# スマホ攻略 V1.8

v1.8

ClassTechStar

# 目录

**一、基礎編（第 0-7 章）** **2**

**第零章：やあ、始める前に** **2**

一、この本は何を手助けしてくれるか?（そして何ができないか） . . . . . 2

二、私たちの「暗号」の説明 . . . . . 2

三、Android と iOS、両方話そう . . . . . 2

四、スマホの容量：なぜ 128G 買ったのに、使えるのは 110G 未満なのか? . . . . 3

五、あなたのスマホは「デジタル資産庫」だ . . . . . 3

**第一章：こんにちは、私のパーソナルコンピューター** **5**

一、スマホハードウェアの「五臓六腑」：深度解析 . . . . . 5

二、オペレーティングシステム：Android VS iOS VS HarmonyOS . . . . . 13

三、スマートフォン端子の全解析 . . . . . 13

四、スマートフォン製品体系と命名ルール . . . . . 15

五、ベースバンドとアンテナ：スマートフォン通信の縁の下の力持ち . . . . . 16

六、あなたのスマートフォンは、一体いくつの役割を演じている? . . . . . 17

付録：Android スマートフォンメモリ拡張解析 . . . . . 18

第一章付録：HarmonyOS オペレーティングシステム特集 . . . . . 21

**第二章：デジタルルームを整理する——ファイル管理** **25**

一、3 層ストレージ体系：あなたのデジタル倉庫 . . . . . 25

二、「ファイルマネージャー」を知る . . . . . 25

三、ファイルシステム構造：より深く理解する . . . . . 26

四、データバックアップと復元：データに保険をかける . . . . . 26

五、ファイルセキュリティ：プライバシーを守る . . . . . 27

六、容量が足りない? 大掃除をしよう . . . . . 27

七、一般的なファイル形式の全解析 . . . . . 27

八、画像とファイルシステム管理：資産を整然と . . . . . 29

**第三章：スマホに「ライフライン」をつなぐ——ネットワーク 32**

〇、接続で迷わない——Wi-Fi とモバイルデータの「道」と「車」を区別 . . . . . 32

一、モバイルネットワーク（4G/5G）とデータ通信 . . . . . 33

二、Wi-Fi：接続、共有、セキュリティ . . . . . 34

三、Bluetooth：短距離無線通信の万能選手 . . . . . 36

四、赤外線リモコン：スマートフォンが万能リモコンに変身 . . . . . 37

五、NFC：タッチする魔法 . . . . . 37

六、テザリング：スマートフォンを Wi-Fi ルーターに . . . . . 38

**第四章：アプリのインストールと管理——スマートフォンの「スキルパック」 40**

一、公式ストア：最も安全な「アプリスーパー」 . . . . . 40

二、ピュア/プロテクトモード：スマートフォンの「アンチウイルス壁」を有効化二、ピュア/プロテクトモード：スマートフォンの「アンチウイルス壁」を有効化 40

三、サードパーティ提供元と「電子寄生虫」：リスク極めて高い！三、サードパーティ提供元と「電子寄生虫」：リスク極めて高い！ . . . . . 41

四、インストール、更新、アンインストール四、インストール、更新、アンインストール . . . . . 42

五、権限管理：あなたはアプリの「ボス」五、権限管理：あなたはアプリの「ボス」 43

Android APK ファイル構造 . . . . . 43

**第五章：スマートフォンをもっと理解させる——基本設定 47**

一、サウンドとバイブレーション一、サウンドとバイブレーション . . . . . 47

二、表示と明るさ二、表示と明るさ . . . . . 47

三、壁紙、テーマとホーム画面三、壁紙、テーマとホーム画面 . . . . . 48

四、効率設定：スマートフォンを「外付け脳」にする四、効率設定：スマートフォンを「外付け脳」にする . . . . . 48

五、アクセシビリティ機能：テクノロジーが善である力五、アクセシビリティ機能：テクノロジーが善である力 . . . . . 50

**第六章：デジタルライフを守る——セキュリティとプライバシー 55**

一、ロック画面：第一の物理的防線一、ロック画面：第一の物理的防線 . . . . . 55

二、「スマートフォンを探す」：究極の盗難防止ツール二、「スマートフォンを探す」：究極の盗難防止ツール . . . . . 55

三、システム更新：パッチを当て、脆弱性を塞ぐ三、システム更新：パッチを当て、脆弱性を塞ぐ . . . . . 56

四、電信詐欺の識別と防止四、電信詐欺の識別と防止 . . . . . 56

五、「開盒」とソーシャルエンジニアリング攻撃の防御：プライバシー漏洩の連鎖を断ち切る五、「開盒」とソーシャルエンジニアリング攻撃の防御：プライバシー漏洩の連鎖を断ち切る . . . . . 58
六、プライバシー保護：あなたのデータはあなたの資産六、プライバシー保護：あなたのデータはあなたの資産 . . . . . 59
七、紛失 / 盗難後の「救急」チェックリスト七、紛失 / 盗難後の「救急」チェックリスト . . . . . 60
八、SIM カードと eSIM のセキュリティ八、SIM カードと eSIM のセキュリティ . 60

**第七章：最先端機能の完全解析 62**

Apple . . . . . 62
アウトドア活動 . . . . . 63
カウントダウン終了時に応答がない場合、自動的に緊急電話を発信。 . . . . . 63
高速 . . . . . 64

**二、ソフトウェア編（第 8-14 章） 67**

**第八章：コミュニケーション——SMS・電話・SNS 67**

一、電話アプリの高度な機能 . . . . . 67
二、SMS と iMessage / RCS . . . . . 69
三、WeChat / QQ：国民的 SNS アプリの管理 . . . . . 70
四、ビデオ通話の比較 . . . . . 72
五、メール：スマホでの設定と使い方 . . . . . 74

**第九章：モバイルオフィスと学習 76**

スマホ画面でドキュメントを編集するのは、パソコンとは確かに感覚が違います。最大の問題は画面が小さく、指が太く、レイアウトがずれやすいこと。でもいくつかのコツで体験はかなり良くなります： . . . . . 76
ノートを取るという事について、多くの人は最初から複雑なシステムを作ろうとして、三日も続きません。私の提案は、まず軽く始めて重くする。「記録する」習慣を身につけてから、「管理する」問題を考える。 . . . . . 77
スマホスキャンは、うまく使えば印刷屋に走る手間をかなり省けます。前回ビザ申請で大量の書類の電子版が必要になった時、すべてスマホで対応しました。 . 77
クラウドストレージ選びについて私は長く考えましたが、結局「メインのエコシステム」が何かによるという結論に達しました。 . . . . . 78
私自身の実体験をお話ししましょう。ある時出張先で、クライアントから急に契約条項の修正を求められました。私のワークフローはこうでした： . . . . . 79

**第十章：モバイル決済と生活サービス** **81**

Alipay でも WeChat でも、初回のカード登録は以下の手順を踏みます： . . . . . 81

多くの人が支払いコードと収款コード（受取コード）の違いを混同していますが、この違いは重要です。 . . . . . 81

指紋/顔認証決済 . . . . . 82

最近外出してバスや地下鉄に乗る時、もう物理の交通カードを使う人は稀です。スマホを改札機にかざす方法は 2 つあります： . . . . . 82

水道光熱費 . . . . . 83

ここ数年の変化は大きく、以前は窓口に並んだ多くの手続きが、今はスマホで数分で完了します。 . . . . . 83

**第十一章：撮影と画像管理** **85**

まず最も一般的な問題をいくつか挙げます。いくつ当てはまるか確認してください： 85

これらは理論知識不要、一度使えば効果がわかります： . . . . . 85

ポートレートモード . . . . . 86

多くの人がレタッチについて誤解しており、レタッチすると不自然になると思っています。実はカメラで撮った写真は元々肉眼で見たものと異なり、適度な調整は当時見た画面を再現するためで、偽造ではありません。 . . . . . 87

スマホのアルバムに数千枚の写真が溜まり、特定の一枚を探すのに時間がかかる——この経験は皆あるでしょう。いくつかの実用的な整理方法： . . . . . 87

スクショ . . . . . 88

第十一章付録：プロフェッショナル映像技術知識 . . . . . 88

**第十二章：音声・映像エンターテインメントと読書** **99**

音楽 App は現在機能がほぼ同じですが、それぞれ強みがあり、選び方次第でかなり快適になります： . . . . . 99

ポッドキャストは、聴いたことがない人は自分と無関係だと思い、一度聴き始めると戻れなくなります。本質は「ラジオ番組の Netflix 版」——科技を語る人、歴史を語る人、物語を読む人、どんなジャンルでも揃っています。 . . . . . 100

多くの人が知りませんが、スマホで再生できる形式はストリーミング App が提供するものよりはるかに多い。良いローカルプレーヤーで多くの問題が解決します。 . . . . . 100

スマホでの読書は目に悪い、集中力を削ぐと言われますが、確かに一理あります。しかしスマホ読書の利点も明確——いつでもどこでも、細切れ時間を最大化活用。私は地下鉄、行列、寝る前にスマホで多くの本を読みました。 . . . . . 101

スマホゲームへの私の態度は：遊んで良いが、逆にコントロールされないように。ゲームツールを合理的に使えば、より快適に遊べ、本業も疎かになりません。 . 101

**第十三章：ツール系アプリのおすすめ** **103**

一、効率ツール：時間と注意力を管理する . . . . . 103
二、健康・運動 . . . . . 106
三、旅行・おでかけ . . . . . 107
四、自動化ツール入門 . . . . . 109
五、ブラウザと検索エンジン：情報への入り口 . . . . . 109
六、実用システムツール：クリーンアップ、迷惑対策、セキュリティ . . . . . 112
七、天気警報・自然災害警報・地震速報 . . . . . 114
八、デバイス間連携：遠隔操作、高速転送、アカウントセキュリティ . . . . . 118

**第十四章：パスワードマネージャー特集 123**
一、なぜパスワードマネージャーが必要？ . . . . . 123
二、パスワードマネージャーの仕組み . . . . . 123
三、暗号化原理の概要 . . . . . 123
四、主要パスワードマネージャーのおすすめ . . . . . 125
五、パスワード強度の本当の評価 . . . . . 126
六、Passkey：パスワード不要ログインの未来 . . . . . 127
七、ハードウェアセキュリティキー . . . . . 130
八、家族/チームのパスワード共有 . . . . . 132
九、パスワードマネージャー活用のベストプラクティス . . . . . 134

**三、応用編（第 15-20 章） 137**

**第十五章：バッテリーの科学と連続駆動時間の最適化 137**
一、バッテリーとは？サイクルと健康度 . . . . . 137
二、充電の迷信？急速充電、ワイヤレス充電と黄金の法則 . . . . . 137
こんな経験はありませんか：スマホは急速充電対応なのに、充電器を変えたら遅くなった。これは急速充電が単なる「高出力なら OK」ではなく、充電ヘッド、ケーブル、スマホの三者が「同じ言語を話す」必要があるから——これが急速充電プロトコルです。 . . . . . 138
四、バッテリー新技術：シリコン炭素負極バッテリー . . . . . 142
五、これこそ本当の省エネテクニック . . . . . 142

**第十六章：アカウント体系とクラウドサービス 144**
多くの人がアカウントは App ダウンロード用だと思っています。実はスマホアカウントの背後には想像以上のものが紐付いています： . . . . . 144
二要素認証を有効化 . . . . . 144

Apple スマホを使っている人なら、遅かれ早かれ iCloud 容量が満杯になったらどうするかという難問に直面します。 . . . . . 145

この 2 つの言葉は似ていますが、結果は全く異なります： . . . . . 146

Android 側は Apple より少し複雑で、データが複数のアカウントに分散する可能性があります： . . . . . 147

Apple の「ファミリー共有」は家族と App 購入、音楽、映画、iCloud スペースを共有でき、位置情報も共有可能。最大 6 人まで追加。設定方法：設定あなたの名前ファミリー共有。Huawei と Xiaomi にも同様の機能があり、アカウント設定で「ファミリー共有」や「ファミリークラウド」を見つけられます。 . . . 147

**第十七章：スマホと他のデバイスの連携 149**

正直、以前最も嫌だったのはスマホで撮った写真をパソコンに置いてレタッチしたくて、ケーブルを探し、転送を待つこと。後に、実はケーブル不要で、良い連携はケーブルより 10 倍速いことが分かりました。 . . . . . 149

お正月に実家に帰った時、従妹がテレビで彼女の卒業旅行動画を流したがっていました。長く苦労しましたが、最終的に非常にシンプルだと分かりました： . . 150

母にバンドを買いました。最初は「運動もしないのにこれを買ってどうするの」と言っていました。2 週間使って、自分が毎日 5 時間ちょっとしか寝ていないことに気づき、自発的に作息を調整し始めました。これがウォッチ/バンドの最も実用的な価値——目に見えない健康データを目に見える数字にする。 . . . 150

スマートホームは難しそうに聞こえますが、要するに 3 つのこと：スマホでリモート制御、自動化ルールの設定、音声操作。 . . . . . 151

有線イヤホンから Bluetooth イヤホンへの変更は、私にとってスマホ体験のアップグレードで最も顕著なものでした。もうバッグから絡まった糸の塊を取り出すことも、ドアノブにケーブルが引っかかることもありません。 . . . . . 151

運転中のスマホ操作は違法で、命に関わります。しかしナビ、電話、音楽が必要な時もあります。この時に CarPlay（Apple）と Android Auto（Android）が役立ちます。 . . . . . 153

デバイスが多く連携されるほど、プライバシー漏洩のリスク面が拡大します。いくつか注意点： . . . . . 153

**第十八章：データバックアップと移行 155**

異なるデータの「生存状態」は異なり、区別する必要があります： . . . . . 155

全体バックアップはスマホの完全な「スナップショット」を撮るようなもの——現在のシステム設定、デスクトップレイアウト、App データ、写真、連絡先、すべてパッケージ化。 . . . . . 155

全体バックアップは最後の砦ですが、一部のものは重要すぎて別途保存する価値があります： . . . . . 156

新スマホへの変更は嬉しいことですが、データが移行できないと頭痛に。3 つのケースをそれぞれ対応： . . . . . 157

これはデータセキュリティ分野の古典的原則で、シンプルで覚えやすい： . . . . . 158

**第十九章：トラブルシューティングと自己救助 160**

正直、スマホを 1 年ほど使うと重くなるのは正常です。原因は必ずしもスマホの故障ではなく、多くの場合「埃溜まり」——ゴミファイル、バックグラウンドプロセス、ストレージ容量が徐々に埋まる。 . . . . . 160

ある App を開くとすぐ落ちる、または使っているうちに突然フリーズする。処理フローは医師の問診のように、一歩ずつ： . . . . . 161

充電できないのは特に焦りを引き起こす問題ですが、多くの場合原因はアクセサリにあり、スマホの故障ではありません。 . . . . . 161

外でモバイルデータは正常だが、家に帰ると Wi-Fi に接続できない。または Wi-Fi に接続しているがネットに繋がらない。この順序で確認： . . . . . 162

タッチ反応不良 . . . . . 162

この状況は最も心拍数を上げますが、冷静に、まずこれらを試しましょう： . . . . 163

スピーカーから音が出ない . . . . . 163

上記の方法をすべて試してもダメなら、専門修理に頼る必要があります。しかし修理市場は水深が深く、私自身の失敗経験と教訓を： . . . . . 163

第二十章：開発者モードと高度な権限 . . . . . 165

第二十章付録：Termux——Android デバイス上のポータブル Linux ターミナル環境 173

**四、発展編（第 21-26 章） 178**

**第二十一章：モバイルネットワークと情報セキュリティの深掘り 178**

：あなたのスマホ公共ネットワーク対象サイト。中間のルーター、スイッチ、どの段階でもあなたが送信するデータを「覗き見」される可能性。特にカフェ、空港のような公共 Wi-Fi では、正直「裸走」状態。 . . . . . 178

——例えばあなたがアクセスした HTTP ウェブページの内容、TLS 未対応の App 通信——は彼らが傍受・閲覧できました。 . . . . . 179

サーバーが返答：「OK、あなたのリストからこの暗号化方案を選んだ、これが私のデジタル証明書、公開鍵が含まれ、某某 CA 機関が署名認証」。 . . . . . 179

で、 . . . . . 180

シークレットモードがすること：ローカルに痕跡を残さない——シークレットウィンドウを閉じると、ブラウザは閲覧履歴、Cookie、フォーム入力記録、検索記録を保存しない。他人のパソコンで何かを調べる、プレゼントを買って家族に閲覧履歴から見つかりたくない場合、シークレットモードが適切。 . . . . . 181

：定期的に設定で Cookie、キャッシュ、履歴をクリア。「追跡防止」のためではありません——Cookie クリアではもうあまり効かない——スマホが重くなるのを防ぐため。ブラウザキャッシュは数 GB の容量を占めることがある。 . . . 181
：iPhone は設定-Wi-Fi で、Android は Wi-Fi 設定で。 . . . 182

**第二十二章：AI はスマホでどう働くのか 184**

聞き取る（音声認識） . . . 184
理解する（自然言語理解） . . . 184
2024-2025 年、AI 大規模モデル（DeepSeek、ChatGPT、文心一言、通義千問など）の爆発がスマホの使い方を深く変えました。スマホはもはや指示を実行するだけのツールではなく、理解、推論、創作できるスマートパートナー。 . . . 184
AI 大規模モデル（Large Language Model、LLM）は膨大なテキストデータで訓練された AI システムで、自然言語を理解・生成できます。インターネット上のほぼすべての文字を読んだ「スーパー優等生」と想像でき、質問に答え、文章を書き、翻訳し、プログラミングし、データ分析できる。 . . . 185
実際の表現 . . . 186
を紹介し、大量の実例を交える。 . . . 187
：AI がもっともらしくでたらめを言うことがあります（専門用語で Hallucination）。存在しない歴史的事件、誤った引用、虚偽の統計データをでっち上げます。重要な情報は必ず人工で確認してください。 . . . 191

**第二十三章：プログラミング的思考とオートメーション入門 194**

一、オートメーションとは何か？ . . . 194
二、iOS「ショートカット」——Apple ユーザーのオートメーション最強ツール . . . 195
三、Android オートメーション——Tasker とその仲間たち . . . 196
四、オートメーション的思考の日常応用 . . . 197
五、落とし穴ガイド：オートメーションのよくある問題 . . . 197

**第二十三章付録：GitHub 入門ガイド——コードホスティングからコラボレーションプラットフォームまで 199**

一、GitHub とは何か？ . . . 199
二、Git と GitHub の違い（重要概念！） . . . 199
三、GitHub の核心概念と機能 . . . 200
四、登録と使い始め：5 分で最初のリポジトリを作成 . . . 202
五、基本ワークフロー：ローカルからクラウドへ . . . 202
六、チームコラボレーションフロー：プルリクエストはどう機能するか . . . 203
七、GitHub の典型的な応用シナリオ . . . 204
八、初心者へのアドバイス . . . 204

九、まとめ：なぜ誰もが GitHub を知るべきか . . . . . 206

**第二十四章：Root 権限と BL ロックの深掘り解説 207**

一、Bootloader の深い理解 . . . . . 207
二、BL ロック解除の詳細フロー（Xiaomi を例に） . . . . . 209
三、Root の複数の方案 . . . . . 210
四、フラッシュ入門 . . . . . 210
五、Samsung Knox：BL 解除の不可逆な代償 . . . . . 210
六、よくある質問と回答（Q& A） . . . . . 211

**第二十五章：中古スマートフォンの処理とデジタル遺産 213**

一、古いスマートフォンの「安楽死」：安全な売却・譲渡ガイド . . . . . 213
二、デジタル遺産：あなたのソーシャルな魂をどう安置するか？ . . . . . 215
三、エコな最後の旅路 . . . . . 220

**第二十六章：今のスマホ技術の最前線 222**

一、AI スマホ：マーケティングの煽りか、真の変革か？ . . . . . 222
二、各社の AI 戦略比較：本気で取り組むのは誰、OEM 的なのは誰？ . . . . . 223
三、衛星通信：フラッグシップの「目玉」から中端機の標準装備へ . . . . . 223
四、折りたたみ画面エコシステムの成熟度：メイン機として使えるか？ . . . . . 224
五、6G と Wi-Fi 7：次世代接続の速度と境界 . . . . . 225
六、シリコン炭素負極バッテリー：ついに材料レベルの突破口が到来 . . . . . 225
七、本章の概要：一言でこれらの最先端を記憶 . . . . . 226

**五、実践編（第 27-32 章） 228**

**第二十七章：お父さんお母さんもラクラク——スマホのシニア向け完全ガイド 228**

まず心構えを正そう：これは“お年寄り使いこなせない”ではなく“スマホが作れていない”のだ . . . . . 228
一、第一歩：「簡単モード」をオンにする——スマホがスッキリする . . . . . 228
二、簡単モードを使わなくても手動で OK——“大きな文字 + 大きなアイコン”の五ステップ . . . . . 229
三、詐欺防止——この章は必ずお年寄りと一緒に読もう . . . . . 230
四、リモートサポート——離れていても親のスマホを修理できる . . . . . 232
五、WeChat の“シニア改造”——お年寄りにこれだけ伝えれば OK . . . . . 232
六、スマホを失くしたら？お年寄り向け“緊急対応プラン”を作る . . . . . 233
七、ワイヤレスイヤホンと補聴器の互換性——見落とされがちな便利機能 . . . . . 233

八、健康関連機能——スマホは通信ツールにとどまらない . . . . . 234

**第二十八章：お金を賢く使ってお気に入りを買う——スマホ選びの究極ガイド 235**

三段階選び方：ニーズ予算機種 . . . . . 235

一、コアパラメータの見方——全部を理解する必要はない、この五項目が分かれば十分 . . . . . 235

二、予算別のおすすめ（2026 年中期市場を基準） . . . . . 237

三、シーン別選び方——あなたのニーズは？ . . . . . 238

四、陥りやすい五つの罠——購入前にチェック . . . . . 239

五、いつ買うのが一番お得？ . . . . . 240

六、購入チャネルの選び方——どこで買えば安心？ . . . . . 240

**第二十九章：デジタルヘルス——スマホとの健康的な関係づくり 242**

一、まず“スクリーンタイム”を見よう——数字が目を覚まさせる . . . . . 242

二、App に“使用制限”を設定する——自分にスピードリミッターを付ける . . . . 242

三、“おやすみモード”の正しい使い方——通知を消すのではなく不安を消す . . . 243

四、目の保護は App ではなく習慣——20-20-20 ルール . . . . . 243

五、“スマホ首”の頸椎自救ガイド . . . . . 244

六、睡眠とスマホ——寝室で最も変えるべき習慣 . . . . . 244

七、情報不安症——“選択的盲目”を身につける . . . . . 244

**第三十章：スクリーンミラーリング、プレゼンとカーナビ連携 246**

一、ワイヤレスキャストの三大陣営——混同しないように . . . . . 246

二、有線キャスト——より安定、ゼロ遅延 . . . . . 247

三、スマホがパソコンに——デスクトップモード . . . . . 247

四、カースル連携——運転中のスマホの正しい使い方 . . . . . 248

五、会議室キャスト実戦——もうバタバタしない . . . . . 248

**第三十一章：ショート動画撮影とコンテンツ制作の入門 250**

一、撮る前に明確に：誰に見せるのか？ . . . . . 250

二、スマホ撮影の基本設定——撮影前に必須の 5 件事 . . . . . 250

三、カメラワーク五手——画面を“動かす” . . . . . 251

四、編集入門——CapCut（剪映）で十分 . . . . . 251

五、サムネイルの重要性——70% の人がサムネイルだけで見るか決める . . . . . 252

六、コンテンツ創作の三つの心法 . . . . . 252

**第三十二章：スマホ安全救急マニュアル 254**

シーン 0：地震 / 極端な天候——スマホが命を守るしくみ . . . . . 254

シーン 1：スマホを紛失した / 盗まれた——ゴールデン 30 分 . . . . . 255
シーン 2：詐欺に騙された——72 時間の勝負期間 . . . . . 256
シーン 3：スマホがウイルス / トロイの木馬に感染したら——どうする？ . . . . . 257
シーン 4：アカウントを乗っ取られた——素早く主導権を取り戻す . . . . . 257
シーン 5：脅迫メッセージが届いた——怖がらない、払わない . . . . . 258
シーン 6：AI 顔すり替え / AI 声まね詐欺——見たものが真実ではなくなる時代 . 258
シーン 7：SIM スワップ攻撃（SIM 乗っ取り）——あなたの番号が「盗まれる」 . 259

**六、展望編とコネクテッドカー編（第 33-34 章） 264**

**第三十三章：未来展望——スマホの次のステップ 264**
一、AR/VR と空間コンピューティング：スマホ以外の“第二のデスクトップ” . . 264
二、ブレイン・マシン・インターフェース：思考がインタラクションの媒介に . . . 265
三、スマホの“隠身”と新しいアイデンティティの基盤 . . . . . 265
四、倫理的思考：私たちはまだスマホの主人か？ . . . . . 265
五、繋がりを保ち、独立も保つ——あなたのデジタルライフ哲学 . . . . . 266

**第三十四章：車載連携ディープガイド——スマホとクルマの融合 268**
一、カースル連携の三大ルート . . . . . 268
二、CarPlay / Android Auto / HiCar の比較 . . . . . 268
三、HarmonyOS コックピット：スマホとカースルのシームレスな連携 . . . . . 269
四、新興自動車メーカーのカースルシステム . . . . . 269
五、スマホをカースルに——使えるか？ . . . . . 270
六、実用的なヒント . . . . . 270

**子どもとスマホ特集（第三十五章） 274**

**第三十五章：子どもとスマホ——デジタル時代の教育ガイド 274**
一、何歳で子どもにスマホを与えるか？ . . . . . 274
二、ペアレンタルコントロールシステム完全攻略 . . . . . 275
三、ティーンモード . . . . . 276
四、ネットリテラシー教育 . . . . . 277
五、健康的な利用習慣 . . . . . 277
六、デジタル遺産と子ども . . . . . 278

**第三十六章：スマホとスマートホーム——家を賢くする 280**

スマートホームのプロトコル：どれを選ぶかが大事 . . . . . 280
三大エコシステムの比較 . . . . . 281
スマホをホームハブにする . . . . . 281
入門セットのおすすめ . . . . . 282
セキュリティ上の注意点 . . . . . 283

**七、AI プログラミング実践編（第 37-47 章） 286**

**第三十七章：パラダイム革命——「コードを書く」から「AI にコードを書かせる」へ 286**
Vibe Coding の誕生：ひと言が引き起こしたムーブメント . . . . . 286
2026 年の現在地：Agentic Engineering の時代 . . . . . 287
核心となる認識の転換：Coder から Conductor へ . . . . . 288
四、なぜ方法論がツールより大事なのか . . . . . 289

**第三十八章：AI プログラミング五大思考原則 291**
一、原則一：Think Before Coding（先に考え、それからコードを書く） . . . . . 291
二、原則二：Simplicity First（シンプル・ファースト） . . . . . 292
三、原則三：Surgical Changes（ピンポイント修正） . . . . . 293
四、原則四：Goal-Driven Execution（目標駆動実行） . . . . . 294
五、原則五：Iterate and Verify（反復検証） . . . . . 294
六、Harness Engineering：Agent = Model + Harness . . . . . 295

**第三十九章：コンテキストエンジニアリング——AI にあなたの意図をわからせる 297**
一、プロジェクト記憶ファイル：AI の「長期記憶」 . . . . . 297
二、仕様駆動開発（SDD） . . . . . 298
三、コンテキストウィンドウ管理 . . . . . 299
四、コンテキストエンジニアリングの実践ポイント . . . . . 300

**第四十章：要件エンジニアリング——アイデアを AI が実行できる指示に変える 302**
一、あいまいなアイデアから、構造化された要件へ . . . . . 302
二、要件を小さく分解する技法 . . . . . 303
三、PRD 作成のテンプレートと実践 . . . . . 304
四、実践：アイデアから具体案までを完全に一巡してみる . . . . . 305

**第四十一章：2026 年 AI プログラミングツールの全景 307**
一、三大流派と選択のフレームワーク . . . . . 307
二、IDE 内蔵 AI 流派 . . . . . 308

三、ターミナル Agent 流派 . . . . . 309
四、クラウド Builder 流派 . . . . . 309
五、ツールは変わる、方法論は変わらない . . . . . 310

**第四十二章：大規模モデルの全景と API 実践 312**
一、世界の主流大モデル：三強の構図 . . . . . 312
二、中国国産の主流大モデル：百花繚乱 . . . . . 315
三、モデル選択の意思決定フレームワーク . . . . . 318
四、API 呼び出し入門：ゼロから大規模モデルにつなぐ . . . . . 319
五、Token プラン購入ガイド . . . . . 322
六、実践：プロジェクトに大規模モデル API を組み込む . . . . . 326

**第四十三章：対話エンジニアリング——AI と効率的に協働するための対話技術 330**
一、プロンプトの階層戦略 . . . . . 330
二、デバッグ対話：エラーの説明で AI の問題特定を速くするには . . . . . 331
三、反復戦略：小刻み前進 vs 一括リファクタリング . . . . . 332
四、マルチターン対話のコンテキスト管理 . . . . . 333

**第四十四章：0 から 1 への実践——プロジェクト完成までの全プロセス 334**
一、プロジェクト 1：ミニマル個人ツールサイト（入門レベル） . . . . . 334
二、プロジェクト 2：フルスタック家計簿システム（中級レベル） . . . . . 335
三、プロジェクト 3：AI 拡張アプリ（上級レベル） . . . . . 337
四、汎用の実践フローチェックリスト . . . . . 338

**第四十五章：品質保証——AI が書いたコードを信頼できるものに 340**
一、テスト駆動：AI にテストを書かせる . . . . . 340
二、コードレビューのテクニック . . . . . 341
三、バージョン管理：AI プログラミングにおける Git の役割 . . . . . 342
四、よくある落とし穴とその回避ガイド . . . . . 342

**第四十六章：ステップアップへの道——使い手からアーキテクトへ 345**
MCP プロトコルとツール拡張 . . . . . 345
カスタムワークフローと Skills . . . . . 345
マルチエージェント協働のパターン . . . . . 346
継続学習の道 . . . . . 346

**第四十七章：クイックリファレンスとリソース 348**
思考モデルの早見カード . . . . . 348

ツール選択の決定木 . . . . . 348
よくある質問（FAQ） . . . . . 349
おすすめ学習リソース . . . . . 350
おわりに：道具は手に、方法論は心に . . . . . 350
参考文献 . . . . . 351

**八、Android 開発実践編（第 48-58 章） 353**

**第四十八章：Android を知る——OS から開発エコシステムまで 353**
Android の生い立ち：「小さなロボット」から「世界最大の OS」へ . . . . . 353
どの言語で書く?：Java か、Kotlin か . . . . . 354
アプリはどうやって生まれるのか? . . . . . 355
Android 技術スタックの全体図：あなたの「武器リスト」 . . . . . 356
Android vs iOS：開発者向け最初の「陣取り」の授業 . . . . . 356

**第四十九章：開発環境を整える——Android Studio と最初のプロジェクト 358**
Android Studio をインストールする . . . . . 358
最初のプロジェクトを作る . . . . . 359
動かしてみる：エミュレータと実機 . . . . . 359
プロジェクト構造の解体：各ファイルの役割 . . . . . 360
さっそく試す：「Hello」を自分の名前に変える . . . . . 361
よくあるエラーの応急処置：初心者の最初 2 週間の「救急室」 . . . . . 361

**第五十章：Kotlin はじめの一歩——Android 開発の第一言語 364**
変数と型：データに名前を付ける . . . . . 364
関数：ロジックを「ボタン」にまとめる . . . . . 365
条件とループ：プログラムに「考える」力をつける . . . . . 365
Null 安全性：Kotlin の「命綱」 . . . . . 366
クラスとオブジェクト：「部品」を作る鋳型 . . . . . 366
ラムダと高階関数：Kotlin の「シンタックスシュガー」 . . . . . 367
超初心者向けの文法早見カード . . . . . 367

**第五十一章：UI 開発——Jetpack Compose 入門 369**
Compose の発想：UI ＝関数 . . . . . 369
基本コンポーネント：文字、ボタン、入力欄、画像 . . . . . 370
レイアウトの 3 点セット：Column、Row、Box . . . . . 370
状態：UI を「生きている」ものにする . . . . . 371

テーマとダークモード：1 回の設定で全体に効く . . . . . 372
プレビュー、アニメーション、ライブ編集：Compose の「チート機能」 . . . . . 372

**第五十二章：4 大コンポーネントと Activity のライフサイクル 375**
4 大コンポーネント：Android の「大黒柱」 . . . . . 375
Activity のライフサイクル：Android 開発で最初の「宝の地図」 . . . . . 376
Intent：アプリの内と外をつなぐ「切符」 . . . . . 376
複数画面とナビゲーション . . . . . 377
ついでに知っておく：Fragment とは何か（そして本書で扱わない理由） . . . . . 378

**第五十三章：データ保存とネットワーク通信 379**
保存方式の選び方：まずこの表を見る . . . . . 379
DataStore：小さな設定を保存する . . . . . 380
Room：データに「本物の家」を用意する . . . . . 380
Retrofit：サーバーと会話する . . . . . 381
JSON 解析：ネットのデータに「翻訳」をつける . . . . . 382
コルーチン：Android の並行処理の「共通語」 . . . . . 382
コルーチン：Android の並行処理の「共通語」 . . . . . 382

**第五十四章：プロジェクト 1——わたしの家計簿（ローカル版） 384**
要件を考える：まず 5 つの質問 . . . . . 384
データ層：記録に「住まい」を与える . . . . . 385
MVVM：画面とロジックを「別々の家」にする . . . . . 386
画面：一覧画面＋入力画面 . . . . . 386
編集と削除：データを「入れて出せる」ように . . . . . 388
検収：3 層のチェック＋ 2 つの上級チャレンジ . . . . . 388
検収：3 層のチェック＋ 2 つの上級チャレンジ . . . . . 388

**第五十五章：プロジェクト 2——天気チェックアプリ（ネットワーク版） 390**
無料の天気 API を申し込む . . . . . 390
ネットワーク層：Retrofit ＋ JSON 解析 . . . . . 391
3 つの画面状態：読み込み中 / 成功 / 失敗 . . . . . 391
実戦の画面コード：天気を描き出す . . . . . 392
画面と操作の細かい工夫 . . . . . 393
画面と操作の細かい工夫 . . . . . 393

**第五十六章：仕上げとリリース——テスト、デバッグ、Google Play への出品 394**

デバッグの三種の神器：Logcat、ブレークポイント、UI インスペクター . . . . . 394
テスト：コードに保険をかける . . . . . 395
署名とパッケージング：アプリに「ハンコ」を押す . . . . . 395
Google Play 出品の全ステップ . . . . . 396
ストア素材の作り方：第一印象がそのままダウンロード数になる . . . . . 397
パフォーマンス最適化とクラッシュ報告の小チェックリスト . . . . . 397

**第五十七章：ステップアップへの道——権限、通知、モダンなアーキテクチャ 399**
実行時権限：まずノックして、それから入る . . . . . 399
通知：正しいタイミングで、役に立つことを伝える . . . . . 400
アーキテクチャのステップアップ：依存性注入とモジュール化 . . . . . 400
2026 年の新しいやり方：AI 支援の Android 開発 . . . . . 401
Compose の性能最適化とデバッグツール . . . . . 402

**第五十八章：クイックリファレンスとリソース 403**
よく使うコードのクイックカード . . . . . 403
よくある質問 FAQ . . . . . 404
おすすめ学習リソース . . . . . 405
Gradle のよく使うコマンド速査 . . . . . 405
おわりに：最初のアプリから N 個目のアプリへ . . . . . 406
おわりに：最初のアプリから N 個目のアプリへ . . . . . 406
参考文献 . . . . . 406

**九、iOS 開発実践編（第 59-69 章） 408**

**第五十九章：iOS を知る——Apple の生態系と開発の生態系 408**
iOS の生い立ち：1 台のスマホがひとつの業界を変えた . . . . . 408
壁に囲まれた庭：Apple の生態系の「ルール」と「配当」 . . . . . 409
どの言語で書くか：Objective-C？　それとも Swift？ . . . . . 409
アプリはどうやって生まれるのか？ . . . . . 410
iOS 技術スタックの全体像：あなたの「武器リスト」 . . . . . 411

**第六十章：開発環境を整える——Xcode と最初のプロジェクト 412**
Mac を 1 台用意する . . . . . 412
Xcode をインストールする . . . . . 413
最初のプロジェクトを作る . . . . . 413
プロジェクト構造の分解：ファイルごとの役割 . . . . . 414

実機デバッグ：アプリをあなたの iPhone に入れる . . . . . 414
Xcode の時短テクニック：初心者に効く「ショートカット裏ワザ」5 選 . . . . . 414
実機デバッグ：アプリをあなたの iPhone に入れる . . . . . 415
さっそく試す＋よくあるエラーの応急処置 . . . . . 415
さっそく試す＋よくあるエラーの応急処置 . . . . . 415

**第六十一章：Swift にすばやく入門——Apple 開発の第一言語 417**
変数と型：データに名前をつける . . . . . 417
オプショナル（Optional）：Swift の「クラッシュ防止のお守り」 . . . . . 418
関数とクロージャ：ロジックを「ボタン」にまとめる . . . . . 418
構造体とプロトコル：Swift の「レゴ」システム . . . . . 419
ゼロからの初心者のための文法チートシート . . . . . 420

**第六十二章：画面づくり——SwiftUI 入門 421**
SwiftUI の考え方：宣言型であって命令型ではない . . . . . 421
基本のビュー：テキスト、ボタン、入力欄、画像 . . . . . 422
レイアウトの 3 点セット：VStack、HStack、ZStack . . . . . 422
状態とバインディング：画面を「生き」させよう . . . . . 423
プレビュー、テーマ、アニメーション：SwiftUI の「裏ワザ」 . . . . . 424
よく使うモディファイア早見表：この 1 ページがあれば、画面づくりはもう安心 . 425

**第六十三章：ビュー、状態、ナビゲーション 427**
状態管理の「データフロー全体図」 . . . . . 427
NavigationStack：ページ移動の「スタック」 . . . . . 428
ポップアップと下部タブ：sheet と TabView . . . . . 429
List：iOS の「国民的リスト」 . . . . . 429
アプリのライフサイクル：フォアグラウンドとバックグラウンド . . . . . 430

**第六十四章：データ保存とネットワーク通信 432**
保存方式の選び方：まずこの表を見る . . . . . 432
UserDefaults：小さな設定を保存する . . . . . 433
SwiftData：データに「ちゃんとした家」を用意する . . . . . 433
Codable：JSON を「翻訳」する . . . . . 434
URLSession ＋ async/await：サーバーと会話する . . . . . 435
ネットワークエラーも「レベル分け」を：すべてのエラーをユーザーに丸投げしない 436

**第六十五章：プロジェクト 1——わたしの家計簿（ローカル版） 437**

要件を考える：まず 5 つの質問 . . . . . 437
データ層：記録に「家」を用意する . . . . . 437
MVVM：画面とロジックの「世帯分け」 . . . . . 438
画面：リストページ＋入力ポップアップ . . . . . 439
編集、削除、集計 . . . . . 440
検収：3 層チェック＋ 2 つの上級チャレンジ . . . . . 441

**第六十六章：プロジェクト 2——天気チェックアプリ（ネットワーク版） 442**
無料の天気 API を申し込む . . . . . 442
ネットワーク層：URLSession ＋ Codable . . . . . 442
3 つの画面状態：読み込み中 / 成功 / 失敗 . . . . . 443
実戦の画面コード：天気を描き出す . . . . . 444
操作のディテールとお気に入り都市 . . . . . 445
ネットワーク通信のデバッグ：出力、パケットキャプチャ、低速ネットの再現 . . . 445

**第六十七章：仕上げとリリース——テスト、デバッグ、App Store 公開 447**
デバッグの基本三ツール：print、ブレークポイント、プレビュー . . . . . 447
テスト：コードに保険をかける . . . . . 448
署名とパッケージング：App に「認め印」を押す . . . . . 448
App Store 公開の全ステップ . . . . . 449
ストア素材の作り方：第一印象がダウンロード数を左右する . . . . . 450
クラッシュ報告とパフォーマンス改善のミニチェックリスト . . . . . 450

**第六十八章：ステップアップへの道——権限、通知、モダンなアーキテクチャ 452**
権限：まずノック、それから入る . . . . . 452
通知：正しいタイミングで役立つ言葉を届ける . . . . . 453
アーキテクチャのステップアップ：MVVM の深掘りとテスト容易性 . . . . . 454
2026 年の新常識：AI 支援の iOS 開発 . . . . . 454
Swift の並行処理の要点：@MainActor と Task . . . . . 455

**第六十九章：クイックリファレンスと学習リソース 457**
よく使うコードのクイックカード . . . . . 457
よくある質問 FAQ . . . . . 458
Xcode のよく使うショートカット一覧 . . . . . 459
おすすめ学習リソース . . . . . 460
おすすめ学習リソース . . . . . 460
おわりに：最初の App から N 個目の App へ . . . . . 460

参考文献 . . . . . 461

**付録 A：新端末の開封「超親切」ガイド** **463**

開封・検品「3 ステップ」： . . . . . 463

アクティベーションと初期設定の「重要リマインダー」： . . . . . 463

アプリのインストールは「セキュリティ第一」： . . . . . 463

**付録 B：実践練習の参考** **464**

**付録 C：ミニマル用語「隠語」対照表** **466**

**付録 D：信頼できるリソースはどこで探す？** **467**

公式サポートが第一選択： . . . . . 467

実用サイトとブロガー： . . . . . 467

セキュリティ自己チェック一言リスト： . . . . . 467

**付録 E：感謝とご説明** **468**

バージョン 1.6 の更新ポイント . . . . . 468

バージョン 1.7 の更新ポイント . . . . . 469

バージョン 1.8 の更新ポイント . . . . . 470

コンテンツの追加：実用アプリと迷惑電話・SMS のブロック . . . . . 471

コンテンツの追加：気象警報 / 自然災害警報 / 地震警報 . . . . . 471

コンテンツの追加：複数デバイスの連携と効率化ツール . . . . . 472

**付録 F：折りたみ画面の選び方と使い方特集** **473**

一、折りたたみ画面は今どこまで発展しているか？ . . . . . 473

二、どんな人が折りたたみスマートフォンに向いているか？ . . . . . 474

三、選び方のポイント：この 6 つの指標を見る . . . . . 474

四、折りたたみスマートフォンならではの使い方のコツ . . . . . 475

五、メンテナンスのポイント：折りたたみ画面を長持ちさせる . . . . . 476

六、折りたたみスマートフォンの「落とし穴」と心理的期待値 . . . . . 477

七、主流ブランドの選び方（2026 年） . . . . . 477

**おわりに：好奇心を持ち続け、清醒でいる** **479**

**付録 G：本書まるごとクイック索引** **480**

**付録 H：バージョンロードマップ** **484**

# 一、基礎編（第 0-7 章）

# 第零章：やあ、始める前に

本格的な探求の前に、まず基本ルールを話しておこう。これで後の説明がすべて理解できるはずだ。急がば回れ、ということ。

## 一、この本は何を手助けしてくれるか？（そして何ができないか）

スマホの使い方を体系的に理解し、機能の背後にある道理を明白にする手助けをする。Android 派でも Apple 派でも、ほとんどの考え方は共通だ。実践練習に付き添い、「使える」から「理解して使える」へとレベルアップさせる。

特定のスマホの取扱説明書ではなく、新製品を追うテックニュースでもなく、スマホ依存を教えるガイドでもない。正直に言えば、「スマホで動画をより快適に見る方法」を期待しているならがっかりするかもしれない　私たちがより関心があるのは、スマホを逆にあなたに奉仕させるのではなく、本当にあなたのために使えるようにする方法だ。

## 二、私たちの「暗号」の説明

より分かりやすく説明するため、本書では小さなマークを使う。これらのマークは私たち間の暗黙の了解のようなものだ：

- **[ボタン名]**：画面上の具体的なボタンやアプリ名を表す。例：[設定]、[OK]。
- **>**：操作順序を表す。例えば「[設定] > [サウンド] をタップ」とあれば、先に設定をタップし、それからサウンドをタップする。
- **太字**：非常に重要！重要な概念やセキュリティ警告の可能性があるので、必ず注意すること。
- *斜体*：補足的な小さなアイデアや比喩で、理解を助ける。

## 三、Android と iOS、両方話そう

実は、呼び方は違ってもやることは同じものがある。例：アプリ（Android）と App（iOS）、設定（どちらも設定）、ファイル管理（Android）とファイル（iOS）。同じものだと知っていれば十分、用語に惑わされないように。

また、現在 Android 陣営には Huawei の HarmonyOS もあり、独自の特色を持っている。後で専用に説明する。要するに、どんなスマホを使っていても、この本は役に立つ。

## 四、スマホの容量：なぜ 128G 買ったのに、使えるのは 110G 未満なのか？

容量の単位は皆知っている：1TB = 1024GB、1GB = 1024MB、1MB = 1024KB……しかし落とし穴がある：ストレージメーカーは通常 1000 進法で計算し、システムは 1024 進法で計算するため、買った「128G」ストレージの実際の容量は「縮む」。

ただしスマホの容量が縮む主な理由は 3 つある：

1. **システム自体が場所を占める：**オペレーティングシステムは家の耐力壁のようなもので、必ず存在する。現在のシステムは軽く 10-20GB を占めるが、これは正常だ。
2. **メーカープリインストールアプリ：**新しいスマホに最初から入っているアプリも容量を占める。アンインストールできるものとできないものがある。
3. **パーティションとフォーマットの損失：**技術的な理由で、まるでリュックの裏地も体積を占めるようなもの。

**スマホ購入のちょっとしたアドバイス**

現在 128GB のスマホは基本的に足りない。256GB から始めるのが無難だ。撮影やゲームが好きなら、512GB 以上を直接お勧めする。覚えておいてほしいのは、ストレージ容量は余るくらいがよく、足りなくなるよりはましだということ。

**実践してみよう：スマホの「身分証」を見てみよう**

[設定] を開き、デバイス情報（または類似オプション）を見つけ、スマホの型番、システムバージョン、総ストレージ容量と空き容量をメモする。これはスマホを理解する第一歩だ。ついでにバッテリーの健康状態も見ておこう。

## 五、あなたのスマホは「デジタル資産庫」だ

具体的な知識に入る前に、まず核心的な認識を確立しよう：**スマホはもはや通信ツールではなく、私たちの最も重要なデジタル資産庫だ。**連絡先、チャット履歴、家族の写真、仕事のドキュメント、銀行アプリ、支払いコード、健康データ、電子証明書……これらのデータを合わせると、その価値はスマホ本体を超えることが多い。

**なぜこれが重要なのか？**

もしスマホを失くしたり壊したりしたら、失うのは数千元のハードウェアだけではない。**むしろ長年蓄積した再生不能なデータ**　子どもの成長の写真、亡くなった家族の音声、バックアップのない重要な契約書。科学的な整理と防護の習慣は、3 つの次元からあなたを守れる：

- **実行効率の向上：**規範的なファイル分類でシステムの検索が速くなり、スマホが長期間フリーズしない。
- **使用寿命の延長：**合理的なストレージ管理でチップの読み書き負担を減らし、バッテリーもストレージもより長持ちする。
- **データセキュリティの保障：**階層管理＋定期バックアップ＋権限の絞り込みで、「データ災難」を遠ざける。

### 私たちの核心理念：「予防を主とし、予防と治療を結合する」

健康管理は病気になってから医者に行くのではないように、**スマホ管理も「日常の習慣」から始めるべき**。本書が繰り返し強調するのは特定の高度なテクニックではなく、これらの素朴な原則だ：

- **予防を主とする：**アプリは公式ストアからのみインストール、写真を送る前にマスキング、パスワードを使い回さない　これらの「小さな行動」はコストが極めて低いが、大多数のリスクを防げる。
- **定期健診：**毎月 1 回ダウンロードフォルダを整理、四半期ごとに 1 回権限監査、半年ごとに 1 回フルバックアップ　決まったリズムはとっさの火消しより 100 倍効果的だ。
- **逃げ道を残す：**クラウド同期を有効にし、アカウントパスワードを記録し、「デジタル遺産リスト」を書く　万が一の時にあなたと家族が落ち着いて対応できるようにする。

# 第一章：こんにちは、私のパーソナルコンピューター

あなたのスマホは、実はポケットの中で最もパーソナルなスーパーコンピューターだ。かつて人類を月に送ったコンピューターより何倍も強力なのに、静かにあなたの手のひらに収まっている。今日は、この「沈黙の巨人」をしっかり知ろう。

## 一、スマホハードウェアの「五臓六腑」：深度解析

スマホは小さいが五臓は揃っている。解剖の授業のように、これらの精密なコンポーネントを一つずつ知ろう。

### SoC（システムレベルチップ）：スマホの「脳＋心臓」

SoC の正式名称は System on Chip、訳すと「システム全体を 1 つのチップに詰め込む」という意味だ。CPU（中央演算処理装置）だけでなく、CPU、GPU（グラフィックス処理装置）、NPU（ニューラルネットワーク処理装置）、ISP（画像信号処理装置）、ベースバンド（通信を担当）などがすべて統合されている。

これはまるで、都市の政府庁舎、発電所、通信センター、警察署をすべて 1 つのビルに建てるようなもので、効率が極めて高い。

### CPU：汎用計算の核心

CPU は SoC の中で最も核心的なコンポーネントで、大多数の計算タスクの処理を担当する。スマホの CPU は ARM アーキテクチャを採用し、パソコンの x86 アーキテクチャとは異なる。ARM アーキテクチャは縮小命令セット（RISC）を特色とし、消費電力が低く、モバイルデバイスに適している。

**CPU の大小コア設計：**現代のスマホ CPU は普遍的に「大小コア」アーキテクチャ（ARM の big.LITTLE 技術）を採用し、コアを異なる性能レベルに分ける：

- **超大コア/大コア：**性能が最も強く、重度のタスクを担当。ゲームのロード、アプリの起動、複雑な計算など。周波数が最も高い（3GHz+）が、消費電力も最大。

- **中コア：**性能と消費電力のバランスが取れ、日常の中程度の負荷タスクを処理。Weibo の閲覧、動画視聴、マルチタスクの切り替えは主にこれに頼る。

- **小コア/効率コア：**消費電力が最も低く、軽量なバックグラウンドタスクを処理。待機、音楽鑑賞、メッセージ受信など、ほとんど電力を消費しない。

3 つのレベルのコアが協調して動作する：システムスケジューラーがタスク負荷に応じて自動的に割り当てる　軽量タスクは小コアに任せて省電力、重い作業は大コアが強力に処理し、終わったら小コアに戻す。これがスマホがスムーズにゲームを遊べるだけでなく、一日中待機できる理由だ。

**各ブランドのアーキテクチャ特徴：**

- **Qualcomm Snapdragon 8 Elite：**Oryon 自社開発 CPU アーキテクチャ（Nuvia 買収由来）を採用、2 つのスーパーコア（4.32GHz）+ 6 つの性能コア（3.53GHz）、すべて TSMC 第 2 世代 3nm プロセスベース。ARM 公開版アーキテクチャに完全に別れを告げ、シングルコアとマルチコア性能がともに飛躍的に向上。GPU は Adreno 830 でスライスアーキテクチャを採用し、グラフィック性能が 40% 向上、Hexagon NPU の演算力は約 45 TOPS、AI 推論性能が 45% 向上。（注：Samsung Galaxy カスタム版は周波数がより高く、スーパーコアは 4.47GHz に達する。）

- **MediaTek Dimensity 9400：**1 つの超大コア（Cortex-X925、3.62GHz）+ 3 つの性能コア（Cortex-X4、3.3GHz）+ 4 つの効率コア（Cortex-A720、2.4GHz）、全大コアアーキテクチャを継続、TSMC 第 2 世代 3nm プロセス。GPU は Immortalis-G925（12 コア）、ピーク性能が 41% 向上、レイトレーシング性能が 40% 向上。AI プロセッサーは第 8 世代 NPU 890 で、エッジ大モデルの実行、エッジ LoRA トレーニング、エッジ動画生成をサポート。

- **Apple A18 Pro：**2 つの性能コア（約 4.04GHz）+ 4 つの効率コア（約 2.4GHz）、TSMC 第 2 世代 3nm プロセス。6 コア GPU がハードウェアレベルのレイトレーシングをサポート、16 コア NPU（約 35 TOPS）がエッジ AI と Apple Intelligence に強力な演算力を提供。Apple 自社開発アーキテクチャはシングルコア性能と電力効率で依然としてリードし、統合メモリアーキテクチャが AI 推論に高帯域幅の優位性を提供。

- **Huawei Kirin 9020：**Mate 70 シリーズに搭載、自社開発 Taishan CPU アーキテクチャを採用、NPU と ISP 能力を継続的に強化、5.5G と衛星通信をサポート、国産チップの最新ブレイクスルーを代表。

## GPU：グラフィックレンダリングエンジン

GPU（Graphics Processing Unit）はグラフィックと並列計算タスクを専門に担当する。コア数は CPU よりはるかに多く（軽く数百から数千コア）、ただし各コアの機能は比較的シンプルで、大量の類似計算を同時に処理するのに適している。

**グラフィックレンダリングフロー：**

1. **頂点処理：**3D モデル内の各頂点の位置座標を計算。
2. **ラスタライズ：**3D グラフィックを 2D 画面上のピクセル点に変換。
3. **ピクセルシェーディング：**各ピクセルの色、テクスチャ、光照効果を計算。
4. **出力合成：**すべてのレイヤーを合成して最終画面とし、画面に送って表示。

**日常使用への GPU の影響：**ゲームだけではない！インターフェースのスクロール、アニメーション効果、動画再生、写真フィルター、AR アプリはすべて GPU に依存。GPU 性能が低いスマホは、インターフェースのスクロールでフレーム落ちが発生する可能性があり、ゲームでは直接カクつく。

**各ブランドの GPU：**Qualcomm は自社開発 Adreno GPU を使用；MediaTek は ARM Mali GPU を使用；Apple は自社開発 GPU を使用し、性能と電力効率はともに業界のベンチマーク。

### NPU：AI アクセラレーションエンジン

NPU（Neural Processing Unit）はニューラルネットワーク演算のために特別に設計されたプロセッサーだ。CPU や GPU に比べ、NPU は AI タスクの処理時に効率が高く、消費電力が低い。

**ニューラルネットワーク演算方式：**NPU の核心演算は行列乗算と畳み込み演算　これらはニューラルネットワークで最も基礎的で最も頻繁な計算だ。NPU はハードウェアレベルの最適化により、大量の行列演算を並列処理でき、速度は CPU より数十倍速いが、消費電力ははるかに低い。

**AI 処理フロー：**

1. **データ入力：**カメラ画像、マイク音声、センサーデータなど。
2. **前処理：**データのフォーマットとサイズを調整し、ニューラルネットワークの入力要件に適合させる。
3. **推論計算：**NPU が訓練済みモデルを読み込み、入力データに対して前方計算を行い、結果を出力する。
4. **後処理：**AI の出力を利用可能な結果に変換する。例えば顔の座標、文字の内容、シーンのタグなど。

**実際の応用シーン：**

- **コンピュテーショナルフォトグラフィ：**夜景合成、ポートレートボケ、HDR 処理、AI ビューティー。
- **音声アシスタント：**音声認識、自然言語理解、音声合成。
- **リアルタイム翻訳：**AI 字幕、会話翻訳。

- **スマート認識：**文字認識（OCR）、物体認識、シーン認識。
- **オンデバイス大模型：**最新のフラッグシップチップはスマートフォン端末で小型 AI 大模型（7B パラメータクラス）の実行をサポートし、オフライン AI 対話、ドキュメント要約、スマートライティングなどの機能を実現している。Snapdragon 8 Elite の Hexagon NPU の演算力は約 45 TOPS、Dimensity 9400 の NPU 890 もオンデバイス大モデルをサポートし（LLM 推論速度が 80% 向上）、Apple A18 Pro の 16 コア NPU は約 35 TOPS の演算力を提供する。
- **AI 画像処理：**AI 消去（写真内の通行人をワンクリックで削除）、AI 画像拡張（画像の境界を拡張）、AI スタイル変換、テキストからの画像生成など。

### ISP：イメージングシグナルプロセッサ

ISP はカメラから伝えられる生データを処理し、光信号を美しい写真に変換する。

**性能の見方は？**型番を見る！Apple A18 Pro、Qualcomm Snapdragon 8 Elite、MediaTek Dimensity 9400、Huawei Kirin 9020 はいずれもフラッグシップ級。ミドルレンジは Snapdragon 8s Gen3/7+ Gen3、Dimensity 8400 を見る。コア数だけを見ず、アーキテクチャとプロセス（ナノ数値）の方がより重要。3nm プロセスは現在最も先進的で、電力効率は 5nm や 4nm をはるかに上回る。

### RAM（メモリ）：臨時の作業台

RAM はあなたのデスクのようなもので、大きいほど同時に広げられるファイルが多くなる。12GB が現在の主流で、16GB はヘビーユーザー向け、24GB 以上はゲーマーやギークの選択だ。

RAM のタイプも重要：LPDDR5 は LPDDR4X より速く省電力。Apple は RAM 容量が大きくないが、システム最適化が優れているため「十分」に見える。

**RAM が不足するとどうなる？**アプリはシステムにバックグラウンドでキルされ、切り替え時に再読み込みが必要になる。WeChat から戻ったらゲームが消えていた、というのはこのためだ。

### ストレージ（ROM/フラッシュメモリ）：長期倉庫

ストレージは本棚のようなもので、すべての写真、動画、アプリが保存される場所。現在の主流は UFS 3.1 または UFS 4.0 で、速度は古い eMMC より数倍速い。

**なぜストレージ速度が重要？**アプリの起動速度、ゲームのロード速度、写真の保存速度を決定する。長く使うと、低速ストレージはスマートフォンをどんどん重くさせる。

### 画面：あなたの目

画面はスマートフォンとやり取りする主要な窓口であり、良し悪しが体験に直接影響する。

- **素材：**OLED は色彩が鮮やかでコントラストが高く省電力（黒を表示する時は画素が発光しない）だが、低輝度ではフリッカーが目に悪い可能性がある。LCD は目に優しくコストが低いが、コントラストと色彩は OLED に劣る。

- **解像度：**1080P が主流で、2K はより鮮明だが消費電力が多い。Apple の「Retina ディスプレイ」の概念は、一定の距離では肉眼で画素を見分けられないということ。

- **リフレッシュレート：**60Hz が標準、90Hz/120Hz はより滑らかで、スクロール時の滑らかさが際立つ。高リフレッシュレートは電力を消費するため、多くのスマートフォンはコンテンツに応じて動的に調整する可変リフレッシュレート（LTPO）をサポートしている。

- **タッチサンプリングレート：**タッチの応答速度に影響。ゲーマーは要チェック、高いほど指に追従する。

- **調光方式：**DC 調光は PWM 調光より目に優しく、低輝度でも目が疲れにくい。現在多くのスマートフォンは高周波 PWM 調光をサポートしており、これも目の保護のため。

### 画面 HDR 技術

HDR（High Dynamic Range）は画面に明るい部分と暗い部分を同時に表示し、画面の階調感を大幅に向上させる。

- **実現原理：**通常の画面の輝度範囲は限られており（SDR）、最も明るい部分と最も暗い部分の差が十分でない。HDR 画面のピーク輝度はより高く（通常 1000 ニット以上）、コントラストが大きく、色域が広く、より多くのディテールを表示できる。

- **主流規格：**HDR10（最も基本的で無料オープンソース）、HDR10+（Samsung 推進、動的メタデータ）、Dolby Vision（最高画質だがライセンスが必要）。スマートフォンがどの規格をサポートするかは画面ハードウェアとシステムライセンスによる。

- **実際の効果：**HDR 動画を視聴する際、太陽光がよりまぶしく、影がより深く、色彩がより豊かになる。前提は動画自体が HDR 制作され、画面も HDR 表示をサポートしていること。

### イメージング HDR 技術

撮影時の HDR と画面の HDR は別物。イメージング HDR はアルゴリズムで「明るい部分の露出オーバー、暗い部分の真っ黒」問題を解決する。

- **アルゴリズム原理：**スマートフォンはシャッターを押す際、実際には異なる露出の複数の写真を撮影し（露出不足で明部ディテールを保持、露出オーバーで暗部ディテールを保持、正常露出で中間調を保持）、それらを 1 枚のハイダイナミックレンジの写真に合成する。

- **処理フロー：**マルチフレームキャプチャアライメント補正（手ブレ防止）露出融合/トーンマッピング色補正最終写真を出力。
- **効果最適化：**現在のスマートフォンは「リアルタイム HDR」を実現しており、ファインダーで見た画面が HDR 処理後の効果そのもので、WYSIWYG。Apple の Smart HDR と Android の自動 HDR はどちらも非常に成熟している。

### バッテリーと充電：エネルギーの源

バッテリーはスマートフォンで最も「短命」の部品で、2 3 年使うと明らかに劣化する。現在の主流容量は 5500-7500mAh。

- **充電功率：**Android 陣営はすでに 100W さらには 200W まで競争し、10 数分で満充電。Apple はまだ 20-30W で停滞。急速充電はバッテリーを傷めるか? 純正アクセサリを使えば問題ない。スマートフォンには温度制御保護がある。
- **ワイヤレス充電：**便利だが効率が低く発熱が大きい。オフィスでの手軽な充電に適し、急いで出かける時には不向き。
- **リバース充電：**スマートフォンがモバイルバッテリーになり、イヤホンや時計に充電する。緊急用。

### カメラ：光と影の魔法

現在のスマートフォンのカメラは 3 4 つが当たり前で、それぞれ役割がある：

- **メインカメラ：**最も頻繁に使用する。センサーが最も大きく、画質が最も良い。
- **超広角：**風景、建物を撮影。視野がより広い。
- **望遠：**遠くを撮影。光学ズームとデジタルズームの区別がある。
- **マクロ：**極めて近い距離の小さなものを撮影。

**画素数がすべてではない！**センサーサイズ（大きいほど有利）、絞りサイズ（f/1.8 は f/2.4 より大きい）、手ブレ補正（OIS 光学手ブレ補正が重要）はすべて画質に影響する。コンピュテーショナルフォトグラフィ（AI アルゴリズム）はますます重要になり、複数の写真を 1 枚の良い写真に合成できる。

### ペリスコープ式望遠レンズ

従来の望遠レンズは長い光路が必要で、スマートフォンが厚くなる。ペリスコープ式望遠は 90 度のプリズムで光を屈折させ、スマートフォン内部で光路を横方向に展開することで、スマートフォンの薄さを保ちながら 5 倍さらには 10 倍の光学ズームを実現する。

- **構造原理：**光がレンズから入るプリズムで 90 度屈折スマートフォンの幅方向に沿ってレンズ群を通過センサーに到達。縦置きの望遠鏡を横置きにするようなもの。

- **光学的特性：**光学ズーム倍数が高く（5x-10x）、画質はデジタルズームをはるかに上回る。ただしセンサーサイズは通常メインカメラより小さく、暗所での表現は一般的。
- **撮像優位性：**月を撮る、遠くの建物を撮る、ステージのクローズアップを撮るなど、一般のスマートフォンでは到達できない。

### OIS 光学手ブレ補正 vs EIS 電子手ブレ補正

手ブレ補正は撮影と録画の画質に影響する重要技術で、2 つの方式は原理が全く異なる：

- **OIS（光学手ブレ補正）：**レンズまたはセンサーを物理的に移動させて手ブレを相殺する。レンズ内部にマイクロジャイロスコープがあり、ブレの方向と振幅を検出してモーターで鏡片群を逆方向に移動させる。長所は画質を損なわないこと、短所はコストが高くスペースを取ること。
- **EIS（電子手ブレ補正）：**ソフトウェアアルゴリズムで画面をクロップおよびオフセットしてブレを相殺する。スマートフォンはやや広い範囲の画面を録画し、ジャイロスコープデータに基づいて安定した領域をクロップする。長所は追加ハードウェアが不要なこと、短所は画面がクロップされる（視野が狭くなる）ことと、暗所でブレる可能性があること。
- **実際の選択：**撮影では OIS を優先し、録画では両者を組み合わせるのが最も効果的。現在多くのフラッグシップスマートフォンは「Sensor-Shift」センサーシフト手ブレ補正をサポートし、従来のレンズ手ブレ補正より効果が良い。

### 光学ズーム vs デジタルズーム

- **光学ズーム：**鏡片群を物理的に移動させて焦点距離を変更し、画質を損なわない。スマートフォン上の光学ズームは通常固定倍数のみ（2x、3x、5x、10x など）で、カメラのように連続ズームはできない。
- **デジタルズーム：**画面の中央領域を拡大し、本質はクロップ＋補間拡大。拡大するほど画質が劣化する。100 倍デジタルズームは見た目はかっこいいが、実際の画質はかなりぼやけている。
- **ハイブリッドズーム：**光学とデジタルを組み合わせ、光学ズーム倍数の間をマルチフレーム合成アルゴリズムで補完する。純デジタルより効果は良いが、光学には及ばない。

### オーディオ：スピーカーと独立グラフィックチップ

**モノラルスピーカー vs デュアルスピーカー：**

- **モノラルスピーカー：**底部に 1 つの発音ユニットのみで、音が片側に偏り立体感がない。中低価格帯のスマートフォンでよく見られる。

- **デュアルスピーカー（ステレオ）：**上部と下部に 1 つずつのスピーカーで、ステレオ音場を形成。動画視聴や音楽鑑賞時に左右のチャンネルがはっきり分かれ、没入感が大幅に向上。受話器を上部スピーカーとして兼用するのが一般的な方式。
- **音質の違い：**デュアルスピーカーは単に「音が大きい」だけでなく、空間感と階調感が鍵。モノラルスピーカーで映画を見ると、何かが足りないと感じる。

**ゲーミングスマートフォンの独立グラフィックチップ：**

一部のゲーミングスマートフォンとハイパフォーマンススマートフォンには独立した表示処理チップが搭載されており、その役割は：

- **フレーム補間：**30fps のゲーム画面をアルゴリズムで 60fps や 120fps に補間し、画面をより滑らかにする。この処理は独立グラフィックチップ上で行われ、SoC の GPU リソースを消費しない。
- **超解像度：**低解像度画面を AI アルゴリズムで高解像度に引き上げ、画質をより鮮明にする。
- **消費電力削減：**フレーム補間と超解像度は独立グラフィックチップが担当するため、SoC の負荷が下がり、発熱と消費電力が減少し、ゲームがより安定する。

### GPS 測位：シングル周波数とデュアル周波数

スマートフォンの測位は主に GPS などの衛星ナビゲーションシステムに依存している。

- **シングル周波数 GPS：**L1 周波数帯の信号のみを受信し、測位精度は約 3-5 メートル。開けた場所では十分だが、高層ビルが密集する都市の峡谷では、信号の反射により測位のズレが大きくなる。
- **デュアル周波数 GPS：**L1 と L5 の 2 つの周波数帯の信号を同時に受信。L5 周波数帯は波長がより長く、耐干渉能力がより強い。デュアル周波数の連合計算で電離層誤差の大部分を除去でき、測位精度は 1-3 メートルに達する。都市でのナビゲーション、配車アプリの測位で優位性が顕著。
- **マルチコンステレーションサポート：**現代のスマートフォンは GPS だけでなく、北斗（中国）、GLONASS（ロシア）、Galileo（ヨーロッパ）など複数の衛星システムをサポート。同時に接続する衛星が多いほど、測位が速く正確になる。

### センサー：スマートフォンの小さな触角

スマートフォンの中には十数個のセンサーが隠れて、黙々と働いている：

- **加速度計＋ジャイロスコープ：**スマートフォンの姿勢を感知し、ゲームでの方向転換や画面の自動回転を行う。
- **光センサー：**画面の明るさを自動調整。

- **近接センサー：**通話時にスマートフォンが耳に近づくと自動で画面を消灯。
- **指紋センサー：**現在の主流は画面下光学式または超音波式指紋。
- **顔認識：**Apple の Face ID は 3D 構造光でセキュリティレベルが高い。多くの Android は 2D 写真認識で、セキュリティは一般的水準。
- **画面の 2 種類の素材：**近距離無線通信。交通カード、ドアカード、銀行カードをかざす。
- **気圧計：**海抜高度を測定。階段を登るのも検出できる。
- **ホールセンサー：**マグネットケースの開閉を検出。

## 二、オペレーティングシステム：Android VS iOS VS HarmonyOS

ハードウェアは体、オペレーティングシステムは魂。3 つのシステムはそれぞれ特色がある：

- **Android：**オープンソースコミュニティのようで、スマートフォンメーカーが深くカスタマイズでき、選択肢が多い。テーマの変更、サードパーティアプリストアのインストール、各種遊び方を試せる。しかしオープンゆえにフラグメンテーションが深刻で、ブランドごとに体験の差が大きい。
- **iOS：**手入れされた庭のようで、Apple がハードウェアとソフトウェアを一手に握り、体験が統一され滑らか。アプリの品質が高く、プライバシー保護も良いが、自由度が低く、多くのことは Apple のルールに従う必要がある。
- **HarmonyOS：**Huawei の新種で、分散能力を主打し、異なるデバイスをシームレスに連携できる。後で専用に解説する。

3 つのエコシステムはいずれも極めて豊富で、好きなライフスタイルを選べばよい。絶対的な善し悪しはなく、適しているかどうかだけだ。

## 三、スマートフォン端子の全解析

スマートフォン端子は外部世界とつなぐ橋で、端子により差が大きい：

### Micro USB

- **物理的特性：**平らな台形の端子で、表裏の区別がある（逆挿しすると入らない）。スマートフォンでは基本的に廃止されたが、一部のローエンド機器や旧機器ではまだ使用されている。
- **転送制限：**USB 2.0 規格のみサポート、最高転送速度 480Mbps、充電功率は通常 10W を超えない。

- **欠点：**表裏が汎用でない、抜き差し寿命が短い、転送速度が遅い、充電功率が低い。

### Lightning（Apple Lightning 端子、廃止済み）

- **物理的特性：**Apple 自社開発の 8 ピン端子で、表裏どちらでも挿入可能。iPhone 14 およびそれ以前の機種で使用され、現在は全面廃止。
- **転送制限：**Lightning 端子は物理的に USB 2.0 規格のみサポートし、最高転送速度 480Mbps。USB-C to Lightning ケーブルを使用しても、転送速度は Lightning 端子自体に制限される。
- **廃止理由：**EU が 2024 年からすべてのスマートフォンに USB-C 端子の統一使用を法規で要求し、Apple は iPhone 15 シリーズから全面的に Type-C に移行。Lightning 端子は歴史の舞台から正式に退場。
- **注意：**Apple の MFi 認証ケーブルには認証チップが内蔵されており、非認証ケーブルは正常に充電やデータ転送ができない場合がある。旧型 iPhone をまだ使用している場合は、Lightning ケーブルを用意しておくことをお勧めする。

### USB Type-C

- **物理的特性：**24 ピンの楕円形端子で、表裏どちらでも挿入可能。スマートフォンと多くの電子機器の標準端子になった。EU は 2024 年からすべてのスマートフォン、タブレット、イヤホンなどの携帯機器に USB-C 端子の統一使用を立法で要求し、世界の他の地域も追随している。
- **転送プロトコル：**Type-C は端子形状にすぎず、実際の転送能力は内部プロトコルによる。USB 2.0（480Mbps）、USB 3.0/3.1/3.2（5-20Gbps）、さらには USB4（40Gbps）をサポート。
- **重要な区別：**すべての USB-C ポートが USB 3.0 に対応しているわけではありません！多くのスマホは USB-C ポートを採用していても、内部は USB 2.0 のピンしか接続されておらず、転送速度は旧ポートと同じくらい遅いです。iPhone 16/17 標準版は USB 2.0 速度のみ対応、Pro 版のみ USB 3.0 に対応（USB-C to USB-C ケーブルが必要）です。
- **充電能力：**USB PD などの急速充電プロトコルをサポートし、最大 240W 充電をサポート可能。
- **ビデオ出力：**DisplayPort Alt Mode をサポートし、変換ケーブルでモニターにビデオを出力できる。

### 5mm オーディオ端子

- **物理的特性：**円形の 3 極または 4 極ジャックで、3 極は左右チャンネルのオーディオを転送し、4 極はマイクチャンネルが 1 つ追加。

- **特徴：**アナログ信号転送で、プラグアンドプレイ、ドライバーや変換が不要。音質が安定し、遅延ゼロ。

- **現状：**大部分のフラッグシップスマートフォンは 3.5mm 端子を廃止し、Type-C 変換ケーブルまたは Bluetooth イヤホンが必要。一部のミドルレンジ機やゲーミングスマートフォンはまだ保持している。

## 四、スマートフォン製品体系と命名ルール

各ブランドの目まぐるしい製品ラインに対し、命名ルールを理解すれば自分に合った機種を素早く見つけられる。

### 主流ブランドの製品シリーズ：

- **Apple：**iPhone（標準版）/ iPhone Plus（大画面版）/ iPhone Pro（プロ版）/ iPhone Pro Max（大画面プロ版）/ iPhone SE（低価格版）/ iPhone 16e（低価格 AI 版、A18 チップと Apple 自社開発 C1 ベースバンド搭載）。2026 年 iPhone 17 シリーズは全面的に A19 Pro チップを搭載し、AI 能力がさらに向上しています。

- **Huawei：**Mate シリーズ（ビジネスフラッグシップ）/ P/Pura シリーズ（イメージングフラッグシップ）/ Mate X シリーズ（折りたたみフラッグシップ）/ nova シリーズ（若者向けミドルレンジ）/ Enjoy シリーズ（エントリー）

- **Xiaomi：**Xiaomi 数字シリーズ（フラッグシップ）/ Xiaomi MIX（探求フラッグシップ/折りたたみ）/ Redmi K シリーズ（ハイパフォーマンスフラッグシップ）/ Redmi Note シリーズ（ミドルレンジ）/ Redmi 数字シリーズ（エントリー）

- **OPPO：**Find X シリーズ（フラッグシップ）/ Reno シリーズ（ミドルレンジ）/ A シリーズ（エントリー）/ OnePlus 数字シリーズ（ハイパフォーマンスフラッグシップ）

- **vivo：**X シリーズ（イメージングフラッグシップ）/ iQOO 数字シリーズ（ハイパフォーマンスフラッグシップ）/ S シリーズ（セルフィーミドルレンジ）/ Y シリーズ（エントリー）

- **Samsung：**Galaxy S シリーズ（フラッグシップ）/ Galaxy Z Fold/Flip（折りたたみフラッグシップ）/ Galaxy A シリーズ（ミドルレンジ）

### 一般的なサフィックスの意味：

- **Pro：**Professional の略で、プロ版。通常標準版をベースにカメラ、画面、プロセッサなどのコア構成をアップグレード。

- **Max：**最大版。画面が最大、バッテリーも最大だが、構成は必ずしも Pro より高いとは限らない。

- **Ultra：**究極版。通常そのシリーズの最高構成を代表し、スペックが最も豪華で、価格も最高。Samsung S24 Ultra、Xiaomi 14 Ultra など。
- **SE：**Special Edition の特別版で、実際には低価格版が多い。構成を削減し、価格を下げる。iPhone SE など。
- **Plus：**拡大版。画面は標準版より大きく、バッテリーも大きいが、構成は基本的に同じ。
- **Lite/青春版：**軽量版。構成を簡素化し、価格をより低く。
- **Note：**元々は大画面スマートフォンを指したが、現在は各ブランドで用法が異なる。Redmi Note はミドルレンジシリーズ。
- **Neo：**新鋭版。通常はパフォーマンス志向のミドルレンジ機種を指す。

## 五、ベースバンドとアンテナ：スマートフォン通信の縁の下の力持ち

スマートフォンが通話やインターネット接続ができるのは、ベースバンドチップとアンテナのペアのおかげ。

### ベースバンドチップ：

ベースバンドはスマートフォンで無線通信を担当するプロセッサで、デジタル信号を無線電波に変調して送信し、受信した無線電波を復調してデジタル信号にする。ベースバンドの性能は通話品質、インターネット速度、信号の安定性に直接影響する。Qualcomm が最大のスマートフォンベースバンドサプライヤーで、MediaTek、Samsung も自社開発ベースバンドを持つ。Apple は iPhone 16e から自社開発の C1 ベースバンドチップを搭載し、Apple が正式に自社開発ベースバンド時代に突入したことを示している。C1 ベースバンドは 4nm プロセスを採用し、消費電力性能に優れる。iPhone 16/16 Plus/16 Pro/16 Pro Max は引き続き Qualcomm ベースバンド（Snapdragon X75 5G）を使用し、後続機種は段階的に自社開発ベースバンドに移行すると予想される。より早期の iPhone XS/11 シリーズが Intel ベースバンドを使用していた時は信号性能が悪く、多くのユーザーの悩みの種だった。

### アンテナ設計：

スマートフォンのアンテナは本体内部に隠され、通常はフレーム周りに配置される。アンテナ設計は信号受信性能に直接影響する：

- **MIMO 技術：**マルチプルインプットマルチプルアウトプットで、複数のアンテナを使用して同時にデータを送受信し、速度と安定性を向上。4×4 MIMO は 2×2 MIMO より速い。
- **アンテナ数：**フラッグシップ機は通常十数個のアンテナを持つ（セルラー、Wi-Fi、Bluetooth、NFC、GPS それぞれ独立したアンテナが必要）。

- **金属ボディの影響：**金属は信号を遮蔽するため、金属ボディのスマートフォンはアンテナ帯（フレーム上のプラスチックの帯）を設計する必要がある。ガラスとプラスチックのボディは信号への影響が小さい。

### ネットワーク技術の進化：

- **：現在カバレッジが最も広く、速度も十分（理論上最高 100Mbps+）、消費電力が比較的低い。：**理論ピーク速度は下り 1Gbps/上り 500Mbps（Cat.18）、実際の体験は通常 50-150Mbps。カバレッジが最も広く、十分に使える。

- **5G NR：**理論ピーク速度は下り 10Gbps+。Sub-6GHz 周波数帯はカバレッジが比較的良い。mmWave ミリ波は速度が極めて速いがカバレッジが極めて小さい。

- **5G-A（5G Advanced）：**5G の拡張版で、5.5G とも呼ばれます。下り速度は最大 10Gbps、上り 1Gbps に達し、接続数密度と測位精度が大幅に向上します。2024 年から中国移動、中国電信、中国聯通が主要都市で 5G-A ネットワークの展開を開始し、Huawei Mate 70/80 シリーズ、Xiaomi 15/16 シリーズなどのフラッグシップ機が 5G-A に対応しています。5G-A はまた「通信と感知の統合」能力をもたらし、基地局が同時に通信と環境感知を行うことで、低空経済（ドローン配送）や車載 IoT 向けのインフラを提供します。

### ネットワーク周波数帯の分類：

スマートフォンがサポートするネットワーク周波数帯は、どの国、どのキャリアのネットワークで使用できるかを決定する：

- **FDD-LTE Band（B シリーズ）：**B1、B3、B5 など、4G 周波数帯。国により使用周波数帯が異なり、中国は主に B1/B3/B5/B8/B34/B38/B39/B40/B41 を使用。

- **TDD-LTE Band：**4G のもう一つの方式で、中国は B38/B39/B40/B41 を大量に使用。

- **5G NR Band（N シリーズ）：**n1、n3、n28、n41、n78、n79 など。n78（3.5GHz）は中国 5G の主力周波数帯で、n41（2.5GHz）は China Mobile が多く使用。

- **購入アドバイス：**海外で使用する必要がある場合、購入前にスマートフォンが対象国の周波数帯をサポートしているか確認すること。中国国内版スマートフォンは通常中国の周波数帯のサポートが最も充実しているが、一部の海外周波数帯が欠けている場合がある。

## 六、あなたのスマートフォンは、一体いくつの役割を演じている？

それはもはや電話ではない：

1. **通信ツール：**電話、SMS、WeChat、メール……

2. **スマートカメラ**：撮影、録画、ドキュメントスキャン……
3. **財布**：支払い、交通カード、銀行カード、会員カード……
4. **情報ポータル**：ニュース、天気、株価、知識……
5. **ゲーム機**：パズルゲームから 3A タイトルまで……
6. **モバイルオフィス**：ドキュメント、スプレッドシート、会議、コラボレーション……
7. **ヘルスアシスタント**：歩数計、心拍数、睡眠、運動……
8. **スマートホームリモコン**：照明、エアコン、テレビ、ロボット掃除機の制御……
9. **デジタル身分証**：電子身分証、電子運転免許、健康コード……
10. **エンターテインメントセンター**：音楽、動画、電子書籍、ポッドキャスト……

考えてみてほしい、小さな一台のデバイスが、どれほどの生活を担っているか！だから、それを使いこなすことを学ぶことは、より良い生活を学ぶことなのだ。

**実践してみよう：ハードウェア構成を理解する**

[設定] で、プロセッサの型番、RAM、ストレージ（ROM）の具体的な容量を見つけられるか試してみよう。スマートフォンの能力を把握しておく。Android ユーザーは「安兔兔」や「DevCheck」をダウンロードして詳細なハードウェア情報を確認できる。

## 付録：Android スマートフォンメモリ拡張解析

スマートフォンを買う時、「*12GB+8GB RAM*」といった宣伝をよく見かけるだろう——まるで *8GB* の *RAM* が無料で増えたかのように。しかし、それは本当にスマートフォンを速くするのか？逆にスマートフォンを傷めないか？この節では「メモリ拡張」という技術を徹底的に解説する。

### 一、本質的定義

Android スマートフォンの「メモリ拡張」（メモリ拡張、メモリ融合、RAM Plus などとも呼ばれる）は**物理的に RAM チップを増設するわけではなく**、システムレベルのソフトウェア技術だ：スマートフォンの内部ストレージ容量（ROM/UFS フラッシュ）の一部を動的に切り分け、仮想 RAM として使用する。

例えるなら：物理 RAM は「高速作業台」で、拡張メモリは「臨時物置」。作業台が足りなくなると、システムは一時的に不要なデータを物置に移動し、必要な時に戻す。物置は容量が大きいが、移動には直接作業台に置くより時間がかかる。

### 二、技術原理

主流メーカーの実装方式は主に 2 種類ある：

### zRAM 圧縮

RAM 内部に圧縮データ空間を切り分け、非アクティブなアプリコードを圧縮保存する。圧縮率は約 2:1。データは RAM 内にあり、速度が速くフラッシュを消耗しないが、CPU が圧縮・解凍に関与する必要がある。

### swap パーティション

UFS フラッシュ内に固定空間を直接切り分け、仮想メモリとしてバックグラウンドプロセスのデータをストレージに書き込む。ストレージ帯域幅に依存し、読み書き速度は物理メモリよりはるかに低い。

メーカーが宣伝する「12GB+8GB RAM」は通常 12GB 物理 RAM+8GB フラッシュ仮想メモリの組み合わせ。注目すべきは、zRAM はデフォルトで本体ストレージを占有せず、zRAM がデータで完全に満たされた時のみシステムがストレージ上にスワップファイルを作成し、この状況は極めて稀だ。

## 三、速度差が主要なボトルネック

この技術の根本的な限界は速度差にある。現在主流の UFS 4.0 フラッシュのシーケンシャル読み取り速度は約 4200MB/s で、LPDDR5X 物理メモリの 1/10  1/15 にすぎない。そのため軽負荷の zRAM 圧縮段階では速度が実 RAM に近いが、高負荷シーンに入ると速度は数十分の一に急減する。

## 四、適用シーンと効果の限界

メモリ拡張は万能ではなく、機種や使用シーンにより効果が異なる：

- **効果的なケース：**6GB 物理メモリ +UFS 3.1 以上、空き容量 ≥30GB の機種で、3GB 拡張を有効化するとバックグラウンド保持数が 3-4 個増え、再ロード率が 68% 低下。デリバリードライバーや配車ドライバーなどのヘビーマルチタスクユーザーに適する。
- **日常シーン：**WeChat のマルチ起動、ブラウザタブの保持、インスタントメッセージの応答などに顕著な改善があり、バックグラウンドアプリのコールド起動時間が明確に短縮。
- **効果が限定的なシーン：**大型ゲームのロード、4K 動画編集などの高帯域幅シーンでは向上が限定的。
- **有害なケース：**8GB メモリ +UFS 2.2 で空き容量不足の機種では、有効化するとゲームのフレームレート変動が逆に拡大し、ショート動画視聴で消費電力が増え、切り替えがカクつく。
- **無効なケース：**16GB 以上の大容量メモリ機種では効果はほぼゼロ。最大でもう 1 個アプリを保持できるだけで、CPU 負荷が 12% 増加し、発熱が 2-3℃ 上昇する。RAM が 12GB より大きい機種では、無意味な圧縮計算を避けるため直接無効化を推奨。

## 五、各メーカーの命名と有効化方法

ブランドにより呼び方が異なる：Xiaomi は「メモリ拡張」、Huawei は「メモリ融合」、Samsung は「RAM Plus」、vivo は「Extended RAM」、OnePlus は「Memory Extension」と呼ぶ。この機能はシステムネイティブ能力で、Android の基盤バージョンとは直接バインドされない。

一般的な有効化パス：【設定】【詳細設定】【メモリ】または【パフォーマンス】サブ項目「メモリ拡張」を有効化し容量レベルを選択（通常 2GB 7GB）、再起動後に有効化。有効化後、システムは「8GB+4GB」のようなメモリ組み合わせを表示する。

**注意事項**

- メモリ拡張は本体ストレージ容量の一部を占有するため、残りストレージが 20-30GB 以上あることを確認すること。そうでないとシステムの滑らかさに影響する可能性がある。
- フラッシュを仮想メモリとして使用すると読み書きによる消耗がある。現代の UFS フラッシュは寿命が長いが、長期高頻度の swap はストレージの劣化を加速する可能性がある。
- スマートフォン自体の物理メモリが小さい場合（4GB 以下など）、メモリ拡張を有効化しても一時的なしのぎにしかならず、根本的にカクつきの問題は解決しない。

**まとめ**

メモリ拡張は成熟したシステムレベルの最適化技術で、ハードウェアを変えずに中低価格帯機種のマルチタスク能力とバックグラウンド保持率を向上できるが、フラッシュと物理メモリの大きな速度差に制限され、「万能薬」ではない。有効化するか、どの程度にするかは、機種の物理メモリサイズ、フラッシュ規格、残りストレージ容量、使用習慣を総合的に判断する必要がある。

**やってみよう：あなたのスマートフォンにメモリ拡張があるか確認**

1. 【設定】を開き、「メモリ拡張」「メモリ融合」「RAM Plus」または「Extended RAM」を検索。
2. 見つけたら現在の有効/無効、および拡張容量設定を確認。
3. 本節の内容と照らし合わせて判断：あなたの機種の物理メモリはどれくらい？フラッシュ規格は？残りストレージ容量は十分か？
4. 拡張容量を無効化または下げてみて、1 週間バックグラウンド保持と発熱の変化を観察し、自分に最適な設定を見つける。

# 第一章付録：HarmonyOS オペレーティングシステム特集

*HarmonyOS* は *Huawei* が自主開発したオペレーティングシステムで、*2019* 年の発表から現在までにいくつかのメジャーバージョンを経てきた。それは単なる「もう一つの *Android* 代替品」ではなく、独特な設計理念を持つ新種だ。この台頭しつつあるオペレーティングシステムを一緒に深く理解しよう。

## 一、HarmonyOS とは？なぜ他と違うのか？

HarmonyOS の核心理念は「万物互聯」。従来のオペレーティングシステムが主に単一デバイスにサービスを提供するのとは異なり、HarmonyOS は設計当初からマルチデバイス連携を考慮している。

**分散アーキテクチャ**は HarmonyOS の魂だ。簡単に言うと、複数のデバイスを仮想的に 1 つの「スーパーターミナル」にできる。あなたのスマートフォン、タブレット、パソコン、時計、テレビはもはや孤立した個体ではなく、1 つの全体の「器官」となり、互いの能力を呼び出し合える。

例えば：スマートフォンで動画を見ていて大画面で見たくなった時、従来の方法はスクリーンミラーリングだが、HarmonyOS では動画を直接テレビに「流転」して続きを見られ、スマートフォンはリモコンになる。これは単なるスクリーンミラーリングではなく、アプリ自体がデバイスをまたいで実行されているのだ。

## 二、HarmonyOS 4.3：AOSP ベースの成熟バージョン

HarmonyOS 4.3 は現在 Huawei スマートフォンに広くデプロイされているバージョンだ。説明が必要なのは、HarmonyOS 4.3 は Android オープンソースプロジェクト（AOSP）のコードフレームワークと互換性があるため、Android アプリを完全に互換実行できる——Huawei アプリマーケットでダウンロードする大部分のアプリは本質的に依然として Android アプリで、HarmonyOS の互換レイヤーを通じて実行されている。しかし HarmonyOS 4.3 は単なる Android の皮ではなく、AOSP をベースに Huawei 自社開発の分散能力、方舟エンジンなどの技術を大量に追加している。

**コア特性：**

- **Android アプリ互換**：AOSP ベースで開発されているため、HarmonyOS 4.3 はほぼすべての Android アプリを正常に実行でき、ユーザーはほとんど差を感じない。
- **方舟エンジン**：Huawei 自社開発のコンパイラとランタイムで、アプリの実行をより滑らかにより省電力にする。早期バージョンより滑らかさが明確に向上。
- **スーパーターミナル**：コントロールセンターで周囲の他の HarmonyOS デバイスを直感的に確認でき、ドラッグで接続。スマートフォンがパソコンのキーボードマウスを呼び出したり、タブレットがスマートフォンのカメラを呼び出したりするのが便利。

- **サービスカード：**iOS のウィジェットに似ているがより柔軟。アプリアイコンを上スワイプしてカードを展開し、アプリを開かずに主要情報を表示。
- **または：**公式アプリマーケットからのアプリインストールのみを許可し、マルウェアリスクを低減。
- **プライバシーセンター：**アプリ権限を一元管理し、アプリの行動記録を確認。

**ユーザーインターフェースの特徴：**

- **マガジンロック画面：**ロック画面に美しい画像と文章を表示し、スワイプするとデスクトップに入る。
- **万能カード：**デスクトップアイコンをカードに展開し、より多くの情報を表示。
- **スーパー中継ステーション：**一時保存領域で、文字、画像、ファイルを一時保存し、任意のアプリにドラッグして使用。アプリ間共有が極めてスムーズに。
- **スマート検索：**グローバル検索でアプリ、ファイル、設定、ウェブページを検索。

## 三、HarmonyOS NEXT（HarmonyOS 6）：Huawei 自主オペレーティングシステムの真の起点

HarmonyOS NEXT（HarmonyOS NEXT 5.0）は HarmonyOS 4.3 とは根本的に異なる——Huawei 自社開発の HarmonyOS マイクロカーネルを採用し、Linux カーネルと AOSP コードに依存せず、**Android アプリを直接サポートしない**。これは HarmonyOS が「Android 互換の過渡形態」から「完全に独立したオペレーティングシステム」へ正式に移行する重要な一歩だ。

**重大な変革：**

- **完全自主カーネル：**HarmonyOS NEXT は Huawei 自社開発のカーネルを使用し、AOSP コードを一切含まない。システムはより簡素で、より安全で、より効率的。
- **Android アプリ非サポート：**これが最大の変化。HarmonyOS NEXT はネイティブ HarmonyOS アプリしか実行できず、APK ファイルのインストールと実行は不可。アップグレード前に常用アプリに HarmonyOS ネイティブ版があるか確認すること。
- **ネイティブアプリエコシステム：**Huawei は開発者がアプリを HarmonyOS ネイティブプラットフォームに移行することを全力で推進。現在 WeChat、Alipay、Taobao などの主流アプリは HarmonyOS ネイティブ版をリリース済みだが、一部のニッチアプリはまだ適応されていない可能性がある。
- **星盾セキュリティアーキテクチャ：**全く新しいセキュリティ体系で、ハードウェアからソフトウェアまでの全リンク保護。より細粒度の権限制御をサポート、例えば「あいまい測位」——アプリはあなたの大体の位置しか分からず、正確な位置は分からない。

- **AI 深融合：**システムレベルの AI 能力で、単なる特定のアプリに AI があるのではない。例えば AI 字幕は動画をリアルタイム翻訳し、AI 切り抜きは主体をスマート認識。
- **デバイス間シームレス移行：**さらに進んだ分散能力。アプリ状態はデバイス間でシームレスに切り替えられ、スマートフォンで編集したドキュメントがパソコンの前に来ると自動的にパソコンで開く。
- **元サービス：**インストール不要で使用可能な軽量サービス。例えば QR コードをスキャンして直接特定機能を開き、アプリを先にダウンロードする必要がない。

**HarmonyOS NEXT アップグレード前の必読事項**

HarmonyOS NEXT にアップグレードすると、スマートフォンは Android アプリを実行できなくなる。アップグレード前に必ず確認すること：(1) 常用アプリに HarmonyOS ネイティブ版があるか；(2) 仕事や生活に Android アプリに依存する必須シーンがないか。アップグレード後は HarmonyOS 4.3 に戻すことはできない。

**操作ロジックの変化：**

- **ジェスチャーナビゲーション：**左右のエッジスワイプで戻る、下部上スワイプでデスクトップへ、上スワイプ長押しでマルチタスクへ。
- **スマートマルチウィンドウ：**分割画面、フローティングウィンドウ、平行視界など複数のマルチタスク形態をサポート。
- **Changlian 通話：**Huawei のビデオ通話サービスで、画面共有、ビューティー、背景ボケをサポート。

## 四、HarmonyOS エコシステム：デバイスとアプリ

HarmonyOS は単なるスマートフォンシステムではなく、多種デバイスをカバーするエコシステムだ。

**サポートするデバイスタイプ：**

- **スマートフォンとタブレット：**コアデバイスで機能が最も完全。
- **スマートウォッチとバンド：**単独実行またはスマートフォンと連携。
- **スマートスクリーン（テレビ）：**家庭エンターテインメントの中心。
- **スマートスピーカー：**音声インタラクションの入り口。
- **自動車：**HarmonyOS コックピット、スマート運転席体験。
- **スマートホーム：**冷蔵庫、エアコン、洗濯機など。

**アプリエコシステムの現状：**

HarmonyOS アプリマーケットには数百万のアプリがあり、主要なアプリはほぼ揃っています。Huawei も開発者によるネイティブ HarmonyOS アプリの開発を積極的に推進しています。2026 年中盤時点で、WeChat、Alipay、Taobao、Douyin、Meituan、JD.com などの主要アプリはすべて HarmonyOS ネイティブ版をリリースしており、日常使用はほぼ問題ありません。一部のニッチアプリや業界向けアプリはまだ適配中の可能性があるため、アップグレード前に常用アプリが対応しているか確認することをお勧めします。

## 五、HarmonyOS 実用テクニック

- **スーパーターミナル：**画面右上から下スワイプしてコントロールセンターを開き、「スーパーターミナル」をタップすると周囲の他のデバイスが表示され、ドラッグで接続。
- **スーパー中継ステーション：**文字/画像/ファイルを長押し、「中継ステーションにコピー」を選択し、他のアプリで長押しペーストまたはドラッグで取り出す。
- **サービスカード：**アプリアイコンを上スワイプしてカードを展開、カードを長押しでサイズ調整やデスクトップに追加。
- **スマートマルチウィンドウ：**画面の左右エッジから内側にスワイプして長押し、サイドバーを開き、アプリをタップしてフローティングウィンドウで開く。
- **プライバシー保護：**設定 > プライバシーで、アプリ権限使用記録の確認、あいまい測位の設定などが可能。

**注意事項**

HarmonyOS システムは絶えず更新反復されており、バージョンにより多少の差異がある可能性がある。本記事の記述は HarmonyOS 4.3 と 6 の主要特性に基づいており、具体的な機能はあなたのスマートフォンの実際のバージョンによる。システム更新を維持し、最新機能とセキュリティパッチを取得することを推奨。

**今すぐ試そう：HarmonyOS の特色機能を体験**

1. コントロールセンターを開き、「スーパーターミナル」を見つけ、周囲に他の HarmonyOS デバイスがあるか確認。
2. 「スーパー中継ステーション」を試す：文字を中継ステーションにコピーし、メモ帳でドラッグして取り出す。
3. 常用アプリのアイコンをいくつか上スワイプし、サービスカードが何を表示するか見てみる。
4. 設定 > プライバシーで、最近どのアプリがセンシティブな権限を使用したか確認。

# 第二章：デジタルルームを整理する——ファイル管理

スマートフォンを長く使うと乱雑になる。良い習慣はファイル管理から始まる。部屋を定期的に整理する必要があるのと同じように、デジタル空間も手入れが必要だ。整然としたスマートフォンは、物を探しやすいだけでなく、動作もより滑らかになる。

## 一、3 層ストレージ体系：あなたのデジタル倉庫

1. **本体ストレージ**：コア領域で、速度が速く、システムとアプリがここにある。スマートフォンで最も貴重な空間で、慎重に管理すべき。
2. **SD カード（サポートの場合）**：拡張クローゼットで、速度は遅いが、映画、写真などの大容量ファイルに適する。注意：現在多くのフラッグシップ機は SD カードをサポートしない。
3. **クラウドストレージ**：金庫＋同期ドライブ。紛失防止と複数デバイス間同期が可能。iCloud、Baidu Netdisk、OneDrive などがよく使われる。

## 二、「ファイルマネージャー」を知る

Android には通常 [ファイル] または [ファイル管理] アプリがあり、iOS は標準の [ファイル] アプリだ。これらはスマートフォンのすべてのファイルを表示する「エクスプローラー」だ。

### 一般的なフォルダの意味：

- **DCIM**：カメラで撮影した写真と動画。
- **Pictures**：その他の画像、スクリーンショット、保存した画像など。
- **Download**：ブラウザでダウンロードしたファイル。
- **Music**：音楽ファイル。

- **Movies**：動画ファイル。
- **Documents**：ドキュメントファイル。
- **Android**：Android システムフォルダで、中に各アプリのデータがある。通常は手動削除しないこと。

## 三、ファイルシステム構造：より深く理解する

Android は Linux ベースで、ファイルシステムには一定の階層構造がある。一般ユーザーが深く理解する必要はないが、基本的な概念を知っておくと役立つ。

### ストレージパーティション：

- **/system**：システムパーティションで、オペレーティングシステムファイルを格納。一般ユーザーはアクセス権限がなく、root 権限が必要。
- **/data**：データパーティションで、ユーザーデータとアプリデータを格納。日常使用する空間。
- **/sdcard または /storage/emulated/0**：ユーザーがアクセス可能なストレージ空間で、写真やダウンロードファイルはここにある。

### アプリデータストレージ：

- **内部ストレージ**：アプリのプライベート空間で、他のアプリはアクセス不可。アプリアンインストール時に削除される。
- **外部ストレージ**：アプリがユーザーアクセス可能領域に保存するファイル、例えば写真、ダウンロードコンテンツ。アプリアンインストール時に必ずしも削除されない。

## 四、データバックアップと復元：データに保険をかける

データはプライスレス、バックアップは必須の習慣だ。

### バックアップ戦略：

1. **クラウドバックアップ**：自動で便利だが、ネットワークとクラウド容量が必要。iCloud、Huawei Cloud、Xiaomi Cloud などがサポート。
2. **ローカルバックアップ**：データケーブルでパソコンに接続し、公式ツールでバックアップ。速く容量が大きいが、手動操作が必要。
3. **二重バックアップ**：クラウドバックアップ＋ローカルバックアップで最も安全。

### 何をバックアップする？

- **写真と動画：**最も容量を占有し、最も貴重。
- **連絡先：**クラウドアカウントに保存するのが最も便利。
- **WeChat チャット履歴：**WeChat 内蔵のバックアップ機能を使用。
- **重要ドキュメント：**クラウドドライブやパソコンに保存。

## 五、ファイルセキュリティ：プライバシーを守る

- **センシティブファイルの暗号化：**多くのスマートフォンには「セキュアフォルダ」や「プライベート空間」機能があり、センシティブファイルを入れ、パスワードや指紋でアクセスできる。
- **削除 ≠ 抹消：**通常の削除は空間が上書き可能というマークをつけるだけで、データは残っている。完全に削除するには、専用の安全消去ツールを使用するか、複数回上書き書き込みが必要。
- **クラウドセキュリティ：**信頼できるクラウドサービスプロバイダーを選び、二要素認証を有効化し、定期的にログインデバイスを確認。

## 六、容量が足りない？大掃除をしよう

- **キャッシュクリーン：**アプリの一時記憶で、クリーンすると空き容量が増える。設定 > ストレージでクリーン可能。
- **大容量ファイルの検索：**ファイルマネージャーでサイズ順にソートし、容量を占有する「犯人」を見つける。
- **残存物の削除：**アプリアンインストール後、ゴミファイルが残ることがあるので手動で削除。
- **アルバムのクリーン：**重複写真、ブレた写真、不要なスクリーンショットを削除。
- **WeChat のクリーン：**WeChat は容量の大口で、定期的なチャット履歴とキャッシュのクリーンが必須。

## 七、一般的なファイル形式の全解析

スマートフォン内の各種ファイル拡張子は複雑だが、理解すればファイル管理がより上手になる：

| 形式 | タイプ | 特性 | 推奨される開き方 |
|---|---|---|---|
| JPG/JPEG | 画像 | 非可逆圧縮、サイズ小、互換性最高。スマートフォン撮影のデフォルト形式。 | アルバム/任意の画像ビューアー |
| PNG | 画像 | 可逆圧縮、透明背景サポート。スクリーンショット、アイコンに常用。JPG よりサイズ大。 | アルバム/任意の画像ビューアー |
| MP3 | オーディオ | 非可逆圧縮、サイズ小、互換性極めて高い。最も汎用的なオーディオ形式。 | 任意の音楽プレーヤー |
| FLAC | オーディオ | 可逆圧縮、音質は CD と同等、サイズは MP3 の 3-5 倍。オーディオマニアの第一選択。 | 音楽プレーヤー/プロフェッショナル再生アプリ |
| WAV | オーディオ | 無圧縮の生オーディオ、音質最高だがサイズ巨大。プロ録音に常用。 | 音楽プレーヤー |
| M4A | オーディオ | AAC エンコード、MP3 の同ビットレートより音質が良い。Apple エコシステムで常用。 | Apple Music/音楽プレーヤー |
| MP4 | 動画 | 最も汎用的な動画コンテナ形式、互換性極めて高い。スマートフォン録画のデフォルト形式。 | アルバム/任意の動画プレーヤー |
| MKV | 動画 | オープンソースコンテナ形式、マルチオーディオトラック、マルチ字幕サポート。映画リソースに常用。 | VLC/nPlayer |
| DOC/DOCX | ドキュメント | Word ドキュメント。DOCX は圧縮形式でサイズがより小さい。DOC は旧形式。 | WPS/Office |
| PPT/PPTX | プレゼン | PowerPoint プレゼン資料。PPTX は圧 | WPS/Office |

**練習：「ダウンロード」フォルダを整理**

ファイルマネージャーを開き > 「ダウンロード」フォルダに入る。中に何があるか？不要なものを削除し、有用なものを分類（例えば「仕事資料」「スタンプ」フォルダを作成して入れる）。ついでにストレージ容量を確認し、どのタイプのファイルが最も多く占有しているか把握する。

# 八、画像とファイルシステム管理：資産を整然と

前節で「フォルダを知る」ことを学んだが、知るだけでは不十分。本節では**長期的に実行可能な管理規範を主体的に構築する方法**を説明し、ファイルがいつでも見つかり、失われず、覗き見されないようにする。

## ファイル分類と命名規範

混乱の根源はルールがないこと。ルールの構築は実は簡単で、重要なのは**継続**。

- **「時間＋イベント」命名法：**フォルダ名にタイムスタンプとイベント記述を含め、後日の回想と検索に便利。例えば 202606_ 旅行チケット、2025 年年末まとめ、2024 子供の誕生日。これでタイムラインが見え、内容も一目で分かる。
- **3 階層分類体系：**
  - **第 1 層：**ファイルタイプ別（ドキュメント / 画像 / 動画 / オーディオ）
  - **第 2 層：**使用シーン別（仕事 / 生活 / 学習）
  - **第 3 層：**具体的なイベントやプロジェクト別（例：「2026Q2_ プロジェクト A」）
- **月次クリーン計画：**毎月固定日付（給料日や月初 1 日を推奨）に、「ダウンロード」ディレクトリを一括整理：**有用なものをアーカイブ、不要なものを削除、貴重なものを転送保存**。年 12 回の小整理は、1 年分の大掃除よりはるかに楽。

**今月のタスクリスト**

暇な週末を選び、以下の順序で操作：

1. ファイルマネージャーで 2026Q2_ プロジェクト A 形式の今月の仕事フォルダを作成；
2. 「ダウンロード」ディレクトリ内の今週のファイルをすべて確認し、有用なものを残し、アーカイブし、不要なものを削除；
3. 今年の家族写真に 2025 家族、2026 家族という年次フォルダを作成。

## 冗長ファイル処理戦略

スマートフォンを長く使うと「太る」が、多くは冗長ファイルの仕業だ。

- **72 時間自動クリーン：**WeChat、QQ などソーシャルアプリが受信する一時ファイル（スタンプ、チャット画像、ショート動画）は、ほとんど一度見たら不要。[WeChat] > [設定] > [一般] > [ストレージ] で「自動クリーン」を有効化し、期限切れキャッシュを 72 時間に設定。スマートフォンが瞬時にさっぱりし、重要な記録も失われない。
- **ストレージ容量の指向性クリーン：**Android [設定] > [ストレージ] > [クリーン加速]、iOS [設定] > [一般] > [iPhone ストレージ] で、システムが「キャッシュ / 一時ファイル / アンインストール可能アプリ / 大容量ファイル」別にリスト化。重点クリーン：(1) キャッシュファイル；(2) 期限切れインストーラ（アプリアップグレード後に残る旧版インストーラ）；(3) 重複ファイル（同名同サイズの写真）。
- **自動分類ルール：**大部分のブランドのアルバムアプリはスマート分類をサポート：人物、風景、ドキュメント、スクリーンショット。有効化すると、新しく撮った集合写真は自動的に「人物 > 張三」に分類され、スキャンした契約書は自動的に「ドキュメント」に分類される。**月次整理と組み合わせることで、「物が見つからない」確率を大幅に低減。**

## 画像安全処理規範

画像はスマートフォンで最も「センシティブ」なデータタイプだ——何気なく撮った 1 枚であなたの位置、デバイス、時間が漏れる。私たちは**「二重保護」ワークフローの構築：**

**証明写真/センシティブスクリーンショット送信の標準フロー**

1. **可視情報のマスキング：**システム標準またはサードパーティツール（Meitu XiuXiu、Snapseed など）で**身分証番号の中間数桁、ナンバープレート、部屋番号、氏名、金額**などのキーフィールドを塗りつぶす；
2. **メタデータ剥離：**送信前にアルバムで「共有」または「エクスポート」をタップすると、システムが位置情報を保持するか尋ねるので**「保持しない」**を選択。これで撮影デバイスの型番、GPS 座標、撮影時間が画像と一緒に送信されない；
3. **一時透かし**（オプション）：センシティブスクリーンショットに「XX 手続き専用」のテキスト透かしを追加し、悪用を防止。

メタデータ（EXIF）削除の一般的な方法（基本操作、詳細な保護は第六章「写真のセンシティブ情報処理」を参照）：

- **システム標準：**iOS [設定] > [プライバシーとセキュリティ] > [カメラ] で「正確な位置」をオフにできる；共有時に「現在の写真のみ」を選択すると**大部分の EXIF を自動剥離。**
- **Android：**大部分のブランドのアルバムアプリは「共有」時に「位置情報を削除するか」を確認し、チェックを入れるだけ。
- **プロフェッショナルツール：**ExifTool（PC 版）、Metapho（iOS）、Photo Metadata Remover（Android）ですべてのメタデータを完全に削除可能。

長期保存が必要なセンシティブ画像（身分証、銀行カード、戸籍謄本のスキャン、パスポート）は、暗号化アルバムに保存することを推奨：：

- Huawei/Xiaomi：[アルバム] > 写真を長押し > 「セキュアボックスに移動」；
- サードパーティ暗号化アプリ：
- 360 セキュアアルバム、、、、など、単独パスワード保護で隠蔽性がより高い。第三章：スマートフォンに「生命線」をつなぐ——ネットワーク

# 第三章：スマホに「ライフライン」をつなぐ——ネットワーク

ネットワークがないと、スマートフォンは「智障」になってしまう。ネットワークは現代スマートフォンの生命線で、それを理解し、使いこなし、保護することはすべてのスマートフォンユーザーの必修科目だ。

## 〇、接続で迷わない——Wi-Fi とモバイルデータの「道」と「車」を区別

多くの人がスマートフォンの画面上部で Wi-Fi と信号アイコンが同時に点灯しているのに、両者の本質的な違いを全く理解していない。最も典型的な誤解は：**Wi-Fi に接続しているのにページが読み込めない**——ルーターがネットワークから切断されているのに、スマートフォンが自動的にモバイルデータに切り替わらないからだ。この問題を完全に理解するために、「道と車」の認知モデルを構築する必要がある。

**コア比喩：道と車**

インターネットを都市と想像してほしい。「**道**」はスマートフォンからインターネットまでの通道で、「**車**」は道を走るデータだ。Wi-Fi とモバイルデータは 2 つの異なる道だが、データ（車）自体は同じだ。

### 本質的違い：ローカル通道 vs リモート高速道路

Wi-Fi は本質的に**ローカルエリアネットワーク技術**で、自宅の階下に無線で敷かれた道のようなもので、ルーターからスマートフォンまでデータを伝送する。しかしブロードバンドの未払いや光ケーブルの切断があれば、道はあっても外の「高速道路」が切れており、どこにも行けない。一方モバイルデータ（モバイルデータ通信）はキャリアの 4G/5G 基地局を通じて伝送され、あなたが直接キャリアの高速専用線に乗るようなもので、信号さえあれば直接インターネットに接続できる。

### 費用とシーン：定額月額 vs 従量課金

Wi-Fi は通常一回限りの投資（ブロードバンド月額）で、電気代が続く限り家族全員が無制限で使用でき、映画のダウンロードやゲームなど大容量トラフィックシーンに非常に適する。一方モバイルデータは使用量に応じて課金され、パッケージを超えると速度制限や追加課金される可能性があり、地下鉄や出張などモバイルシーンでの軽量タスク処理に適する。

**試してみよう：データパッケージを確認**

スマートフォンの [設定] > [モバイルネットワーク] > [データ管理] を開き、今月の残りデータ容量と、最もデータを消費するアプリを確認。把握してから出かければ慌てない。

### セキュリティ考慮：公共 Wi-Fi は隠された「フィッシング場」

セキュリティの面では、モバイルデータは通常公共 Wi-Fi より安全だ。モバイルデータはキャリアが暗号化するが、公共 Wi-Fi はハッカーが「偽ホットスポット」を構築して暗号化されていないトラフィックを傍受しやすい。ただしここで区別が必要：HTTPS が全面普及した今日、攻撃者が傍受できるのは**あなたがどのウェブサイトにアクセスしたか**（ドメイン）と、**暗号化されていない HTTP コンテンツ**のみ。WeChat のメッセージ、銀行アプリのログインパスワードなど HTTPS で暗号化されたデータは、攻撃者は**直接読み取れない**——より複雑な SSL ストリッピング攻撃を実施し、ユーザーが証明書警告を無視しない限り。とはいえ、外出時の決済操作では、直接スマートフォンのモバイルデータを使用するのが最も確実な選択だ。

**セキュリティリマインダー**

空港、ショッピングモール、カフェなどの公共場所では、名前の不審な無料 Wi-Fi（例：「Free_WiFi_2024」）には接続しないでください。たとえ接続しても、銀行アプリへのログインやパスワード入力は絶対に行わないでください。

### スマートフォン側での判断：ステータスバーの「暗号」を読む

スマートフォンが現在何でネットワーク接続しているかを知るには？Huawei スマートフォンを例にすると、ステータスバーのキャリア信号または WLAN 信号の横に**双方向矢印アイコン**が表示されれば、対応する方式でインターネットに接続中を表す。両方が同時に点灯していれば、ネットワーク加速が有効で、スマートフォンが 2 つの通道を同時に利用していることを示す。

iPhone ユーザーはより直感的：Wi-Fi アイコンが点灯していれば Wi-Fi を使用中、消灯時は右上の信号バーを見ればモバイルデータを使用中。一部の Android スマートフォンはステータスバーに「5G」や「4G」と表示し、現在のモバイルネットワークタイプを直接伝える。

## 一、モバイルネットワーク（4G/5G）とデータ通信

モバイルネットワークはいつでもどこでも接続する基盤だ。2G から 5G まで、速度は千倍以上向上したが、原理は実は似ている。

### ネットワーク方式：

- **4G LTE：**現在カバレッジが最も広く、速度も十分（理論上最高 100Mbps+）、消費電力が比較的低い。
- **5G：**速度が速く（理論上最高 10Gbps）、カバレッジはまだ整備中で、消費電力が高い。大容量ファイルのダウンロードや高画質動画の視聴に適しています。
- **5G SA vs NSA：**SA は独立ネットワーク、NSA は 4G 基地局に依存。SA の方が体験が良いが、キャリアとスマートフォンの両方がサポートする必要がある。

### データ管理：

- **データ通信量とは：**スマートフォンがインターネットに接続して発生するデータ量。動画視聴、画像閲覧が最もデータを消費し、テキストと音声は比較的少ない。
- **節約のヒント：**設定で「データ節約モード」を有効化し、Wi-Fi 環境下でアプリの更新や高画質動画を視聴する。
- **信号バー ≠ 速度：**信号が強いのは接続が良いことを示すだけ。速度は基地局の混雑度、基地局からの距離にも依存する。

**重要警告：海外渡航前に【データローミング】をオフに！**

海外でモバイルデータを使用すると天文学的な請求が発生する可能性がある！海外渡航前に必ず設定でデータローミングをオフにするか、現地の SIM カードや国際ローミングパッケージを事前に購入すること。

## 二、Wi-Fi：接続、共有、セキュリティ

Wi-Fi は家庭やオフィスのインターネット接続の主力で、速度が速くデータ消費がない。しかし Wi-Fi も絶えず進化しており、規格により差が大きい。

### Wi-Fi 接続：

[設定] > [WLAN] でネットワークを選択しパスワードを入力。現在多くのスマートフォンは QR コード接続をサポートし、より便利。

### パスワード共有：

Android と iOS はどちらも接続済みのスマートフォンから友人にパスワードを素早く共有できる。Android は QR コードを生成し、iOS は近接で自動共有できる。

### Wi-Fi 規格の進化：

Wi-Fi 規格は IEEE が制定し、各世代で顕著な向上がある：

| 規格 | プロトコル | 周波数帯 | 最高速度 | 特徴 |
|---|---|---|---|---|
| Wi-Fi 4 | 802.11n | 2.4/5GHz | 600Mbps | 旧規格、カバレッジはそこそこ |
| Wi-Fi 5 | 802.11ac | 5GHz | 3.5Gbps | 現在最も普及、速度も十分 |
| Wi-Fi 6 | 802.11ax | 2.4/5GHz | 9.6Gbps | 多数のデバイスで高性能、遅延が少ない |
| Wi-Fi 6E | 802.11ax | 2.4/5/6GHz | 9.6Gbps | 6GHz 帯追加、混雑なし |
| Wi-Fi 7 | 802.11be | 2.4/5/6GHz | 46Gbps | 超高速転送、マルチリンク集約 |

- **Wi-Fi 6 の主要技術：**OFDMA（直交周波数分割多元接続）が複数デバイスの同時転送を可能にし、MU-MIMO（マルチユーザーマルチプルインプットマルチプルアウトプット）がマルチデバイス同時効率を向上、TWT（ターゲットウェイクタイム）がデバイスの省電力化を実現。

- **Wi-Fi 6E：**Wi-Fi 6 をベースに 6GHz 帯を追加し、より多くのチャネルを提供して干渉を低減します。スマートフォンとルーターの両方が対応している必要があります。

- **Wi-Fi 7：**MLO（マルチリンク操作）を導入し、複数の周波数帯を同時に使用してデータ転送が可能になり、レイテンシが大幅に削減されます。320MHz の超広帯域チャネルにより速度が飛躍的に向上します。Snapdragon 8 Elite Gen5、Dimensity 9400 などのフラッグシップチッププラットフォームが Wi-Fi 7 に対応しており、Xiaomi 17、Huawei Mate 80、Samsung S26 などの機種で Wi-Fi 7 ネットワークを利用できます（Wi-Fi 7 ルーターとの組み合わせが必要）。

- **2.4GHz vs 5GHz：**2.4GHz は壁貫通が良いが速度が遅く干渉が多い。5GHz は速度が速いが壁貫通が弱い。Wi-Fi 6E/7 の 6GHz 周波数帯は速度が最も速いが壁貫通が最も弱く、同一部屋内での使用に適する。

### 公共 Wi-Fi のリスク：

公共 Wi-Fi ではハッカーがデータを傍受したりフィッシングホットスポットを設置する可能性がある。ルール：

- 銀行アカウントへのログイン、パスワード入力を避ける。

- 可能なら VPN で暗号化接続を使用。

- 知らない Wi-Fi に自動接続しない。
- HTTPS ウェブサイト（アドレスバーに鍵アイコン）を使用。

## 三、Bluetooth：短距離無線通信の万能選手

Bluetooth はスマートフォンで最も使用頻度の高い無線技術の一つかもしれない——ワイヤレスイヤホン、スマートウォッチ、カーシステム、キーボードマウス、すべてそれで接続する。

### 技術原理：

Bluetooth は 2.4GHz ISM 帯で動作し、周波数ホッピング拡散（FHSS）技術でデータを 79 チャネルに分散して高速に切り替えながら伝送するため、干渉を効果的に回避できます。Bluetooth の通信距離は 3 つのクラスに分かれます：Class 1 は約 100 メートル、Class 2 は約 10 メートル（スマートフォンでよく使われる）、Class 3 は約 1 メートルです。

### バージョン進化：

- **Bluetooth 4.0**：低消費電力 Bluetooth（BLE）を導入し、バンド、心拍センサーなどの小電池デバイスも無線接続可能になり、バッテリー持続が日単位から月単位に。
- **Bluetooth 4.2**：転送速度とプライバシー保護を向上、IPv6 直接接続をサポート。
- **Bluetooth 5.0**：転送速度 2 倍（2Mbps）、転送距離 4 倍（理論 300 メートル）、ブロードキャスト能力 8 倍。デュアルオーディオ（同時に 2 つのイヤホン接続）をサポート。
- **Bluetooth 5.1**：方向測位機能（AoA/AoD）を導入し、測位精度はセンチメートル級に到達。
- **Bluetooth 5.2**：LE Audio 新規格、LC3 コーデックサポート（音質向上、消費電力低下）、マルチストリームオーディオ、ブロードキャストオーディオ。
- **Bluetooth 5.3/5.4**：消費電力と接続安定性をさらに最適化、周期的ブロードキャストを強化。

**LE Audio 普及状況**：2025-2026 年、LE Audio は急速な普及段階に入っています。主要ブランドの新型完全ワイヤレスイヤホン（Huawei FreeBuds Pro 5、Xiaomi Buds 6 Pro、Samsung Galaxy Buds3 Pro など）は LE Audio と LC3 コーデックに対応しており、より低いレイテンシとより良い音質を提供します。新しいイヤホンを購入する際は、LE Audio 対応製品を優先することをお勧めします。

### 活用シーン：

- **オーディオ伝送**：ワイヤレスイヤホン、カーオーディオ。Bluetooth 5.2 の LE Audio で音質と遅延が質的に飛躍。

- **データ転送：**近距離でのファイル、写真転送。速度は Wi-Fi Direct に劣るが、シンプルさで勝る。
- **デバイスペアリング：**スマートウォッチ、バンド、キーボードとマウス、ゲームコントローラ。
- **位置サービス：**Bluetooth ビーコン（Beacon）は屋内ナビゲーション、物品追跡（Apple AirTag など）に使用。
- **スマートホーム：**Bluetooth Mesh ネットワークで電球やスマートロックなどを制御します。

## 四、赤外線リモコン：スマートフォンが万能リモコンに変身

赤外線リモコンは最も古い無線制御技術の一つだが、スマートフォンでも依然として実用的だ。一部の Android スマートフォン（Xiaomi、Huawei の一部機種など）には赤外線送信機が内蔵され、テレビ、エアコン、セットトップボックスなどの家電をリモコン操作できる。

### 動作原理：

赤外線リモコンは、赤外線発光ダイオードから特定の周波数（通常 38kHz）の赤外光パルス信号を放ち、家電の赤外線受信器がこのパルスを復号して対応する操作を実行します。ブランドごとに異なるエンコードプロトコルが使われており、一般的なのは NEC、RC5、RC6、SONY などです。スマートフォンの赤外線アプリには各ブランドのコードライブラリが内蔵されており、ブランドと型番を選ぶだけでマッチします。

### 制限事項：

- **方向性：**赤外線は直線伝播で、デバイスに向けて発信する必要があり、壁越しは不可。
- **距離が短い：**有効距離は一般に 5-8 メートル。
- **Apple 非サポート：**iPhone には赤外線送信機がなく、この機能は使用不可。

## 五、NFC：タッチする魔法

NFC（Near Field Communication、近距離無線通信）は極短距離（10 センチ以内）の高周波無線通信技術で、動作周波数 13.56MHz、転送速度最高 424Kbps。

### 通信原理とモード：

NFC は RFID 技術から派生し、3 つの動作モードをサポート：

- **読み取り/書き込みモード：**スマートフォンが NFC タグの情報を読み取る。NFC ステッカーの読み取りでアクションをトリガーするなど。
- **ピアツーピアモード：**2 台の NFC デバイスが相互にデータ転送。Android Beam（廃止済み）など。
- **カードエミュレーションモード：**スマートフォンがスマートカードをエミュレート。これが最もよく使われるモード——交通カード、ドアカード、銀行カードはすべてこのモード。

**セキュリティメカニズム：**

- **超短距離：**10 センチ以内の通信距離自体が物理的セキュリティ保障で、遠隔からの盗聴はほぼ不可能。
- **セキュリティチップ：**スマートフォンに独立したセキュリティチップ（SE）を内蔵するか HCE（ホストカードエミュレーション）を使用し、決済鍵はセキュアエリアに保存され、スマートフォンが root 化されても抽出不可。
- **指紋/顔認証：**決済時に生体認証確認が必要で、スマートフォン紛失時も不正使用不可。
- **トークン化：**Apple Pay、Huawei Pay などは実際のカード番号を転送せず、一次性トークンを使用し、傍受されても再利用不可。

**典型的な応用：**

- **交通カード/ドアカード：**実体カードの情報をスマートフォンに書き込み、スマートフォンをかざすだけ。
- **モバイル決済：**Apple Pay、Huawei Pay などで、銀行カードをスマートフォンにバインドし、タッチで決済。
- **デバイスペアリング：**NFC タッチで Bluetooth イヤホン、スピーカーと迅速ペアリング。
- **本人認証：**NFC 身分証読み取り、電子運転免許認証。

## 六、テザリング：スマートフォンを Wi-Fi ルーターに

自分のデータ通信量が余っている時、テザリングでタブレットやパソコンに提供できる。

- **有効化方法：**設定 > テザリング/ネットワーク共有で有効化。
- **注意事項：**電池とデータ通信量を大きく消費するので、複雑なパスワードを設定し、接続デバイス数を制限することを忘れないでください。
- **USB 共有：**データケーブルでパソコンと接続。ワイヤレステザリングより安定で速い。

### やってみよう：テザリングの設定と管理

設定 を開き、「テザリング」または「ネットワーク共有」を見つけて有効化し、名称とパスワードを設定。

1. 別のデバイス（パソコンなど）でこの Wi-Fi を検索して接続。
2. スマートフォン側で、接続済みデバイスを管理したり、知らないデバイスをキックできるか確認。
3. 第四章：アプリのインストールと管理——スマートフォンの「スキルパック」

# 第四章：アプリのインストールと管理——スマートフォンの「スキルパック」

アプリがスマートフォンを万能にするが、管理も必要だ。新入社員に席を用意し権限を割り当てるように、アプリのインストールにも規範フローが必要だ。

## 一、公式ストア：最も安全な「アプリスーパー」

- **Android**iOS
- **iOS**：App Store のみ。
- **HarmonyOS** コアのセキュリティ意義：審査済み、更新保証あり、決済安全。行動ガイド：

公式ストアからのダウンロードを優先! 二、ピュア/プロテクトモード：スマートフォンの「アンチウイルス壁」を有効化

## 二、ピュア/プロテクトモード：スマートフォンの「アンチウイルス壁」を有効化二、ピュア/プロテクトモード：スマートフォンの「アンチウイルス壁」を有効化

ピュアモードまたはプロテクトモードを内蔵し、これはシステムレベルのセキュリティスイッチだ。すべてのユーザーに有効化を強く推奨。。

### それは一体何?

コア機能はサードパーティアプリのインストールをブロックすること：有効時、システムは公式アプリストアからのアプリインストールのみ許可し、ブラウザ、**QR** コード、チャットソフトなどのチャネルからダウンロードしたインストーラは直接ブロックされ、悪意あるソフトを根源から排除する。2. 適用対象

### 適用対象

- **：「公式」と「偽物」の広告を区別できず、ピュアモードが最も確実なお守り。**アプリ権限に敏感でないユーザー
- **：ニッチアプリ、ベータ版、海外版ゲームをインストールする必要がない。**デバイスのセキュリティを向上させたいすべてのユーザー
- **：マニアであっても、有効にしておく方が無効より良い——一時的な必要があれば、その時だけ無効にすればよい。**3. 有効化パス（Huawei/Xiaomi の場合）

### 有効化パス（Huawei/Xiaomi の場合）

- **：**[設定] > [システムと更新] >[ピュアモード]、**「有効化」を選択。**、「オンにする」を選択。
- **Xiaomi**[プライバシー保護] **または** [設定] > [アプリ設定] > [アプリインストール]、**「提供元不明アプリのインストール」をオフ。**OPPO/vivo
- **：一部機種はアンロック可能、申請が必要。**[外部提供元アプリのインストール]、**個別に認証をオフ。**4. アプリストアのセキュリティ検査メカニズム

### アプリストアのセキュリティ検査メカニズム

ウイルススキャン

- **：各アプリは上架前に複数のアンチウイルスエンジンによる合同スキャンを受け、既知のトロイの木馬やウイルスの特徴を識別。**権限審査
- **：ストアはアプリが申請する権限が「合理的」かを審査——懐中電灯アプリが連絡先権限を要求する？即座に却下または下架。**開発者認証
- **：上架には企業資格、個人身分証明またはクレジットカード認証が必要。問題発生時に人物を特定でき、「ヤミアプリ」ではない。**継続監視
- **：上架済みアプリも継続的に再スキャンされ、新たに発見された悪意ある挙動は強制下架される。**三、サードパーティ提供元と「電子寄生虫」：リスク極めて高い！

## 三、サードパーティ提供元と「電子寄生虫」：リスク極めて高い！三、サードパーティ提供元と「電子寄生虫」：リスク極めて高い！

「電子寄生虫」。。

**トロイの木馬が植入された「クラッキング版」ソフトウェア**

を指す——あなたは「節約した」と思うが、実際は「罠にはまった」。一般的な形式：**「クラッキング版」「無料版」「広告除去版」の有料ソフトウェア；**「いいね増やし」「紅包搶奪」「ゲームチート」などのグレーツール；

- グループチャットの「広告」が推奨する「内部版」アプリ。
- これらのプログラムは表面上は使えるが、裏で
- 連絡先を盗み、SMS を読み、マイクを盗聴、バックグラウンドでマイニング

している可能性がある。ピュアモードはそれらを防ぐ**第一の防線**——そもそもインストールさせない。**第二の防線の推奨：**定期全スキャン

### 第二の防線の推奨：

- **センシティブ権限の慎重な付与：**連絡先、SMS、位置、カメラ、マイク——これら 5 項目の「高危権限」は厳格に審査必要。
- **消費電力とデータ通信量の異常への注目：**スマートフォンが突然発熱、バッテリー急消費、深夜のデータ通信——八割バックグラウンドに「寄生虫」が隠れている。
- **万が一サードパーティ製アプリをインストールする必要がある場合：**提供元が絶対に信頼できることを確認する。

インストール前に権限要求を注意深く確認する。

1. インストール後にセキュリティアプリでスキャンする。
2. 最終的なアドバイス：システム設定で「提供元不明のアプリのインストール」権限をオフにする。
3. 四、インストール、更新、アンインストール

インストール

## 四、インストール、更新、アンインストール四、インストール、更新、アンインストール

- **更新：**
- **必ず更新すること!：アンインストール：**アイコンを長押ししてアンインストール領域にドラッグ。より完全に削除したい場合は、[設定] > [アプリ管理] で該当アプリを見つけてアンインストール。
- **五、権限管理：あなたはアプリの「ボス」**アプリが権限（連絡先や位置など）を要求するのは、合理的な場合（地図アプリが位置を要求）もあれば、不合理な場合（懐中電灯が連絡先を要求）もある。

# 五、権限管理：あなたはアプリの「ボス」五、権限管理：あなたはアプリの「ボス」

：「常に許可」「使用中のみ許可」「禁止」「毎回確認」。

- **管理原則：**
- **最小化：不要なものは与えない。：**

1. 状況に応じた判断：タクシー配車アプリのように、使用時のみ位置権限を与える。
2. 重点監視すべき権限
3. 連絡先/SMS：迷惑 SMS 送信に悪用される可能性。

**リアルタイム位置：行動追跡される可能性。**

- カメラ/マイク：盗撮・盗聴される可能性。
- ファイルアクセス：プライベートファイルが読み取られる可能性。
- 六、ソフトウェアインストールパッケージの構成要素
- アプリストアからダウンロードするインストールパッケージの中に何が入っているか考えたことがあるだろうか？インストールパッケージの構成を理解することで、アプリの仕組みをより良く理解でき、問題発生時にどこから調査すべきかも分かるようになる。

## Android APK ファイル構造

APK（Android Package Kit）は本質的に ZIP 圧縮ファイルで、解凍すると以下のコアディレクトリとファイルが見える：

### Android APK ファイル構造

：アプリの「身分証明書」であり「取扱説明書」。アプリのパッケージ名、バージョン番号、必要な権限、サポートするシステムバージョン、入口 Activity などのコア情報を記録。システムがアプリをインストールする際、最初にこのファイルを読み取る。

- **classes.dex：** アプリのコアコード。Java/Kotlin ソースコードがコンパイルされて Dalvik バイトコードになり、このファイルに格納される。アプリによっては複数の dex ファイル（classes2.dex、classes3.dex など）を持つ場合がある。
- **resources.arsc：** コンパイル済みのリソースインデックステーブル。すべてのリソースの ID と対応関係を記録。

- **res/ディレクトリ：**リソースフォルダ。画像、レイアウトファイル、文字列、色定義などを含む。異なる解像度の画像は異なる drawable フォルダ（drawable-hdpi、drawable-xhdpi など）に配置される。
- **assets/ディレクトリ：**生リソースフォルダ。コンパイル不要のリソース（ウェブファイル、フォント、オーディオなど）を格納。アプリ実行時に AssetManager で読み取り可能。
- **lib/ディレクトリ：**ネイティブライブラリフォルダ。コンパイル済みの.so ファイル（C/C++ コードのコンパイルの産物）を CPU アーキテクチャ別（arm64-v8a、armeabi-v7a など）に格納。ゲームや高性能計算を必要とするアプリには通常このディレクトリがある。
- **META-INF/ディレクトリ：**署名情報フォルダ。MANIFEST.MF、CERT.SF、CERT.RSA の 3 ファイルを含み、APK の完全性と署名の検証に使用。
- **2. iOS IPA ファイル構造** IPA（iOS App Store Package）も同様に圧縮ファイルだが、構造は若干異なる：

### iOS IPA ファイル構造

：コアコンテンツの格納場所。内部に.app フォルダがあり、アプリのすべてのコンテンツを含む。

- **Info.plist：**アプリの設定ファイルで、Android の AndroidManifest.xml に類似。アプリ ID、バージョン、権限要件、サポートデバイスなどを記録。
- **実行可能ファイル：**.app フォルダと同名のメインプログラムで、コンパイル済みの Mach-O 形式バイナリファイル。
- **Frameworks/ディレクトリ：**アプリが依存する動的ライブラリとフレームワーク。
- **リソースファイル：**画像、オーディオ、動画などのリソース。通常.app フォルダ内に格納。
- **_CodeSignature/ディレクトリ：**コード署名情報。アプリの完全性と出所の検証に使用。
- **3. 署名検証メカニズム**なぜ公式ストアからダウンロードするアプリの方が安全なのか？ 署名検証が鍵だ。

### 署名検証メカニズム

：開発者が秘密鍵で APK に署名し、システムインストール時に公開鍵で検証する。APK が改ざんされた場合（ウイルスの植入など）、署名は無効になり、システムはインストールを拒否する。Google Play はさらに App Signing を要求し、Google が署名鍵を保管することでアプリのセキュリティをさらに保護する。

- **iOS 署名：**Apple の審査はより厳格。各 IPA には Apple が発行した証明書の署名が必須で、なければインストール不可。これが iOS が App Store または企業証明書からしかアプリをインストールできない理由だ。企業証明書で署名されたアプリは有効期限があり、Apple はいつでも取り消し可能。
- **署名の役割：**
- **出所の検証：アプリが自称する開発者から実際に来ていることを確認。：**

1. なりすまし防止：悪意あるアプリが他のアプリに偽装することを防ぐ。
2. 4. インストールパッケージのサイズと最適化
3. アプリストアに表示されるアプリサイズと、インストール後に占有する容量が異なることに気づいたことがあるかもしれない。理由は以下の通り：

### インストールパッケージのサイズと最適化

：圧縮後の APK/IPA のサイズ。

- **インストールサイズ：**解凍後のサイズで、通常ダウンロードサイズよりはるかに大きい。
- **実行時占有容量：**アプリ実行時にキャッシュやデータファイルが生成され、さらに容量を占有する。
- **現代アプリの最適化手法：**App Bundle（Android）

：Google Play の動的配信技術。ユーザーはダウンロード時に自分のデバイスに適したリソースのみを取得（画面解像度に対応する画像のみなど）し、ダウンロードサイズを大幅に削減。

- **App Thinning（iOS）：**類似の技術で、Apple がデバイスに応じて不要なリソースを自動的に切り詰める。
- **オンデマンドダウンロード：**ゲームなどの大型アプリは、まずコアパッケージをダウンロードし、後からリソースパッケージをダウンロード可能。
- **豆知識：なぜあるアプリのインストールパッケージは特に大きいのか？**ゲーム系アプリは通常最も大きく、大量の HD テクスチャ、3D モデル、オーディオファイルを含むため。ソーシャル系アプリも比較的大きく、各種機能モジュールをサポートする必要があるため。ツール系アプリは通常最も小さい。あるツール系アプリが特に大きい場合、不要な機能がバンドルされていないか警戒すべき。

**実践セクション：3 つのアプリの権限を審査**

よく使うアプリを 3 つ（ソーシャル、ショッピング、ツール系など）選び、[設定] のアプリ権限管理ページで項目ごとに審査し、不要なセンシティブ権限をオフにするか「使用

中のみ許可」に変更する。

### 第五章：スマートフォンをもっと理解させる——基本設定

数分かけて設定するだけで、体験は大幅に向上する。これらの設定はスマートフォンに「チューニング」を行うようなもので、あなたの使用習慣に合わせることができる。

# 第五章：スマートフォンをもっと理解させる——基本設定

数分かけて設定するだけで、体験は大幅に向上する。これらの設定はスマートフォンに「チューニング」を行うようなもので、あなたの使用習慣に合わせることができる。

## 一、サウンドとバイブレーション一、サウンドとバイブレーション

：聞こえて、かつ恥ずかしくないものを選ぶ。

- **バイブレーションモード：**マナー時にバイブで通知。
- **おやすみモード：**例外を設定可能。連絡先からの着信を許可するなど。
- **音量分割：**メディア音量、着信音量、アラーム音量を個別に調整可能。
- **二、表示と明るさ**自動明るさ

## 二、表示と明るさ二、表示と明るさ

- **アイケア/ナイトシフトモード：**ブルーライトをカットし、夜間の閲覧を快適に。
- **ダークモード：**省電力（OLED 画面で特に有効）、かつ夜間に眩しくない。多くのアプリもダークモードをサポート。
- **画面リフレッシュレート：**一部の Android スマートフォンは調整可能。高いほど（120Hz など）スクロールが滑らか。
- **自動ロック：**15 秒または 30 秒に設定。安全で省電力。
- **フォントサイズ：**字が小さいと感じる？少し大きくしよう。
- **三、壁紙、テーマとホーム画面**素敵な壁紙（静的または動的）に変更するだけで、気分も変わる。Android ブランドのスマートフォンにはテーマストアがあり、アイコンやスタイルを丸ごと変更可能。

## 三、壁紙、テーマとホーム画面三、壁紙、テーマとホーム画面

：天気、スケジュール、音楽コントロールをホーム画面に配置し、ワンタップでアクセス。iOS と Android の両方がサポート。ホーム画面の空白部分を長押しして追加。

**四、効率設定：スマートフォンを「外付け脳」にする**このセクションは「一度設定したら戻れない」機能で、どれも日常効率を大幅に向上させる。

## 四、効率設定：スマートフォンを「外付け脳」にする四、効率設定：スマートフォンを「外付け脳」にする

核心となる考え

### 集中 / おやすみモード：通知に奪われた注意力を取り戻す

- **使用のアドバイス：**
- **・仕事モード：「よく連絡する」連絡先と必要なアプリのみ許可；：**
  • 運動 / 読書 / 会議：現在の活動に応じてカスタマイズ。
  応用テクニック
  ：自動化 + 位置 / 時間トリガーでモードを自動切替。例：iOS ショートカットで「会社に到着したら自動的に仕事モードに切り替え」を設定可能。
- **2. 通知管理：赤いバッジの「しつこい連呼」を止める**マーケティング通知をオフ

### 通知管理：赤いバッジの「しつこい連呼」を止める

- **重要度で並べ替え：**人間系の通信（電話、SMS、インスタントメッセージ）をトップに固定；ニュース、ショッピング、ソーシャルプラットフォーム系は「サイレントプッシュ」に設定可能。
- **グループ化通知：**同種の通知をまとめて折りたたみ、一度のロック解除で数十件のメッセージに埋もれるのを防ぐ。
- **3. ショートカットジェスチャー：操作を一歩リード**スクリーンショット

### ショートカットジェスチャー：操作を一歩リード

- **長尺スクリーンショット / スクロールキャプチャ：**通常のスクリーンショット後に「長尺スクリーンショット」オプションをタップ。
- **ワンタップ QR スキャン：**コントロールセンター / クイック設定に「スキャン」アイコンを追加。
- **背面タップ（iPhone）：**[設定] > [アクセシビリティ] > [タッチ] > [背面タップ] で「スクリーンショット」や「ロック」などのアクションを割り当て。

- **音声起動：**Siri / 小愛同学 / 小芸 / 小布、一声かければ天気確認やアラーム設定が可能。

- **4. 入力メソッド：最も過小評価されている効率化ツール**適切な入力メソッドの選択

### 入力メソッド：最も過小評価されている効率化ツール

- **・Sogou / Baidu：辞書が大きく、流行語の更新が速い；：**
  • Gboard（Google）：シンプル、クロスプラットフォーム同期が強力、フリック入力をサポート；
  • iOS 標準：プライバシー保護が最も良く、システムと深く統合。
  必須のスイッチ
  ：スマート補正、予測変換、クラウド辞書同期、上スワイプで数字 • 記号入力。

- **プライバシーの注意：**「文章予測」や「個別化推奨」機能は慎重に使用。入力内容がアップロードされる可能性がある。

- **5. ホーム画面とマルチタスク：ワークフローを滑らかに**アプリ分割表示 / ピクチャーインピクチャー

### ホーム画面とマルチタスク：ワークフローを滑らかに

- **小ウィンドウ / フローティングウィンドウ：**WeChat / TikTok は「小ウィンドウモード」をサポートし、返信後にすぐ閉じられる。

- **常用アプリを固定：**Dock バー（画面下部）には最も高頻度の 4 つのみ配置し、検索時間を削減。

- **「初日勤務」チェックリスト**新規スマートフォンの取得や大型アップデート後、10 分かけて以下の設定を確認することを推奨：

**(1) ロック画面パスワード + 生体認証 (2) 通知グループ化 + マーケティングプッシュをオフ (3) フォントサイズ / リフレッシュレート / ダークモード**

(4) デフォルトの入力メソッド / ブラウザ / メール (5) 緊急連絡先 / メディカル ID (6) バックアップ設定

五、アクセシビリティ機能：テクノロジーが善である力

「アクセシビリティ機能」という名前に恐れる必要はない。これらの機能は誰でも使える。元々は障害者のために設計されたが、一般人にも役立つ——一時的に手を怪我した、赤ちゃんを抱えている、地下鉄でイヤホンが壊れて聞こえない、老眼鏡で小さな文字を見る……。どのような状況でも助けになる。結局のところ、アクセシビリティデザインは

## 五、アクセシビリティ機能：テクノロジーが善である力五、アクセシビリティ機能：テクノロジーが善である力

理念を体現している：すべての人が平等にデジタルライフを享受できるようにする。**1. 視覚障害者の「目」——スクリーンリーダー完全ガイド**スクリーンリーダーは視覚障害者がスマートフォンを使用するためのコアツール。画面上の

### 視覚障害者の「目」——スクリーンリーダー完全ガイド

、ユーザーはジェスチャーと音声でスマートフォンを操作する。以下では iOS「VoiceOver」と Android「TalkBack」の使用方法をそれぞれ紹介する。**iOS VoiceOver** 有効化の手順

**：[設定] > [アクセシビリティ] > [VoiceOver]、スイッチをオン。サイドボタン（またはホームボタン）を 3 回クリックして素早く切り替えも可能。**

**基本ジェスチャー：**
**指 1 本で左右スワイプ：フォーカス要素を切り替え；：**

1. 指 1 本でダブルタップ：現在のフォーカス要素をアクティブ化（「タップ」に相当）；
2. 指 3 本でスワイプ：ページを切り替え；
3. 指 1 本でトリプルタップ：拡大または選択（通常のトリプルタップとは異なる）。
4. 個別設定
5. ：[VoiceOver] > [音声] で、

**読み上げ速度**（デフォルト約 50%、70-80% に調整すると聴感がよりスムーズ）、**ピッチ**、異なる音声パッケージの切り替えが可能。iOS には「拡張音声」と「音声コントローラ」が内蔵され、前者は呼吸音や間を減らし、より自然な聴感を実現。**音声エンジン**有効化の手順

**：[設定] > [アクセシビリティ] > [TalkBack]、スイッチをオン。または音量上下キーを 3 秒間長押しで素早く起動（一部ブランドがサポート）。**

**基本ジェスチャー：**
**指 1 本で左右スワイプ：フォーカス要素を切り替え；：**

1. 指 2 本で上下/左右スワイプ：スクロール；
2. 指 3 本でシングルタップ：読み上げモード（連続読み上げ）に移行；
3. 指 4 本でシングルタップ：読み上げを一時停止/再開。
4. 個別設定

5. **：**[TalkBack] > [テキスト読み上げ設定] で、

**読み上げ速度、ピッチ、音声エンジン**の切り替えが可能（Google テキスト読み上げエンジンのインストールを推奨、より自然な読み上げ）。**2. 小さな字を大きく、ぼやけを鮮明に——視覚調整**拡大ジェスチャー

## 小さな字を大きく、ぼやけを鮮明に——視覚調整

### 拡大ジェスチャー

- **iOS** で拡大。デフォルトは全画面ズーム、ウィンドウズーム（部分拡大ウィンドウ）も選択可能。**2 本指のピンチで倍率調整（1.5x□15x）**、3 本指でダブルタップすると現在のズーム倍率を固定。**ピンチイン・ピンチアウトで倍率を調整（1.5 倍□15 倍）：**[設定] > [アクセシビリティ] > [拡大]、3 本指でタップして拡大、もう一度 3 本指でタップして終了；または

- **Android** して一時拡大を有効化（最大 8 倍）。**ハイコントラストテキスト / 太字テキスト** iOS

### ハイコントラストテキスト / 太字テキスト

- **iOS：**[設定] > [表示] > [フォントサイズ]、「小、標準、大、特大」の 4 段階を調整可能。Huawei/Xiaomi は「太字」と「ハイコントラストテキスト」スイッチを追加提供。

- **Android**iOS

### カラーフィルター（色覚バリアフリー）

- **iOS**、および「グレースケール」と「色を反転」オプションを提供。まず色覚テスト（システム内蔵）を行ってから対応するフィルターを選択することを推奨。**赤 / 緑 / 青 / 黄の 4 つの色覚サポートモード：**[設定] > [アクセシビリティ] > [色補正] で、

- **Android**+「色を反転」スイッチを含む。**3. AI 画像記述：AI をあなたの「目」にする**スクリーンリーダーが画像に遭遇した場合、従来はスキップするか「画像」という 2 文字だけを読み上げていた。しかし今、AI は

### AI 画像記述：AI をあなたの「目」にする

でき、認証コードの識別、画像の読み取り、メニューの識別に非常に役立つ。**各システムの AI 画像記述サポート状況** iOS 15+

：[設定] > [アクセシビリティ] > [VoiceOver] > [音声記述] で「画像記述」をオン。システムは

- **ローカル AI モデル**を呼び出して画像のテキスト記述を自動生成（iPhone 12 以降が必要）。**新版 TalkBack アプリ：** Google TalkBack は比較的新しいバージョンで
- **画像 Q& A** 機能を統合。「この画像に何がある?」と質問すると、AI がリアルタイムに回答（Android システムバージョンに厳密に依存せず、TalkBack アプリを更新するだけで体験可能）。**サードパーティアプリ：**「Seeing AI」（iOS）、「Lookout」（Android）、「小芸幫幫」（Huawei）はいずれも優れた AI 視覚アシスタントで、物品認識、書類読み取り、顔の表情認識が可能。
- **使用シーン**街撮り写真に向かって「この画像には何人がいますか? 彼らは何をしていますか?」と聞いてみよう。またはデリバリーアプリを開き、AI にメニュー画像の料理名を読み上げさせよう——

これこそが真の「テクノロジーが生活をより平等にする」

。**4. 聴覚障害者の「耳」——リアルタイム字幕完全ガイド。**

## 聴覚障害者の「耳」——リアルタイム字幕完全ガイド

機能で、聴覚障害者が騒がしい環境、図書館、外国語シーンで特に役立つ。**リアルタイム字幕の有効化と最適化** iOS

：[設定] > [アクセシビリティ] > [リアルタイム字幕]、スイッチをオン。システムはデバイスのすべての音声（通話、動画、ポッドキャスト、FaceTime、ボイスメモ）を検出し、自動的に画面上部に字幕を表示。

**iOS：**
**音量キー**を押すポップアップボックスの「リアルタイム字幕」アイコンをタップ；または [設定] > [アクセシビリティ] > [リアルタイム字幕] でオン。**精度向上のヒント：**
**できるだけ：**

1. で使用し、エアコンや機械の轟音から離れる；**話者が**はっきり発音し、適切な速度
2. の時に認識率が最も高い；**複数人の対話は精度を下げる、「1 人だけ表示」モードを使用可能；**中国語普通話、広東語、英語などの主要言語をサポート、
3. 少数言語の認識率は低い
4. ；**話の途中に BGM や人の声のノイズがあると、認識が途切れ途切れになる。**5. 通話シーンのデュアルチャネル：音声テキスト変換 + テキスト音声変換
5. ユーザーが

### 通話シーンのデュアルチャネル：音声テキスト変換 + テキスト音声変換

場合（突発性失声、喉の手術回復期、重度の吃音など）、スマートフォンは「伝言役」になる。**通話リアルタイム翻訳（TTY / RTT）**iOS

：[設定] > [アクセシビリティ] > [RTT/TTY]、有効化すると通話中に画面にキーボードが表示され、相手の音声がリアルタイムにテキスト化され、

**iOS。入力した文字も相手に読み上げられます：**[設定] > [アクセシビリティ] > [リアルタイム文字起こし] + [音声通知]。「リアルタイム文字起こし」は相手の発話をリアルタイムにテキスト表示;「音声通知」は音声が不便な時にあなたの入力を音声で再生。
**Android：**
**USB PD 3.1：**

- **TTY（テレタイプライター）：**専用 TTY デバイスが必要、RTT に代替されつつある;
- **WeChat/QQ** の音声テキスト変換機能
- も代替案：音声メッセージを長押し「テキスト変換」。**6. すべての人向けの「小さくても実用的な」機能**背面タップ

### すべての人向けの「小さくても実用的な」機能

- **ダブルタップ/トリプルタップ**でスクリーンショット、ロック、コントロールセンターなどのアクションを割り当て可能。両手で荷物を持っている時に特に便利。**アシストタッチ / フローティングボール：**iOS [アクセシビリティ] > [タッチ] > [アシストタッチ]、Android [アクセシビリティ] > [フローティングナビゲーション]。仮想ボタンでジェスチャーや物理ボタンを代替、ボタン不調や片手操作時に助けに。
- **スイッチコントロール：**外部スイッチ、シングルタップ、頭部の動作でスマートフォンを操作、
- **肢体障害者向けに設計**。iOS [アクセシビリティ] > [スイッチコントロール]、Android [アクセシビリティ] > [スイッチコントロール]。**サウンド認識**（iOS）：[アクセシビリティ] > [サウンド認識]、有効化するとスマートフォンがドアベル、赤ちゃんの泣き声、火災警報、サイレンなどの音を「聞き」、
- **認識するとバイブレーション + 画面通知**。聴覚障害者や睡眠中に非常に有用。**LED フラッシュ通知：**iOS [アクセシビリティ] > [オーディオ/視覚] > [LED フラッシュで通知]、Android [設定] > [通知] > [フラッシュライト通知]。着信や通知時にフラッシュライトが点滅、
- **騒がしい環境でも音なしで感知可能。モノラルオーディオ。**
- **Apple のアクセシビリティデザイン哲学** Apple はアクセシビリティ分野に多大な投資を行い、毎年アクセシビリティ機能の更新と追加を行っている。Apple 公式サイトには

専用のアクセシビリティページがあり、各機能の使用方法を詳しく紹介。iOS のアクセシビリティ機能は通常 Android より完璧で、体験もよりスムーズ。これが多くの障害者が Apple を選ぶ重要な理由だ。

**30 秒で体験してみよう**

空き時間を見つけて、以下の体験をしよう：

**VoiceOver / TalkBack**

をオンにし、目に頼らず 3 本指でスワイプして WeChat を閲覧してみる；

1. リアルタイム字幕**をオンにし、スマートフォンに向かって話して認識精度を確認**；3 本指タップ
2. でテキストを 200% に拡大し、視覚障害者の日常を体験；**すべての機能をオフにしてから、「もし私が永遠にこれらの機能を必要とするなら、どう設計してほしいか」を考える。**第六章：デジタルライフを守る——セキュリティとプライバシー
3. スマートフォンのセキュリティ＝プライバシーセキュリティ＋資産セキュリティ。データが資産となる時代において、スマートフォンを守ることは自分自身を守ること。**一、ロック画面：第一の物理的防線**ロック画面はスマートフォンの第一の防線であり、最も見落とされやすい。
4. セキュリティレベルの順位：

# 第六章：デジタルライフを守る——セキュリティとプライバシー

スマートフォンのセキュリティ＝プライバシーセキュリティ＋資産セキュリティ。データが資産となる時代において、スマートフォンを守ることは自分自身を守ること。

## 一、ロック画面：第一の物理的防線一、ロック画面：第一の物理的防線

：セキュリティと利便性の完璧なバランス。

### セキュリティレベルの順位：

1. ：最も便利だが、セキュリティは技術による（3D 構造光は安全、2D 写真は騙されやすい）。パターンロック解除
2. ：便利だが痕跡が残り推測されやすい。強力なロック画面の設定：
3. 必ず複雑なパスワードを設定（誕生日、123456 は不可）。生体認証を有効化。
4. ロック時間を「即時」または最短に設定。二、「スマートフォンを探す」：究極の盗難防止ツール

### 強力なロック画面の設定：

- 以下のことが可能：
- 位置特定
- ：スマートフォンをどこに置いたか、マップで検索。

## 二、「スマートフォンを探す」：究極の盗難防止ツール二、「スマートフォンを探す」：究極の盗難防止ツール

：スマートフォンを鳴らす。家で探す時に適している。紛失モード

- **：リモートでスマートフォンをロックし連絡先番号を表示。**リモートワイプ
- **：最後の手段、すべてのデータを消去（ただしワイプ後は位置特定不可）。**[設定] で、Apple は「iPhone を探す」、Android は「デバイスを探す」またはブランドの類似サービスを探し、緑色の有効状態であることを確認。
- **三、システム更新：パッチを当て、脆弱性を塞ぐ**システム更新の主な目的はセキュリティ脆弱性の修正で、家の壁の穴を補修するようなもの。したがって、システム更新の通知を見たら、できるだけ早くアップグレードすること。
- **もちろん、大型バージョンアップ（iOS 17 から iOS 18 など）には互換性の問題がある可能性があるため、数日様子を見てからアップグレードしてもよい。**四、電信詐欺の識別と防止

AI 技術が急速に発展する今日、従来の「見るものは真」の認識モードは完全に失効した。詐欺師はディープフェイク（Deepfake）などの AI 技術を利用し、「網を広く張る」式の詐欺から「精密包囲狩り」式の犯罪へと全面的にアップグレードしている。

## 三、システム更新：パッチを当て、脆弱性を塞ぐ三、システム更新：パッチを当て、脆弱性を塞ぐ

(1) AI 顔交換・音声模倣詐欺

不法分子がソーシャルメディアに公開された高解像度写真や、わずか数秒の音声サンプルを違法に収集し、AI ディープフェイク技術で本物と見分けがつかない「親友の動画」や「音声」をクローン。よくあるシナリオ：

## 四、電信詐欺の識別と防止四、電信詐欺の識別と防止

など。この種の詐欺は家族愛や友情などの感情的紐帯を利用し、

### AI 時代の最も一般的な詐欺の手口

。

(2) 権威 / カスタマーサービス偽装詐欺 **AI 技術は公式カスタマーサービスや公安・検察・司法機関職員の声や語調を精密に模倣でき、さらに高精度に逼真な偽造送金スクリーンショット、電子公文、指名手配書などの文書を合成し、**パニック心理や権威の圧力を作り出し、**被害者に送金や画面共有機能を開かせてアカウント情報を窃取。**。

**AI アルゴリズムで「絶対に損をしない」偽造収益スクリーンショット、仮想クオンツ株選びソフトウェアのインターフェースや「マスター級」の投資分析レポートを偽造し、高額リターンを餌に高価な投資サービスを販売、または被害者の資金を自身が管理する取引アカウントに振り込ませ、**

最終的に資金を持ち逃げ。2. 破りの手法——「照妖鏡」の防護思路

**AI は声と顔をクローンできるが、無敵ではなく、いくつかの綻びを利用できる：**

動作検証法：ビデオ通話時に、相手に事前設定した連続動作指令の完了を積極的に要求する。例：「手を顔の前で素早く振る」「横顔で耳の輪郭を表示」「5 回連続で素早く瞬きする」など。現在の AI 顔交換技術は複雑な動的遮蔽、高速運動シーン、光と影の変化を処理する際、。

## 破りの手法——「照妖鏡」の防護思路

暗号検証法

- ：事前設定したプライベート検証質問を確立して使用する。例：「前回一緒に食事した時にあなたが注文した料理」「私があなたに贈った最初のプレゼント」など、双方のみが知る詳細。相手に明らかなためらいや質問に答えない状況があれば、直ちに通話を終了しセキュリティプランを起動すべき。二次確認法

- ：公安部サイバー安全局が発表した「四不一確認」原則を厳格に遵守——見知らぬ情報を軽信しない、随意に送金・振込しない、個人のセンシティブ情報を漏洩しない、疑わしいコンテンツを拡散しない。お金のやり取りに関わる場合は、必ず。

- 。振込操作を行う際は「24 時間到着」モードを選択することを推奨し、潜在的な詐欺の識別と資金回復のための十分な時間窓を確保。3. 従来のよくある手口（依然として警戒が必要）：当選 / キャッシュバック詐欺：利益で誘い、まず少し甘い汁を吸わせてから大金を騙し取る。ネット恋愛詐欺：養豚詐欺、感情を育ててからお金を騙し取る。

## 従来のよくある手口（依然として警戒が必要）：

- ：アカウント乗っ取り後にお金を借りる（AI 時代はさらに見分けが困難）。4. スマートフォン内蔵の防護：

- 迷惑電話ブロック。認証コード窃取防止。

- SMS リンクのセキュリティ検査。5. 詐欺防止の標語：

## スマートフォン内蔵の防護：

- 「急ぎの用事」に遭遇したらまず 5 分間冷静になること。ほぼすべての詐欺はこの瞬間に尻尾を出す。

- 五、「開盒」とソーシャルエンジニアリング攻撃の防御：プライバシー漏洩の連鎖を断ち切る

- 「開盒」（ソーシャルエンジニアリング攻撃）の核心的な原理は、

### 詐欺防止の標語：

**すること。防御の鍵は個人プライバシーに明確な赤線を引き、多層的な防護体系を構築すること。**1. 写真の匿名化処理

## 五、「開盒」とソーシャルエンジニアリング攻撃の防御：プライバシー漏洩の連鎖を断ち切る五、「開盒」とソーシャルエンジニアリング攻撃の防御：プライバシー漏洩の連鎖を断ち切る

写真のメタデータ（EXIF 情報）を完全に削除**する必要がある。撮影時間、デバイス型番、GPS 位置座標などのセンシティブデータを含む。写真に含まれる**門牌番号、ナンバープレート情報、乗車券バーコード、ID カード番号

### 写真の匿名化処理

基本的な EXIF 削除方法は第二章「画像とファイルシステムの管理」を参照してください。ソーシャルメディアで写真を共有する前は、さらに **Android：一部のフォトアプリ（Google フォトなど）は共有時に位置情報を保持するかどうかを確認。**汎用ツール：ExifTool、Metapho などで EXIF を表示/編集可能。**2. アプリ権限の収束** E コマース系、ツール系アプリの

- マイク、連絡先、位置情報
- などの不要な権限を定期的に審査してオフにし、アプリがバックグラウンドで日常会話を「盗み聞き」したり、個人の関係ネットワークを違法に収集することを防止。長期間未使用だが高権限を占有するアプリは速やかにアンインストールし、源からプライバシー漏洩のリスクポイントを削減。
- 「連絡先権限」の申請を見た時、まず問いかける：このアプリは本当に私のすべての連絡先を知る必要があるか？

### アプリ権限の収束

基本的な権限管理方法は第四章「権限管理」を参照してください。「個人情報特定攻撃（開盒）対策」の観点では、さらに定期的に EC 系・ツール系アプリの**スマートカメラ、スマートドアロック、スマートスピーカーなどの IoT デバイスは初回使用時に**直ちにデフォルトのログインパスワードを変更

- し、大文字・小文字、数字、特殊記号を含む高強度の独立パスワードを設定。使用頻度の低いリモートアクセス機能はオフ状態を保ち、デバイスのファームウェア更新を定期的にチェックしてセキュリティ脆弱性を適時に修復し、
- ハッカーの侵入と家庭プライバシー空間の覗き見を防止

### スマートデバイスのセキュリティ防護

実例の警告 **2022 年ある地域の警察の通報によると、ハッカーがあるブランドの家庭用カメラのデフォルトパスワード（多くのユーザーが未変更）を破解し、千戸以上の家庭の寝室の映像を窃取してネット上で販売。**デフォルトパスワードを変更しない＝家のドアを開け放しにする。。

スマートフォン内の写真、チャット履歴、位置軌跡は本質的にすべてあなたの「デジタル資産」。一度漏洩すれば、軽ければ広告の嫌がらせ、重ければ個人情報の暴露、詐欺、恐喝に遭う。

1. アプリ権限の「最小化原則」：**位置。**

## 六、プライバシー保護：あなたのデータはあなたの資産六、プライバシー保護：あなたのデータはあなたの資産

**：**必要な場合（WeChat の友達追加など）のみ許可、それ以外は一律に禁止。

### アプリ権限の「最小化原則」:

- **：iOS 14+ から「選択した写真」を権限付与可能、Android 13+ も同様に詳細な権限付与をサポート。**マイク / カメラ
- **連絡先**バックグラウンド追跡
- **：iOS 14.5+ からアプリが IDFA を使用する前に必ず質問することを強制；Android は「広告 ID」をオフに可能。**2. プライバシー管理の実践：
- **プライバシーレポート（iOS）：**[設定] > [プライバシーとセキュリティ] > [アプリプライバシーレポート] で、どのアプリがこっそりあなたの位置、カメラ、マイクにアクセスしているか確認。
- **権限使用履歴（Android）：**[設定] > [プライバシー] > [権限使用履歴] で、誰がいつセンシティブな権限を呼び出したか確認。

### プライバシー管理の実践：

- **：システム設定で「広告 ID」や「広告追跡を制限」をオフにすることで、「精密な広告の嫌がらせ」を減らせる。**七、紛失 / 盗難後の「救急」チェックリスト
- **スマートフォンを失くしても慌てず、以下の順序で処理：**直ちに SIM カードを凍結
- **：キャリアに連絡して紛失届けを出し、認証コードの受信に悪用されるのを防止。**リモートロック

## 七、紛失 / 盗難後の「救急」チェックリスト七、紛失 / 盗難後の「救急」チェックリスト

重要パスワードの変更

1. **：スマートフォンにログインしたことのあるすべての重要アカウント（決済、銀行、メール）のパスワードを直ちに変更。**被害届
2. **：警察署に被害届を出し、IMEI 番号を記録。スマートフォン発見時にも身元証明に役立つ。**親族・友人への通知
3. **：詐欺師があなたの名義でお金を借りるのを防止。**（最後に）リモートワイプ
4. **：発見が絶望的と確認した時に実行。ワイプ後は位置特定機能が失効。**八、SIM カードと eSIM のセキュリティ
5. **SIM カード PIN の設定：**有効化すると、スマートフォンの再起動やカード交換のたびに PIN コードの入力が必要、「カードを抜いてスマートフォンを交換」してロック画面をバイパスするのを防止。
6. **eSIM の優位性：**物理カードが抜かれることがなく、セキュリティがより高い；同時に複数番号の切り替えをサポートし、旅行時に非常に便利。

## 八、SIM カードと eSIM のセキュリティ八、SIM カードと eSIM のセキュリティ

- **：「キャリア二次カード交換」詐欺に警戒——詐欺師があなたの身分を冒用して SIM カードを再発行し、あなたのスマートフォン番号を乗っ取る。「カード再発行は本人が身分証を持って店舗で手続きが必要」と設定するのが最も有効な防線。**絶対にやってはいけないこと
- **誕生日 / 123456 / qwerty などの弱いパスワードを使用；**認証コードを誰かに送信（「カスタマーサービス」であっても）；
- **SMS 内の不明なリンクをクリック、特に 00 で始まる海外番号；**公共 Wi-Fi でネットバンキングや決済にログイン；

**見知らぬ人に「画面共有」や「リモートコントロール」を有効化。**

- 「リモートロック」の有効化とシミュレーション
- 「スマートフォンを探す」機能が有効になっていることを確認。
- PC のブラウザまたは別のスマートフォンで、対応するクラウドサービスアカウント（iCloud.com またはブランドクラウドサービスサイト）にログイン。
- 「デバイスを探す」機能を見つけ、自分のスマートフォンに対して「サウンド再生」

または「ロック」のシミュレーション指令を送信し、全体の操作フローに慣れる。

- 第七章：最先端機能の完全解説

テクノロジーは日々進化し、スマートフォンの機能も絶えず進化している。衛星通信、ネットワーク不要通信、衝突検出、星閃技術……。これらの SF のように聞こえる機能は、実はすでに私たちの生活に入り込んでいる。これらの最先端技術を一緒に理解しよう。

1. 一、衛星通信：信号がなくても救助要請可能
2. 衛星通信は近年のスマートフォン分野の大きなブレイクスルー。荒野や深山幽谷でスマートフォンが完全に信号がない時、衛星通信が命綱になる可能性がある。
3. 動作原理：

# 第七章：最先端機能の完全解析

対応機種：

## Apple

：iPhone 14 シリーズ以降、衛星経由での緊急 SOS 送信をサポート。iPhone 16 シリーズはさらに衛星経由での iMessage と SMS 送信をサポート。

### 対応機種：

：Mate 50 シリーズから北斗衛星メッセージ（テキストのみ）をサポート、Mate 60 Pro シリーズはさらに天通衛星通話（電話可能）をサポート、Mate 70 シリーズは衛星通信能力がさらに強化。

### 注意事項：

- **Apple：**iPhone 14 シリーズ以降、衛星経由での緊急 SOS 送信に対応しています。iPhone 17 シリーズはさらに衛星経由の iMessage と SMS 送信に対応し、より多くの地域でのカバレッジを拡大しています。
- **Huawei** 使用シーン：
- **アウトドア探検、登山中の遭難。**海上での遭難。
- **自然災害による地上通信の中断。**使い方：

### 開けた場所にいることを確認し、空が見えること。

- スマートフォンの衛星通信機能をオンにする（通常は緊急発信インターフェースにある）。
- 画面の指示に従い、衛星の方向に向ける。
- 事前設定された救助要請メッセージと位置を送信。

### 豆知識

1. 衛星通信にはスマートフォンに特殊なハードウェアアンテナが必要で、すべてのスマートフォンがサポートしているわけではない。また、衛星通信は通常有料のサブスクリプションサービスが必要だが、緊急救助機能は一般的に無料。

2. 二、ネットワーク不要通信：近距離通信の新方式

3.「ネットワーク不要通信」とは、モバイルネットワークや Wi-Fi がない状況で、2 台のスマートフォン間で直接通信する技術を指す。

4. 技術原理：

**Bluetooth や専用の無線周波数を利用し、近距離（通常数百メートルから数キロメートル）でポイントツーポイント接続を確立。**

応用シーン：

## アウトドア活動

：ハイキング、キャンプ時のチームメンバー間の通信。

### 災害救援

：地震、洪水後の通信中断時。

### 活用シーン：

- **：コンサート、スポーツイベントで人が多く信号が悪い時。**対応プラットフォーム：
- **Apple：**iOS 17 の「NameDrop」と「オフラインで友達を探す」機能。
- **大型イベント：**Changlian アプリがネットワーク不要通話をサポート。

### 対応プラットフォーム：

- **Apple** 三、衝突検出：決定的瞬間に命を救う
- **Huawei** 動作原理：
- **Xiaomi** カウントダウンインターフェースをポップアップし、あなたが無事かどうかを確認。

## カウントダウン終了時に応答がない場合、自動的に緊急電話を発信。

緊急連絡先にあなたの位置を送信。

### 対応機種：

Apple

1. ：iPhone 14 シリーズ以降。
2. カウントダウン内に応答がない場合、自動で緊急電話を発信します。
3. ：一部のハイエンド機種が順次サポート。

### 注意事項：

- **Apple** 誤検出の可能性あり（ジェットコースター、激しい運動など）、その際は速やかにキャンセルすればよい。
- **Android** 四、星閃技術：次世代ワイヤレス接続

**星閃（SparkLink）は Huawei が主導して開発した次世代ワイヤレス通信技術で、特定のシーンで Bluetooth と Wi-Fi を代替することを目指す。主要指標で既存のソリューションに匹敵、あるいは超越：空中速度は 900Mbps に達し（Bluetooth の 50Mbps を遥かに超越）、レイテンシは 20 $\mu$ s まで低減（Bluetooth は約 15ms）、単一ネットワークで 4096 デバイスの同時接続をサポート（Bluetooth は通常 8 台以内）。**

- 技術的特徴：
- 超低レイテンシ
- ：エンドツーエンドのレイテンシはわずか 20 マイクロ秒（0.02ms）、Bluetooth より 2 桁低く、ゲームや音楽の同期はほぼ知覚できない。

## 高速

：単一リンクのピーク速度 900Mbps、Bluetooth オーディオの数十倍で、ロスレスオーディオや HD 動画の転送が可能。

### 低消費電力

- **：待機時の消費電力は Bluetooth と同等だが、伝送効率が高く、単位データあたりのエネルギー消費が低い。**広カバレッジ + 高同時接続
- **：接続距離は Bluetooth より 50% 以上長く、単一ネットワークで数千デバイスの同時オンラインをサポート。**応用シーン：
- **ワイヤレスイヤホン：**より低いレイテンシ、ゲーム時に音画同期。
- **スマートウォッチ：**より安定した接続。

活用シーン：

- ：より多くのデバイスの同時接続。車載連携
- ：よりスムーズなスマートフォン-カーナビ接続。対応デバイス：
- スマートホーム技術展望
- 星閃技術はワイヤレス通信の未来の方向を代表する。より多くのメーカーが参加することで、星閃対応デバイスが増え、より優れたワイヤレス体験を享受できるようになるだろう。確認してみよう：あなたのスマートフォンがこれらの機能をサポートしているか

設定で「衛星」を検索し、衛星通信関連のオプションがあるか確認。

「衝突検出」や「緊急 SOS」を検索し、有効になっているか確認。

スマートフォンが Huawei の場合、星閃デバイス接続をサポートしているか確認。

これらの機能の使用方法を理解し、いざという時に役立てる。

二、ソフトウェア編（第 8-14 章）

1. 基本操作をマスターした後、各種アプリの使用テクニックを深く理解しよう。通信からオフィス、決済からエンターテインメントまで、これらの章はあなたを真のスマートフォン達人にする。
2. 第八章：通信：SMS、電話とソーシャル
3. 通信はスマートフォンの最も原始的で最も核心的な使命だ。現在、それは単純な通話と SMS から、インスタントメッセージ、ソーシャルネットワーク、電子メールを含む複雑なエコシステムへと進化した。
4. 一、電話アプリの高度な機能

# 二、ソフトウェア編（第 8-14 章）

# 第八章：コミュニケーション——SMS・電話・SNS

コミュニケーションは、スマホにとって最も原始的で中核的な使命です。今では、シンプルな通話や *SMS* から、メッセージアプリ、*SNS*、メールを含む複雑なエコシステムへと進化しました。

## 一、電話アプリの高度な機能

現代のスマホの電話アプリは、もはや単なるダイヤルパッドを超え、録音、迷惑電話ブロック、スマート識別などの強力なツールを統合しています。

### 通話録音（法令遵守に注意）：

- 役割：重要な通話内容を記録し、業務メモ、取材、顧客との約束、紛争時の証拠として活用します。
- 法律上の注意：中国国内では、通話録音を証拠として使うには、相手の同意を得るか、自らの合法的権利を守るためであることが前提です。
- 一方的録音と双方同意：法的には、「相手の同意なし」に取得した録音が当然に無効になるわけではありません。最高人民法院の関連司法解釈では、録音の取得方法が他人の合法的権益を深刻に侵害しておらず、法律の禁止規定に違反しておらず、公序良俗に反していなければ、証拠として提出できます。ただし、一部の省の裁判所は民事裁判で「こっそり録音」の採用基準をより厳しくしており、刑事事件では通常さらに厳格な適法性審査が求められます。
- 地域差：北京、上海、広東などの裁判所は一方的録音を民事証拠として比較的寛容に扱います。一方、営業秘密や個人のプライバシーに関わる録音は、どの省でも特に慎重に扱う必要があり、プライバシー侵害を理由に排除される可能性があります。
- 操作方法：Android は通常、通話画面に明確な「録音」ボタンがあります。iOS は標準の通話録音機能を提供していないため、サードパーティアプリや録音機能付きの Bluetooth ヘッドセットを使います。

**録音利用のレッドライン**

- 録音を恐喝、拡散、他人への公開侮辱に使ってはいけません。
- 第三者のプライバシーに関わる内容（たまたま他人の会話を録ってしまった場合など）を勝手に拡散してはいけません。
- 重要な場面では、通話の冒頭で「この通話は録音されます」と口頭で伝えることをおすすめします。コンプライアンス上も、証拠としての効力も高まります。

### 迷惑電話対策の深掘り：

- 仕組み：電話アプリがネットワーク経由で膨大な番号データベースを照会し、着信時に「迷惑電話」「詐欺電話」や会社名を表示します。
- 有効化方法：Android は電話アプリ > 設定 > 迷惑ブロック、iOS は設定 > 電話 > 着信拒否と発信者表示。ブランドごとの詳しい設定パスと推奨項目は、第十三章「実用システムツール」のブランド対応表をご覧ください。
- 主要な表示プラットフォーム：国内の番号表示は主に 3 社のデータサービス——騰訊手机管家、360 Mobile Guard、電話邦（Tapir）——が提供しています。通信事業者（China Mobile / China Unicom / China Telecom）の「名義人表示」や「迷惑ブロック」機能も、多くの場合これらのデータを参照します。スマホメーカー（Huawei、Xiaomi、OPPO、vivo）の表示データベースは、多くが上記サービスと提携しています。
- 誤表示されたら：仕事用の番号が「出前」「宅配」「迷惑電話」などと誤って表示された場合、次の窓口で申し立てできます：電話邦：番号申訴プラットフォーム（haoma.tk.cn）にアクセスして身分証明を提出します。
- 360：360 Mobile Guard アプリ内の「番号申訴」機能から提出します。
- 騰訊：「騰訊手机管家」ミニプログラムの「番号申訴」ページから提出します。
- 大手 3 事業者：China Mobile のユーザーは SMS「KTFSR」を 10086 へ送信して迷惑防止機能を有効化できます。China Telecom のユーザーは「天翼防騒擾」公式アカウントをフォロー、China Unicom のユーザーは「聯通手机管家」で設定します。

迷惑対策の応用：通信事業者の「高頻度発信ブロック」（例：China Mobile の高頻度発信制限）を有効にすると、AI 音声による自動発信を自動的にブロックできます。

スマホ標準のブロック主な機能：着信識別 / スマートブロック：番号データベースに基づき、迷惑・詐欺・広告勧誘を自動的に表示・ブロックします。

ブラックリスト / ホワイトリスト：既知の迷惑番号を手動でブロック。連絡先や宅配・出前番号はホワイトリストに入れて、誤ブロックを防ぎます。

SMS キーワードフィルタ：「当選」「融資」「リンクをクリック」などのキーワードを含む SMS は、自動的にブロックボックスへ振り分けられます。

iOS の不明な着信を消音：連絡先にない着信はボイスメールへ送られるだけで、中断されません。迷惑電話が多い人に向いていますが、見知らぬ番号からの大事な着信を逃す点には注意が必要です。

おやすみモードとの連携：夜間や会議中は連絡先からの着信だけを許可し、タイマーと組み合わせると、睡眠の質がすぐに改善します。

実用ソフトの組み合わせ方：まずはスマホ標準のブロックを優先し、その上で国家反詐中心や通信事業者の迷惑対策を重ねます。クリーン系は Qingzhuo などの軽量ツールが選べますが、セキュリティ / クリーン系は 1 系統を入れれば十分です。詳しくは第十三章をご覧ください。

### VoLTE / VoNR とは？有効化方法：

- VoLTE（Voice over LTE）：4G ネットワーク上の音声通話技術。通話中に 2G / 3G へ落ちることがなく、通話しながらネットも使え、音質は従来の通話よりクリアで、つながりも速くなります。

- VoNR（Voice over NR）：5G 上の音声通話で、VoLTE の次世代版です。遅延が低く、音質が良く、高精細ビデオ通話にも対応します。使えるには、5G 対応スマホ＋ 5G プラン＋現地の 5G カバレッジの 3 つが必要です。

- 有効化方法：iOS は「設定 > モバイル通信 > モバイルデータオプション > 音声とデータ」で「VoLTE」をオンにします。Android は「設定 > モバイルネットワーク > モバイルデータ > 詳細設定」で「VoLTE 通話」のスイッチを見つけます。通信事業者側で既定オンになっていることが多いのですが、通話中にネットが切れて使えなくなる場合は、VoLTE が有効になっていない可能性が高いです。

**豆知識**

通話中にネット速度が急に遅くなったり切断されたりする場合、多くは VoLTE / VoNR がオフで 2G / 3G に落ちているためです。オンにすれば「通話しながら動画視聴」ができます。

## 二、SMS と iMessage / RCS

従来の SMS はまだ廃れていません。認証コード、銀行通知、公式情報を受け取るうえで、今も信頼できるチャネルです。

- iMessage：Apple デバイス間でインターネット経由で送受信するメッセージで、青い吹き出しに表示されます。高画質の画像・動画、既読通知などに対応し、完全無料でエンドツーエンド暗号化されています。

- RCS：世界の通信事業者と Google が推進する次世代メッセージング規格です。iMessage や WeChat に似た機能ですが、電話番号ベースです。

- iMessage と RCS の相互接続：iOS 18 以降、Apple は RCS 標準に対応し始めました。これにより、iPhone と Android の間でも RCS で高画質画像、既読通知、入力中表示が送れ、「緑の吹き出し」による体験の断絶がなくなります。ただし、双方の通信事業者が RCS に対応しており、端末の OS も十分に新しいことが条件です。

### RCS の国内展開状況：

- China Mobile：RCS の展開が比較的早く、一部の省では「5G 消息」サービスに対応済み。リッチメディアやインタラクティブカードを送れます。端末側の対応と、営業窓口での開通手続きが必要です。
- China Telecom：「電信 5G 消息」として提供。一部のプランは既定で対応しており、既存ユーザーは指定の SMS コマンドを送って開通します。
- China Unicom：主要都市で 5G 消息を展開していますが、端末対応はまちまちです。多くの Android 機種では、SMS アプリ内の「拡張メッセージ」または「5G 消息」スイッチを手動でオンにする必要があります。
- 実際の体験：国内の RCS 普及は予想より遅く、WeChat エコシステムの力の強さや端末対応の不統一などが主な要因です。日常の通信は依然 WeChat が主役で、RCS はむしろ「公式プッシュ」（銀行通知や行政メッセージなど）の役割を担っています。

**認証コードのセキュリティ・レッドライン**

- 認証コードを転送しない：「認証コードを読み上げて」「SMS を転送して」と言ってくる人は、たとえ「カスタマーサービス」「宅配員」「警察・検察・裁判所」を名乗っていても詐欺の可能性があります。認証コード＝あなたの財布です。
- フィッシング SMS に注意：「ETC が期限切れ」「医療保険カードが凍結」「荷物紛失で補償」などの SMS に含まれるリンクに注意してください。正規の機関が、SMS のリンク経由でカード番号やパスワードを求めることはありません。
- 番号の見分け方：銀行の公式 SMS は通常 95 始まりの短縮番号か 1069 ＋企業署名から届きます。一般の 11 桁の携帯番号から来た「銀行通知」は、ほぼ詐欺だと思ってください。
- 二要素認証のバックアップ：重要なアカウントに紐づく携帯番号は、安易に変更しないでください。変更せざるを得ない場合は、必ず先にすべての重要アカウントの紐付けを解除してください。そうしないと、新しい名義人が SMS 認証コードであなたのパスワードをリセットできるおそれがあります。

## 三、WeChat / QQ：国民的 SNS アプリの管理

WeChat と QQ はもはや通信ツールを超え、SNS・決済・情報収集を一体化した「スーパーアプリ」になっています。

### チャット履歴のバックアップと移行：

1. パソコンで PC 版または Mac 版 WeChat にログインします。
2. 左下のメニュー > [移行とバックアップ] > [バックアップと復元] をクリックします。
3. 「チャット履歴をパソコンにバックアップ」を選び、スマホでバックアップしたいチャットを選択します。
4. 新しいスマホへの移行：元のスマホの WeChat で [マイページ] > [設定] > [全般] > [チャット履歴の移行とバックアップ] > [移行] > [スマホ / タブレットへ移行] を開き、QR コードを読み取れば完了です。

### ストレージ容量のクリーンアップ深掘り：

- 通常の入口：WeChat の [マイページ] > [設定] > [全般] > [保存スペース]。キャッシュの削除に加え、日時やファイル種別でチャット履歴を細かく整理できます。
- チャット内での整理：個別のチャットを開く > 右上の「…」> [チャット履歴] > [保存スペースの管理]。特定のチャットの画像・動画・ファイルだけを削除し、テキストは残せます。
- 元画像 / 元動画の整理：グループチャットで受け取った高画質の元画像・元動画が一番容量を取ります。[保存スペースの管理] で「元画像」にチェックして一括削除できます。
- 自動ダウンロードのオフ：[設定] > [全般] > [写真・動画・ファイルと通話] で「自動ダウンロード」をオフにすると、Wi-Fi 環境で大量のグループファイルが黙ってキャッシュされるのを防げます。
- ミニプログラムのキャッシュ：[発見] > [ミニプログラム] で使っていないミニプログラムを長押しして削除すると、ローカルキャッシュが解放されます。
- チャット履歴の圧縮バックアップ：PC 版 WeChat でバックアップする際に「圧縮」にチェックすると、数 GB のチャット履歴を元の 30　50%まで圧縮できます。

### グループ管理のテクニック：

- グループ公告：オーナー / 管理者がグループチャット右上の「…」> [グループ公告] で発信します。全員に強制通知されるので、重要なお知らせに最適です。新メンバーが入ったときも自動的に表示されます。
- グループ ToDo：オーナーがピン留めメッセージを「グループ ToDo」に設定できます。全メンバーの頭上に赤いアイコンが出て、完了すると自動で消えます。イベント申し込みやタスク収集に便利です。
- グループ連鎖（接龍）：入力欄で「# 接龍」と入力すると連鎖を開始できます。注文や申し込みの集計に向いています。結果はグループチャット上部に自動でまとめられ、スクロールの嵐を防げます。

- グループライブ：グループチャット右上の「＋」＞[グループライブ]。このグループだけが見られるライブを開始でき、社内研修、家族の集まり、製品のクローズド　に最適です。
- グループチャットを折りたたむ：グループ一覧でグループ名を左にスワイプ＞[折りたたむ]。重要でないグループを「折りたたまれたグループ」にまとめ、自分への @ メンションがあるときだけ通知されます。
- @ 全員はオーナー / 管理者のみ：メンバーが 500 人を超えるグループでは、一般メンバーは @ 全員を使えなくなり、不要な通知を防げます。

### 朋友圏（モーメンツ）の権限を細かく管理：

- グループ別公開範囲：投稿時に「誰が見られるか」をタップし、「一部の人に見せる」または「一部の人に見せない」を選びます。事前に設定したタググループ（「同僚」「家族」「顧客」など）に基づいて正確に配信できます。
- 友だちの閲覧範囲：[マイページ] ＞ [設定] ＞ [友だちの権限] ＞ [朋友圏] ＞ [友だちに見せられる朋友圏の範囲]。[直近 3 日 / 1 か月 / 半年 / すべて] から選べます。プライバシーと SNS のバランスを考えるなら「半年」がおすすめです。
- 特定の朋友圏を非表示：朋友圏の投稿を長押し ＞ [彼 / 彼女の朋友圏を見ない] または [フォローを解除]。相手には通知されません。
- 朋友圏広告の管理：[マイページ] ＞ [設定] ＞ [WeChat について] ＞ [個人情報と権限] ＞ [パーソナライズ広告の管理] で、行動ベースのパーソナライズ推薦をオフにできます。
- 見知らぬ人への 10 件表示：[友だちの権限] ＞ [朋友圏] ＞ [見知らぬ人に朋友圏を 10 件見せる]。オフにすると、見知らぬ人に過去の投稿を見られるのを防げます。

## 四、ビデオ通話の比較

ビデオ通話ツールにはそれぞれ得意分野があります。シーンに合ったツールを選ぶと、コミュニケーションがスムーズになります。

- WeChat ビデオ：国民的な普及率で、年齢を問わず使えます。親友や家族との日常連絡、すばやいやり取りに最適。最大 20 人のビデオ通話に対応（ミニプログラム「グループビデオ」が必要）。弱点は、回線が悪いと画質が大きく落ちることです。
- FaceTime：Apple エコシステム専用。画質が優れ、エンドツーエンド暗号化に対応。iOS 15 以降は「リンク」で Android / Web ユーザーも招待できますが、ホストは Apple デバイスである必要があります。家族でのデバイスをまたぐ連絡に向いています。
- Zoom：国際会議の定番。最大 100 人までの無料会議を 40 分まで実施できます。画面共有、バーチャル背景、ブレイクアウトルームなど機能が充実し、グロボーバル / リモート協働に最適です。

- 騰訊会議（Tencent Meeting）：国内オフィスの定番。WeChat と深く連携し、QR コードですぐ参加できます。300 人会議（有料版）、クラウド録画、リアルタイム字幕、議事録機能が強力で、社内研修や大規模会議に向いています。

- 利用シーン別のおすすめ：1 対 1 の日常連絡は WeChat、Apple ユーザー同士なら FaceTime、正式なビジネス会議は Zoom か騰訊会議（国内は騰訊、クロスボーダーは Zoom）。

### 必要帯域とデータ消費の目安：

- WeChat ビデオ通話：約 500Kbps　1Mbps の帯域が必要。標準画質で毎分約 3　5MB、高画質で約 8　12MB を消費します。

- FaceTime：SD で約 1Mbps、720p で約 2Mbps、1080p では 4Mbps 以上が必要。データ消費は毎分約 5　15MB です。

- Zoom / 騰訊会議：720p で約 1.5　3Mbps、1080p で 3　5Mbps が必要。1 時間あたり約 300　800MB で、多人数会議ではホストの消費がさらに大きくなります。

- アドバイス：長時間のビデオ通話はできるだけ Wi-Fi を。モバイル回線では画質を意識的に下げ（アプリ設定で「高画質ビデオ」をオフに）、データを節約しましょう。

### ビデオ通話の画質を上げるコツ：

- 照明：窓に背を向け、光源の方向を向く（自然光が最適）。逆光で顔が暗く映るのを避けます。

- アングル：スマホのカメラは目より少し高く。あご下が見えにくくなります。横向き通話では両手で持つと安定します。

- オーディオ：ヘッドセットを使うとエコーやハウリングを防げます。静かな環境では「スピーカー」モードはオフに。

- ネットワーク：Wi-Fi を優先。4G / 5G では窓際に近づいて電波の減衰を避けます。多人数会議では他の映像をオフにして話者だけを表示しましょう。

- 背景ぼかし：騰訊会議と Zoom はどちらも AI 背景ぼかしに対応し、散らかった背景を隠せます。無地の服装を選び、背景色と近いと「クロマキー失敗」のようになる点に注意しましょう。

**ヒント**

回線がカクついたら、切る前にまず映像をオフにして音声だけにする方がマナー上も丁寧です。重要な会議の前には、騰訊会議の「ネットワーク診断」機能で帯域を事前に確認できます。

## 五、メール：スマホでの設定と使い方

- メールアカウントの追加：iOS は設定 > メール > アカウント > アカウントを追加。Android には通常「メール」アプリがあります。
- 活用のコツ：メールアカウントごとに通知方法を変える。スワイプ操作でアーカイブ、削除、マークをすばやく行う。

### IMAP・POP3・Exchange の違い：

- POP3：サーバーからメールをダウンロードした後、既定では元のメールを削除し、ローカルだけに残します。1 台の端末だけで使い、オフライン閲覧したい場合に適しています。弱点は、端末を変えると過去のメールが見えなくなること。現在はあまり推奨されません。
- IMAP：メールは常にサーバーに保存され、ローカルはキャッシュとして扱います。複数端末（スマホ / PC / タブレット）で状態が同期——片方ですべて既読になります。現在の主流で、推奨プロトコルです。
- Exchange（ActiveSync 含む）：Microsoft のプロトコル。メール以外にカレンダー、連絡先、タスクも同期します。企業メールの第一候補。Outlook、NetEase 企業郵箱、騰訊企業郵箱などに対応しています。
- 選び方の目安：個人メールは IMAP。会社のメールは Exchange を優先し、次点で IMAP。特別なアーカイブ要件がない限り、POP3 は避けましょう。

### 企業メールの設定方法：

1. iOS：[設定] > [メール] > [アカウント] > [アカウントを追加] > 「Microsoft Exchange」または「その他」を選択します。
2. Android：標準の「メール」アプリまたは Outlook アプリで「アカウントを追加」を選び、完全なメールアドレスとパスワードを入力します。
3. サーバー設定（Exchange の例）：サーバーアドレスは通常「mail. 会社ドメイン.com」または「outlook.office365.com」。ドメインには会社の略称を入れます。
4. 「サーバーを検証できません」と表示された場合は、手動で入力します。受信サーバー（IMAP）はポート 993、暗号化は SSL。送信サーバー（SMTP）はポート 465 または 587、暗号化は TLS です。
5. ログイン後は [アカウント設定] で「プッシュ」をオンにしてメールをリアルタイム受信し、「メールプレビュー」をオフにして企業秘密を守ることをおすすめします。

**ヒント**

企業メールでは、管理者が発行する「クライアント認証パスワード」や「IMAP / SMTP 認証コード」が必要なことが多く、Web ログイン用パスワードはそのまま使えません。ログインに失敗したら、まず Web 版メールの設定で認証コードを発行し、それをスマホに入力してください。

**実践：WeChat のチャット履歴をパソコンにバックアップする**

1. パソコンでバックアップを開始：PC 版 WeChat を開く > 左下メニュー > [移行とバックアップ] > [バックアップと復元]。
2. パソコンへのバックアップを選択：「チャット履歴をパソコンにバックアップ」をクリックします。
3. スマホでバックアップ内容を選び、完了するまで待ちます。

# 第九章：モバイルオフィスと学習

一、ドキュメント処理：掌中での編集と創作

スマホ画面でドキュメントを編集するのは、パソコンとは確かに感覚が違います。最大の問題は画面が小さく、指が太く、レイアウトがずれやすいこと。でもいくつかのコツで体験はかなり良くなります：

WPS Office

- ：ローカライズが最も充実しており、**Microsoft** 形式（**docx/xlsx/pptx**）と完全互換、内蔵の **PDF** ツールセットが豊富。私が特に気に入っているのは「音声メモ」機能です。契約書を見る時にタイピングしにくい場合、スマホに向かって意見を数言話すだけで、自動的にテキスト注釈になってドキュメントに貼り付けられます。また **WPS** には大量の無料テンプレートが内蔵されており、履歴書や報告 **PPT** を作る時に時間を大幅に節約できます。Microsoft 365

- ：会社が正規の **Office** を使っているなら、これがベストチョイス。デスクトップ版とのフォーマット互換性はほぼ完璧で、**WPS** で複雑なレイアウトを開くとわずかにずれるようなことがありません。**OneDrive** と深く統合されており、パソコンで編集途中のドキュメントも、スマホで開けば最新版になっています。石墨ドキュメント/テンセントドキュメント

- ：この **2** つは「オンライン協業」の代表。複数人で同時に一つのドキュメントを編集するのに適しており、各自の変更がリアルタイムで表示されます。私は石墨ドキュメントを使って友人と旅行ガイドを一緒に作りました。二人で同時に表に情報を入力する効率は、一人でやるよりはるかに高いです。実用的なワークフロー

：私の現在の習慣は、重要なファイルは **OneDrive** に保存、日常的な編集は **WPS**、チーム協業は石墨ドキュメントを使う。ファイルをクラウドに置くようになってから、「**USB** メモリを忘れた」という恥ずかしい思いはもうありません。二、ノートアプリ：手軽なメモから知識体系へ

ノートを取るという事について、多くの人は最初から複雑なシステムを作ろうとして、三日も続きません。私の提案は、まず軽く始めて重くする。「記録する」習慣を身につけてから、「管理する」問題を考える。

手軽なメモ——システム標準で十分

- **：iOS** メモは起動が極めて速く、書類スキャン、手書き、録音にも対応しています。**Android** 各社のメモ帳も劣らず、ロック画面状態から素早く起動できます。これらのツールは一時的なアイデア、買い物リスト、会議の要点を記録するのに適しています。重要なのは起動が速く、思いついたらすぐ記録でき、**App** の読み込みを待つ必要がないこと。後で読む——情報収集
- **：WeChat** の「フローティングウィンドウ」機能は実は隠れた神器です。良い記事を見てじっくり読む時間がない時、右上の「フローティングウィンドウ」をタップすると、小さな円いアイコンになって画面の端に貼り付けられ、いつでもタップして読み続けられます。「ファイル転送アシスタント」に転送するよりずっと便利。また、**Cubox** や **Pocket** のような「後で読む」ツールは、どこで見た素晴らしいコンテンツでもクロスプラットフォームで収集できます。体系構築——Notion と飛書
- **：**散在するノートが探しにくくなってきたら、**Notion** に移行しましょう。これは本当に柔軟性が高い——「読書ノートデータベース」を作って、書名、著者、評価で自動的に整理できますし、「プロジェクトカンバン」を作って作業タスクの進捗を可視化することもできます。飛書の「多次元表」も同様で、かつ国内からのアクセス速度が速いです。私は **Notion** ですべての執筆素材、学習ノート、プロジェクトの進捗を管理しており、もう **3** 年以上使っています。実用的なノート戦略

**：**私自身の習慣は「三種の神器」——一時的なアイデアはシステムメモ帳（秒で起動）、後で読む記事は **WeChat** フローティングウィンドウ（手軽）、長期蓄積が必要な知識は **Notion**（体系化）。最初からすべてを一つのツールに詰め込もうとせず、そうすると逆に挫折しやすくなります。三、スキャンと OCR：紙の世界をスマホに取り込む

スマホスキャンは、うまく使えば印刷屋に走る手間をかなり省けます。前回ビザ申請で大量の書類の電子版が必要になった時、すべてスマホで対応しました。

iPhone ユーザー

- **：iOS** メモ標準のスキャン機能がすでに十分優れています。メモを開くカメラアイコンをタップ「書類をスキャン」を選択、書類に向けると自動的に枠を認識して透視を補正します。撮影後に色調整ができます——「白黒」モードを選ぶとスキャナーに最も近

い効果になり、「グレースケール」は画像入りの資料に適しています。Android ユーザー

- **Apple ユーザー** OCR 文字認識
- **：スキャンした書類の文字を抽出して編集するには、OCR が必要です。WeChat 標準の「文字抽出」機能が最も簡単——画像を長押しすれば使え、認識精度もなかなかです。大量処理が必要な場合は、白描 App や全能掃描王を使いましょう。中国語認識では、Baidu の OCR エンジンの効果が比較的良いです。WPS の画像からテキストへの変換機能も精度が高いです。**スキャンの小技
- **：明るい場所で撮影し、できればデスクライトを横から当てて、書類に手の影が落ちないようにします。書類の角が折れている場合は、手で平らにしてから撮りましょう。複数ページの書類は連続スキャン後に一つの PDF に統合でき、ページごとに別々に保存する必要はありません。**四、クラウドストレージの選び方：ベストはない、あなたに最適なものだけ

## クラウドストレージ選びについて私は長く考えましたが、結局「メインのエコシステム」が何かによるという結論に達しました。

Apple エコシステム iCloud

- **：その存在を感じさせないほどシームレスに同期されます。スマホで撮った写真が、数分後にはパソコンに自動的に入っています。また Apple のプライバシー保護は最も厳格です。ただし欠点も明らかで、Android や Windows ユーザーへのクロスプラットフォーム共有が面倒、無料枠は 5GB のみ、拡張には費用がかかります。**Windows メイン OneDrive
- **：Windows パソコンと Office 365 を使っているなら、OneDrive はまさに天作の組み合わせ。Word でドキュメントを保存すると自動的にクラウドに保存されます。1TB の容量が Office サブスクに付属しておりお得です。スマホ App の自動写真バックアップ機能も使いやすいです。**国内共有 Baidu ネットディスク
- **：正直、Baidu ネットディスクのダウンロード速度制限はずっと批判されてきましたが、その共有エコシステムは確かに他の追随を許しません。中国語インターネット上の大量の学習資料や映像素材は Baidu ネットディスクで共有されています。たまに他人が共有したファイルをダウンロードするだけなら、会員にならなくてもなんとかなります。ヘビーな資源収集家なら、会員になった方がいいでしょう。節約した時間の方が価値があります。**Alibaba クラウドドライブ/Quark ネットディスク
- **：この 2 年で台頭してきた新鋭。Alibaba クラウドドライブはアップロード・ダウンロード無制限（現時点）、Quark ネットディスクは Quark ブラウザと深く連携しており、資料検索時に直接ネットディスクに保存できて便利です。**私の組み合わせ

**：iCloud で個人写真と日常ファイル（プライバシー優先）、Baidu ネットディスクで資源保存と共有受信（エコシステム優先）、OneDrive で仕事ドキュメント（Office 統合優先）。3 つのクラウドストレージがそれぞれの役割を担い、干渉しません。**五、実際のモバイルオフィスシーン

## 私自身の実体験をお話ししましょう。ある時出張先で、クライアントから急に契約条項の修正を求められました。私のワークフローはこうでした：

ホテルでクライアントから WeChat で修正意見を受け取る（テキストメッセージ＋手書き注釈の写真 1 枚）。

1. OneDrive を開いて契約書原本を探し、WPS で開いて編集、クライアントの WeChat メッセージを参照しながら条項を一つずつ修正。
2. 修正完了後、WPS の「PDF として出力」機能で書き出し、クライアントの WeChat グループに送って確認。
3. クライアントから第二ラウンドの意見が出され、WPS のペンツールで直接 PDF 上に修正箇所を丸で囲み、音声注釈で修正の意図を説明。
4. 確認 OK 後、WPS の「電子署名」機能でオンライン署名し、相手にも同様に署名してもらう。
5. 最後に署名済みの契約書 PDF を Baidu ネットディスクと iCloud に 1 部ずつ保存し、同時に Notion の作業ログに記録。
6. 全過程は開始から終了まで約 40 分で、パソコンは一切開きませんでした。正直、3 年前はスマホでこれらができるとは信じませんでしたが、今では確実にできます。重要なのは、これらのツールに事前に慣れておくこと。必要な時に慌ててダウンロード・登録するのでは遅すぎます。

やってみよう：モバイルオフィスのワークフローを構築する

**メインのドキュメント App（WPS または Microsoft 365）を選び、ログインしてクラウドストレージを連携。**

1. メモまたは付箋を開き、スキャン機能で書類を撮影し、PDF として書き出して自分に送信。
2. クラウドストレージ App を一つ選び、スマホ写真の自動バックアップに設定（「Wi-Fi 時のみバックアップ」で通信量節約）。
3. Notion または飛書で新規ページを作成し、簡単な「作業ログ」テンプレートを構築してみる。

4. 第十章：モバイル決済と生活サービス

# 第十章：モバイル決済と生活サービス

一、銀行カードの登録方法とは？

## Alipay でも WeChat でも、初回のカード登録は以下の手順を踏みます：

App を開き、「マイ」または「ウォレット」を見つける。「銀行カードを追加」をタップ。

1. カード番号を入力。最近の App はほとんど写真でカード番号を認識できるので、一桁ずつ入力する必要はありません。
2. 銀行から事前登録した携帯番号に認証コード SMS が送信されます。入力して、6 桁の数字の支払いパスワードを設定。
3. この支払いパスワードは、銀行カードのパスワードやロック画面のパスワードとは別々にすること。楽だからといって同じにしないでください。
4. 一つの小技：セキュリティが心配なら、銀行 App で II 類または III 類の電子口座を開設し、スマホ決済専用にしましょう。このカードには日常の小銭だけを入れておけば、問題が起きても損害は限定的です。

二、支払いコードは「見た者が受け取る」

## 多くの人が支払いコードと収款コード（受取コード）の違いを混同していますが、この違いは重要です。

支払いコード

- **：あなたがレジ係に見せてスキャンさせるバーコード＋ QR コード。このコードはあなたの「電子財布」に相当し、スキャンした者はあなたの口座からお金を引き落とせます。だから絶対にスクリーンショットを撮って誰かに送らないでください。このコードは 1 分ごとに自動更新され、盗用を防いでいます。**収款コード

- **：他人がこのコードをスキャンしてあなたに送金します。このコードはスクショして保存でき、他人に送っても安全です。**この違いをしっかり認識すれば、多くの詐欺の罠を避けられます。「支払いコードのスクショを送って」と言う人は、100% 詐欺師です。

三、これらのセキュリティ設定、5 分で済ませて安心を得る

## 指紋/顔認証決済

- **：Alipay と WeChat の決済設定で、生体認証決済をオンに。これで毎回の決済時に指紋または顔スキャンが必要になり、スマホをなくしても拾った人はあなたのお金を使えません。**セキュリティロック
- **：Alipay と WeChat には「セキュリティロック」機能があります。オンにすると、App の「ウォレット」ページに入る時に追加の本人確認が必要。パス：Alipay マイ設定アカウントとセキュリティセキュリティセンターセキュリティロック。**暗証不要決済の確認
- **：多くの人が各種暗証不要決済に契約していて、自分でも忘れています。Alipay と WeChat の「決済設定」で「暗証不要決済/自動引き落とし」を探し、契約中のサービスを確認し、不要なものはオフに。**ログインデバイス管理
- **：定期的にどのデバイスがあなたのアカウントにログインしたか確認。知らないデバイスがあれば、すぐにキックアウト。パス：Alipay 設定アカウントとセキュリティログインデバイス管理。**四、バス・地下鉄：スマホが交通カード

## 最近外出してバスや地下鉄に乗る時、もう物理の交通カードを使う人は稀です。スマホを改札機にかざす方法は 2 つあります：

スマホウォレットの仮想交通カード

- **：Apple ウォレット、Huawei ウォレット、Xiaomi ウォレットで仮想交通カードを直接発行。この方法の利点は、かざす時にスマホをロック解除する必要がなく、直接近づけるだけで済むこと。応答速度が最も速く、ネット接続がなくてもかざせます。一部の都市の仮想交通カードには物理カードと同じ乗り継ぎ割引もあります。**Alipay/WeChat の乗車コード
- **：App で「乗車コード」を検索し、お住まいの都市のバスや地下鉄の乗車コードを開通。この方法の利点は、チャージ不要で、残高や連携した銀行カードから引き落とされること。ただしネット接続が必要で、かざす前に App を開いてコードを探す必要があります。**私は個人的にスマホウォレットの仮想交通カードをおすすめします。初回は 10　20 元チャージする必要がありますが、使い勝手が本当に段違いに良いです。特に地下鉄に急ぐ時に、立ち止まってスマホをロック解除して App を探す必要がありません。

五、生活費支払い、デリバリー、タクシー

## 水道光熱費

- **：Alipay または WeChat の「生活支払い」で、ご家庭の水道、電気、ガスの戸号を連携。毎月請求書が届いたら、そこから直接支払えます。戸号はご家庭の紙の請求書で確認できるか、カスタマーサービスに電話して聞けます。**デリバリー比較
- **：同じ店でも美団と餓了麼で価格と割引が異なることがよくあります。注文前に両方見比べると、数元節約できることがあります。また、美団の「拼好飯」機能は、友人と一緒に注文すると価格がかなり安くなります。**タクシーの安全
- **：滴滴や高徳タクシーで、「緊急連絡先」と「行程共有」を設定しておきましょう。タクシーに乗ると、App が自動的にあなたの行程を設定した連絡先に共有します。また、深夜のタクシーは「快車」より「タクシー」を優先しましょう。タクシーは業界の監督があり、安全度が高めです。**六、スマホで医療保険、社会保険、行政手続き

## ここ数年の変化は大きく、以前は窓口に並んだ多くの手続きが、今はスマホで数分で完了します。

医療保険電子証憑

- **：Alipay または WeChat で「医療保険電子証憑」を検索し、有効化すれば、病院受診や薬局購入時にスマホをかざすだけで済み、物理の医療保険カードを持参する必要がありません。初回は顔認証が必要で、以降は使用時に動的コードが生成されます。**社会保険照会
- **：「掌上 12333」App をダウンロードするか、Alipay の「市民センター」で、社会保険納付記録、年金口座残高、失業給付申請などを照会できます。**所得税
- **：毎年 3 月から 6 月の所得税年度調整は、「個人所得税」App で完結。控除対象の特別追加控除（子女教育、住宅ローン利子、老人扶養など）があるか確認しましょう。これらはすべて納税額を減らしたり還付を受けたりするのに役立ちます。**これらを覚えて、詐欺を防ぎ安心を

**警察、銀行、カスタマーサポートは「安全口座」への送金を指示しません。**

- 誰かに支払いコードのスクショを送らせる人は、すべて詐欺師です。
- 見知らぬリンクをタップしない、特に「ETC 期限切れ」「社会保険カード停止」などの SMS。
- ネットショッピングの返金は元の経路で戻り、銀行カード番号の提供は不要です。
- 今すぐやる：決済 App にセキュリティチェックを実施

**Alipay と WeChat を開き、指紋/顔認証決済が有効か確認。**

1.「セキュリティロック」機能を見つけてオンに。

2.「暗証不要決済/自動引き落とし」リストを確認し、不要なサービスをオフに。

3.「ログインデバイス」リストを確認し、知らないデバイスを削除。

4. 医療保険電子証憑を有効化し、次回病院では物理カードを持参不要に。

5. 第十一章：撮影と写真管理

# 第十一章：撮影と画像管理

一、あなたもこれらの間違いをしていませんか?

## まず最も一般的な問題をいくつか挙げます。いくつ当てはまるか確認してください：

レンズが汚れている

- **：これが本当に最も一般的な問題。スマホを取り出す時に指紋がレンズにつき、撮影結果が霞んで、スマホが壊れたと思い込みがち。習慣づけて、撮影前に服の裾でレンズを拭きましょう。**露出を調整しない
- **：デフォルトモードでは、画面をタップするとフォーカスされますが、タップ後に上下にスワイプすると明るさを調整できます。多くの人が撮った写真が暗すぎたり明るすぎたりするのは、この操作を知らないからです。**フラッシュを常にオン
- **：スマホの LED フラッシュで撮った効果は本当に見苦しく、光が硬くて冷たく、人が幽霊のように写ります。本当に真っ暗闇でない限り、オフにしましょう。暗所では、夜景モードの方がフラッシュよりはるかに効果的です。**ピンチアウトで拡大
- **：画面上で二本指ピンチはデジタルズームで、本質的に画像を切り抜いて拡大するだけなので、画質が明確に低下します。遠くを撮りたい場合は、ファインダー上のズームボタン（0.5x/1x/2x/3x）を直接使いましょう。これらは光学ズームで画質を損ないません。：**画面上でピンチズームするとデジタルズームになり、本質的には画像を切り抜いて拡大するため画質が明らかに低下します。遠くを撮りたい場合は、ファインダー上のズームボタン（0.5x/1x/2x/3x）を直接使ってください。0.5x（超広角）と 1x（メインカメラ）はネイティブ光学レンズで画質劣化がありません。2x/3x は「光学品質ズーム」——高画素センサーの切り抜きで実現し、画質は光学ズームに近いですが独立レンズではありません。3x を超えるズームは通常ハイブリッドズームまたはデジタルズームで、画質が明らかに低下します。

## これらは理論知識不要、一度使えば効果がわかります：

三分割法構図＋グリッド線

- ：カメラ設定で「グリッド線」（ガイド線とも）をオン。撮影時に強調したいものを 4 本の線の交点に置くと、画面がすぐに「正しく」なります。これは写真で最もシンプルかつ最も効果的なルールで、神秘性ではなく、人間の視覚習慣に合致するだけです。「リーディングライン」を探す

- ：風景や街並みを撮る時、画面内に自然な線がないか留意——道路、手すり、壁、並木——これらの線を画面の端から被写体まで伸ばします。このテクニックを使いこなすと、通りすがりの人があなたの写真を見て写真を学んだことがあると思います。立ったまま撮らない

- ：多くの人が撮影する時は常に目の高さなので、視点がワンパターン。しゃがんで低アングルから撮ったり、高いところから俯瞰撮影してみたりすると、同じ場所でも全く違う印象に。子供やペットを撮る時は特にしゃがむ必要があり、目線の高さで撮ると自然な感じになります。光が第一要素

- ：真昼の太陽の下でポートレートを撮ると、顔の影が硬くて醜くなります。最も良い光は早朝と夕方の「ゴールデンアワー」で、太陽が斜めで色が暖色寄り、何を撮っても美しくなります。曇りの日の光は地味ですが、均一で柔らかく、ポートレートは真昼の太陽より撮りやすいです。夜景モードの使い方

- ：夜景モードをオンにすると、スマホが自動的に複数枚連写して合成します。この時に最も重要なのは——絶対に手ブレしないこと。壁に寄りかかれるなら寄りかかり、机に置けるなら置き、ダメなら息を止めましょう。夜景モードは 2□4 秒安定を保つ必要があり、ブレると画面がぼやけます。三、ポートレートモードとプロモード

## ポートレートモード

- ：背景がぼけて被写体がくっきりする効果（浅い被写界深度）を撮れます。スマホはレンズの物理サイズが小さいため、本物のボケ能力は限定的で、AI アルゴリズムで実現しています——簡単に言うと、AI が人物を切り抜き、背景をぼかします。使用時は 1□2 メートルの距離が最も効果的で、遠すぎても近すぎてもアルゴリズムがエラーを起こしやすい。またポートレート撮影時は、背景に複雑な線がないよう注意。そうでないと AI の切り抜きで髪の辺りに「失敗」が生じます。プロモード

- ：画面効果を完全にコントロールしたい場合は、プロモードを試しましょう。いくつかの重要なパラメータを知っておく必要があります：ISO が低いほど画質がクリーンですが、画面は暗くなります。シャッターが遅いほど光量が増えますが、手ブレでぼやけます。ホワイトバランスは色の暖かさ冷たさを管理し、調整が悪いと写真が青や黄色に偏ります。一般人は常時プロモードを使う必要はありませんが、これらのパラメータを理解すれば、カメラが何をしているか把握できます。四、レタッチ：やりすぎず、ちょうど良く

多くの人がレタッチについて誤解しており、レタッチすると不自然になると思っています。実はカメラで撮った写真は元々肉眼で見たものと異なり、適度な調整は当時見た画面を再現するためで、偽造ではありません。

Snapseed

- **:Google** 製の無料レタッチツール、機能はプロ級だが広告なし。私の日常的なレタッチフローは**:** まず「画像調整」で明るさとコントラストを調整「ディテール」でシャープ化「ホワイトバランス」で色かぶり補正最後に「切り抜き」で希望の構図に。**Snapseed** の「局部」機能は便利で、画面内の特定領域だけを調整でき、全画像を一括変更する必要がありません。Xingtu / Meitu
- **:** この **2** つは人物写真のレタッチに適しています。ただし原則として**:** 美顔フィルターは **30%** 以内に抑えるべきで、それを超えると不自然になり、肌がプラスチックのようになります。完璧より自然な方が美しい。Lightroom モバイル版
- **:** 色にこだわるなら、これが最もプロフェッショナル。トーンカーブ調整、**HSL** 色別調整などの高度な機能があります。ただし多くの人には大げさすぎるかもしれません。レタッチワークフローの提案
- **:** 私の習慣は、まず良い原画像を撮り、帰宅後にパソコン画面や **iPad** でじっくりレタッチすること。スマホ画面は小さく、色もあまり正確ではありません。どうしてもスマホでレタッチする必要がある場合は、画面の明るさを中□上程度に調整し、省電力モードでレタッチしないように。五、写真が多すぎる？ 整理戦略

スマホのアルバムに数千枚の写真が溜まり、特定の一枚を探すのに時間がかかる――この経験は皆あるでしょう。いくつかの実用的な整理方法**:**

検索を活用

- **:**最近の **iPhone** と多くの **Android** スマホのアルバムはスマート検索に対応。「猫」「海辺」「身分証明書」「花」などのキーワードで検索すると、システムが **AI** で画像内容を認識して探し出します。日付や場所でも検索でき、例えば「昨年 **5** 月」「北京」。この機能を知る人は少ないですが、本当に使いやすい。定期的にゴミを掃除
- **:** スクショ、重複写真、ピンボケの失敗作、これらが容量を食う元凶。**iOS** アルバムには「重複項目」自動統合機能があり、**Android** はファイルマネージャーで「**screenshots**」フォルダを検索し、不要なスクショを一括削除できます。毎月 **5** 分かけて掃除するだけで数 **GB** 空きます。アルバムを作成、デフォルト並び順だけに頼らない

- ：システムデフォルトの時系列並び順は間違いではないですが、「子供の成長写真」「旅行写真」「重要書類」のような時期をまたぐコンテンツは、手動でアルバムを作って分類すべき。Apple の「共有アルバム」は家族と一緒に写真を追加でき、一人が作成すれば家族全員が閲覧できます。バックアップは最低限の要件

- ：写真は最も代替不能なデータ——ファイルは再構築できますが、チャット履歴はなくなればそれまで、でも子供の写真や老人の動画はなくなると本当に戻りません。必ずクラウドバックアップを有効に。iCloud 写真が最も手軽、Android は各社独自のクラウドサービスがあるか、Baidu ネットディスク/Alibaba クラウドドライブで自動バックアップ。原則は：少なくとも 1 つのクラウドバックアップを持ち、卵を一つのカゴに盛らないこと。六、スクショと画面録画

### スクショ

- ：電源キー＋音量上げキーの同時押し、すべてのスマホで共通の方法。多くの Android スマホは三指下スワイプスクショ、関節ダブルタップスクショにも対応。長尺スクショ（スクロールスクショ）は Android で成熟——1 枚スクショ後に「スクロール」ボタンをタップすると、スマホが自動的にページを下に送って長尺画像に結合。チャット履歴やウェブページのスクショに適しています。画面録画

- ：コントロールセンターに「画面録画」を追加し、タップで録画開始、もう一度タップで停止。録画前に長押しするとマイク音声を録画するか選択可能——チュートリアル解説ならマイクオン、操作画面だけならオフ。録画完了した動画は自動的にアルバムに保存されます。練習：スマホで撮影＋レタッチ＋整理の完全フローを一度実行

カメラ設定で「グリッド線」をオンにし、自然光のある場所で 3 枚撮影（レンズ拭き、露出調整に注意）。

1. 最も良い 1 枚を選び、Snapseed で基本調整（明るさ、コントラスト、切り抜き）。
2. この写真に専用アルバムを作成し、「私の作品」と命名。
3. クラウドバックアップが有効か確認し、写真がクラウドに 1 部保存されていることを確保。
4. 第十一章付録：プロフェッショナル映像技術知識

## 第十一章付録：プロフェッショナル映像技術知識

スマホ撮影と動画の奥義を真に理解するには、いくつかの専門的な映像技術知識を理解する必要があります。これらの知識はスマホカメラをより良く使うのに役立つだけでなく、デバイス選びの際の判断力も高めてくれます。

## 一、解像度、フレームレート、ビットレート：動画品質の三要素

**解像度は画面の鮮明度を決定し、ピクセル数で表されます。**

720P (HD)

- **：1280×720 ピクセル、エントリーレベルの鮮明度。**1080P (Full HD)
- **：1920×1080 ピクセル、現在主流の解像度。**2K (QHD)
- **：2560×1440 ピクセル、1080P より鮮明。**4K (UHD)
- **：3840×2160 ピクセル、超高画質、ディテール豊富。**8K
- **：7680×4320 ピクセル、未来の標準、現状は対応デバイスが限定的。**解像度が高いほど画面は鮮明ですが、ファイルも大きくなり、ストレージと処理能力の要求が高まります。

フレームレート（Frame Rate）：

**フレームレートは画面の滑らかさを決定し、毎秒フレーム数（fps）で表されます。**

24fps

- **：映画標準フレームレート、独特の「シネマ感」があります。**30fps
- **：テレビやネット動画で一般的、滑らかで自然。**60fps
- **：高フレームレート、スポーツシーンやゲーム録画に適し、非常に滑らか。**120fps/240fps
- **：超高フレームレート、スローモーション動画に使用可能。**フレームレートが高いほど画面は滑らかで、スローモーション効果も良くなりますが、ファイルも大きくなります。

ビットレート（Bitrate）：

**ビットレートは動画のデータ密度を決定し、毎秒データ量（Mbps）で表されます。**

ビットレートが高いほど画質が良く、ディテールが多く保持されますが、ファイルも大きくなります。

- ビットレートが低すぎると、画面にモザイクや色塊などの圧縮痕が現れます。
- プロフェッショナルな動画撮影では通常、高いビットレートが必要で、後処理の余地を多く残せます。
- 二、動画コーデック形式：H.264、H.265 と ProRes

## 二、動画コーデック形式：H.264、H.265 と ProRes

特徴

- 特徴長所
- ：後処理の調整余地が極めて大きく、露出やホワイトバランスなどのパラメータを画質劣化なしに変更可能。短所
- 欠点適用シーン
- ：ハイエンドプロフェッショナル制作。**iPhone 15 Pro シリーズは Apple ProRes** 動画撮影に対応（最大 **4K 60fps**）しますが、**ProRes RAW** 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。H.265 (HEVC)：

特徴

- 特徴長所
- ：後処理の調整余地が極めて大きく、露出やホワイトバランスなどのパラメータを画質劣化なしに変更可能。短所
- 欠点適用シーン
- ：ハイエンドプロフェッショナル制作。**iPhone 15 Pro シリーズは Apple ProRes** 動画撮影に対応（最大 **4K 60fps**）しますが、**ProRes RAW** 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。Apple ProRes：

特徴

- 特徴長所
- ：後処理の調整余地が極めて大きく、露出やホワイトバランスなどのパラメータを画質劣化なしに変更可能。短所
- 欠点適用シーン
- ：ハイエンドプロフェッショナル制作。**iPhone 15 Pro シリーズは Apple ProRes** 動画撮影に対応（最大 **4K 60fps**）しますが、**ProRes RAW** 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。ProRes RAW：

特徴

- 特徴 RAW とは
- ：未処理の生センサーデータ、最大情報量を保持。長所
- ：後処理の調整余地が極めて大きく、露出やホワイトバランスなどのパラメータを画質劣化なしに変更可能。短所

- 欠点適用シーン
- **：ハイエンドプロフェッショナル制作。iPhone 15 Pro シリーズは Apple ProRes 動画撮影に対応（最大 4K 60fps）しますが、ProRes RAW 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。**：ハイエンド専門制作向け。iPhone 15 Pro シリーズ以降は Apple ProRes 動画撮影に対応（最大 4K 60fps）、iPhone 17 Pro はさらに ProRes RAW の本体直接録画に対応しています。

## 三、HDR：ハイダイナミックレンジ技術

HDR10

- **：最も基本的な HDR 標準、互換性が良い。**HDR10+
- **：Samsung が推進する標準、ダイナミックメタデータに対応。**Dolby Vision
- **：Dolby 開発のトップ HDR 標準、画質が最も良いがライセンスが必要。**HLG
- **：放送級 HDR 標準、通常モニターとの互換性あり。**HDR 動画撮影時は、HDR 対応ディスプレイが必要で完全な効果を確認できます。

四、色空間と色深度

## 四、色空間と色深度

**sRGB**

- **：標準色空間、互換性が最も良い。**Display P3
- **：Apple が推進する広色域標準、色がより豊か。**Rec.2020
- **：HDR 動画の標準色空間、色域が最も広い。**色深度：

**8-bit**

- **：1670 万色、通常動画標準。**10-bit
- **：10.7 億色、プロフェッショナル動画標準、色のグラデーションがより滑らか。**12-bit
- **：680 億色、ハイエンドプロフェッショナル標準。**実用的なアドバイス

**日常撮影：1080P/4K + 30fps + H.265 エンコード、画質とファイルサイズのバランス。**

- スポーツ撮影：1080P/4K + 60fps、滑らかさを確保。
- プロフェッショナル創作：4K + ProRes（対応場合）、最大の後処理余地を保持。
- ストレージの提案：高ビットレート動画は容量を食うので、定期的にパソコンやク

ラウドストレージにバックアップ。

- 探索してみよう：スマホの動画設定

**カメラ App を開き、動画モードに切り替え。**

1. 解像度とフレームレートのオプションを確認し、異なる組み合わせを試す。
2. 対応していれば、HDR や ProRes などの高度なオプションがあるか確認。
3. 異なる設定で数本の動画を撮影し、画質とファイルサイズを比較。
4. 第十二章：映像エンターテインメントと読書

## 五、RAW フォーマットと HEIF/JPEG の比較

**RAW フォーマットとは：**

RAW フォーマットはカメラセンサーが記録した「生データ」——一切の圧縮や処理を経ていない純粋な原始情報です。生鮮食品のように、後処理で「調理」して完成させます。スマホはストレージと処理能力が限られているため、デフォルトで撮影されるのはすでに「調理済み」の JPEG または HEIF で、RAW は特地に有効にする必要があります。

**RAW vs HEIF vs JPEG の比較：**

- **JPEG：**最も古いフォーマットで、互換性が最も高い（どのデバイスでも開ける）。8-bit 色深度、非可逆圧縮、ファイルサイズが最小（1 枚 2-5MB）。後処理の余地が限られており、無理に暗部を明るくすると色ノイズが出やすいです。
- **HEIF/HEIC：** Apple が推進する新フォーマットで、同じ画質でファイルサイズは JPEG の半分（1 枚 1-3MB）。10-bit 色深度と透明チャネルに対応。欠点は一部の旧デバイスや Windows ではプラグインのインストールが必要なことです。
- **RAW：**非圧縮の生データで、ファイルサイズが最大（1 枚 20-100MB）。色深度は通常 12-16bit で、後処理の余地が最大。欠点は専門ソフト（Lightroom、Snapseed など）でないと開いて処理できず、直接共有できないことです。

**なぜプロカメラマンは RAW で撮るのか：**

- **ホワイトバランスを後処理で無劣化調整できる：** JPEG でホワイトバランスを間違えると（例：白熱灯下で黄色く偏る）、後処理で補正すると画質が劣化します。RAW は色温度を再設定でき、完全に無劣化です。
- **露出許容範囲が広い：** JPEG で 2 段アンダー露出の暗部はほぼ真っ黒で、明るくするとノイズだらけです。RAW は 3-4 段アンダーの暗部から「救える」使用可能なディテールを取り出せます。過曝も同様です。

- **暗部とハイライトのディテールがより豊か：**RAW のダイナミックレンジは JPEG よりはるかに大きく、明部が飛ばず、暗部が潰れない余裕がずっと大きいです。
- **シャープ化、ノイズ除去、レンズ補正がすべて後処理で制御可能：**カメラの本体処理は「一発勝負」ですが、RAW は何度でも調整でき、毎回生データから出発できます。

**スマホで RAW を有効にする方法：**

- **iPhone ProRAW：**iPhone 12 Pro 以上の Pro 機種のみ対応。「設定カメラフォーマット」で「Apple ProRAW」をオンにします。カメラ画面上部に「RAW」ボタンが表示されるのでタップして有効化。ファイルサイズは約 25MB/枚、必要な時のみオンにすることをお勧めします。
- **Android Motion Raw / Ultra RAW：**ブランドによって名称が異なります。Xiaomi/OPPO/vivo のフラッグシップ機はカメラの「プロモード」または「その他」で RAW スイッチが見つかります。Huawei は「プロモード」で RAW を有効化。Samsung Galaxy S シリーズは「プロ」モードに RAW オプションがあります。ファイルサイズは通常 20-50MB。
- **一般的なアドバイス：**日常のスナップ撮影で RAW を有効にする必要はありません。ファイルが大きく容量を占めます。重要なシーン（風景、ポートレート、後処理が必要な場合）のみ有効にしましょう。

## 六、コンピュテーショナルフォトグラフィの原理

現代のスマホ撮影は「コンピュテーショナルフォトグラフィ」なしでは成り立ちません——つまり、アルゴリズムによってレンズやセンサーだけでなく写真を生成します。これらの核心原理を理解すると、スマホがシャッターを切った瞬間に何をしているかがよりわかります。

**マルチフレーム合成：**

スマホでシャッターを一度切ると、実は複数枚の写真を撮っています。アルゴリズムが複数フレームを整列させ、加重融合して高品質な一枚を作ります。

- **ナイトモード：**1-3 秒で複数枚のアンダー露出写真を連写し、整列後に重ね合わせ——ノイズが互いに相殺され、明るさが累積します。結果として暗所でもクリーンな画像が撮れます。
- **HDR（ハイダイナミックレンジ）：**アンダー、正常、オーバーの 3 枚（または複数枚）を撮影し、明部と暗部の両方にディテールがある 1 枚に合成します。逆光シーンで特に効果的です。
- **Deep Fusion（Apple）/Photonic Engine（光子エンジン）：**中低光シーン向けに、ピクセル単位で複数フレームを分析・融合し、テクスチャとディテールを重点的に向上させます。セーターや髪の毛などのディテールが明らかに鮮明になります。

**セマンティックセグメンテーション：**

AI は単に画像全体を一律に処理するのではなく、まず画面に何があるかを「理解」します——空、顔、芝生、建物、衣服——そして各領域に異なる処理戦略を適用します。

- **空：**青色を個別に強調し、明度を下げて過曝を防ぎます。
- **顔：**肌色領域を認識し、個別に明るくし、肌を滑らかにします。目鼻立ちや髪のディテールはいじりません。
- **芝生/葉：**緑の彩度を強調し、ディテールのシャープネスを向上させます。
- **背景ぼかし：**ポートレートモードで、AI が被写体の輪郭を認識し、背景をぼかして被写体を鮮明にします。

**ディープラーニングの応用：**

- **AI シーン認識：**カメラが「グルメ」「ペット」「夜景」「書類」などのシーンを自動認識し、対応するパラメータを最適化します。例えばグルメモードは暖色と彩度を強調し、料理がより魅力的に見えます。
- **AI ポートレートぼかし：**スマホレンズは物理的に一眼レフのような浅い被写界深度を実現できないため、ディープラーニングモデルで被写体の輪郭を予測し、背景をぼかします。最新アルゴリズムは髪の毛やガラスの端などのディテールも正しく処理できます。
- **AI 消去/AI 拡張：**画面内の通行人や不要物を選択すると、AI がコンテキストに基づいて隠された部分を「補完」します。これは生成 AI の映像分野での応用です。

**コンピュテーショナルフォトグラフィ vs 光学撮影：**

- **コンピュテーショナルフォトグラフィの優位性：**小型でも高画質が得られ、暗所や逆光などの過酷なシーンでも撮影でき、一般ユーザーが写真の知識がなくても「綺麗な」写真が撮れます。
- **コンピュテーショナルフォトグラフィの弱点：**画像が時に「過度処理」になる——過剰シャープ化、色の歪み、不自然な空の色。被写体が動くとマルチフレーム合成でゴーストが残る。ディテールは「計算で作られた」ものであり、真に記録されたものではない。
- **光学撮影の優位性：**大型センサー + 大口径レンズで光をリアルに捉え、画質が自然で後処理の余地が大きい。アルゴリズムの「補完」によるアーティファクトが発生しない。
- **トレンド：**現代のフラッグシップスマホは「光学 + コンピュテーショナル」の融合——大型センサーでリアルな情報を取得し、アルゴリズムで表示を最適化。両者は対立するものではありません。

## 七、レンズタイプと焦点距離

現代のフラッグシップスマホは通常 3　4 つのレンズを搭載し、それぞれが異なる焦点距離に対応します。焦点距離を理解すると、シーンに応じたレンズ選択ができます。

### 一般的なレンズと換算焦点距離：

- **超広角（13-24mm 換算）：**視野が最も広く、風景、建築、屋内空間に適しています。目の前の大シーンを撮影できますが、エッジに歪みがあり（直線が曲がる）、ポートレートでは顔が引き伸ばされます。「大シーン」に適し、「クローズアップ」には不向きです。

- **メインカメラ（24-28mm 換算）：**日常で最もよく使うレンズで、画質が最も良く、絞りが最大。視野と画質を両立し、人物も風景も撮れます。ほとんどのスマホの「1x」がメインカメラです。

- **望遠（70-200mm 換算）：**遠景やポートレートのクローズアップに使用。望遠は空間の圧縮効果があり（遠くの山と近くの人がより近く見える）、ポートレートではより自然な目鼻立ちの比率と背景ぼかしが得られます。

- **ペリスコープ式望遠（200mm+ 換算）：**反射式光路設計により、スマホの厚み内でより長い焦点距離を実現。遠距離でもディテールをクリアに撮影でき、コンサート、スポーツイベント、野生動物に適しています。フラッグシップ機のみ搭載。

### センサーサイズと画素サイズのトレードオフ：

センサーは「ベース」であり、受け取れる光量を決定します。業界では「ベースが一段大きいと圧倒的」と言われます——同じ画素数なら、センサーが大きいほど 1 画素あたりのサイズも大きく、集光量が多く、画質が良い。

- **大型センサー：**集光量が多く、夜景の画質が良く、ダイナミックレンジが広い。欠点はスマホが厚くなり（カメラモジュールが突出）、コストが高いこと。

- **高画素・小型センサー：**昼間の光が十分な時は、高画素でより鮮明な写真が撮れ、後処理の切り抜きに便利。ただし暗所では 1 画素あたりの集光が不足し、ノイズが多くなる。

- **主流フラッグシップの選択**: メインカメラは 1/1.3 インチから 1 インチ（一部機種は 1 インチ大型センサーを採用）、画素数は約 5000 万——これが画質と体積のバランスポイントです。

### 画素融合技術（Pixel Binning）：

昼間の高画素と夜間の大画素を両立するため、スマホメーカーは「画素融合」を使用——複数の小画素を 1 つの大画素に統合して出力します。

- **4in1 画素（Quad Bayer）**：5000 万画素センサーで 1250 万画素写真を出力。各出力画素は 4 つの隣接画素から合成され、集光量が 4 倍向上し、画質がよりクリーンに。
- **9in1 画素（Nonacell）**：1 億画素以上のセンサーに使用し、約 1100-1200 万画素を出力。同じ原理で、集光量がさらに大きい。
- **実際の効果**：デフォルト撮影モードでは、融合後の低画素写真が得られます——画質が良く、ファイルが小さい。究極のディテールが必要な時（書類撮影や遠景など）は、手動で「高画素モード」に切り替えて全解像度出力できます。

## 八、特殊撮影モード

日常撮影以外に、スマホにはいくつか特殊モードがあり、使いこなせば普段は撮れない画面が撮れます。

**パノラマモード：**

- 撮影時に案内矢印に従ってスマホをゆっくり水平に移動してください。速すぎると——継ぎ目がずれます。
- スマホの垂直方向を安定させ、水平にのみ移動してください。上下に揺れると水平線が波状になります。
- 動く人や車がある場合は注意——合成時に「体が半分」や「ゴースト」が現れます。
- パノラマモードは縦撮影も可能——縦に撮って横に合成し、高い建物や滝に適しています。

**スローモーションとタイムラプス：**

- **スローモーション**：120fps または 240fps の高フレームレートで撮影し、再生時にスロー再生。水滴、スポーツの瞬間、ペットのジャンプに適しています。光が十分必要で、スローモーションモードは露出時間が短く、暗所では画質が低下します。
- **タイムラプス**：間隔撮影で複数枚の写真を合成し早送り動画を作成。車の流れ、雲の動き、日落、花の開花に適しています。撮影時間は数分から数時間で、必ず三脚でスマホを固定してください。手持ちでは確実にブレます。
- **テクニック**：タイムラプスで雲を撮る場合、スマホを固定して 15-30 分撮影すると、最終的に約 10 秒のタイムラプス動画が得られ、インパクトがあります。

**星空・天文モード：**

- 都市の光害から遠ざかり、街灯の干渉がない郊外や山間部を見つけてください。都市では天の川は撮れません。

- 必ず三脚を使用し、手持ちは不可。スマホを数秒から数十秒しっかり固定する必要があります。
- 天気は晴れて雲がないこと、月が大きすぎないこと（満月は星光を隠します）。月初めや月末の暗い夜が最適です。
- iPhone の「ナイトモード」は星空を自動認識して露出時間を延長（最大 30 秒）。Android フラッグシップの「星空モード」や「天文モード」も同様の効果。辛抱強く待ち、シャッターを切った後はスマホを動かさないでください。

**マクロモード：**

- 花、昆虫、宝石、文字のクローズアップなど小さな被写体の撮影に使用。1-2cm の距離まで寄ってピントが合わせられます。
- 一部のスマホには専用マクロレンズ（多くは 200 万画素または 500 万画素の小型センサー）があり、画質は一般的。フラッグシップ機は超広角レンズでマクロを兼用し、画質がより良い。
- 寄って撮影する際はレンズが被写体に非常に近く、光を遮るため、必要に応じてデスクライトや懐中電灯で補光してください。
- ピント距離が極めて近く、被写界深度が非常に浅いため、被写体が前後に数ミリ動くだけでピントがずれる可能性があります。撮影時は手を安定させ、被写体を動かさないのがベスト。

**静止画撮影の実用アドバイス**

- 日常撮影：HEIF/JPEG のデフォルトで十分。ファイルが小さく、画質も実用十分。
- 重要なシーン（風景、ポートレート、後処理が必要）：RAW を有効にし、最大の後処理余地を確保。
- レンズ選び：風景は超広角、ポートレートは望遠、日常はメインカメラ。
- 夜景と星空：必ずスマホを固定し、ナイトモードまたは天文モードを有効化。
- 大型センサー搭載スマホ（1 インチセンサー機など）は画質の上限が高いですが、操作習慣がやはり鍵。

**試してみよう：あなたのカメラの能力を把握する**

1. スマホが RAW 撮影に対応しているか確認（設定カメラフォーマット）。有効にして 1 枚撮影し、同じシーンの JPEG と RAW のファイルサイズを比較。
2. 超広角、メイン、望遠の 3 つの焦点距離に切り替え、それぞれ同じ遠景を撮影して視角と画質の違いを体験。
3. ナイトモードまたは星空モードがあるシーンを見つけ、スマホを固定して夜景写真を 1 枚撮影。

4. パノラマモードで横方向に大シーンを撮影してみる。スマホを安定して移動させることに注意。

# 第十二章：音声・映像エンターテインメントと読書

一、音楽鑑賞：どの *App* を選ぶか、どう聴くと快適か

## 音楽 App は現在機能がほぼ同じですが、それぞれ強みがあり、選び方次第でかなり快適になります：

網易雲音楽

- ：強みはプレイリストコミュニティとコメント欄。「雨の日に聴く曲」や「勉強疲れ時にリラックスできるインスト」を検索すれば、他人が整理したプレイリストがすぐ見つかります。コメント欄が曲より面白いことも。欠点は一部の楽曲の著作権が不完全で、聴いている途中でグレーになる曲があること。QQ 音楽
- ：著作権が最も充実、ジェイ・チョウ、五月天などの人気アーティストの楽曲は基本的に **QQ** 音楽にしかありません。音質オプションも豊富で、会員はハイレゾ音質を利用可能。「聴きたい曲がすべて検索できる」ことを重視するなら、**QQ** 音楽が最も確実。Apple Music
- ：**Apple** ユーザーのファーストチョイス。インターフェースがクリーン、広告なし、音質が良く、ロスレスや **Dolby Atmos** に対応。ただしレコメンドアルゴリズムは国内 **App** ほど中国語ユーザーの好みを理解せず、プレイリスト文化も網易雲ほど発展していません。シンプルな体験を好み、ソーシャル機能を気にしない人に適しています。Spotify
- ：世界的にユーザーが最も多く、レコメンドアルゴリズムは公認トップ。「毎週の発見」と「デイリーミックス」の精度が驚異的で、長く使うほどユーザーを理解。欠点は無料版の広告が多く、一部の中国語曲の著作権カバーが不完全。利用には特殊なネットワークが必要。聴曲の小技

：**Bluetooth** イヤホンを使っている場合、音楽 **App** の設定で「音響調整」や「イコライザー」があるか確認し、低音と高音を少し調整すると効果が明確に向上。また、ストリーミング聴曲は通信量を消費するので、設定で「**Wi-Fi** 時のみロスレス再生」をオンにするか、

お気に入りのプレイリストを事前ダウンロードしてオフラインで聴きましょう。二、ポッドキャスト（Podcast）：通勤路上の「耳の伴侶」

ポッドキャストは、聴いたことがない人は自分と無関係だと思い、一度聴き始めると戻れなくなります。本質は「ラジオ番組の Netflix 版」——科技を語る人、歴史を語る人、物語を読む人、どんなジャンルでも揃っています。

小宇宙アプリ

- ：国内で最も良くできたポッドキャスト App、インターフェースが清潔で余計な機能なし。「毎日のおすすめ」と「編集者厳選」が新番組の発見を助け、コメント欄も活発。私が最もよく聴くのは《忽左忽右》（文化歴史系）と《不合時宜》（社会問題系）で、毎回賢い人たちの会話を傍聴しているようです。Apple ポッドキャスト
- ：iPhone に標準搭載、追加 App 不要。CarPlay との統合が最も良く、運転中に便利。聴取のコツ
- ：ほとんどのポッドキャスト App は 1.2 倍速や 1.5 倍速再生に対応、慣れると時間を大幅に節約。私は中国語は 1.2 倍、英語は 1.5 倍で聴いています。また「新エピソードの自動ダウンロード」と「再生完了後の自動削除」を必ずオンに。そうしないとスマホに聴き終わった音声ファイルが溜まっていきます。三、動画視聴：プレーヤーからストリーミングまでの正しい開き方

多くの人が知りませんが、スマホで再生できる形式はストリーミング App が提供するものよりはるかに多い。良いローカルプレーヤーで多くの問題が解決します。

VLC プレーヤー

- ：オープンソース無料、ほぼあらゆる形式を再生——MKV、AVI、FLAC、さらには ISO ディスクイメージも。パソコンでは再生できるがスマホでは再生できない動画をダウンロードした場合、十中八九フォーマット問題で、VLC を入れれば解決。外付け字幕にも対応、.srt 字幕ファイルを動画と同じフォルダに置けば VLC が自動的に読み込みます。nPlayer
- ：より洗練された体験を求めるなら、この有料 App は買い。インターフェースが美しく、ジェスチャー操作が快適——左側上下スワイプで明るさ調整、右側上下スワイプで音量調整、左右スワイプで早送り巻き戻し。DTS や Dolby 音響にも対応、映画視聴時の臨場感が大幅に向上。Bilibili

- **：**もはや二次元サイトだけでなく、学習リソースが異常なほど豊富。大学入試、プログラミング、撮影、料理、あらゆるチュートリアルが揃います。弾幕文化は **Bilibili** の魂で、動画視聴時に弾幕をオンにすると、一緒に見ている人がいる感覚。ただし学習動画視聴時は集中力を削がれるので弾幕オフ推奨。テレビにキャスト

- **：**スマホ画面は大きくてもテレビには敵いません。同じ **Wi-Fi** 環境で、動画 **App** 右上の「**TV**」キャストボタンをタップすれば、テレビに画面を投影可能。ほとんどのスマートテレビやプロジェクターが対応、ドラマや映画の視聴体験が大幅に向上。**iPhone** ユーザーは **AirPlay** で画面全体をミラーリング可能。四、電子書籍読書：スマホでもしっかり読める

スマホでの読書は目に悪い、集中力を削ぐと言われますが、確かに一理あります。しかしスマホ読書の利点も明確——いつでもどこでも、細切れ時間を最大化活用。私は地下鉄、行列、寝る前にスマホで多くの本を読みました。

WeChat 読書

- **：**現在国内で体験が最も良い読書 **App**。書庫が充実、出版書もネット小説も揃います。「読書時間でコイン交換」モデルで、多く読めば無料で本と交換でき、学生に優しい。「アンダーライン」機能で他人がどの段落にマークしたか見え、他人の注釈した良い文に共感することも。ただし「読書ランキング」は少し競争が激しく、「時間稼ぎのために読む」になりがちなので、オフ推奨。Kindle App

- **：Kindle** デバイスを持っていれば、スマホ版 **Kindle App** で読書進捗を同期。**Kindle** の正規電子書籍はレイアウト品質が海賊版 **PDF** よりはるかに良いですが、**Kindle** 中国ストアは閉鎖され、海外リージョンへの切り替えが必要。Apple Books（ブックス）

- **：iPhone** 標準、**EPUB** と **PDF** の読書に適しています。インターフェースが極めてシンプル、ページめくりアニメーションが快適。ネットから電子書籍ファイルをダウンロードしたら、「ブックスに共有」でインポート可能。読書設定の提案

- **：**どの **App** でも、背景を暖黄色調（アイケアモード）にし、フォントを少し大きくし、行間を広めに。夜の読書では「ダークモード」をオン——黒背景白文字で目が楽。絶対に白背景黒文字小フォントを暗所で長時間読まないで、目の疲れは蓄積します。五、スマホゲーム：楽しく、スマホにも優しく

スマホゲームへの私の態度は：遊んで良いが、逆にコントロールされないように。ゲームツールを合理的に使えば、より快適に遊べ、本業も疎かになりません。

ゲームモード/ゲームスペース

- ：最近の主流スマホにはこの機能があります。オンにするとスマホが自動的にバックグラウンドクリア、通知ブロック、誤操作防止——団体戦中に突然電話が鳴るような事態をゲームモードが防いでくれます。一部のスマホはパフォーマンスモード/E-sportsモードにも対応、プロセッサを強制フル稼働させますが、代償は発熱と消費電力の増大。画質とフレームレートの調整

- ：無闇に最大にしない。まずフレームレート（滑らかさ）を確保してから画質を追求。60fps は 30fps よりはるかに快適ですが、120fps はスマホに負荷が大きく、ミドルレンジ機では無理する必要なし。競技系ゲーム（王者、PUBG など）では、フレームレートの優先度は画質よりはるかに高い——滑らかな操作感は綺麗な芝生より百倍重要。ゲーム中のスマホ発熱対策

- ：これは正常ですが、長期の高温はバッテリー寿命を損ないます。いくつかの冷却策：スマホケースを外して放熱、画面輝度を 70% 程度に下げる（輝度は高発熱源）、充電しながらゲームをしない（発熱が重なり、バッテリーが急速に劣化）。スマホが握れないほど熱い場合は、すぐにゲームを終了して休ませましょう。依存防止ツール

- ：iOS の「スクリーンタイム」と Android の「デジタルウェルビーイング」で、ゲーム App に 1 日の時間制限を設定可能。私は平日最大 1 時間、週末 2 時間に設定。時間に達すると App アイコンがグレーアウトし、タップできなくなります。この制限は本当に効果的で、形だけのものではありません。クラウドゲーム

- ：スマホ性能が足りないが大作を遊びたい場合は、クラウドゲームプラットフォーム（Tencent START クラウドゲーム、NetEase クラウドゲームなど）を試す。ゲームはリモートサーバーで実行され、映像が動画ストリームとしてスマホに送信。ネットワーク要件は高いがスマホ性能はほぼ不要。欠点は遅延が避けられないことで、戦略系、RPG 系に適し、FPS 系には不向き。やってみよう：スマホエンターテインメントシステムを構築

音楽 App でお気に入りのアルバムやプレイリスト 3 つをダウンロードし、オフライン再生を確保。

1. 小宇宙または Apple ポッドキャストで興味あるチャンネル 3 つを登録し、1 エピソードをダウンロードして試聴。
2. VLC プレーヤーをインストールし、パソコンからコピーした動画ファイルの再生を試す。
3. WeChat 読書で読みたい本を探し、暖色背景とアイケアモードを設定、自分に合ったフォントサイズに調整。
4. ゲームをする場合、ゲームモードを開いて最適化設定を確認（迷惑防止、フレームレート優先など）。
5. 第十三章：ツール系 App のおすすめ

# 第十三章：ツール系アプリのおすすめ

この章では、効率・健康・旅行系のツールを厳選して紹介し、実用的なシステム機能やデバイス間連携も補足します。*Qingzhuo* などのクリーン / 迷惑対策、スマホ標準のブロック設定、天気と地震速報、向日葵 / *ToDesk* による遠隔操作、*Trae* などの *AI* オフィス、人工デスクトップ、*LocalSend* などの *LAN* 転送、デバイス間クリップボード、*Aegis* / *Microsoft Authenticator* の動的パスワード、*TickTick* のスケジュール管理と紙の書類スキャンなどです。

## 一、効率ツール：時間と注意力を管理する

効率ツールはデジタル生活の「骨格」です。ばらばらのアイデア、タスク、資料を整理整頓してくれます。シーン別に紹介します。

### ToDo リスト（タスク管理）

- TickTick（TickTick） 対象：仕事・生活・学習のタスクとスケジュールを一元管理したい人。本節の重点推奨のひとつ。強み：スマホ / PC / タブレット / Web で同期。リスト＋カレンダー＋ポモドーロの 3in1。「明日の午後 3 時に会議」「毎週月曜に週報提出」のような自然言語入力で日時と繰り返しルールを自動認識。サブタスク、タグ、優先度、添付、共同リストに対応。「スケジュール＋タスク消化＋日次レビュー」を一体で管理できます。
- TickTick をもっと使いこなすコツ：収集箱：まず全部「今日 / 収集箱」に放り込み、夜に仕事・家庭・学習のリストへ振り分けます。
- カレンダービュー：会議や締切をカレンダーにドラッグして予定の重なりを確認。ToDo リストだけを見るより直感的です。
- 繰り返しタスク：服薬、支払い、運動はルール化して自動繰り返しに。毎回入力する手間がなくなります。
- ポモドーロ：タスクの上でそのまま集中モードを開始。今週「本当に完了したもの」を集計できます。
- スマホ機能との連携：音声アシスタントで「リマインドして…」と言えばシステム通知に。重要な締切はシステムカレンダーにも入れて二重の安心を。

Todoist　対象：ミニマル志向のクロスプラットフォームユーザー。強み：すっきりした UI、明確なプロジェクト階層（タスクサブタスク孫タスク）、karma ポイントで習慣化を後押し。

Microsoft To Do　対象：オフィスワーカー、Outlook ユーザー。強み：完全無料・広告なし。「My Day」で毎日リセットされ、心理的負担が軽い。Microsoft エコシステムと深く連携。

システムのメモ / リマインダー　対象：サードパーティアプリを入れたくないライトユーザー。強み：ネイティブ体験で遅延ゼロ。Siri / 小愛で音声追加でき、システムウィジェットとも連動。

### ポモドーロ（集中力トレーニング）

- Forest　対象：スマホを触りすぎてしまう人。強み：集中中に木を育て、途中で離れると木が枯れる「仮想の命」でポジティブなフィードバックを形成。友だちと共同育成も可能。
- 潮汐（Tide）　対象：ホワイトノイズで集中 / 入眠したい人。強み：自然音（雨、波、森）＋ポモドーロ＋瞑想の 3in1。ミニマルで癒やされる UI。
- 原則：定番は 25 分集中＋ 5 分休憩。自分の状態に合わせて 50 / 10 や 90 / 20 に調整しても OK です。

### カレンダー（時間の可視化）

- システムカレンダー　対象：全ユーザー。強み：ネイティブ通知が最も安定し、バッテリー消費も最小。Siri / 小愛で音声追加できます。
- Google Calendar　対象：複数デバイス利用者、他人と予定を共有したい人。強み：「目標」機能が運動 / 学習時間を自動で調整。Gmail と連携し、メール中のイベントを賢く認識します。

### メモ系（知識の定着）

- Notion　対象：メモ、タスク、データベース、wiki を一元管理したい人。学生、フリーランス、チーム協働に。強み：万能なナレッジベース。Block 単位のモジュール編集で、個人ページ、読書リスト、プロジェクトボードを構築可能。テンプレートも豊富です。
- Obsidian　対象：深い知識管理をする人、研究者、ライター。強み：ローカルの Markdown 保存、双方向リンク＋グラフビューでノートが「第二の脳」に。データは完全にあなたのもの。
- flomo　対象：断片的なアイデアを残したいけど長文は書きたくない人。強み：カード型メモ。朋友圏のように気軽に記録でき、# タグで分類、タイムラインで振り返れます。

- Apple メモ　対象：iPhone / Mac ユーザー、軽い記録需要。強み：起動が速く、書類スキャン / 手描き / チェックリスト / iCloud 同期に対応。無料でシステムと深く統合。
- 印象筆記（Evernote 中国版）　対象：Web クリップや資料アーカイブを大量に使う上級者。強み：Web クリップ拡張が強力。画像内の文字を OCR 認識。老舗で安定しているが、上位プランは有料です。

### PDF ツール

- PDF Expert　対象：iPad / iPhone ユーザー、契約書に批注が必要なビジネスパーソン、学生。強み：iOS で定評のある最強 PDF エディタ。注釈が滑らかで、署名、結合・分割、フォーム入力に対応。
- WPS　対象：Word / Excel / PPT / PDF をまとめて扱うオフィスワーカー。強み：オールインワンのオフィススイート。国内のクラウドストレージや WeChat ファイルとの互換性が高く、無料版でも日常用途は十分です。
- Adobe Acrobat　対象：PDF の規格準拠を重視する人（法律、デザイン、出版）。強み：公式標準。PDF/A 形式への変換が正確で、デバイス間同期と電子署名もプロ級。

### スキャン系（紙の書類をデジタル化）

紙の契約書、領収書、宿題、ホワイトボードメモ、身分証のコピー……スマホのカメラでスキャナーの大半は代替できます。基本フロー：きれいに撮る自動トリミングと歪み補正 PDF / 画像を書き出し任意で OCR。

- スキャナ全能王（CamScanner）　対象：契約書、チケット、証明書を頻繁にスキャンするビジネスパーソンと学生。強み：端の自動検出、遠近補正、OCR 文字認識。複数ページ PDF の生成とクラウド同期に対応。
- Microsoft Lens　対象：学生、ホワイトボードや書籍を Word にしたい人。強み：無料で広告なし。OneNote / Word へワンタップ書き出し。文字認識の精度が高い。
- Apple メモのスキャン　対象：iPhone ユーザー、軽いスキャン需要。強み：アプリ不要。メモ内のカメラアイコンからスキャンでき、影の除去や署名注釈も自動。無料で使いやすい。
- 白描 / スキャナ全能王 Web 版 / メーカー純正カメラの「書類スキャン」：Huawei、Xiaomi、OPPO、vivo の標準カメラには「書類 / スキャン」モードが搭載されていることが多い。Android では Google カメラや標準のスキャン機能も使えます。急ぎで契約書 1 枚を撮るのに便利です。
- 撮影のコツ：光は均一に、スマホは紙面と平行に、四隅を合わせる。複数ページは順番に撮ってから PDF に結合。機密書類は透かしと保存場所に注意し、使い終えたら「最近削除した項目」から完全削除しましょう。

## 二、健康・運動

健康系アプリは、スマホの中でもっとも過小評価されている「秘書」です。睡眠、心拍、運動量を継続的に追跡し、たまに病院に行くより長期的な状態を映し出してくれます。

### 運動・フィットネス

- Keep　対象：自宅トレーニング派、完全初心者、ダイエットや体型づくりをしたい人。強み：HIIT からヨガまで充実した無料講座が豊富。AI がトレーニングプランを自動生成。コミュニティも活発です。

- Nike Training Club（NTC）　対象：プロの指導を受け、体系的にトレーニングしたい人。強み：Nike 公式。講座はすべて無料で、動作解説はプロのコーチが収録。筋群 / 器具別に整理されています。

- Apple Fitness　対象：Apple Watch ユーザー、活動リングで運動習慣を続けたい人。強み：毎日の 3 つのアクティビティリング（Move / Exercise / Stand）で目標を可視化。ウォッチの心拍データとシームレスに連携します。

### 睡眠トラッキング

- AutoSleep　対象：Apple Watch ユーザー、毎晩の深い睡眠 / 浅い睡眠の比率を知りたい人。強み：腕時計をつけて寝るだけで自動トラッキング。手動の開始・停止は不要。睡眠の質スコアと就寝アドバイスを提供します。

- Sleep Cycle　対象：時計をつけずに寝る人、スマホで睡眠を記録したい人。強み：マイクで寝返りやいびきを解析し、浅い睡眠のタイミングでスマート起床。つらい目覚めを防ぎます。

- 小米運動健康　対象：Xiaomi スマートバンド / ウォッチユーザー。強み：無料で Xiaomi バンドと深く連携。心拍・血中酸素・睡眠を一体でトラッキングでき、コスパが高い。

### 瞑想・リラックス

- Calm　対象：ストレスの多い社会人、入眠困難な人。強み：「毎日の平静」瞑想ガイド＋睡眠ストーリー（有名人の朗読）。美しい UI で、英語コンテンツが豊富です。

- 潮汐（Tide）　対象：ホワイトノイズで集中 / 入眠したい中国語ユーザー。強み：国産の高品質アプリ。自然音＋瞑想講座＋ポモドーロ。中国語ガイドが親しみやすい。

- 小睡眠　対象：入眠音や、寝言・いびきの記録が必要な人。強み：豊富なホワイトノイズを重ねて組み合わせ可能（雨＋扇風機＋焚き火）。就寝前にタイマー OFF。寝言の録音は面白くて実用的です。

### 女性の健康

- Flo　対象：生理記録、妊娠準備、女性の健康を気にする人。強み：AI が次の生理 / 排卵日を高精度で予測。コミュニティで匿名質問可能。洗練された UI です。
- 美柚　対象：国内の女性ユーザー、中国語のコミュニティや解説を読みたい人。強み：ローカライズされた内容が充実。生理 / 妊娠準備 / 妊娠中 / 育児の段階ごとに役立つ知識を提供します。

### 服薬リマインダー

- 用薬助手　対象：長期服薬の慢性疾患患者、家族の服薬を支える子ども。強み：バーコードで薬を識別し、服薬リストを自動作成。時間になるとポップアップ通知。服薬記録は家族と共有できます。

### データハブ

- Apple Health / Google Fit　対象：すべてのスマートフォンユーザー。強み：スマホ、バンド、サードパーティ健康アプリのデータ（歩数、心拍、睡眠、生理など）を集約する、統一された「健康カルテセンター」です。

## 三、旅行・おでかけ

出先では、使いやすいアプリがあるだけで無数の失敗を防げます。旅行の流れに沿って紹介します。

### 航空券・高速鉄道

- 航旅縦横（UMetrip）　対象：よく飛行機を使うビジネスパーソン。強み：公式の民間航空データ。フライト状況 / 搭乗口変更 / 遅延予報が最も正確。機材種別や座席表も確認できます。
- 12306　対象：国内で鉄道を使うすべての人。強み：公式唯一の信頼できる購入チャネル。待機購入（キャンセルが出たら自動で確保）に対応。学生割引や団体券の手続きも簡単です。
- 飛常準（VariFlight）　対象：定時性に敏感な人、送迎が必要な人。強み：遅延予測が正確。旅程を家族と共有して、現在地と到着予定をリアルタイムで確認できます。

### 地図・ナビ

- 高徳地図（Amap）　対象：国内都市のユーザー、自家用車 / タクシー派。強み：リアルタイム交通情報が正確で、渋滞回避力が高い。タクシー配車や相乗りもワンストップ。公共交通の乗り換え案内も充実。

- 百度地図（Baidu Maps）　対象：スポット情報や屋内ナビが必要な人。強み：パノラマ地図 / AR ナビ。観光地や商業施設の屋内地図が詳細。音声パックも豊富です。
- Google Maps　対象：海外旅行者。強み：世界最大級のカバレッジと現地のレビュー。ストリートビューとオフライン地図は、異国の地での救世主です。

### 翻訳

- Google 翻訳　対象：海外旅行者、語学学習者。強み：「会話」のリアルタイム双方向翻訳、「カメラ」の即時単語翻訳は旅行の必需品。オフライン言語パックのダウンロードにも対応。

### 両替・為替換算

- 極簡匯率（Minimal Exchange）　対象：海外旅行、越境 EC、海外ショッピングをする人。強み：極限まですっきりした UI。160 以上の通貨に対応し、為替はリアルタイム更新。複数通貨の同時比較、数字の長押しでクイック換算も可能です。

### 天気

- システム天気 / 中国天気網　対象：全員。予報と公式気象警報を確認する第一候補。強み：メーカーの天気アプリと「中国天気」などの公式チャネルが気象台の警報と連携。豪雨、台風、猛暑などの警報がバナー / 通知で届き、信頼性が高い。
- 彩雲天気（Caiyun Weather）　対象：分単位の降雨予報が必要な人。強み：「今後 2 時間に降るか」の精度は業界トップクラス。降雨の動的地図が直感的で、雨が降る前に通知してくれます。
- Carrot Weather　対象：ユーモアと豊富なウィジェットを求める天気ファン。強み：毒舌 AI が天気を読み上げ。カスタマイズ可能なウィジェットが大量。レーダーもプロ級。有料版はさらに多機能です。
- 警報のポイント：天気アプリは通知を許可しないと警報を受け取れません。警報は青・黄・橙・赤の 4 段階で、赤は災害級の天候リスクが最も高いことを意味します。表示に従って行動を調整しましょう。有効化方法の詳細は、本章第 7 節「天気警報 / 自然災害警報 / 地震速報」をご覧ください。

### 旅行ガイド

- 馬蜂窩（Mafengwo）　対象：詳細な旅行計画を立てる人、ドライブ旅行 / じっくり旅派。強み：ユーザーのオリジナル紀行文が充実。目的地の遊び方 / ルート / 失敗回避ガイドが詳しく、オフラインガイドもダウンロードできます。
- 小紅書（RED）　対象：リアルな体験談や、穴場スポット / グルメを探したい若い旅行者。強み：画像・短文・動画が直感的。「 都市 3 日間モデルコース」などの情報が新しく、鮮度が高い。検索体験も優秀です。

## 四、自動化ツール入門

自動化は「一度の設定で長く楽になる」レバーです。毎日繰り返している何十回もの操作をゼロにできます。この節では概念の入り口だけを扱い、具体的な使い方は第二十三章「プログラミング的思考と自動化入門」で詳しく解説します。

- iOS「ショートカット」：Apple 公式の自動化ツール。プログラミング不要で、「帰宅したら Wi-Fi をオン＋輝度を下げる＋天気を読み上げ」などの複合アクションをワンタップで実行できます。
- Android の自動化：システムレベルのシーン連動（Xiaomi「自動タスク」、Huawei「情景インテリジェンス」など）が最も簡単。Tasker は多機能ですが学習曲線が急なので、上級者向けです。
- 基本の考え方：まず「毎日繰り返している操作」を特定し、「1 つのトリガー条件＋複数のアクションで代替できるか」を考えます。詳しい例と応用は第二十三章をご覧ください。

## 五、ブラウザと検索エンジン：情報への入り口

ブラウザはインターネットへの窓、検索エンジンは情報の海を進むコンパスです。使いこなせば、遠回りがぐっと減ります。

### ブラウザの仕組み：

ブラウザに URL を入力すると、(1)DNS サーバーへドメインの IP アドレスを照会、(2) その IP のサーバーに HTTP/HTTPS リクエストを送信、(3) サーバーが Web ページのコード（HTML、CSS、JavaScript）を返す、(4) ブラウザがコードを解析して画面に描画します。

### 主要ブラウザのおすすめ：

- Microsoft Edge：Chromium ベースで高速・高互換。Copilot AI アシスタントを内蔵し、ページ要約、翻訳、会話が可能。デバイス間同期に対応し、メインブラウザとして強くおすすめします。
- Safari：Apple デバイスの標準ブラウザ。システムと深く統合され、省電力で効率的。プライバシー保護も良好ですが、拡張機能は少なめです。
- Chrome：世界で最も普及率が高く、拡張も豊富。ただしメモリ消費が大きく、スマホではバッテリーをやや使いやすい。

### 検索エンジンの選び方：

- Bing：Microsoft の検索エンジンで、Edge と深く連携。AI 要約が答えを直接提示し、検索効率が高い。画像検索の体験も良好です。

- Yandex：ロシア最大の検索エンジン。画像検索と逆画像検索に強く、類似画像の検索や物体の識別に便利です。
- 百度（Baidu）：国内情報検索の主力。ただし広告が多いため、見極める力が必要です。
- Google：世界で最も精度の高い検索エンジン。ただし国内では特別なネットワークが必要です。

**効率的な情報検索ガイド：**

- AI 検索を活用：Bing の AI 要約や Edge の Copilot は質問に直接答え、大量の結果から要点を抽出してくれます。リンクを 1 つずつ開くよりはるかに効率的です。
- 偽情報と広告の見分け方：検索結果で「広告」や「スポンサー」と表示されるものは有料プロモーションで、自然検索結果ではありません。情報源の信頼性を確認し、公式サイトや著名メディアの内容を優先しましょう。
- 正しいアプリのダウンロード：検索エンジンで「アプリ名＋公式」と入力（例：「WeChat 公式」）し、公式ドメインの結果を優先します。サードパーティのダウンロードサイトからのインストールは避けてください。不正ソフトが同梱されていることがよくあります。
- 公式サイトの見分け方：Windows 11 なら microsoft.com、Chrome なら google.com。ドメインがブランドと一致しているかを見るのが、最も基本的な見分け方です。

**Web サイトのドメイン知識：**

ドメインはサイトの「住所」です。読めば、サイトの性質と出どころが分かります：

- 一般的な拡張子の意味：.com ―商業組織。最も一般的
- .cn ―中国の国別ドメイン
- .org ―非営利団体
- .net ―ネットワーク関連サービス
- .edu ―教育機関
- .gov ―政府機関
- .xyz/.tech/.info ―新興の汎用ドメイン。取得のハードルが低く、要注意

国・地域ドメイン：.jp ―日本

.kr ―韓国

.uk ―イギリス

.de ―ドイツ

.tw —台湾

.hk —香港

セキュリティのヒント：.gov や.edu のサイトは取得ハードルが高く、比較的信頼できます。.xyz や.top などの安価なドメインはフィッシングに悪用されやすいので、アクセス時は注意しましょう。

### 広告ブロック：

- AdGuard　対象：ポップアップ、インストリーム広告、アプリ内広告に悩むすべてのユーザー。強み：システムレベルの広告ブロック（ブラウザだけでなく）。アプリ内のバナー広告や起動時トラッキングもフィルタ。データ節約とページ表示の高速化にもつながり、有料版は全デバイス対応。

### プライバシー重視ブラウザ：

- Brave　対象：プライバシーを重視し、広告トラッキングされたくない日常ユーザー。強み：Chromium ベースで、初期設定のまま広告とトラッカーをブロック。Tor 非公開モードを内蔵し、Chrome より高速でメモリ消費も軽い。

- DuckDuckGo　対象：「一度検索しただけで全 Web から広告を追われたくない」ユーザー。強み：同名の検索エンジンを内蔵（検索履歴を記録しない）。ワンタップですべての閲覧痕跡を消去（「燃焼ボタン」）。プラットフォームをまたいだ統一感のある体験。

### ブラウザ比較一覧：

| ブラウザ | 主な強み | 向いている人 |
|---|---|---|
| Microsoft Edge | Chromium ベース＋ Copilot AI 内蔵。ページ要約 / 翻訳 / 会話が可能 | 効率を重視するオフィスワーカー、AI ヘビーユーザー（メイン推奨） |
| Safari | Apple エコシステムと深く統合。省電力・高効率、プライバシー保護が強い | iPhone / Mac ユーザー、バッテリーとシステム連携を重視する人 |
| Chrome | 世界最大の普及率、最も豊富な拡張エコシステム | 拡張を多用する上級者、デバイス間同期派 |
| Brave | 初期設定で広告とトラッカーをブロック、高速 | プライバシー重視、拡張なしで広告をブロックしたい人 |
| DuckDuckGo | 同名検索エンジンが履歴を記録せず、ワンタップで痕跡消去 | パーソナライズ追跡を避けたいプライバシー敏感なユーザー |

「就寝前リラックス」の自動化を作ってみよう

1. トリガー条件を作成：特定時刻（例：22:30）。
2. 実行アクションを追加：音量を下げる > おやすみモードをオン > リラックス用プレイリストを再生 > 輝度を下げる。
3. 保存してテスト。

## 六、実用システムツール：クリーンアップ、迷惑対策、セキュリティ

効率・健康系アプリ以外に、日常体験を本当に左右するのは「迷惑が少ない」「ストレージが足りる」「詐欺が入ってこない」の 3 つです。この節では定番の実用ソフトを整理したあと、スマホ標準の迷惑ブロック機能を掘り下げます。スマホを何年使っても、これらのスイッチを一度も開いたことがない人が多いのです。

### システムクリーンアップとストレージ最適化

- Qingzhuo　対象：Android ユーザー。キャッシュ、残留ファイル、空フォルダを軽く掃除したいけど、「スマホマネージャー」の詰め合わせには煩わされたくない人。強み：軽量で広告なし。アプリ残留の深層スキャン、大容量ファイルの特定、使用量分析に対応。商業の「一括加速」より控えめで、自力メンテナンス好きな人に。注意：掃除前にパスを確認し、WeChat、写真、書類フォルダを無闇に全選択しないこと。システムキャッシュはメーカー標準のクリーンアップを優先し、サードパーティは補助にとどめましょう。
- スマホ標準のストレージクリーンアップ　対象：全員、特にご高齢や設定を変えるのが苦手な人。強み：Android は [設定] > [ストレージ] > [クリーンアップ]、iOS は [設定] > [一般] > [iPhone のストレージ]。自社アプリのキャッシュ構造を最も理解しており、誤削除リスクが最低。第一候補にすべきです。
- WeChat / QQ 内蔵のストレージ管理　対象：チャットをよく使う人。強み：WeChat の [マイページ] > [設定] > [全般] > [保存スペース]。チャットやファイル種別で細かく整理でき、外部の「WeChat 掃除」系ツールより安全です。
- 原則：システム / アプリ標準の機能があればサードパーティは使わない。パスや種別で選べるなら「一括全削除」はしない。クリーンアップの目的は容量の解放であり、大事なデータまで消すことではありません。

### 迷惑対策・セキュリティ系の実用ソフト

- 国家反詐中心　対象：全員、特にご高齢、経理関係、ネット通販や知らない番号からの着信が多い人。強み：公式提供。着信警告、SMS リスク通知、疑わしい URL 検出に対応。公安の詐欺対策データベースと連携し、「三つの防衛線」の中で最も公式の裏付けがある一環です。

- メーカーのセキュリティセンター / スマホマネージャー　対象：各ブランドのユーザー。強み：Huawei スマホマネージャー、Xiaomi セキュリティセンター、OPPO スマホマネージャー、vivo スマホマネージャーなど。迷惑ブロック、ウイルス対策、権限管理、決済保護を統合。通常無料で、システム下層との連携も良い。2 つ目の商業セキュリティを入れて権限衝突を起こすのは避けましょう。

- 通信事業者の迷惑対策サービス　対象：高頻度の自動発信や AI 音声電話に悩む人。強み：China Mobile は KTFSR を 10086 へ送信して開通。China Telecom は「天翼防騒擾」をフォロー。China Unicom は「聯通手机管家」で設定。ブロックは事業者側で行われるため、スマホが鳴らなくても迷惑番号を防げます。

- AdGuard　対象：ポップアップ、インストリーム広告、アプリ内広告に悩むすべてのユーザー。強み：システムレベルの広告ブロック（ブラウザだけでなく）。アプリ内のバナー広告や起動時トラッキングもフィルタ。データ節約とページ表示の高速化にもつながり、有料版は全デバイス対応。

- 「クリーンアップ加速 / 迷惑ブロック」の詰め合わせに注意：一部のサードパーティ管理ツールは常駐し、連絡先や SMS の権限を何度も要求します。ブロック力がシステムに勝るとは限らず、電池消費とプライバシーリスクが増えるだけのことも。まず標準機能を有効にし、次に公式の反詐アプリを検討しましょう。

#### 実用ソフトの選定原則

- システム優先：ブロック、クリーンアップ、権限管理は、まずスマホ標準のスイッチをオンに。

- 公式で補完：国家反詐中心や通信事業者の迷惑対策は公式チャネルなので、重ねて使えます。

- サードパーティは控えめに：同じ機能は 1 つで十分。権限の衝突とバックグラウンドのリソース争奪を避けましょう。

- 少ないほど多い：ツールを入れるほど通知と権限が増え、「ツール自体」に邪魔される確率が上がります。

### スマホ標準の迷惑ブロック：ブランド別の有効化方法

以下にブランド別の入口を示します。名称は OS バージョンで多少変わることがあるので、設定内で「迷惑」「ブロック」「不明な着信」と検索してください。第八章「迷惑電話対策」との違い：ここはスイッチの場所が中心、あちらは番号表示と誤表示の申立てが中心です。

| プラットフォーム | 入口パス | 推奨オン項目 |
|---|---|---|
| iPhone / iOS | [設定] > [電話] / [メッセージ] | 不明な着信を消音。ブロック済み連絡先。メッセージの「不明な送信者をフィルタ」 |
| Xiaomi / Redmi | 電話アプリ > 設定 > 迷惑ブロック。またはセキュリティセンター | スマートブロック、知らない番号の一回鳴り切断、SMS キーワードフィルタ |
| Huawei / Honor | 電話 > 迷惑ブロック。スマホマネージャー > 迷惑ブロック | 着信識別、ブラックリスト、SMS スマートブロック |
| OPPO / realme / OnePlus | 電話アプリ > 迷惑ブロック。または本機 > 迷惑ブロック | 着信ブロック、詐欺電話識別、SMS ブロックルール |
| vivo / iQOO | 電話 > 迷惑電話ブロック。i マネージャー | 迷惑ブロック、詐欺電話警告、メッセージフィルタ |
| Samsung | 電話 > 設定 > 番号をブロック / 発信者表示と迷惑電話 | 不明な番号のブロック、迷惑電話識別（地域により対応が異なる） |

**3 層防御の重ね方**

日常は「システムブロック＋通信事業者の迷惑対策＋国家反詐中心」の 3 層を重ねるのがおすすめ。システムが表示とブラックリスト、事業者がネットワーク側で高頻度発信を遮断、反詐中心が公式警告を補完します。3 層すべてを最強にしなくても OK。宅配、出前、医師からの電話まで誤ブロックしないよう、連絡先と最近の通話相手は許可しましょう。

**実践：今日から迷惑ブロックをオンに**

1. 電話アプリの設定を開き、「迷惑ブロック / 不明な着信を消音」を見つけてスマートブロックをオンに。
2. 親のスマホも確認：同じパスで＋通信事業者の迷惑対策を開通（China Mobile は KTFSR を 10086 へ送信）。
3. アプリストアで「国家反詐中心」をインストールし、登録して着信警告をオンに。
4. 表示済みのテスト番号にかけて（またはブロック記録を確認して）、機能が効いているか確かめましょう。

## 七、天気警報・自然災害警報・地震速報

迷惑ブロックが防ぐのは「電話詐欺」、警報機能が防ぐのは「天災と異常気象」です。天気アプリを入れて気温だけ見ている人は多いのですが、通知や地震速報をオフのままにして

います。豪雨の赤色警報、台風、地震が来る前の数秒から数十秒——スマホは本来、貴重な避難時間を確保する道具なのです。

## 天気警報：豪雨・台風・猛暑の受け取り方

- 公式チャネル優先：中国気象局 / 中央気象台 / 各省政府気象局の警報が最も信頼できます。「中国天気」の WeChat 公式アカウントや Weibo をフォローするか、「中国天気」などの公式アプリをダウンロードしましょう。
- スマホ標準の天気アプリ：Huawei 天気、Xiaomi 天気、OPPO 天気、vivo 天気、Apple「天気」など。多くが現地気象台の警報をプッシュします。天気アプリの設定で「気象警報 / 災害性天候」の通知がオンになっているか確認してください。
- 彩雲天気などの分単位降雨予報：通勤や子どもの送迎前に「この 30 分、傘が必要か」を判断するのに便利。ただし警報は公式気象台を基準にしましょう。
- 警報色の意味（共通）：青：注意喚起。通常どおり外出し、更新を確認。
- 黄：警戒を強化。不要な屋外活動を減らす。
- 橙：重大。できるだけ屋内に留まり、危険区域から離れる。
- 赤：特に重大。休校・休業・避難の指示に従い、冠水路、土砂災害、強風区間を無理に通行しない。

主な警報の種類：豪雨、大雪、台風、猛暑、寒波、強風、雷、ひょう、道路凍結、濃霧、干ばつ、地質災害リスク。

## 自然災害警報：「雨」だけではない

- 発表主体：気象系は気象台が担当。地質災害や土砂災害などは自然資源、水利、応急管理部門が共同で発表することも。地方政府が応急放送、SMS、12379 などのチャネルで伝えることもあります。
- 複数チャネルでの受信方法：スマホのシステム通知＋天気アプリの警報。
- WeChat で現地の「応急管理局」「気象局」の公式アカウントをフォロー。
- 12379 の国家警報発信関連チャネル（SMS / サービス、地域により開通状況が異なる）。
- テレビ、応急放送、村・地域の拡声器——山間部や洪水期はスマホだけに頼らない。

屋外・外出時の注意：台風時は看板の近くや木の下を避ける。豪雨では地下駐車場、トンネル、アンダーパスに入らない。山岳旅行で「地質災害気象リスク」の警報が出たら、すぐに急斜面や谷から離れる。

救急マニュアルとの連携：災害後に連絡が取れない、スマホが切れた、位置情報で救助を求めたい場合は、第三十二章「地震・災害シナリオ」と第七章「衛星通信・圏外通信」をご覧ください。

## 地震速報:「地震予知」とは違う

地震速報（EEW）は、地震波（特に破壊力の大きい S 波）が対象地域に到達する前に、電磁波が地震波より速く伝わることと、観測所が P 波を先に捉える仕組みを利用し、影響を受ける可能性のある地域へ数秒　数十秒前に警報を送るものです。地震が起きるかどうかを予測するものではなく、「その場で身を守る、火を止める、エレベーターから出る」時間を稼ぎます。震央に近いほど予告時間は短く、間に合わないこともあります。

- 公式・業界のソリューション：中国大陸では中国地震警報網関連の機能や、成都高新減災研究所などが推進する警報サービスが一般的。一部のメーカーはスマホにプリインストールするか、ネット経由で受信できます。

- サードパーティアプリ：アプリストアで「地震速報」などの公式・機関系アプリを検索（インストール前に開発者を確認し、関係のない権限を要求する偽アプリを避ける）。

- 警報を受けたら：屋内ではすぐ近くで身を守る（机の下、耐力壁の角、頑丈な家具のそば）。頭と首を守り、窓、外壁、高い家具から離れる。階段や窓から飛び降りない。エレベーター中は最寄りの階ですぐ降りる。屋外では開けた場所へ移動し、電線、看板、危険建築物から離れる。震後はガスと電気を確認し、余震に注意。

- おやすみモードで遮断しない：地震速報は「消音 / おやすみモードを突破」できるようにしておく。寝る前は手の届く場所にスマホを置き、着信音量を 0 にしない。

## スマホ標準の地震速報：ブランド別の有効化方法

以下の入口は OS バージョンで多少変わることがあります。設定内で「地震」「警報」「安全警報」と検索してください。iOS 自体の国内地震速報対応は限定的なため、信頼できるサードパーティの警報アプリをインストールし、位置情報と通知の権限を許可したままにしておく必要があります。

| プラットフォーム | 主な入口 | 説明 |
| --- | --- | --- |
| Xiaomi / Redmi | スマホマネージャー / セキュリティセンター > 地震速報。または設定で「地震」を検索 | 一部バージョンはシステム内蔵の警報サービス。ネット接続と位置情報の有効化が必要 |
| Huawei / Honor | 設定 > セキュリティ > 緊急警報通知 / 地震速報 | 名称は「緊急警報通知」の場合あり。地震などを含む |
| OPPO / realme / OnePlus | 設定 > セキュリティ（SOS 緊急連絡）> 地震速報 | 機種と地域サービスのカバレッジによる |
| vivo / iQOO | 天気アプリ、または設定 > セキュリティ / その他 > 地震速報 | 実際の OS バージョンのメニューに従うこと |
| iPhone / iOS | システム設定に完全な内蔵スイッチがないことが多い。信頼できる地震速報アプリをインストールして通知を許可 | アプリにはバックグラウンド実行と位置情報の権限が必要。おやすみモードで警報を遮断しない |
| Samsung など | 設定 > セキュリティと緊急事態 / ワイヤレス緊急警報 | 「緊急警報」のカバレッジは地域と通信事業者で異なる |

**地震速報に関するよくある誤解**

- 速報 ≠ 予知：「明日地震があるか」を教えてくれるものではなく、地震が発生し、波がまだ届いていないときにだけ警報します。
- どこでも何十秒も前もってくれるわけではない：震央付近では数秒しかなく、間に合わないことも。主に周辺地域が恩恵を受けます。
- 通知を切れば入れたのと同じ：どんなに優れた警報でも、消音、バックグラウンド停止、位置情報拒否では受信できません。
- 偽アプリ注意：連絡先や SMS の権限を求める「地震予知」ソフトに注意。システム内蔵か正規機関のソリューションを優先しましょう。

**警報機能の組み合わせ方**

日常のおすすめ：「システムの天気警報通知＋公式の気象 / 応急チャネル＋地震速報（システムまたは正規アプリ）」。迷惑対策と同様、同種のソフトを山ほど入れる必要はありません。大事なのは通知権限、位置情報の許可、おやすみモードの突破。ご高齢のスマホで迷惑ブロックを設定したら、ついでに天気警報と地震速報も確認しましょう。

**実践：10 分で警報設定をチェック**

1. スマホ標準の天気アプリを開き、設定で「気象警報 / 災害性天候」の通知をオンに。
2. システム設定で「地震」または「警報」を検索し、地震速報 / 緊急警報を見つけて

オンに（Android）。iPhone は正規の地震速報アプリを入れて通知を許可。

3. 現地の気象局または応急管理局の公式アカウントをフォローし、プッシュを許可。
4. 警報音がおやすみモードで完全に消音されないか確認。親のスマホも同じようにチェックしましょう。

## 八、デバイス間連携：遠隔操作、高速転送、アカウントセキュリティ

スマホが使いやすいのは、パソコンやタブレットと「チーム」を組んでいるからです。この節では、スマホからのパソコン遠隔操作、AI オフィスとリモート協働、ライブ壁紙、LAN 高速転送、デバイス間クリップボード、動的パスワードの 2FA、紙のスキャンとスケジュール管理の連携を補足します。より体系的なマルチデバイス連携は第十七章をご覧ください。

### スマホからパソコンを遠隔操作：向日葵 / ToDesk

- 向日葵遠隔控制（Sunlogin） 対象：外出先から自宅や会社の Windows / Mac を操作したい人。ご高齢のスマホを遠隔でサポートするのにもよく使われます（第二十七章参照）。強み：国内エコシステムが成熟。スマホアプリでパソコンデスクトップ操作、ファイル転送、電源オン / オフ（一部はハードウェア必要）。無料版で個人の軽い用途は十分。

- ToDesk 対象：接続の安定性と低遅延を求めるリモートワーク / 運用担当者。強み：デバイスコード＋パスワード接続。スマホからパソコン操作、パソコンからスマホ操作（バージョンによる）、ファイル転送、マルチモニタ切替がスムーズ。国内ノードの体験は良好。

- 基本の使い方：パソコンにクライアントをインストールしてログイン / オンライン維持スマホにアプリをインストールデバイス ID ＋接続パスワード（または同一アカウント）でセッション確立スマホのタッチパッド / ジェスチャーでデスクトップ操作。

- セキュリティのレッドライン（必ず覚える）：見知らぬ人が「カスタマーサービス / 警察・検察・裁判所 / 通信事業者」を名乗り、向日葵や ToDesk を入れさせてスマホやパソコンを遠隔操作させようとするのは、すべて拒否——典型的な詐欺手口です（第二十七章参照）。

- 自分で使うとき：二段階認証をオン、初期パスワードを変更、使わないときはログアウトまたは無人モードをオフ。会社のパソコンは社内のセキュリティ規定に従うこと。

- 公共の場でリモートワークするときは、画面やパスワードをのぞき見されていないか注意しましょう。

### AI オフィスソフト：Trae / WorkBuddy / Dumate

- Trae　対象：プログラミングと文書協働をするユーザー。中国語環境に親しみやすい。強み：ByteDance の AI IDE / プログラミングアシスタント。SOLO モードで一文からプロジェクトを生成。日常のコーディングや文書修正が効率的。パソコンで実行する場合、モバイル側は遠隔操作や公式の付随機能でタスクの確認・追跡が可能（最新の製品形態に準拠）。第四十一章「AI プログラミングツール全景」も参照。

- WorkBuddy　対象：会議メモ、メール、報告書、オフィス文書の整理を AI に任せたい社会人。強み：オフィスシーン向けの AI アシスタント。「下書き—推敲—要約」のような高頻度の文章作業に最適。一部のワークフローはスマホから開始でき、重い作業はパソコンで完結。

- Dumate　対象：複数人での協働、文書と知識管理を一体で行いたいチーム / 個人。強み：AI 協働型のオフィスツール。資料の集約と共同編集を重視。スマホでは閲覧と軽い編集、複雑なレイアウトはパソコンの大画面で。

- 「スマホからパソコンを遠隔操作」の意味：多くの AI オフィス機能はパソコン / Web で動きます。スマホから向日葵 / ToDesk でパソコンに戻るか、製品のスマホアプリ / Web を使えば、通勤中にパソコン上の AI タスクを「遠隔操作」できます。原則：重い計算と長文書はパソコンへ。進捗確認と数行の修正はスマホで。

- プライバシーの注意：契約書、身分証、顧客データをアップロードする前に、製品のプライバシー条項と企業のコンプライアンス要件を確認。会社の資料は社内指定のツールを優先しましょう。

### 人工デスクトップ：miHoYo ライブ壁紙（スマホ＋ PC）

- 人工デスクトップ　対象：miHoYo のキャラクターが好きで、無料のライブ壁紙が欲しい人。強み：miHoYo 提供の無料ライブ壁紙アプリ。スマホと PC に対応。キャラクターの Live2D / モーション演出を壁紙にでき、機種 / OS によってはロック画面やデスクトップ壁紙に設定可能。

- 省電力とパフォーマンス：ライブ壁紙は静止画より電池と GPU を消費。バッテリー残量が少ないときや古い機種では、充電中のみ使うか、フレームレートを下げる（アプリにオプションがあれば）。

- インストール元：公式サイト / 正規のアプリストアを優先し、サードパーティの「クラック版」にトロイの木馬が同梱されているのを避ける。PC 版はインストールパスと起動時の自動起動が必要かどうかを確認しましょう。

### LAN ファイル高速転送：LocalSend / FeiGe / AirDrop

- LocalSend　対象：Android / Windows / iOS / Mac / Linux を混在させ、クロスプラットフォームで、クラウドを経由せずに転送したい人。強み：オープンソース、LAN

直接接続。同じ Wi-Fi 下で写真・動画・文書を転送。ケーブル不要、メーカーのサーバーを経由せず、速度はルーター次第。

- FeiGe（IPMsg） 対象：オフィスの Windows パソコン間、昔ながらの LAN オフィス環境。強み：老舗の LAN 転送 / チャットツール。社内ネットでの大容量ファイル転送に便利。正規チャネルからダウンロードし、企業内ネットでは IT ポリシーに従うこと。
- AirDrop 対象：iPhone、iPad、Mac ユーザー。強み：Apple エコシステムでワンタップ共有。Bluetooth 探索＋ Wi-Fi 直接転送で通信量ゼロ。「連絡先のみ」に設定すれば、見知らぬ人からの迷惑を減らせます。
- その他：Xiaomi Share、Huawei 共有、vivo Share などのブランド方案は、同ブランド端末間で高速。WeChat の「ファイル転送アシスタント」は便利ですが、ファイルサイズとログイン端末に制限があります。大容量は LocalSend / AirDrop がおすすめ。
- 使い方のヒント：両方の端末を同じ Wi-Fi に接続。公共 Wi-Fi での機密ファイル転送はリスクがあるため、プライベートホットスポットを優先。転送後は受信側でファイルの完全性を確認しましょう。

### デバイス間クリップボード同期：KDE Connect / スマホ・パソコン互いに転送

- Apple「ユニバーサルクリップボード」 対象：iPhone ＋ Mac ユーザー。強み：同じ Apple ID でログインし、ハンドオフを有効にすると、スマホでコピー Mac でペースト（その逆も）。テキストと一部ファイルに対応。
- KDE Connect 対象：Android ＋ Windows / Linux / Mac。オープンソースのクリップボード同期、通知ミラー、リモート入力が欲しい人。強み：LAN ペアリング後にクリップボードを同期、スマホ通知をパソコンへ転送、スマホをスライド送り / タッチパッドとして使用。データはサードパーティアカウントを経由しません。
- ブランドの「スマホ・パソコン互いに転送」：Huawei 多屏協同 / スーパーターミナル、Xiaomi 妙享、vivo Share、OPPO 互連、Honor 互聯など。デバイス間クリップボードとファイルドラッグを備え、同ブランドなら体験が最良です。
- Windows ＋ Android の公式チャネル：「スマホ接続 / Link to Windows」（Your Phone の進化版）。パソコン上でスマホの通知や写真を確認でき、対応シーンではコピー&ペーストも可能。Samsung などはさらに深い連携があります。
- セキュリティの注意：クリップボードにはパスワードや認証コードが残ることがあります。共有パソコンでは全体同期を慎重に使い、パスワード欄で再ペーストによる漏えいを避ける。一部の入力メソッドもクリップボードを読むため、権限に注意しましょう。

### 動的パスワード 2FA：Aegis / Microsoft Authenticator

- SMS ではなく認証アプリを使う理由：SMS 認証コードは SIM スワップ攻撃のリスクがあります。TOTP の動的パスワードはローカルで時間アルゴリズムにより生成され、

携帯電波に依存せず、フィッシングや盗聴に強い（トロイの木馬とスクリーンショットには依然注意が必要）。

- Aegis Authenticator　対象：Android ユーザー。オープンソースとローカル管理を好む人。強み：オープンソースで無料。暗号化バックアップとインポート / エクスポートに対応。すっきりした UI で、指紋 / 顔認証でトークン一覧をロック解除可能。
- Microsoft Authenticator　対象：Microsoft アカウント、Office 365 / 企業アカウントを使う人。iOS / Android 対応。強み：プッシュ承認ログイン、パスワード監視、アカウントバックアップ（製品バージョンによる）。Windows / 企業 ID 基盤と深く統合。
- Google Authenticator / 2FAS など：各エコシステム内でもよく使われています。大事なのは暗号化バックアップがあって、機種変更してもトークンを失わないこと。
- 機種変更時に必ずやること：旧機種から移行するか、サービス内の「バックアップコード / リカバリーキー」を使います。バックアップコードは紙に印刷して 2 か所に保管し、同じ 1 台のスマホだけに残さないこと。
- パスワード管理ツールとの併用：一部のパスワード管理ツールでも TOTP を保存できますが、便利なぶんリスクが 1 か所に集中します。重要度の高いアカウントは「パスワード管理ツール＋専用認証アプリ」で役割を分けるのがおすすめ。詳しくは第十四章を参照。

## おすすめの組み合わせ：1 日の流れでどうつなげるか

### 実用的なワークフロー例

- 朝：TickTick で今日の予定を確認天気 / 警報をひと目チェックパソコンで Trae やドキュメントを開き、スマホは脇に置く。
- 日中：紙の資料はスキャン系アプリで PDF に変換し、LocalSend / AirDrop でパソコンへ送る。長文はパソコンの AI オフィスツールで草案づくり。
- 外出先：ToDesk / Sunlogin（向日葵）でパソコンにつないで作業を続行。重要なアカウントへのログインは Aegis / Microsoft Authenticator のワンタイムパスワードで。
- 夜：TickTick で完了したタスクにチェック。必要に応じて、デスクトップの壁紙を手動で変えて一息つきましょう。

**リモート操作とアカウント安全の心得**

- 見知らぬ人にリモート操作ソフトのインストールを求められたら電話を切り、自分で公式サポートに連絡する。
- 認証コードやワンタイムパスワードは、「配達員」「カスタマーサポート」を含め、誰にも教えない。
- 会社のパソコンのリモート操作や AI ソフトのインストールの前に、まず IT 部門

に可否を確認する。

- 公衆ネットワークでのファイル送信やリモートログインでは、とりわけ警戒を。

**実践練習：「デバイス間連携」を一度通しで試そう**

1. スマホとパソコンを同じ Wi-Fi につなぎ、LocalSend（または AirDrop）で写真を 1 枚やり取りする。
2. TickTick に「今夜 8 時：スキャン書類の整理」というタスクを作り、スマホとパソコンの両方から見えることを確認する。
3. メールや WeChat で 2FA（対応している場合）を有効にし、Aegis か Microsoft Authenticator で連携して、バックアップコードを保存する。
4. 自分のパソコンに Sunlogin（向日葵）か ToDesk を入れ、自分のスマホから接続に成功したら、ログアウトするかパスワードを強化する。流れに慣れつつ、安全な抜け道は残さないこと。

# 第十四章：パスワードマネージャー特集

一、なぜパスワードマネージャーが必要?

## 一、なぜパスワードマネージャーが必要?

パスワードマネージャーがこのジレンマを解決：マスターパスワード一つだけ覚えれば、他のすべてのパスワードを覚えてくれます。

二、パスワードマネージャーの仕組み

## 二、パスワードマネージャーの仕組み

1. **：すべてのパスワードは高強度暗号化で保存され、あなただけが復号可能。**マスターパスワード
2. **：パスワード保管庫をロック解除するためのマスターパスワードを設定。マスターパスワード自体はサーバーに保存されません。**自動入力
3. **：ウェブサイトや App のログイン画面で、パスワードマネージャーが自動的に ID とパスワードを入力。**パスワード生成
4. **：ランダムで複雑なパスワードを生成し、セキュリティを向上。**三、主流パスワードマネージャーのおすすめ

## 三、暗号化原理の概要

パスワードマネージャーの暗号化原理を理解すると、サービスプロバイダーが信頼に値するかを判断できます。以下、分かりやすい言葉でいくつかの核心概念を説明します。

### AES-256 暗号化

AES（Advanced Encryption Standard）は米国国立標準技術研究所（NIST）が認可した共通鍵暗号アルゴリズムで、銀行、軍、政府で広く使用されています。「256」は鍵長が 256 ビットであることを意味し——理論上、宇宙の年齢を超える時間がないと総当たり攻撃で破

れません。主流のパスワードマネージャー（1Password、Bitwarden、KeePass など）はすべて AES-256 でパスワード保管庫を暗号化しています。

### ゼロナレッジアーキテクチャ（Zero-Knowledge Architecture）

「ゼロナレッジ」とはサービスプロバイダーが何も保存しないということではなく、**暗号化と復号はあなたのデバイスのローカルで完結し、サーバーは暗号化済みの暗号文のみを保存する**。具体的な流れ：

1. クライアントでマスターパスワードを入力。
2. クライアントはマスターパスワードから暗号化鍵を派生させます（この処理はあなたのスマホ/PC 上で行われます）。
3. 暗号化された暗号文のみがサーバーにアップロードされます。
4. 次回ログイン時、サーバーは暗号文を送信し、クライアントが再びマスターパスワードで復号します。

重要なポイント：**マスターパスワード自体はあなたのデバイスから一切送信されません**。サーバー上にもありません。そのため、仮にサービス提供元がハッキングされても、攻撃者が手にするのは解読不可能な暗号文の山に過ぎません——マスターパスワードを推測されない限り。

**なぜ「ゼロナレッジ」はサービス提供者にもパスワードを見せないのか？**

暗号化鍵はあなたのマスターパスワードから派生し、マスターパスワードはあなたの頭の中にのみ存在し、ローカルデバイス上でのみ使用されるからです。サービス提供者のサーバーにはマスターパスワードも暗号化鍵もなく、暗号文しかありません。だからこそ、Bitwarden、1Password などの正規サービス提供者は「私たちもあなたのパスワードを見ることはできません」と明確に宣言しているのです——見たくないのではなく、技術的に不可能なのです。

### 鍵派生関数：PBKDF2 と Argon2

マスターパスワードは暗号化鍵としてそのまま使えません——「鍵派生関数」による処理が必要です。この関数の役割は**あえて計算を遅くすること**ことで、ハッカーの総当たり攻撃のコストを非現実的なレベルまで引き上げます。

- **PBKDF2**: 古い標準規格で、iCloud キーチェーン、Bitwarden、LastPass などに広く使われています。反復ハッシュ（通常 10 万回以上）で破解を遅延させます。
- **Argon2**: 2015 年パスワードハッシュコンテスト優勝で、PBKDF2 よりモダンです。「計算コスト」に加え「メモリコスト」も導入し、GPU/ASIC などの専用ハードウェアでも加速破解が困難です。Bitwarden、KeePassXC などが既にサポートしています。

反復回数が多いほど安全ですが、ロック解除速度も遅くなります。正規サービス提供者は使用する派生アルゴリズムと反復回数を公開し、セキュリティコミュニティによる監査を可能にしています。

### 事例警鐘：LastPass 2022 年データ漏洩

2022 年 8 月、老舗パスワードマネージャー LastPass は開発環境への侵入を公表しました。同年 11 月にはさらに、ハッカーがユーザーのパスワード保管庫のバックアップ（暗号化された暗号文）を持ち出したことを明らかにしました。この事件の重要な教訓：

- **ゼロナレッジアーキテクチャが最低限を守った**：パスワード保管庫自体が暗号化されているため、ハッカーは暗号文を手にしても解読できません——ユーザーのマスターパスワードが十分に強いことが前提です。
- **弱いマスターパスワードのユーザーが最も危険**：ハッカーは弱いマスターパスワードをオフラインで総当たり攻撃できるため、弱いマスターパスワードや使い回しているユーザーは実質的なリスクに直面します。
- **メタデータも漏洩**：パスワードの内容は暗号化されていますが、サイト URL やメモなどのメタデータは暗号化されていない可能性があり、ユーザーがどのサービスを利用しているかが露出します。

**LastPass 事件から学んだこと**

第一に、マスターパスワードは長くユニークである必要があります（16 文字以上、大文字・小文字・数字・記号を含む）。第二に、Argon2id をサポートし反復回数の多いサービス提供者を優先しましょう。第三に、高価値アカウント（銀行、メール）には必ず独立した二要素認証を設定し、パスワードマネージャーだけに頼らないようにしましょう。

## 四、主要パスワードマネージャーのおすすめ

### iCloud キーチェーン

- **：Apple エコシステムユーザーの第一選択。Safari、App とシームレスに統合、無料、安全。欠点はクロスプラットフォーム利用が不便。**Google パスワードマネージャー
- **：Android ユーザーに便利。Chrome、Android システムと統合、無料。**Huawei パスワード金庫
- **：HarmonyOS 内蔵、Huawei ユーザーに適しています。**サードパーティ専門ツール：

### Bitwarden

- **：オープンソース無料、機能強力、すべてのプラットフォームに対応。プライバシー重視ユーザーに強くおすすめ。**1Password

- **：インターフェース美麗、機能豊富、家庭やチームに適しています。有料。**LastPass
- **：老舗パスワードマネージャー、無料版は機能制限あり。**KeePass
- **：完全ローカル保存、オープンソース無料。ギークユーザーに適しています。**四、パスワードマネージャー活用のベストプラクティス

## 五、パスワード強度の本当の評価

パスワードマネージャーは一般的に「パスワード強度」インジケーター（赤/黄/緑）を表示しますが、この評価は長さと文字の種類しか見ておらず、包括的ではありません。パスワードが本当に安全かを判断するには「エントロピー」と「漏洩の有無」を考慮する必要があります。

### エントロピー（Entropy）：パスワード強度の科学的尺度

「エントロピー」はパスワードの不確実性を測る指標で、単位はビットです。エントロピーが高いほど、総当たり攻撃に必要な試行回数が多くなります。

- **”123456”**: エントロピーが極めて低く、瞬時に破解されます。
- **”P@ssw0rd”**: 一見複雑に見えますが、すべての辞書攻撃リストに載っており、エントロピーは依然として低いです。
- **”correct horse battery staple”**: 4 つのランダムな単語、エントロピー約 44 ビットで、破解難度は「P@ssw0rd」をはるかに超えます。
- **16 文字のランダム文字（大文字・小文字 + 数字 + 記号を含む）**: エントロピー約 100 ビット、現在の計算力では総当たり攻撃不可能です。

結論:**「長さ + ランダム性」は「複雑な記号」よりも重要**。これがパスワードマネージャーが生成するランダムな長いパスワードが本当に安全である理由です。

### HaveIBeenPwned：あなたのパスワードが漏洩していないか確認

セキュリティ研究者 Troy Hunt が運営する haveibeenpwned.com は過去の大規模データ漏洩で流出したアカウントとパスワードを収録しています。以下のことができます：

1. メールアドレスを入力して、あなたのアカウントがいずれかの漏洩事件に関与していないか確認。
2. Pwned Passwords 機能を使って、特定のパスワードが既知の漏洩データベースに含まれているか確認（パスワードマネージャーやブラウザにこの機能が組み込まれています）。

もしパスワードが漏洩データベースに含まれていたら、**どんなに複雑に見えてもすぐに変更する必要があります**——辞書攻撃はこれらの既知パスワードを優先的に試すからです。

**k-匿名性クエリの原理**

HaveIBeenPwned にパスワードを入力するとまた漏洩するのでは？心配いりません。現代のクエリは「k-匿名」プロトコルを採用しています：ブラウザはパスワードハッシュの最初の 5 文字だけをサーバーに送信し、サーバーはその 5 文字で始まる全ハッシュリストを返し、ローカルで照合します。つまり、サーバーはあなたの完全なパスワードハッシュを見ることはありません。1Password、Bitwarden の「漏洩パスワード検出」もこの仕組みで動作しています。

### パスワードマネージャー内蔵の監視機能

- **1Password Watchtower**: 弱いパスワード、重複パスワード、漏洩パスワード、期限切れパスワード、2FA 未設定サイトをリアルタイムで監視し、赤/オレンジ/黄のレベル分けで表示します。
- **Bitwarden Vault Health**: 無料版でも弱いパスワード、重複パスワードを確認可能。有料版では漏洩パスワードの検出が可能です（HaveIBeenPwned ベース）。
- **iCloud キーチェーン「セキュリティレコメンデーション」**：iOS 設定パスワードセキュリティレコメンデーションで、どのパスワードが既知の漏洩に含まれているか、どれが重複使用されているかを教えてくれます。
- **Google パスワードマネージャー「パスワードチェックアップ」**：Chrome に内蔵され、漏洩したパスワード、弱いパスワード、重複パスワードをワンクリックでチェックできます。

**今すぐ確認：あなたのパスワードは安全か**

1. haveibeenpwned.com を開き、よく使うメールアドレスを入力して、どの漏洩事件に関与しているか確認しましょう。
2. iPhone ユーザー：設定パスワードセキュリティレコメンデーションで、赤色の警告を一つずつ処理しましょう。
3. Chrome ユーザー：設定パスワードパスワードチェックアップで、指示に従って変更しましょう。
4. 1Password/Bitwarden ユーザー：Watchtower/Health Reports を開き、「漏洩」と「重複」の項目を優先的に処理しましょう。

## 六、Passkey：パスワード不要ログインの未来

2024-2026 年、パスワード技術は革命を遂げようとしています——**Passkey（パスキー）**が従来のパスワードを完全に淘汰する可能性があります。Apple、Google、Microsoft の 3 大

プラットフォームが共同推進し、FIDO Alliance と W3C が支持する、数十年で最大のログイン方式の変革です。

### Passkey とは？

Passkey は FIDO2 / WebAuthn 標準に基づき、**デバイス上の鍵ペア**でパスワードを置き換えます。原理を簡略化すると：

- あなたのスマホが鍵のペアを生成します：**公開鍵**（ウェブサイトに登録）＋ **秘密鍵**（デバイスのセキュリティチップに永久保存）。
- ログイン時、ウェブサイトが「チャレンジ」を送信し、あなたのスマホが秘密鍵で署名し、ウェブサイトが公開鍵で検証します。
- 署名前に生体認証（Face ID / 指紋）またはデバイスの PIN で本人確認が必要です。
- このプロセス全体で、**パスワードは一切ネットワーク上で送信されません**。

### Passkey vs 従来のパスワード

| 比較項目 | 従来のパスワード | **Passkey** |
|---|---|---|
| フィッシング耐性 | 低い、フィッシングサイトがパスワードを盗める | 本質的にフィッシング耐性、秘密鍵は本物のドメインにのみ署名 |
| サーバー漏洩の影響 | パスワードハッシュが破解される可能性がある | サーバーは公開鍵のみ保存、漏洩の影響なし |
| 利便性 | 記憶/入力/管理が必要 | Face ID/指紋で 1 秒ログイン |
| リプレイ攻撃 | 中間者攻撃で傍受・再生される可能性 | 毎回使い捨て署名、再生不可 |

### 各プラットフォームのサポート状況

- **Apple iCloud キーチェーン Passkey**: iOS 16+ / macOS Ventura+ でネイティブサポート。Passkey は同じ Apple ID でログインしている全デバイス間で同期でき、新デバイスでサイトにログインする際に再作成不要です。
- **Google パスワードマネージャー Passkey**: Android + Chrome でネイティブサポート。Passkey は Google アカウントに保存され、Android デバイス間で同期。2023 年以降、Google 個人アカウントで Passkey ログインをサポート。
- **Microsoft**: Windows 11 / Edge が Passkey をサポート。Microsoft アカウント、Outlook、Office などで順次 Passkey ログインを提供し、Microsoft Authenticator で管理可能。
- **1Password / Bitwarden**: Passkey の作成、保存、クロスプラットフォーム同期をサポートし、単一エコシステムに縛られません。

**クロスデバイス同期が重要なブレークスルー**

初期の Passkey の最大の課題は「単一デバイスにバインド」——スマホを替えるたびに再作成が必要でした。2023 年以降、Apple、Google、Microsoft は Passkey のクラウド同期を実現：iCloud キーチェーンは Apple デバイス間で同期、Google パスワードマネージャーは Android デバイス間で同期。1Password などのサードパーティはクロスエコシステム同期を実現しました。これは Passkey がパスワードを代替する実用性を備えたことを意味します。

## スマホで Passkey を作成・使用する方法

**iOS（iPhone）の操作手順：**

1. Passkey をサポートするウェブサイト（Google、Microsoft、Amazon、eBay、PayPal など）のアカウントセキュリティページで「パスキーを追加 / Add passkey」を選択。
2. システムが Face ID / Touch ID の認証画面を表示し、認証成功後 Passkey が自動生成され iCloud キーチェーンに保存されます。
3. 次回そのサイトにログインする際、ユーザー名入力後に「iPhone でログイン」を選択し、Face ID を一度認証するだけです。
4. 管理画面：設定パスワード該当サイトを見つけると Passkey エントリーが表示されます。

**Android の操作手順：**

1. Passkey をサポートするウェブサイトのアカウントセキュリティページで「パスキーを追加」を選択。
2. Google パスワードマネージャーのポップアップ Google アカウントを選択画面ロック認証（指紋/パターン/PIN）。
3. Passkey は Google パスワードマネージャーに自動保存され、同じアカウントの Android デバイス間で同期されます。
4. 次回ログイン時に「Passkey を使用」を選択し、画面ロックを解除するだけです。
5. 管理画面：Android 設定パスワード、パスキー、自動入力パスワードマネージャー google.com。

## Passkey のクロスデバイスログイン

Windows PC で、iPhone にしか Passkey がないサイトにログインする場合、QR コードで認証できます：

1. サイトのログインページで「Passkey を使用」を選択。

2.「他のデバイスを使用」を選択。

3. PC に QR コードが表示されるので、iPhone でスキャン。

4. iPhone に Face ID 認証が表示され、確認後 PC 上でサイトのログインが完了します。

このプロセスは Bluetooth + インターネットの二重暗号化通信で行われ、Passkey の秘密鍵が iPhone を離れることはありません。

**今すぐ試す：パスワード不要ログインを体験**

1. Google アカウントにログインセキュリティパスキーパスキーを追加で、Face ID/指紋ログインを体験。
2. Microsoft アカウントにログインセキュリティ詳細セキュリティオプションパスキーを追加。
3. タオバオ、JD（京東）、eBay、PayPal など Passkey 対応サービスで試してみましょう。
4. 従来のパスワードをバックアップとして残し、高価値アカウントを徐々に Passkey に移行しましょう。

**移行期のアドバイス**

Passkey はパスワードを完全に置き換えたわけではありません——多くのサイトはまだ「Passkey + パスワード」の併用段階です。高価値アカウントで Passkey を優先的に有効化しますが、**従来のパスワードをすぐに削除しないでください**、バックアップとして残しましょう。同時にパスワードマネージャーの使用も続けてください。すべてのサイトが Passkey に対応しているわけではないからです。Passkey は「パスワードマネージャーの強力な補完」として捉え、「即時置き換え」とは考えないでください。

## 七、ハードウェアセキュリティキー

一般ユーザーより高いセキュリティを求める場合——例えば大量の暗号資産を保有している、経営者/ジャーナリスト/活動家である、企業のコアアカウントを保護するなど——ハードウェアセキュリティキーは SMS 認証コードより信頼性の高い選択肢です。

### ハードウェアセキュリティキーとは？

ハードウェアキーは物理デバイス（USB メモリのような形状）で、内部のセキュリティチップにあなたの秘密鍵を保存し、物理的接触または NFC で認証を完了します。代表的な製品：

- **YubiKey**（Yubico 社）：最も主流。FIDO2/WebAuthn、U2F、OTP、PIV、OpenPGP をサポートし、USB-C / USB-A / NFC など多様なインターフェースに対応。5 シリーズで 50-70 ドル。

- **Google Titan**: Google 製。USB-C + NFC / Bluetooth、約 30-40 ドル。
- **OnlyKey**: オープンソースの代替品、複数プロトコルをサポート。
- **中国産代替品**: 飛天誠信 ePasskey など、より手頃な価格。

### なぜハードウェアキーは SMS 認証コードより安全なのか?

- **フィッシング耐性**: Passkey と同様、ハードウェアキーは本物のドメインにのみ応答し、フィッシングサイトは認証コードを騙し取れません。
- **SIM ハイジャック耐性**: SMS 認証コードは SIM スワップ攻撃で傍受される可能性がありますが、ハードウェアキーは電話番号を必要としません。
- **中間者攻撃耐性**: FIDO2 プロトコルは毎回の署名にチャレンジ値を含み、リプレイできません。
- **物理的分離**: 秘密鍵は常にハードウェアチップ内にあり、マルウェアが PC を侵害しても読み取れません。

### 適用シーン

- 暗号資産取引所アカウント、ウォレット復元。
- 企業 SSO、リモートワーク VPN、SSH サーバーログイン。
- メール、GitHub、ソーシャルメディアなどの「メインアカウント」——一度侵害されると他の全アカウントがリセットされます。
- ジャーナリスト、弁護士、経営者など高感度人物。

### ハードウェアキーを二要素認証に設定する方法

Google アカウント + YubiKey を例に：

1. YubiKey を購入（2 本まとめて購入を推奨、1 本は日常用、1 本はバックアップ、別々の場所に保管）。
2. myaccount.google.com にログインセキュリティ 2 段階認証。
3.「セキュリティキーを追加」を選択 YubiKey を USB-C ポートに挿入、またはスマホの NFC センサーにかざす。
4. 指示に従って YubiKey の金属接点に触れて登録を完了。
5. 2 本目のバックアップキーの登録を推奨。
6.「2 段階認証」の設定で、ハードウェアキーを優先認証方式にし、SMS 認証コードをバックアップにします。

主流サポートサイト：Google、GitHub、Microsoft、Apple ID、Facebook、Twitter、Dropbox、GitLab、AWS、Coinbase、Binance など。

**デュアルキー戦略**

ハードウェアキーの最大のリスクは「紛失」です。推奨：(1) 異なる 2 本のキーを同時に登録；(2) 1 本は随身携帯（キーホルダー）、1 本は自宅の金庫または銀行貸金庫に保管；(3) 一部サイトでは「バックアップ復旧コード」を印刷してオフライン保存可能。これで主キーを紛失してもアカウントから永久に締め出されることはありません。

**購入ルートに注意**

必ず公式ストアまたは正規代理店から購入し、中古や出所不明のキーは買わないでください——理論上、プリセットされたハードウェアにバックドアが仕込まれている可能性があります。YubiKey のファームウェアは出荷後に更新できませんが、Yubico は継続的にセキュリティ公告を発信するので公式サイトを確認してください。

## 八、家族/チームのパスワード共有

家庭では Netflix、家庭用 Wi-Fi、ルーター管理画面、スマートロックのパスワードを家族で共有。チームではサーバー、データベース、ソーシャルメディアアカウントを複数人で共同利用します。**WeChat でパスワードを送信したり、共有ドキュメントに書いたりするのは高危険**——チャット履歴やドキュメント権限が漏洩すれば、パスワードも漏洩します。

### 1Password の「共有保管庫」（Vault）

1Password は家族・チーム共有の代表的な製品です。その核心仕組みは**複数の保管庫**：

- **個人保管庫**: あなただけが閲覧可能。銀行、メールなどのプライベートアカウント。
- **共有保管庫**: 家族やチームメンバー全員が閲覧可能。Netflix、家庭用 Wi-Fi、共有クレジットカードなど。
- **権限レベル分け**: チーム版では一部メンバーを「閲覧のみ」や「閲覧のみ・編集不可」に設定できます。

家族やチームメンバーはマスターパスワードを共有する必要がなく、各自のアカウントでログインしますが、権限を付与された共有保管庫の内容を閲覧できます。パスワード変更後、全メンバーにリアルタイムで同期されます。

### Bitwarden の「組織」機能

Bitwarden の無料版は個人パスワード共有をサポート（「Send」機能で期限付きで 1 件のパスワードを共有）。有料版では「組織」を作成し、複数エントリーを家族やチームメンバーと共有でき、権限を細かく設定可能。Bitwarden はセルフホストオプションも提供し、データ主権を重視する企業に適しています。

### 家庭シーン：Netflix、Wi-Fi などのパスワードを安全に共有する方法

- **Netflix / 動画サイト**: 1Password Families の「共有保管庫」にログインパスワードを保存し、家族メンバーが各自でログイン。グループチャットに送信する必要なし。
- **家庭用 Wi-Fi パスワード**: iOS デバイス間では AirDrop で来客に Wi-Fi パスワードを直接共有でき、口頭で伝える必要なし。Android 12+ にも「近距離共有で Wi-Fi 共有」機能があります。
- **スマートロック、スマート家電**: なるべくメーカー App の「家族メンバー」機能で各自に個別アカウントを作成し、1 つの管理者アカウントを共有しないようにしましょう。
- **家庭用ルーター管理画面**: 管理者パスワードは 1Password 共有保管庫に保存し、ゲスト用 Wi-Fi は別の SSID で設定。

**1Password Families のコストパフォーマンス**

1Password Families は約 4.99 ドル/月で最大 5 人の家族メンバーをサポート。各人に独立した個人保管庫 + 4 つの共有保管庫。一人当たり 1 ドル/月未満で、現在家庭共有で最も優れた体験を提供します。Bitwarden の家庭版は 10 ドル/年で 6 人サポートし、コストパフォーマンスはより高いですが、インターフェース体験はやや劣ります。

### チームシーン：サーバー/データベースパスワード共有

- **1Password Teams / Business**: 部門やプロジェクトグループごとに複数の保管庫を作成し、きめ細かい権限管理が可能。すべてのアクセス行為を監査できます。
- **Bitwarden Teams / Enterprise**: オープンソース方案、セルフホスト可能。エンタープライズ版は SSO 連携、ディレクトリ同期をサポート。
- **職務分離の原則**: 管理者パスワードを一人だけが掌握すべきではありません——2-3 名の「緊急連絡先」を設定し、キーパーソンの離職/休暇時も業務が継続するように。
- **離職プロセス**: 従業員の離職時にパスワードマネージャーのアクセス権を即時取り消し、必要に応じてその従業員が接触した高感度パスワードをすべてローテーション。

### 緊急アクセス（Emergency Access）：万が一の時のために

こんなシーンを想像してください：あなたが家庭のすべての重要アカウントを単独で管理しているが、意図せず入院したり亡くなったりした場合、家族が銀行、メール、保険のアカウントにログインできない。**「緊急アクセス」機能**はまさにこのために設計されました——信頼する家族/友人を認可し、あなたが応答できない時にパスワード保管庫にアクセスできるようにします。

**Bitwarden 緊急アクセスの設定：**

1. Bitwarden Web Vault にログイン設定緊急アクセス。

2. 信頼する連絡先のメールアドレスを追加し、相手を招待。
3. 「待機期間」を設定（7-14 日を推奨）：相手がリクエストしてからあなたが確認するまでの時間枠。
4. 待機期間内にあなたが応答しない（拒否もしない）場合、期間満了後に相手が自動的にアクセス権を取得します。
5. いつでも手動でリクエストを拒否したり緊急アクセス権を取り消すことができます。

**1Password 緊急アクセス：**

1. 1Password は直接「緊急アクセス」とは呼びませんが、「家族/チーム組織」機能で実現できます——家族を家族版に追加した後、一部エントリーを「共有保管庫」に配置。
2. 個人版ユーザーには、マスターパスワードを高度に信頼する家族に伝えるか、密封した封筒で金庫に保管することを推奨。
3. 1Password は「復旧コード」も提供しており、二要素認証を有効にする際は必ず印刷して保管してください。

**今すぐ設定：家族のために「スペアキー」を残す**

1. 信頼する家族 1 人を「緊急連絡先」に選定。
2. パスワードマネージャーで緊急アクセスを設定（Bitwarden）するか、家族共有保管庫に追加（1Password）。
3. マスターパスワード + 復旧コードを紙に書き、封筒に入れて自宅の金庫に保管するか信頼できる家族に伝える。
4. 半年に一度見直し、緊急連絡先が引き続き適切か確認。

## 九、パスワードマネージャー活用のベストプラクティス

- **：マスターパスワードは最後の砦、必ず長く複雑なパスワードを設定し、しっかり記憶。**二要素認証を有効化
- **：パスワードマネージャーアカウントで二要素認証をオンにし、保護を強化。**定期的に弱パスワードをチェック
- **：ほとんどのパスワードマネージャーには「セキュリティ監査」機能があり、弱パスワードや重複パスワードを見つけ出します。**マスターパスワードを忘れない
- **：マスターパスワードは復元不可、忘れるとすべてのパスワードを失います。紙に書いて安全な場所に保管推奨。**緊急アクセス
- **：緊急連絡先を設定、万が一アクセスできなくなった場合、信頼できる人があなたのパスワードを取得可能。**重要な注意

パスワードマネージャーは便利ですがリスクもあります：マスターパスワードが漏洩すると、すべてのパスワードが露出します。だからマスターパスワードは他のパスワードと絶対に同じにせず、見つけやすい場所に書かないでください。また、良好なセキュリティ実績のあるサービスプロバイダーを選ぶことをおすすめします。

今すぐ試す：パスワードマネージャーを使い始める

パスワードマネージャーを一つ選ぶ（Bitwarden またはシステム標準を推奨）。

1. アカウント登録、強力なマスターパスワードを設定。
2. 既存のパスワードをインポート（あれば）または新規パスワードの追加を開始。
3. ブラウザプラグインまたはスマホ App をインストールし、自動入力機能を体験。
4. セキュリティ監査レポートを確認し、改善が必要なパスワードを見つける。
5. 三、応用編（第 15-20 章）

# 三、応用編（第 15-20 章）

# 第十五章：バッテリーの科学と連続駆動時間の最適化

一、バッテリーとは？サイクルと健康度

## 一、バッテリーとは？サイクルと健康度

使いながら充電

- **：旧式バッテリーと異なり、メモリー効果がなく、使い切ってから充電する必要はありません。**熱に弱い
- **：高温はバッテリーの頭号キラー。**経年劣化する
- **：長く使うと最大容量が低下しますが、これは正常な現象。**核心指標：サイクル回数と健康度

**1 サイクルは 1 回の充電ではありません。例えば今日 60% 使って満充電、明日 40% 使って満充電、合計 100% 消費で 1 サイクル。**

- バッテリー健康度は現在の最大容量の新品時に対する割合。80% を下回ると駆動時間が明確に短くなると感じます。
- 確認方法
- **：iPhone：設定 > バッテリー > バッテリーの状態と充電。Android：通常は設定でバッテリー健康度を検索。**二、充電の迷信？急速充電、ワイヤレス充電と黄金の法則

## 二、充電の迷信？急速充電、ワイヤレス充電と黄金の法則

- **：純正または認証済みの急速充電アクセサリを使えば、ダメージはわずか。スマホと充電ヘッドには温度制御があり、過熱すると自動的に出力を下げます。**ワイヤレス充電
- **ワイヤレス充電**黄金の養生法則

- **：電池残量を 20%-80% の間に保つよう努め、常に飢え（0%）や満腹（100%）状態にしない。**三、急速充電プロトコル完全解説：なぜあなたの充電器は速く充電できない？

**iPhone「80% 充電制限」機能——ゴールデンルールをシステム設定に**

iPhone 15/16 シリーズ（iOS 17 以上）は「20%-80% の法則」をシステム機能にしました。有効化すると、充電が 80% に達した時点で自動的に停止し、バッテリー劣化を大幅に遅らせます。設定パス：
設定 > バッテリー > バッテリーの状態と充電 > 充電の最適化 > 80% 制限
3 つのオプションの意味：(1) **制限なし**——100% まで充電；(2) **バッテリー充電の最適化**——生活リズムを学習し、早朝の最終段階まで 100% 到達を遅延；(3)**80% 制限**——80% で停止、最もバッテリーに優しい。長期的に自宅/オフィスで充電環境がある人には 80% 制限を強く推奨。頻繁に出張し、満充電で出かけたい人は「バッテリー充電の最適化」を選択。

**GaN 充電器——出張組の良い相棒**

最近の急速充電アダプタがどんどん小型化しているのに気づいているかもしれません。その立役者は **GaN（窒化ガリウム）**という新型半導体材料です。従来のシリコンベース充電器と比べ、GaN 充電器にはいくつかの明確な利点があります：

- **小型**: 同出力で、GaN 充電器は従来シリコン製の 1/2 から 1/3 のサイズにでき、65W の GaN アダプタは Apple 旧 5W アダプタより小さい。
- **高効率・低発熱**: 窒化ガリウムのスイッチング損失はシリコンよりはるかに低く、エネルギー変換効率は 95% 以上に達し、充電時に熱くなりにくい。
- **出張に適している**: 65W または 100W の GaN マルチポート充電器 1 つで、スマホ、タブレット、ノート PC を同時に充電でき、「レンガ」3 つ持参よりはるかに軽量。
- **選び方のアドバイス：**PD プロトコル対応の GaN 充電器を選ぼう（65W 以上ならノート PC も充電可能）。Baseus、Anker、Nubia、Xiaomi などが定評あり。ケーブルの互換性も要確認（大電力には E-Marker チップ内蔵の Type-C ケーブルが必要）。

**こんな経験はありませんか：スマホは急速充電対応なのに、充電器を変えたら遅くなった。これは急速充電が単なる「高出力なら OK」ではなく、充電ヘッド、ケーブル、スマホの三者が「同じ言語を話す」必要があるから——これが急速充電プロトコルです。**

1. 急速充電の基本原理

**充電電力 = 電圧 × 電流（P = U × I）。電力を上げるには、電圧を上げるか、電流を上げるか、または両方を上げる。異なるプロトコルが異なる技術路線を選択しています。**

急速充電のワークフロー：

ハンドシェイク段階

1. **：スマホと充電ヘッドがケーブルを通じて通信し、それぞれ対応するプロトコルと電力情報を交換。**ネゴシエーション段階
2. **：双方が共通対応する最高電力レベルを確定。**充電段階
3. **：充電ヘッドがネゴシエートされた電圧・電流を出力、スマホ側で電源管理。**調整段階
4. **：電池残量が増加するにつれ、スマホは段階的に充電電力を下げ、バッテリーを保護。**
2. 主流急速充電プロトコル詳細

### USB PD（Power Delivery）

——汎用プロトコルの王特徴

- 特徴電圧レベル
- **：5V、9V、15V、20V、PD 3.1 は 28V、36V、48V にも対応。：**5V、9V、15V、20V。PD 3.1 ではさらに 28V、36V、48V にも対応。
- **USB PD 3.1** 優位性
- **：汎用性が高く、一つの充電ヘッドで多種デバイスを充電可能。PPS（プログラマブル電源）に対応し、電圧電流を細かく調整可能。**劣位性
- **：Type-C インターフェースと専用チップが必要、コストが高め。**Qualcomm QC（Quick Charge）

——**Android 陣営の元老** QC 3.0

- **：3.6V-20V を 0.2V ステップで可変電圧に対応、最大 18W。**QC 4.0/4+
- **：USB PD プロトコルと互換、最大 27W に対応、デュアルチャネル充電技術を追加。**QC 5.0
- **100W+：**最大 100W 以上に対応し、0  50% まではわずか 5 分。発熱も大幅に改善。
- **USB PD 3.1**OPPO VOOC/OnePlus WARP/realme Dart

——低電圧大電流の代表技術路線

- 技術路線 VOOC 1.0

- **：20W（5V/4A）、2014 年発表、急速充電時代を開創。**SuperVOOC
- **：50W-65W、デュアルセル直列設計を採用。**SuperVOOC 2.0
- **：65W、30 分で 4000mAh バッテリーを満充電。**150W/240W スーパーフラッシュ充電
- **：現在最速の量産急速充電技術。**特徴
- **特徴** Huawei SuperCharge

——**高電圧急速充電の進化** FCP（Fast Charge Protocol）

- **：9V/2A、18W、QC 2.0 と互換。**SCP（Super Charge Protocol）
- **：より高い電力に対応、22.5W から 66W、さらに 100W へ。**技術的特徴
- **：デュアルセル直列＋チャージポンプ方式を採用、複数プロトコル互換に対応。：**チャージポンプ技術を採用し、高電圧を低電圧・大電流に変換してバッテリーへ供給。効率は最大 98%。
- **USB PD 3.1**Xiaomi 澎湃急速充電

——**自研プロトコルの台頭**澎湃 P1 チップ

- **：Xiaomi 自研の急速充電管理チップ、120W 有線急速充電に対応。**澎湃 G1 チップ
- **：バッテリー管理チップ、P1 と組み合わせてより効率的な急速充電を実現。**技術的特徴
- **：デュアルセル直列＋チャージポンプ方式を採用、複数プロトコル互換に対応。**Apple 急速充電

~**27W** プロトコル

- **最大電力**電力
- **：iPhone 8 以降が急速充電に対応、最大約 27W（iPhone 15 Pro Max は 27W+）。**特徴
- **特徴**アクセサリ要件
- **：USB-C to Lightning ケーブル（または USB-C to USB-C ケーブル）と PD 充電ヘッドが必要。**3. 急速充電プロトコル比較表

**プロトコル**

| 最大電力 | 技術路線 | 技術路線 | **USB PD 3.1** |
|---|---|---|---|
| 240W | 低電圧大電流 | 最も広範 | QC 5.0 |
| 100W+ | 高電圧可変 | Qualcomm 機種 | VOOC/Dart |
| 240W | 低電圧大電流 | OPPO 系専用 | SuperCharge |
| 100W | チャージポンプ | Huawei/Honor | Apple 急速充電 |
| ~27W | PD プロトコル | iPhone/iPad | 4. ワイヤレス充電プロトコル |

**ワイヤレス充電にも独自のプロトコル標準があります：**

Qi 標準

- **：WPC（ワイヤレス充電アライアンス）が制定、最も汎用的な標準。基本 Qi は 5W-15W に対応。**Qi2 標準
- **：2023 年発表、マグネット位置決め（Apple MagSafe 類似）を導入、15W+ に対応。**プライベートプロトコル
- **：各社が独自の高速ワイヤレス充電を持つ、Xiaomi 50W ワイヤレス充電、Huawei 50W ワイヤレス充電など、専用充電器が必要。**5. 急速充電選びのガイド

**プロトコルマッチングを確認**

- **：充電器購入時、あなたのスマホの急速充電プロトコルに対応しているか確認。サードパーティ充電器は通常対応プロトコルを表示しています。**ケーブルを確認
- **：大電力急速充電には専用ケーブルが必要。普通のケーブルは 60W 以下しか対応しない可能性。**認証を確認
- **：Apple ユーザーは MFi 認証ケーブルを選ぶことを推奨。Android ユーザーはブランド純正または有名サードパーティ製を選ぶ。**発熱を確認
- **：急速充電時のスマホ発熱は正常ですが、火傷するほど熱いのは異常です。充電を一時停止するか充電器を交換することを推奨。**急速充電の安全ヒント

充電しながら大型ゲームをプレイするのを避ける、発熱が悪化。

- 粗悪な充電器やケーブルを使用しない、安全上のリスクがあり。
- 充電時は分厚いスマホケースを外し、放熱を助ける。
- 充電器やスマホが異常発熱、異臭がする場合は直ちに使用中止。
- 四、これが本当の節電テクニック

## 四、バッテリー新技術：シリコン炭素負極バッテリー

2024-2026 年のフラッグシップスマホ発表会に注目しているなら、「シリコン炭素負極」という言葉がハイエンドバッテリーの代名詞になっているのに気づくでしょう。これはリチウムイオンバッテリー負極材料の重要なアップグレードで、原理はそれほど複雑ではありません：

- **従来の黒鉛負極**: スマホバッテリーは従来黒鉛を負極に使用してきました。技術は成熟し安定していますが、エネルギー密度は理論上限界（約 372mAh/g）に接近しています。
- **シリコン材料の導入**: シリコンの理論比容量は黒鉛の約 10 倍（約 4200mAh/g）です。少量のシリコンを黒鉛に混ぜて「シリコン炭素複合負極」を作ると、バッテリー全体のエネルギー密度を大幅に向上できます。
- **エネルギー密度の向上**: 同等体積で、シリコン炭素負極バッテリーの容量は約 **15%-20%** 向上します。これが 2024-2026 年のフラッグシップが 7-9mm のボディに 5500-6000mAh の大容量バッテリーを搭載できる決定的な理由です。

**どの機種が採用しているか？**

- **HONOR（栄耀）**: Magic シリーズがシリコン炭素負極を業界初採用。Magic6 Pro、Magic7 シリーズ、Magic V3 折りたたみスクリーンに採用。
- **Xiaomi**: Xiaomi 14 Ultra、Xiaomi 15 シリーズに次世代シリコン炭素負極バッテリーを搭載。
- **vivo**: X100 Ultra、X200 Pro シリーズに高密度シリコン炭素負極方案を採用。
- **Huawei/OPPO など**: 2025-2026 年のフラッグシップに広く普及し、シリコン炭素負極は「ブラックテクノロジー」からハイエンド機の標準装備へと移行しています。

**ユーザーへの実際の影響**: シリコン炭素負極の最大のメリットは「同体積でバッテリー容量が大きい」こと——スマホはより薄くでき、同時にバッテリー持ちも犠牲になりません。あるいはボディはそのままでもバッテリーが 4500mAh から 5500mAh に増加します。つまり、バッテリー持ちのために「分厚く重く」する代償を払う必要がなくなったのです。ただし、シリコン炭素負極バッテリーは低温環境と満充電管理において急速充電チップとの連携最適化がまだ必要で、これもメーカーが長期的な 80% 充電維持を推奨する理由の一つです。

## 五、これこそ本当の省エネテクニック

- **：輝度を下げる、自動ロック時間を短縮。OLED 画面スマホはダークモードでかなり節電。**電波不良
- **：エレベーター、地下室ではスマホが必死に電波を探し、消費が激しい。**5G

- **：電波が悪い場合、手動で 4G に切り替えると駆動時間が大幅に改善。**位置情報サービス
- **：非ナビ系 App は位置情報権限を「使用時のみ許可」に。**バックグラウンド活動を制限
- **：iPhone は設定 > 一般 > バックグラウンド App 更新。Android はアプリバッテリー設定で。**チェックしてみよう：バッテリーの健康診断

健康度を確認：バッテリーの最大容量を見つけて記録。

1. 消費詳細を分析：バッテリー設定のグラフと App リストで消費の元凶を見つける。
2. 対策を策定：バックグラウンド消費異常の App を制限。
3. 第十六章：アカウント体系とクラウドサービス

# 第十六章：アカウント体系とクラウドサービス

一、スマホアカウントはどれだけのものを管理している？

## 多くの人がアカウントは App ダウンロード用だと思っています。実はスマホアカウントの背後には想像以上のものが紐付いています：

連絡先

- **連絡先**写真と動画
- **写真と動画**メモ
- **：ふと思いついたアイデア、日記、アカウントのパスワード（そう、多くの人がパスワードをメモに記録）。**App 購入履歴
- **：数百元使った有料 App やアプリ内課金。**スマホ検索
- **：スマホ紛失後、位置特定、ロック、リモートワイプを助ける唯一のツール。**Apple スマホを使っている場合、これらすべてが Apple ID に紐付いています。Android を使っている場合、Huawei/Xiaomi/OPPO/vivo アカウント、そして Google アカウントに紐付いています。これらのアカウントはデジタル生活の「戸籍謄本」で、戸籍を失うと多くのものが不明確になります。

二、今すぐやるべき 3 つのこと

## 二要素認証を有効化

1. **：Apple でも Android でも、最初にアカウント設定で「二要素認証」または「2 段階認証」をオンに。これはドアに内鍵を追加するようなもので、パスワードだけでは不十分、スマホで認証コードを受け取る必要があります。ネットでパスワードを推測されても、あなたのスマホがなければ入れません。**信頼できる復元方法を登録

2. **：二要素認証は安全ですが、リスクもあります——自分のスマホをなくしたらどうする？そこでシステムは「復元キー」や「予備メール/電話番号」の設定を促します。これは緊急時にアカウントを取り戻す唯一の手段で、紙に記録しスマホに保存しないことをおすすめします。**「スマホを探す」が有効か確認

3. **：この機能はスマホ紛失後に位置特定、ロック、音再生、さらにはリモートワイプを助けます。最後の砦。多くの人がスマホ購入時にこのステップをスキップし、後で使おうとした時に有効でないことがわかり、手遅れに。**三、iCloud：Apple ユーザーの「空気」

## Apple スマホを使っている人なら、遅かれ早かれ iCloud 容量が満杯になったらどうするかという難問に直面します。

Apple は無料で 5GB を提供しますが、正直 1 回のバックアップにも足りません。だからほとんどの人が最終的にあの煩わしいポップアップ「iCloud ストレージがいっぱいです」を受け取ります。この時 3 つの選択肢があります：

クリーンアップ

- **：設定あなたの名前 iCloud ストレージを管理、誰が容量を占めているか確認。通常、写真とバックアップが大半。不要な古いバックアップを先に削除。**アップグレード
- **：50GB は月 6 元、200GB は月 21 元。個人的には 50GB でほとんどの人には十分、主にバックアップと写真サムネイル保存用。**迂回策
- **：お金を使いたくない場合、写真を定期的にパソコンに書き出し、iCloud 写真同期をオフに。ただしこれでは写真自動バックアップの利便性を失います。**重要な注意：iCloud 写真ライブラリは「クラウドストレージ」ではなく「同期」です。スマホで写真を削除すると iCloud でも削除されます。「クラウドにバックアップしてスマホを空にする」を望むなら、iCloud 写真はあなたの求めるものではなく、iCloud クラウドバックアップまたはサードパーティのクラウドストレージが必要です。

四、同期とバックアップを混同しない

**iCloud+：有料ユーザーだけが使えるプライバシー三種の神器**

有料 iCloud プラン（50GB/200GB/2TB など）にアップグレードすると、自動的に **iCloud+** ユーザーになり、3 つのプライバシー保護機能が解放されます：

- **Private Relay（プライベートリレー）**：Apple 版「VPN」。Safari のトラフィックが Apple サーバーを経由してからウェブサイトに送信され、誰も（Apple 含む）あなたが誰か、何にアクセスしたかを同時に知ることはできません。公共 Wi-Fi でのプライバシー保護に適しています。設定パス：設定 > あなたの名前 > iCloud > プライベートリレー。
- **Hide My Email（メールを隠す）**：ウェブサイト/App 登録時に使い捨てのランダムメールアドレスを生成し、メールは自動的に本当のメールアドレスに転送されま

す。このサイトからのメールを受け取りたくなくなったら、ランダムアドレスを無効にするだけで、本当のメールアドレスは露出しません。設定パス：設定 > あなたの名前 > iCloud > メールを隠す。

- **カスタムメールドメイン**: 自分のドメイン（例：@yourname.com）で iCloud メールを送受信でき、個人ブランドや家庭用メール管理に適しています。

ヒント：Private Relay は一部の国/地域では利用できません。中国国内ユーザーは別の方案で補完する必要があるかもしれません。

**iCloud 高度データ保護（ADP）——真のエンドツーエンド暗号化**

2022 年末、Apple は **iCloud 高度データ保護（Advanced Data Protection, ADP）**を発表し、エンドツーエンド暗号化（E2EE）を従来の健康、パスワードなどの限られたデータから **iCloud データの大部分**——写真、メモ、iCloud バックアップ、デバイス情報など計 23 種類のデータに拡大しました。

- **E2EE とは**: 暗号化鍵はあなたのデバイスにのみ存在し、Apple サーバーにもありません。仮に Apple がハッキングされても、法執行機関から要求されても、Apple は復号できる平文データを提供できません。
- **基本ジェスチャー：**設定 > あなたの名前 > iCloud > 高度データ保護で、指示に従って復旧連絡先/復旧キーを設定（**必須**、キーを紛失するとデータも復旧できません）。
- **注意事項**: (1) 有効化後は復旧キーを必ず適切に保管してください。紛失するとアカウント喪失時にデータを取り戻せません。(2) 共有アルバム、iCloud 共有フォトライブラリなどの一部機能は E2EE の対象外です。(3) 中国本土の Apple ID は現在 ADP をサポートしていません。他の地域のアカウントに切り替える必要があります。

一言アドバイス：アカウントに重要なプライバシーデータがあり、アカウントの地域がサポートしている場合、ADP の有効化を強く推奨します——これは一般ユーザーが得られる最強のクラウドデータ保護です。

## この 2 つの言葉は似ていますが、結果は全く異なります：

同期

- は双方向：スマホで削除するとクラウドでも削除、クラウドで削除するとスマホでも削除。iCloud 写真、連絡先、メモはこれらは同期。利点は複数デバイス間で一貫性を保てること、欠点は「削除したら本当に消える」。バックアップ
- は一方向：スマホが現在の状態の「スナップショット」をクラウドに保存。その後スマホで削除しても、クラウドバックアップ内のコピーは残存。iCloud クラウドバックア

ップ、パソコンローカルバックアップはこれらはバックアップ。利点はデータがより安全、欠点は容量を食い、リアルタイム更新されない。五、Android ユーザーのアカウントマトリクス

## Android 側は Apple より少し複雑で、データが複数のアカウントに分散する可能性があります：

ブランドアカウント

- （Huawei/Xiaomi/OPPO/vivo）：クラウドスペース、スマホ検索、アプリストア、ウォレットを管理。各ブランドが独自のクラウドサービスを持ち、通常は基本無料＋拡張有料モデル。Google アカウント
- ：Gmail、Google フォト、Google 連絡を使っている場合、これらは Google アカウントにあります。Google は無料で 15GB 提供、Apple より気前が良いですが、Gmail、Drive と共用。WeChat/QQ
- ：これらはスマホシステムから独立。チャット履歴と WeChat お気に入りは、ブランドアカウントにも Google アカウントにもなく、WeChat 独自のサーバーにのみ存在。だから Android ユーザーがバックアップする時は、ブランドクラウドバックアップ、Google バックアップ、WeChat チャット履歴バックアップの 3 つを個別にチェックする必要があり、どれも欠かせません。

六、ファミリー共有：家族も恩恵を

## Apple の「ファミリー共有」は家族と App 購入、音楽、映画、iCloud スペースを共有でき、位置情報も共有可能。最大 6 人まで追加。設定方法：設定あなたの名前ファミリー共有。Huawei と Xiaomi にも同様の機能があり、アカウント設定で「ファミリー共有」や「ファミリークラウド」を見つけられます。

非常に実用的なシーン：両親のスマホにファミリー共有を設定すれば、いつでも彼らの位置を確認でき（同意が前提）、万が一老人が迷子になったり連絡が取れなくなったりした時に位置特定が可能。

アカウント乗っ取りの結果は想像より深刻

ハッカーがあなたのアカウントパスワードを入手した場合、可能なのは：スマホをリモートロックして身代金要求、あなたの写真や連絡先を閲覧、あなたの決済方法で買い物。さらに厄介なのは、アカウントに多くを紐付けすぎていると、アカウント回復自体が悪夢。だから二要素認証を今すぐ有効化を。

実践：アカウントに包括的な健康診断を実施

**スマホ設定を開き、アカウントページに入り、二要素認証が有効か確認。**

1.「信頼できる電話番号」と「復元キー」が設定されているか確認、キーは紙に記録。

2. iCloud またはクラウドスペース管理に入り、どのデータが同期されているか確認、不要なものはオフに。

3.「iPhone を探す」が有効か確認。

4. Android の場合、ブランドクラウドバックアップ、Google バックアップ、WeChat バックアップの 3 つがカバーされているか確認。

5. 第十七章：スマホと他デバイスの連携

# 第十七章：スマホと他のデバイスの連携

一、パソコンとの連携：スマホとパソコンはもう二つの世界ではない

正直、以前最も嫌だったのはスマホで撮った写真をパソコンに置いてレタッチしたくて、ケーブルを探し、転送を待つこと。後に、実はケーブル不要で、良い連携はケーブルより 10 倍速いことが分かりました。

Apple エコシステム（AirDrop + ユニバーサルクリップボード）

- **：iPhone と Mac を使っているなら、AirDrop は最初に覚えるべき機能。スマホで共有をタップ AirDrop Mac を選択、写真が瞬時に移動し、通信量も Wi-Fi も不要、Bluetooth と Wi-Fi ダイレクトを使います。もう一つ見落とされがちな機能が「ユニバーサルクリップボード」——スマホでテキストをコピーし、Mac で直接ペーストすると出てきて、まるで魔法のよう。**Android + Windows（Link to Windows）

- **：Microsoft には「Link to Windows」という App があり、Samsung スマホに内蔵、他の Android スマホはアプリストアからダウンロード可能。ペアリング後、パソコンでスマホの通知確認、SMS 返信、写真閲覧、さらには電話の発着信が可能。2 年使っていますが、会議中にスマホを何度も見る必要がなく、パソコン画面に通知がポップアップするだけで十分。**ブランドマルチスクリーン連携

- **：Huawei の「マルチスクリーン連携」と Xiaomi の「Xiaomi 妙享」はさらに徹底——スマホ画面を直接パソコンに投影し、マウスでスマホからパソコンへファイルをドラッグ＆ドロップでき、パソコンで Douyin を見たり WeChat に返信したりも可能。クロスデバイスのコピーペースト、ファイルドラッグもでき、体験は非常にスムーズ。**二、テレビにキャスト：家族みんなで視聴

お正月に実家に帰った時、従妹がテレビで彼女の卒業旅行動画を流したがっていました。長く苦労しましたが、最終的に非常にシンプルだと分かりました：

iPhone テレビ

- ：右上からコントロールセンターを下ろし、二つの四角が重なったアイコン（画面ミラーリング）をタップ、テレビ名を見つけてタップすれば接続。前提はテレビとスマホが同じ **Wi-Fi** に接続し、テレビが **AirPlay** に対応していること（ほとんどのスマートテレビが対応）。Android テレビ

- ：各社呼び方が異なり、**Huawei** は「ワイヤレスキャスト」、**Xiaomi** は「投影」、**OPPO/vivo** は「マルチスクリーン連携」。通常はクイックスイッチパネルで見つけられ、見つからない場合は設定で「キャスト」を検索。動画 App から直接キャスト

- ：テレビで **iQiyi**、**Bilibili**、**YouTube** を見たいだけなら、さらに簡単——再生画面で **Wi-Fi** シンボル付きの四角い小アイコンを探し、タップしてテレビを選択すると動画が移動。この時スマホで別のことができ、動画はテレビで継続再生、相互に影響しません。キャスト失敗時の対策

- ：**90%** は同じ **Wi-Fi** に接続していないこと。残り **10%** はテレビのキャスト機能がオフで、テレビ設定で「マルチスクリーン連携」や「**AirPlay**」スイッチを探してください。三、スマートウォッチ/バンド：手首に常駐する健康管理者

母にバンドを買いました。最初は「運動もしないのにこれを買ってどうするの」と言っていました。**2** 週間使って、自分が毎日 **5** 時間ちょっとしか寝ていないことに気づき、自発的に作息を調整し始めました。これがウォッチ/バンドの最も実用的な価値——目に見えない健康データを目に見える数字にする。

通知フィルター

- ：スマホがバッグやポケットで震え続ける時、手首を上げて一目見る——上司からの **WeChat** か **Pinduoduo** の販促か、**1** 秒で判断、スマホを取り出す価値のないメッセージはそのままに。健康データ同期

- ：ウォッチは **24** 時間心拍数、血中酸素、睡眠、歩数をモニタリング、これらのデータは自動的にスマホの健康 **App** に集約。長期的に見れば自分のトレンドが分かる——今月は歩くのが少なすぎないか、今週の睡眠の質は良くなったか。交通カード/キーカード同期

- **：スマホで登録したバスカードやキーカードは、ほとんどのブランドが自動的にウォッチに同期。地下鉄でスマホを取り出す必要がなく、手首を改札機にかざすだけで通過、本当に「未来は既に来ている」と感じます。**スマホ検索の神器
- **：ウォッチには通常「スマホを探す」機能があり、タップするとスマホが鳴ります。私がこの機能を使う頻度は予想をはるかに超える——スマホをどこに置いたかすぐ忘れる。**四、スマートホーム：スマホは家の「メインスイッチ」

## スマートホームは難しそうに聞こえますが、要するに 3 つのこと：スマホでリモート制御、自動化ルールの設定、音声操作。

私自身のいくつかの実際の使い方をお話しすると、これがいかに実用的か分かります：

エアコン

- **：夏の帰宅途中にスマホで事前にエアコンをオン、ドアを開けると涼しい。100 元余りでエアコンパートナーコンセントを買えば実現でき、エアコンを買い替える必要なし。**照明
- **：就寝前にベッドに横になり、起きるのが面倒ならスマホでタップまたは「Hey Siri/小愛同学、電気を消して」と声かけ。**ロボット掃除機
- **：外出前にスマホで「掃除開始」をタップ、帰宅すると床が綺麗に。毎日定時掃除も設定可能。**カメラ
- **：家に老人、子供、ペットがいれば、スマホでいつでも家庭の状況を確認。**どのエコシステムを選ぶか？一言でまとめると：

iPhone 使用 Apple HomeKit 対応デバイスを優先（少し高いがプライバシー保護が良い）。

- デバイス多数でコスパ重視 Xiaomi 米家エコシステムが最も充実、照明、コンセント、ロボット掃除機など何でも揃う。
- Huawei スマホ使用 HarmonyOS Connect を優先、Huawei デバイスとの体験が最も良い。
- 五、Bluetooth イヤホンとスピーカー：ワイヤレスの自由

## 有線イヤホンから Bluetooth イヤホンへの変更は、私にとってスマホ体験のアップグレードで最も顕著なものでした。もうバッグから絡まった糸の塊を取り出すことも、ドアノブにケーブルが引っかかることもありません。

ペアリングは簡単

- **：イヤホンは初回電源オンで通常自動的にペアリングモード（点滅）に入り、スマホの Bluetooth をオンにすれば検索でき、タップするだけで接続。以降は取り出すたびに自動接続、何もする必要なし。**同ブランドの相乗効果
- **：AirPods と iPhone の組み合わせはバッテリー残量が自動ポップアップし、複数の Apple デバイス間でシームレスに切り替え可能。Huawei FreeBuds と Huawei スマホ、Xiaomi イヤホンと Xiaomi スマホにも同様のポップアップと最適化があります。**Bluetooth スピーカー
- **：家庭集会、アウトドアキャンプの時、一つの Bluetooth スピーカーで雰囲気が倍増。接続後、スマホがあなたの DJ ブースに。**遅延に注意
- **：ゲーム時に Bluetooth イヤホンを使うと、わずかな遅延（銃声がしてから 0.5 秒後に音が聞こえる）が生じる可能性。遅延に敏感な場合は、有線イヤホンを使うか、「低遅延モード」対応の Bluetooth イヤホンを購入。**六、カーシステム：運転中の安全なパートナー

### UWB 超広帯域技術——Bluetooth より高精度な「空間感覚」

Bluetooth のほか、新型スマホには UWB（Ultra-Wideband、超広帯域）チップも内蔵されています。その能力は**高精度な測距と空間定位**——他のデバイスがどれくらい離れているか、どの方向にあるかを計算でき、精度はセンチメートル級に達します。Bluetooth は「近くにいる」しか分かりませんが、UWB は「左前方 1.2m」まで分かります。主な用途：

- **AirTag/検索の高精度化**: iPhone が AirTag を検索する際に「7m 先、左へ」の矢印ナビを表示するのは UWB のおかげです。通常の Bluetooth は「範囲内にいるかいないか」しか分かりません。
- **デジタルカーキー**: UWB 搭載の iPhone と Android フラッグシップは、カーキーとして車に近づくと自動解除、離れると自動ロックでき、運転席と助手席を区別し、リレー攻撃も防げます。
- **屋内ナビゲーション**: ショッピングモール、空港で UWB ビーコンによる「通路レベル」のナビゲーションが可能で、GPS よりはるかに正確です。
- **サポートする機種**: iPhone 11 以降の機種に UWB チップ（U1/U2）搭載。Android 陣営では Xiaomi、Samsung、vivo、OPPO の一部フラッグシップにも搭載済み。

### LE Audio と Auracast——Bluetooth オーディオの次世代標準

20 年使われてきた従来の Bluetooth オーディオ（A2DP）は **LE Audio**（低消費電力オーディオ）に置き換えられつつあります。これは Bluetooth 5.2 以降の次世代オーディオ標準です：

- **より省電力、より高音質**: 全新の LC3 コーデックを採用し、従来の SBC より高音質・低消費電力で、イヤホンのバッテリー持ちが顕著に延びます。

- **デュアルオーディオパス**: 左右のイヤーピースが独立して伝送し、片耳モードへの切り替えでも切断しません。
- **Auracast ブロードキャストオーディオ**: LE Audio の目玉機能。1 台のスマホ/TV から同時に**無数の**イヤホンにオーディオをブロードキャスト——空港ロビー、博物館、ジム、教室で 1 つの送信源から、複数の人が自分のイヤホンで同じ音声を聴けます。
- **現状**: 2024-2026 年に新発売のフラッグシップイヤホンとスマホは LE Audio を広くサポート。旧デバイスはハードウェアアップグレードが必要でソフトウェアでは解放できません。新イヤホン購入時に LE Audio/Auracast 対応表示を確認しましょう。

## 運転中のスマホ操作は違法で、命に関わります。しかしナビ、電話、音楽が必要な時もあります。この時に CarPlay（Apple）と Android Auto（Android）が役立ちます。

接続方法

- **：車が対応していれば（近年のほとんどの新車が対応）、ケーブルでスマホを車の USB ポートに接続、カーディスプレイが自動的に CarPlay または Android Auto 画面に切り替わります。一部の新車はワイヤレス接続に対応、さらに便利。**できること
- **：ナビ（カーナビよりはるかに使いやすい）、電話（ハンドルで応答）、音楽（スマホの音楽 App を直接車に投影）、音声 WeChat（Siri や小愛がメッセージの読み上げと返信をサポート）。**古い車でも利用可能
- **：車が古く対応していない場合、数百元で外部 CarPlay ディスプレイを買い、ダッシュボードに貼り付けば、体験はほぼ同じ。**七、連携中のセキュリティとプライバシー

## デバイスが多く連携されるほど、プライバシー漏洩のリスク面が拡大します。いくつか注意点：

公共の場でキャストする前、スマホの通知バーにプライベートメッセージがないか確認、家族に見せるべきでない内容を見せないように。

- デバイスを他人に譲渡または売却する前、連携解除を——スマホ設定でそのデバイスを見つけ、「削除」または「ペアリング解除」を選択。
- スマートホームカメラは、ブランド品で暗号化対応のものを購入、数十元の無名品は避ける。カメラがハッカーに生配信されたニュースは多すぎます。
- やってみよう：スマホ画面をテレビにキャストし、スマートシーンを一つ設定

**スマホとテレビが同じ Wi-Fi に接続されていることを確認。**

1. テレビで「画面ミラーリング」または「AirPlay」機能を見つけてオン。
2. スマホで：iPhone はコントロールセンターから「画面ミラーリング」をタップ。Android は下ろして「ワイヤレスキャスト」を探す。
3. お気に入りの写真を一枚テレビにキャストし、家族に見せましょう。
4. スマートホームデバイスがあれば、スマホ App で「外出モード」を作成してみる：ワンタップで照明オフ、エアコンオフ、ロボット掃除機起動。
5. 第十八章：データバックアップと移行

**事例 1：バッテリー持ち改善の実戦**

- **シーン**: ユーザーの Xiaomi 14 バッテリー持ちが悪く、半日で切れる。
- **手順**: (1) 設定バッテリー消費ランキング確認 (2) 異常なバックグラウンド App を特定 (3) 5G を 4G に切替 (4) バックグラウンド制限 (5) 省電力モード有効化 (6) 充電時にケースを外して放熱。
- **期待される結果**: バッテリー持ちが 30-50% 延長、バッテリー劣化度データが読み取り可能。

# 第十八章：データバックアップと移行

一、まず確認：スマホ内のどれがなくなると本当に消えるのか

## 異なるデータの「生存状態」は異なり、区別する必要があります：

スマホローカルにあり、なくなると本当に消えるもの

- **：写真動画（クラウド同期していない場合）、メモ、SMS、通話履歴、ダウンロードしたファイル。**App サーバーにあり、機種変更して再ログインすればあるもの
- **：WeChat の友達リスト、Taobao の注文、銀行 App の取引記録。これらはスマホ内ではなく対応する企業のサーバーに保存されています。**グレーゾーン、特別な操作が必要なもの
- **：WeChat チャット履歴（WeChat サーバーに保存されていますが、自分でサーバーからダウンロードできません）、スマホバンキングのデバイス証明、2 段階認証 App 内のシークレット（Google Authenticator など）。**だから重点的に保護すべきは、第一類と第三類です。

二、全体バックアップ：一度にすべて対応

## 全体バックアップはスマホの完全な「スナップショット」を撮るようなもの——現在のシステム設定、デスクトップレイアウト、App データ、写真、連絡先、すべてパッケージ化。

方案一：パソコンローカルバックアップ（最も信頼できる、パソコン保有者に推奨）

### iPhone

- **iPhone:** ケーブルでスマホをパソコンに接続し、iTunes（旧 Windows 版）または Finder（Mac）を開いて、デバイスページで「今すぐバックアップ」を選択。「バックアップを暗号化」にチェックしないと、パスワードやヘルスケアデータはバックアップされません。

- **Android** パソコンバックアップの利点はクラウド容量を買う必要がなく、復元速度がクラウド復元よりはるかに速い。欠点は定期的にケーブル接続してバックアップする必要がある——ほとんどの人ができない。
- 方案二：クラウドバックアップ（最も手間いらず、すべての人に推奨）

**iPhone**

- **iPhone:** 設定ユーザの名前 iCloud iCloud バックアップオン。電源接続 • Wi-Fi 接続 • ロック画面の状態で自動バックアップされます。無料の 5GB ではまず足りないので、有料の 50GB プランへのアップグレードがおすすめ。
- **Android** クラウドバックアップの最大の利点は「気にしなくていい」、自動で完了。最大の欠点は復元時にダウンロードに長時間かかり、無料容量は通常足りない。
- 三、分類バックアップ戦略：重要なものは別途保存

## 全体バックアップは最後の砦ですが、一部のものは重要すぎて別途保存する価値があります：

写真と動画（最も貴重なデジタル資産）

### 二重保険方案

- **：iCloud 写真/Google フォトの自動同期をオンにしつつ、数ヶ月おきに手動で写真をパソコンや外付け HDD にコピー。クラウド 1 部、ローカル 1 部、相互にバックアップ。**お金を使いたくない方案
- **：Google フォトは無料で 15GB 提供、「圧縮画質」に設定して容量節約可能。または 1 ヶ月おきにケーブルで写真をすべてパソコンにコピー。**連絡先（見落としがちだが極めて重要）

**iPhone 連絡先はデフォルトで iCloud と同期、同期をオンにしていれば追加バックアップ不要。ただしより安全にするなら、iCloud.com で vCard ファイルを書き出してパソコンに保存。**

- Android 連絡先は通常 Google またはブランドクラウドと同期。連絡先 App の設定で.vcf ファイルを書き出せば、完全なバックアップになります。
- WeChat チャット履歴（個別処理、最重要）

**これは個別に説明する必要があります、全体バックアップでは WeChat チャット履歴は多くの場合不完全で、機種変更ツールでは全く移動できないから。**

方法一：新スマホに移行

- 。**WeChat マイ設定一般チャット履歴の移行とバックアップ移行スマホ/タブレットに移行。新旧スマホを同じ Wi-Fi に接続し、QR コードをスキャンして転送。注意：移行完了まで旧スマホの WeChat をログアウトしない。**方法二：パソコンにバックアップ
- 。**パソコンに WeChat PC 版をインストール、左下の三本線移行とバックアップバックアップと復元チャット履歴をパソコンにバックアップ。機種変更後に新スマホに復元可能。**方法三：クラウドにバックアップ
- 。**WeChat は iCloud やスマホクラウドスペースへのバックアップにも対応しますが、注意：このバックアップは現在のスマホでのみ復元可能、クロスデバイス復元不可。機種変更する場合は、前 2 つの方法を使う必要があります。**四、機種変更ガイド：旧スマホから新スマホへ

## 新スマホへの変更は嬉しいことですが、データが移行できないと頭痛に。3 つのケースをそれぞれ対応：

同ブランドアップグレード（最も簡単）

**iPhone 新しい iPhone**

- **：起動アクティベーション時に旧スマホを横に置くと、「クイックスタート」が自動ポップアップ、旧スマホで新スマホのパターンをスキャンするだけ。すべてのデータ、設定、App レイアウトがそのまま移行、Wi-Fi パスワードも自動的に引き継がれます。約 20□30 分。**Android 同ブランド
- **：ブランド標準の機種変更ツールを使用。Huawei は「スマホクローン」、Xiaomi は「Xiaomi 機種変更」、OPPO は「スマホ引越し」、vivo は「相互転送」。手順はほぼ同じ：新旧スマホに同じ App をインストール、QR コードで接続、移行したいものを選択、転送開始。**クロスブランド機種変更（Android 間、異ブランド）

**サードパーティの機種変更ツールまたは旧ブランド標準の機種変更 App を使用（ほとんどのブランドの機種変更 App は他の Android スマホへの移行にも対応）。移行できるもの：連絡先、写真、SMS、通話履歴、一部 App。移行できないもの：App 内のログイン状態（再ログイン必要）、WeChat チャット履歴（WeChat 独自の移行機能を使用）。**

クロスプラットフォーム機種変更（AndroidiPhone、またはその逆）

**Android iPhone**

- **：Apple の「iOS に移行」App があり、Android スマホでダウンロード後、新 iPhone アクティベーション時に「Android デバイスからデータを移行」を選択。連絡先、写

**真、SMS などの基本データは移行できるが、App データは移行不可。WeChat チャット履歴は WeChat 独自の移行機能を使用。**iPhone Android

- **：各 Android ブランドに対応する移行ツールがありますが、正直クロスプラットフォーム移行体験は同プラットフォームほど良くない。最も重要なことを先に個別に：連絡先書き出し、写真バックアップ、WeChat チャット履歴移行。残りはゆっくり。**五、バックアップの黄金の法則：3-2-1 原則

## これはデータセキュリティ分野の古典的原則で、シンプルで覚えやすい：

3

- **：重要なデータは少なくとも 3 部保持（原本 1 部、バックアップ 2 部）。**2
- **：バックアップは 2 種類の異なるストレージメディアを使用（例：クラウド 1 部、外付け HDD1 部）。**1
- **：少なくとも 1 部のバックアップを異なる物理的位置に配置（火災、盗難などで同時にすべてのデータが失われるのを防止）。**一般人にとっては、「クラウド 1 部、パソコン 1 部」で既に十分。複雑にする必要はなく、本当に実行することが重要。

WeChat チャット履歴の特別な注意

**WeChat チャット履歴はどのスマホ機種変更ツールでも移行できません！全体バックアップのカバー範囲外です。WeChat 標準の「チャット履歴移行」機能を使用するか、パソコン版 WeChat にバックアップする必要があります。多くの人が新スマホに機種変更した後にチャット履歴がすべてなくなったことに気づき、その時はもう手遅れ——旧スマホは既に出荷時リセットされている可能性。**

練習：完全なバックアップ訓練を実施

**iCloud クラウドバックアップまたはブランドクラウドバックアップが有効か確認、前回のバックアップ時間を確認。**

1. 手動で写真をバックアップ：ケーブルでパソコンに接続、DCIM と Pictures フォルダをパソコンにコピー。
2. 連絡先をバックアップ：iPhone ユーザーは iCloud.com で vCard を書き出し。Android ユーザーは連絡先 App の設定で.vcf ファイルを書き出し。
3. WeChat チャット履歴をバックアップ：WeChat を開くマイ設定一般チャット履歴の移行とバックアップパソコンにバックアップ。
4. 書き出した連絡先 vcf と写真フォルダを外付け HDD や USB メモリにコピーし、「3-2-1」の「1 部を別の場所にオフラインで」を完了。

5. 第十九章：トラブルシューティングと自己救助

# 第十九章：トラブルシューティングと自己救助

一、スマホが重い、発熱——最もよく聞かれる問題

**正直、スマホを 1 年ほど使うと重くなるのは正常です。原因は必ずしもスマホの故障ではなく、多くの場合「埃溜まり」——ゴミファイル、バックグラウンドプロセス、ストレージ容量が徐々に埋まる。**

まず診断、急いで修理しない：

ストレージ容量を確認

1. **：設定ストレージで残容量を確認。10% 未満だとシステムは非常に重くなります、場所を空けようと必死だから。私の経験では：15%-20% 以上の残容量を保てば、スマホは比較的スムーズに動作。**消費電力ランキングを確認

2. **：バッテリー設定で過去 24 時間の消費電力ランキングを確認。ある App が大部分を占めているのにほとんど使っていない場合、バックグラウンドで勝手に動いています。アンインストールまたはバックグラウンド活動を制限。**最近何をしたか思い出す

3. **：システムアップデート後に重くなった？（大型バージョン更新後の最初数日は重くなるのが正常、システムがバックグラウンドでインデックス再構築中）。それとも新しい App をインストール後に重くなった？（アンインストールして試す）。**使用環境を確認

4. **：スマホを太陽の下に放置、分厚いケースを付けて充電しながらゲームをすると、発熱は必然。高温は CPU の自動クロックダウンを引き起こし、スマホが突然重く感じます。冷ましてから再試行。**治療法（簡単なものから複雑なものへ）

**：スマホを再起動（60% の問題を解決）ストレージ容量をクリア（不要な App と動画を削除）ある App が問題ならキャッシュをクリアまたはアンインストールして再インストール最後の手段：データバックアップ後に出荷時リセット。**二、アプリが落ちる、応答なし——App が言うことを聞かない

## ある App を開くとすぐ落ちる、または使っているうちに突然フリーズする。処理フローは医師の問診のように、一歩ずつ：

まず閉じて再度開く

1. ：バックグラウンドから App をスワイプで消去し、再度開く。ほとんどの一時的バグはこれで解決。App をアップデート
2. ：アプリストアでアップデートがあるか確認。App バージョンが古すぎてシステムと互換性がない場合があります。権限を確認
3. ：一部の App は特定の権限が必要でないと正常動作しませんが、付与していません。例えばカメラ App にカメラ権限を付与していないと開くとすぐ落ちる。設定でその App を見つけ、必要な権限をすべて付与。キャッシュをクリア
4. ：設定アプリ管理その App を見つけるストレージキャッシュをクリア。注意:「キャッシュをクリア」であって「データをクリア」ではない!「データをクリア」は App 内のログイン状態と設定をすべて削除、App を出荷時状態にリセットします。アンインストールして再インストール
5. ：最後の手段。アンインストール前にこの App のデータが失われないか確認——WeChat なら先にチャット履歴をバックアップ、ゲームならアカウントが携帯番号に紐付いているか確認。三、充電できない、充電が遅い——バッテリーのせいではないかも

## 充電できないのは特に焦りを引き起こす問題ですが、多くの場合原因はアクセサリにあり、スマホの故障ではありません。

まず充電ポートを確認

- ：爪楊枝や非導電の小工具で、充電ポートの埃や綿ぼこりを優しく取り出す。私自身も経験——ポケットから落ちた糸くずが充電ポートに詰まり、挿すと接触不良かと思ったが、取り出したら直った。ケーブルと充電ヘッドを交換して試す
- ：ケーブルの破損率は充電ヘッドよりはるかに高い、特に頻繁に巻くケーブル。問題ないと分かっているケーブルと充電ヘッドに交換してクロステストすれば、どのアクセサリが壊れたかすぐ特定できる。ワイヤレス充電の問題
- ：ワイヤレス充電を使用している場合、スマホケースが厚すぎないか、ケース裏に金属片（マグネットスタンドなど）がないか確認。また、スマホを充電パッドの中央に正確に置く——ずれると充電できない。充電が遅い
- ：充電ヘッドの電力が十分か確認（iPhone の急速充電には少なくとも 20W の PD 充電ヘッドが必要）、ケーブルが急速充電対応か（すべてのケーブルが 3A 以上の電流

に対応しているわけではない）、スマホが熱すぎないか（熱すぎると自動的に出力を下げてバッテリーを保護）。四、ネットワークに接続できない、Wi-Fi がよく切れる

## 外でモバイルデータは正常だが、家に帰ると Wi-Fi に接続できない。または Wi-Fi に接続しているがネットに繋がらない。この順序で確認：

機内モードのオン/オフ

1. ：コントロールセンターを下ろし、機内モードをオンにして 5 秒待ってからオフに。これはスマホにネットワークを再検索させるもので、非常に効果的。Wi-Fi を「忘れる」て再接続

2. ：Wi-Fi 設定で、ネットワーク名の横の感嘆符（Android は歯車）をタップし、「このネットワークを無視」または「忘れる」を選択、再度パスワードを入力して接続。ルーターを再起動

3. ：多くの家庭のルーターは数ヶ月電源を切っておらず、時間が経つと不安定に。電源を抜いて 30 秒待ってから再挿入、ほとんどの Wi-Fi 問題が解決。スマホの問題か確認

4. ：他のデバイスが同じ Wi-Fi に正常に接続できるか？他は正常で自分のスマホだけダメなら、スマホの再起動またはネットワーク設定のリセットが必要（設定一般転送またはリセットネットワーク設定をリセット、データは削除されないが、すべての Wi-Fi パスワードはクリアされる）。五、画面問題：タッチ反応不良、画面ちらつき、残像

## タッチ反応不良

- ：まず強化ガラスフィルムを剥がして、フィルムのせいか確認——安い強化ガラスフィルムは長く使うとタッチに影響。画面に水や手に油があるとタッチがずれることも。どちらでもなければ、再起動して再試行。画面に跡/残像（焼き付き）

- ：長時間同じ画面を表示（例えばずっと画面を点灯して Douyin を見る）と、OLED 画面に「残像」が残ります。これは画面故障ではなく、一時的。画面を消して 30 分休めば通常回復。画面ちらつき

- ：低輝度でちらつく場合、PWM 調光の問題の可能性が高く、OLED 画面で多い。設定で「DC 調光」や「フリッカー防止」オプションを探し、オンにするとかなり改善。すべてのスマホにある機能ではありません。画面の一部が完全に黒/白

- ：これは比較的深刻で、画面の物理的損傷の可能性が高く、画面交換が必要。非公式修理の画面は、色と輝度が通常純正に劣り、防水にも影響。六、起動できない、Logo 画面で止まる——まず焦らない

## この状況は最も心拍数を上げますが、冷静に、まずこれらを試しましょう：

まず 30 分充電してから

- ：多くの場合、単にバッテリーが完全に切れているだけ。純正充電器を挿し、少なくとも 30 分待ってから起動を試みる。一部のスマホはバッテリー切れ後、一定量まで充電されないと反応しません。強制再起動
- ：全面屏 iPhone は音量上げ音量下げ電源キー長押しで Logo 表示まで。Android は電源キー 10□15 秒長押し、振動まで。起動画面でループ再起動
- ：この状況は「白リンゴ」（iPhone）または「Logo 卡まり」（Android）と呼ばれます。強制再起動が効かない場合、リカバリーモードに入ってパソコンで修復が必要。iPhone をパソコンに接続し iTunes/Finder を開き、リカバリーモードに入った後「アップデート」を選択（「復元」ではない）、システム再インストールでデータ保持。Android は Fastboot または Recovery モードへ、各ブランドでキーの組み合わせが異なるので、「スマホ型番＋リカバリーモード入り方」を検索推奨。上記すべて効かない
- ：専門修理に頼るしかありません。データをバックアップしていれば、あまり心配不要。七、音声問題：スピーカーが鳴らない、通話音が小さい

### スピーカーから音が出ない

- ：まずマナーモード（iPhone 側面のサイレントスイッチ）、おやすみモード、音量が最低になっていないか確認。馬鹿げているように聞こえるが、この状況が 40% を占めます。次に Bluetooth がイヤホンやスピーカーに接続していないか確認。通話相手に声が聞こえない
- ：マイク穴が塞がれていないか確認——スマホ底部の小穴は埃で塞がれやすい。柔らかいブラシで優しく掃除。針や爪楊枝で強く突かない、防塵ネットを破損。音が割れる/ノイズ
- ：スピーカーの浸水が原因の場合——ドライヤーの熱風を当てない！ティッシュで表面の水分を吸い取り、自然乾燥。ネットでは音波排水 App（低周波音を再生して水を振動で飛ばす）がある程度効果的と言われており、試す価値あり。八、公式修理を探す、落とし穴を避ける

## 上記の方法をすべて試してもダメなら、専門修理に頼る必要があります。しかし修理市場は水深が深く、私自身の失敗経験と教訓を：

本物の公式を見つける、「公式っぽい」ではなく

- **：ブランド公式サイト（Apple.com.cn、Huawei.com.cn など）で「認定サービスプロバイダー」リストを探す。街角で Apple ロゴを掲げる非認定店は、純正アクセサリを使っていない可能性があり、修理後に防水がなくなります。**修理に出す前に必ずやること

- **：すべてのデータをバックアップ（修理中にデータが失われないことを期待しない）、すべての重要なアカウントをログアウト（Apple ID、WeChat、Alipay）、「iPhone を探す」をオフ。一部の修理店はロック画面パスワードの提供を要求しますが、プライバシーリスクあり——自分で先にアカウントをログアウトしてから持ち込むのが良い。**画面交換の落とし穴

- **：非公式画面は「原圧画面」（純正ガラス交換）、「後圧画面」（非純正ガラス）、「組立画面」（全非純正）に分かれる。価格は数倍違うが、表示効果と耐久性も大きく異なる。スマホがまだ保証期間内なら、大人しく公式ルートを。**バッテリー交換の落とし穴

- **：非純正バッテリーは容量を偽装している可能性があり、半年で純正バッテリーの 2 年分の劣化を示す。Apple バッテリー交換は公式価格が非公式よりそれほど高くなく、公式推奨。**データ復元の落とし穴

- **：スマホの浸水/落下破損後、修理店が「データ復元は追加料金」と言う場合、警戒——一部の悪徳業者はわざと損傷程度を誇張して多く請求。正規のデータ復元は先に見積もりしてから操作、復元失敗は無料。**万能の第一の手段：スマホを再起動

どんな問題に直面しても——動作が重い、アプリが落ちる、ネットワークに繋がらない、音声異常——まずスマホを再起動。これは適当な対応に聞こえるが、私自身の経験では、少なくとも 6 割の小さな問題は再起動で解決。スマホもパソコンと同じで、長く動作するとメモリに様々なゴミが蓄積し、再起動はクリアしてやり直すこと。だから次にスマホに問題が起きたら、急いで「どうしよう」と検索する前に、再起動してから確認を。

今すぐ覚える：これらの救命操作は完全に暗記すべき

**強制再起動**

1. **：全面屏 iPhone は音量上げ音量下げ電源キー長押しで Logo 表示まで。Android は電源キー 10□15 秒長押し、振動まで。**充電ポートを掃除

2. **：小工具（爪楊枝や SIM 取り出しピン）で、充電ポートの埃を優しく掃除。**Wi-Fi 再接続

3. **：Wi-Fi 設定で現在の Wi-Fi を「忘れる」、再度パスワードを入力して接続。**機内モードのオン/オフ

4. **：コントロールセンターを下ろし、機内モードをオン、心の中で 5 秒数え、オフにしてネットワークが復帰するか確認。**これらの操作手順をスクショで保存するか、メモに書く。問題発生時に慌てて検索する必要がなく、直接メモを見れば OK。

5. 第二十章：開発者モードと高度な権限

# 第二十章：開発者モードと高度な権限

一、開発者モード：隠し設定のアンロック

## 一、開発者モード：隠し設定のアンロック

有効化方法：

### 有効にする方法：

- **Android**：設定端末情報「ビルド番号」を 7 回連続タップし、「開発者モードになりました」と表示されるまで操作。
- **HarmonyOS**iOS
- **iOS** よく使う機能：

### よく使う機能一覧：

- **：パソコンに接続して高度な操作、フラッシュやデータ復元など。**：PC に接続して高度な操作（フラッシュ、データ復元、ADB コマンドなど）を行うために必ず有効化するスイッチ。本章の「ADB 実用コマンド」小節を参照。
- **：画面にタッチポイントを表示、チュートリアル録画に適する。**：画面にタッチポイントを表示（小さな丸が指に追従）。操作チュートリアルやデモの録画に適しています。

### パフォーマンスと快適さ関連：

- **アニメーション速度調整（ウィンドウアニメーションスケール / トランジションアニメーションスケール / アニメーション時間スケール）**：これら 3 つのオプションは App を開く時やページ切り替え時のアニメーション時間を制御します。デフォルトは 1x。0.5x にするとスマホが「見た目上」速くなります（実際の処理速度は変わらず、アニメーションが速くなるだけ）。完全にオフにするとアニメーションなしで直接ジャンプします。多くのギークは 0.5x に設定し、遷移感を残しつつ速さを演出。完全オフは操作がぎこちなくなり、本来のカクつきを隠して「とても滑らか」と誤認させる原因にもなります。
- **GPU レンダリングの強制（Force GPU Rendering）**：すべての 2D 描画を CPU ではなく GPU に強制的に処理させます。旧機種や最適化の不十分な App でスクロールの滑らかさが向上することがありますが、GPU 負荷と消費電力も増加します。現代のスマホはデフォルトで GPU レンダリングを多用しているため、このスイッチは中高級機種への影響は限定的で、歴史的な名残のオプションです。一般ユーザーはデフォルトのままで問題ありません。

- **GPU レンダリングプロファイリングの表示（Profile GPU Rendering）**：画面上に棒グラフで毎フレームのレンダリング所要時間をリアルタイム表示。緑色の横線は16.6ms（60fps のフレーム時間閾値）を表します。棒が緑線を超えるとフレームドロップが発生しています。これは開発者が App のカクつきを調査する強力なツールで、一般ユーザーもどの App の最適化が不十分かを比較できます——例えば、ある App のスクロール時に棒が頻繁に赤線を超えるなら、ほぼその App が原因です。

**バックグラウンドとバッテリー持ち関連：**

- **：バックグラウンドで動作するアプリ数を制限し、省電力。**：「標準制限」「最大 1 プロセス」「最大 2 プロセス」などが設定できます。一見バックグラウンドプロセスを制限すると省電力になりそうですが、実際には Android のバックグラウンド管理は既に十分機能しており、強制制限は App の頻繁な再読み込みを引き起こし（コールドスタートはかえって消費電力が多く、体験も低下します）。このオプションはデバッグ向けで、一般ユーザーが長期有効にすることは推奨されません。「標準制限」のままで問題ありません。
- **アクティビティを保持しない**：画面から離れるたびに即座に破棄します。非常に非推奨。App が頻繁にホワイトスクリーンになり、再読み込みが発生します。App の復元ロジックのテスト用のみ。

**注意事項**

二、BL ロック（Bootloader ロック）：フラッシュの第一の門

## 二、OEM ロック解除：開発者オプション内の「フラッシュスイッチ」

開発者オプションの中に目立たないが重要なスイッチがあります——「OEM ロック解除」（一部ブランドでは「OEM アンロック許可」や「デバイスアンロック」と呼ばれます）。このスイッチ自体は BL ロックを解除しませんが、「事前許可」です：これをオンにして初めて、メーカーが fastboot コマンドやブランドツールで BL の解除を許可します。

- **スイッチの役割**：メーカーに「私は保証を放棄し、自己責任で BL の解除を許可してほしい」と宣言するようなものです。このスイッチがオフの場合、BL 解除コマンドはすべて直接拒否されます。
- **BL ロックとの関係**：OEM ロック解除は「資格申請」、BL 解除が「実際の操作」です。前者はスマホの設定でタップするだけ、後者はメーカーの公式サイトで解除コードを申請し、PC で fastboot コマンド（例：`fastboot flashing unlock`）を実行する必要があり、両方が不可欠です。
- **初期化するとリセットされるか**：はい。「OEM ロック解除」スイッチは初期化後にオフに戻りますが、BL ロックは一度解除すると物理レベルの状態となり、初期化では再ロックされません——手動で `fastboot flashing lock` を実行して再ロックする必要があります。

- **いつオンにすべきか**: フラッシュや Root を確実に行う場合を除き、オフのままにしてください。「USB デバッグ」とは独立したスイッチで、フラッシュ時は通常両方をオンにします。

**豆知識：なぜメーカーはこのスイッチを設けるのか**

初期の Android にはこのスイッチがなく、マルウェアは権限を取得すれば密かにシステムをフラッシュできました。Google は後にすべての Android メーカーに対し、開発者オプションに「OEM ロック解除」スイッチを追加し、「ユーザーの知情同意」の確認とすることを要求しました。これが、BL ロックを解除していても OEM ロック解除スイッチがオフなら再フラッシュが失敗する理由です。

## 三、BL ロック（Bootloader ロック）：フラッシュの第一の門

BL ロックの役割：

### セキュリティ保護

- **：マルウェアのシステムフラッシュを防止。**盗難防止
- **：スマホがフラッシュされても、元のアカウントパスワードがなければ使用不可。**保証制限
- **：BL アンロックは通常、公式保証の喪失を意味します。**各ブランド BL ロックポリシー：

### 各ブランドの BL ロック方針：

- **Xiaomi：**ロック解除可能。アカウント連携と待機期間（7　30 日）が必要。
- **Huawei**OnePlus
- **：アンロック可能、比較的緩和。**OPPO/vivo
- **：一部機種はアンロック可能、申請が必要。**Samsung
- **Samsung**Apple
- **Apple** 三、Root 権限：Android の「スーパーアドミン」

## 四、ADB 実用コマンド：PC からスマホを操作

ADB の正式名称は Android Debug Bridge で、Google が公式提供するコマンドラインツールです。PC から USB または Wi-Fi 経由で Android スマホと通信できます。Root 不要で、スマホの「USB デバッグ」がオンなら使えます。「PC からスマホに指示を出すリモコン」と理解してください——フラッシュ、App インストール、ファイル転送、ログ確認にすべて必要です。

**USB で接続する方法：**

1. スマホ側：開発者オプション「USB デバッグ」をオン。
2. PC 側：Google 公式から Platform Tools（adb3. タスク（Task）：AI に何をするか明確に伝える fastboot の 2 つの実行ファイルを含む）をダウンロードし、任意のディレクトリに解凍。
3. データケーブルでスマホと PC を接続。スマホに「USB デバッグを許可しますか?」のポップアップが表示されるので、「常に許可」にチェックを入れて確定。
4. PC のコマンドライン（Windows の PowerShell や cmd）で `adb devices` を実行し、`List of devices attached /xxxxxxxx device` のような出力があれば接続成功です。

**ドライバーと認証ポップアップを見逃さないで**

Windows ユーザーで `adb devices` が「unauthorized」と表示されるかデバイスがリストされない場合、多くはドライバーが未インストールかスマホで「許可」を押し忘れたのが原因です。Samsung/Xiaomi などのブランドは各社の「USB ドライバー」や「スマホアシスタント」をインストールしてドライバーを補完できます。Mac と Linux は通常ドライバー不要です。スマホのポップアップを誤って消してしまった場合は、ケーブルを挿し直すと再表示されます。

**よく使う ADB コマンド：**

- `adb install app.apk`: PC 上の APK をスマホにインストール（`-r` は上書きインストールでデータ保持；`-s` は SD カードにインストール）。
- `adb uninstall com.xxx.xxx`: パッケージ名で App をアンインストール。
- `adb push PC パス /sdcard/目標パス`: PC のファイルをスマホにプッシュ。
- `adb pull /sdcard/ファイル PC パス`: スマホのファイルを PC に取り込む（WeChat で自分に送ってダウンロードするよりはるかに速い）。
- `adb shell`: スマホの Linux Shell に入り、直接 `ls`、`pm`、`am` などのコマンドを実行できます。
- `adb reboot bootloader`: スマホを Fastboot モードで再起動（フラッシュの入口）。
- `adb reboot recovery`: Recovery モードで再起動。
- `adb shell pm list packages`: スマホにインストールされたすべての App のパッケージ名をリスト（パッケージ名検索に必須）。`-s` を付けるとシステム App のみ、`-3` を付けるとサードパーティ App のみ。
- `adb shell pm uninstall --user 0 パッケージ名`: 現在のユーザーのプリインストール App をアンインストール。Root 不要。下記の実戦を参照。

**Root 不要でプリインストール App をアンインストールする実戦：**

これは ADB の最も実用的な機能の一つです：`pm uninstall --user 0` コマンドで「現在のユーザー」のプリインストール App をアンインストールでき、Root 不要でシステムパーティションにも影響しません。アンインストールされた App はシステムから「消滅」しますが、実際にはシステムパーティションに残っており、初期化すれば戻ります。

1. まず `adb shell pm list packages -s` で全システム App をリストし、アンインストールしたいパッケージ名を見つけます。例えば Xiaomi ブラウザは `com.android.browser`、Huawei アプリストアは `com.vmall.client`。
2. パッケージ名がどの App か不明な場合は、PC で `adb shell pm list packages | findstr` キーワードで検索するか、サードパーティ App「Package Viewer」をインストールしてスマホ上で各 App のパッケージ名を直接確認できます。
3. アンインストールを実行：`adb shell pm uninstall --user 0` パッケージ名、例えば `adb shell pm uninstall --user 0 com.android.browser`。戻り値が `Success` なら成功です。
4. 復元したい場合は？2 つの方法：(1) 初期化（最も徹底だが面倒）；(2) 再インストール：`adb shell cmd package install-existing` パッケージ名。

**システムコアパッケージをむやみに削除しないでください**

「pm uninstall –user 0」は Root 不要の安全な方法ですが、重要システム App（`com.android.systemui` ステータスバー、`com.android.phone` 電話、`com.android.settings` 設定など）をむやみに削除すると、システムクラッシュ、ブラックスクリーン、起動不能などの深刻な問題を引き起こします。削除前に「このパッケージ名は削除できるか」を検索し、少なくとも間違えないようにしてください。一度に 1-2 個だけ削除し、問題ないことを確認してから続行することを推奨します。

**豆知識：–user 0 とは何か**

Android 4.2 以降はマルチユーザーをサポートし、各ユーザーに独立した ID があります（機主は 0、追加ユーザーは 10 から開始）。`--user 0` は「現在の機主のみアンインストール」を意味し、システムパーティション内の APK ファイルは依然存在し、機主に表示されなくなるだけです。これが Root 不要の理由——システムファイルは変更せず、「ユーザー設定」を変えただけです。Root 後のアンインストールとの最大の違いもここにあります：Root アンインストールはシステムから本当に削除し、全ユーザーから消えますが、ADB アンインストールは「現在のユーザーに非表示」にするだけです。

## 五、Root 権限：Android の「スーパー管理者」

Root 権限は Linux のスーパーユーザーに相当し（/ ディレクトリに対して完全な制御権を持ち、任意のファイルの読み書きが可能）、Android システムの最高権限です。Windows の Administrator とは本質的な違いがあります：Windows はセキュリティ上の理由から管理

者に多くの制限を設けており（真の最高権限は TrustedInstaller）、Administrator であっても Linux root のように直接 `sudo rm -rf /*` を実行できません。Root 権限を取得すると、システムファイルの変更、プリインストール App のアンインストール、Root 必要な高度な App の使用が可能になります。

**アンインストールできないプリインストールアプリを削除。**

- 高度な機能 App の使用（Titanium Backup、Greenify など）。
- システムインターフェースと機能の修正。
- サードパーティ Recovery とカスタム ROM のインストール。
- Root のリスク：

**保証喪失**

- **ROM 焼きにはリスクがあり、操作には慎重さが必要です。次のような問題が発生する可能性があります：**セキュリティリスク
- **：マルウェア App が Root 権限を取得し、深刻な結果を招く可能性。**システム不安定
- **：誤操作でシステムクラッシュの可能性。**銀行 App 使用不可
- **：多くの銀行・決済 App は Root を検出すると実行を拒否。**OTA 更新失敗
- **：公式システム更新を正常に受信できない可能性。**深刻な警告

**Root 操作にはリスクがあり、スマホがレンガ化（起動不可）する可能性があります。初心者の場合、まず関連知識を十分に学び、完全なバックアップがあることを確認してください。操作前にチュートリアルを注意深く読み、あなたの機種とシステムバージョンを確認してください。**

四、一般ユーザーへのアドバイス

## 六、SafetyNet と Play Integrity 認証：なぜ Root 後に銀行 App が使えないのか

多くの Root ユーザーがこんな絶望的瞬間を経験しています：Root 後、意気揚々とモジュールをインストールしたのに、銀行 App を開くと——「お使いのデバイスは安全でないため使用できません」。Google Pay も使えず、Netflix すら高画質再生を拒否されます。その背後にあるのは Google の 2 つのデバイス完全性認証メカニズム：旧式の SafetyNet と新型の Play Integrity API です。

**SafetyNet / Play Integrity とは：**

- **SafetyNet Attestation（旧版、2017 年□）**：Google が早期に発表したデバイス完全性チェック API。App はこれを通じて Google サーバーに「このデバイスは CTS 互換

性テストに合格しているか、未変更の公式状態か」を問い合わせます。戻り値は Basic Attestation（基本検証）と Strong Attestation（強力検証、ハードウェアキー依存）に分かれます。Google は SafetyNet を 2024 年以降順次廃止すると発表しています。

- **Play Integrity API（新版、2022 年□）**：Google が発表した代替方案で、SafetyNet より厳格かつ包括的です。デバイス完全性に加え、「アプリ完全性」（App が改ざんされていないか）と「アカウント/認可完全性」（ユーザーアカウントが Google に信頼されているか）も含みます。新規 App は Play Integrity に移行すべきです。

- **誰が使うか**: 銀行、金融、決済系 App（Google Pay、各銀行のネットバンキング、支付宝/WeChat Pay の一部シーン）、ストリーミング（Netflix の高画質著作権保護）、一部ゲームのアンチチート（Tencent/NetEase の一部ゲーム）、企業オフィス App（MDM デバイスコンプライアンスチェック）。

**Play Integrity の 3 段階認証結果：**

- **MEETS_DEVICE_INTEGRITY（デバイス完全性合格）**：デバイスは正規 Android システムを実行し、Bootloader は未解除、Root なし、CTS 互換性テスト合格。銀行・決済系 App を正常に使用できます。

- **MEETS_BASIC_INTEGRITY（基本完全性合格）**：デバイスは基本的に信頼できるが、軽微な異常の可能性あり。一部 App はダウングレード使用（Netflix は再生可能だが画質制限など）、厳格な銀行 App は直接拒否します。

- **MEETS_STRONG_INTEGRITY（強力完全性合格）**：最高レベル。ハードウェアキーの裏付けが必要（Google Pixel シリーズの Titan M チップなど）。ほとんどのサードパーティブランドは Root なしでもこのレベルに達しません。

- **ラベルなし（NO_INTEGRITY）**：デバイスが Root 済み、BL 解除済み、サードパーティ ROM フラッシュ済み、またはエミュレータと認識され、ほとんどの決済/銀行 App が直接実行を拒否します。

**なぜ Root 後にこれらの App が使えなくなるのか：**

App は Play Integrity API で Google サーバーに認証リクエストを送信し、Google はデバイスの複数指標をチェックします——Bootloader ロック状態、システムパーティションの変更有無、su バイナリの存在、SELinux が Permissive モードかなど。いずれかの異常が認証レベルの低下または空を引き起こし、銀行 App は「デバイスが安全でない」と判断してサービスを拒否します。

**現代の隠蔽方案：Magisk Hide / Shamiko / Zygisk：**

Root の利便性を享受しながら銀行 App も使えるよう、コミュニティは一連の「Root 状態隠蔽」方案を開発しました。核心的な考え方はすべて同じです：検出プログラムが見るのは変更されたシステムではなく、「正常に見える」デバイスにする。

- **：古参の Root 方案、現在は更新停止。**：現在最も主流な Systemless Root 方案——システムパーティションを変更せず、マウント方式で Root を実現します。この設計の利点は OTA 更新失敗の確率が低く、アンインストールが比較的クリーンで、後続のすべての隠蔽方案の基盤となることです。ただし Magisk の使用には BL ロック解除が依然として必要です。

- **Magisk Hide（開発終了）**: Magisk に初期内蔵されていた隠蔽モジュールで、指定 App に対して Root を隠せました。しかし 2021 年以降 Magisk 公式から削除されました。理由は Google が検出手法を継続強化し、双方が「いたちごっこ」に陥ったためです。

- **Zygisk**: Magisk 24.0+ で導入された新メカニズム。Riru に似て、Zygote プロセスへの注入でモジュール読み込みを実現し、システム直接変更より隠蔽性が高いです。Zygisk の有効化は現代隠蔽方案使用の前提です。

- **Shamiko**: 現在最も主流な隠蔽モジュール。Zygisk と連携し、指定 App（「リバースホワイトリスト」モード）に対して Root を隠蔽、Magisk 自身を隠蔽、Bootloader ロック状態を偽装します。Shamiko は LSPosed リポジトリまたは GitHub から配布されます。

- **Play Integrity Fix（osm0sis）**：Google Play Integrity 検出に特化した Magisk モジュール。デバイスのフィンガープリントを変更し、Google に未 Root の公式機種と認識させます。

**いたちごっこは永遠に終わらない**

Google は定期的に検出ルールを更新し、App メーカーも Play Integrity 検証をアップグレードします。今日使える隠蔽方案が明日認識されるかもしれません。もし銀行/決済 App に完全に依存している（日常送金、QR コード決済など）なら、最も確実な選択は依然として：Root しない、BL ロックを解除しない。隠蔽方案は「メイン機 Root なし + サブ機で Root 遊び」のプレイヤー構成により適しています。

## 七、一般ユーザーへのアドバイス

- **：有効化して、アニメーション加速などの機能を体験しても良いが、他の設定は変更しない。**：オンにして、アニメーション加速、GPU レンダリングプロファイリングなどの機能を体験できますが、理解できない他のオプションはむやみに変更しないでください。

- **OEM ロック解除と BL ロック**: フラッシュや Root の予定がなければ、両方のスイッチをデフォルト（オフ）のままが最も安全です。

- **ADB**: Root 不要の強力ツールで、一般でも使えます——例えば `pm uninstall --user 0` でプリインストール App をアンインストール。ただし削除前にパッケージ名が削除可能か必ず確認してください。

- **ワイヤレスシステムアップデート**: 一般ユーザーの Root は推奨しません。リスクが利益を上回ります。どうしても必要な機能がある場合は、Root 不要の代替方案を先に調べましょう（ADB でプリインストール削除、Shizuku で一部 Root 機能を代替など）。

設定 ＞ デバイス情報に入る。

1.「ビルド番号」を連続 7 回タップ、開発者モードが有効化されたと表示されるまで。
2. 設定に戻り、「開発者オプション」を見つける（通常は設定下部または「システム」内）。
3.「ウィンドウアニメーションスケール」、「トランジションアニメーションスケール」、「アニメーション時間スケール」を見つけ、すべて 0.5x またはオフに設定。
4. スマホが速くなったか感じてみる。
5. 第二十章付録：Termux——Android デバイス上のポータブル Linux ターミナル環境

# 第二十章付録：Termux——Android デバイス上のポータブル Linux ターミナル環境

開発者モードでもまだ「ハードコア」不够だと思い、スマホで完全な *Linux* コマンドラインを体験したいなら、*Termux* はまさにあなたのためにあります。*Root* 不要で、普通の *Android* スマホをポータブル *Linux* ワークステーションに変え、応用ユーザーがモバイル開発と運用保守を探求する強力なツールです。

## 一、Termux とは

Termux の核心的優位性は軽量性と高い拡張性にあり、開発、学習、システム管理など多くのシーンに適しています。標準的なソフトウェアパッケージと精密なターミナルエミュレーションをオープンソースソリューションに統合しています。

二、核心機能と特徴

## 二、核心機能と特徴

### 完全な Linux 環境

Termux**Bash のみ内蔵**という 1 種類の Shell のみです。Zsh や fish などの他の Shell を使いたい場合は、パッケージマネージャーで手動インストールが必要です。例：`pkg install zsh` プロテクトモード `pkg install fish`。また Termux は Vim、Nano、Emacs などのエディターをサポートし、ネイティブに近いコマンドライン体験を提供します。ユーザーは「pkg install」で 2000 以上のパッケージをインストールでき、GCC、OpenSSH、SQLMap などの開発・セキュリティツールも含まれます。

### 強力なパッケージ管理

セキュリティとネットワーク機能

### セキュリティとネットワーク機能

- **：SSH クライアントとサーバー機能をサポート、他デバイスへのリモート接続やスマホを SSH ホストにすることが可能。**暗号化ツール
- **：OpenSSL などの暗号化ツールを提供、データ暗号化とセキュア通信をサポート。**サーバー構築
- **：Nginx や Apache を設定して静的ウェブページサーバーを構築可能。**システム統合と拡張

### システム統合と拡張

- Termux:API プラグインでセンサー、カメラなど Android ハードウェア機能を呼び出し。
- Bluetooth キーボードと外部ディスプレイの接続をサポート、完全なマウスサポートあり。
- 三、プラグインエコシステム

## 三、プラグインエコシステム

プラグイン名

| 機能説明 | **Termux:API** |
| --- | --- |
| コマンドラインプログラムに Android API 機能を公開 | Termux: Boot |
| デバイス起動時のスクリプト実行を許可 | Termux: Float |
| フローティングターミナルウィンドウを提供 | Termux: Styling |
| ターミナルでカラースキームとフォントカスタマイズを有効化 | Termux: Tasker |
| Termux スクリプトを Tasker 自動化アプリと統合 | Termux: Widget |
| ホーム画面ウィジェットからスクリプト実行を許可 | 重要なヒント |

**Termux とそのプラグインは同じソースからインストールする必要があります、同じ sharedUserId を共有するため。異なるソース（F-Droid と GitHub など）からの混在インストールはインストールエラーになります。**

四、インストールと設定ガイド

## 四、インストールと設定ガイド

### インストール手順

1. **：公式チャネル（F-Droid または GitHub）から Termux をダウンロードインストール、Root 不要。**システム初期化
2. **：初回起動後「Installing bootstrap packages」の完了を待つ。**ストレージ権限の取得
3. **：**pkg update && pkg upgrade を実行してスマホファイルシステム権限を取得。ソフトウェアソースの更新
4. **：**pkg update && pkg upgrade`Python` をインストール**：**基本最適化

### 基本最適化

- **：**termux-change-repo コマンドでグラフィカルインターフェースからミラーソースを変更（清華大学ミラーソースを推奨）。ショートカットキー設定
- **：音量下キーが Ctrl を、音量上キーが Esc をエミュレート、入力効率を向上。**インターフェース美化
- **：Zsh + Oh My Zsh をインストール、テーマとプラグインを設定。**五、典型的な応用シーン

## 五、典型的な応用シーン

### モバイル開発テスト

サーバー運用保守

### サーバー運用保守

Linux 操作の学習

### Linux 操作の学習

リアルな Linux 環境を提供し、コマンドラインやシステム管理の学習に理想的なプラットフォームです。Shell スクリプト作成、Linux ファイル権限管理（例：`chmod 755` で rwxr-xr-x 権限を設定、`chown user: group` でファイル所有者を変更、`ls -l` で owner/group/other の

3 組 9 桁の権限を確認）などの基本コマンドを練習できます。これは Unix/Linux の古典的な「3 組 9 桁」権限モデル（owner / group / other × 読み / 書き / 実行）で、Windows NTFS の ACL アクセス制御リストとは完全に異なる体系であることに注意してください。

#### ネットワークセキュリティテスト

六、バージョンと互換性

## 六、バージョンと互換性

まとめ

**Termux は強力な Linux ターミナル体験を Android デバイスにもたらし、スマホをポータブルな開発・学習ツールにします。開発者、システム管理者、技術愛好家のいずれも、Termux を活用してモバイルデバイスで完全なコマンドライン体験を楽しみ、「ポケットの中の Linux」という理念を実現できます。**

やってみよう：スマホで最初の Linux コマンドを実行

**F-Droid から Termux をダウンロードインストール、初回起動で初期化完了を待つ。**

1. pkg update && pkg upgrade
2. を実行してパッケージを更新。`Python` をインストール：pkg install python
3. 。`python3 -c "print('Hello from Termux!')"`。
4. 四、超越編（第 21-26 章）この編ではより深いデジタル世界にご案内します。情報セキュリティから `AI` の原理、プログラミング思考からデジタル遺産まで、これらのコンテンツはあなたを真のデジタル市民にします。第二十一章：モバイルネットワークと情報セキュリティの深掘り

# 四、発展編（第 21-26 章）

# 第二十一章：モバイルネットワークと情報セキュリティの深掘り

通常のネットアクセス

**：あなたのスマホ公共ネットワーク対象サイト。中間のルーター、スイッチ、どの段階でもあなたが送信するデータを「覗き見」される可能性。特にカフェ、空港のような公共 Wi-Fi では、正直「裸走」状態。**

VPN 使用

- **：あなたのスマホ VPN 暗号化トンネル VPN サーバー対象サイト。データは暗号化カプセル化され、「覗き見」者は乱数しか見えない。また、対象サイトが見るのは VPN サーバーの IP アドレスで、あなたの本当の IP ではありません。**VPN で何ができるか？簡単に言うと：第一、
- **公共 Wi-Fi でのプライバシー保護**。第二、

地域制限の回避（**特定の国でのみ配信されているコンテンツの視聴など**）。**第三、**ネットワークプロバイダー（ISP）の閲覧習慣記録の防止。**ただし一言注意：**無料 VPN を使わないでください！**無料の背後はしばしばあなたのデータが彼らの「商品」であることを意味します。ある友人が無料 VPN を使ったら、スマホに謎の広告がポップアップし始め、後にその VPN が彼の閲覧データを広告業者に売却していたことが判明。だから VPN 選びは、有料の大手ブランド（NordVPN、ExpressVPN など）を選ぶか、使わないか。**また、VPN は万能ではありません。「転送中」のデータしか保護できず、対象サイト自体が安全でない、またはスマホにマルウェアが感染している場合、VPN は助けになりません。**二、公共 Wi-Fi の「甘い罠」——想像より危険**無料 Wi-Fi に接続するだけだろ、どれほどのリスクがあると思う？実話をお話しします：セキュリティ研究者が実験を行い、空港に「Starbucks_Free」という Wi-Fi ホットスポットを設置、1 時間足らずで百人以上が接続。実際にはこのホットスポットは研究者が自分で設置したもので、このホットスポットを経由するすべての

暗号化されていないトラフィック

## ——例えばあなたがアクセスした HTTP ウェブページの内容、TLS 未対応の App 通信——は彼らが傍受・閲覧できました。

ただし重要な前提：今日日常的に使っている WeChat、ネットバンキング、Alipay は、ほとんどが HTTPS 暗号化を採用。つまり、攻撃者がデータパケットを傍受しても、見えるのは乱数だけで、**チャット内容や決済パスワードを直接読み取ることはできない**。ただし彼らはあなたがどのサイトにアクセスしたか（ドメイン名）は依然として分かります。一部の古い App やウェブページがまだ HTTP 平文送信を使っている場合、内容は完全に露出。この攻撃を「中間者攻撃」（Man-in-the-Middle）と呼び、原理は攻撃者があなたと本当の Wi-Fi ルーターの間に「割り込む」こと。さらに恐ろしいのは、多くのスマホが「既知 Wi-Fi に自動接続」を設定しており、攻撃者が自分のホットスポット名をあなたが以前接続した Wi-Fi（例えば「ChinaNet」や「CMCC」）に設定するだけで、あなたのスマホが自動的に接続。

どう防ぐか？いくつかの実用的なアドバイス：**HTTPS を確認**：アドレスバーの先頭が https://で、余分な「s」は Secure を意味。パスワード入力が必要なサイトにアクセスする時、http://（s なし）を見たら、すぐに止まって入力しない！最近の主流ブラウザは非 HTTPS サイトに「安全ではない」警告を表示、このマークを見たら警戒を。

「自動接続」をオフ

- **：iPhone でも Android でも、Wi-Fi 設定で「自動参加」または「自動接続」をオフに。毎回手動でタップする方が、スマホに判断させるより良い。**スマホのモバイルデータで機密操作

- **：送金、ネットバンキングログイン、ID 番号入力……これらの操作は自分の 4G/5G モバイルデータで着実に行い、あの通信量を節約しようとしない。**使わない Wi-Fi を忘れる

- **：出張や旅行から戻ったら、以前一時的に接続したホテル Wi-Fi、空港 Wi-Fi をすべて「忘れる」。これで次回スマホが同名の偽ホットスポットに自動接続しない。**三、HTTPS はどうあなたを保護する？——SSL/TLS ハンドシェイク

- **先ほど HTTPS に言及しましたが、実際にどう動作するのでしょう？HTTPS サイトにアクセスするたび、ブラウザとサーバー間で「SSL/TLS ハンドシェイク」が行われ、全体は大体こうです：**ブラウザが「ねえ、サーバー、安全な通信したいんだけど、これが私の暗号化能力リスト（対応暗号化アルゴリズムなど）」と言う。

## サーバーが返答：「OK、あなたのリストからこの暗号化方案を選んだ、これが私のデジタル証明書、公開鍵が含まれ、某某 CA 機関が署名認証」。

ブラウザが証明書の真偽を検証：信頼できる CA が発行？期限切れ？ドメインが一致？

1. 検証通過後、双方が公開鍵暗号で「セッションキー」をネゴシエート、以降の通信はすべてこのキーで暗号化。

2. これは銀行で手続きするようなもの：まず身分証（証明書）を提出、窓口があなたの身分を確認、それから二人で合言葉を決め、以降の会話は二人だけが理解できる。誰かが傍聴していても、暗号化された乱数しか聞こえない。

3. ただし注意：HTTPS は「転送過程」が暗号化されていることだけを保証し、対象サイトが善人であることは保証しない。フィッシングサイトも HTTPS を使える——あの鍵アイコンは「あるサイト」に接続していることだけを示し、そのサイトがあなたが行きたいサイトであることは保証しない。だから自分で URL が正しいか確認する必要があります。

4. 四、DNS セキュリティ：あなたの「ネットワーク電話帳」は安全?

DNS（ドメイン名システム）はインターネットの電話帳のようなもの：あなたが「baidu.com」と入力すると、DNS が対応する IP アドレス（例えば 110.242.68.66）を調べ、ブラウザが接続できます。問題は、伝統的な DNS クエリは

平文

## で、

暗号化されていない！**つまり、あなたの ISP、Wi-Fi 提供者、または中間ノードは誰でもあなたがどのサイトにアクセスしたか見える。**さらに厄介なのは「DNS ハイジャック」：攻撃者が DNS 段階で細工し、「baidu.com」を偽のフィッシングサイトに向ける。あなたが正しい URL を入力しても、全く同じだが実際は偽のログインページに連れて行かれ、アカウントパスワードを入力した瞬間に終わり。**幸い暗号化 DNS 方案が登場：**DNS over HTTPS（DoH）

：DNS クエリを HTTPS トラフィックに隠し、通常のウェブ閲覧トラフィックに混ぜる、どれが DNS クエリか区別できない。

DNS over TLS（DoT）

- **：TLS 暗号化プロトコルで DNS クエリを専用保護、ポートが独立、ネット管理者は DoT 使用が分かるが内容は見えない。**私は Cloudflare の 1.1.1.1 または Google の 8.8.8.8 を DNS サーバーとして推奨、両方とも暗号化 DNS に対応。Android の「プライベート DNS」設定で「one.one.one.one」または「dns.google」を直接入力するだけで、超簡単。iPhone は少し手間がかかり、プロファイルのインストールまたは App 設定が必要だが、やる価値あり。

- **五、「シークレットブラウジング」は本当に透明になれる？**まず結論：

シークレットモードはあなたの家のカーテンであって、透明マントではありません。

シークレットモードがすること：ローカルに痕跡を残さない——シークレットウィンドウを閉じると、ブラウザは閲覧履歴、Cookie、フォーム入力記録、検索記録を保存しない。他人のパソコンで何かを調べる、プレゼントを買って家族に閲覧履歴から見つかりたくない場合、シークレットモードが適切。


**シークレットモードがすること：ローカルに痕跡を残さない——シークレットウィンドウを閉じると、ブラウザは閲覧履歴、Cookie、フォーム入力記録、検索記録を保存しない。他人のパソコンで何かを調べる、プレゼントを買って家族に閲覧履歴から見つかりたくない場合、シークレットモードが適切。**

折りたたみ画面は「未来のテクノロジー」から「今買える」成熟カテゴリーになりました。しかし、ストレート型スマートフォンとはフォームファクター、使用習慣、メンテナンス方法に大きな違いがあり、買う価値があるか、どう選ぶか、どう使うか、この記事で整理しましょう。**あなたがアクセスしたサイトは**

あなたが来たことを同じように見える

、あなたの IP アドレス、デバイス情報、閲覧行動はサイト側で完全に把握。

- あなたのネットワークプロバイダーは**あなたがどのサイトにアクセスしたか同じように見える**、ただし具体的な内容は見えない（HTTPS 使用場合）。
- 会社/学校のネットワーク管理者は**同じように監視できる**あなたのネットアクセス行動。
- あなたがダウンロードしたファイル、追加したブックマークは**残る**、ウィンドウを閉じても消えない。
- だから覚えて：シークレットモードは「ローカル消去器」で、「ネットワーク透明着」ではない。本当にネットアクセス痕跡を隠す必要があるなら、VPN ＋ Tor ブラウザのようなコンボが必要ですが、日常使用ではこれらを理解していれば十分。**六、ブラウザプライバシー：ブラウザにあなたを売らせない**気づいていないかもしれませんが、ブラウザ自体が巨大な情報収集器。Cookie、閲覧履歴、キャッシュ、自動入力、位置権限……それぞれがあなたのプライバシーを漏洩する可能性。いくつかのアドバイス：

定期的に閲覧データをクリア

**：定期的に設定で Cookie、キャッシュ、履歴をクリア。「追跡防止」のためではありません——Cookie クリアではもうあまり効かない——スマホが重くなるのを防ぐため。ブラウザキャッシュは数 GB の容量を占めることがある。**

「Do Not Track」の幻想を捨てる

- **：ブラウザの「Do Not Track リクエストを送信」オプションは、本質的にサイトに「追跡しないで」とお願いするもの。しかしサイトが聞くかどうかは全く別の話で、ほとんどのサイトは無視。期待しないで。**プライバシー保護ブラウザを活用

- **：Firefox Focus、DuckDuckGo ブラウザ、Brave はデフォルトでトラッカーと広告をブロック、Chrome や Safari のデフォルト設定よりずっとクリーン。**検索エンジンも重要
- **：Baidu/Google の代わりに DuckDuckGo で検索、検索履歴を記録せず、ユーザープロファイルを構築しない。正直検索結果は Google ほど精確ではないが、日常調べ物には十分。**App 権限を確認
- **：スマホ設定でブラウザに付与された権限を確認。本当に位置、マイク、連絡先へのアクセスが必要？おそらく不要。**七、実用的なセキュリティチェックリスト：今から自分を守る
- **多くを語りましたが、最後にシンプルなセキュリティチェックリストを提供します。30 分で完了すれば、スマホのセキュリティは質的に向上：**Wi-Fi 自動接続をオフ

## ：iPhone は設定-Wi-Fi で、Android は Wi-Fi 設定で。

暗号化 DNS に変更

1. **：Android はプライベート DNS を「dns.google」に直接設定、iPhone は App またはプロファイルで 1.1.1.1 の DoH を設定。**二要素認証（2FA）を有効化
2. **：WeChat、Alipay、Apple ID、Google アカウント、2FA を有効にできるものはすべて有効化。SMS 認証コードは最低レベルの 2FA で、TOTP（Google Authenticator、Microsoft Authenticator など）が使えればさらに良い。**App 権限を一通り確認
3. **：各 App の権限を確認、特に位置、連絡先、SMS、マイク、カメラの 5 項目。懐中電灯 App が連絡先を要求するようなものを見つけたら——すぐアンインストール。**定期的にブラウザデータをクリア
4. **：カレンダーリマインダーを設定、毎月 1 回、キャッシュ、Cookie をクリア。**公共充電スタンドのケーブルを使わない
5. **：USB ポートは充電もデータ転送もでき、悪意のある充電スタンド（「Juice Jacking」と呼ばれる）が充電中にデータを盗む可能性。外出時は自分の充電ヘッドとケーブルを持参。**今すぐやる：スマホに「セキュリティ健康診断」を実施
6. **DNS アップグレード：**Android：設定 > 接続と共有 > プライベート DNS >「one.one.one.one」または「dns.google」を入力。iPhone：Safari で「1.1.1.1 DNS profile」を検索しプロファイルをインストール、または Cloudflare の 1.1.1.1 App をダウンロード。

**Wi-Fi 監査**

1. **：Wi-Fi リストを開き、知らない、使わない、名前の怪しいホットスポットをすべて「忘れる」。**権限大掃除

2. **：設定 > プライバシー/権限管理に入り、App ごとに権限を審査。権限要求が不合理（電卓が連絡先を要求など）な App をマークし、アンインストールまたは代替品を検討。**アカウントセキュリティを確認

3. **：Apple ID または Google アカウントのセキュリティページを開き、不明なデバイスのログインがないか確認、2FA が有効か確認、復元連絡先が信頼できるか確認。**第二十二章：AI はスマホでどう動くのか

4. **スマホの最もクールな「魔術ショー」へようこそ、主役は：人工知能。あなたが毎日使っている「ワンタップで綺麗な写真」、口を開けば応える音声アシスタント、使うほどにあなたを理解する入力メソッド、その背後にこの見えない頭脳が立っています。**一、AI カメラ：あなたの「現像室」がスマホに

# 第二十二章：AI はスマホでどう働くのか

二、音声アシスタント：あなたの「声の秘書」はどうやってあなたを理解する？

## 聞き取る（音声認識）

：音波をテキストに変換。

## 理解する（自然言語理解）

1. ：意図を理解、キーエンティティを抽出。実行と回答

2. ：対応する App やサービスを呼び出し、音声合成で「話して」聞かせる。三、オンデバイス AI vs. クラウド AI

3. オンデバイス AI は計算をスマホ内に留め、速度が速く、プライバシーが高く、オフラインで利用可能。クラウド AI はネット接続が必要だが、より複雑なタスクを処理できる。四、AI 大規模モデルとスマホ：あなたの常時携帯スマートアシスタント

## 2024-2025 年、AI 大規模モデル（DeepSeek、ChatGPT、文心一言、通義千問など）の爆発がスマホの使い方を深く変えました。スマホはもはや指示を実行するだけのツールではなく、理解、推論、創作できるスマートパートナー。

AI 大規模モデルとは？

# AI 大規模モデル（Large Language Model、LLM）は膨大なテキストデータで訓練された AI システムで、自然言語を理解・生成できます。インターネット上のほぼすべての文字を読んだ「スーパー優等生」と想像でき、質問に答え、文章を書き、翻訳し、プログラミングし、データ分析できる。

2025-2026 年、AI 大規模モデル（DeepSeek R1、ChatGPT-5、文心一言 5.0、通義千問 3 など）の爆発的普及はスマホの使い方を根本的に変えました。スマホはもはや命令を実行するだけのツールではなく、理解し、推論し、創作するインテリジェントなパートナーです。オンデバイス AI チップ（NPU 演算力 80 TOPS 突破）により、大規模モデル推論が初めてスマホ上でローカル実行可能になり、インターネット接続なしでも AI が使えます。

## 対話アシスタント

**：**DeepSeek、ChatGPT、豆包、Kimi などの App で、いつでも質問、翻訳、文案作成、記事要約が可能。伝統的な検索エンジンよりスマート——リンクだけでなく、直接答えをくれる。

## 執筆と創作

- **：AI にメール作成、会議議事録要約、文章推敲、クリエイティブ文案生成を依頼。**画像生成と編集
- **：AI でワンタップ写真の通行人を消去、画像境界を拡張、スタイル変換、さらにはテキスト記述から画像を生成。**リアルタイム翻訳
- **：通話リアルタイム翻訳、対面会話翻訳、写真翻訳、海外旅行で言語の壁なし。**学習補習
- **リアルタイム翻訳**オンデバイス大規模モデル：スマホ自身で AI を実行
- **最新のフラッグシップスマホはローカルで小型 AI 大規模モデル（通常 7B パラメータ程度）を実行可能、もたらす変化は：**ネットワーク不要

**：ネット切断でも AI 機能使用可能、機内、地下鉄でも OK。**

プライバシーがより安全

- **：あなたの対話とデータはクラウドにアップロードされず、完全にスマホローカルで処理。**応答がより速い
- **：ネットワーク遅延なし、AI 応答はほぼ瞬時。**現在の制限

- **：オンデバイスモデルの能力はまだクラウド大規模モデルに及ばず、複雑なタスクは依然としてネット接続が必要。ただしチップの AI 演算力向上に伴い、オンデバイスモデルはますます強力に。**システムレベル AI 統合：
- **Apple Apple Intelligence：**iOS 18 で導入された AI システム、執筆ツール、画像マジックワンド、Siri 強化、通知要約などの機能を統合。オンデバイス処理を優先、複雑なタスクのみクラウドへ。

### Huawei 盤古大規模モデル

- **：HarmonyOS に統合、小芸アシスタントが大規模モデル能力を獲得、ドキュメント要約、スマート画面認識、AI レタッチなどをサポート。**Xiaomi 澎湃 AI
- **：システムレベル AI 能力、AI 画像検索、AI 要約、AI リアルタイム字幕、オンデバイス大規模モデル対話などをサポート。**Samsung Galaxy AI
- **：通話リアルタイム翻訳、ノート AI 要約、画像生成編集、サークル検索などの機能。**五、いくつかの典型的な応用シーンと実践
- **スマホ AI の「超能力」をより直感的に感じてもらうため、以下 5 つの具体的なシーンで、AI が日常生活でどう「助け舟を出す」かをお見せします：**シーン 1：リアルタイム翻訳——海外旅行の常時携帯通訳

## 実際の表現

**：**パリのレストランでフランス語が分からない？iPhone の「翻訳」App は対面双方向会話モードをサポート、双方が各自の言語の字幕を見られ、同時通訳も可能。Android 陣営の Samsung Galaxy AI 通話リアルタイム翻訳は、電話で直接中国語を話し、相手には英語が聞こえ、その逆も可能。

### 背後の原理

**：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。**シーン 2：AI 消去——通行人に「破壊された」観光写真を救済

**実際の表現：**観光地で絶景の自撮りを撮ったが、背景に他の観光客がぎっしり？「マジック消去」で通行人を囲むと、スマホが自動認識して通行人を「塗り消し」、さらに背景をスマートに埋め合わせ、効果はしばしば驚異的。

### 背後の原理

**：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。**シーン 3：AI レタッチ——ワンタップで「撮影マスター」

**実際の表現：**曇りの日の灰色っぽい風景写真を撮った？AI「マジックスカイ」でワンタップで空を晴れ、夕焼け、さらには星空に交換。AI 画像拡張で縦構図を横構図にでき、スマホが画面外のシーンを「推測」。

### 背後の原理

**：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。**シーン 4：ドキュメント処理——オフィスワーカーの効率倍増器

**実際の表現：**会議終了後、会議録音を AI に渡すと、自動的に構造化された会議議事録を整理、ToDo 項目を抽出、要約を生成。10 ページの英語 PDF レポートを AI に渡せば、10 秒で核心結論を抽出。

### 背後の原理

**：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。**シーン 5：AI 画像検索と物体認識——あなたの「視覚検索エンジン」

**実際の表現：**街で知らない花を見かけ、カメラを起動すると AI が即座に「これはムクゲです」と教える。アルバムで去年海辺で撮った写真を探す時、「海」と検索すると、AI が海を含むすべての写真を自動検出、手動で探す必要なし。

### 背後の原理

**：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。**六、プロンプト最適化：AI をあなたの指揮下に置く「コミュニケーション公式」

**実際の表現**最も実用的で最も使いやすい「役割-背景-タスク-制約」コミュニケーション思路

## を紹介し、大量の実例を交える。

なぜフレームワークが必要？**多くの人が AI を Baidu のように使う——キーワードをいくつか入力する。しかし AI は検索エンジンではなく、「読解してから回答する」言語モデル。**より多くの文脈を与え、何を欲しているかを明確にし、脱線を制限する

と、精確な結果をくれる。フレームワークはあなたの頭の中の「良い答えとは何か」を AI が理解できる指示に翻訳するもの。

1. 役割（Role）：AI に誰であるかを伝える **AI に具体的な「アイデンティティ」を与えると、対応分野の専門知識と表現方法を呼び出す。**役割設定が具体的なほど、出力が専門的

。

AI に具体的な「役割」を与えると、その分野の専門知識と表現スタイルを引き出してくれます。「あなたは。

で。」（長さ＋形式）

- なぜ有効? 複雑な概念をシンプルに説明するのが得意な中学物理の先生です。」（科学普及問題）
- なぜ有効? 審美眼があり、多くを読み、独自のスタイルを持つ図書編集者です。」（執筆問題）
- なぜ有効? 毎日会社員に朝食を作る料理ブロガーです。」（生活問題）
- なぜ有効? 忍耐強く、穏やかで、初心者を決して嘲笑しない中医師です。」（健康問題）
- なぜ有効? 役割は **AI** の知識範囲

良いプロンプトを書くのは一回きりのことではなく、表現スタイル、、を設定。役割のない **AI** は「何でもできるが何も深くない」ゼネラリスト。役割があれば、そのアイデンティティに入り、その専門家のように思考できる。3. タスク（Task）：AI に何をするか明確に伝える「あなたは誰、現状は何、制約条件は何」を事前に明確にすると、**AI** は本当にあなたに適した

答えを出せ、漠然とした回答ではなくなる。

「自分は誰か、今どんな状況か、どんな制約があるか」を先にはっきり伝えると、AI は「私のスマホは Xiaomi 14、買って半年、最近ゲーム中に発熱がひどい

で。」（長さ＋形式）

- 「私は **60** 歳の退職したおばさんで、目があまり良くなく、スマホをあまり使いこなせません。」（ユーザー像）
- 「私は **2026** 年 **6** 月の英語六級試験を準備中、基礎は普通です。」（学習背景）
- 「私は三都市の小さなカフェを経営、月商 **3** 万、二人の従業員を雇用しています。」（ビジネス背景）
- 「私は出産後家で産後休養中、睡眠が著しく不足しています。」（生活段階）
- なぜ有効? 背景情報は **AI** に「誰のためにサービスするか」

良いプロンプトを書くのは一回きりのことではなく、「どんな条件下か」を知らせる。同じ問題でも、学生と親には説明が異なり、都市人と地方人にはアドバイスが異なる。背景が欠けると、**AI** は「最大公約数」の答えしか出せず、しばしばピンとこない。3. タスク（Task）：AI に何をするか明確に伝えるこのステップは動詞で始める

（書く、列挙、比較、翻訳、説明、要約……）、タスクを

具体的で実行可能に記述。曖昧なタスクは曖昧な答えしか生まない。（書く、列挙、比較、翻訳、説明、要約……）。タスクは「私がすぐに閉じるべき 5 つの権限を列挙

で。」（長さ＋形式）

- 「以下の英語メールを中国語に翻訳し、ビジネス口調を維持してください。」（タスク具体）
- 「この 4 機種の 3000 元帯のスマホを比較し、長所短所比較表を作成してください。」（形式明確）
- 「小学生にも分かる言葉で「5G とは何か」を説明してください。」（スタイル明確）
- 「私の朝食店に 3 つの短動画文案を執筆し、それぞれ会社員、カップル、子供連れの親向けにしてください。」（セグメント明確）
- 「タスク部分は AI の行動の「指令中枢」。動詞が明確で要求が具体的なほど、AI は「何を納品するか」を把握

良いプロンプトを書くのは一回きりのことではなく、4. 制約（Constraints）：AI に何をしないかを伝える「制約」は「拘束」のように聞こえるが、実は AI をあなたに最も適した出力に収束させる

強力なツール。一般的な次元は：文字数、形式、トーン、回避項目、必須項目。

「制限」は「縛り」のように聞こえますが、実際には「200 字以内、箇条書き

で。」（長さ＋形式）

- 「トーンは親しみやすく自然に、友人との会話のように、『まず、次に、さらに』のような役所語は使わないで。」（トーン）
- 「専門用語は使わず、専門用語に出会ったらついでに説明して。」（用語制限）
- 「各段落は 3 行以内、スマホで読みやすく。」（レイアウト）
- 「必ず含める：使用シーン、長所短所、私の予算との適合度。」（必須項目）
- 「具体的な投資アドバイスはしない、原理だけ説明して。」（境界）
- 「制約がないと、AI は「自由発揮」しやすい——学術的すぎたり、長すぎたり、脱線したり。明確な制約＝明確な「納品基準」、双方の期待が揃い、結果がコントロール可能に。5. プロンプト品質評価：良いプロンプトの 5 つの特徴

良いプロンプトを書くのは一回きりのことではなく、書いたらこの 5 項目を自己チェック

してから AI に送信：

具体的で抽象的でない：「記事を書いて」ではなく「800 字、3 セクション、初心者ママ向けのスマホ選びガイドを書いて」。背景を含み孤立していない

1. ：AI に「私は誰、なぜこれを聞くか」を伝えれば、「あなた用にカスタマイズ」された答えが出せる。明確な出力形式
2. ：表、リスト、段落、それとも JSON？事前に言う。制約が適切
3. ：文字数、スタイル、回避項目、3 つのうち 1 つでも明確にすれば顕著な向上。AI が検証可能
4. ：聞き終わった後、AI にあなたの要求を復唱させる；はっきり言えなければ、はっきり伝わっていない。6. よくある問題と解決法
5. 症状解決法

回答が漠然、実用性がない

| 背景を追加（あなたは誰/どんなシーン） | 回答が学術的/公式すぎる |
|---|---|
| トーン制約を追加＋「役所語」拒否リスト | 回答が脱線/質問に答えていない |
| タスクを動詞で言い直し、「これだけ答えて」を要求 | 回答が長すぎて読めない |
| 文字数を硬く制限（200 字以内） | AI がもっともらしく嘘をつく |
| 「不確かな場合は『分かりません』と表示」を追加＋重要事実の自己チェック | 第一版が不満足 |
| 書き直さず、元のプロンプト後に「もう少し…して」を追加 | 7. 3 つの完全なシナリオ事例 |
| シナリオ 1：生活アシスタント（朝食組み合わせ提案） | プロンプト |

：

あなたは栄養バランスに優れた朝食ブロガーです（

シニア行政アシスタント：
背景）。7 日間飽きない朝食の組み合わせをリストアップしてください（タスク）。各朝食の要件：(1) 準備時間 15 分以内 (2) 栄養バランス（タンパク質＋炭水化物＋野菜・果物）(3) 老若男女向け (4) 一般的なスーパーで買える食材（制限）。）。各朝食の要件：(1)15 分以内で作れる (2) 栄養バランス（タンパク質＋炭水化物＋野菜）(3) 老若問わず食べられる (4) 食材は普通のスーパーで買える（：役割が専門性を定義し、背景が 3 世代の共通点を示し、タスクが明確な成果物を指定し、4 つの制限が実行可能性を保証します。ログイン後、右上の「+」をクリックし、「New repository」を選択。
プロンプト：

あなたは

**シニア行政アシスタント：**

Q2 マーケティング推進案を議論しました（**背景**）。実行可能な 12 週間の試験対策計画を立ててください（**タスク**）。**要件：(1) 毎週のリスニング・スピーキング・リーディング・ライティングの時間配分を明確に (2) 毎月の重点突破方向を提示 (3) 適切な復習資料を 3 冊推奨（選定理由も記載）(4) 表形式で出力（制限）。**（背景）。**：役割が専門的背景を定義し、背景がスコアと時間のアンカーを示し、タスクが「段階別計画」を明確にし、制限が「表化」と「実行可能性」を要求します。**シナリオ 3：仕事効率化（議事録整理）**

**プロンプト：**

あなたは

**シニア行政アシスタント：**

Q2 マーケティング推進案を議論しました（**背景**）。以下の会議記録を構造化された議事録に整理してください（**タスク**）。**要件：(1)「決議事項 / ToDo 事項 / リスクと異論」の 3 部構成 (2) ToDo 事項には担当者と期限を明記（言及がなければ「要確認」と記載）(3) 総文字数は 600 字以内 (4) Markdown 形式で出力（制限）。**（背景）。**：役割が行政の視点を定義し、背景が会議テーマを示し、タスクが構造化出力を明確にし、4 つの制限が結果をそのまま再利用可能にします。**応用編：AI をあなたの「専属アシスタント」にする**

**プロンプト**複数回対話の最適化

**：AI の回答に不満がある時、「これは専門的すぎるので、小学生にもわかる言葉で説明し直してください」と言えます。**

思考の連鎖（CoT）

- **：AI に「段階的に思考させる」と、回答の質が通常高くなります：「この数学の問題を段階的に分析してください」。**複数 AI 連携
- **：A で初稿を書き、B で推敲し、C で翻訳する——組み合わせて使うと効果的です。**システムプロンプト
- **：AI の設定で自分の身分、好み、タブーなどを事前設定し、回答を自分のニーズに合わせます。**七、AI の限界と課題：清醒な判断力を保つ
- **チャレンジ 1：複雑な指示**「ハルシネーション」問題

## **：AI がもっともらしくでたらめを言うことがあります（専門用語で Hallucination）。存在しない歴史的事件、誤った引用、虚偽の統計データをでっち上げます。重要な情報は必ず人工で確認してください。**

知識の期限

- **：ほとんどの AI の訓練データには期限があり（例：2024 年初頭）、それ以降の出来事を知りません。**推論能力の限界

- **：複雑な多段推論、数学的証明、論理的罠において、AI は依然として間違いを犯しやすいです。**プライバシーリスク

- **：クラウド AI サービスでは、あなたの会話がモデル訓練に使用される可能性があります。機密情報には端末側 AI を優先使用してください。**偏見と価値観

- **：AI は訓練データの偏見を持つ可能性があり、独立した思考を保つ必要があります。**AI 使用のヒント

- **AI 生成の内容は不正確な場合があります（「ハルシネーション」問題）、重要な情報は必ず確認してください。**AI に個人情報（身分証番号、パスワード、銀行情報など）を入力しないでください。

**プロンプトを活用：より具体的で明確な記述ほど、AI の回答の質が高くなります。**

- AI はツールであり権威ではありません、独立した思考と判断力を保ってください。
- 重要な意思決定（医療、法律、財務）では、AI に頼らず専門家に相談してください。
- 応用編：AI をあなたの「専属アシスタント」にする
- これらの応用使い方を試して、AI の真の実力を体験してください：
- 複数回対話の最適化

**チャレンジ 2：AI 画像編集**

1. **：通行人が写った観光写真を撮り、「魔法消去」や「AI 消しゴム」機能を使って通行人をきれいに消去してください。AI が背景をどう「補完」するか観察してください。**チャレンジ 3：プロンプト最適化

2. **：まず「旅行ガイドを書いて」という簡単なプロンプトで AI に聞き、次に CRISPE フレームワークで詳細なプロンプトを書いてください。2 回の回答の質の違いを比較してください。**第二十三章：プログラミング的思考とオートメーション入門

3. **前二章では、スマートフォンに安全な「境界壁」を築き、壁の中で日に日に賢くなる「AI の頭脳」を見てきました。ここで、スマートフォンの究極のポテンシャルの一つを解放しましょう：プログラマーのように「考え」、設定した連続タスクを自動実行させること。「プログラミングなんてできないし、自分には関係ない」と思うかもしれません。安心してください、本章では一行のコードも書く必要はありません——現代のスマートフォンは「プログラミング」のハードルを、ドラッグ＆ドロップで済むレベルまで下げています。**一、オートメーションとは何か？

：AI がもっともらしくでたらめを言うことがあります（専門用語で Hallucination）。存在しない歴史的事件、誤った引用、虚偽の統計データをでっち上げます。重要な情報は必ず人工で確認してください。



**事例 2：機種変更データ移行の完全フロー**

- **シーン**: iPhone 13 Xiaomi 15 のクロスブランド機種変更。
- **手順**: (1) 旧機で iCloud/WeChat バックアップ (2) Apple ID ログアウト +「検索」をオフ (3) 新機で「Xiaomi 機種変更」App で QR スキャン (4) 移行項目を選択 (5) WeChat は独自の「チャット履歴移行」を使用 (6) 重要データ確認（連絡先/アルバム/WeChat）(7) 旧機を暗号化消去。
- **期待される結果**: データ損失ゼロ、旧機は安全に売却可能。

# 第二十三章：プログラミング的思考とオートメーション入門

前二章では、スマートフォンに安全な「境界壁」を築き、壁の中で日に日に賢くなる「*AI* の頭脳」を見てきました。ここで、スマートフォンの究極のポテンシャルの一つを解放しましょう：プログラマーのように「考え」、設定した連続タスクを自動実行させること。「プログラミングなんてできないし、自分には関係ない」と思うかもしれません。安心してください、本章では一行のコードも書く必要はありません——現代のスマートフォンは「プログラミング」のハードルを、ドラッグ＆ドロップで済むレベルまで下げています。

## 一、オートメーションとは何か？

例えば：毎朝起きた時、手動で「おやすみモード」をオフにし、WeChat を開いてメッセージを確認し、天気アプリを開いて今日の服装を決め、NetEase Cloud Music を開いて曲を再生する。これらのステップを一つのオートメーションフローにまとめ、「毎朝 7 時に自動実行」と設定すれば、起きてスマートフォンを手に取るだけで、すべてが準備完了です。想像するだけで素晴らしいと思いませんか？**オートメーションの本質は「もし……なら……」という論理チェーン、プログラミングにおける if-else です。この考え方を理解すれば、一生コードを書かなくても、問題処理の条理が格段に良くなります。核心となる要素：**トリガー条件

**：**いつ開始するか？「毎朝 7 時」「自宅の Wi-Fi に接続した時」「特定の場所に到着した時」「バッテリー残量が 20% を下回った時」などが設定できます。

アクション

1. **：具体的に何をするか？WeChat メッセージの送信、特定のアプリを開く、音量調整、壁紙の切り替え、音楽再生、通知の送信……スマートフォンで手動できることのほぼすべてを、オートメーションで実行できます。**中間論理

2. **（任意だが強力）：状況 A なら X を実行、そうでなければ Y を実行。例えば「平日ならニュースポッドキャストを再生、週末なら音楽を再生」など、これを条件分岐と呼びます。**二、iOS「ショートカット」——Apple ユーザーのオートメーション最強ツール

3. **Apple の「ショートカット」（Shortcuts）アプリは標準搭載されており、ダウンロー**

ド不要です。その核心概念は、各種操作を積み木のように組み合わせることです。まずいくつかの重要な概念を説明します：「ショートカット」

## 二、iOS「ショートカット」——Apple ユーザーのオートメーション最強ツール

「オートメーション」

- ：トリガー条件により自動実行される指示。手動でトリガーする必要がありません。例えば「スマートフォンがカーブルートゥースに接続した時、自動的に地図アプリを開く」。オートメーションは「オートメーション」タブで設定し、「ショートカット」とは分かれています。「アクション」
- ：最小の実行単位。アプリを開く、メッセージを送信する、天気を取得する、明るさを調整する——それぞれが一つのアクションです。何百、何千ものアクションを繋げることができます。実用的な例をいくつか紹介します。実際にやってみれば理解できるはずです：
- 実例 1：ワンタップ「おはようフロー」毎朝タップする（または Siri に「おはよう」と言う）だけで、スマートフォンが自動的に：今日の天気を取得天気と気温を朗読今日のカレンダー予定を表示よく使う音楽アプリを開いて再生を開始。操作手順：新規ショートカットアクション「現在の天気を取得」を追加「テキストを朗読」を追加「カレンダーイベントを取得」を追加「リマインダーを表示」を追加「アプリを開く」で NetEase Cloud Music を選択。これだけです、すべてドラッグ＆ドロップで完了します。

実例 2：自動「省電力モード」

### 実例 1：ワンタップ「おはようフロー」

実例 3：スキャン自動家計簿

### 実例 2：自動「省電力モード」

もう一つ、知らない人が多い使い方：ショートカットをホーム画面に追加して、アプリのアイコンにできます。例えば「タクシーで帰宅」というショートカットを作り、アイコンをデスクトップに置けば、ワンタップで DiDi を開き、目的地を自宅に設定できます。アプリを開いて手動入力するより遥かに速いです。

### 実例 3：スキャン自動家計簿

Android 側は少し複雑ですが、上限もより高いです。まず簡単なものから：

入門レベル：オートクリッカー

## 三、Android オートメーション——Tasker とその仲間たち

中級レベル：Tasker——Android オートメーションの王

### 入門レベル：オートクリッカー

プロファイル（Profile）

### 中級レベル：Tasker——Android オートメーションの王

タスク（Task）

- **：「トリガー後に何をするか」を定義。一つのタスクは複数のアクションを含み、順番に実行されます。アクションの種類には、システム設定の調整（音量、明るさ、Wi-Fi、Bluetooth、機内モード）、SMS 送信、HTTP リクエスト、Shell コマンドの実行、メディア再生の制御、ダイアログの表示などがあります。**Tasker の定番実用シナリオをいくつか紹介します：

- **帰宅時自動で Wi-Fi オン：**トリガー条件を「特定の基地局に接続」（自宅付近では固定の基地局に接続するため）タスク「Wi-Fi をオンにする」。GPS 不要、省電力で、スマートフォンが小区の範囲に入ると自動的に Wi-Fi がオンになります。

会議モード

- **：トリガー条件を「カレンダーイベントのタイトルに『会議』を含む」タスク「サイレントモード設定、メディア音量オフ、WeChat 自動返信（プラグインが必要）」。会議中にスマートフォンが突然鳴る心配がなくなります。**夜間の目のケア

- **：トリガー条件を「時間 22:00-06:00」タスク「アイケアモードをオン、明るさを 20% に下げる、画面を白黒（グレースケールモード）に設定」。白黒画面はショート動画を見る欲求に絶大な効果があります——画面がグレーになると、見る気が失せます。**イヤホン接続時自動ポッドキャスト

- **：トリガー条件を「有線イヤホン挿入/Bluetooth イヤホン接続」タスク「小宇宙アプリを開き、最新エピソードの再生を開始」。イヤホンを装着するだけで聴取でき、スマートフォンのロック解除してアプリを探す手間が省けます。**Tasker のハードルが高すぎると感じる場合は、簡易版の兄弟アプリ

- **MacroDroid** を試してみてください。インターフェースがはるか使いやすく、テンプレートも豊富で、こだわりたくないユーザーに適しています。また、現在多くの中国系 Android システム（Xiaomi の MIUI、Huawei の HarmonyOS、OPPO の ColorOS）に「シーン」や「スマートアシスタント」機能が標準搭載されており、Tasker ほど強力ではありませんが、日常的なオートメーションには十分です。

四、オートメーション的思考の日常応用**オートメーションを学ぶ最大の収穫は、少しの時間を節約することではなく、**問題に対する考え方を変えること

## 四、オートメーション的思考の日常応用

写真を撮るたびに、自動的にクラウドストレージにバックアップし、ローカルの写真を小サイズに圧縮して容量を節約。**毎朝自動的にカレンダーから今日の予定を読み取り、テキストを生成して WeChat で家族に送信。**荷物の受領 SMS を受信したら、自動的にメモに「荷物を受け取る」の ToDo を追加。

- バッテリーが満充電になったら自動的に音声で充電器を抜くようリマインド（過充電によるバッテリー劣化を防止）。
- 毎日の退勤時間になっても、まだ会社の近くにいる場合（基地局位置情報で判定）、自動的に「退勤の時間です」とリマインド。
- これらは高度に聞こえますが、「トリガー＋アクション」という 2 つの基本概念をマスターし、ショートカットや Tasker と組み合わせれば、すべて実現できます。重要なのは
- 「この作業は自動化できる」とまず気づくこと
- ——多くの人は「できない」のではなく、そういう発想すらないのです。

五、落とし穴ガイド：オートメーションのよくある問題**オートメーションも順風満帆ではありません。私が先に踏んだ落とし穴をいくつか紹介します：**位置トリガーが反応しない場合がある

## 五、落とし穴ガイド：オートメーションのよくある問題

ショートカットの「オートメーション」には認証が必要

- **：iOS では、「オートメーション」タブのトリガーの多くは、実行前に手動確認が必要です（iOS のセキュリティメカニズム）。ただし、Wi-Fi 接続、Bluetooth 接続、アラーム停止のトリガーは、確認なしで完全に自動実行できます。**Tasker はユーザー補助権限が必要
- **：多くの高度な機能（例えばタップのシミュレーション）にはユーザー補助サービス権限が必要で、設定画面に常駐通知が表示されます。これは Android のセキュリティ設計であり、削除できません。**オートメーションを複雑にしすぎない
- **：オートメーションを始める際は、「2-3 のアクション」の小さなフローから始めてください。いきなり 20 ステップのスーパーフローを作ろうとしないでください——デバッグで発狂します。**ヒント：オートメーション入門の正しい姿勢
- **まず 5 分間考えてみてください：毎日スマートフォンで最も多く繰り返している 3 つの作業は何ですか？例えば毎朝のルーティン、Bluetooth イヤホン接続後の習慣、外出前の設定など……これらの高頻度の小さな作業から始めて、それぞれ簡単なオート**

メーションを作ってみてください。1 つのオートメーションが 5 秒節約するだけでも、積み重なれば大きな節約になります。しかも、完成した時の達成感は、本当に癖になります。練習：3 つの入門オートメーションチャレンジ

自分に最も適したものを 1 つ完了すれば、入門完了です：

チャレンジ 1：おはようオートメーション

：iOS：ショートカットアプリで新規ショートカット「現在の天気を取得」「テキストを朗読」「アプリを開くで NetEase Cloud Music を選択」を追加。完了後、Siri に「おはよう」と言ってトリガーできます。Android：Tasker で新規タスク「天気を取得」「朗読」「アプリを開く」を追加トリガー時間を毎日 7:00 に設定。

チャレンジ 2：低バッテリー自動省電力

1. ：iOS：オートメーションタブ新規「バッテリー残量」25% を下回った時「低電力モードをオン」＋「明るさ 25%」を追加。Android：Tasker で新規 Profile 状態電源バッテリー 0-25% タスク「省電力モードオン」＋「明るさを下げる」。チャレンジ 3：カーブルートゥース接続時自動ナビ

2. ：iOS：オートメーション新規「Bluetooth」カーブルートゥースを選択「アプリを開くで Amap を選択」＋「メディア音量を 80% に設定」を追加。Android：TaskerProfile 状態 Bluetooth 接続済みカーブルートゥースを選択タスク「Amap を開く」＋「メディア音量を上げる」。第二十三章付録：GitHub 入門ガイド——コードホスティングからコラボレーションプラットフォームまで

3. 前章で「プログラミング的思考」を説明しましたが、プログラミング的思考を実践し、他の人と一緒にプロジェクトを行うには、GitHub という名前を避けて通れません。コンピュータ専攻の学生がコンテストに参加する、自分のコード作品を保存したい、あるいは単に世界中の開発者が何に取り組んでいるか見たい場合でも、GitHub は必ず知っておくべきプラットフォームです。この節では最も基礎的な概念から説明し、明確な認識を構築します。一、GitHub とは何か？

事例 3：自動化「おはようフロー」

- シーン: 毎日 7:00 に自動で天気予報 + マナーモード解除 + ニュースを開く。
- 手順: iOS: ショートカットオートメーション時間トリガー「天気を取得」「音量を設定」「App を開く」を追加。Android: MacroDroid トリガー (時間 7:00) アクション (メディア音量/URL を開く/TTS 天気読み上げ)。
- 期待される結果: インストール不要で実現可能、導入ハードルが低い。

# 第二十三章付録：GitHub 入門ガイド——コードホスティングからコラボレーションプラットフォームまで

前章で「プログラミング的思考」を説明しましたが、プログラミング的思考を実践し、他の人と一緒にプロジェクトを行うには、*GitHub* という名前を避けて通れません。コンピュータ専攻の学生がコンテストに参加する、自分のコード作品を保存したい、あるいは単に世界中の開発者が何に取り組んでいるか見たい場合でも、*GitHub* は必ず知っておくべきプラットフォームです。この節では最も基礎的な概念から説明し、明確な認識を構築します。

## 一、GitHub とは何か？

例えで理解すると：**Git はあなたのパソコン上の日記帳、GitHub は公開のブログプラットフォーム**——自分の作品を公開し、他の人のプロジェクトをフォローし、議論やコラボレーションに参加できます。それは「プログラマーのバックヤード倉庫」にとどまらず、世界中の開発者が共同で構築する「テクノロジーシティ」のようなものです——プロジェクトはここで生まれ、反復され、議論され、再利用されます。

一般の人にとっても GitHub は決して縁遠いものではありません。多くの非プログラマーがノート、電子書籍、個人ブログのホスティングに利用し、設定ファイルのバックアップにも使っています。「自分のものを整理して共有したい」というニーズが少しでもあれば、GitHub を知る価値があります。**二、Git と GitHub の違い（重要概念!）** 初めて触れる人はこの 2 つの言葉を混同しがちです。ここで整理しましょう：

比較項目

## 二、Git と GitHub の違い（重要概念!）

GitHub

| 本質 | バージョン管理ツール（ソフトウェア） | コードホスティングプラットフォーム（ウェブサイト） |
| --- | --- | --- |
| 実行場所 | ローカルパソコン上 | クラウド（ブラウザでアクセス） |
| 核心の役割 | ファイルの各変更を記録（何を変更したか、誰が変更したか、いつ変更したか） | Git リポジトリをクラウドに置き、保存・表示・コラボレーションを容易に |
| ネットワーク要否 | 不要、オフラインで使用可能 | ネットワーク接続が必要 |
| 帰属 | オープンソースソフトウェア、Linus Torvalds（Linux の父）が作成 | Microsoft 傘下製品 |
| 同等の競合製品 | SVN、Mercurial | GitLab、Gitee（碼雲）、Bitbucket |
| 簡単に言うと： | Git は「変更をどう記録するか」を管理し、GitHub は「変更をどこに置き、どうコラボレーションするか」を管理します | 。Git はツールでローカル実行、GitHub はプラットフォームで Git リポジトリをクラウドに置き、コラボレーションと表示を容易にします。Git だけを使って GitHub を使わないこともできます（例えば自分でコードを書いてバージョン管理するだけ）が、GitHub を使うには必ず Git が必要です（GitHub は Git に基づいているため）。 |

三、GitHub の核心概念と機能**基礎概念**作業を始める前に、いくつかの高頻度ワードを知っておきましょう。そうしないと GitHub のインターフェースを見ても混乱するだけです：

## 三、GitHub の核心概念と機能

### 基礎概念

コミット（Commit）

- **：コードの一回の変更記録。各コミットには説明情報が付属し、「今回何を変更したか、なぜ変更したか」を記録します。これが Git/GitHub で「タイムトラベル」ができる基礎です。**ブランチ（Branch）
- **：メインラインから分岐した独立した開発ライン。新機能を追加したい場合、ブランチを切って単独で開発し、完了後にメインラインにマージすれば、他の人に影響を与えません。**クローン（Clone）

- **：クラウドのリポジトリをローカルパソコンに完全にコピーし、以降はローカルでコードを修正できます。**プッシュ（Push）/ プル（Pull）
- **：Push はローカルコードを GitHub クラウドにアップロードすること、Pull は GitHub から最新コードをローカルにダウンロードすること。チームコラボレーションでコードを同期する 2 つのコアアクションです。**プルリクエスト（Pull Request、略称 PR）
- **：あるブランチでコードを修正し終え、メインブランチにマージしたい時、PR を発行し、チームメンバーにコードをレビューしてからマージしてもらいます。GitHub コラボレーションの核心メカニズムです。**イシュー（Issue）
- **：バグ、機能要件、議論トピックを記録するために使用。各 Issue は一つの掲示板スレッドのようなもので、担当者を割り当てたり、ラベルを付けたりできます。**Fork
- **：他人のリポジトリを自分のアカウントにコピーし、自分のコピー上で自由に修正でき、元のリポジトリに影響を与えません。修正完了後、元の作者に PR を送り、改善のマージを依頼できます。**核心機能
- **GitHub が提供するのはコード保存だけではありません：**バージョン管理

## 核心機能

チームコラボレーション

- **：複数ユーザーによる同一プロジェクトの並行開発をサポートし、複数人が同時に開発しても上書きや競合が発生しません。**問題追跡
- **：Issue でバグや機能要望を記録し、チームでのフォローアップが容易に。**プロジェクト管理
- **：ToDo リスト、プロジェクトかんばん（Projects）などの管理ツールを統合し、軽量な Trello を内蔵しています。**オートメーション
- **：**GitHub Actions
- **でコードコミット後の自動コンパイル、テスト、パッケージング、デプロイを実現し、現代の「CI/CD（継続的インテグレーション / 継続的デプロイ）」の中核ツールです。**ドキュメント管理**：README でプロジェクトを紹介し、Wiki で詳細なアーキテクチャや開発規範を保存し、Pages では静的ウェブサイトを直接ホスティングできます。**コードホスティングと表示
- **：各リポジトリには美しいウェブページ表示があり、シンタックスハイライトをサポートし、あなたの技術作品の「看板」になります。**オープンソースコミュニティ
- **：好きなプロジェクトを Star（お気に入り）し、プロジェクトの動向を Watch（フォロー）し、開発者を Follow（フォロー）して、グローバルなオープンソースコラボレーションに参加できます。**四、登録と使い始め：5 分で最初のリポジトリを作成

- **アカウント登録** github.com

## 四、登録と使い始め：5 分で最初のリポジトリを作成

### アカウント登録

1. github.com/yourname）。 メール認証とロボット認証を完了。
2. 無料プラン（Free）を選択すれば十分です。個人ユーザーには完全に対応しています。最初のリポジトリを作成ログイン後、右上の「+」をクリックし、「New repository」を選択。
3. リポジトリ名（Repository name）を入力、例：
4. my-first-project

### 最初のリポジトリを作成

1. 簡単な説明（Description）を書き、他の人がこのリポジトリの目的をわかるようにします。
2. Public（公開）または Private（非公開）を選択。公開リポジトリは誰でも閲覧可能、非公開は自分と共同作業者のみ閲覧可能。無料アカウントでも無制限の非公開リポジトリをサポートしています。「`Add a README file`」にチェックを入れる——`README` はプロジェクトの「取扱説明書」で、各リポジトリに用意することを推奨します。。
3. 豆知識：README とは
4. README.md は Markdown 形式のファイルで、他の人があなたのリポジトリを開いた時に最初に目にする内容です。良い README には通常、プロジェクトの概要、インストール方法、使用例、ライセンス、連絡先が含まれます。README を書くことはプログラマーの基本素養であり、プロジェクトが他の人に使用されるかどうかの最初の関門でもあります。
5. 五、基本ワークフロー：ローカルからクラウドへ
6. GitHub のワークフローを理解するには、このサイクルを覚えるのが核心です：

ローカルで修正ステージコミットプッシュクラウド更新

。各ステップを分かりやすく説明します：

## 五、基本ワークフロー：ローカルからクラウドへ

：ローカルパソコンでエディタを使ってコードを書いたりドキュメントを修正したりします。**git add**：修正したファイルを「ステージングエリア」に追加し、Git に「これらの変更を記録する準備ができた」と伝えます。

1. **git commit：**ステージングエリアの変更を正式に一つの「コミット」として記録し、説明を付けます。例：「ログインバグを修正」。このステップはローカルでのみ完了します。

2. **git push：**ローカルのコミットを GitHub クラウドにアップロードし、他の人が見られるようにします。

3. **git pull：**作業を始める前に、まずクラウドから最新コードをプルし、ローカルがチームの他のメンバーと同期していることを確認します。

4. **初心者はこの呪文を覚えてください：**「まず pull してから修正、修正したら add、commit には説明を付け、最後に push」

5. **。この習慣を身につければ、チームコラボレーションが乱れることはありません。**六、チームコラボレーションフロー：プルリクエストはどう機能するか

GitHub の最も強力な点はコラボレーションメカニズムです。典型的なチームコラボレーションフローは次の通りです：**ブランチを切る：**メインブランチ（通常 main または master）から新しいブランチを切り、例：

## 六、チームコラボレーションフロー：プルリクエストはどう機能するか

、このブランチで新機能を開発します。

1. **開発してコミット：**新しいブランチでコードを書き、複数回 commit して進捗を記録します。`GitHub` にプッシュ：ブランチをクラウドリポジトリに push します。

2. **プルリクエストを発行：**GitHub ウェブページで PR を発行し、あなたのブランチをメインブランチにマージするよう依頼します。この PR で何を変更したか、なぜ変更したかを明確に書きます。

3. **コードレビュー（Code Review）：**チームメンバーがあなたのコードを確認し、意見や修正提案を提示します。フィードバックに基づいてコミットを続けると、PR は自動的に更新されます。

4. **マージ：**レビュー通過後、「Merge」をクリックしてコードをメインブランチにマージします。あなたの貢献が正式にプロジェクトに取り込まれました！

5. **このフローにより、**コードがメインブランチに入る前に必ずレビューが行われる

6. **ことが保証され、エラー発生確率が大幅に下がります。オープンソースプロジェクトもこのメカニズムで世界中の見知らぬ人からの貢献を受け入れています——有名なオープンソースプロジェクトで PR を提出し、採用されれば、あなたはそのプロジェクトの貢献者になれます。**七、GitHub の典型的な応用シナリオ

学生のコンテスト参加と課題プロジェクトコンピュータ専攻の学生が ACM、ハッカソン、卒業制作に参加する際、GitHub はほぼ欠かせません。チームコラボレーションプラットフォームとしても、作品展示ウィンドウとしても機能します——コンテストの審査員はあなたのリポジトリを直接見て、コード品質とコラボレーション過程を理解できます。多くの企業の採用時にも GitHub ページの添付を求め、技術力の証明にします。個人ナレッジベースとノートホスティング

## 七、GitHub の典型的な応用シナリオ

### 学生のコンテスト参加と課題プロジェクト

GitHub はグローバルなオープンソース運動の核心拠点です。好きなプロジェクトを Fork し、修正後に PR でコミュニティに還元できます。また、単に Star でお気に入りのプロジェクトをブックマークし、更新を追跡することもできます。オープンソースへの参加は技術向上と評価蓄積の絶好の道です。

### 個人ナレッジベースとノートホスティング

GitHub Actions を活用し、「コードを push したら、自動テスト＋サーバーへの自動デプロイ」というフルオートフローを実現できます。多くの個人ブログやドキュメントサイトがこの方法で構築され、記事を書いて push するだけでサイトが自動更新されます。

### オープンソースプロジェクトへの参加

コードリポジトリをクラウドストレージとして使い、パソコンを替えたりマルチデバイスで作業したりする際、git pull 一回で最新コードを取得でき、USB メモリでコピーし回る必要がなくなります。

### オートメーションデプロイと CI/CD

コミットメッセージを明確に書く

### コードバックアップとマルチデバイス同期

こまめにコミット、小刻みに進める

## 八、初心者へのアドバイス

- **ブランチで開発し、メインブランチを直接修正しない：**個人プロジェクトでもブランチを切る習慣を身につければ、将来チームコラボレーション時に慌てません。

- **.gitignore を活用：**このファイルで Git にどのファイルをバージョン管理から除外するかを指定します（例：パスワード設定、コンパイルの産物、依存フォルダ）。ゴミファイルのアップロードを防ぎます。
- **優秀なオープンソースプロジェクトを多く見る：**GitHub Trending を巡回し、高スターのプロジェクトのコード構造、README の書き方、コラボレーションフローを見るのは、最も速い無料の向上方法です。
- **GitHub 学生パックを申請：**学生認証後、多数の有料ツールやサービスの無料枠（Copilot、ドメイン、クラウドサービスなど）を獲得でき、価値は数千ドルに上ります。
- **セキュリティリマインダー**パスワード、API キー、データベース接続文字などの機密情報を GitHub にコミットしないでください。特に公開リポジトリでは——一度漏洩するとクローラーに即座に取得され、深刻な結果をもたらします。
- **誤って機密情報をコミットした場合、ファイルを削除するだけでは不十分です（履歴に残っています）。**git filter-branch

**セキュリティに関する注意**

- 非公開リポジトリも絶対に安全ではありません。機密情報は環境変数で管理し、コードリポジトリに入れないのが最善です。
- 中国国内からの GitHub アクセスの注意点 `GitHub` は中国国内からのアクセスが時に不安定で、開けない、クローンが遅い場合があります。次の対策を試してください：(1) 国内ミラーサイト（`GitClone`、`ghproxy` など）でクローンを加速；(2) `Hosts` の設定やプロキシの使用；(3) 主に国内コラボレーションが目的なら、国産代替の Gitee（碼雲）
- も検討でき、機能は類似しアクセス速度も速いです。多くのチームが GitHub と Gitee の両方でホスティングし、グローバル化と国内速度を両立しています。

**応用編：GitHub Pages で無料個人ブログを構築**

GitHub Pages を使えば無料で静的ウェブサイトをホスティングでき、Jekyll、Hexo、Hugo などの静的サイトジェネレーターと組み合わせれば、個人ブログを簡単に構築できます。Markdown 記事を書いてリポジトリに push するだけでブログが自動更新され、HTTPS とカスタムドメインも標準装備です。無数の技術ブロガーのブログがこの方法で構築され、コストゼロ、広告なし、バージョン管理可能です。**九、まとめ：なぜ誰もが GitHub を知るべきか** GitHub はもはや「プログラマーのツール」ではなく、

**デジタル時代の仕事方式**

です：バージョン管理がすべてを追跡可能にし、コラボレーションメカニズムがチームを効率的に運営し、オープンソース精神が知識を自由に流通させます。コードを書かなくても、「コミット、ブランチ、マージ」という概念を理解すれば、「反復修正＋多人数コラボレーション」が必要なあらゆるコンテンツの管理に役立ちます——論文からドキュメント、ノートから企画書まで。

## 九、まとめ：なぜ誰もが GitHub を知るべきか

実践：GitHub 初体験を完了 **GitHub アカウントを登録し、プロフィールを整える（アバター、自己紹介、興味のある技術分野）**。hello-github

という公開リポジトリを作成し、「Add a README file」にチェックを入れる。

**リポジトリに入り、README.md の編集ボタン（鉛筆アイコン）をクリックし、Markdown で自己紹介を書いて変更をコミット。**

1. GitHub の検索バーで興味のあるキーワード（例：

2. python-tutorial）を検索し、好きなプロジェクトを 2 つ `Star` する。（応用）パソコンに Git をインストールし、リポジトリをローカルに Clone し、README を修正後にクラウドに push し、完全な「ローカルクラウド」ワークフローを完了。

3. 第二十四章：Root 権限と BL ロックの深掘り解説

4. この章は第二十章の内容を深く掘り下げるもので、スマートフォンの基盤メカニズムを真に理解したい、あるいはフラッシュを試みたいユーザー向けです。これらの操作にはリスクがあるため、慎重に行ってください。一、`Bootloader` の深い理解 Bootloader はスマートフォン起動時に最初に実行されるプログラムで、ハードウェアの初期化、OS カーネルの読み込みを担当します。Bootloader はスマートフォンの「門番」と理解できます。

5. BL ロックの動作原理：

# 第二十四章：Root 権限と BL ロックの深掘り解説

この章は第二十章の内容を深く掘り下げるもので、スマートフォンの基盤メカニズムを真に理解したい、あるいはフラッシュを試みたいユーザー向けです。これらの操作にはリスクがあるため、慎重に行ってください。

## 一、Bootloader の深い理解

：ユーザーが旧バージョンのシステムをフラッシュして既知の脆弱性を悪用するのを防ぎます。

### データ保護

- **：新システムをフラッシュしても、元のアカウントパスワードがなければユーザーデータにアクセスできません。**二、BL ロック解除の詳細フロー（Xiaomi を例に）
- **ロック解除権限の申請：**Xiaomi のロック解除ツールサイトで解除を申請。
- **アカウント連携：**スマートフォンの設定で Xiaomi アカウントを連携。

### ハードウェアレベルの保存と検証

BL ロックの状態（Locked/Unlocked）およびメーカーの公開鍵は、通常保護されたハードウェア領域（Qualcomm の devinfo パーティション、eFuse や RPMB パーティションなど）に保存され、ソフトウェアによる改ざんを防ぎます。起動のたびに Bootloader はこのセキュア領域から状態を読み取り、厳格な署名検証を実行するかどうかを決定します。

### デジタル署名検証

メーカーはシステム更新やイメージを発表する際、自身の秘密鍵で署名します。デバイス起動時、Bootloader は内蔵の公開鍵でシステムイメージ（boot.img、recovery.img など）を検証します。その仕組みは：

1. システムイメージのハッシュ値を再計算。

2. 公開鍵でイメージに付属の署名を復号し、ハッシュ値を取得。

3. 2 つのハッシュ値を比較し、一致すれば検証合格。これによりシステムイメージが第三者に改ざんされていないことを保証します。

### セキュアブートチェーン

BL ロックはセキュアブートチェーンの第一段階です。検証合格後、次段階のブートプログラム（カーネルなど）をロードして実行し、システム起動環境の信頼性を段階的に保証します。

### ロック vs 解除：BL ロックの両面影響

BL ロックの利害をより直感的に理解するため、5 つのコア次元での違いを対比表で示します：

| 核心機能 | ロック状態（デフォルト） | 解除後の状態 |
| --- | --- | --- |
| システムセキュリティ | 未署名の非公式システムのフラッシュを防止し、システム完全性と低層セキュリティを保護<br>高セキュリティ：公式署名イメージのみロード、悪意あるファームウェア改ざんを効果的に防御 | セキュリティ低下：サードパーティ ROM のフラッシュを許可、システム完全性検証メカニズムが無効化、マルウェアが Root 権限を取得するとシステムを深く改ざん可能 |
| システム安定性 | メーカーがテスト・最適化した公式システムの実行を保証し、安定性と快適さを確保<br>安定：公式ファームウェア使用、機能完備、OTA 更新が信頼できる | リスク増大：サードパーティ ROM に互換性問題がある可能性、システムクラッシュ、文鎮化、ハードウェア機能異常（カメラ、急速充電など）、OTA 更新失敗の可能性 |
| 保証権益 | メーカーは BL ロックで安定した公式システムを提供し、保証サービスを提供<br>保証有効：公式無料保証サービスを享受（非意図的損傷） | 保証無効：ほとんどのメーカーは BL 解除を能動的な保証放棄とみなします。再ロックしても履歴解除記録が検出されれば保証拒否される可能性 |
| データセキュリティ | 解除プロセスは通常データの強制消去を行い、旧データが新システム環境で悪用されるのを防止<br>データ安全：日常使用中、BL ロックがマルウェアのシステムパーティション変更能力を制限 | データリスク：解除時に全ユーザーデータが強制消去；解除後に正しく暗号化しないとデータ保存が不安定；紛失後にフラッシュでデータを抽出されやすい |
| 機能制限 | セキュア環境に依存する機能（決済、銀行 App など）の正常動作を保証<br>機能完備：すべての公式機能が正常に使用可能 | 機能制限：一部セキュリティ敏感 App（Google Pay、銀行 App など）が BL 解除状態を検出して実行を拒否する可能性；DRM サービスがダウングレードされ、高画質動画再生に影響 |

## 二、BL ロック解除の詳細フロー（Xiaomi を例に）

1. **：アカウントランクに応じ 7-30 日の待機が必要。**解除操作

2. **：Xiaomi ロック解除ツールを使用し、パソコンに接続して解除。**三、Root の複数の方案
3. **Magisk：**現代的な Root 方案、Systemless 設計、Root 状態を隠蔽可能。
4. **Kernelsu：**カーネルベースの Root 方案、より低層、隠蔽性が高い。

## 三、Root の複数の方案

- **：古参の Root 方案、現在は更新停止。**四、フラッシュ入門
- **フラッシュとはスマートフォンに新しい OS をインストールすることです。一般的なフラッシュの種類：**公式 ROM のフラッシュ
- **：公式システムのインストール、文鎮化からの復旧やダウングレードに使用。**サードパーティ Recovery のフラッシュ

## 四、フラッシュ入門

サードパーティ ROM のフラッシュ

- **：LineageOS、Pixel Experience など、異なるシステムを体験。**ROM 焼きとは、スマートフォンに新しいオペレーティングシステムをインストールすることです。一般的な ROM 焼きの種類：
- **公式 ROM 焼き：**公式システムをインストールし、文鎮復旧やダウングレードに使用。
- **サードパーティ Recovery 焼き：**TWRP など、バックアップ復元やサードパーティ ROM の書き込みに使用。

## 五、Samsung Knox：BL 解除の不可逆な代償

他の Android メーカーと異なり、Samsung スマホの BL 解除メカニズムには独特の「ヒューズ」設計があります：**Samsung スマホにおいて、解除は Knox ヒューズ作動をトリガーし、一部セキュリティ機能（Samsung Pay など）を永久に喪失；保護データをすべて消去；保証に影響。**具体的には：

- **ヒューズは使い切り**: スマホ内部に eFuse ヒューズがあり、BL 解除を検出すると永久に焼き切れ、復元できません。
- **影響を受ける機能**: Samsung Pay（交通系/銀行カード）、Knox セキュアフォルダー、企業 MDM 管理制御、一部健康保険系 App がすべて無効化されます。
- **データを強制消去：**ヒューズがトリガーされるとスマホは強制的に初期化され、すべてのローカルデータが消去されます（クラウドデータは影響を受けません）。

- **保証への影響：**Knox ヒューズトリガー後は保証サービスの対象外となることが公式に明示されています。

**行動する前に慎重に**

Samsung の Knox ヒューズは**不可逆的**で、一度トリガーされると元の工場状態には戻せません。Root 権限が必要な機能を使いたいだけなら、Root 不要の代替手段があるかまず調べてください。

**サードパーティ ROM 焼き**

：LineageOS、Pixel Experience など、異なるシステムを体験。

- 重要な警告
- ROM 焼きにはリスクがあり、操作には慎重さが必要です。次のような問題が発生する可能性があります：
- スマートフォンが文鎮化（起動不可）
- 保証の喪失

データの消失

**決済機能の異常**

- **ROM 焼きの前に必ず完全なバックアップを行い、チュートリアルを注意深く読み、機種とバージョンの一致を確認してください。**おすすめリソース
- **XDA Developers：**世界最大の Android 開発者コミュニティ、豊富な ROM 焼きチュートリアルと ROM リソースがあります。
- **Coolapk：**中国国内で有名なデジタルコミュニティ、中国語のチュートリアルや議論が豊富です。

## 六、よくある質問と回答（Q& A）

**Q1：BL ロック解除後に元に戻せますか？**

**A：**再ロック可能です。通常、公式の完全なフラッシュパッケージを書き込み、特定のフラッシュスクリプト（Xiaomi の `flash_all_lock.bat`）を使って再ロックします。ただし、再ロック後も、解除によってトリガーされたセキュリティヒューズ（Samsung の Knox など）は元に戻らない場合があります。

**Q2：ロック解除後に Root 化するには？**

**A：**BL ロック解除は最初のステップにすぎません。Root 化には通常以下が必要です：

1. カスタム Recovery（TWRP など）のフラッシュ。

2. Recovery 経由で Magisk などの Root 管理ツールをフラッシュし、boot イメージをパッチして Root 権限を取得。

# 第二十五章：中古スマートフォンの処理とデジタル遺産

ねえみんな、これまで *20* 章以上の「修行」を経て、あなたは「デジタル初心者」からかなり落ち着いたスマートフォンユーザーへと成長したことでしょう。でも、テクノロジー生活には常に巡りがあります——新しい恋人もいつかは古い恋人になります。古い相棒を「引退」させようとするとき、どうやって堂々と、安全に別れを告げればいいか知っていますか？

## 一、古いスマートフォンの「安楽死」：安全な売却・譲渡ガイド

核心原則：データ消去とは「削除」ではなく「上書き」です。写真をゴミ箱に入れてアルバムを空にすれば完了だと思う人が多いですが、スマホを譲渡した後、新しい持ち主がデータ復元ソフトでスキャンすると、古い写真、チャット履歴、身分証写真がすべて戻ってきます——このような「社会的死」現場は毎年上演されています。

1. **一、古いスマートフォンの「安楽死」：安全な売却・譲渡ガイド：**最後の完全バックアップを完了し、すべてのアカウント（iCloud、Huawei/Xiaomi/Samsung アカウント、Google アカウント）からサインアウトし、「スマホを探す」機能をオフにしてください——そうしないと次の使用者がアクティベーションロックで締め出されます。

2. **自分で「大掃除」：**現代のスマホの「初期化」は通常、暗号化消去を含みます——暗号鍵自体を破体を破壊し、鍵がなくなると全データが瞬時に意味のない文字列になります。

3. **最後の「目くらまし」（旧デバイスのみ必要）：**現代のスマホ（iOS 8+、Android 6+）はデフォルトでフルディスク暗号化（FBE）が有効で、初期化すると暗号鍵が破壊され、データは理論上復元不可能です。**この手順は不要**。極めて古い非暗号化デバイス（2015 年以前）のみ、初期化後にジャンクファイルでストレージを満たし、再度初期化する必要があります。

**各ブランドのデータ消去パス早見表：**原理は共通していますが、各ブランドで「初期化」のメニュー位置がわずかに異なります。主な違いは以下の通りです。

| ブランド | 消去エントリー | 主な違いと注意事項 |
|---|---|---|
| Huawei | 設定 > システムと更新 > リセット > 初期化 | 事前に Huawei アカウントからサインアウトし「スマホを探す」をオフにする必要があります。そうしないとアクティベーションロックがトリガーされます。純正モードではパスワード認証が必要な場合があります。 |
| Xiaomi / Redmi | 設定 > マイデバイス > 初期化 | 下部に「仮想 SD カードをフォーマット」と「すべてのコンテンツと設定を消去」の両オプションがあり、両方ともチェックする必要があります。Xiaomi アカウントからサインアウトする前に「デバイスを探す」をオフにしてください。 |
| OPPO / OnePlus | 設定 > その他の設定 > スマホをリセット > すべてのデータを完全に消去 | ColorOS は「完全消去」モードを提供し、ストレージを複数回上書きします。ロック画面のパスワード入力で確認が必要です。 |
| vivo | 設定 > システム管理 > バックアップとリセット > すべてのデータを消去 | 「スマホのストレージをフォーマット」にチェックを入れると SD カード領域も消去されます。vivo アカウントからサインアウトし、「スマホを探す」をオフにしてください。 |
| Samsung | 設定 > 一般管理 > リセット > 初期化 | 事前に Samsung アカウントを削除し「スマホを探す」をオフにする必要があります。一部機種には「自動上書き」オプションがあり、有効化を推奨します。 |
| iPhone | 設定 > 一般 > 転送または iPhone をリセット > すべてのコンテンツと設定を消去 | 事前に Apple ID からサインアウト必須。そうしないと次のユーザーが iCloud アクティベーションロックで締め出されます。iOS 15+ は「サインアウト時に iCloud データを保持」に対応し、後で確認できます。 |

**最も見落とされやすい 3 つのこと**

- **アカウントからサインアウトしていない：**スマホを買い手に売却した後に遠隔ロックされ、自分でも解除できず、最終的に身分証を持って公式サポートに行くしかなくなります。
- **SD カードを取り出していない：**初期化では外付け SD カード内の写真はフォーマットされません。手動で取り出すか、個別にフォーマットしてください。
- **USB メモリ/OTG 外付けデバイスを切断していない：**消去中に外付けデバイスが接続されていると、外付けデバイスのデータが誤消去されたり、消去処理が中断される可能性があります。

## 二、デジタル遺産：あなたのソーシャルな魂をどう安置するか？

スマホの中のデータは、もはや「仮想」のものではありません。あなたの資金、ソーシャル関係、創作物、さらには「デジタルアイデンティティ」を担っています。**人が亡くなった後、これらの資産をどう処理すべきでしょうか？**これはもはや抽象的な問題ではなく、ますます多くの家庭が現実に直面する難題です——家族がパスワードを知らず、プラットフォームが継承を許可せず、暗号資産が永遠に失われます。

**法的根拠：個人情報保護法第 49 条**

2021 年に施行された「中華人民共和国個人情報保護法」第 49 条は明確に規定しています：**自然人が死亡した場合、その近親者は自身の合法かつ正当な利益のために、死者の関連個人情報について本章に規定する閲覧、複製、訂正、削除等の権利を行使できる。ただし、死者が生前別の取り決めをした場合を除く。**
これは以下を意味します：

- **近親者には法的根拠があり**プラットフォームに死者のアカウントと個人情報の処理を申請できます。
- 「合法かつ正当な利益」には通常、遺産相続の手続き、アカウントの閉鎖、機密プライバシーの削除などが含まれます。
- 死者が生前に遺言や契約で別の取り決め（遺産連絡先の指定など）をしていた場合、その取り決めに従います。
- プラットフォームは「ユーザープライバシー」を理由に一律に協力を拒否することはできませんが、具体的なフローは各社で異なります。

### WeChat と Alipay：アカウントは継承できないが、資金は可能

この 2 つの中国人が最もよく使うアプリは、ルールがそれぞれ異なります：

- **WeChat アカウント：**「Tencent WeChat ソフトウェア利用許諾およびサービス契約」

に基づき、アカウントの所有権は Tencent に帰属し、ユーザーは使用権のみを有します。**アカウント自体は継承も譲渡もできません**。ただしアカウント内の**残高、理財通の資金**は個人財産に属し、近親者は死亡証明、親族関係証明、相続公証書を用いて TenPay に引き出しまたは移転を申請できます。

- **Alipay：**アカウントは同様に直接継承できませんが、Alipay 残高、Yu'ebao、理財通などの**資金は相続可能な財産に属します**。相続人は Alipay の「カスタマーサービスホットライン」または窓口で死亡証明、相続公証書を提出して手続きできます。Ant Fortune 系の理財商品はファンド会社のフローを経る必要があります。
- **チャット履歴：**法的には個人のプライバシーに属し、プラットフォームが自発的に家族に提供することはありません。遺産の手がかりに関わる場合、訴訟や公証申請で取得できます。

**見知らぬ人に「画面共有」や「リモートコントロール」を有効化。**

- **死者のアカウントのパスワードを家族に直接渡して「そのまま使い続ける」のは避ける**——プラットフォーム契約違反となり、資金が凍結されたりリスク管理がトリガーされた際に立証がより困難になります。
- **死者のスマホで認証コードを受け取って「引き出す」のは避ける**——不正使用と認定され、かえって法律に抵触する可能性があります。
- **引き延ばさない：**一部のアカウントは長期間未ログインで自動的に停止され、資金の回収がより困難になります。

### Apple「レガシーコンタクト」：事前に指定すれば労半して功倍す

iOS 15.2 以降、Apple は「レガシーコンタクト」機能を提供し、ユーザーが事前に信頼する連絡先を指定できます。亡くなった後、その連絡先は**死亡証明 ＋ レガシーコンタクトアクセスキー**で死者の iCloud データ（写真、メモ、連絡先など）へのアクセスを申請できます。操作手順：

1. iPhone で [**設定**] > [**ユーザー名**] > [**サインインとセキュリティ**] > [**レガシーコンタクト**]。
2. 「レガシーコンタクトを追加」をタップし、Face ID で本人確認します。
3. 連絡先を選択（連絡先から選択）すると、システムが**アクセスキー**。
4. アクセスキーをメッセージ、メール、または印刷した紙面で相手に渡し、大切に保管するよう伝えてください。
5. 複数の連絡先をバックアップとして追加でき、単一障害点を回避できます。

指定された人は Apple デバイスがなくても、キーと死亡証明で Apple 公式サイトからアクセスを申請できます。全体のフローは通常 7 日以内に完了します。

### Google「非アクティブアカウントマネージャー」：亡き後の事務を自動処理

Google は「非アクティブアカウントマネージャー（Inactive Account Manager）」を提供し、ユーザーが長期間ログインしない場合に事前設定した操作を自動実行できます——これは現在最も完成されたデジタル遺産メカニズムの一つです。

1. github.com/yourname**myaccount.google.com/inactive**、Google アカウントにログインします。
2. タイムアウト期間を設定：3 / 6 / 12 / 18 ヶ月未ログインでトリガー。
3. 信頼できる連絡先を追加（最大 10 人）、各人がアクセスできるデータ（Gmail、写真、Google ドライブなど）を設定。
4. オプション：自動返信メールを設定し、差出人に対応できないことを通知。
5. 最後に選択：期限到達後に**連絡先にデータを共有する**、それとも**アカウントを直接削除する**。
6. 電話番号をフォールバック認証として設定し、誤トリガーを防ぎます。

### 国内ソーシャルアカウント：追悼化と削除フロー

国内の主要ソーシャルプラットフォームは逝者のアカウント処理を 2 つのアプローチに分けます：**「追悼化」（アカウントを残し親族が偲べるようにする）**3. タスク（Task）：AI に何をするか明確に伝える**「削除」（完全に削除）**。家族は死亡証明、親族関係証明、申請人の身分証でプラットフォームのカスタマーサービスに申請できます。

- **Weibo：**アカウントの「追悼化」に対応し、親族が死亡証明を提出して申請します。アカウントはロックされてログイン不可になりますが内容は保持され、プロフィールに「このアカウントは追悼アカウントに設定されています」と表示されます。削除申請も可能で、相続公証の提出が必要です。
- **Douyin：**「私 - 設定 - フィードバックとヘルプ - アカウントとセキュリティ - アカウント削除」またはカスタマーサービス経由で、死亡証明、親族関係証明、申請人の身分証を提出し、審査通過後に削除または追悼状態に移行できます。
- **Bilibili：**カスタマーサービスチャネルで逝者の証明と親族証明を提出し、アカウント削除または追悼ページとしての保留を申請できます。UP 主の過去の動画の処理は著作権宣言と組み合わせる必要があります。
- **小紅書：**公式カスタマーサービスに申請し、死亡証明、親族関係証明を提出してアカウントの削除とコンテンツの削除を申請できます。

**重要なリマインダー**

- 各プラットフォームのルールは更新されるため、手続き前に公式カスタマーサービスの最新ガイダンスを確認してください。
- すべての証明書類は提出前にスキャンしてバックアップし、原本は大切に保管してください。
- 複数プラットフォームを一括手続きする際、統一的に**相続公証書**を 1 部申請すれば、手続きの往復を省けます。

## デジタル資産の相続の難題

ソーシャルアカウント以外にも、ますます多くの人が**真に価値のあるデジタル資産**を保有しており、その相続問題はより複雑です：

- **暗号資産（Bitcoin、Ethereum など）：**「秘密鍵が所有権」の原則に基づき、**秘密鍵がなければ資産は消滅したも同然**。相続人は法的文書でチェーン上に「申し立て」できません。助言：ニーモニック/秘密鍵を複数の物理的に安全な場所（銀行貸金庫 + 自宅金庫など）に分散保管し、遺言にアクセス方法を明記してください。
- **取引所アカウント：**銀行口座と同様に、相続公証書で取引所（Binance、OKX など）に処分を申請できますが、越境取引所のフローは長引きます。
- **ドメイン名：**相続可能な財産に属し、相続人がドメイン登録業者（Alibaba Cloud、GoDaddy など）に相続公証を提出して移転手続きができます。
- **知的財産権（オリジナル記事、動画、特許、ソフトウェア著作権）：**著作権の財産権は相続可能（人格権は非相続）で、収益権は相続人が著者の死後 50 年まで享受できます。
- **クラウドストレージとオンラインストレージ：**コンテンツデータは個人に帰属し、相続可能です。ただし一部のプラットフォーム契約ではアカウント自体の譲渡が不可とされ、先にダウンロードしてから削除する必要があります。

### 暗号資産が「消滅」する真のコスト

Chainalysis のレポートによると、世界中で既に **300 万枚以上の Bitcoin** が秘密鍵の紛失により永遠に回収不可能で、現存総量の 6 分の 1 に相当します。暗号資産を保有している場合、少なくとも 1 人の絶対的に信頼できる人に**復元パス**を知らせてください——さもなければこれらの富は永遠にブロックチェーン上で眠り続けます。

## 「デジタル遺言」テンプレート

従来の財産遺言ではデジタル資産を管理できません。毎年「デジタル遺言」添付書類を更新し、正式な遺言と一緒に保管することを推奨します。以下のテンプレートを参考にしてください：

**《個人デジタル資産処分添付書類》（テンプレート）**

**第 1 部：アカウントリスト**（アカウント名と所属プラットフォームのみ記録し、**パスワードは書かない**）

- ソーシャル系：WeChat（アカウント xxx）、Weibo、Douyin、Bilibili、小紅書など
- 決済系：Alipay、WeChat Pay、各銀行カードアプリ
- クラウド系：iCloud、Huawei Cloud、Xiaomi Cloud、Google アカウント、Microsoft アカウント
- メール系：QQ メール、NetEase メール、Gmail など
- その他：オンラインストレージ、サブスクリプションサービス、ゲームアカウントなど

**第 2 部：パスワード管理方案**

- マスターパスワードの保管方法：パスワードマネージャー（Bitwarden、1Password など）を使用し、マスターパスワードは紙に書いて自宅の金庫に封印。
- 復旧キー/ニーモニック：分散保管し、少なくとも 2 カ所の物理的場所（銀行貸金庫 + 自宅金庫など）。
- 授権査問人：氏名、身分証番号、本人との関係、連絡先。

**第 3 部：処分の意思**

- 各アカウントを「追悼化」するか「削除」するか。
- iCloud / Google データに「レガシーコンタクト」または「非アクティブアカウントマネージャー」を指定しているか。
- 暗号資産を誰が相続するか、秘密鍵の取得パスの説明。
- 著作権系作品の収益を誰が享受するか。

**第 4 部：連絡先とプラットフォーム**

- 信頼する執行人の氏名、連絡先、予備連絡先。
- 常駐プラットフォームのカスタマーサービス窓口と必要な証明書類リスト。
- 本添付書類の更新日：_____ 年 ___ 月 ___ 日、次回確認日：_____ 年 ___ 月 ___ 日。

**「パスワード」に関する法的注意**

パスワードを遺言に直接書き込むと漏洩リスクがあり、遺言公開後はすべての相続人が見られます。**より安全な方法は：遺言には「アカウントリスト + 処分の意思 + パスワード保管場所」のみを書き、実際のパスワードはパスワードマネージャーのマスターパスワードで信頼する執行人に別途伝える**。これにより法的要件を満たしつつ、パスワー

ド漏洩リスクを低減できます。

## 三、エコな最後の旅路

スマホ 1 台に金、銀、銅、希土類など数十種類の金属が含まれており、無断で廃棄すると土壌汚染と資源浪費につながります。正しいリサイクルルートは 3 種類あり、それぞれ特徴が異なります：

| ルート | 買取価格 | データセキュリティ | ：ハイエンドプロフェッショナル制作。iPhone 15 Pro シリーズは Apple ProRes 動画撮影に対応（最大 4K 60fps）しますが、ProRes RAW 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。 |
|---|---|---|---|
| 公式下取り（Huawei/Apple/Xiaomiなど） | 中程度、補助券が併用されることが多い | 最高、公式の標準フローあり | 同ブランドの新機種に買い替える予定なら最もお得 |
| サードパーティ回収プラットフォーム（愛回収、転転、回収宝など） | やや高い、価格比較可能 | 比較的高い、データ消去レポートあり | 直接現金化したい、ブランドに縛られたくない |
| 公益寄付（農村学校、慈善団体など） | 経済的リターンなし | 自分で完全消去が必要 | デバイスがまだ使える、善行をしたい |
| 再利用（自宅で改造） | 新デバイス費用を節約 | 元の所有者が使い続ける、リスクなし | デバイスの性能がまだ十分、家庭で改造ニーズがある |

- **：最も確実な方法、定期的に貴重なデータをローカルハードディスクにダウンロード。：** 最も推奨される方法で、フローが標準化され、データセキュリティが保証されます。Apple、Huawei、Xiaomi などのブランド公式サイトに「下取り」の入り口があり、評価後に新機種代に充当できます。
- **公式回収/下取り：** 信用できる大型回収プラットフォームを選び、注文後に集荷に来てもらい、検査後に送金されます。必ず**「データ消去レポート」を提供する**プラットフォームを選び、証憑を保管してください。

- **公益寄付：**地元の農村学校やコミュニティ公益団体に連絡し、旧スマホを教育用タブレットや高齢者向けスマホに改造して使用できます。**寄付前に必ず完全な消去フローを実行し、すべての SIM/SD カードを取り外してください。**
- **正規回収プラットフォーム：**車載ナビ、スマートホームハブ、監視カメラ、ベビーモニター、電子フォトフレームに改造。旧スマホ 1 台であと数年活用できます。

**電子ゴミの真実**

スマホ 1 台の回路基板には**金、銀、パラジウム、銅**などの貴金属が含まれ、廃棄回路基板 1 トンあたりの金含有量は金鉱石の数十倍です。しかし無断廃棄すると、鉛、水銀、カドミウムが土壌と地下水に浸透し、汚染は数十年続きます。**正しくリサイクルすれば、環境を保護し、希少金属を循環再生できます。**

**活用**

1. 最後の完全バックアップ：写真をクラウドにアップロード、重要ファイルをパソコンにコピー、チャット履歴をエクスポートして保存。
2. すべてのアカウントからサインアウト：iCloud / Huawei / Xiaomi / Google アカウント、「スマホを探す」をオフに。
3. SIM カードと SD カードを取り出し、すべての OTG 外付けデバイスを切断。
4. 暗号化初期化を実行、必要に応じて手動で「ストレージをフォーマット」にチェック。
5.（旧デバイスのみ）ジャンクファイルでストレージを満たし、再度初期化。
6. 次の行き先を選ぶ：公式下取り、正規回収プラットフォーム、公益寄付、または家族への譲渡。
7.「デジタル遺産リスト」を同期更新し、廃棄済みデバイスをリストから削除。

**今月のタスクリスト**

1. 現在のメインスマホに Apple「レガシーコンタクト」を指定、または Google「非アクティブアカウントマネージャー」を設定。
2. 本節のテンプレートで《個人デジタル資産処分添付書類》を起草し、家族と保管場所を相談。
3. 自宅に 2 台以上の未使用旧スマホがないか確認し、公式または正規プラットフォームの回収を予約。

# 第二十六章：今のスマホ技術の最前線

旧スマホの「亡き後の事」について語り終えたところで、目を現在に戻しましょう。*2024*□*2026* 年はスマホ技術の集中爆発期です：*AI* がクラウドからポケットに詰め込まれ、衛星通信がフラッグシップから中端機に降りてきて、折りたたみ画面が「お試し」から「常用」になり、バッテリー技術も長年見られなかった材料レベルの突破口を迎えました。本章では、これらの起きている技術の最前線を整理し、「今買い替えるべきか」「何に買い替えるか」の判断を助けます。

## 一、AI スマホ：マーケティングの煽りか、真の変革か？

2024 年以降、ほぼすべての Android メーカーがフラッグシップ機で「AI スマホ」の旗を掲げ、Apple も iOS 18 で Apple Intelligence を提供開始しました。しかし、一体どんなスマホが「AI スマホ」と言えるのでしょうか？ 業界ではおおよそ 3 つの判断基準が形成されています：

- **オンデバイス大模型：**スマホ本体で 7B 程度のパラメータ数の言語モデル（Qwen、Llama、DeepSeek の軽量版など）を稼働でき、クラウドに依存せずテキスト要約、翻訳、文案生成などのタスクを完了できる。判断方法：機内モードでも AI 機能が使えればオンデバイスモデル。

- **NPU 演算力（TOPS）：**NPU（ニューラルプロセッシングユニット）の演算力は TOPS（1 秒間に 1 兆回の演算）で表されます。2024 年のフラッグシップチップ（Snapdragon 8 Gen3、Dimensity 9300 など）の NPU 演算力は 40-70 TOPS の範囲で、2025-2026 年の新世代チップは概ね 100 TOPS を突破し、これがオンデバイスで大モデルを稼働するハードルです。

- **AI Agent シーン：**「質問応答」だけでなく、アプリをまたいでタスクチェーンを実行できる——例えば「昨日の会議の写真をアルバムにまとめて張三に共有して」と指示すると、システムがアルバム、ファイル、WeChat など複数のアプリを自動的に呼び出して完了。これが AI スマホと「AI アシスタント付きスマホ」を区別する核心の指標です。

## 二、各社の AI 戦略比較：本気で取り組むのは誰、OEM 的なのは誰？

各社の AI アプローチには明らかな差があり、購入時に注目できます：

- **Apple Apple Intelligence（iOS 18+）：**システムレベルアプリとの統合を重視——メール要約、通知の優先度、文章の推敲、アルバムの AI レタッチ。オンデバイスモデルが主体で、複雑なタスクは Private Cloud Compute にクラウド処理。長所はプライバシー保護とシステムの滑らかさ、短所は中国語シーンの提供が遅く、一部機能は 2025 年に段階的に対応。

- **Google Gemini Nano:** Pixel 8/9 シリーズに内蔵、「サークルで検索」、Magic Compose、録音要約などに対応。Gemini Nano は真のオンデバイス小モデルで、オープンソースエコシステムに優しく、Android AI スマホのリファレンス実装です。

- **：システムレベル AI 能力、AI 画像検索、AI 要約、AI リアルタイム字幕、オンデバイス大規模モデル対話などをサポート。：**HarmonyOS の小芸に接続し、オンデバイスでの文案生成、AI レタッチ、AI 消去（通行人の除去）、通話要約に対応。HarmonyOS NEXT 5.0 以降はさらにシステムレベルで統合され、AI 能力とシステムが深く融合。

- **Xiaomi MiLM:** HyperOS に内蔵、AI ポートレート、AI 要約、AI 画師、超解像度に対応。Xiaomi は「端末-クラウド協同」路線で、オンデバイスで軽量モデルを稼働し、重いタスクはクラウドへ。

- **OPPO（Andes 大モデル）と vivo（BlueLM 大モデル）:** ColorOS 15 と OriginOS 5 に深く統合、AI 消去、AI 書類要約、AI 画像認識 Q& A、自然言語での写真検索（「去年海辺で撮った写真を探して」）を主力。

**選び方のアドバイス** AI で仕事の書類を処理したり情報を整理することが頻繁にあるなら、オンデバイス演算力が強く、AI 機能がシステムレベルで統合された機種を優先。たまに AI レタッチを使うだけなら、主流の中端機で十分です。

## 三、衛星通信：フラッグシップの「目玉」から中端機の標準装備へ

衛星通信はこの 2 年で最も実質的な技術の下方普及の一つです。2022 年の Huawei Mate 60 と iPhone 14 の発表時はフラッグシップ専用でしたが、2025 年には 2000-3000 元の価格帯にまで降りてきました。

- **北斗ショートメッセージが中端機に降下：**北斗衛星のショートメッセージ機能（1000 漢字以内の SMS+ 位置情報の送信可）は Redmi、realme、iQOO などの中端機に降下し、月額不要、無料で利用可能。アウトドアハイク、自運旅行などのシーンに適しています。

- **天通衛星通話：**Huawei Mate 60/70 シリーズ、Honor Magic6 シリーズなどが天通一号衛星通話機能に対応し、衛星電話の発着信、衛星メッセージの送信が可能。キャリアの衛星プランの契約が必要（約月 10-200 元）。無人地域に頻繁に入るユーザーに適しています。

- **iPhone 衛星メッセージ（SOS）：**iPhone 14 以降の機種が衛星 SOS に対応し、緊急サービス機関にテキストメッセージを送信し位置を共有できます。2 年間無料で提供され、一部地域では継続利用も可能。

- **日常撮影：1080P/4K + 30fps + H.265 エンコード、画質とファイルサイズのバランス。：**都市ユーザーはまず使いませんが、アウトドア愛好家、辺境地域へ出張するビジネスマンには衛星機能付き機種を強く推奨します——緊急時に命を救えます。

## 四、折りたたみ画面エコシステムの成熟度：メイン機として使えるか？

折りたたみ画面は「技術のお試し」から「メイン機として使用可能」へと移行しました。折りたたみ画面の成熟度を判断するには 3 点を見ます：ハードウェアの耐久性、App の適配度、インタラクションエコシステム。

- **ハードウェアの耐久性：**ヒンジ寿命は普遍的に 50 万回以上（1 日 100 回開閉で 13 年使用可）、水滴ヒンジが画面の折り目を減らし、IPX8 級防水が標準装備に。内画面の UTG ガラス + 保護フィルムにより画面がもはや「繊細」ではなくなりました。

- **App の適配：**主要 App（WeChat、Taobao、Douyin、Bilibili、Office、WPS）は大画面向けに適配済みですが、まだ約 30% のロングテール App が未適配で、引き伸ばしや黒帯で表示されます。購入前によく使う App を試すことを推奨します。

- **ハングング状態のインタラクション：**90°-120° に折りたたむと、一部 App が「上半分にコンテンツ表示、下半分で操作」のハンギングモードに対応し、ビデオ通話、自撮り、動画視聴に適しています。カメラのハンギング自撮りは折りたたみ画面独自の体験です。

- **分割画面とマルチタスク：**大画面で 2 つの App を分割画面で同時に開き、さらには 3 つの App を小窓で並存させることができます。HyperOS、ColorOS、OriginOS の分割画面ロジックは比較的成熟していますが、iOS（iPadOS スタイル）の分割画面は iPhone ではまだ提供されていません。

- **価格：**横折り（本タイプ）は 1 万元級から 5000-7000 元帯に下がり、縦折り（フリップタイプ）は 4000-5000 元帯に下がりました。2025 年に一部メーカーが 3000 元帯の縦折りを発表し、折りたたみ画面が真正に大衆市場に入りました。

**選び方のアドバイス：**ヘビーな読書、書類処理、動画ユーザーなら横折り画面が検討に値します。携帯性とソーシャル属性を追求するなら縦折り画面がより適しています。実店舗で少なくとも 30 分は触ってから決めることを推奨します。

## 五、6G と Wi-Fi 7：次世代接続の速度と境界

6G はまだ本格的に落地していませんが、関連技術はすでに進行中です：

- **6G の進展：**3GPP は 2027 年に 6G 標準策定を完了し、2030 年頃に商用化する計画です。6G の理論ピーク速度は 1 Tbps（5G の 50 倍）で、遅延は 0.1ms と低く、「通信・感知一体化」（通信 + 感知）をサポートし、スマホが低精度レーダーとしても使えるようになります。現在は各社ともプロトタイプ検証段階で、コンシューマー製品にはまだ程遠いです。

- **Wi-Fi 7（802.11be）：**2024 年以降フラッグシップ機の標準装備となり、理論ピーク速度は 46 Gbps（Wi-Fi 6 の 4 倍）で、320MHz 帯域幅、4K-QAM、MLO（マルチリンク同時伝送）をサポート。実測では Wi-Fi 7 ルーター環境下でフラッグシップ機の転送速度は 2-5 Gbps に達し、Wi-Fi 6E より 2-3 倍高速です。Wi-Fi 7 スマホに買い替えたら、性能をフルに発揮するには Wi-Fi 7 ルーターの併用が必要です。

- **Bluetooth 5.4 と LE Audio：**2024-2026 年の新機種は普遍的に Bluetooth 5.4 と LE Audio（低消費電力オーディオ）に対応し、2 組のイヤホンを同時に接続して同じ曲を聴けます。Auracast ブロードキャストオーディオで公共場所で複数人が同じ音源を同時に聴けます。

## 六、シリコン炭素負極バッテリー：ついに材料レベルの突破口が到来

スマホのバッテリー持ちは長年「1 日 1 回充電」と批判されてきましたが、2024 年以降のシリコン炭素負極バッテリー技術の普及で状況が実質的に改善しました。

- **技術原理：**従来の黒鉛負極の理論容量は約 372 mAh/g ですが、シリコンの理論容量は 4200 mAh/g（約 10 倍）に達します。ただしシリコンは充放電時に 300% の体積膨張を起こし、粉化しやすいです。シリコン炭素負極はナノシリコン-炭素複合構造により、シリコンの膨張問題を許容範囲に制御し、バッテリーのエネルギー密度を 15%-30% 向上させます。

- **：コンピュータビジョン（CV）モデルが画像内容をベクトル化（Embedding）、テキストクエリと類似度マッチング。この「マルチモーダル検索」はスマホアルバム管理の大きな飛躍。：**2024-2025 年にシリコン炭素負極を採用したフラッグシップ機（Xiaomi 14 Ultra、Honor Magic 6 至臻版、vivo X100 Pro など）は、7-8mm のボディに 5000-6000 mAh のバッテリーを搭載し、前世代より約 800-1000 mAh 多く、ヘビー使用でも 1 日半持続します。

- **百ワット急速充電との組み合わせ：**シリコン炭素負極は急速充電との互換性が良く、100W/120W 急速充電で 17 分で満充電はもはやニュースではありません。2025 年の一部機種は 150W-240W に対応し、10 分以内に 80% まで充電します。

- **選び方のアドバイス：**ヘビーユーザーなら、「シリコン炭素負極 + 百ワット急速充電」の組み合わせを採用した 2024 年以降の機種を優先すると、バッテリー不安が大幅に軽減されます。

## 七、本章の概要：一言でこれらの最先端を記憶

- AI スマホの 3 基準：オンデバイス大モデル、NPU 演算力 TOPS、AI Agent のアプリ間シーン。
- 衛星通信：北斗ショートメッセージが中端機に降下、天通通話は依然フラッグシップ専用。
- 折りたたみ画面：ハードウェア成熟、App 適配は 7 割、ハンギング/分割画面は独自体験。
- 6G はまだ道半ば、Wi-Fi 7 は既に落地、Bluetooth LE Audio は普及開始。
- シリコン炭素負極 + 百ワット急速充電：バッテリー持ちと充電のダブル突破。

**試してみよう：次のスマホに「技術ニーズリスト」を作る**

紙を 1 枚出し、「必須/プラス/不要」の 3 段階で、AI、衛星、折りたたみ画面、Wi-Fi 7、急速充電、バッテリー容量へのニーズをリストアップ。次に実店舗に行った時、店員が推奨する「セールスポイント」が本当のニーズスか押し売りかを素早く判断できます。例えば：

- 必須：5000mAh 以上のバッテリー、80W 以上の急速充電、NFC、IP68 防水。
- プラス：オンデバイス AI 要約、Wi-Fi 7、北斗ショートメッセージ。
- 不要：折りたたみ画面（バー型が好み）、衛星通話（遠出しない）、240W の極限急速充電。

**本章のまとめ**

- AI スマホの 3 大判断基準：オンデバイス大モデル、NPU TOPS、AI Agent シーン
- 各社の AI 戦略：Apple はプライバシー統合重視、Google はオンデバイスオープンソース、Huawei/Xiaomi/OPPO/vivo はそれぞれ得意分野が異なる
- 衛星通信：北斗ショートメッセージは中端機に降下、天通通話は依然フラッグシップ専用
- 折りたたみ画面：ハードウェアは成熟、App 適配は約 70%、ハンギング状態と分割画面は独自体験
- 6G 標準は 2027 年策定、Wi-Fi 7 は普及済み、Bluetooth LE Audio は落地開始
- シリコン炭素負極バッテリーが 5000-6000mAh をフラッグシップ標準にし、バッテリー不安が大幅に軽減

# 五、実践編（第 27-32 章）

# 第二十七章：お父さんお母さんもラクラク——スマホのシニア向け完全ガイド

正直に言うと、この章を書いている時に両親のことを思い出した。母は“システム更新”のポップアップが出るたびに、スマホを壊すのが怖くてタップできない。父の *WeChat* の文字は蟻より小さいが、大きくする方法を知らず、二年間目を細めて見ていた。これは“バカ”だからではなく、スマホのデフォルトが全員を二十代だと思い込んでいるからだ。この章では、一番分かりやすい言葉で、お年寄りがスマホを使う際のもっとも頭痛の種を一つずつ解決していく。

## まず心構えを正そう：これは“お年寄り使いこなせない”ではなく“スマホが作れていない”のだ

設定をいじる前に、一つの考え方を明確にしておこう：**現在のスマホの UI は、設計段階からあまり高齢ユーザーを考慮していない**。ポップアップは小さくて見えない、操作ジェスチャーは正確なタッチを要する、アイコンがどれも似ている——これらはユーザーの問題ではなくプロダクトの問題だ。だからお年寄りにスマホを教える際、最初にすべきは教えることではなく、環境を整えることだ。**環境を整える**。環境さえ整えば、問題の大半は消えてしまう。

お年寄りにスマホを教える三つの鉄則：(1) **急かさない**——あなたは十年スマホを使っているが、彼らはまだ三ヶ月かもしれない。(2) **例えを使う**——“Wi-Fi は家の水道管のようなもの、蛇口をひねって初めて水が出る”、こういう例えは専門用語より一万倍分かりやすい。(3) **書き留める**——口頭で一度教えても忘れるかもしれない。紙に重要ステップを書いてスマホケースの裏に貼っておくのが一番効く。

## 一、第一歩：「簡単モード」をオンにする——スマホがスッキリする

現在のスマホにはほぼ“簡単モード”（機種によっては“ケアモード”“シニアモード”“お年寄りモード”）が内蔵されており、オンにするとスマホ全体が大きく変わる：

- **文字とアイコンが大きくなる**——もう老眼鏡なしで見られる。

- **ホーム画面のアイコンが大幅に整理される**——電話、SMS、カメラ、アルバム、WeChat など最もよく使う App だけを残し、残りはすべて隠す。
- **操作がシンプルになる**——派手な機能（左端のパネル、スマートアシスタント、フローティングウィンドウなど）はデフォルトでオフになり、UI はスッキリ志向。
- **音が大きくなる**——着信音、通知音、通話音量がすべて自動で最大になる。

**各ブランドの簡単モードのオンにする方法（そのまま操作すれば OK）：**

| ブランド | モード名 | オンにする方法 |
|---|---|---|
| iPhone | アクセシビリティ（専用の簡単モードはないが、個別に大きく設定可能） | 設定 > 表示と明るさ > 文字サイズ、最大までドラッグ。さらに” アクセシビリティ > 表示と文字サイズ” で” 太字テキスト”” ボタン形状” をオン |
| Apple 急速充電 | 簡単モード | 設定 > システムと更新 > 簡単モード > オン |
| Xiaomi/Redmi | シンプルモード | 設定 > その他の設定 > シンプルモード > オン |
| OPPO/OnePlus | 簡単モード | 設定 > 特殊機能 > 簡単モード |
| vivo/iQOO | シンプルデスクトップ | 設定 > デスクトップ、ロック画面と壁紙 > シンプルデスクトップ |
| Samsung | 簡単モード | 設定 > 表示 > 簡単モード |

他のブランドを使っている場合、“設定”上部の検索ボックスに“簡単”または“シニア”と入力すればだいたい見つかる。見つからない場合は、下記の“手動で大きくする”方法を一つずつ設定すれば同じ効果が得られる。

## 二、簡単モードを使わなくても手動で OK——“大きな文字 + 大きなアイコン”の五ステップ

お年寄りによっては簡単モードの UI を“年寄りくさい”と嫌がる。その場合は以下の五項目を手動で調整すれば同じ効果がある：

1. **文字を最大にする：**設定 > 表示 > 文字サイズ、一番右までドラッグ。ついでに” 表示サイズ”（機種により” 表示スケール”）も最大にすると、すべての UI 要素が一緒に大きくなる。

2. **アイコンを大きくする：**ホーム画面で二本指でピンチ（写真を縮小するように）してデスクトップ設定 > デスクトップレイアウト（またはアイコンサイズ）に入り、マス目が最も少ないレイアウト（例：3×5）を選ぶと、アイコンが自然に大きくなる。

3. **”高コントラストテキスト”をオンにする：**文字の輪郭をよりシャープにする。”アクセシビリティ”の中にこのスイッチがある。

4. **不要な通知をオフにする：**設定 > 通知、電話、SMS、WeChat 以外の App 通知をすべてオフにする。これでスマホがいちいち鳴らず、うるさくなくなる。

5. **よく使う連絡先をホーム画面に置く：**ホーム画面の空白部分を長押し > ウィジェット追加 > 連絡先、配偶者、子供、医者、緊急連絡先のクイックダイヤルをホーム画面に置く。ワンタップで発信でき、連絡帳を探す必要がない。

**実践：お年寄りのスマホを”リフォーム”しよう**

1. 簡単モードをオンにする（上記のブランド別表通りに操作）。
2. バッテリー残量の数値表示をオン（設定 > バッテリー > バッテリー残量%）、残量がいつでも分かるようにする。
3. 通知パネルを開き、Bluetooth、NFC、機内モードなどお年寄りには不要なクイックスイッチを削除または隠す。
4. ホーム画面に 4 つのクイックダイヤルを置く：配偶者、子供、かかりつけ医（いれば）、社区/管理事務所。

## 三、詐欺防止——この章は必ずお年寄りと一緒に読もう

高齢者はテレコミ詐欺の最大の被害層だ。警戒心が低いのではなく、詐欺師の手口が綿密に設計され、高齢者の心理的弱点を狙っているからだ。以下は私が整理した最もよくある詐欺の手口と対処法だ。お年寄りに直接読み聞かせるか、印刷して電話の横に貼っておくことをお勧めする。

### 六大よくある詐欺（見たらすぐ電話を切る）：

- **(1)”お孫さんが事故に遭った、すぐ送金して”**——詐欺師は”お孫さんが外でトラブルを起こして警察に捕まった、保証金が必要”と言う。覚えておこう：**送金しないと”釈放しない”という電話はすべて詐欺だ。**公安・検察・裁判所が電話で金銭を要求することはない。

- **(2)”あなたの医療保険カードが停止されました”**——SMS で医療保険カードに異常があると言い、リンクをタップして”有効化”させる。覚えておこう：**医療保険局、社会保障局が SMS でリンクを促すことはない。**この場合は 12333（社保公式ホットライン）に電話して確認する。

- **(3) ” 高収益の財テク、絶対に儲かる”**——WeChat グループや SMS で各種” シニア財テク”” 国家支援プロジェクト” を推奨し、高額リターンを約束する。覚えておこう：**年利 5% 以上で” 元本保証” を謳うものは 99.9% 詐欺だ。**

- **(4) ” 私はあなたの上司/古い友人だ、番号が変わった”**——知人を装って金を借りる。覚えておこう：**電話して確認するのが一番効く**、” 電話に出られない” と言うやつは八割詐欺師だ。

- **(5) ” 無料で卵/健康食品の講座”**——線下の無料イベントで” 健康講座” を聴かせ、法外な価格の健康食品を売りつける。覚えておこう：**棚から牡丹餅は落ちない、無料イベントの裏には商売がある。**

- **(6) ” スマホがウイルス感染した、リモート修復が必要”**——詐欺師がリモートコントロール App（向日葵、TeamViewer など）をダウンロードさせ、リモートでスマホを操作して金を奪う。覚えておこう：**誰にもリモートでスマホを操作させてはいけない！**スマホメーカー、通信キャリア、銀行が電話でリモートコントロールソフトのインストールを促すことはない。

**三重の防御ラインを敷けば、90% の詐欺を防げる：**

1. **第一のライン：スマホ内蔵のブロック機能**——Android スマホは” 迷惑ブロック” をオン（電話 App またはスマホ管理 App 内）、iPhone は” 不明な発信者を消音” をオン（設定 > 電話 > 不明な発信者を消音）。

2. **第二のライン：キャリアのブロック**——SMS で”KTFSR” を 10086 に送信、China Mobile の高頻度迷惑電話ブロックを無料で有効化。China Telecom や China Unicom にも類似サービスあり。キャリアのカスタマーサービスに電話して開通してもらえばよい。

3. **第三のライン：国家反詐欺センター App**——アプリストアで” 国家反詐騙センター”App をダウンロード、登録して着信警告をオンにする。この App は公安のデータベースと連携し、詐欺師の番号がかかってくると即座に識別する。

**お年寄り向け” 防詐欺のお呪い”（印刷して電話の横に貼る）**

- **知らない電話で送金要求切る！**
- **SMS のリンクをタップさせようとする削除！**
- **高収益・元本保証と言うブロック！**
- **リモートコントロールソフトを入れさせようとする即切る！**
- **本物の警察は電話で事件を処理せず、ましてや金を要求しない！**
- **迷ったら、まず子供に電話して確認しよう。**

## 四、リモートサポート——離れていても親のスマホを修理できる

お年寄りが問題に遭遇した時、電話でいくら説明しても該当するボタンが見つからないことがある。そんな時に“リモートサポート”機能が役立つ——自分のスマホから相手のスマホ画面を見て、操作を手伝える。各ブランドの方法は異なる：

- **Huawei/Honor：**内蔵の”Changlian” アプリに” 画面共有” 機能がある。親子で Changlian を入れ、ビデオ通話を開始して” 画面共有” をタップすれば、相手の画面に丸を描いて” ここをタップ” と教えられる。
- **Xiaomi/Redmi：**Xiaomi のビデオ通話 App に” リモートサポート” があり、画面共有 + 音声通話の同時進行に対応。
- **：一部機種はアンロック可能、申請が必要。：**内蔵の” ファミリースペース” または” リモートガード”、設定で” リモート” と検索すれば見つかる。
- **汎用ソリューション（全ブランド対応）：**” 向日葵リモートコントロール”App をダウンロード（無料版で十分）、双方にインストール。同じアカウントで登録するか友達追加し、親にスマホに表示された認証コードを読んでもらえば、リモートで画面を見て操作できる。
- **：iOS メモ標準のスキャン機能がすでに十分優れています。メモを開くカメラアイコンをタップ「書類をスキャン」を選択、書類に向けると自動的に枠を認識して透視を補正します。撮影後に色調整ができます——「白黒」モードを選ぶとスキャナーに最も近い効果になり、「グレースケール」は画像入りの資料に適しています。：**FaceTime 通話中に” 画面共有” に対応。FaceTime 通話中に画面共有ボタンをタップすれば、相手があなたの画面を見られる。ただし iOS のセキュリティ制限によりリモート操作はできず、” 指示型の指導” のみ。

## 五、WeChat の“シニア改造”——お年寄りにこれだけ伝えれば OK

WeChat はお年寄りが最もよく使う App だが、UI が密集しすぎている。以下は即効性のある調整：

- **WeChat 内の文字を大きくする：**WeChat > 私 > 設定 > 一般 > 文字サイズ、最大までドラッグ。これは WeChat 内の文字にのみ影響し、システム全体の文字サイズには影響しない。
- **” ケアモード” をオンにする：**WeChat > 私 > 設定 > ケアモード > オン。オンにすると文字とボタンが大きくなり、色のコントラストが強化され、ワンタップで” 文字メッセージを読み上げ”（テキストメッセージをタップするだけで読み上げられる）。
- **重要な連絡先に分かりやすいメモを付ける：**子供や親戚の名前を連絡先で分かりやすく変更する——例えば” 長男-小明”” 次女-小紅” など、WeChat を開けば誰だかすぐ分かるように。

- **”音声入力”を教える（タイピングの代わり）：**チャット欄のキーボードで、スペースキーを長押し（Android）またはマイクアイコンをタップ（iPhone）、直接話しかければ文字になって送信される。認識精度はすでに非常に高く、ピンイン入力ができないお年寄りには”画期的”な便利さだ。

- **”発見”ページの不要な機能をオフにする：**WeChat > 私 > 設定 > 一般 > 発見ページ管理、”モーメンツ”” 動画アカウント”” ライブ”” ショッピング”” ゲーム”など使わないものをすべてオフにし、干渉を減らす。

## 六、スマホを失くしたら？お年寄り向け“緊急対応プラン”を作る

備えあれば憂いなし。スマホを失くす前に以下を済ませておこう：

1. **”スマホを探す”がオンになっているか確認：**iPhone は設定 > あなたの名前 > 探す > iPhone を探す；Android は設定で”デバイスを探す”を検索。このスイッチがオンになっていることを必ず確認。

2. **重要なパスワードとアカウントを紙に書く：**Apple ID / Huawei アカウント / Xiaomi アカウントのパスワードを家の引き出しに一部、信頼できる子供に一部。スマホ紛失後にこれらのアカウントにログインしてリモートロックと位置追跡が必要になるので、パスワードを忘れると大変だ。

3. **お年寄りに一言教える：”スマホを失くしたら、すぐ家族に電話して。”**家族はすぐに iCloud または Android のデバイス探索 Web ページにログインし、リモートで”紛失モード”を設定できる。そうすれば拾った人も開けず、使えない。

## 七、ワイヤレスイヤホンと補聴器の互換性——見落とされがちな便利機能

多くのお年寄りは聴力に問題があるが、スマホが補聴器と連携できることを知らない。iPhone と一部の Android スマホは MFi（Made for iPhone）補聴器に対応し、通話音声を直接補聴器に送れる。スマホの受話器を耳に当てるよりずっとクリアだ。

- **接続方法：**iPhone の設定 > アクセシビリティ > 補聴器、Bluetooth をオンにし、補聴器のペアリング手順に従う。

- **補聴器を使わない場合：**”アクセシビリティ > オーディオ/視覚”で”通話ノイズキャンセル”と”モノラルオーディオ”をオンにすると、通話の明瞭度が改善する。

- **Android：**”アクセシビリティ > 聴力強化”に類似の設定がある。ブランドにより位置が少し異なるので”補聴”で検索する。

## 八、健康関連機能——スマホは通信ツールにとどまらない

- **SOS 緊急求助：**電源ボタンを 5 回連続押し（大部分のスマホ）、自動的に緊急電話を発信し、設定済みの緊急連絡先に位置情報を送信する。お年寄りのために事前に設定しておこう——iPhone：”設定 > 緊急 SOS”、Android：”設定 > セキュリティ > 緊急求助”。教える前に一度リハーサルし、本番で焦って押し間違えないようにする。
- **転倒検出：**Apple Watch にこの機能があり、転倒を検出してしばらく動かないと、自動的に救急電話を発信し家族に通知する。予算があればお年寄りに一台贈ろう。緊急の瞬間に命を救える。
- **服薬リマインダー：**iPhone の”ヘルスケア”App に服薬リマインダー機能が内蔵。薬名、時間、服用量を設定すれば、時間になるとリマインドが表示される。Android は”繰り返しアラーム”や”服薬リマインダー”系 App をダウンロードする。

**本章のまとめ**

- まず簡単モードをオンにするか手動で文字/アイコンを大きくし、環境を整えれば問題の大半は解決する
- お年寄りにスマホを教える三鉄則：急かさない、例えを使う、書き留める
- 六大よくある詐欺は繰り返し注意し、三重の防御ラインで 90% の詐欺を防げる
- リモートサポートで離れていても親のスマホを修理できる
- WeChat の”ケアモード”+”音声入力”がお年寄りに最も実用的な二つの機能
- SOS 緊急求助と転倒検出は、緊急の瞬間に命を救える設定。5 分かけて設定する価値がある

# 第二十八章：お金を賢く使ってお気に入りを買う——スマホ選びの究極ガイド

スマホ選びで、一週間レビューを見て一ヶ月価格比較しても、結局五六千元も使って自分には不要なフラッグシップ機を買ってしまう人が多い。この章はスペック表を並べない——それは公式サイトで見られる。私がしたいのは“選び方の思考法”を渡すことで、今後どんな新機種が出ても、自分に合うかどうかを自分で判断できるようになる。

## 三段階選び方：ニーズ予算機種

ネットで“3000 元でどんなスマホがいいか”と聞く人が多いが、この質問には答えられない——スマホで主に何をしたいか分からないからだ。正しい思考順序は：

**選び方の三手**

1. **まず何を最も重視するかを明確にする：**写真？ゲーム？バッテリー？画面？システムの快適さ？それとも電話と WeChat ができればいい？ニーズに優先順位を付ける。
2. **次に予算上限を決める：**”だいたいいくら”ではなく”最高いくらまで出せるか”。厳しい上限を決めれば、選ぶ時にどんどん高くなるのを防げる。
3. **最後に機種を選ぶ：**ニーズの優先順位と予算を持って、機種をマッチングする。

## 一、コアパラメータの見方——全部を理解する必要はない、この五項目が分かれば十分

スペック表には何十項目ものデータがあるが、一般ユーザーが注目すべきはこの五つの次元だけ：

| 比較項目 | 見方 | 一言まとめ |
| --- | --- | --- |
| プロセッサ（SoC） | スマホが何年スムーズに使えるかを決める。2026 年の基準：Snapdragon 8 Elite Gen5/Dimensity 9500 以上 = トップクラス；Snapdragon 8s Gen4/Dimensity 8400= 中高級；Snapdragon 7+ Gen4/Dimensity 7350=3 年使える。Snapdragon 6 系以下は要注意。 | 新しいものを買え、SoC がスマホが何年卡なくなく使えるかを決める |
| 画面 | OLED は LCD より色彩が良く省電力;リフレッシュレート 120Hz は 60Hz よりスクロールがずっと滑らか;解像度は最低 1080P 以上。高周波 PWM 調光（1920Hz 以上）は目に優しい。 | OLED+120Hz+ 高周波調光 = 良い画面の合格ライン |
| : 家に老人、子供、ペットがいれば、スマホでいつでも家庭の状況を確認。 | **画素数ではなくセンサーサイズを見る**。センサーが大きいほど圧倒的に有利で、画素数よりはるかに重要。超広角は実用的；望遠（3x 以上の光学ズーム）は遠くのものを撮る;マクロレンズは大抵おまけ。 | 大センサーメインカメラ + 実用的な望遠 > おまけレンズ複数 |
| バッテリーと充電 | 4500mAh は最低合格ライン、5000mAh 以上なら一日一充で安定。急速充電はワット数を見る：60W 以上で速い、100W 以上で超速い。ワイヤレス充電は便利だが遅い。 | 5000mAh+60W 急速充電 = 不安なしの組み合わせ |
| システム更新の約束 | スマホがどれだけ安全に使えるかを決める。Apple は通常 5-6 年のシステム更新;Samsung フラッグシップは 4 年のシステム +5 年のセキュリティ更新；Google Pixel は 7 年;国産ミッドレンジは更新期間が総じて短い（2-3 年）。 | 長く使いたいなら、システム更新の約束が長いブランドを選ぶ |

## 二、予算別のおすすめ（2026 年中期市場を基準）

以下のおすすめは 2026 年中国本土市場の状況に基づく。価格は公式発表の発売価格。実際の価格は変動するので、購入時の実価格を基準に。

### 1000 元以下：電話と WeChat ができれば OK

**対象者：**学生、サブ機、スマホにこだわらない人、主に電話と WeChat 用途。

**買えるもの：**Redmi Note シリーズ、Honor X シリーズ、realme 数字シリーズの中低構成。プロセッサは Snapdragon 6 系または Dimensity 6000 シリーズ、画面は LCD 中心、写真はクリアに撮れるが AF が遅い、システムの快適さは普通。

**選び方のアドバイス：**この価格帯では二点を見る：バッテリーが十分大きいか（5000mAh 以上を選ぶ）、メモリが十分か（6GB+128GB 以上、4GB は卡なるので避ける）。Redmi がこの価格帯で最も優位。

### 1000-2000 元：コスパ最高、3-4 年卡なく使えるスイートスポット

**対象者：**学生、予算が限られているが長く使いたい人、写真やゲームに極限のこだわりがない人。

**代表的な機種：**Redmi Turbo シリーズ、iQOO Z シリーズ、Honor X シリーズの高構成、OnePlus Ace シリーズのエントリー。

**選び方のアドバイス：**Snapdragon 7+ Gen3 または Dimensity 7300 以上のプロセッサを搭載し、8GB+256GB メモリの機種を選ぶ。この構成で日常使用 + たまに King of Glory をするには十分で、通常 3-4 年は卡なく使える。

### 2000-3500 元：中堅層、性能とカメラが明確に向上

**対象者：**良い写真、卡ないゲーム、長く使いたい一般ユーザー。最も人が買う価格帯。

**代表的な機種：**Xiaomi 数字シリーズ（Xiaomi 17 標準版の値下がり後など）、vivo S シリーズ高構成、OPPO Reno シリーズ、Honor 数字シリーズ、Redmi K シリーズ。

**選び方のアドバイス：**この価格帯の最大の特徴は“バランス機”——明確な弱点がない。2000 元以下と比べ、主な向上点は：カメラが明確に良くなる（手ブレ補正あり）、高品質 OLED 画面、急速充電が速い（80W 以上が一般的）、作りもより上質。12GB+256GB または 12GB+512GB を選べば、四五年は問題ない。

### 3500-5000 元：準フラッグシップ、ガジェット好き以外には完全に十分

**対象者：**カメラにこだわるユーザー、最高級の質感を味わいたいユーザー。

**代表的な機種：**Xiaomi 数字シリーズ標準版、vivo X シリーズ、OPPO Find X シリーズ標準版、iPhone 旧フラッグシップ（iPhone 16 など）。

**選び方のアドバイス：**この価格帯では昨年のフラッグシップ機が買える。私のおすすめは：**昨年のフラッグシップを買え、今年のミッドレンジは買うな**。昨年のフラッグシップの素材、作り、カメラハードウェアは通常、今年のミッドレンジより一ランク上。“最新型”でないことを受け入れられる前提で。

### 5000 元以上：フラッグシップ域、お金に余裕があるかある体験に極限のこだわりがある人

**対象者：**写真愛好家、ヘビーゲーマー、最高級画面と做工を求めるユーザー、スマホで動画撮影してコンテンツを作るクリエイター。

**代表的な機種：**iPhone 17 Pro/Pro Max、Samsung S26 Ultra、Huawei Pura 80 Ultra、Xiaomi 17 Ultra、vivo X300 Pro。

**選び方のアドバイス：**この価格帯では各社の差がますます縮まっており、実はどのシステムが好きかによる。写真なら vivo か Huawei、動画なら iPhone、画面と S Pen なら Samsung、バランスなら Xiaomi。正直この価格帯になると、**好みで、気に入った一台を選べばいい**。

## 三、シーン別選び方——あなたのニーズは？

以下は最もよくある“利用シナリオ”、当てはまるものを選べばいい：

| これこそが真の「テクノロジーが生活をより平等にする」 | 注目ポイント | おすすめの方向 |
|---|---|---|
| 写真愛好家 | 大センサーメインカメラ（1インチがベスト）、光学手ブレ補正、望遠（3x 以上光学ズーム）、画像処理チップ | vivo X シリーズ、Huawei Pura/Mate シリーズ、Xiaomi Ultra シリーズ |
| ゲーマー | トップクラスのプロセッサ、高いタッチサンプリングレート、冷却システム、大バッテリー + 急速充電 | iQOO 数字シリーズ、ROG Phone、Red Magic、Redmi K シリーズ |
| バッテリー優先 | 5500mAh 以上のバッテリー、効率の良いプロセッサ、LTPO 画面 | Honor Magic シリーズ、OnePlus、Redmi Note 大バッテリー版 |
| お年寄り向け | 簡単モード、大きな文字、大きな音量、大バッテリー、アフターサービス拠点が多い | Huawei Enjoy、Honor Play、OPPO A シリーズ、Redmi Note |
| 学生向け | コスパ高い、頑丈、大バッテリー、ペアレンタルコントロール機能あり | Redmi Note/Turbo、realme GT、iQOO Neo |
| Apple エコシステムユーザー | すでに Mac/iPad/AirPods を持っており、iPhone を選ぶのが唯一の解 | iPhone 17 標準版（コスパ）または Pro 版（フルスペック） |
| 海外旅行 | eSIM 対応、全世界周波数バンド対応、Google サービス | iPhone、Samsung S シリーズ、Pixel、OnePlus |

## 四、陥りやすい五つの罠——購入前にチェック

1. **罠一：レンズ 4 個 5 個 = 写真が良いわけではない**——メーカーはスペックにレンズ数を積みたがるが、その中に 200 万画素の” おまけレンズ”（マクロ + 被写界深度）が二つあることが多く、実質的に無意味。スマホの写真能力を見る時は、**メイン + 超広角 + 望遠だけを見る**この三つだけで十分。残りは存在しないものとしてよい。

2. **罠二：1 億画素に騙されるな**——画素数が高い = 画質が良いではない。センサーサイズ、絞り、アルゴリズム調整の方が画素数よりはるかに重要。多くのフラッグシップ機は 5000 万画素の大センサーを使い、1 億画素の千元機よりずっと良い写真を撮る。

3. **罠三：キャリアロック機を買うな**——” 通信費積立でスマホプレゼント” はお得に見えるが、キャリアカスタム機はシステム更新が遅く、削除できないプリインストール App がある可能性があり、通常キャリアロックされている（そのキャリアの SIM しか使えない）。お金を節約したように見えて、実は” ロック” されている。

4. **罠四：” 曲面ディスプレイは高級”——マーケティングであり真理ではない**——曲面デ

ィスプレイはかつてフラッグシップの象徴だったが、今ではその問題に気づく人が増えている：保護フィルムが貼りにくい、縁での誤タッチ、割れると修理が高い。2026 年のトレンドは”微曲面”さらには”完全フラット”への回帰。”曲面 = 高級”という理由で選ぶべきではない。

5. **罠五：すべての”AI スマホ”が本当に有用なわけではない**——2026 年すべてのメーカーが”AI スマホ”を謳うが、多くは既存の音声アシスタントの名前を変えただけ。本当に有用な AI 機能は：リアルタイム翻訳、AI 消去（写真から人を消す）、AI 写真編集、オンデバイス大モデル推論、ローカライズされたスマート要約。その他の派手な機能は基本的にギミック。

## 五、いつ買うのが一番お得？

- **618（5 月末から 6 月中旬）：**一年でスマホの値下がり幅が最も大きい時期。多くのメーカーが販売台数を伸ばすため、フラッグシップ機が 800-1500 元値下がりする。上半期にスマホを買うベストタイミング。

- **ダブル 11（10 月末から 11 月中旬）：**下半期の大型セール、割引幅は 618 とほぼ同じ。この時期は多くのブランドの新型フラッグシップが発表されたばかりで、旧型フラッグシップの値下がり幅が最大。

- **新機種発表から一ヶ月後：**発表直後は定価、一ヶ月後には流通価格（第三者店舗）が通常 200-500 元安くなる。

- **春節前二週間から元宵節：**実店舗が在庫処分をする”年越しセール”、店員と値段交渉できる。

- **おすすめしない時期：**新製品発表直後に初回購入——最初のユーザーになりたい人以外は、発売直後価格が最も高い。

## 六、購入チャネルの選び方——どこで買えば安心？

- **ブランド公式モール/Tmall 旗艦店/JD 自営：**最も安心、価格面では不利だがリファービッシュや偽物の心配なし、アフターサービスが最も良い。

- **Pinduoduo”百億補助”：**最低価格、ただし”百億補助”と表示された店舗を選ぶ必要がある。表示のない店は避ける。商品受け取り時に開封動画を撮り、問題があれば返品できる。

- **実店舗：**手に取って触れる、使い心地を試せる。注意：店員は利益率の高い機種（あなたに最も適した機種ではなく）を勧めるので、来店前にネットで 2-3 機種に絞り、店に行ったら”この機種だけ見たい”と言うのがよい。

- **中古プラットフォーム（Xianyu/Zhuanzhuan）**：安いがリスクあり。中古を買うなら三つを守る：購入時の領収書または注文スクリーンショットの提供を要求、IMEI 番号の確認を要求、**プラットフォーム取引のみ、絶対に私的に送金しない**。

**実践：この方法でスマホを一台選ぶ**

1. 紙を一枚取り、二つのコアニーズを書く（例：バッテリー第一、写真第二）。
2. 予算上限を決める（例：3500 元）。
3. JD か Tmall を開き、価格で絞り込み、3-5 機種を候補に挙げる。
4. Bilibili で”XX 機種一ヶ月使用体験” を検索（初日のレビューではなく、最低一ヶ月使用後のリアルな感想）、二三本の動画を見る。
5. チャネルの信頼性を確認してから注文、受け取り時に開封動画を撮る。

**本章のまとめ**

- 選び方の三手：ニーズ予算機種
- コアパラメータは五項目のみ：プロセッサ、画面、カメラ、バッテリー充電、システム更新の約束
- 1500-3500 元はコスパ最高、最もおすすめのスイートスポット
- 昨年のフラッグシップを買う方が今年のミッドレンジよりお得
- 五大罠：多レンズの罠、高画素の罠、キャリアロック、曲面信仰、AI ギミック
- 618 とダブル 11 は一年で最もスマホを買いやすい二つのタイミング

# 第二十九章：デジタルヘルス——スマホとの健康的な関係づくり

こんな経験はないか：5 分だけ短動画を見るつもりが、気がつくと 2 時間経っていた？夜ベッドでスマホを見ていたら、見れば見るほど目が冴えて、結局深夜 2 時に寝た？スマホは私たちの生活を変えたが、静かに私たちの時間と健康をむさぼり食っている。この章はスマホを“やめる”ことが目的ではなく、スマホに対する主導権を取り戻すことが目的だ。

## 一、まず“スクリーンタイム”を見よう——数字が目を覚まさせる

変える前に、まず現状を把握しよう。スマホには多くの人が一度も開いたことのない機能がある。“スクリーンタイム”（iPhone）または“デジタルウェルビーイング”（Android）と呼ばれ、毎日スマホにどれだけ時間を使ったか、その時間がどの App に使われたかを教えてくれる。

- **iPhone：** 設定 > スクリーンタイム > すべてのアクティビティを表示。毎日のロック解除回数、スマホを手に取った回数、各 App の使用時間が分かる。
- **Android：**設定 > デジタルウェルビーイングとペアレンタルコントロール。Huawei は”健康的なスマホ使用”、Xiaomi は” スクリーンタイム管理” と呼ぶ。機能は類似し、毎日の使用データが見られる。

**初めてこのデータを見る人の 80% は驚く**。ある友人はデータを見た後、毎日 TikTok に 4 時間以上使っていることに気づき、一週間で 28 時間——アルバイト一台分に相当する。この“数字の衝撃”が変化の第一歩だ。

## 二、App に“使用制限”を設定する——自分にスピードリミッターを付ける

問題が分かったら、ターゲットを絞って制限を設定できる。App を一括削除する必要はなく、“一日の枠”を設定するだけでいい：

1. **iPhone：**設定 > スクリーンタイム > App 制限 > 制限を追加。制限したい App（例：TikTok）を選び、毎日 30 分に設定。時間が来ると App アイコンがグレーアウトし、開くには追加でパスワード入力が必要。
2. **Android：**設定 > デジタルウェルビーイング > App タイマー。App を選んで毎日の使用上限を設定、時間が来ると App アイコンがグレーアウト。
3. **テクニック：**制限のパスワードを普段使わない覚えにくいものにする。簡単に入力できると毎回”延長”してしまい、制限が形骸化する。

## 三、“おやすみモード”の正しい使い方——通知を消すのではなく不安を消す

多くの人のスマホは一日に何十回も通知で鳴り、鳴るたびに注意力が中断される。“ちょっと通知を見るだけ”だと思っていても、実際は中断後に注意力を取り戻すのに 15-20 分かかる。おやすみモードは“連絡絶ち”ではなく、注意力を取り戻すためのものだ。

- **時間指定でオン：**おやすみモードを夜 10 時から朝 7 時まで自動オンに設定。これで睡眠中に迷惑 SMS で起こされることがなくなる。
- **重要な連絡先を通過させる：**おやすみモードの設定で、”着信を許可”を”お気に入りの連絡先”または”繰り返し着信”（3 分以内の 2 回目の着信は通常通り鳴る）に設定。これで家族の緊急電話を見逃さず、営業電話にも邪魔されない。
- **仕事集中モード：**iPhone の”集中モード”はカスタムシーンを設定できる——例えば”仕事”モードでは仕事関連 App の通知のみ許可、”読書”モードではすべての通知をミュート。

## 四、目の保護は App ではなく習慣——20-20-20 ルール

長時間スマホ画面を見ると、目の乾きや疲れは避けられない。アイケアモード（ブルーライトカット）はある程度役立つが、本当に効果的なのは目の使い方を変えることだ：

**20-20-20 ルール**

画面を見るごとに **20 分**、視線を **20 フィート（約 6 メートル）**以遠に移し、**20 秒**維持する。この簡単な動作が毛様体筋の疲労を効果的に和らげる。

- **アイケアモードをオンにする：**設定 > 表示 > アイケアモード（または Night Shift）、オンにすると画面の色温度が暖色になり、ブルーライトが減少。日没から日の出まで自動オンに設定することを推奨。
- **画面の明るさを調整する：**周囲が暗い時は画面も暗くする。多くの人が暗闇でスマホを見る時明るさを上げすぎるが、これが最も目を痛める行為だ。”自動明るさ”をオンにし、正常に動作していることを確認。

- **距離を保つ：**スマホと目の距離は最低 30 センチ。多くの人がスマホをとても近くに持って見るが、こうすると毛様体筋が常に緊張収縮状態になり、近視の進行が早い。

## 五、“スマホ首”の頸椎自救ガイド

下を向いてスマホを見る時、首が受ける圧力は通常立っている時の 5-6 倍だ。長期間下を向くと頸椎のカーブがなくなり、骨棘形成、めまい、手のしびれを引き起こす。以下は簡単な改善方法：

- **スマホを高く上げる：**スマホを見る時はできるだけ目の高さまで上げる。膝の上に置いたスマホを下を向いて見ない。腕は疲れるが、首は感謝する。
- **30 分ごとに首を動かす：**ゆっくり左に首を回して限界まで、3 秒キープ、正面に戻す；次に右に回し、3 秒キープ。5 回繰り返す。次に” 米字操”——顎で空中に” 米” の字を書く。
- **壁に寄りかかって 3 分立つ：**後頭部、肩甲骨、臀部、かかとの四点を壁に付け、毎日 3 分立つ。猫背・下向きの姿勢を効果的に改善できる。

## 六、睡眠とスマホ——寝室で最も変えるべき習慣

スマホ画面のブルーライトはメラトニン分泌を抑制し、“見れば見るほど目が冴える”状態にする。多くの人は“見ているうちに眠くなった”と思うが、実際はブルーライトで脳が騙されており、スマホを置いた後かえって眠りにくい。以下は改善のアドバイス：

- **寝る 1 時間前はスマホを見ない：**最も効果的だが最も難しい一つ。できない場合は、少なくとも”Night Shift/アイケアモード” をオンにしてブルーライトを最低限に。
- **スマホを手の届かない場所に置く：**充電器を寝室の反対側に置く。ベッドサイドではなく。これで思わず手に取って見てしまうのを防ぎ、アラームが鳴っても起きて止めなければならない——一石二鳥。
- **” 睡眠モード” をオンにする：**iPhone の” 睡眠” 集中モードは睡眠時間帯に自動で画面をロックし、ロック画面をシンプルにする；Android 各ブランドにも類似機能あり。
- **スマホの代わりに目覚まし時計を使う：**十数元の物理的目覚まし時計を買い、スマホをアラームにしない。” 時間を見るついでに思わずスクロール” の罠がなくなることに気づく。

## 七、情報不安症——“選択的盲目”を身につける

私たちが毎日受け取る情報量は、昔の人が一生で受け取る量だ。しかし人間の脳はこれほど多くの情報を処理するようにはできておらず、情報過多は不安、注意力散漫、意思決定

疲労を引き起こす。解決策は“より多くの情報を得る”ことではなく“能動的に遮断する”ことだ：

- **フォローを解除する：**フォローしているすべての公式アカウント、UP 主、ブロガーのリストを開き、自問する：”過去一ヶ月で実用的な価値を得たか？”なければフォロー解除。情報チャネルは多さより質。
- **不要な App の通知をオフにする：**WeChat、電話、SMS、カレンダー以外の App 通知をすべてオフに。ニュースは通知不要——自分から見に行けば自然に目に入る。
- **”情報断食日”を設ける：**毎週一日（例：日曜）を選び、終日スマホでニュースを見ず、短動画をスクロールしない。空が崩れ落ちないこと、そして月曜の精神的状態が明らかに良くなることに気づく。

**実践：一週間デジタルヘルスチャレンジ**

1. 今日：スクリーンタイム/デジタルウェルビーイングを開き、先週の使用データを見る。
2. 明日：最も時間を費やしたエンタメ App に毎日の制限を設定（30-60 分を推奨）。
3. 明後日：おやすみモードを時間指定オン（22:00-07:00）に設定、お気に入り連絡先の着信を許可。
4. 四日目：寝る 1 時間前はスマホを見ない、充電器を寝室の反対側に置く。
5. 五日目：もう見ていない 5 つの公式アカウントやブロガーのフォローを解除。
6. 六日目：丸一日”情報断食”に挑戦——ニュースを見ず短動画をスクロールしない。
7. 七日目：今週のスクリーンタイムレポートを見て、先週と比較する。

**本章のまとめ**

- まずスクリーンタイムデータを見て現状を認識することが変化の第一歩
- エンタメ App に制限を設定し、おやすみモードを時間指定でオンにして注意力を取り戻す
- 20-20-20 ルールは最もシンプルで効果的なアイケア習慣
- 下を向いてスマホを見る時の首への負担は 5-6 倍、スマホを高く上げる + 定期的に首を動かす
- 寝る 1 時間前はスマホを見ない、充電器を手の届かない場所に置く
- 情報不安の特効薬は”能動的に遮断”することであり、”より多く得る”ことではない

# 第三十章：スクリーンミラーリング、プレゼンとカーナビ連携

スマホ画面は大きくても 7 インチだが、テレビは 55 インチ、プロジェクターは 100 インチだ。スマホ画面を大画面に“投げる”と、体験は全く次元が異なる。この章ではスクリーンキャスト、プレゼン、カースル連携の三つの高頻度シーンを明確にし、スマホの能力をより大きな画面に拡張する方法を説明する。

## 一、ワイヤレスキャストの三大陣営——混同しないように

ワイヤレスキャストはどれも“スマホ画面をテレビに”のように見えるが、基盤の技術は三種類あり、互いに互換性がない：

| 技術 | 使用者 | 特徴 |
| --- | --- | --- |
| AirPlay | iPhone/iPad/Mac | Apple エコシステム、体験最も滑らか。Apple TV または AirPlay 対応テレビが必要 |
| Miracast | Android スマホ | Wi-Fi Direct 直接接続、ルーター不要。Android スマホの”ワイヤレスキャスト”はデフォルトでこれを使用 |
| DLNA | 全プラットフォーム | メディアファイル（動画/音楽/画像）のみプッシュ、画面ミラーリングなし。テレビに動画を送って再生するのに適している |

### 各プラットフォームの操作方法：

- **iPhone：** コントロールセンター > 画面ミラーリング > テレビ/Apple TV を選択。テレビが AirPlay に対応していない場合、Apple TV ボックスまたは AirPlay 対応のテレビスティックを購入できる。

- **Android：**通知パネルをプルダウン ＞ ”ワイヤレスキャスト”/” スマートキャスト”/” マルチスクリーンインタラクション” ボタンを見つける ＞ テレビを選択。スマホとテレビが同じ Wi-Fi に接続している必要がある。

- **汎用ソリューション：**テレビが古すぎてワイヤレスキャストに対応しない場合、Chromecast や Xiaomi ボックス/Huawei ボックスを買ってテレビの HDMI 端子に挿せばキャスト受信ができる。

## 二、有線キャスト——より安定、ゼロ遅延

ワイヤレスキャストは便利だが遅延があり、ゲームやプレゼンでは体験が良くない。有線キャストはこの問題を解決できる：

- **iPhone：**Lightning/USB-C to HDMI 変換ケーブルが必要（約 100-200 元）。接続するとスマホ画面がテレビ/プロジェクターの HDMI 端子に直接出力される。

- **Android：**一部のスマホは USB-C to HDMI 直接接続に対応（スマホが DP Alt Mode をサポートする必要あり、大部分のフラッグシップが対応）。USB-C to HDMI ケーブルを一本買えばよい。

- **会議室シーン：**頻繁にスマホでプレゼンする場合は、バッグに HDMI 変換ケーブルを常備することを推奨。会議室のテレビ/プロジェクターはほぼ HDMI 端子があり、挿せば使える。ワイヤレスキャストをあれこれするよりはるかに頼りになる。

## 三、スマホがパソコンに——デスクトップモード

一部の Android スマホは“デスクトップモード”に対応——モニターに接続すると、パソコンのデスクトップに似た UI を出力し、マウスとキーボードで操作できる：

- **Samsung DeX：**Samsung フラッグシップの看板機能。モニター接続後にデスクトップ UI が起動し、マルチウィンドウ操作が可能。本当のパソコンに最も近い体験。Bluetooth キーボード/マウスと組み合わせれば、出張時にノート PC を持たずに済む。

- **Huawei デスクトップモード：**Huawei Mate シリーズ/Pura シリーズがモニター接続後に類似機能あり。マルチウィンドウ、ファイル管理、オフィス App に対応。

- **Motorola Ready For：**Moto フラッグシップも対応、機能は DeX に類似。

- **：ハイエンドプロフェッショナル制作。iPhone 15 Pro シリーズは Apple ProRes 動画撮影に対応（最大 4K 60fps）しますが、ProRes RAW 形式は現状外部録画デバイスとの組み合わせが必要で、スマホ単体での直接撮影は未対応。：**ライトなオフィス作業（メール返信、文書編集）、PPT プレゼン、大画面で動画視聴。ヘビーな Excel 操作や専門的なデザインには不適。

## 四、カースル連携——運転中のスマホの正しい使い方

運転中のスマホ操作は違法だが、ナビ、音楽、電話は必須だ。カースル連携システムで車載画面上でスマホのコア機能を使い、スマホは置いて触らないでおける：

- **Apple CarPlay：** iPhone ユーザーの第一選択。USB またはワイヤレスでカースルに接続、車載画面に iPhone のシンプル UI が表示される——ナビ（Amap/Baidu Map）、音楽、電話、メッセージ、ポッドキャスト。Siri 音声操作で完全ハンズフリー。
- **Android Auto：** Android ユーザー向けの対応策。機能は CarPlay に類似、Google Maps、Spotify などに対応。国内は Google サービス制限により一部機能使用不可。
- **Huawei HiCar：** Huawei 自社開発方案、国内車種のサポート率が高い。ナビ・音楽のほか、Huawei ビデオ通話、駐車映像統合などの独自機能も使える。
- **Baidu CarLife：** クロスプラットフォーム方案、iOS と Android の両方で使用可能。機能は少ないが互換性が広く、多くの旧型車種は CarLife のみ対応。

**セキュリティに関する注意**

どのカースル連携方案でも、運転中はできるだけ音声操作を使うべきだ。ナビは事前に目的地を設定し、音楽は事前にプレイリストを選んでおき、走行中に車載画面を操作しない。

## 五、会議室キャスト実戦——もうバタバタしない

会議はキャストの最も高頻度なシーンであり、最も問題が起きやすいシーンでもある。以下は実戦の経験則：

- **事前到着してテスト：** 会議開始直前にキャスト接続を試すのは絶対に避ける。10 分前に到着し、接続方法をテストする。Wi-Fi キャストの失敗確率は想像より高い。
- **有線方案を Plan B として準備：** バッグに HDMI 変換ケーブルを入れておく。ワイヤレスキャストが繋がらない時に有線が命綱になる。
- **通知プレビューをオフにする：** キャスト中、WeChat メッセージが会議室の大画面に表示される。キャスト前に設定で”ロック画面通知のプレビュー”をオフにするか、”プレゼンモード”をオンにする（一部スマホ対応、キャスト時に通知を自動非表示）。
- **”プレゼン文書”を使う而非”スマホ操作”：** PPT を展示する場合、PPT をスマホに保存して WPS で全画面再生する方が、スマホ画面全体をキャストするよりよい。よりプロフェッショナルでスムーズ。

**実践：スマホからテレビへのキャストを一度完了する**

1. テレビで「画面ミラーリング」または「AirPlay」機能を見つけてオン。
2. スマホで写真または動画を一枚開く。
3. ワイヤレスキャストを試す（iPhone は画面ミラーリング、Android はワイヤレスキャストボタン）。
4. 失敗した場合、テレビが対応プロトコルをサポートしているか確認、または DLNA プッシュを試す（アルバム/動画 App で”キャスト”ボタンをタップ）。
5. キャスト成功後、スマホで他の操作をして、大画面が同期表示されるか確認する。

**本章のまとめ**

- ワイヤレスキャストの三大陣営：AirPlay（Apple）、Miracast（Android）、DLNA（クロスプラットフォーム・メディアプッシュ）
- 有線キャストはより安定・ゼロ遅延、会議室シーンに HDMI 変換ケーブルが必須
- Samsung DeX/Huawei デスクトップモードでスマホがライトオフィス PC に変身
- カースル連携：CarPlay/Android Auto/HiCar/CarLife の四大方案
- 会議室キャストの鉄則：事前テスト、Plan B 準備、通知プレビューオフ

# 第三十一章：ショート動画撮影とコンテンツ制作の入門

*2026* 年、短動画はもう“若者の玩具”ではなく、すべての人が情報を得て自己表現する基礎手段になった。生活を記録しようが、知識を共有しようが、副業をしようが、スマホが最高の創作ツールだ——カメラを買う必要も *Premiere* を学ぶ必要もない。一台のスマホで始められる。この章はゼロからのスタートを助ける。

## 一、撮る前に明確に：誰に見せるのか？

多くの人が動画撮影で最初に犯す間違いは“スマホを手に取ってすぐ撮る”ことで、撮り終わっても誰に見せるか分からない。撮る前にまず三つの質問に答えよう：

1. **視聴者は誰か？**——子供向け、同年代向け、お年寄り向けでは、内容のスタイルが全く異なる。

2. **どんな価値を提供できるか？**——笑わせる（エンタメ）、学ばせる（知識）、食欲をそそる（グルメ）、温める（感情）。少なくとも一つ。

3. **どのプラットフォームで継続更新できるか？**——Douyin/Kuaishou（総合エンタメ）、REDnote（ライフスタイル）、Bilibili（深い内容）、動画アカウント（ソーシャル圏）。一つのメインプラットフォームを選んで深耕し、最初から全プラットフォーム配信はしない。

## 二、スマホ撮影の基本設定——撮影前に必須の 5 件事

1. **解像度は 1080P、フレームレートは 30fps に設定：**これは短動画プラットフォームの標準だ。4K はファイルが大きすぎてアップロードが遅く、60fps は SNS ではメリットがなくファイルがより大きい。

2. **レンズをきれいに拭く：**最初で最も重要。撮る前に服でレンズを拭くだけで、画質が一ランク上がる。

3. **フォーカスと露出をロック：**ファインダー内で被写体を 2 秒長押しすると”AE/AF ロック”の表示が出る。これで撮影中に移動しても画面が明るすぎたり暗すぎたりしない。

4. **横撮りか縦撮りか？**——Douyin/Kuaishou/動画アカウント/REDnote に投稿**縦撮り**（9:16）、Bilibili/YouTube に投稿**横撮り**（16:9）。プラットフォームを考えてから方向を決める。

5. **グリッド線をオンにする：**設定 > カメラ > グリッド。三分割法の構図の基礎で、画面の重心を素早く見つけるのに役立つ。

## 三、カメラワーク五手——画面を“動かす”

初心者が動画撮影で最大の問題は画面が“静”すぎること——ずっと立ったまま動かずに撮る。以下の五つの基本カメラワークを覚えれば、画面にすぐプロ感が出る：

- **プッシュ（Push）：**スマホをゆっくり被写体に近づける。”近づく”緊張感や親密感を生む。
- **プル（Pull）：**スマホをゆっくり被写体から遠ざける。ディテールから全景に引き、環境を示す。
- **パン（Pan）：**スマホの位置はそのままで左右に回転。風景や空間の展示に適している。
- **トラック（Track）：**スマホを平行移動させ、被写体についていく。手ぶれ補正が必要で、ジンバルを使うのがベスト。
- **フォロー（Follow）：**スマホが被写体の後ろまたは前について歩く。Vlog で最もよく使うカメラワーク。

**ブレないことが第一：**手ブレした映像は素人っぽく見える。両手持ち片手持ちより安定；歩く時は膝を少し曲げ“忍び足”で上下の揺れを減らす；予算があればスマホジンバル（DJI OM シリーズ約 500-800 元）を買うと、効果が劇的に変わる。

## 四、編集入門——CapCut（剪映）で十分

剪映（CapCut の中国版）は現在初心者に最も適したスマホ編集ツールで、無料、機能充実、テンプレート豊富。以下は基本ワークフロー：

1. **素材をインポート：**剪映を開く > 作成開始 > 撮影済み動画クリップを選択してインポート。

2. **カット編集：**タイムラインをドラッグし、各クリップの冒頭と末尾の不要部分を切り取る。原則：各セグメントは情報量のある部分だけ残し、長さより短さを優先。

3. **トランジションを追加：**二つの動画クリップの間で”トランジション”ボタンをタップし、シンプルな効果を選ぶ。初心者は”ディゾルブ”か”フェード”を推奨、派手な 3D トランジションは避ける。

4. **字幕を追加：**剪映には”字幕認識”機能があり、ワンタップで自動生成。精度は約 90%、手動で一度校正すればよい。

5. **BGM を付ける：**剪映の音楽ライブラリに著作権フリーの音楽が大量にある。内容に合うリズムの BGM を選び、音量を動画元音声の 1/3 程度に調整。

6. **エクスポート：**1080P でエクスポート、フレームレート 30fps、ビットレートは”推奨”を選べばよい。

## 五、サムネイルの重要性——70% の人がサムネイルだけで見るか決める

短動画のサムネイルは“看板”で、タップするかどうかを決める。サムネイル作成のポイント：

- **サムネイルに一言入れる：**大きな文字で動画のコア内容を書く。例えば”3 分でスマホ写真編集をマスター”。文字は大きく、色は明るく、背景とのコントラストをつける。
- **サムネイルと内容を一致させる：**”タイトル詐欺”をしない——サムネイルで一つのことを言っておきながら中身が違うと、視聴者はすぐスクロールし、アルゴリズムの評価が下がる。
- **人物の表情を生き生きと：**サムネイルに人がいる場合、生き生きとしたまたは誇張された表情の瞬間を選ぶ。人の顔は文字よりクリックを惹きつける。
- **剪映でサムネイルを作る：**剪映で最も良い一フレームを選び、テキストを重ねて、画像としてエクスポートしサムネイルにする。

## 六、コンテンツ創作の三つの心法

- **最初の 3 秒で生死が決まる：**短動画プラットフォームでは、視聴者は 3 秒以内にスクロールするか見続けるかを決める。最初の 3 秒で”フック”を出す——一つの問い、一つのサスペンス、一つの意外な映像。
- **一本の動画は一つのことだけ話す：**初心者が最も犯しやすい間違いは一本の動画に内容を詰め込みすぎること。一本の動画は一つの問題だけを解決し、網羅より深掘りが大事。
- **才能より継続が大事：**アルゴリズム推薦には”アカウント育成”が必要。最初の 20 本の動画はほぼ再生数がないが、これは正常だ。毎週 2-3 本更新し続け、アルゴリズムに内容の方向性を認識させれば、トラフィックは徐々に伸びる。

**実践：30 秒の短動画を一本撮る**

1. 得意な小ネタを一つ選ぶ（例：”スマホの文字を大きくする方法”）。
2. スマホ縦画面で 3-5 クリップ撮影、各 5-10 秒。
3. 剪映で結合、カット、字幕追加、BGM 付与。
4. 大きな文字入りサムネイル画像を作る。
5. エクスポート後、まず友人に見せてフィードバックを集めてから公開する。

**本章のまとめ**

- 撮る前に明確に：視聴者は誰か、どんな価値を提供するか、どのプラットフォームに投稿するか
- 撮影前の 5 件事：1080P/30fps、レンズ拭き、フォーカス露出ロック、横縦の確認、グリッドオン
- カメラワーク五手：プッシュ、プル、パン、トラック、フォロー——ブレないことが第一
- 編集は剪映：カットトランジション字幕 BGM エクスポート、六ステップ
- 最初の 3 秒で生死が決まる、一本の動画は一つのことだけ、才能より継続が大事

# 第三十二章：スマホ安全救急マニュアル

この章は「緊急時対応マニュアル」です。できれば一生使わないでほしいのですが、もしスマホの紛失・盗難・詐欺に遭ったとき、このマニュアルがあれば被害を最小限に抑えられます。地震や大雨などの災害が来たときも、スマホは警報の受信、居場所の伝達、連絡の維持に欠かせない道具です。今のうちに一度通読して、大事な手順をスクリーンショットで保存するか、印刷して家に置いておくのがおすすめです。何かあったときにはスマホがもう手元にないかもしれず、災害時には電池切れや圏外の可能性もあるからです。

## シーン 0：地震 / 極端な天候——スマホが命を守るしくみ

守るべきは命の安全であって、アカウントやお金ではありません。警報の設定方法は第十三章「天気警報・自然災害警報・地震警報」を参照。ここでは警報が鳴ったあとの行動だけを説明します。

**地震警報が鳴ったら：まず身を伏せてから避難。動画撮影に夢中にならない**

1. 近くで身を守る：まずしゃがみ、頭を覆い、しっかりつかまる（テーブルの下、耐力壁の隅、頑丈な家具のそば）。頭と首を保護し、ガラス・外壁・シャンデリアから離れます。
2. むやみに建物の外へ走り出さない：揺れの本体は多くの場合十数秒から数十秒で、階段のほうがかえって危険なことも。エレベーターは絶対に使わない。
3. 揺れが収まってから落ち着いて避難：階段を使い、余震と落下物に注意。開けた場所へ移動し、電柱・看板・倒壊しそうな建物から離れる。
4. スマホは本当に必要な場面で：緊急連絡先へ短いメッセージを送る（「私は　にいます。無事です / 負傷しています。助けが必要です」）ほうが、長電話より電池がもちます。送れる状況なら、まずはテキスト（SMS / WeChat）で。
5. 圏外のとき：第七章の衛星 SOS / 衛星メッセージ（対応機種の場合）を参照。または高い場所・開けた場所へ移動してから再試行。状況によっては、ネット不要の Bluetooth トランシーバーも役立ちます。
6. 不要な機能を切る：緊急時は機内モードを入れて定期的に解除、Bluetooth のスキャンをオフ、画面の明るさを下げる——だけで数時間は長持ちします。最後の電力は、安否連絡と救援情報のために取っておきましょう。

**大雨 / 台風 / 山あいの洪水：警報の段階で動き出す**

- 赤色警報を見たら：外出はできるだけ控え、地下空間・アンダーパス・河川・急な斜面から離れる。
- 屋外で取り残されたら：頑丈な建物の上の階へ移動し、無理に水の中へ入らない（冠水した場所にはマンホールの蓋がないところや、漏電の危険がある）。119 / 110 にかけるときは、目印と水位をはっきり伝える。
- 出かける前：天気アプリと地域の警報を流す公式アカウントで危険を確認。長距離ドライブでは、山沿い・水辺・崖沿いの区間を避ける。
- 家族での取り決め：「連絡が取れなくなったら、最後の WeChat / SMS の位置情報を正とする」と決めておく。全員が必死に発信し続けて電池を使い切るのを防げます。

**ふだんから済ませておく 4 つのこと**

1. システムの地震警報や正規の警報アプリを有効にし、マナーモードでも鳴るよう許可しておく。
2. 連絡先の先頭に「緊急連絡先」を 2 3 人置き、ロック画面に表示できるようにする（iOS / Android のメディカル ID やロック画面情報）。
3. 家とカバンにモバイルバッテリーを用意。長距離移動の前には満充電してケーブルも持参。
4. 連絡帳を見なくてもかけられる番号（家族の携帯番号など）を最低 1 つ覚え、紙に書いて財布に入れる。

## シーン 1：スマホを紛失した / 盗まれた——ゴールデン 30 分

スマホを紛失してからの最初の 30 分は「ゴールデンタイム」です。適切に対処すれば、お金とプライバシーを守れます。以下の順に、一歩も飛ばさず実行してください：

**紛失後 30 分の標準的な手順**

1. 人のスマホを借りて自分の番号に発信（最初の 1 分）：近くに落ちていて、親切な人が取ってくれれば取り返せるかもしれません。
2. スマホを遠隔ロック（2 5 分目）：人のスマホかパソコンから「探す」系サービスにログインします：iPhone：iCloud.com ＞探す＞自分のデバイスを選択＞紛失としてマーク
3. Android の各ブランド：HUAWEI は cloud.huawei.com / Xiaomi は i.mi.com / OPPO は cloud.oppo.com など
4. SIM カードの利用停止（5 10 分目）：キャリアのサポート窓口（移動 10086 / 聯通 10010 / 電信 10000）に電話し、スマホを紛失したと伝えて SIM カードのサービス停

止を依頼します。ここが最重要——詐欺師があなたの SIM カードを手にすると、認証コードを受け取られ、すべてのパスワードをリセットされてしまいます。

5. 決済アカウントの凍結（10　20 分目）：Alipay：95188 で利用停止、または人のスマホで Alipay にログイン＞マイ＞設定＞セキュリティセンター＞緊急凍結

6. WeChat：110 に通報したうえで WeChat サポートに連絡、または WeChat セキュリティセンター＞アカウントを凍結

7. 銀行アプリ：キャッシュカード裏面のサポート電話にかけ、ネットバンキングの凍結を依頼

8. 知人・家族への連絡（20　25 分目）：モーメンツで知らせるか、家族に拡散を頼み、スマホを紛失したことを伝えて、詐欺師があなたになりすまして他人を騙すのを防ぎます。

9. 警察への通報（25　30 分目）：110 に電話するか、最寄りの派出所で被害を届け出て、受理票を受け取ります。この後の保険請求や銀行とのやり取りでも、この受理票が必要になります。

## シーン 2：詐欺に騙された——72 時間の勝負期間

騙されたと気づいたら、いちばん大事なのは冷静さです。慌てて間違った判断をすると、二次被害につながります。以下の手順で進めてください：

1. まずすべての送金を即座に中止:詐欺師が何と言おうと、1 円も送ってはいけません。「もう一度保証金を送れば前のお金が戻る」は、典型的な二次詐欺の口上です。

2. 証拠をすべて残す：チャット履歴を消さない！通話履歴を消さない！チャットのスクリーンショット、送金記録、相手のアカウント情報を保存します。これらは通報と資金取り戻しのカギとなる証拠です。

3. すぐに 96110 へ電話：全国の詐欺被害相談専用ダイヤルです。オペレーターが緊急の支払い停止を案内してくれます——送金から間もない場合（通常 24 時間以内）なら、銀行が相手の口座を凍結して資金を止められる可能性があります。

4. 110 に通報：スマホ、身分証、送金の証憑を持って最寄りの派出所へ行き、事情聴取を受けます。

5. 銀行 / 決済サービスに連絡：被害状況を伝え、関与した口座の凍結を依頼します。クレジットカードを不正利用された場合は、チャージバック（支払い取消し）の手続きを求めましょう。

6. パスワードをすべて変更：別の安全なデバイスで、Apple ID / 各ブランドのアカウント、WeChat、Alipay、銀行アプリのパスワードを変更し、二段階認証を有効にします。

## シーン 3：スマホがウイルス / トロイの木馬に感染したら——どうする？

次のような症状が出たら——理由もなく広告がポップアップする、知らないアプリが勝手に入る、通話料が急増する、データ通信量が異常に減る——マルウェアに感染した可能性が高いです：

- ステップ 1：セーフモードに入る——Android は電源ボタンを長押しして電源を切り、もう一度長押しして起動。起動時のバイブレーションが来たら音量下ボタンを押し続け、セーフモードに入るまで待ちます（画面左下に「セーフモード」と表示）。セーフモードではサードパーティ製アプリがすべて無効になるので、原因の切り分けに便利です。
- ステップ 2：怪しいアプリを削除——セーフモードで［設定］>［アプリ管理］を開き、インストール日時順に並べて、最近入れた心当たりのないアプリを見つけて削除します。
- ステップ 3：セキュリティアプリでスキャン——「Tencent Mobile Manager」や「360 Mobile Guard」をダウンロードして、フルスキャンを実行します。
- ステップ 4：初期化（最終手段）——ここまでで解決しない場合は、大事なデータをバックアップしてから工場出荷状態に戻します。注意：初期化するとすべてのデータが消えるので、写真・連絡先などの重要なファイルは必ず先にバックアップしてください。
- iPhone は基本的にウイルスに感染しません——iOS のサンドボックス機構のおかげでマルウェアが生き残りにくいためです。iPhone に異常がある場合、ほとんどはアプリのバグか設定の問題で、ウイルスではありません。初期化すれば 99% の問題は解決します。

## シーン 4：アカウントを乗っ取られた——素早く主導権を取り戻す

1. すぐにパスワードを変更：まだログインできるなら、真っ先に変えます。新しいパスワードは旧パスワードとまったく別のものに。
2. ほかのデバイスを締め出す：アカウントのセキュリティ設定で「ログイン端末の管理」を探し、知らないデバイスをすべて強制ログアウトさせます。
3. ログイン履歴を確認：直近 7 日間のログイン記録を見て、知らない場所からのログインがあれば、アカウントは間違いなく盗まれています。
4. 二段階認証を有効にする：まだオンにしていないなら、今すぐオンに。アカウントの再被害を防ぐいちばん効果的な方法です。
5. 連携情報を確認：アカウントに紐づく電話番号やメールアドレスが勝手に変わっていないか確認します。変わっていたら、アカウントの復旧申し立て機能で取り戻しましょう。

6. 決済設定を確認：被害が WeChat / Alipay なら、パスワードなし決済や自動引き落としの設定が変更されていないか、銀行カードが見知らぬアカウントに追加されていないかを確認します。

## シーン 5：脅迫メッセージが届いた——怖がらない、払わない

よくある脅迫の手口は 2 つです。(1)「あなたのわいせつ動画 / 写真を手に入れた。金を払わなければ連絡先全員に送る」。(2)「あなたのスマホはハッカーに支配された。金を払わなければロックする」。こうした状況に遭ったら：

- 99.9% は詐欺です：相手の手元にあなたのプライバシーデータは何もありません。同じ脅迫文を何万人かに一斉送信して、怖がって払う人がいればもうけ、という仕組みです。
- 返信しない、お金を払わない：返信は「この番号は生きている」と相手に教えるのと同じで、「脅せそうなターゲット」の印をつけられたも同然。その後の嫌がらせが増えます。
- スクリーンショットで証拠を保存：そのうえで相手をブロックします。
- パスワードを変更：心配なら、メインのアカウントのパスワードを変えておけば安心です。
- 本当にプライバシー漏えいの危険がありそうなら：警察に通報を。警察にはこの種の事件を扱う経験があります。

## シーン 6：AI 顔すり替え / AI 声まね詐欺——見たものが真実ではなくなる時代

2024 年以降、AI による顔すり替えと声のまね技術が進み、詐欺師がビデオ通話の中であなたの家族・上司・友人の顔や声に「なりすます」ことができるようになりました。「ママ、トラブルに遭ったから、早くお金を送って」——映っている人の顔も声も本物の息子そっくり。でも、偽物です。いま最も防ぎにくく、危険な詐欺手口の一つです。

**ステップ 1：破綻を見抜く——AI 偽装によくある 4 つの弱点**

- まばたきの頻度が不自然：人は 1 分間に 15　20 回まばたきしますが、AI 顔すり替えではまばたきが少なすぎたり、リズムがぎこちなかったりしがちです。
- 口の動きと音声がずれる：口の形と聞こえる声の間に、0. 数秒のズレがないか、よく見ましょう。
- 顔のふちがにじむ：顔の輪郭、耳のあたり、髪の生え際に「にじみ」やちらつきが出やすくなります。

- 光と影の向きが合わない：顔のハイライトや影が、背景の光の当たり方と食い違っていたり、横顔に向いたときに目鼻立ちがうまく「ついて回らなかったり」します。

**ステップ 2：3 つの方法で相手が本物か確かめる**

- 特定の動作をお願いする：相手に顔を向けさせたり、顔や口を手で覆わせたりして、口を開け閉めさせます。AI 顔すり替えは、顔が隠れたり大きく首を回したりするときに最も「ボロが出やすく」、顔がゆがんだり動きに追いつかなかったりします。
- 本人しか知らない質問をする：たとえば「この前、うちの犬をどこの病院に連れて行ったっけ？」といった嘘の質問を用意します——詐欺師なら答えられないか、口ごもるはずです。
- わかっている番号にかけ直す：一度ビデオ通話を切り、連絡先に登録してある番号へ直接かけ直します。詐欺師は本物の電話番号を操れないので、かけた瞬間に正体が分かります。

**ステップ 3：騙されたと確かめたあとの通報の手順**

1. まず送金を即停止：どんなに「急いで」と言われても一度ストップ。「早く振り込め、電話を切るな」と急かす相手は、ほぼ間違いなく詐欺師です。
2. 証拠をすべて残す：ビデオ通話の画面を録画・スクリーンショットし、チャット履歴、相手のアカウント、送金の証憑を保存します——これらは技術鑑定の重要な材料です。
3. 96110（詐欺被害相談専用ダイヤル）へ電話：「AI 顔すり替え / 声まね」タイプの詐欺だと伝えると、オペレーターが緊急の支払い停止を始めてくれます。
4. 110 に通報：証拠を一式そろえて派出所へ行き、事情聴取を受けます。AI による偽装技術が使われたことを特にはっきり伝えると、警察の立件と性質の判断に役立ちます。
5. 知人・家族に知らせる：詐欺師があなたの偽造動画で他人を騙すおそれがあるので、モーメンツや家族グループであらかじめ注意を呼びかけましょう。

## シーン 7：SIM スワップ攻撃（SIM 乗っ取り）——あなたの番号が「盗まれる」

聞いたことがない人が多いのに、被害が非常に大きい攻撃です。SIM スワップ（SIM カード乗っ取り）とは、攻撃者があなたの個人情報を偽装してなりすまし、キャリアショップで新しい SIM カードを再発行することです。再発行に成功すると、あなたのスマホの電波は突然消え、攻撃者のスマホはその新しいカードで、あなた宛ての SMS 認証コードをすべて受け取れます——Alipay も銀行も WeChat も、すべての扉が大きく開いた状態になります。

**どう気づく？SIM スワップの 3 つの予兆サイン**

- スマホが突然「圏外」/「SIM カードがありません」と表示される。カードには触れていないのに、再起動しても直らない——多くの場合、これが乗っ取られた瞬間です。
- キャリアから「カード再発行が完了しました」「手続きが完了しました」という SMS が届く——しかし自分では何も申し込んでいない。
- 突然認証コードが届かなくなり、ほかの SMS も届かなくなる——SMS が攻撃者のカードへ横取りされているからです。

**防御策——SIM カードにも「鍵」をかける**

- SIM カードの PIN を設定：［設定］＞［電話］＞［SIM PIN］から。有効にすると、スマホの再起動やカードの差し替えのたびに PIN の入力が必要になります。攻撃者がカードを手に入れても使えません。
- キャリアの二段階確認を有効にする：中国移動 / 聯通 / 電信のアプリにログインし、「セキュリティセンター」や「手続き時の確認」を探して、本人以外による再発行の SMS 通知と、手続き時の二重パスワード確認をオンにします。こうすれば、他人があなたのカードを再発行しようとしたとき、窓口の係員が追加の本人確認を求めてくれます。
- 重要なアカウントは SMS 認証だけにしない：銀行や Apple ID など重要度の高いアカウントは、SMS の認証コードではなく、認証アプリ（Google Authenticator、Microsoft Authenticator）やハードウェアキーを優先しましょう。
- 身分証のコピーには透かしを：手続きに必要なコピーには「　の手続きにのみ使用」と書き添えます——攻撃者は、流出した身分証の情報であなたになりすますことが多いからです。

**乗っ取られたあとの緊急対処——スピードがすべて**

1. すぐにキャリアのサポートへ電話して利用停止：移動 10086 / 聯通 10010 / 電信 10000。ただちに回線を止め、カード再発行の手続きを凍結するよう依頼し、攻撃者があなたの番号を使い続けられないようにします。
2. 別のデバイスで重要アカウントにログイン：パソコンか家族のスマホから、銀行、Alipay、WeChat、Apple ID のパスワードをすぐに変更し、見知らぬデバイスを締め出します。
3. 決済アカウントを凍結：Alipay は 95188 で利用停止、WeChat はセキュリティセンターで凍結、銀行はサポートにネットバンキングの凍結を依頼します。
4. 身分証を持ってキャリアショップへ：本人が身分証を持参して直営店に行き、新しい SIM カードを再発行して、番号の利用権を取り戻します。

5. 警察に通報：SIM スワップは「個人情報の侵害」にあたります。証拠（回線停止の時刻の記録、金銭的損失、再発行の記録）をそろえて被害を届け出ましょう。

6. 総点検：番号が戻ったら、直近のログイン記録と入出金明細を一つずつ確認し、なりすましの形跡がないか確かめます。

**予防は治療に勝る——ふだんからの 5 つのこと**

1. ロック画面のパスワード＋二段階認証——両方オンに
2.「デバイスを探す」が有効か確認
3. SIM カードに PIN を設定（［設定］＞［電話］＞［SIM PIN］）——スマホをなくしても、他人にカードを使われません
4. 出どころの分からないアプリを入れない、知らないリンクを開かない
5. 大事なデータ（写真・連絡先・チャット履歴）を定期的にバックアップ

**やってみよう：あなた専用の「緊急連絡カード」をつくる**

1. 紙を 1 枚用意して、次の番号を書きましょう：キャリアのサポート（移動 10086 / 聯通 10010 / 電信 10000）、銀行のサポート、96110 詐欺被害相談専用ダイヤル、110（警察通報）、家族の電話番号。
2. Apple ID / 各ブランドのアカウントの登録メールアドレスと、予備の連絡手段も書いておきます。
3. この紙は財布に入れるか冷蔵庫に貼っておく——スマホの中にしまわないこと。何かあったときスマホが手元にないかもしれないからです。
4. 今すぐチェック：「デバイスを探す」が有効か、SIM カードの PIN が設定済みか確かめましょう。

**この章のまとめ**

- スマホ紛失のゴールデン 30 分：借りたスマホで発信遠隔ロック SIM 利用停止決済凍結知人・家族に連絡警察へ通報
- 騙されたあとの 72 時間：送金を止める証拠を残す 96110 に電話通報銀行に連絡パスワード変更
- スマホの感染：セーフモード怪しいアプリを削除セキュリティスキャン初期化（最終手段）
- アカウント乗っ取り：パスワード変更デバイスを締め出すログイン履歴を確認二段階認証を有効化連携情報を確認決済設定を確認
- 脅迫メッセージの 99.9% は詐欺。返信も支払いもせず、スクリーンショットを撮ってブロック

- AI 顔すり替え / 声まね詐欺：まばたき・口の動き・輪郭・光影の破綻をチェック。顔を向ける・口を覆う動作を頼み、本人しか知らない質問をする、登録済みの番号へかけ直す
- SIM スワップ乗っ取り：スマホが突然「圏外」になったら警戒。SIM PIN の設定、キャリアの二段階確認、重要アカウントは認証アプリへ切り替え
- ふだんの予防 5 か条：ロック画面＋ 2FA、デバイスを探す、SIM PIN、出どころ不明アプリを入れない、定期バックアップ

# 六、展望編とコネクテッドカー編（第33-34章）

# 第三十三章：未来展望——スマホの次のステップ

スマホは一夜で消えないが、その“独占的地位”は次世代デバイスに分流される可能性がある。この章では三つのことを語る：次世代デバイスはどんな姿か、スマホがいかに“隠身”して新しいアイデンティティの基盤になるか、そして私たちが前もって考えるべき倫理的議題。

## 一、AR/VR と空間コンピューティング：スマホ以外の“第二のデスクトップ”

次世代デバイスの競争はすでに始まっており、スマホの時間を最も分流しそうなのは AR/VR と空間コンピューティングデバイスだ：

- **Vision Pro 後継：** Apple Vision Pro の発表は”空間コンピューティング”概念の正式な実装を示したが、初代製品の価格（29999 元から）と重量は依然ハードル。2026-2027 年により軽くて安い普及版 Vision が出ると予想され、空間コンピューティングが”開発者のおもちゃ”から”初期マス層”へと移行する。その時、iPhone は Vision グラスの演算パートナーとデータハブになる可能性がある。

- **折りたたみスマートフォン：** 目の前に”第二のデスクトップ”を開き、デジタル情報が空気のように至る所に存在するように。Meta と Ray-Ban のコラボスマートグラスはすでに数百万個売れ、国内の Rokid、XREAL、INMO などもコンシューマー向け AR グラスを推出。2025-2026 年の AR グラスはリアルタイム翻訳、ナビ HUD、AI アシスタント対話ができるが、光導波ディスプレイとバッテリーの制限があり、”スマホ代替”までまだ 3-5 年ある。

- **スマートリングとウェアラブル：** Samsung Galaxy Ring、Oura Ring などのスマートリングが流行し始め、スマホの”肌身につけたセンサー”として健康モニタリングを補完。スマホの役割は”万能端末”から”ハブ + 複数のライト化アクセサリ”の組み合わせへと移行している。

## 二、ブレイン・マシン・インターフェース：思考がインタラクションの媒介に

思考自体がインタラクションの媒介になるが、この道のりは想像より長い：

- **Neuralink 最新の進展：** Musk の Neuralink は 2024 年に初の人体埋め込みを完了し、最初の被験者 Noland Arbaugh が思考でコンピュータのマウスを操作してチェスを指した。現在、埋め込み型 BCI（ブレイン・マシン・インターフェース）は主に麻痺患者の補助シーンに向けられ、彼らが再びデジタル世界を”操作”できるようにすることが目標だ。
- **非侵襲型 BCI：** 複数の企業が非侵襲型ヘッドバンド（Kernel、OpenBCI など）を開発し、手術なしで脳波信号を読み取れるが、精度は埋め込み型に遥かに劣る。”思考でタイピング”までにはまだかなりの距離がある。
- **コンシューマー向けの展望：** ブレイン・マシン・インターフェースがコンシューマー向けスマホインタラクションに到達するまであと 10-20 年だが、短期的にはアクセシビリティ補助や集中力モニタリングなどのシーンで先に使われるかもしれない。倫理とプライバシーの問題（”私の脳波は誰のプライバシーか”）が技術の成熟に先んじて議論の焦点になる。

## 三、スマホの“隠身”と新しいアイデンティティの基盤

スマホのもう一つの未来の進化方向は“隠身”かもしれない——至る所に存在する“黒子”となり、あなたのデジタルアイデンティティの唯一の基盤になる：

- **二、スマートフォンの「不可視化」と新たなアイデンティティの礎：** 瞬時に”あなたはあなた”の認証を完了し、あらゆるサービスのログインで手動認証コード入力が不要に。
- **クラウド二要素認証：** 新規スマホ起動時にデジタル資産継承方案を事前設定し、”身後の事”を出荷時デフォルトに。
- **デジタル遺産が初期設定に：** あなたのクラウド上の個人データ金庫こそが中心で、ハードウェアはアクセスの入口にすぎない。機種変更、デバイス変更しても、アイデンティティとデータは常にあなたについてくる。

## 四、倫理的思考：私たちはまだスマホの主人か？

：あなたのクラウド上の個人データ金庫こそが中心。

- **AI 倫理：** オンデバイス AI が自分よりも好みを理解する時、あなたの「選択」は自主的な意志か、誘導された結果か？AI Agent が代わりに注文やメッセージ返信をする時、責任と境界はどこにありますか？

- **データ主権：**あなたの健康データ、脳波データ、生体特徴データは、あなたに帰属するのか、デバイスメーカーに帰属するのか、クラウドサービスプロバイダーに帰属するのか？「個人情報保護法」と EU の GDPR は出発点にすぎず、「データポータビリティ権」と「忘れられる権利」の真の実装にはまだ長い道のりがあります。
- **デジタル・ディバイド：**AI スマホ、折りたたみ画面、衛星通信はすべて”使える”と”使えない”の格差を広げている。デジタルインフラへの公平なアクセスは、20 年前の”ネットが使えるか”と同じように重要な新たな社会課題になる。
- 三、倫理的考察：私たちは、まだスマートフォンの主人なのか？
- 一つの究極の問いが常に存在します：私たちはテクノロジーによって力を与えられた「神」なのか、それともアルゴリズムに餌付けされた「水槽の脳」なのか？

## 五、繋がりを保ち、独立も保つ——あなたのデジタルライフ哲学

技術がどう進化しようと、以下のデジタルライフ哲学は 10 年後も適用できる：

1. 数分おきに無意識に画面を点灯させているとき、それはあなたの習慣ですか、それとも設計された「依存行動」ですか？
2. 四、つながりを保ちながら、独立も保つ——あなたのデジタルライフ哲学
3. リソースの分配者であり、プッシュ通知の受け手ではない。
4. 自動化をあなたの忠実な「コパイロット」にする。
5. あなたの堅牢な「デジタル後方拠点」を構築する。

**定期的に「デジタル断捨離」と「ソーシャル断食」を実行する。**

あなたとスマホ（そして将来の AR グラスや AI アシスタント）のために簡単な“利用規約”を書き、関係を定義する。例えば：

- 試してみよう：あなたの「個人デジタルデバイス利用サービス規約」を作成する
- あなたとあなたのスマートフォンのために簡単な「サービス規約」を書き、関係を定義しましょう。例えば：
- 第 1 条：私の注意の時間は私のものです。あなたは私が仕事/家族と過ごす時に自ら「サイレントモード」に入るべきです。

**本章のまとめ**

- Vision Pro と AR グラスはスマホ以外の新たなコンピューティング入口だが、短期的にはスマホを代替できない
- ブレイン・マシン・インターフェースはコンシューマー向けまであと 10-20 年、倫理問題が技術の成熟に先んじる

- スマホは”隠身”方向に進化し、デバイス中心からアイデンティティ中心へ
- AI 倫理、データ主権、デジタル・ディバイドは今後 10 年の核心議題
- 繋がりを保ち独立も保つことは、デジタルライフの不変の哲学

# 第三十四章：車載連携ディープガイド——スマホとクルマの融合

自動車は“四つの車輪とソファ”から“四つの車輪とスマートフォン”へと進化している。ガソリン車でも新エネルギー車でも、スマホとカースルの関係が運転体験を決める。この章ではカースル連携の三大ルート、各陣営のカースルシステムの特徴、そして運転をよりスムーズにする小技を整理する。

## 一、カースル連携の三大ルート

現在、スマホと自動車の“連携”には主に三つのアプローチがあり、これらを理解すればエコシステム全体が理解できる：

- **スマホをカースルにキャスト**：カースルは”ディスプレイ”と”スピーカー”としてのみ機能し、演算とマップデータはすべてスマホから。代表方案：Apple CarPlay、Google Android Auto、Huawei HiCar、Baidu CarLife。利点はスマホのエコシステムをそのまま車に持ち込め、App を再インストール不要；欠点はカースル本体のハードウェア能力が遊休化する。
- **スマホとカースルの深い融合**：カースルは独立システムだが、スマホとアカウント、アプリ、データを共有し、タスクを”リレー”できる。代表方案：HarmonyOS コックピット（AITO/Luxeed）、BYD DiLink、NIO NIO OS、Li Auto カースルシステム。利点は体験の一体化；欠点は通常同じブランドエコシステムに縛られる。
- **CarPlay 2.0 深い統合版**：2025 年に Apple が発表した次世代 CarPlay は、単なる”キャスト”ではなく、カースルのメーターパネル、エアコン制御、シート調整などのコア機能を引き受ける。Ferrari、Aston Martin、Porsche などのブランドが先行搭載し、カースルシステムが”スマホメーカー主導”の新段階に入ったことを示す。

## 二、CarPlay / Android Auto / HiCar の比較

三大主流キャスト方案はそれぞれ重点が異なり、車選びやスマホ選びの際に以下の表を参照できる：

| 方式 | 対応スマホ OS | 対応車種 | 機能の豊富さ | 国内での利用性 |
|---|---|---|---|---|
| Apple CarPlay | iPhone のみ | 極めて広い、大部分の車種が対応 | 高い、マップ/音楽/電話/メッセージすべて可能 | 良い、ネイティブで利用可能 |
| Android Auto | Android（国内スマホはサービスフレームワークのインストールが必要） | 比較的広い、合合弁/輸入車が中心 | 高い、CarPlay と同等 | 一般、VPN でのアクティベーションが必要 |
| Huawei HiCar | Huawei スマホ中心、一部他の Android も対応 | 国産車と一部合弁車種 | 比較的高い、HarmonyOS コックピットと連携可能 | とても良い、ローカルエコシステム |
| Baidu CarLife | iOS と Android の両方に対応 | 比較的広い、旧型車の後付けで一般的 | 中程度、更新が遅い | とても良い、ローカルエコシステム |

## 三、HarmonyOS コックピット：スマホとカースルのシームレスな連携

CarPlay が“スマホをカースルにキャスト”だとすれば、HarmonyOS コックピットは“スマホとカースルはもともと一家”だ。Huawei スマホと AITO/Luxeed カースルの連携は、現在の国内カースル連携の最高水準を代表する：

- **スーパーデスクトップ：**カースルがスマホ上の App を直接呼び出し、カースルに再インストール不要。スマホで聴いている音楽、ナビしているルートが、乗車時に自動的にカースル画面に引き継がれ、降車時に再びスマホに戻る。全過程シームレス。
- **アプリ引き継ぎ：**スマホで設定したナビは、ドアを閉めるとカースルが自動的に引き継ぐ；駐車場に到着後、降車してからスマホが歩行ナビを続けて最終目的地まで案内。”ラストワンマイル”が途切れない。
- **スマホとカースルのアカウント共有：**Huawei アカウントでログインすると、カースルが自動的に音楽会員、マップのお気に入り、スケジュール、連絡先を同期。カースルとスマホは同じアイデンティティの異なる端末であり、二つの独立システムではない。

## 四、新興自動車メーカーのカースルシステム

国産新エネルギー車のカースルはすでに“ナビ＋音楽”からモバイルスマート端末へと進化し、各社のアプローチはそれぞれ特色がある：

- **NIO NIO OS：**NOMI 音声アシスタントは業界のベンチマークで、複数コマンド認識、

コンテキスト連続対話、感情表情付き。NIO Phone はスマホを”車のキー＋第二画面”にし、NIO エコシステムに深く結びつく。

- **Li Auto カースル：**マルチスクリーンインタラクションがハイライト、助手席スクリーンで独立してドラマ視聴、子どもスクリーンでアニメ視聴可能。”タスクマスター”でシーンをカスタマイズでき、例えば”ドアを開けると音楽自動一時停止、閉めると再生再開”。
- **Xiaomi HyperOS カースル：**Xiaomi スマホ、時計、家電と深く連携。スマホで設定したアラームがカースルで通知；カースルで帰宅ナビすると、家のエアコンが事前に起動。HyperOS が”人・車・家の全エコシステム”を貫通。
- **BYD DiLink：**象徴的な回転スクリーン、縦画面で短動画視聴、横画面でマップ表示、App の方向に適応。アプリストア内蔵で、エコシステムの豊富さは伝統的自動車メーカーの中でトップ。

## 五、スマホをカースルに——使えるか？

カースルが古すぎるか使いにくすぎる場合、スマホで“代用”できるか？答えは：できるが、シーンによる。

- **Motorola Ready For / デスクトップモード：**一部の Motorola、Samsung、Huawei スマホは”デスクトップモード”に対応し、モニターに接続すると簡易パソコンになる。理論上はカースル画面に接続しても使えるが、実際の互換性はまちまちで、自動車専用インタラクション（ステアリングホイールボタンなど）がない。
- **スマホホルダー vs 純正カースル：**純正カースルがカクつく、マップが更新されない場合、安定したスマホホルダー＋スマホナビが現実的な選択。注意点：(1) スマホ画面が小さく運転中に見えにくい；(2) 夏の直射でスマホが過熱しクロックダウンや自動シャットダウンする可能性；(3) 長時間ナビでバッテリー消費が早く、車載充電器が必要。
- **アドバイス：**短距離や臨時使用ならスマホホルダーで十分；日常通勤や長距離は CarPlay/HiCar 対応のカースルにアップグレードするか後付けカースルを設置するのがよく、体験も安全もより良い。

**セキュリティに関する注意**

どの方法を使っても、運転中のスマホ操作は交通規則違反です。スマホホルダーは「ナビを見る」ためのみに使用し、「スマホを操作する」ためには使えません。目的地の設定、楽曲の切り替えなどの操作は、必ず停車してから行うか、音声操作を使用してください。

## 六、実用的なヒント

- **ワイヤレス CarPlay の接続が不安定：**(1) カースルの Bluetooth をオフにして再ペアリング；(2) カースル側のペアリング済みデバイス記録を削除して再接続；(3) スマホ

が複数の Bluetooth デバイスに同時接続して競合していないか確認；(4) カースルファームウェアとスマホシステムを最新版に更新；(5) どうしてもダメなら有線 CarPlay に変更、より安定。

- **カースルのデータ通信量が使い切った：**(1) スマホでテザリングし、カースルをスマホのホットスポットに接続；(2) カースル設定で”データ通信量大”アプリ（動画、オンライン音楽など）を Wi-Fi に切り替え；(3) 定期的にカースルのオフラインマップキャッシュをクリア。

- **運転モードの自動トリガー：**(1)iOS の”集中モード”で”カースル Bluetooth 接続時に自動オン”を設定し、非緊急通知を自動ブロック；(2)Android の”運転モード”やサードパーティ App（Tasker など）で Bluetooth 接続時に自動ミュート、自動返信を実現；(3)Huawei スマホは”スマート感知決済”を設定し、乗車時に自動的にカースルで音楽再生に切り替え。

**車選びのアドバイス**

カースル体験を重視する場合、車選びの際は必ずディーラーで実測する：(1) 自分のスマホを持参し CarPlay/HiCar の接続が安定しているかテスト；(2) 音声アシスタントの反応速度とコマンド認識率を試す；(3) アプリストアによく使う App があるか確認；(4) 今後の OTA アップデート頻度を確認——良いカースルはソフトウェアの反復で作られる。

**実践：カースル連携体験を最適化する**

1. スマホとカースルが対応する連携方案（CarPlay / HiCar / Android Auto / CarLife）を確認。
2. カースルの Bluetooth ペアリングリストを一度クリーンアップし、常用スマホだけ残し接続の競合を減らす。
3. ”運転集中モード”を設定し、カースル Bluetooth 接続時に自動オンにする。
4. カースルブランド App（HarmonyOS Intelligent Driving、Li Auto、NIO App など）をダウンロードし、リモートでのエアコン操作や車況確認ができるか見る。

**本章のまとめ**

- カースル連携の三大ルート：キャスト（CarPlay など）、深い融合（HarmonyOS コックピットなど）、CarPlay 2.0 深い統合
- CarPlay はエコシステムが最も広い；HiCar は国内体験が良い；Android Auto は国内で設定が必要；CarLife は旧型車の後付けに適している
- HarmonyOS コックピットのスーパーデスクトップ、アプリ引き継ぎ、アカウント共有は深い融合の方向を代表
- NIO NOMI、Li Auto マルチスクリーン、Xiaomi 人・車・家、BYD 回転スクリーンそれぞれ特色あり

- スマホのカースル代用は臨時シーンのみに適し、長期的にはアップグレードまたは後付けカースルを推奨
- 実用ヒント：CarPlay 再接続、スマホテザリングでデータ補充、Bluetooth で運転モードトリガー

# 子どもとスマホ特集（第三十五章）

# 第三十五章：子どもとスマホ——デジタル時代の教育ガイド

この章では“スマホは猛獣”とは言わず、“早く触れるほど賢くなる”とも吹聴しない。年齢別にデバイスの付与、ルール設定、リテラシー教育、健康保護を段階的に行う。

## 一、何歳で子どもにスマホを与えるか？

標準答はないが、年齢の基準ラインはある。早すぎると、子どもの脳の前頭葉がまだ成熟しておらず、自制力がアルゴリズムの“誘惑設計”に追いつかない；遅すぎると、仲間内で“社交的断絶”に陥る可能性がある。以下は年齢別の提案：

- **3-6 歳：**できるだけスマホに触れさせない。使う場合、一度に 15 分以内、内容は親が厳選した教育アニメ中心、かつ必ず**一緒に見る**。この段階は画面が視力と注意力の発育に最も影響する。
- **7-9 歳：**親のデバイスで”借用”でき、オンライン授業、お話を聴く、調べ物に使う。”使用前に申請、使い終わったら返す”のルールを確立し、専用スマホは与えない。
- **10-12 歳：**”フィーチャーフォン”またはキッズウォッチの検討を、登下校の連絡に対応。スマホが必要な場合は厳格なペアレンタルコントロールをオンに。
- **13 歳以上：**中学に入り、社交ニーズが高まり、スマホを与えてもよいが、”利用規約”と定期振り返りを組み合わせる。

**”十分でよい”原則**

子どもの最初のスマホはフラッグシップである必要はない。プロセッサの“過剰性能”は往々にしてゲームに使われる。バッテリーが十分、性能中程度、目に優しい画面の千元機や公式リファービッシュ機を選ぶ方が管理しやすい。予算の目安：1500-2500 元で大多数の学生のニーズを満たせる。

### スマートウォッチを過渡期の方案として

“下校後に連絡できるか”の問題を解決したいだけなら、キッズウォッチの方がスマホより適している。電話、音声送信、位置情報のみで、ブラウザや短動画がなく、“依存の入口”

が天然制限される：

- **小天才電話ウォッチ**：エコシステム最も充実、同年代間の社交”タッチで友達追加”が核心的セールスポイントだが、専用 App エコシステムは比較的閉じている。
- **Huawei キッズウォッチ**：Huawei スマホとの連携が良い、位置情報が正確、ビデオ通話対応、コスパが高い。
- **Xiaomi キッズウォッチ**：価格が手頃、基本機能が充実、予算が限られた家庭に適している。

ウォッチ購入時の注意：**4G 全網通対応**のモデルを選ぶ（2G/3G は段階的に廃止中）、キャリアのキッズプラン料金を確認。

## 二、ペアレンタルコントロールシステム完全攻略

デバイスを子どもに渡す前、最初にすべきは電源オンではなくペアレンタルコントロールの設定だ。両プラットフォームともネイティブ方案がある：

### iOS：スクリーンタイム

- **App 制限**：設定 > スクリーンタイム > App 制限、各 App カテゴリごとに毎日の利用可能時間を設定（例：ゲーム 30 分）。
- **使用停止時間**：毎日の”消灯時間帯”（例：22:00-7:00）を設定、期間中すべての App がグレーアウトし電話のみ利用可能。
- **コンテンツ制限**：年齢別に映像、Web、App インストールをフィルタリングし、App 内課金を禁止。
- **ファミリー共有**：親の iPhone で子どものデバイスをリモート管理、ダウンロード要求の承認、利用レポートの確認。

### Android：Google Family Link / ブランド別ペアレンタルコントロール

- **Google Family Link**：Google 公式方案、デバイスの利用時間をリモート管理、デバイスのロック、アクティビティレポートの確認が可能。国内 Android は機種により利用可能性が異なる。
- **Xiaomi キッズスペース**：設定 > キッズスペース、利用時間、ホワイトリスト App を設定、退出には親のパスワードが必要。
- **Huawei 学生モード**：設定 > 健康的なスマホ使用 > 学生モード、利用時間帯と App を制限、リモート管理対応。

- **OPPO/vivo キッズモード：**”設定”または専用”キッズスペース”App でオン、原理は類似。

| 機能 | iOS スクリーンタイム | Android Family Link / ブランド方案 |
|---|---|---|
| App 制限 | 対応、カテゴリ別 | 対応、App 別 |
| 使用停止時間 | 対応 | 対応（ブランド方案により若干差異） |
| コンテンツフィルタ | システムレベル、比較的徹底 | ブランド依存、ばらつきあり |
| リモート管理 | ファミリー共有 | Family Link / ブランドクラウド |
| App インストール承認 | 親の承認が必要 | Family Link 対応 |

**ペアレンタルコントロールは”一度設定で永遠”ではない**

子どもは必ず回避方法を見つける（ネットで“スクリーンタイムの破り方”のチュートリアルが大量にある）。技術的手段は最低限のラインにすぎず、本当の防線は親子のコミュニケーションと信頼だ。毎月子どもと一緒に利用レポートを振り返ることを推奨し、一方的な監視は避ける。

## 三、ティーンモード

主要コンテンツプラットフォームには“ティーンモード”が内蔵されており、オンにすると利用時間制限、不良コンテンツのフィルタリング、課金・投げ銭の無効化が行われる。これは国家法規が要求する強制機能だが、**親が能動的にオンにしてパスワードを設定する必要がある：**

- **Douyin/Kuaishouteen モード：**毎日 40 分制限（22:00-6:00 使用禁止）、内容は厳選された科学・教育動画のみ、ライブ配信、投げ銭、課金不可。

- **WeChat ティーンモード：**WeChat Pay を制限（親の承認枠が必要）、公式アカウント、動画アカウントの内容を制限、シェイク、近くの人などの機能はデフォルトでオフ。

- **ゲーム依存防止システム：**国家レベルの実名認証 + 時間制限。未成年者は金曜、土曜、日曜および法定祝日の 20:00-21:00 のみゲームにログインでき、毎日 1 時間。

**実名認証に親の身分証で“アカウントを借りる”のはダメ**

多くの子どもが親の身分証でゲームアカウントを登録して依存防止を回避していますが、これは利用契約違反であると同時に、子どもを保護から遠ざけます。親は自分の決済パスワードと身分情報を厳重に管理し、手間を省くために「アカウントを貸す」ことは避けてください。

## 四、ネットリテラシー教育

技術で 90% のリスクをブロックできるが、残り 10% は子どもの判断力次第。“ネット禁止”よりも“どうネットを使うか”を教える方が重要だ：

- **プライバシー保護**：見知らぬ人に本名、学校、住所を明かさない；個人写真、身分証、家族写真を送らない；位置情報機能はデフォルトでオフ、使用時のみオン。
- **ネットいじめの識別と対応**：罵倒、孤立、悪意のある画像編集はすべていじめに該当。子どもに遭遇時には罵り返さない、証拠を削除しない、すぐに親に伝えるよう教える。WeChat はメッセージ長押しで通報でき、プラットフォームが加害者アカウントを処理する。
- **コンテンツ批判**：子どもに”事実”と”意見”を見分けるよう教え、タイトル詐欺、センセーショナル主義を識別；アルゴリズムは見たいものだけ推すのであり、世界の全貌ではないと理解；”棚から牡丹餅”の広告は一律に信じない。
- **ネットセキュリティ**：知らないリンクをクリックしない、出所不明の QR コードをスキャンしない、非公式ストアの App をダウンロードしない；パスワードを誰にも（”親友”含め）共有しない；認証コードを要求する相手は一律に詐欺師とみなす。

**実践：子どもと一緒に《スマホ利用規約》に署名する**

ルールを書き出し、双方が署名して目立つところに貼るのは、口頭の約束よりはるかに効果的。以下を含めることを推奨：

- 利用時間帯：例：”宿題完了後 19:30-20:30 まで利用可能”。
- 禁止エリア：食卓、寝室就寝前、授業中はスマホをしまう。
- コンテンツの境界：親に無断で App をインストールしない、見知らぬ人とプライベートチャットしない。
- 違約の後果：例：”一度超過したら、翌日の利用時間を半分に”。
- 親の約束：子どもの前で”スマホスクロール”を合意時間以上しない、模範を示す。

## 五、健康的な利用習慣

子どもの目、頸椎、睡眠はすべて発育期にあり、スマホの身体への悪影響は成人より大きい：

- **スクリーンタイムの提案**：3-6 歳は毎日 1 時間以内；6-12 歳は学習系は制限なし、エンタメ系は 1 時間以内；12 歳以上は子どもと相談、ただしエンタメ系は 2 時間以内。
- **アイケア習慣**：”20-20-20 ルール”を守る——画面を 20 分見るごとに、20 フィート（約 6 メートル）先を 20 秒見る；寝る 1 時間前はスマホを見ない、ブルーライトがメラトニンを抑制し睡眠に影響する。

- **姿勢管理：**スマホ画面を目の高さに、下を向いて猫背にならない；連続使用は 30 分以内で立ち上がって活動する。
- **親子共通のルール：**食卓、寝室、書斎に”スマホ禁止エリア”を設ける；家族で映画を見る、野外活動の時はスマホを一括してしまい、”スマホを置く”ことを子どもへの罰ではなく家族文化にする。

**親が最高の”ファイアウォール”**

研究によれば、親自身のスマホ利用時間は子どもと強い正の相関がある。子どもにスマホを見ないよう要求しながら自分は深夜まで短動画をスクロールすれば、ルールは無効になる。子どもを変えるには、まず自分のスマホを置くことから。

## 六、デジタル遺産と子ども

この節は見落とされがちだが重要だ。子どものデジタル資産（アカウント、写真、課金残高、ゲーム装備）も相続とプライバシーの問題に直面する：

- **子どものデジタル資産の相続：**子どもが不慮の死を遂げた場合、SNS アカウント、ゲームアカウント、クラウド写真の相続プロセスは成人と同じで、死亡証明書、親族関係証明が必要。親は普段から子どもの重要アカウントに”遺産連絡先”または”緊急連絡先”を設定しておくことを推奨（Apple 遺産連絡先、WeChat アカウント相続など）。
- **未成年者のデジタルプライバシー：**《個人情報保護法》では、14 歳未満の未成年者の個人情報を処理するには監護者の同意が必要で、専用のプライバシールールを制定すべきと規定。親は子どもがよく使う App のプライバシーポリシーを定期的に審査し、”パーソナライズされた推奨”と”データ共有”のスイッチをオフにする。
- **アカウント登録の提案：**子どものために登録するアカウントは親が管理できるメール/電話番号を使用し、未成年段階で一元管理しやすくする；子どもが成年に達した後にアカウントの引き渡しができる。

**”子どものデジタル遺産”のもう一面に警戒**

子どもは誕生から“記録”されている——親がアップした成長写真、幼稚園が送った動画、スマートウォッチの健康データ。これらのデータは一度アップロードされると完全削除が困難。投稿前に考えよう：十年後、子どもはこの写真で恥ずかしい思いをしないか？子どものデジタル人格を尊重し、“子ども投稿”を控えることから始める。

**本章のまとめ**

- スマホを与える年齢に標準答はないが、3-6 歳はできるだけ触れさせない、13 歳前はキッズウォッチやフィーチャーフォンで過渡
- ”十分でよい”：最初のスマホは千元機を選びフラッグシップではなく、過剰性能は過剰誘惑に等しい

- iOS スクリーンタイムと Android Family Link/ブランド別ペアレンタルコントロールは二重の技術的な防線、併用でより安定
- ティーンモードとゲーム依存防止は法規要求だが、親が能動的にオンにしパスワードを守る必要がある
- ネットリテラシーは技術的ブロックより重要：子どもにいじめの識別、プライバシー保護、コンテンツ批判を教える
- 健康習慣は親から始める——あなたがスマホを置けば、子どもも置く
- 子どものデジタル遺産とプライバシーは新興議題、普段から”遺産連絡先”を設定し子ども投稿を控える

# 第三十六章：スマホとスマートホーム——家を賢くする

スマホには、ますます重要になる役割がもう一つあります——家じゅうの「リモコン」になることです。電球 *1* 個、スマートプラグ *1* つ、玄関の鍵にいたるまで、スマートホームは「ガジェット好きのおもちゃ」から「ごくふつうの設備」になりつつあります。この章では、プロトコル・エコシステム・ハブ・入門セットを整理し、最後に安全チェックリストで締めくくります。「スマート」に足をすくわれないための知識です。

## スマートホームのプロトコル：どれを選ぶかが大事

スマートデバイスを買う前に、まずどんなプロトコルに対応しているかを見ましょう。スマホやエコシステムがそのデバイスを「思いどおりに動かせる」かどうかが、これで決まります。いまの主流は 4 種類。それぞれの性格は次のとおりです：

- Matter プロトコル：2022 年に CSA（接続標準連盟）が発表。背後には Apple、Google、Amazon、Samsung などの大手が手を組んでいます。目的はただ一つ——すべてのブランドのスマートデバイスに「同じ言葉」を話させること。2023  2026 年には急速に統一標準になりつつあり、対応デバイスは Apple HomeKit、Google Home、Amazon Alexa、Mi Home（米家）に同時に接続でき、単一のエコシステムに縛られません。

- Thread プロトコル：Matter の土台となるネットワークの「配管」です。低消費電力でメッシュ構成（どこかのノードの電源が落ちても、信号が自動的に別の経路を通る）、応答が速く、もたつきません。Thread ボーダールーターを備えたデバイス（HomePod mini、Apple TV 4K、Xiaomi ハブゲートウェイなど）を選ぶと、動作が明らかに安定します。

- Wi-Fi：帯域が広く、ルーターに直結できますが、消費電力が高く、デバイスが増えるとチャンネルを圧迫しがち。カメラやロボット掃除機など、大きなデータをやり取りするデバイス向きです。

- Bluetooth（BLE）：初期設定が簡単で省電力ですが、距離が短く、メッシュ構成はできません。玄関鍵や温湿度計など、小さなデータ量のデバイス向きです。

- Zigbee：昔からある低消費電力のメッシュ型プロトコルです。専用ゲートウェイが必要なものの、デバイスは安価で安定。ただしブランドをまたぐ相互接続が弱く、Matter ＋ Thread に少しずつ置き換えられつつあります。

**ひとことで言うと**

2026 年にスマートデバイスを買うなら、まず「Matter over Thread」の表記があるかを確認——これが、いま最も時間の試練に耐える選択です。古いデバイスが Wi-Fi や Bluetooth までしか対応していなくても、急いで処分する必要はありません。使い続けて、新しく買うときに Matter へ向かえば OK です。

## 三大エコシステムの比較

プロトコルは「言葉」で、エコシステムは「コミュニティ」です。どれも Matter に対応していても、日々のオートメーションや音声操作のためには、メインのエコシステムを 1 つ選ぶことになります。三大主流エコシステムを比較してみましょう：

| エコシステム | 強み | 弱み | 向いている人 |
|---|---|---|---|
| Apple HomeKit | プライバシーが最強（ローカル処理が中心）、iPhone / Siri / ショートカットとの連携が滑らか | デバイスが少なめで高め、HomePod か Apple TV がハブとして必要 | 家じゅう Apple 製で、プライバシーと体験を重視するユーザー |
| Mi Home（Xiaomi） | デバイスが最も豊富でコスパ最高、「Xiao AI（小愛同学）」も使いやすい | エコシステムがやや閉じ気味、データはクラウド処理が多く、広告も多め | 予算が限られていて、家じゅうを一度にそろえたい入門ユーザー |
| HUAWEI スマートホーム（HarmonyOS） | HarmonyOS のスマホ / タブレット / ウォッチとシームレスに連携、対応デバイス数も急増中 | 海外デバイスの対応が少なく、エコシステムはまだ追い上げ中 | HUAWEI スマホのユーザー、「スーパーデバイス」による複数デバイス連携を重視する人 |

**エコシステム選びのアドバイス**

スマホに合わせてエコシステムを選ぶ：iPhone ユーザーは HomeKit を、HUAWEI ユーザーは HUAWEI スマートホームを、その他の Android ユーザーは Mi Home がいちばん楽です。Matter プロトコルのおかげで、別々のエコシステムのデバイスも同じ 1 台のスマホで操作できるので、最初から一つに「決め込む」必要はありません。

## スマホをホームハブにする

入門レベルの多くの場面では、スマホこそがあなたの「ホームハブ」です——アプリを通して照明・エアコン・カーテン・防犯を操作し、オートメーションを組み立てて、家を「自

分で動くもの」にできます：

- デバイス操作：HomeKit / Mi Home / HUAWEI AI Life アプリを開けば、遠隔でも自宅内でも、照明のオン / オフ、色温度の調整、エアコンの温度設定、カーテンの開閉、カメラ映像の確認ができます。
- オートメーション：「条件動作」を組み合わせます。よく使う 3 大シーン：帰宅モード：自宅への到着を検知＋日没玄関灯を点ける＋エアコンを入れる＋カーテンを閉める＋玄関鍵を施錠。
- 外出モード：最後の 1 人が出かける（自宅を離れたことを検知、または玄関鍵の施錠）照明を消す＋エアコンを切る＋スマートプラグを切る＋カメラを起動＋警戒モードに設定。
- おやすみモード：毎晩 23:00 家じゅうの主照明を消す＋カーテンを閉める＋エアコンを 26 度に切り替え＋常夜灯を点ける。

NFC タッチ：玄関に NFC タグを貼っておけば、スマホをかざすだけで対応するシーンが動きます（1 回タッチで玄関灯を点灯、もう 1 回でおやすみモードへ）。iPhone は「ショートカット」のオートメーションで、Android は「ウォレット」系アプリや各メーカーのアプリで書き込めます。

**やってみよう：「帰宅モード」を組み立てる**

スマホのスマートホームアプリの「オートメーション」で新規作成します：トリガーは「特定の場所に到着」（自宅）を選び、アクションには順に「玄関灯を点ける」「エアコンを 26 度に設定」「カーテンを閉める」を追加。保存したら一度外へ出て、帰宅したときに勝手に明かりがつくか確かめましょう——初めて成功したときの達成感は、新しいデバイスをポチるより気持ちいいものです。

## 入門セットのおすすめ

一度に全部そろえる必要はありません。この 4 点セットから始めれば、日常のニーズの 8 割はカバーできます：

- スマートスピーカー：Xiao AI（Mi Home エコシステム、コスパが高い） / HomePod mini（Apple エコシステム、Thread ハブも兼ねる） / Alexa（海外の対応が広い）。注意：スピーカーで音声エコシステムが決まります。まずエコシステムを決めてからスピーカーを買いましょう。
- スマート電球：Yeelight（Mi Home / Matter 対応、色温度と明るさを調節できて安い） / Philips Hue（色がきれいで安定、ただし Hue ブリッジが必要でやや高価）。注意：買う前にソケットの規格（E27 / B22）と、色調整の要否を確認しましょう。

- スマートプラグ：Xiaomi スマートプラグ（安価、電力計測つき） / TP-Link Kasa（海外向けの Matter 対応が良い）。注意：大電力の家電（給湯器、電気ヒーターなど）に使うときは、必ず定格電力を確認して過負荷を避けましょう。
- スマートロック：Lockin / Xiaomi（コスパ、Mi Home 連携） / Kaadas（錠前の伝統的な大手で、機械構造が堅牢）。注意：Matter や自分のエコシステムへの対応を確認し、電池のもちと緊急用キーホールの位置にも注目しましょう。

**購入の順番のおすすめ**

まずスマートスピーカー（エコシステムを決める）次にスマート電球とスマートプラグ（ハードルが低く、すぐ効果を実感できる）最後に玄関鍵とカメラ（安全に関わるので、慎重に選ぶ）。一度に追加するのは 1 種類までにして、慣れてから増やせば、「買ったのに使えない」を避けられます。

## セキュリティ上の注意点

スマートデバイスが多い家ほど、攻撃の入り口も広がります。次の 4 つは最低ラインです。1 つずつ照らし合わせてください：

1. 初期パスワードを変える：カメラ、ルーター、玄関鍵の出荷時パスワードは、ネット上ですべて公開されています。買ったらまず、長くて使い回しのない強いパスワードに変更しましょう。
2. ファームウェアをこまめに更新：メーカーが配るファームウェアには、セキュリティ修正が入っていることが多いもの。アプリで「自動更新」をオンにするか、少なくとも月 1 回は手動で確認しましょう。
3. ネットワーク分離：IoT デバイスは専用の Wi-Fi（またはゲストネットワーク）に置き、スマホやパソコンのメインネットワークとは分けます。万が一どれかのデバイスが突破されても、大事なデータへの横展開の脅威を及ぼしにくくなります。
4. プライバシー保護：カメラは「プライバシーゾーン」のマスキング（寝室や洗面所は撮らないなど）を有効にし、クラウドでの 24 時間録画はオフに。自宅カメラの映像を、サードパーティ製アプリに長期間見せっぱなしにしないこと。

### この落とし穴にはハマらないで

- 知らない人にスマートロックの「遠隔解錠」を使わせない——一時的な暗証番号を優先し、使い終わったら失効させましょう。
- カメラをベッドや窓の外に直接向けない——漏れるのは自分のプライバシーです。
- 安さにつられて無名の「ノーブランド」デバイスを買わない——ファームウェアの更新がほとんどなく、家に裏口を取り付けるようなものです。

**この章のまとめ**

- Matter ＋ Thread が 2023　2026 年の統一の流れ。新しく買うなら、まずこの表記をチェック
- 三大エコシステムにはそれぞれ得意分野がある：Apple はプライバシーと体験、Mi Home はコスパ、HUAWEI は複数デバイス連携
- スマホはハブになる。「帰宅 / 外出 / おやすみ」の 3 大シーンを組み上げれば、家が自分で動き出します
- 入門の 4 点セット：スマートスピーカー、電球、スマートプラグ、玄関鍵。この順番で少しずつ追加
- 初期パスワードの変更、ファームウェア更新、ネットワーク分離、プライバシーのマスキング——4 つの最低ラインは省けません

# 七、AI プログラミング実践編（第 37-47 章）

# 第三十七章：パラダイム革命——「コードを書く」から「AI にコードを書かせる」へ

おめでとうございます。この本を開いたあなたは、いま職業のパラダイム革命のど真ん中に立っています。この革命の起点は、*2025* 年に一人のプログラマーが *SNS* に投稿したひと言でした。少しドラマチックに聞こえますか？でも、本当にあったことです。

## Vibe Coding の誕生：ひと言が引き起こしたムーブメント

2025 年 2 月、Andrej Karpathy——この名前は今のうちに覚えておいてください。この先も何度も登場します——が、後になって繰り返し引用されることになった投稿を SNS に上げました：「Vibe Coding という新しいプログラミングのやり方がある。完全にフィーリング（vibes）で進めて、コードの細かいところは気にせず、AI に全部まかせる。見えたものをそのまま口にして、コピーアンドペースト、だいたい動けばそれでいい。」

とりとめのない独り言に見えるこの発言は、実はすでに静かに起きていたパラダイムシフトに、ぴったりの名前を与えたのです：人間はもうコードを 1 行ずつ書かず、自然言語で意図を記述し、AI が実装を完成させる。Karpathy の言い回しには少し揶揄の色がありましたが、その背後には深い洞察があります：AI のコード生成能力がある閾値を超えたとき、プログラミングのボトルネックは「この行のコードが書けるか」ではなく「この件を自分ははっきり考えられているか」に変わるのです。

**ヒント：Vibe とは何？**

Vibe を直訳すると「雰囲気、フィーリング」。Vibe Coding とは「感覚に従って AI にプログラミングを指揮させる」こと——あなたは欲しいものを記述し、AI が実装を担います。この言葉を覚えておいてください。この革命の「出生証明書」なのですから。

あの瞬間から、「コーディング」という言葉の意味が分かれ始めました：従来のコーディングは、設計を文法に翻訳すること。新しいコーディングは、意図を指示に翻訳することです。前者は指が試され、後者は頭が試されます。

Vibe Coding という言葉は、すぐに冗談めいたものから真面目なエンジニアリングの概

念へと変わっていきました。2026 年半ばまでに、SNS 上のネタから、要件の表現、コンテキスト管理、反復検証、品質保証までをカバーする全プロセスを備えた、完全な方法論の体系へと進化しています。人々は同時に気づきました：「感覚でやる」のは入門段階の状態にすぎず、本当に効率的な AI プログラミングには、体系化された思考の枠組みが必要なのだと。そしてその思考の枠組みこそ、本書があなたに差し出すものです。

## 2026 年の現在地：Agentic Engineering の時代

2026 年 8 月に至るまでに、AI プログラミングの分野は、いくつもの重要なパラダイムの更新をくぐり抜けてきました。Vibe Coding は最初の火種でしたが、ほどなく皆が気づきます：「感覚のまま」AI とおしゃべりしているだけでは、生まれてくるコードの品質が極めて不安定——ときには感動的にうまくいき、ときには大失敗する。そこでパラダイムは進化し始めました：単純な対話型プログラミングから、構造化された Agent エンジニアリングへ。

またしても Karpathy です。2026 年初頭、Agentic Engineering という概念を提唱しました。Vibe Coding の「言えば生成する」とは違って、Agentic Engineering が重視するのは：AI に目標を与え、ステップの計画、ツールの呼び出し、結果の検証、反復修正を自律的に行わせること。AI はもはや単なるコード生成器ではなく、環境を感じ取り、判断を下し、行動を実行できるインテリジェントなエージェント（Agent）なのです。

この転換は、3 つの重要な変化をもたらしました：

1. やり取りの形が変わった：「一問一答」から「目標を設定して権限をゆだねる」へ。

2. 仕事の成果物が変わった：「コードの一片」から「完成したプロジェクト一式」へ。

3. 人の役割が変わった：「コードを 1 行ずつレビューする」から「境界線を定め、結果を検収する」へ。

数字は嘘をつきません：Cursor は 2026 年 4 月にリリースされた Cursor 3 で、Agent モードの使用率が初めて Tab 補完モードを追い越しました。開発者が大規模にこの働き方へ移行した証拠です。また、ターミナルネイティブの AI プログラミングツール Claude Code は、2026 年半ばまでに年間換算で 25 億ドルの収入規模に達しました。次のトレンドグラフで、この交差の瞬間を直感的につかんでください：

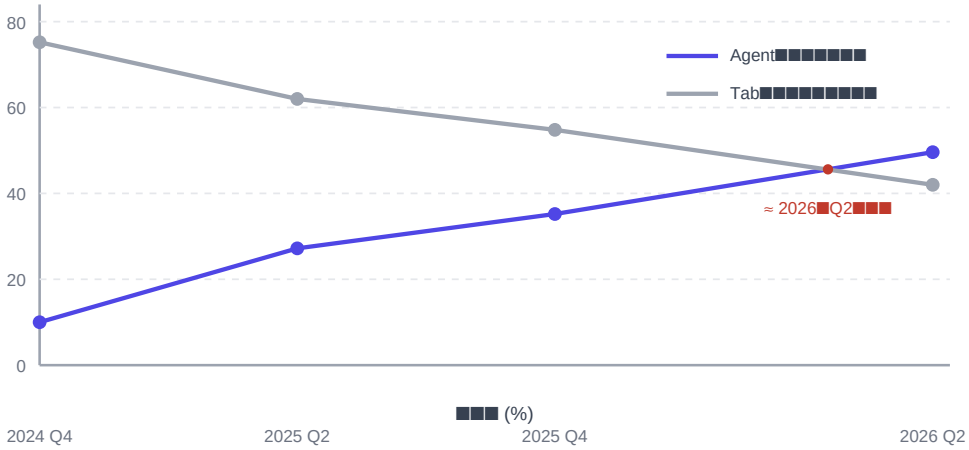



図 1：AI プログラミングの対話モード使用率の変化（業界調査データに基づく推定。2026 年 Q2 に Agent モードが Tab 補完を逆転）

でも正直に言いましょう：ツールとモデルの変化は、見えている表面にすぎません。Cursor を使おうが Claude Code を使おうが、土台のモデルが GPT-5 だろうと Claude 4.5 だろうと、成功している AI プログラミング実践者は皆、同じ一連の根本的な方法論に従っています。このガイドが教えるのは、まさにその部分です。

## 核心となる認識の転換：Coder から Conductor へ

従来のプログラミング教育は、Coder になる方法を教えます——文法を理解し、API を暗記し、ロジックの誤りを直す。AI プログラミングの時代には、この役割が再定義されつつあります。もうすべてのコードを自分の手で書く必要はありませんが、あなたは「指揮官」になる必要があります：戦場を定め、ルールをつくり、タスクを割り振り、成果を検収する Conductor です。

この転換の核心は「コードが書けるかどうか」ではなく、「何をしたいのかをはっきり考えられるかどうか」にあります。優れた指揮官は、必ずしも最良の兵士ではありません。しかし、戦いの目標・地形・兵力配置を誰よりもよく知っていなければなりません。同じように、優れた AI プログラミング実践者は、あらゆる言語の文法の細部に精通している必要はありません。しかし、プロジェクトの目標・アーキテクチャ上の制約・品質基準を誰よりもよく把握していなければなりません。

この表は 2 度見返す価値があります——今回の役割転換の全体像が、ここにまとまっているのです：

| 観点 | 従来の Coder | AI 指揮官（Conductor） |
|---|---|---|
| 中核スキル | 文法の暗記、API の呼び出し、デバッグ | 要件の定義、コンテキスト管理、検収の判断 |
| 働き方 | コードを 1 行ずつ書く | 意図を記述し、成果物を審査する |
| 品質管理 | 自分で書いたコードの中身は、自分が一番わかっている | テストとレビューで AI の成果物を検証する |
| 求められる知識 | 特定の言語 / フレームワークを深く習得する | 技術の全体像を広く、業務ロジックを深く理解する |
| ボトルネックはどこか | コーディングの速さと文法の暗記 | 思考の明確さと表現力 |
| エラーの原因 | 文法ミス、ロジックの抜け漏れ | 要件があいまい、文脈情報が足りない |

従来の Coder と AI 指揮官の能力の比較

これは決して、プログラミングの知識が重要でなくなったという意味ではありません。むしろ逆です：AI が書いたコードが読めるからこそ、効果的にレビューして修正できます。データ構造とアルゴリズムを理解しているからこそ、AI の提案が妥当かどうか判断できます。知識の蓄積はあなたの「指揮の上限」を決めます——ただし知識の役割は、「実行の道具」から「判断のよりどころ」へと変わったのです。

## 四、なぜ方法論がツールより大事なのか

ツールは 3 か月ごとに改良され、方法論は 3 年ごとに進化します。ツールの操作を覚えても 3 か月後には古くなるかもしれない。けれど一つの考え方を身につければ、3 年後もまだ使える。

AI プログラミングを始めたばかりの多くの人は、最初に「どのツールが一番良いか」「どのモデルが一番強いか」と問い立てます。これらの質問が大事でないわけではありませんが、優先順位を間違えています。2026 年 8 月のツールの情勢は、2025 年 8 月とはすでにまったく違います——Trae や Qoder などのツールは栄枯盛衰を繰り返し、Cursor はプラグインから独立した IDE になり、さらに Cursor 3 へ進化し、Claude Code は実験的なプロジェクトからターミナルプログラミングの主流の選択肢になりました。しかし、この激しい変化の裏側で、成功している実践者たちが従っている思考の原則は、ほとんど変わっていません。

だからこそ本書は、特定のツールの具体的な操作を教えることに紙幅を割きません——そうした内容は書き終えた瞬間に古くなるからです。その代わり、本書のすべての重心は、検証ずみの思考方法論というひとつの体系に置かれています：要件をどう考えるか、AI とどうやり取りするか、成果物をどう検証するか、どう反復し続けるか。この方法論を身につければ、どのツールに変わってもすぐ使いこなせます。ツールの操作だけを学んだ人は、ツールが変わるたびにゼロからやり直しです。

**ゼロベースの読者への安心材料**

ゼロから始める人は、「方法論」という三文字におびえないでください。方法論は抽象的な哲学ではなく、「問題に出会ったときどう考えるか」という操作マニュアルです。料理を学ぶのと同じです：レシピどおりに一歩一歩進めることもできます（ツールの操作）が、ほんとうの料理人は「なぜこの味付けなのか」「火加減をどう調整するのか」（方法論）を理解しています。本書はいちばん基本の概念から説明し、原則ごとに具体的な比較例を付けているので、「何か」がわかるだけでなく、「どうやるか」まで身につきます。

**やってみよう：いまの自分の「役割写真」を一枚撮ろう**

紙を一枚取り出して、三つの質問の答えを書きましょう：(1) いまは AI にどうやってコードを書かせているか（使ったことがあれば）？ (2) 自分は上の表の「Coder」に近いか、それとも「指揮官」に近いか？ (3) 三か月後の自分には、自分の働き方をどう表現してほしいか？ 書き終えたら本に挟んでおき、第四十七章を読み終えたらもう一度見返してください——変化は想像より速いと気づくはずです。

**本章のまとめ**

- 2025 年、Karpathy は AI プログラミング革命に「Vibe Coding」という名前を付けました：1 行ずつコードを書くやり方から、自然言語で意図を記述するやり方へ
- 2026 年には Agentic Engineering へ進化：目標を与え、権限をゆだね、AI が自律的に計画・実行し、人は成果の受け入れを担当
- Cursor 3 では Agent モードの利用率が初めて Tab 補完を追い越し、Claude Code の年間収益は 25 億米ドルに達しました
- あなたの新しい役割は Conductor（指揮官）：戦場を定め、ルールを決め、タスクを割り振り、成果を受け入れる
- ツールは 3 か月ごとに更新され、方法論は 3 年かけてしか進化しない——学ぶのは方法論であって、操作ではない

# 第三十八章：AI プログラミング五大思考原則

もしこの本で一章だけ覚えられるとしたら、それは必ずこの章でなければなりません。五大思考原則は本書全体の心臓部であり、このあとのすべての章——コンテキスト、要件、対話、品質——は、この五つの原則をさまざまな場面に展開したものです。ご安心ください。どれ一つ難しくありませんが、どれ一つも、いくつものプロジェクトを通して「筋肉に染み込ませる」値打ちがあります。

物語の出発点がおもしろいのです。Andrej Karpathy は自分のプロジェクトをオープンソースとして公開したとき、わずか 70 行の CLAUDE.md ファイルを添付しました。そこには AI と協働するための四つの中核原則がまとめられていました。このファイルは GitHub で 70,000 以上の Star を獲得し、世界中の AI プログラミング実践者に最も広く引用される指針となりました。この四原則は空虚なスローガンではなく、膨大な実践から磨き出された、実行できる思考の習慣です。本書はその土台の上に五つ目の原則——反復検証——を加え、完全な五大原則の体系にしました。

**70 行のファイルはなぜそれほど価値があるのか？**

70,000 Star はどれほどすごいことか？GitHub のほとんどのプロジェクトは、一生かけてもこの数字に届きません。70 行のテキストファイルがそれを獲得できたということは、ただ一つの事実を証明しています。AI 時代にいちばん希少なのはコードではなく、「考え抜く」能力だということです。この 70 行こそ、「考え抜く」の凝縮版なのです。

## 一、原則一：Think Before Coding（先に考え、それからコードを書く）

これは五つの原則の中で最も直感に反するものです。AI がコードを書いてくれるのに、どうして「先に考える」必要があるのでしょう？答えはシンプルです：AI の出力の質は、あなたの入力の質に比例します。自分でも何をしたいのかはっきり考えていなければ、AI の答えは「直してはまた直す」の果てしないループにあなたを落とし入れるだけ——いわゆる「AI と一緒に道に迷う」状態です。

「先に考える」とは、AI への指示を出す前に、次の三つの質問に答えておくことです。この三つで、一つも欠かせない「思考のトライアングル」ができます：

1. 何の問題を解決するのか？（目標）
2. 期待する結果はどんな形か？（受け入れ基準）
3. 守るべき制約条件は何か？（境界）

| 思考の切り口 | あいまいな入力（悪い例） | 明確な入力（良い例） |
|---|---|---|
| 問題の定義 | ウェブサイトを作って | AI ミニツール 3 つの入口カードを並べた、個人用ツールサイトを作って |
| 期待する結果 | ちょっとオシャレに | レスポンシブ対応、ダークテーマ、カードにはホバーのアニメーション、スマホでも使えること |
| 制約条件 | （未指定） | フレームワークは使わない。素の HTML+CSS+JS、1 ファイルでデプロイ、バックエンドは不要 |

同じ要件の、あいまいな入力と明確な入力の比較

**注意：先に考える ≠ 完璧な文書を先に書くこと**

「先に考える」は「完璧な PRD を書いてから始める」という意味ではありません。短い時間で、しかし筋道立てて考えるプロセスです——上の三問を 30 秒で整理するほうが、30 分かけて会話の中で何度もやり直すよりずっと効率的です。小さなタスクなら三問は短くてよく、大きなプロジェクトなら本格的な要件文書に発展させればよいのです（詳しくは第四十章）。

## 二、原則二：Simplicity First（シンプル・ファースト）

AI プログラミング最大の落とし穴は「作れないこと」ではなく「作りすぎること」です。AI に「ユーザー管理システムを作って」と頼むと、認証、権限、ロール、監査ログ、データエクスポートといった機能一式を生成してくるかもしれません——でも今のあなたに必要なのは、シンプルなログインページだけかもしれないのです。この「作りすぎ」こそ、AI コードがふくれ上がる主な原因です。

「シンプル・ファースト」では、指示を出すたびに「最小限で実現する」原則に従うことが求められます：いちばん少ないコードで中核機能を作り、いちばんシンプルな方法で問題を解き、いちばん少ない機能でアイデアを確かめる。Karpathy はこれをこう言い表しています：「割引価格を計算したいだけなら、EC エンジンは必要ない」。

実践では、「シンプル・ファースト」は次のような具体的な操作の習慣を意味します：

- AI には一度にひとつの機能だけを実装させる。システム全体を一気に説明しない。

- 複雑な依存関係を追加するより、標準ライブラリとシンプルな方法を優先する。
- 50 行で済む問題を、AI に 500 行書かせない。
- AI の出力が期待の範囲を超えたら、思いきって「余計な部分は削って」と伝える。

| 場面 | 作りすぎの案（避けるべき） | シンプルな案（おすすめ） |
|---|---|---|
| データを保存したい | MySQL データベース＋ORM を構築する | まずは JSON ファイルか SQLite を使う |
| ユーザーのログインが必要 | OAuth2.0 ＋ JWT ＋ロール権限 | まずはシンプルなパスワード認証から |
| フロントエンドのフレームワークが必要 | React + Redux + Router | まずは素の HTML ＋ネイティブ JS を試す |
| デプロイして公開したい | Docker + CI/CD + K8s | まずはローカルで動くことを確認してから |

よくある場面での「作りすぎの案」と「シンプルな案」の比較

## 三、原則三：Surgical Changes（ピンポイント修正）

すでにあるプロジェクトを手入れしていくときは、毎回の修正を外科手術のように精密に——直すべきところだけを直し、触れてはいけないところには触れない。これは AI プログラミングではとりわけ大事です。AI には「ついでに直しておこう」という癖があるからです：バグをひとつ直してと頼んだのに、ファイルの半分をリファクタリングして返してくることがあります。

「ピンポイント修正」でいちばん大事なのは、AI に出す指示ごとに、直す範囲と境界をはっきり指定することです。「calculate_discount 関数だけを直すこと」「ほかの部分には触れないこと」「今のコードのスタイルを保つこと」と伝えます。AI の修正が頼んだ範囲を超えたら、すぐに余計な部分を元に戻すよう求めましょう。

Karpathy は diff（差分）の例でこれを説明しています。買い物カゴの割引計算関数に「会員価格」のロジックを追加させるとしましょう。良い出力は、この関数ひとつだけを直して数行を足すもの。悪い出力は、ファイル全体を作り直してしまうもの——新しいコードのほうが「エレガント」に見えても、新しいバグを持ち込んでいないかは保証できません。AI プログラミングでは、「見た目が良くなった」ことは、指示から外れてよい理由には決してなりません。

悪い指示（範囲があいまい）

```
このプロジェクトの 割引計算ロジックを改善して
```

良い指示（範囲をピンポイント指定）

```
cart.pyのcalculate\_discount関数だけを修正： 1. 既存の割引に会員価格のロジックを追加する 2. 会員ユーザーはさらに10\%の割引 3. 関数シグネチャと他の関数は修正しない 4. 既存の変数命名のスタイルを守る
```

## 四、原則四：Goal-Driven Execution（目標駆動実行）

この原則は、大事な問いに答えてくれます：AI には「どうやるか」の指示を出すべきか、それとも「何をするか」の指示を出すべきか？ Karpathy の答えは後者です。「まずクラスを作って、次にメソッドを足して、テストを書いて」と伝えるより、「CSV ファイルを読み込んで、各列のユニークな値の数を数えるツールが欲しい」と伝えるほうがうまくいきます。実装の道筋は AI に任せ、あなたは成果を受け入れる側に回ります。

「目標駆動」のメリットは二つあります。AI にとっては、より広い裁量の余地が生まれ、あなたの途中の手順に縛られず最適な実装ルートを選べます。あなたにとっては、意識を「どうやるか」から「できあがりが正しいか」へ移せる——これこそ指揮官のやるべきことです。

**注意：目標駆動 ≠ すべて丸投げすること**

AI が実行を終えたあとは、厳しく受け入れ確認をすることが前提です：結果は要件を満たしているか？ 境界のケースは処理されているか？ 明白な作りすぎはないか？ 期待と違っていたら、具体的なフィードバックを返します——「書き直して」ではなく、「ここは空のリストを入力すると IndexError が出ます。空チェックを足してください」と伝えるのです。

| 指示のスタイル | 例 | 結果の特徴 |
|---|---|---|
| 命令型（どうやるか） | 「まず DataParser という名の Python クラスを作り、parse メソッドをひとつ追加して、filename 引数を受け取り、csv モジュールで読み込んで…」 | AI はただのタイピストになる。あなたの途中の手順が最適ルートとは限らない |
| 目標型（何をするか） | 「CSV ファイルを読み込み、各列のユニークな値の数を数え、JSON 形式で出力するツールが欲しい。完全な実装を示してください。」 | AI が自ら最適なルートを選び、あなたは成果を受け入れる |

命令型の指示と目標型の指示の比較

## 五、原則五：Iterate and Verify（反復検証）

これは本書が Karpathy の四原則に加えた五つ目の原則です。中核にある信念はこうです：AI の最初の出力は、ほとんどの場合、最終版ではない。効果的な AI プログラミングとは「生成—検証—修正」の反復ループであって、一回きりの「指示—出力」ではない、と。

「反復検証」は受動的な「ダメならもう一回」ではなく、構造化された検証のプロセスです。AI が出力を返してくるたびに、三つの層で確かめます：

- 第一層・機能検証：動くか？入出力は正しいか？
- 第二層・境界検証：空の入力、長すぎる入力、変な入力ではどうなるか？
- 第三層・アーキテクチャ検証：コードの構造は妥当か？ハードコーディングはないか？あとでの拡張はしやすいか？

目標を定める
AI がコードを生成
機能の検証
合格
境界値検証
合格
アーキテクチャレビュー
合格、次のステップへ
不合格
具体的なフィードバックを与える
再生成

図 2：AI プログラミングの反復検証ループ

このループでカギになるのが「具体的なフィードバックを返す」という一手です。検証が通らなかったときは、「違う、やり直して」ではなく、「この関数は空のリストを入力すると IndexError が出ます。15 行目に空チェックを足してください」と伝えます。具体的なフィードバックなら AI はピンポイントで直せます。あいまいなフィードバックは、あなたの意図を推測し直させるだけです。

## 六、Harness Engineering：Agent = Model + Harness

2026 年の半ばごろから、Harness Engineering という概念が AI プログラミングのコミュニティで広まり始めました。その中核となる公式はとてもシンプルです：

Agent の能力 = モデル能力 × Harness の質

モデルの能力は、AI という脳の知能の上限です。Harness はモデルを取り巻くように作られた「作業環境」——コンテキスト管理、ツール統合、処理の流れの設計、記憶システムなどを指します。

この概念は、よくある疑問を説明してくれます：同じ GPT-4 や Claude を使っていても、なぜ驚くようなプロジェクトを作れる人と、平凡なコードしか得られない人がいるのか？違いはモデルではなく Harness にあります。良い Harness には、明確なプロジェクト仕様のファイル、妥当なツール統合、整理された対話コンテキスト、行き届いた検証の流れが含まれます。モデルは知能の上限を決め、Harness はその上限をどこまで発揮できるかを決めるのです。

**天才インターン生のたとえ**

モデルを天才インターン生だと思ってください——知能はとても高いけれど、プロジェクトの背景資料を渡さず、どのツールを使うかも教えず、期限と受け入れ基準も設定しなければ、良いものは作れません。Harness Engineering とは、「天才インターン生のために作業環境をちゃんと整えてあげる」ためのエンジニアリングなのです。

本書のこのあとの章は、本質的にはすべて、良い Harness の作り方を教えています：第三十九章はコンテキスト管理（プロジェクト記憶システム）、第四十章は要件定義（PRD）、第四十三章は対話技術（プロンプトの階層化）、第四十五章は品質保証（検証の流れ）。これらを組み合わせたものが、あなたの Harness です。

**やってみよう：「思考のトライアングル」で次のアイデアを分解しよう**

ずっと作りたかったのに手を出せずにいる小さなもの（例：カウントダウンの Web ページ、水分補給リマインダーのスクリプト）をひとつ選びます。3 分かけて答えましょう：(1) どんな問題を解決するもの？ (2) できあがりはどんな形？ (3) どんな制約（時間、技術、環境）がある？ そして答えをひとつの段落にまとめます。おめでとうございます。あなたは人生で初めて「Think Before Coding」を完了しました——この段落こそ、合格点の高品質な指示です。

**本章のまとめ**

- 五大原則は Karpathy の 70 行の CLAUDE.md（Star 70,000 超）に由来し、本書が五つ目の「反復検証」を補足
- Think Before Coding：口を開く前に三問に答える——何をする？ 結果はどんな形？ どんな制約がある？
- Simplicity First：AI 最大の落とし穴は「作りすぎ」——割引の計算に EC エンジンは不要
- Surgical Changes：外科手術のようにピンポイントで直し、AI の「ついでに直す」を拒否する
- Goal-Driven Execution：「何をするか」を伝え、「どうやるか」は伝えず、AI に自ら道筋を選ばせる
- Iterate and Verify：三層の検証（機能 / 境界 / アーキテクチャ）＋具体的なフィードバックで反復する
- Harness Engineering：能力＝モデル × Harness。AI を「天才インターン生」とみなし、環境を整えて管理する

# 第三十九章：コンテキストエンジニアリング——AI にあなたの意図をわからせる

こんな経験はありませんか：新しい会話ウィンドウに切り替えると、*AI* は記憶喪失になったように、プロジェクトの基本情報をすっかり忘れてしまう。これは *AI* がわざと困らせているのではなく、生まれつき「忘れっぽい」のです。この章では、*AI* に「長期記憶システム」を取り付ける方法を教えます。あわせて、*2026* 年で最も注目されている開発パラダイムの一つ——仕様駆動開発——も紹介します。

## 一、プロジェクト記憶ファイル：AI の「長期記憶」

AI モデルには根本的な制限があります：記憶が「短期的」なのです。会話が始まるたびに、AI はあなたのプロジェクトについてゼロからスタートします。だから会話の中で「使っているのは React」「バックエンドは Node.js」「データベースは PostgreSQL」「コードのスタイルはダブルクォート」と、何度も伝え続けなければなりません。これは非効率なうえ、抜けも出やすくなります——毎日、入ってきたばかりの同僚に入社研修をするようなものです。

解決策が「プロジェクト記憶ファイル」です——プロジェクトのルートディレクトリに置くプレーンテキストのファイルで、AI ツールは会話の開始時に自動で読み込みます。ツールごとに、名前の付け方の慣習が違います：

| ツール | ファイル名 |
|---|---|
| Cursor | `.cursorrules` |
| Claude Code | `CLAUDE.md` |
| GitHub Copilot | `copilot-instructions.md` |
| 汎用 | `AGENTS.md` |

各ツールのプロジェクト記憶ファイルの命名慣習

名前は違っても原理は同じです：AI に「いつも覚えていてほしい」情報をこのファイルに書き込むのです。良いプロジェクト記憶ファイルには次を含めるべきです：プロジェクト概要（これは何のプロジェクトか）、技術スタック（どんな技術を使っているか）、コード規約

（命名、インデント、コメントのスタイル）、ディレクトリ構造（コードがどう整理されているか）、開発ルール（ブランチ戦略、コミット規約、テスト要件）。Karpathy の CLAUDE.md はわずか 70 行ですが、大事な情報をすべてカバーしています——簡潔でありながら完璧なのです。

```
# プロジェクト：個人家計簿アプリ
## 概要
日常の収支を記録するのを助ける、フルスタックの個人家計簿アプリです。
## 技術スタック
- フロントエンド：React + TypeScript + Tailwind CSS
- バックエンド：Node.js + Express + TypeScript
- データベース：SQLite（開発）/ PostgreSQL（本番）
- テスト：Vitest
## コーディング規約
- ダブルクォートを使う
- インデントはスペース2つ
- 関数名はcamelCase
- 型定義はPascalCase
- すべての関数にJSDocコメントを付ける
## ディレクトリ構成
/src
  /frontend  - フロントエンドのコード
  /backend   - バックエンドのコード
  /shared    - 前後で共有する型
  /tests     - テストファイル
## 開発の約束事
- コミットメッセージは中国語で
- 新機能は必ずテストを先に書く
- APIの応答は { success, data, error } 形式で統一
```

**注意：「二枚目の要件文書」にしないこと**

このファイルの長さに注意してください——何十ページもの要件文書にはしないこと。プロジェクト記憶ファイルの目的は「AI にすばやく背景の認識を持たせる」ことであって、「各機能をくわしく記述する」ことではありません。簡潔に、構造化して、「AI が知っておくべきこと」だけを書きましょう。

## 二、仕様駆動開発（SDD）

Spec-Driven Development（SDD、仕様駆動開発）は 2026 年に成熟してきた開発パラダイムです。核心理念はひと言：先に仕様を書き、それからコードを書く。従来の TDD（テスト駆動開発）と違うのは、SDD の「仕様」にはテストケースだけでなく、要件の記述、インターフェースの定義、データモデルなども含まれることです。AI はこれらの仕様に基づいてコードを生成し、人間の役割は仕様を書き、維持することです。

SDD のワークフローは五つのステップです：

1. 自然言語で要件の仕様を書く（.md ファイルに記述する）。

2. インターフェースとデータ構造を決める（仕様ファイルに書く）。

3. AI に、仕様に沿ったコードを実装させる。

4. テストを動かして、成果物が仕様どおりか確かめる。

5. ずれていたら、仕様を直すか、AI にコードを直させる。

この流れの中で、仕様ファイルは「唯一の真実の源」です——コードは作り直せますが、仕様が先に正しくなければなりません。

仕様ファイルを書く
仕様は完全ですか?
完全
AI が仕様からコードを生成
テストを実行して成果を検証
仕様不完全 / テスト不合格
コードの問題コードを修正。仕様の問題仕様を修正
検証に戻る
全部合格納品

図 3：仕様駆動開発（SDD）のワークフロー

SDD の強みは、仕様ファイルがコードより安定していることです。土台のモデルやツールが変わっても、仕様ファイルは依然として有効で、新しい AI ツールにコードを作り直させるだけで済みます。だからこそ本書は「方法論はツールより大事」と強調するのです——仕様ファイルは、あなたの方法論の具体的な担い手なのです。

## 三、コンテキストウィンドウ管理

プロジェクト記憶ファイルがあっても、会話の中で「コンテキストが足りない」という問題には出会います。AI モデルのコンテキストウィンドウ（同時に処理できる情報の量）には限りがあります。会話が長くなりすぎたり、参照するファイルが多すぎたりすると、AI は序盤の情報を「忘れ」て、成果物の質が落ちます——友人と 6 時間もおしゃべりしていたら、冒頭で話した最初の話題なんて忘れているのと同じです。

コンテキストウィンドウ管理の中核戦略は「層わけした参照」です：

| コンテキストの層 | 内容 | 管理方法 |
|---|---|---|
| 第一層：プロジェクト記憶 | 技術スタック、規約、ディレクトリ構造 | CLAUDE.md などのファイルに書き、自動で読み込まれる。繰り返し不要 |
| 第二層：ファイル参照 | いまのタスクに関わるコードファイル | @ 記号で正確に参照する。全文を会話に貼らない |
| 第三層：対話コンテキスト | いまのタスクの会話履歴 | ひとつのタスクが終わったら新しい会話を開く。コンテキストのふくらみを防ぐ |
| 第四層：その場の指示 | いまこの手順での具体的な要求 | 明確・簡潔・的確に。くどい説明はしない |

コンテキストウィンドウの層別管理戦略

## 四、コンテキストエンジニアリングの実践ポイント

コンテキストエンジニアリングは一回きりの設定ではなく、続けて最適化していくプロセスです。実践で繰り返し効いてきたポイントを紹介します：

1. 記憶ファイルはプロジェクトの進化に合わせて随時更新する。新しい技術を取り入れたりアーキテクチャを変えたりしたら、真っ先に記憶ファイルを更新します。会話の中でその場で説明するのは避けましょう。

2. @ 参照を活用し、全文の貼り付けはしない。ほとんどの AI プログラミングツールは @file や @folder という書き方に対応していて、裏側でファイルの中身を AI のコンテキストに注入してくれます。手で貼り付けるより効率的で、会話を汚しません。

3. 会話は「短く、濃く」を心がける。ひとつの会話では、ひとつのタスクかひとつの機能だけを扱います。終わったら新しい会話で次へ進みます。長い会話はコンテキストウィンドウを消耗するだけでなく、AI を「ぼんやり」させます。

4. 結果を「説明」するのではなく、AI に結果を「見せる」。バグに出会ったら、「なんかエラーが出たみたい」と自分の言葉で説明するのではなく、エラーログをそのまま @ 参照で AI に渡しましょう。AI はあなたの伝え聞きより、生の情報を見るほうがずっと効率的です。

5. 定期的に掃除とアーカイブをする。終わった会話や一時ファイルは作業スペースを占領します。無関係なものを定期的に片付けてプロジェクトのディレクトリをきれいに保つと、AI がプロジェクトの構造をより効率的に理解するのにも役立ちます。

**やってみよう：あなたのプロジェクトに 70 行の記憶ファイルを書こう**

いま作っている（または作ろうと思っている）プロジェクトをひとつ選び、CLAUDE.md を書いてみましょう：プロジェクト概要（2　3 文）、技術スタック（書き出す）、コード

規約（3　5 項目）、ディレクトリ構造（ツリーで描く）、開発ルール（2　3 項目）。70 行以内に収めます——書き終えるころには、プロジェクトに対する自分の理解までずいぶんはっきりしていることに気づくはずです。

本章のまとめ

- AI の記憶は「短期的」：新しい会話のたびに、プロジェクトの理解はゼロから
- プロジェクト記憶ファイル（CLAUDE.md など）は AI の「長期記憶」。会話のたびに自動で読み込まれる
- 良い記憶ファイル＝概要＋技術スタック＋規約＋ディレクトリ＋開発ルール。70 行で十分
- SDD（仕様駆動開発）：先に仕様を書き、それからコードを書く。仕様は「唯一の真実の源」で、コードより安定している
- コンテキストは四つの層で管理：プロジェクト記憶 @ ファイル参照会話履歴その場の指示
- 五つの実践ポイント：随時更新、@ 参照、短い会話、生の情報を渡す、定期的な整理

# 第四十章：要件エンジニアリング——アイデアを AI が実行できる指示に変える

コンテキストエンジニアリングが「*AI* に背景をわからせる」ことだとすれば、要件エンジニアリングが解決するのは「*AI* に、今回は具体的に何をするのか知らせる」ことです。これは *AI* プログラミングの中で、人と人の差がいちばん表れるプロセスです——同じツール、同じモデルでも、驚くようなプロジェクトを作れる人もいれば、平凡なおもちゃしか作れない人もいます。その違いは多くの場合、技術力ではなく、要件を表現する力にあります。

## 一、あいまいなアイデアから、構造化された要件へ

どのプロジェクトも、あいまいなアイデアから始まります——「家計簿アプリを作りたい」「個人サイトを作りたい」「ToDo リストのツールを作りたい」。アイデアそのものは問題ありませんが、それをそのまま AI に投げると、成果物の質は完全に運まかせになります。要件エンジニアリングの目標は、このあいまいなアイデアを、AI が正確に実行できる構造化された要件に変えることです。

良いニュースがあります：構造化された要件のために、何十ページもの PRD を書く必要はないのです。五つの質問に答えるだけで十分。私たちはこれを「要件五要素」と呼びます：

| 要素 | 質問 | 例：家計簿アプリ |
| --- | --- | --- |
| Who | だれが使う？ | 個人用。毎日の収支を記録する |
| What | 何を作る？ | 1 件ごとの収支を記録し、月ごとにまとめて、可視化できるアプリ |
| Why | なぜ作る？ | 自分のお金の使い方を把握し、ムダな出費を見つけたい |
| How | ざっくりどう作る？ | フロントエンドは React、データはブラウザのローカルに保存。バックエンドとログインはいったん作らない |
| Constraint | どんな制限がある？ | ひとりでの開発、2 週間以内に完成、公開・デプロイはしない |

要件五要素とその例

この五つの要素を正式な文書にする必要はありません。頭の中でさっと一巡するだけでも十分です。大事なのは、「考えてから口を開く」習慣をつけることです。五つの質問にはっきり答えられるようになると、要件の説明が自然と構造化されていることに気づくはずです。

## 二、要件を小さく分解する技法

大きめのプロジェクトでは、ひとつの要件の記述では足りません。大きな要件を、小さく、独立して確かめられるサブ要件に分解する必要があります。このプロセスを「要件分解」と呼びます。中核原則は：どのサブ要件も、独立に完成でき、独立に検証でき、独立に納品できる、ということです。

「個人用の家計簿アプリを作る」を例にすると、合理的な分解はこうなります：

1. 基本的な収支記録機能を作る（記録の追加・閲覧・削除ができる）。
2. カテゴリー機能を足す（各記録に食費、交通費などのタグを付ける）。
3. 月ごとの集計機能を作る（月単位で収支をまとめる）。
4. データの可視化を足す（グラフでお金の使い方を見せる）。

どのステップも、それ単体で動かして確かめられる機能のひとかたまりです。

要件分解の黄金律：各ステップが終わるごとに、プロジェクトは「使えるもの」になっているべき——機能が粗末でも。永遠に「全部中途半端で、何ひとつ使えない」状態で止めてはいけません。

この分解のしかたには、二つの良さがあります。第一に、進み具合が常に見えること——一歩終えるたびにアプリに機能がひとつ増え、このプラスのフィードバックは長期のプロジェクトにとって非常に大事です（人間は達成感が必要ですからね）。第二に、AI へのひとつひとつのタスクが十分小さくなり、「AI の理解のずれ」のリスクを下げます——タスクが小さいほど AI の理解は正確になり、成果物の質も上がります。

## 三、PRD 作成のテンプレートと実践

大きめのプロジェクト（フロントエンドとバックエンド、複数の機能モジュールを持つフルスタックアプリなど）では、要件五要素だけでは足りないことがあります。そのときは、より正式なプロジェクト要件文書（PRD）を書きます。誤解しやすいのはここです：PRD は AI のために書くものではありません——自分自身のために書くものです。自分の考えを整理できてこそ、AI にさらに正確な指示を出せるようになります。

良い PRD は長くなくて構いませんが、六つの部分を含めるべきです：プロジェクト概要（ひとことで何を作るかはっきり）、機能リスト（各機能点の中身）、技術方式（どんな技術スタックか、おおまかなアーキテクチャ）、データモデル（中核のデータ構造）、画面の記述（主要なページの見た目）、開発計画（何ステップで完成させるか）。

```
# 個人家計簿アプリ PRD
## プロジェクト概要
軽量な個人家計簿アプリ。収支記録・カテゴリー・月次集計・データの可視化に対応。
## 機能一覧
1. 収支の記録：金額、種別（収入/支出）、カテゴリー、メモ、日付
2. カテゴリー管理：食費・交通・買い物・娯楽・その他をプリセット
3. 月次集計：月ごとの総収入・総支出・残高
4. 支出の可視化：円グラフでカテゴリー別の割合を表示
5. データ書き出し：CSVで出力
## 技術方案
- フロントエンド：React + TypeScript + Tailwind CSS
- データ保存：localStorage（バックエンド不要）
- グラフ：Chart.jsを使用
- ビルド：Vite
## データモデル
Record {
  id: string
  amount: number
  type: 'income' | 'expense'
  category: string
  note: string
  date: string  // YYYY-MM-DD
}
## 開発プラン
Step 1: 基本の記録機能（CRUD）
Step 2: カテゴリー管理
Step 3: 月次集計
```

```
Step 4: データの可視化
Step 5: データ書き出し
```

この PRD の書き方のスタイルに注目してください：長文ではなく、簡潔なリストと構造化されたフォーマットを使います。各機能点は明確に定義し、データモデルはシンプルな型で記述します。このスタイルの PRD は自分で読んでもわかりやすく、AI に渡したときも、AI が各要件を正確に理解できます。

## 四、実践：アイデアから具体案までを完全に一巡してみる

机上の空論ではおもしろくないので、ひとつの完全な例で要件エンジニアリングの全プロセスをなぞってみましょう。あなたのアイデアはこうです：「毎日何を学んだかを記録できる、学習トラッカーが欲しい」。

第一ステップ：要件五要素を一巡します。

- Who：個人使用。
- What：毎日の学習内容を記録し、学習時間を追跡し、学習の進み具合をふりかえられるツール。
- Why：続けて学ぶ習慣を身につけ、自分の積み重ねを見たい。
- How：まず Web 版を作り、データはブラウザのローカルに保存する。
- Constraint：ゼロベースでの開発、1 週間以内にコア機能を完成させる。

第二ステップ：要件を分解します。

1. Step 1：「学習項目の記録」機能を作る（日付、内容、時間）。
2. Step 2：「学習時間の集計」を追加する（今日、今週の累計）。
3. Step 3：「学習カレンダー」の表示を作る（GitHub のコントリビューショングラフのように見せる）。
4. Step 4：「学習のふりかえり」機能を追加する（過去のある日の記録をランダムに見せて、復習を助ける）。

第三ステップ：短い PRD を書きます。長くはいりません——上の分解そのものが PRD の本体になります。それを.md ファイルにまとめてプロジェクトのルートに置き、あなたと AI の共通の参照資料にしましょう。

第四ステップ：最初の開発を始めます。AI への指示はこうなります：

```
学習トラッカーの最初の機能——学習記録の登録が必要です。
ユーザーは日付、学習内容、学習時間（分）を含む学習記録を1件追加できます。
データはlocalStorageに保存。
```

```
フロントエンドはReact + TypeScript。
完全な実装を示してください。
```

この指示の特徴に注目してください：いまこのステップの目標（記録機能）だけを書き、ほかの機能を混ぜていない。技術の制約（React、TypeScript、localStorage）をはっきりさせている。AI に「何をするか」「どうやるか」がわかる十分な情報を与えながら、具体的な実装は AI に任せている。これが、要件エンジニアリングを経た、質の高い AI プログラミングの指示です。

**やってみよう：お気に入りのアイデアに「五要素カード」を書こう**

ずっと温めてきた小さなプロジェクトをひとつ選び、5 分かけてカードを一枚埋めましょう：Who / What / Why / How / Constraint、それぞれ一文ずつ。そして「どのステップが終わっても使える状態」になる 3 5 のステップに分解します。書いてみるとわかります：アイデアを書き出した瞬間、それは「白昼夢」から「ToDo リスト」に変わるのです。

**本章のまとめ**

- 要件五要素：Who / What / Why / How / Constraint——考えてから口を開く
- 要件分解の黄金律：どのステップが終わっても、プロジェクトは「使えるもの」になっている
- PRD は自分のために書くもの：概要、機能リスト、技術方式、データモデル、画面、計画
- 良い指示＝いまのステップだけを書く＋制約をはっきりさせる＋「何をするか」を言い「どうやるか」は言わない
- 完全シミュレーションの型：五要素分解 PRD 指示、の四ステップ

# 第四十一章：2026 年 AI プログラミングツールの全景

まず予防線を一本：この章は *2026* 年 *8* 月時点のツールエコシステムの「スナップショット」です。半年後には古くなります——実はこれこそ、この章がいちばん最初に教えたい教訓です。この章の目的は「どのツールを使うか」を教えることではなく、「ツールをどう選ぶか」の判断フレームワークを身につけてもらうことです。さまざまなツールの設計哲学が理解できれば、いつでも自分に合った選択ができるようになります。

## 一、三大流派と選択のフレームワーク

2026 年の AI プログラミングツールは、大きく三つの流派に分かれています。それぞれの流派の背後には、異なる設計哲学があります。まずこの全体図を見てください：

| 流派 | 代表ツール | 設計哲学 | 向いている場面 |
|---|---|---|---|
| IDE 内蔵 AI | Cursor 3, Trae, Windsurf, CodeBuddy | AI の能力をエディタに深く組み込む | 日常の開発、コードレビュー、じわじわ拡張していく使い方 |
| ターミナル Agent | Claude Code, Codex CLI, Aider | ターミナルをインターフェースとし、AI がファイルシステムとコマンドを直接操作する | プロジェクト単位の操作、一括修正、CI/CD との統合 |
| クラウド Builder | v0, Bolt, Replit Agent | ブラウザの中で、ゼロから完全なプロジェクトを作る | 高速プロトタイプ、設定なしですぐ始められる、即時プレビュー |

AI プログラミングツールの三大流派

ツール選びのカギは「どれが一番良いか」ではなく「いまの場面にどれが一番合うか」です。実用的な判断軸が三つあります：

- プログラミングの基礎はどのくらい？ゼロベースならクラウド Builder、基礎があれば IDE 内蔵 AI、上級ユーザーならターミナル Agent。

- プロジェクトはどの段階? ゼロから始めるならクラウド Builder、反復開発なら IDE 内蔵 AI、保守やリファクタリングならターミナル Agent。

- デプロイのニーズは何? すばやく確かめるならクラウド Builder、本格的なプロジェクトなら IDE 内蔵 AI、自動化の流れならターミナル Agent。

## 二、IDE 内蔵 AI 流派

IDE 内蔵 AI は最もメジャーな流派です：AI の能力をコードエディタに組み込み、従来のプログラミング体験を保ちながら AI による拡張を重ねていきます。2026 年のこの流派の代表は、Cursor 3、Trae、Windsurf、CodeBuddy です。

### Cursor 3

Cursor は現在世界中で最も広く使われている AI プログラミング IDE です。2026 年 4 月にリリースされた Cursor 3 は、AI プログラミングツールの重要な転換点となりました：Agent モードの利用率が初めて Tab 補完モードを上回ったのです——開発者は「AI にコードを補完してもらう」から「AI にタスクを任せる」へと移りました。Cursor 3 の中核的な機能には、複数ファイル編集（タスクのために AI が複数のファイルを同時に修正できる）、コンテキスト認識（プロジェクトの構造と依存関係を自動で理解する）、ターミナルとの深い統合（エディタの中で直接コマンドを実行し、出力を確認できる）があります。

### Trae

Trae は ByteDance が送り出した AI プログラミング IDE で、中国市場で大きなユーザーベースを持っています。その SOLO モード（Builder モードとも呼ばれます）は典型的な「ひと言でプロジェクトを作る」能力——要件を記述すれば、AI がプロジェクトの初期化からコード生成までの全流れを自律的に進めます。Trae の強みは中国語対応と中国国内のネット環境への適応で、中国のユーザーがすばやく始めるのに向いています。

### その他のツール

Windsurf（旧 Codeium）は滑らかなコード補完の体験で知られ、「打鍵がとまるほどのストレスなく飛ぶように打つ」体験を求める開発者に向いています。テンセントクラウドの CodeBuddy は VS Code プラグインとしてクラウドエコシステムとの統合に長け、テンセントクラウドへのデプロイや WeChat ミニプログラム開発を予定しているプロジェクトに適しています。アリババの通義霊碼（Tongyi Lingma）や百度の Comate も同様の IDE 統合体験を提供しており、それぞれ中国語の場面向けの最適化に特徴があります。

## 三、ターミナル Agent 流派

ターミナル Agent は 2026 年に最も速く成長している流派です。核心理念：AI はエディタのプラグインであるだけでなく、開発者と同じように開発環境全体を操作できるべき——ファイルの読み書き、コマンドの実行、出力の確認、依存関係のインストール。その代表が Claude Code です。

### Claude Code

Claude Code は Anthropic が送り出したターミナル AI プログラミングツールで、2026 年半ばには年間換算収益 25 億米ドルの規模に達しています。その働き方は従来の IDE とはまったく違います：ターミナルで自然言語でタスクを記述すると、Claude Code が自律的に関係するファイルを読み、コードを直し、テストを実行し、結果を確認し、反復して修正を重ねます——一連の流れの中で、エディタを開く必要はありません。

Claude Code の強みは「プロジェクト全体を見渡せる」こと：エディタのウィンドウに制限されず、プロジェクトのディレクトリ全体を自由に見て回り、ファイルどうしの依存関係を理解できます。CLAUDE.md というプロジェクト記憶ファイルにも対応しており、AI はセッションの開始時からプロジェクトの背景を把握できます。大きなプロジェクトのリファクタリング、一括修正、テスト作成などのタスクでは、ターミナル Agent は IDE プラグインより効率的です。

### Codex CLI と Aider

OpenAI の Codex CLI も同様のターミナルプログラミング体験を提供し、/import 機能に対応——既存のコードベースを取り込んで AI に理解させられます。Aider はオープンソースのターミナル AI プログラミングツールで、複数のモデル（GPT、Claude、DeepSeek など）に対応しており、柔軟にモデルを切り替えたいユーザーに向いています。これらのツールにはそれぞれ特徴がありますが、核心理念は共通しています：AI に、ターミナル上で開発環境をまるごと操作する能力を持たせることです。

## 四、クラウド Builder 流派

クラウド Builder は、ゼロベースのユーザーに最も向いている流派です。ソフトを何もインストールする必要はなく、ブラウザを開いて要件を記述するだけで、AI がクラウド上に完全なプロジェクトを作り、即時プレビューを見せてくれます。

- v0（Vercel 製）：フロントエンドの UI コンポーネントやページの生成が得意で、Web ページのプロトタイプをすばやく作るのに向いています。
- Bolt：ひと言からフルスタックアプリを作成できます。フロントエンド、バックエンド、データベース込みです。

- Replit Agent：Replit のクラウド IDE の中で動き、プロジェクトを作るだけでなく、クラウド上でそのまま実行やデプロイまでできます。

これらのツールに共通する強みは「設定なしですぐ始められる」こと——ブラウザを開けば始められます。限界は：プロジェクトが複雑になると、クラウドの編集環境の柔軟性がローカル IDE に及ばないことです。そのため、これらは「0 から 1」のプロトタイプ段階に最も適しています：クラウド Builder ですばやくアイデアを確かめ、通ったらローカル IDE に移して本格的に開発します。

## 五、ツールは変わる、方法論は変わらない

もし明日、すべての AI プログラミングツールが消えても、あなたが保つべき三つの習慣：よく考えてから手を動かす、一度に小さな一歩だけ、終わったら必ず検証。ツールは増幅器、方法論は土台。増幅器は交換できるが、土台は揺るがない。

2024 年から 2026 年までのツールの変遷をふりかえりましょう：2024 年の王者は GitHub Copilot、2025 年には Cursor がその座を奪い、2026 年には Claude Code が急速に台頭しました。新しいツールが現れるたびに「今回は違う」と言う人がいますが、最後にわかるのは：新しいツールを使いこなせるのは、まさに方法論がしっかりしている人たちだということ。彼らはツールを変えても数日の順応期で済み、ツールの操作だけを学んだ人は、変わるたびに一から学び直しになります。

そのため、本書がすすめる学習の道筋はこうです：まずはひとつのツール（どれでも構いません）で入門し、実践の中で五大思考原則を体感する。方法論が十分固まったら、異なる流派のツールを試して、設計哲学の違いを感じてみる。最終的には自分だけのツールの組み合わせができます——たとえば日常の開発は Cursor、リファクタリングとテストは Claude Code、すばやいプロトタイプは v0 というように。組み合わせは人それぞれですが、方法論は共通です。

**やってみよう：自分だけの「武器の組み合わせ」を揃えよう**

三つの判断軸（プログラミングの基礎、プロジェクトの段階、デプロイのニーズ）で自分を位置づけましょう：ゼロベースなら Bolt か v0 でページを一枚作ってみる。基礎があれば Cursor か Trae で反復タスクをひとつこなす。挑戦したいならターミナルで Claude Code を一度動かしてみる。そして答えてください：どの流派が、いまのあなたの段階に一番合っていますか？

**本章のまとめ**

- 三大流派：IDE 内蔵 AI（日常の開発）、ターミナル Agent（プロジェクト単位の操作）、クラウド Builder（ゼロベースのプロトタイプ）
- ツール選びは三つの問い：プログラミングの基礎は？プロジェクトの段階は？デプロイのニーズは？
- Cursor 3 の Agent モードが初めて Tab 補完を逆転；Claude Code の年間収益は 25

億米ドル

- クラウド Builder は「0 から 1」に最適。検証が通ったらローカル IDE へ移して本格開発
- ツールは 3 か月ごとに更新される：方法論を身につけた人は、ツールを変えても数日の順応期で済む

# 第四十二章：大規模モデルの全景と API 実践

ここ数章は「*AI* とどう協働するか」の話でしたが、本章ではもっと基本的な問題を扱います：「どの *AI* を使う？ どう使う？」*2026* 年 *8* 月の大規模モデルの情勢は、すでに非常に豊かです——世界では *GPT*、*Claude*、*Gemini* の三強が争い、中国では *DeepSeek*、通義千問（*Qwen*）、*GLM*、*Kimi*、豆包（*Doubao*）が百花繚乱です。どのモデルにもそれぞれの性格と得意分野があります：選択が合っていれば仕事ははかどり、合っていなければ労多くして功少なしです。本章ではこれらのモデルを紹介するだけでなく、*API* を通じて自分のアプリに組み込む方法、そして自分のニーズに合った *Token* プラン（用量プラン）の選び方も教えます。

## 一、世界の主流大モデル：三強の構図

2026 年 8 月の世界の大モデル市場では、OpenAI（GPT シリーズ）、Anthropic（Claude シリーズ）、Google（Gemini シリーズ）の三強が並び立つ構図ができています。どこも個性ははっきりしています——ひとつずつ見ていきましょう。

### OpenAI GPT-5.6 シリーズ：万能でバランスの取れたオールラウンダー

OpenAI は大モデルの波の火付け役で、GPT シリーズは今も世界で最も広く使われているモデルです。2026 年 7 月にリリースされた GPT-5.6 シリーズは、三段階の位置づけを採用しています：フラッグシップの GPT-5.6 Sol は高度な推論ニーズ向け、バランス型の GPT-5.6 Terra は日々の開発向け、軽量の GPT-5.6 Luna は高い同時処理と低コストの場面向けです。3 モデルとも 105 万 Token のコンテキストウィンドウを共有し、画像入力、関数呼び出し、構造化出力に対応しています。

| モデル | 位置づけ | 入力（/100 万 Token） | 出力（/100 万 Token） | コンテキストウィンドウ |
|---|---|---|---|---|
| GPT-5.6 Sol | フラッグシップ推論 | US$5.00（約 36 元） | US$30.00（約 217 元） | 105 万トークン |
| GPT-5.6 Terra | バランス・日常 | US$2.50（約 18 元） | US$15.00（約 109 元） | 105 万トークン |
| GPT-5.6 Luna | 軽量・低コスト | US$1.00（約 7 元） | US$6.00（約 44 元） | 105 万トークン |

OpenAI GPT-5.6 シリーズの料金（2026 年 8 月）

GPT シリーズの中核的な強みは「バランス」――推論、プログラミング、ライティング、マルチモーダルと、目立つ弱みがありません。OpenAI の巨大なプラグインエコシステムと開発者コミュニティもあり、対応ドキュメントとツールチェーンが最も整っています。「何を選べばいいかわからない」という場面では、GPT は「目をつぶって選んでも間違いない」デフォルトの答えです。ただし弱みもあります：中国国内からのアクセスには特殊なネット環境が必要で、中国語の表現にはときどき「翻訳くささ」があり、API 価格は中　高水準です。

**良いニュース**

OpenAI は 2026 年 8 月、無料ユーザーのデフォルトモデルを GPT-5.6 Luna に引き上げ、テキストチャットを無制限に開放すると発表しました。つまり GPT の基本能力をコストゼロで体験でき、より強い推論や API 呼び出しが必要なときにだけ支払えばよいのです。

## Anthropic Claude シリーズ：プログラミングと厳密さの頂点

GPT がオールラウンダーなら、Claude はプログラミング分野の「頂点」です。Anthropic の Claude シリーズは SWE-bench（ソフトウェアエンジニアリングのベンチマーク）で長期間にわたり大差をつけて首位を保ち、コード生成の質と、エンジニアリングレベルのプロジェクトを形にする力は業界トップです。さらに貴重なのは、Claude のハルシネーション率（でたらめな情報を生成する確率）が業界最低の約 1% であること――学術や法務など、高精度が求められる場面に特に適しています。

Claude のもうひとつの切り札は超長いコンテキスト：100 万 Token のコンテキストウィンドウに対応し、何百ページもの契約書やコードベース全体を、細部を見落とさず一気に「読み切る」ことができます。これはコードのリファクタリングや長文書の分析などの場面で、ほぼ代替がききません。ターミナル AI プログラミングツールの Claude Code は、2026 年半ばには年間換算収益 25 億米ドルに達しています――これが、Claude のプログラミング能力が最も認められている証拠です。

| モデル | 位置づけ | 入力（/100 万 Token） | 出力（/100 万 Token） | コンテキストウィンドウ |
|---|---|---|---|---|
| Claude Fable 5 | フラッグシップ | US$10.00（約 72 元） | US$50.00（約 362 元） | 100 万トークン |
| Claude Opus 4.8 | プロ向け | US$5.00（約 36 元） | US$25.00（約 181 元） | 100 万トークン |
| Claude Sonnet 5 | コスパ重視 | US$2.00（約 14 元） | US$10.00（約 72 元） | 100 万トークン |

Anthropic Claude シリーズの料金（2026 年 8 月、Sonnet 5 は優遇価格）

Claude の弱みは価格です——Fable 5 の出力価格は全モデル中で最も高く、100 万 Token あたり 50 米ドル。マルチモーダル対応（画像・動画の理解）もやや弱めです。予算の限られた個人開発者には、優遇期間中の Sonnet 5 のコストパフォーマンスがとても際立っています。

### Google Gemini シリーズ：マルチモーダルの王者

Google の Gemini シリーズは 2026 年にバージョン 3.1 Pro まで進化しました。Gemini 最大の切り札はネイティブマルチモーダル——画像とテキストの解析は正確で、OCR の表認識にミスがなく、動画フレームの抽出も効率的、さらに音声・動画の深い解析にも対応します。Google エコシステム（検索、メール、クラウドサービス）と深く結びついており、リアルタイムの情報検索とモーダルをまたいだ理解で際立った強さを発揮します。Gemini は 200 万 Token の超長いコンテキスト入力にも対応しており、現時点で最も大きなコンテキストウィンドウを持つモデルです。

Gemini の位置づけは「科学的推論とマルチモーダルの場面での第一選択」です。プロジェクトに大量の画像分析、動画の理解、科学計算が関わるなら、Gemini にはほぼ対抗馬がいません。ただし、テキストだけのコード生成では Claude に少し劣り、中国語の表現も中国国産モデルほど自然ではありません。

| 比較項目 | GPT-5.6 | Claude Fable 5 | Gemini 3.1 Pro |
|---|---|---|---|
| 総合スコア | 94.8 | 95.0 | 92.1 |
| コーディング力（SWE-bench） | 74.9% | 64.3% (Pro) | 中程度 |
| コンテキストウィンドウ | 105 万トークン | 100 万トークン | 200 万トークン |
| マルチモーダル能力 | バランス | やや弱い | 最強 |
| ハルシネーション率 | 低め | 最低（約 1%） | 低め |
| 中国国内での利用可否 | 特殊なネット環境が必要 | 特殊なネット環境が必要 | 特殊なネット環境が必要 |
| API エコシステム | 最も整っている | 成長中 | Google エコシステムとの一体化 |

国際三大モデルの横断比較（2026 年 8 月）

**注意：中国国内で使うときの現実的なハードル**

中国国内の開発者にとって、GPT、Claude、Gemini の最大の障害はネットワークアクセスです。Azure のプロキシや API 集約プラットフォームなどを通じて間接的に利用することもできますが、安定性と法令遵守については、自分で評価する必要があります。データの安全に厳しい要件があるプロジェクトでは、まず中国国産モデルの利用をおすすめします。

## 二、中国国産の主流大モデル：百花繚乱

2026 年の中国国産大モデルは、「追いかける側」から「並んで走る側」になり、分野によっては「先頭を走る側」にまで成長しました。2023 年の「中身の流用」騒動とは違い、2026 年の国産フラッグシップモデルは、論理推論、コード生成、中国語理解などの面で世界トップクラスのモデルとほぼ互角になり、差は 5% 以内まで縮まっています。さらに大事なのは、中国語の場面、価格、コンプライアンスを満たした運用では、国産モデルに代えられない強みがあることです。

### DeepSeek V4：コストパフォーマンスの王者

DeepSeek（深度求索）は中国国産大モデルの中で現象級の存在です。2025 年の初め、DeepSeek-V3 は「GPT より 10 分の 1 安いのに能力は同等」という位置づけで市場を震撼させ、業界では「汎用 AI 大モデルの斬殺ライン」と呼ばれました——それより安いものは使い勝手で及ばず、それより高いものはコストの安さと提供力で及ばない、という意味です。2026 年 8 月には DeepSeek V4 正式版が登場しました：1.6 兆パラメータの MoE アーキテクチャ、動かすのは約 49B、100 万 Token の劣化なしコンテキストに対応し、コードのコストパフォーマンスは世界の天井と広く見られています。

DeepSeek V4 は Agent 能力で大きく跳ね上がりました——DeepSWE ベンチマークが 12.8 から 62.7 へ（+390%）、NL2Repo が 38.5 から 61.5 へ。API には Responses API と Anthropic API の二つのプロトコル対応が新たに加わり、Codex や Claude Code といった主流のプログラミングツールとの互換性が実現しています。つまり、Claude の API の代わりに DeepSeek の API を使って Claude Code を動かせる——コストは数分の一に下がり、体験はほぼ同じです。

| モデル | 位置づけ | 入力（/100 万 Token） | 出力（/100 万 Token） | コンテキストウィンドウ |
|---|---|---|---|---|
| DeepSeek V4 Pro | フラッグシップ推論 | 3 元（約 US$0.43） | 6 元（約 US$0.87） | 100 万トークン |
| DeepSeek V4 Flash | 軽量・高速 | 1 元（約 US$0.14） | 2 元（約 US$0.28） | 100 万トークン |

DeepSeek V4 シリーズの料金（2026 年 8 月、セール価格）

価格比較：同じく 100 万 Token の入力＋ 100 万 Token の出力で、DeepSeek V4 Flash は約 0.42 米ドル、GPT-5.6 Sol は約 35 米ドル、Claude Fable 5 は約 60 米ドル。差はなんと 80　140 倍にもなります。

**落とし穴に注意：DeepSeek は値上げの予定です**

DeepSeek 公式は 2026 年 8 月 6 日、「近いうちに API 料金を大幅に引き上げる計画」と発表しており、セール価格は長くは続かない可能性があります。一部のメディアでは値上げ幅が 30 倍に達する可能性があると報じられていますが、8 月 14 日時点で公式は最終的な改定案を公表していません。さらに DeepSeek V4 正式版では、ピーク・オフピーク料金の仕組みが導入され——ピーク時間帯の API 呼び出し料金はそのまま 2 倍になります。最新の料金は公式の発表を確認することをおすすめします。

## 通義千問 Qwen3.7 Max：アリババクラウドのエンタープライズ向け選択肢

アリババクラウドの通義千問（Qwen）シリーズは、アリババクラウドのエコシステムを背景に、企業向けの運用で元から強みを持っています。2026 年 7 月の Qwen3.7 Max が現行のフラッグシップ：235B パラメータのデンスモデルで、ネイティブコンテキストは 256K、1M まで拡張可能です。Qwen の中核的な強みは：成熟したオープンソースエコシステム（7B から 235B まで全シリーズがオープンソース）、正確な中国語の場面の理解、アリババクラウド基盤との深い統合です。

| モデル | 出力価格（/100 万 Token） | 向いている場面 |
|---|---|---|
| qwen-turbo | 0.6 元 | ごくシンプルな質疑応答、テキスト分類、高負荷な同時処理 |
| qwen-long | 2.0 元 | 10M の超長コンテキスト処理 |
| qwen3.7-flash | 7.2 元 | 汎用対話、コンテンツ生成 |
| qwen3.7-plus | 12.0 元 | マルチモーダル（画像＋動画）、中国語創作 |
| qwen3.7-max | 36.0 元 | フラッグシップの中国語能力、複雑なタスク |

Qwen（通義千問）シリーズの段階制料金（2026 年 8 月）

Qwen の料金は段階制を採用しています。極めて低コストの qwen-turbo（出力 0.6 元/100 万トークン）から、フラッグシップの qwen3.7-max（出力 36 元/100 万トークン）まで、軽量な質疑応答から複雑な推論まで、あらゆるシーンのニーズをカバーします。すでに Alibaba Cloud を使っている企業にとって、Qwen は最も手軽な統合の選択肢です。

## 智譜 GLM-5.1：国産 SWE-bench 第 1 位

智譜 AI（Zhipu AI）の GLM シリーズは 2026 年に目覚ましい活躍を見せています。GLM-5.1 は SWE-bench で国産第 1 位の成績を獲得しました。744B の MoE（混合エキスパート）

ハイブリッドアーキテクチャで、100 万トークンのコンテキストに対応します。特徴は中国語の意味理解が正確なこと、そして中国国内のコンプライアンス準拠のプライベート導入に対応していることです。これはデータセキュリティへの要求が厳しい金融・行政の場面でとても重要になります。GLM-5.1 の料金は入力 6 元、出力 24 元/100 万トークンで、国産モデルの中堅クラスです。

### Kimi K3：超長文の王

Moonshot AI（月の暗面）の Kimi シリーズは、超長文の処理能力で知られています。2026 年 7 月に公開された Kimi K3 は 2.8 兆パラメータの規模を誇り、複数の難しいタスクの評価で世界の第一集団に入りました。Kimi の主戦場は、大きなコードベースの分析や長文ドキュメントの検索拡張です。本 1 冊分のテキストも、プロジェクト一式のコードも処理でき、「needle in a haystack（干し草の山から針探し）」式の検索タスクで優れた成績を収めます。Kimi の料金は比較的高め（出力約 12　20 元/100 万トークン）ですが、長文ユースケースでのコストパフォーマンスには他の追随を許しません。

### 豆包（Doubao）：ByteDance のオールラウンダー

ByteDance の豆包（Doubao）シリーズは 2026 年も安定したパフォーマンスを見せています。Doubao Seed 2.1 Pro は火山引擎（Volcano Engine）のフラッグシップモデルで、256K のコンテキストを持ち、実測でも総合的に安定した動きを見せ、中国語のユースケースでの体験が特に優れています。豆包の強みは、登録するだけで大量の無料トークンがもらえること、火山引擎エコシステムとの統合が整っていること、そして料金が中国国内で最安クラスのひとつであることです。手早く AI プログラミングを体験してみたい初心者には、豆包がハードルゼロの入門向け選択肢です。

| 比較項目 | DeepSeek V4 | Qwen3.7 Max | GLM-5.1 | Kimi K3 | Doubao 2.1 Pro |
|---|---|---|---|---|---|
| 総合スコア | 87.5 | 89.7 | 90.5 | 88+ | 85+ |
| プログラミング能力 | 極めて強力 | 強 | 国産 SWE 第 1 位 | 強 | 中の上 |
| 中国語理解 | 優秀 | 優秀 | 正確 | 優秀 | 優秀 |
| コンテキストウィンドウ | 100 万 | 256K-1M | 100 万 | 超長文 | 256K |
| 出力価格（元/100 万トークン） | 6 | 36 | 24 | 12-20 | 30 |
| オープンソース | MIT ライセンス | 全シリーズ公開 | 一部公開 | クローズドソース | クローズドソース |
| プライベート導入 | 対応 | 対応 | 対応 | 非対応 | 非対応 |

国産 5 大モデルの横断比較（2026 年 8 月）

重要なトレンドがひとつあります。国産モデルと GPT/Claude の間に感じられた「世代の差」が、もう消えたということです。論理推論や数学コンテストでは、国産フラッグシップと GPT-5 のスコア差はほぼ一桁です。体系立ったアーキテクチャ思考や複雑なエンジニアリング推論では、Claude が依然リードしています。より「ひらめき」のあるアーキテクチャ案を出せる一方で、国産モデルはときどき「マニュアル通り」に見えます。ただし、中国語のユースケース、コスト管理、コンプライアンス対応の導入では、国産モデルが圧倒的な優位に立ちます。




図 4：5 モデルの価格と性能の比較（価格は低いほど、スコアは高いほど良い）

## 三、モデル選択の意思決定フレームワーク

各モデルの特徴がわかったところで、自分のプロジェクトにはどのモデルを選べばいいのでしょうか。ここで「四問決定法」をお渡しします。4 つの質問に答えるだけで、あなたに最も合ったモデルを絞り込めます。

プロジェクトは国内向けですか、国際展開ですか？

国内国産モデルを選ぶ

|

国際海外モデルも選べる

中心の仕事はプログラミングですか、それとも別？

プログラミング Claude または DeepSeek

|

その他判断を続ける

超長コンテキスト（>50 万トークン）が必要？

必要 Gemini または Kimi

|

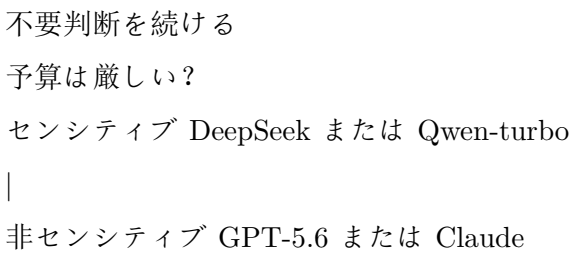



図 5：大規模モデル選択の四問決定法

**ワンポイント：組み合わせ戦略**

この決定法は金科玉条ではありません。多くの場面では、複数のモデルを組み合わせて使う必要があります。よくある組み合わせ戦略はこうです。日常の開発には DeepSeek か Qwen（コストが低く中国語が得意）、重要なアーキテクチャ設計には Claude（推論が強くハルシネーションが少ない）、マルチモーダル分析には Gemini（画像・動画の処理）。「1 つのモデルを使い倒す」のではなく、必要に応じてモデルを切り替えましょう。

## 四、API 呼び出し入門：ゼロから大規模モデルにつなぐ

多くの AI プログラミング解説は対話画面の使い方しか教えてくれませんが、本物の「AI プログラミング」——自分のアプリから大規模モデルを呼び出すこと——には API が必要です。API（Application Programming Interface、アプリケーションプログラミングインターフェース）はプログラム同士の通信プロトコルです。API を通れば、あなたのコードは関数を 1 つ呼び出すように大規模モデルの能力を利用できます。

### トークンとは

API を理解する前に、まず「トークン（Token）」という概念を押さえる必要があります。トークンは大規模モデルの課金の基本単位で、「AI にとっての読み書きの単位」と考えるとよいでしょう。1 トークンはおよそ、英語では 0.75 単語（4 文字）、中国語では 0.5　1 文字に対応します。中国語 1000 字のテキストなら、約 1500　2000 トークンを消費します。

トークンは字数でも文字数でもありません。大規模モデルは専用のトークナイザー（Tokenizer）でテキストをトークンに分割します。同じ語でもモデルによってトークン数は異なることがあります。たとえば「人工智能（人工知能）」は GPT では 3 トークン、DeepSeek では 2 トークンになる、といった具合です。ほとんどのモデルの API は実際に消費したトークン数を返すので、API レスポンスで確認できます。

**トークンコストのクイック見積もり**

中国語 100 万字は約 150 万トークンに相当します。DeepSeek V4 Flash で入力 100 万字＋出力 100 万字を処理すると、コストは約 0.42 米ドル（約 3 元）。同じ量を Claude Fable 5 で処理すると、約 60 米ドル（約 434 元）かかります。モデル選びがプロジェクトのコストを直接左右するのはこのためです。

## API キーの取得

大規模モデル API を使う最初のステップは API キーの取得です。認証に使う一連の文字列のことですね。各モデル提供元にはそれぞれ API プラットフォームがあり、アカウントを登録して API キーを作成する必要があります。主なモデルの API プラットフォームへの入り口と取得手順は以下のとおりです。

| モデル | API プラットフォーム | 取得手順 | 無料枠 |
|---|---|---|---|
| DeepSeek | platform.deepseek.com | 登録実名認証 API キー作成 | 新規ユーザーに利用枠を進呈 |
| Qwen（通義千問） | bailian.aliyun.com（百煉プラットフォーム） | Alibaba Cloud アカウントでログインサービスを有効化キーを作成 | 新規ユーザーは無料トライアル付き |
| 智譜 GLM | open.bigmodel.cn | 登録実名認証 API キー作成 | 新規ユーザーに利用枠を進呈 |
| Kimi | platform.moonshot.cn | 登録実名認証 API キー作成 | 限定的な無料枠 |
| 豆包（Doubao） | volcengine.com（Volcengine） | 登録方舟（Ark）を有効化 API キー作成 | 登録で大量のトークンをプレゼント |
| OpenAI GPT | platform.openai.com | 登録クレジットカード登録 API キー作成 | 無料枠なし（有料） |
| Anthropic Claude | console.anthropic.com | 登録カード登録 API キー作成 | 新規ユーザーに少額の利用枠 |

主なモデルの API プラットフォームとキー取得方法（2026 年 8 月）

**安全上の注意：API キー＝財布のパスワード**

API キーはあなたの財布のパスワードと同じです。キーを手にした人は誰でも、あなたの利用枠でモデルを呼び出せてしまいます。したがって、API キーをコードに書かない（環境変数で管理する）、キーを含むコードを公開 Git リポジトリにコミットしない、キーが映ったスクリーンショットを SNS に共有しない——この 3 つは絶対に守ってください。キーが漏れた場合は、すぐにプラットフォームで削除して作り直しましょう。

## 初めての API 呼び出し

API キーを取得すれば、どんなプログラミング言語でも大規模モデル API を呼び出せます。以下では Python で最もシンプルな呼び出しをデモします。DeepSeek にメッセージを 1 つ送り、返信を取得します。

```
# SDKのインストール：pip install openai
# DeepSeekはOpenAI SDK形式と互換あり、そのまま使える
```

```
from openai import OpenAI
import os

# 1. クライアントを作成（API Keyは環境変数から読み込む）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 2. リクエスト送信
response = client.chat.completions.create(
    model="deepseek-chat",  # モデル指定
    messages=[
        {"role": "system", "content": "あなたはPythonプログラミングの助手です。"},
        {"role": "user", "content": "クイックソートの関数を書いてください。"}
    ]
)

# 3. 返答を取得
print(response.choices[0].message.content)

# 4. トークン消費を確認
print(f"入力トークン: {response.usage.prompt_tokens}")
print(f"出力トークン: {response.usage.completion_tokens}")
print(f"合計トークン: {response.usage.total_tokens}")
```

このコードの核となるロジックは次のとおりです。クライアントを作成し（API キーとモデルのアドレスを渡す）メッセージリストを組み立てる（system が役割を設定し、user がユーザー入力）リクエストを送る返信を受け取る。このパターンは、OpenAI 形式に互換するほぼすべてのモデルに使えます。api_key と base_url を差し替えるだけでモデルを切り替えられます。多くの国産モデルが OpenAI SDK 互換形式を選ぶのはこのためです。開発者は 2 行変えるだけでモデルを乗り換えられ、移行コストが極めて低くて済みます。

以下は他のモデルへ切り替えるときのコードの差分です。変更が必要なのはわずか 2 行であることに注目してください。

```
# DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)
# model="deepseek-chat"

# Qwen（百煉プラットフォーム）
client = OpenAI(
    api_key=os.environ.get("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
```

```
)
# model="qwen-plus"

# Zhipu GLM
client = OpenAI(
    api_key=os.environ.get("GLM_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4"
)
# model="glm-4"

# OpenAI GPT
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY"),
    # base_urlはデフォルトのままでOK
)
# model="gpt-5.6-terra"
```

### ストリーミング応答の処理

実際のアプリでは、AI が全部書き終わるのを待って一度に表示させるのは避けたいものです。ユーザーを長く待たせてしまうからです。ストリーミング応答（Streaming）なら、AI が生成しているのと並行して一文字ずつ表示でき、ユーザー体験が大きく向上します。タイプライターのように文字がひとつずつ飛び出してくるのを見ると、体感はまったく違います。

```
# ストリーミング応答：一文字ずつ出力
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "Pythonのバブルソートの解説を書いてください"}
    ],
    stream=True  # ストリーミングを有効化
)

# ひとかたまりずつ受け取って表示
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
# 出力はタイプライターのように一文字ずつ流れる
```

## 五、Token プラン購入ガイド

ほとんどのモデル提供元には 2 つの課金方式があります。従量課金（Pay-as-you-go）と、定額サブスクリプション（Token プラン / Subscription）です。この 2 つの違いを理解すれ

ば、利用コストを大幅に下げられます。

### 従量課金 vs サブスクリプション

| 観点 | 従量課金（Pay-as-you-go） | サブスクリプション（Subscription） |
|---|---|---|
| 課金方式 | 使ったトークン分だけ支払う | 定額を前払いして利用枠を取得 |
| 向いている場面 | 利用量が不安定、たまに呼び出す程度 | 利用量が安定、高頻度で呼び出す |
| 価格面のメリット | 標準価格 | 従量課金より 20　50% 安い |
| 柔軟性 | いつでも利用でき、縛りなし | 前払いが必要で、期限切れの利用枠は失効する場合も |
| 超過時の扱い | 自動的に従量課金 | 利用枠を超えると従量課金に切り替わり |
| 代表プラットフォーム | 各モデルの公式 API プラットフォーム | OpenAI　Plus/Pro,　Claude Pro, Cursor Pro |

2 つの課金方式の比較

初心者はまず従量課金から始めるのがおすすめです。前払いが不要で、使った分だけ払えばよいからです。月の使用量が一定の水準で安定してきたら（たとえば月 50 元を超えるなど）、より安い単価を得るためにサブスクリプションへの切り替えを検討しましょう。

### 主なサブスクリプションプラン一覧

| プラン | 月額 | 含まれる内容 | 向いている人 |
|---|---|---|---|
| ChatGPT Free | 無料 | GPT-5.6 Luna でテキスト対話し放題 | 初心者の体験用 |
| ChatGPT Plus | US$20/月 | GPT-5.6 Sol、画像生成、データ分析、プラグイン利用 | 中程度の利用者 |
| ChatGPT Pro | US$200/月 | 全モデル無制限、新機能への優先アクセス | ヘビーなプロユーザー |
| Claude Free | 無料 | 回数制限ありの Claude 対話 | 初心者の体験用 |
| Claude Pro | US$20/月 | Claude を無制限に利用、新モデルへの優先アクセス | プログラミング開発者 |
| Cursor Free | 無料 | 回数制限ありの AI 補完と対話 | 初心者の体験用 |
| Cursor Pro | US$20/月 | AI 補完し放題、月 500 回の高度なリクエスト | 日常的に開発する人 |
| Cursor Business | US$40/月/人 | チームコラボレーション、管理画面、プライバシーモード | チーム利用 |

主な AI ツールのサブスクリプションプラン（2026 年 8 月）

中国国産モデルのサブスクリプション事情は少し異なります。DeepSeek は基本的に従量課金で、月額のサブスクプランはありません。チャージしておくと、実際の使用量に応じて精算されます。Qwen（通義千問）は百煉プラットフォームで前払いのリソースパック（一定量のトークン利用枠を割引価格で購入）を提供しています。豆包も火山引擎に同様の前払いプランがあります。中国国内の開発者にとって最も経済的な組み合わせは、たいてい「DeepSeek の従量課金（日常の開発）＋豆包の無料枠（試して確認）」です。

### API 集約プラットフォーム

複数のモデルを同時に使いたいなら、API 集約プラットフォームは手間いらずの選択肢です。集約プラットフォームは統一された API キーを 1 つ提供し、それだけで世界中の何十種類ものモデルを呼び出せます。各プラットフォームで個別に登録する必要はありません。

| プラットフォーム | 特徴 | 価格面のメリット | 向いている場面 |
|---|---|---|---|
| OpenRouter (openrouter.ai) | 海外の集約プラットフォーム。1 つの Key で世界中のモデルを呼び出せる | 価格が透明で、リアルタイムの価格比較に対応 | 複数の海外モデルを柔軟に切り替えたい場合 |
| SiliconFlow (siliconflow.cn) | 中国国内の集約プラットフォーム。各種国産モデルの API を提供 | 公式より 20 30% 安い | 中国国内の開発者、複数モデルの価格比較 |
| Azure OpenAI（マイクロソフト経由） | マイクロソフトの Azure が仲介する OpenAI API | Batch API なら 5 割引き | エンタープライズ向け OpenAI 利用、大量の非同期タスク |
| 百度千帆（qianfan.baidubce.com） | 百度の AI クラウドプラットフォーム。文心（ERNIE）などのモデルを提供 | 新規ユーザーに利用枠をプレゼント | 百度エコシステムの利用者 |

主な API 集約プラットフォーム（2026 年 8 月）

**落とし穴注意:「共有 Key」を買わない**

淘宝（タオバオ）や閑魚（Xianyu）などのプラットフォームで、いわゆる「共有 API キー」を買ってはいけません。こうしたキーは出所が不明なことが多く、いつアカウント停止になってもおかしくありませんし、データ漏えいのリスクもあります。API キーは必ず公式プラットフォームか正規の集約プラットフォームから入手しましょう。

## コスト最適化の戦略

適切なモデルを選ぶことは第一歩にすぎず、「お得に使う」工夫も同じくらい重要です。以下は、実践で効果が確認されたコスト最適化の戦略です。

1. キャッシュヒットを活用する。多くのモデルには「キャッシュ」機能があります。リクエストに、以前処理したことのある内容（システムプロンプトなど）が含まれる場合、その部分は極めて低い価格で課金されます。DeepSeek のキャッシュヒット価格は通常価格のわずか 1/60 です。固定のシステムプロンプトをメッセージリストの先頭に置くと、キャッシュヒットを最大化できます。

2. レベル別ルーティング——簡単なタスクは安いモデルに、複雑なタスクは高いモデルに振り分けます。たとえばテキスト分類には DeepSeek V4 Flash（出力 2 元/100 万トークン）、アーキテクチャ設計には Claude（出力 181 元/100 万トークン）という具合です。プロジェクト内の呼び出しの 80% は、安いモデルでまかなえます。

3. 出力の長さを制限する。API は出力トークンで課金されるため、出力が長いほど高くつきます。長文を必要としない場面では max_tokens パラメータを設定して出力長を制限し、AI の「余計なおしゃべり」を防ぎましょう。

4. 非リアルタイムのタスクは Batch API を使う。即時の応答を必要としないタスク（一括データアノテーション、文書要約など）が大量にあるなら、Batch API を使うと 5 割引きになります。OpenAI も Azure もこの機能を提供しています。

5. トークン使用量をモニタリングする。ほとんどの API プラットフォームには使用量統計のダッシュボードがあります。定期的に使用量の内訳を確認し、トークンを最も消費している呼び出しを見つけて、そこを重点的に最適化しましょう。プロンプトを少し変えるだけ（たとえば「詳しく説明してください」を「3 文で説明してください」に変える）でも、トークンを 50% 節約できることがあります。

| 最適化の戦略 | 節約幅 | 実施の難易度 | 説明 |
|---|---|---|---|
| キャッシュヒット | 最大 98% | 低 | 固定の内容をメッセージの先頭に置く |
| レベル別ルーティング | 50%-80% | 中 | 簡単なタスクは安いモデルで処理 |
| 出力の長さを制限 | 30%-60% | 低 | max_tokens パラメータを設定 |
| Batch API | 50% | 低 | 非リアルタイムタスクはバッチ処理に |
| プロンプトの最適化 | 20%-50% | 中 | プロンプトを簡潔にし、冗長さを減らす |

コスト最適化の戦略と効果（2026 年 8 月）

## 六、実践：プロジェクトに大規模モデル API を組み込む

本章の締めくくりは、ひとつの完全な実践です。「AI 要約ジェネレーター」を作りましょう。ユーザーが長文を入力すると、大規模モデル API を呼び出して要約を生成します。この実践を通じて、環境設定、API 呼び出し、エラー処理、コスト管理という API 利用の一連の流れを学べます。

```
# ai_summarizer.py - AI要約ジェネレーター
import os
from openai import OpenAI

# 1. 設定（API Keyは環境変数から読み込む。ハードコード禁止）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def summarize(text: str, max_length: int = 200) -> dict:
    """テキスト要約
```

```
    Args:
        text: 要約したい元テキスト
        max_length: 要約の最大長（トークン数）

    Returns:
        dict: 要約とトークン消費情報を含む
    """
    try:
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=[
                {
                    "role": "system",
                    "content": "あなたは要約ジェネレーターです。"
                               "ユーザーが提供したテキストを簡潔にまとめてください。"
                               "要約は3文以内で。"
                },
                {
                    "role": "user",
                    "content": f"以下のテキストを要約してください:\n\n{text}"
                }
            ],
            max_tokens=max_length,  # 出力の長さを制限
            temperature=0.3  # 温度低=より安定
        )

        return {
            "success": True,
            "summary": response.choices[0].message.content,
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
            "total_tokens": response.usage.total_tokens
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 2. 使用例
if __name__ == "__main__":
    long_text = """
    （長文をここに貼り付け...）
    """

    result = summarize(long_text)
```

```
    if result["success"]:
        print(f"要約: {result['summary']}")
        print(f"トークン消費: {result['total_tokens']}")
        # コスト概算：DeepSeek V4 Flash
        cost = result['total_tokens'] * 0.000004  # 約0.4厘/1000トークン
        print(f"概算コスト: {cost:.4f} 元")
    else:
        print(f"エラー: {result['error']}")
```

このコードには、API 呼び出しのベストプラクティスがいくつか盛り込まれています。API キーは環境変数から読み込み、コードにハードコードしない。関数にはわかりやすいドキュメントと型ヒントを付ける。max_tokens を設定して出力長を制限する。temperature は低め（0.3）にして、より安定した出力を得る。エラー処理を完備する。コスト追跡のためにトークン消費情報を返す。このコードを AI に書かせるときは、こうした要件を伝えれば OK です。AI にはそれを実現する十分な力があります。

この章の最後の教え：モデルは「弾薬」、方法論は「戦術」。弾薬を正しく選べば戦いは楽になるが、戦術が悪ければどんな弾薬でも勝てない。各モデルの特徴と API の使い方を知れば「弾薬庫」は十分——ただしこの戦いをどう戦うかは、前の章で学んだ思考方法論にかかっています。

**やってみよう：初めての API 呼び出しを動かそう**

国産モデルのプラットフォームに 1 つ登録しましょう（おすすめは DeepSeek か智譜。どちらも新規ユーザーに利用枠をプレゼントしてくれます）。API キーを作成し、システムの環境変数に書き込んで（コードの中ではなく！）、先ほどの「初めての API 呼び出し」のコードを実行します。AI の返答が表示されたその瞬間、あなたは「AI を使う側」と「AI アプリを作る側」の境界線を越えたのです。

**本章のまとめ**

- 国際の三強：GPT-5.6（バランス型のオールラウンダー）、Claude（プログラミング能力の頂点＋ハルシネーション率わずか 1%）、Gemini（マルチモーダルの王＋ 200 万コンテキスト）
- 国産の五強：DeepSeek V4（コスパの王、値上げ公告に注意）、Qwen3.7（エンタープライズ向け）、GLM-5.1（SWE 国産第 1 位）、Kimi K3（超長文）、豆包（無料で入門）
- 四問決定法：デプロイ先は？プログラミングかそれ以外か？超長文コンテキストは必要か？予算はシビアか？
- トークンは課金単位：中国語 1 文字は約 1.5　2 トークン。100 万字で約 150 万トークン
- API キー＝財布のパスワード：環境変数で管理。ハードコードは絶対 NG、Git に

アップロードしない

- 国産モデルはほぼ OpenAI SDK 互換：モデルの切り替えは api_key と base_url の 2 行を変えるだけ
- 節約の 5 つのコツ：キャッシュヒット、レベル別ルーティング、出力制限、Batch API、使用量モニタリング

# 第四十三章：対話エンジニアリング——AI と効率的に協働するための対話技術

対話は *AI* プログラミングの中核となるやり取りの方法です。ただし「対話」は「おしゃべり」と同じではありません。効率的な *AI* プログラミングの対話とは、構造化されたコミュニケーション技術のことで、本書ではこれを「対話エンジニアリング」と呼びます。その研究対象は、いかに少ない対話ターンで最高品質のコードを引き出すか。はっきり言えば、この章は *AI* と「物事をはっきり伝える」方法を教える章なのです。

## 一、プロンプトの階層戦略

AI との対話は 3 つの階層に分けられ、それぞれ担う役割が違います。この 3 層を理解すれば、積み木を組み合わせるように効率的なプロンプトを作れるようになります。

| 階層 | 機能 | 例 | 頻度 |
| --- | --- | --- | --- |
| システム層 | AI の役割と全体的な制約を設定する | 「あなたは React の専門家です。コードは必ず TypeScript で書くこと」 | プロジェクト開始時に 1 度だけ設定 |
| タスク層 | 今から取り組む具体的なタスクを記述する | 「ユーザー名とパスワードの入力欄を含むフォームコンポーネントを実装してください」 | タスクごとの開始時に与える |
| フィードバック層 | AI の出力を評価して修正させる | 「パスワード欄に表示/非表示の切り替えがないので追加してください」 | 出力の質に応じて与える |

対話プロンプトの 3 つの階層

システム層のプロンプトは通常、プロジェクト記憶ファイル（CLAUDE.md など）に書いておきます。対話のたびに繰り返す必要はありません。タスク層は対話の中で最もよく使

う層で、「何を」「どの技術で」「どんな制約のもとで」という 3 要素を含めるべきです。いちばん腕の見せどころはフィードバック層です。良いフィードバックは AI に的確に修正させられますが、悪いフィードバックは AI に再び当てずっぽうをさせるだけです。

**ワンポイント：フィードバックの 3 つのコツ**

実用的なフィードバックのテンプレートが「問題を指摘＋期待を説明＋文脈を提供」です。たとえば「25 行目のエラー処理が console.log だけになっています（問題を指摘）。本番環境では共通のエラーハンドラを使い、ユーザーに分かりやすいメッセージを返すべきです（期待を説明）。プロジェクトには handleError 関数があるので使えます（文脈を提供）」。このようなフィードバックは「エラー処理を直して」と頼むより 10 倍効きます。

## 二、デバッグ対話：エラーの説明で AI の問題特定を速くするには

デバッグは AI プログラミングで最も頻繁に登場する場面のひとつです。バグに遭遇したとき、問題を AI にどう伝えるかが、そのまま素早く特定して直せるかどうかを左右します。デバッグ対話の 4 つのポイントは次のとおりです。

1. あなたの言い換えではなく、生の情報を AI に「見せる」。エラーログや例外のスタックトレース、関連コードは「なんかエラーが出たみたい」と説明するのではなく、@ で直接引用して AI に渡しましょう。AI はあなたの解釈を見るより、生の情報を見るほうが正確に把握できます。

2. 「期待する動作」と「実際の動作」の差を説明する。「ボタンをクリックしたらホームに遷移するつもりでした（期待）が、実際にはクリックしてもページが反応しません（実際）」——こうした説明は「ボタンが壊れています」よりずっと役に立ちます。

3. 再現手順を示す。「先にユーザー名を入力し、次にログインボタンをクリックすると、このエラーが出ます」——再現手順があれば AI はあなたの操作の流れをなぞれるので、問題を特定しやすくなります。

4. すでに試したことをはっきり伝える。「ネットワークリクエストは確認済みで、200 が返ってきてデータも正常です」——これは AI が検証済みの方向を除外し、同じ調査を繰り返して時間を無駄にするのを防いでくれます。

悪いデバッグの説明

```
ボタンをクリックしても反応がありません。見てください
```

良いデバッグの説明

```
ログインボタンをクリックしてもページが反応しません。 期待する動作：クリックすると /home
    ページに遷移する 実際の動作：クリックしても何も反応せず、コンソールにエラーなし 再
    現手順： 1. ログインページでユーザー名adminとパスワード123456を入力 2. 「ログイ
    ン」ボタンをクリック 3. ページが反応しない 確認済みの項目： - ネットワークリクエス
    トは正常、200を返す - onClickイベントはバインド済み - LoginButtonコンポーネントの
```

```
コードを確認済み 関連ファイル：@src/components/LoginButton.tsx @src/pages/Login.
tsx
```

## 三、反復戦略：小刻み前進 vs 一括リファクタリング

AI プログラミングにおける反復には 2 つの基本的な戦略があり、向いている場面がそれぞれ違います。この 2 つの違いを理解しておくと、状況に応じて正しい選択ができます。

「小刻み前進」戦略は、機能開発やバグ修正に向いています。核心は、毎回 AI に小さな修正を 1 つだけさせ、検証が通ってから次に進むこと。メリットは、一歩ごとに検証も取り消しもでき、リスクを制御できること。デメリットは、スピードが遅めで、対話のやり取りを頻繁に要ることです。向いているのは：新機能の開発、最適解が定まらないときの探索、AI の出力にあまり安心できないときの慎重な反復。

「一括リファクタリング」戦略は、目標が明確な大規模な変更に向いています。核心は、完成形の目標を一度に記述し、変更全体の計画と実行を AI に任せること。メリットは、スピードが速く、AI が全体像を見ながらより良いアーキテクチャ判断を下せること。デメリットは、リスクが高く、AI の理解がズレた場合の修正コストが大きいことです。向いているのは：明確なアーキテクチャの刷新、大規模なコード移行、十分なテストで裏付けられたリファクタリング。

| 観点 | 小刻み前進 | 一括リファクタリング |
| --- | --- | --- |
| 1 回あたりの修正量 | 機能 1 つまたはバグ 1 つ | モジュール 1 つまたはアーキテクチャ層 1 つ |
| 検証の頻度 | 修正のたびにすぐ検証 | すべての修正が完了してからまとめて検証 |
| 対話のターン数 | 多い（機能あたり 5 10 ターン） | 少ない（モジュールあたり 1 3 ターン） |
| リスク | 低い。各ステップで取り消し可能 | 高い。ズレを直すコストが大きい |
| 向いている場面 | 新機能の開発、探索的な開発 | 目標のはっきりしたリファクタリングや移行 |
| 前提条件 | 特別な要件なし | テストカバレッジが必要 |

2 つの反復戦略の比較

実践では、この 2 つの戦略を組み合わせて使うことがよくあります。たとえばモジュールをリファクタリングするとき、まず「一括リファクタリング」で AI に全体案を出させ、その案をレビューしてから「小刻み前進」で少しずつ実行します。こうすれば全体を見渡す視点を保ちつつ、実行のリスクも抑えられます。

## 四、マルチターン対話のコンテキスト管理

対話のターンが重なるにつれ、対話のコンテキストを自分から管理して、「AI が最初の情報を忘れてしまう」問題を避ける必要があります。実用的なコンテキスト管理のテクニックをいくつか紹介します。

第一に、重要な情報を「固定（アンカー）」する。対話の最初に重要な制約を明確に書き、その後の対話でも適宜繰り返します。たとえば「覚えておいてください。データは localStorage に保存していて、データベースは使えません」——関連する修正の依頼の中でこの制約を重ねて伝え、AI の忘却を防ぎます。

第二に、「要約して引き継ぐ」テクニックを活用する。対話が長くなってきたら、AI に「ここまでの進捗と残りのタスクを要約してください」と頼み、その要約を出発点に新しい対話を始めます。こうすれば重要なコンテキストを保ちつつ、長い対話による劣化を避けられます。

第三に、「1 つの対話に 1 つのタスク」。これは最もシンプルで、最も効果的なコンテキスト管理の戦略です。1 つの対話では 1 つのタスクか機能だけを扱い、終わったら新しい対話で次に進みます。対話の長さが自然に制限され、コンテキストが明快に保たれます。

第四に、ツールのコンテキスト管理機能を利用する。ほとんどの AI プログラミングツールには「コンテキストをクリア」や「新規セッション」のボタンがあります。AI の回答の質が落ちてきたと感じたら、ためらわずに新しいセッションを開きましょう。会話履歴を少し失うほうが、劣化したコンテキストの中で足掻き続けるよりずっとましです。

**やってみよう：AI の「患者」を一度やってみる**

ブラウザでわざと小さなページを作り、バグを仕込んでみましょう（たとえばボタンが反応しない、など）。そして本章の「良いデバッグの説明」の形式に従って、AI に一度きちんと説明してみます。生の情報、期待 vs 実際、再現手順、確認済みの項目。AI が問題を特定する速さの違いを体感すれば、「問題の説明」それ自体がひとつの技術だと分かるはずです。

**本章のまとめ**

- プロンプトの 3 層：システム層（記憶ファイルに書く）、タスク層（何を＋技術＋制約）、フィードバック層（問題の指摘＋期待＋文脈）
- デバッグの 4 つのポイント：生の情報を渡す、期待 vs 実際、再現手順、確認済み項目
- 小刻み前進：低リスク・ターン数多めで新機能に最適。一括リファクタリング：高リスク・ターン数少なめでテストの裏付けが必要
- コンテキスト管理の 4 つの技：重要情報の固定、要約して引き継ぐ、1 対話 1 タスク、思い切って新しいセッションを開く

# 第四十四章：0 から 1 への実践——プロジェクト完成までの全プロセス

理論は学び終えました。いよいよ手を動かす番です。この章は単なる「写経」チュートリアルではなく、「どう考えるか、どう聞くか、どう確かめるか」という実践の方法論です。難易度の異なる 3 つのプロジェクトを使って、要件から完成品までの思考プロセスのすべてをお見せします。どのツールを使っていても、この流れはそのまま使えます。

## 一、プロジェクト 1：ミニマル個人ツールサイト（入門レベル）

### 要件を考える

目標：実用的な小さなツールへの入り口を 3 4 個並べて表示する、個人ツールサイトの Web ページを作ること。フロントエンドだけのプロジェクトで、バックエンドもデータベースも不要です。ポイントはツールそのものの複雑さではなく、「要件から完成品まで」の流れを一通り体験することです。

要件の 5 要素に沿って整理します。Who——個人で使う。What——ツールへの入り口を並べて見せる Web ページ。Why——よく使うツールを 1 ページにまとめてアクセスしやすくする。How——単一ファイルの HTML+CSS+JS で、フレームワークは使わない。Constraint——30 分以内に完成、デプロイは不要。

### 指示を出す

Think Before Coding の原則に基づき、指示には AI が目標を理解するのに十分な情報を含めます。ただし、実装の細部まで指定する必要はありません。

```
個人ツールサイトのWebページを作成してください。要件：
1．単一HTMLファイル（CSSとJSをインライン）、フレームワーク不使用
2．ダークテーマ、モダンなミニマルスタイル
3．4つのツールカードを表示：
   - テキスト要約ツール
   - 文字数カウントツール
   - カラーピッカー
   - QRコード生成ツール
```

```
4．各カードにアイコン、タイトル、短い説明
5．カードはホバーアニメーション付き
6．レスポンシブ、スマホで使える
7．上部にタイトルバー
完全なHTMLファイルを直接示してください。
```

### 検収と反復

AI からコードが出てきたら、反復検証の 3 層チェックで受け入れます。第 1 層・機能の検証：ファイルはブラウザで開けるか？ 4 枚のカードはすべて表示されるか？ ホバーのアニメーションは正しく動くか？ 第 2 層・境界の検証：スマホ画面のサイズでもレイアウトは崩れないか？ ウィンドウが細いときカードは折り返されるか？ 第 3 層・アーキテクチャの検証：コードは読みやすいか？ CSS は 1 つの style タグの中に収まっているか？ 目につく重複や無駄はないか？

カードの入り口だけでなく、ツールの機能そのものも実装したい場合は、少しずつ足していきましょう。1 回に 1 つのツールの機能を追加し、確認してから次へ。これが「小刻み前進」戦略の実戦での使い方です。

## 二、プロジェクト 2：フルスタック家計簿システム（中級レベル）

### 要件の検討と PRD

目標：フロントエンド、バックエンド、データベースを備えたフルスタックの家計簿アプリ。このプロジェクトは最初のものよりずっと複雑で、第四十章の要件エンジニアリングの手法が必要になります。

まず PRD を書きます。第四十章第 3 節のテンプレートに沿って、プロジェクト概要、機能リスト、技術案、データモデル、開発計画をまとめましょう。技術案の選択では Simplicity First の原則を考慮します。フロントエンドは React を使うが Redux などの複雑な状態管理は入れない。バックエンドは Node.js を使うがマイクロサービスにはしない。データベースは開発段階では PostgreSQL ではなく SQLite を使う、という具合です。

開発計画は 5 ステップに分解します。Step 1：バックエンド API の骨組み（Express サーバー＋ SQLite ＋ CRUD のエンドポイント）。Step 2：フロントエンドの基本ページ（記録リスト＋追加フォーム）。Step 3：フロントとバックのつなぎ込み（API 呼び出し＋データ表示）。Step 4：カテゴリと絞り込み機能。Step 5：月次の集計と簡単なグラフ。各ステップが終わるごとに「使えるもの」になっているのが理想です。

### プロジェクト記憶ファイルを設定する

コーディングを始める前に、まずプロジェクト記憶ファイルを作成します。これがコンテキストエンジニアリングの中核となる実践です。対話が始まるたびに AI がプロジェクトの背景を把握できるようにします。

```
# プロジェクト：フルスタック家計簿アプリ
## 概要
収支記録・カテゴリー・月次集計に対応した個人家計簿アプリ。
## 技術スタック
- フロントエンド：React + TypeScript + Vite
- バックエンド：Node.js + Express + TypeScript
- データベース：SQLite（better-sqlite3経由）
- 前後で共有する型定義
## APIの約束事
- 基本パス：/api
- 応答形式：{ success: boolean, data: any, error: string }
- RESTfulスタイル
## データモデル
Record { id, amount, type, category, note, date }
## コーディング規約
- TypeScriptのstrictモード
- インデントはスペース2つ
- 関数にはJSDocコメント
- コメントとコミットメッセージは中国語で
```

## 少しずつ実装する

開発計画に沿って、各ステップを独立した対話で完成させます。最初のステップの指示例は次のとおりです。

```
バックエンドAPIの骨格を作成する必要があります。
要件：
1. Expressサーバー、ポート3001
2. SQLiteデータベース、テーブル名records
3. recordsテーブルのフィールド：
   id (TEXT PRIMARY KEY)
   amount (REAL)
   type (TEXT: 'income'|'expense')
   category (TEXT)
   note (TEXT)
   date (TEXT: YYYY-MM-DD)
4. CRUD API：
   GET /api/records - 全記録の取得
   POST /api/records - 記録の追加
   PUT /api/records/:id - 記録の更新
   DELETE /api/records/:id - 記録の削除
5. エラー処理ミドルウェア
完全なバックエンドコード構成を示してください。
```

この指示の特徴に注目してください。今やるべきこの 1 ステップ（バックエンドの骨組

み）だけを記述し、フロントエンドの要件を混ぜていない。フィールドとインターフェースを明示して（AI に十分な制約を与えつつ）、具体的な実装は AI に任せている。そして Goal-Driven Execution の原則に従い、「どう作るか」ではなく「何を作るか」を語っている、という点です。

AI からコードが出たら、すぐに検証します。サーバーを起動し、curl か Postman で各エンドポイントが正しく動くかをテストしましょう。問題がないことを確認したら、新しい対話を開いて Step 2（フロントエンドの基本ページ）に進みます。各ステップで「指示検証新しい対話」のサイクルを守ります。

## 三、プロジェクト 3：AI 拡張アプリ（上級レベル）

### プロジェクトの構想

目標：大規模モデル API を組み込んだアプリ。たとえば「AI 学習アシスタント」です。ユーザーが学習内容を入力すると、AI が要約を生成し、関連する知識ポイントを結び付け、学習リソースのおすすめを出します。このプロジェクトの特別なところは、あなたのアプリ自体が AI を呼び出すという点です。API 呼び出し、プロンプトエンジニアリング、エラー処理といった新しい問題に取り組む必要があります。

このプロジェクトが教えてくれる最も大切なことは、「AI に、AI を呼び出すコードを書かせる」ときは、プロンプトエンジニアリングに格段に気を配る必要がある、ということです。あなたのアプリが大規模モデル API を呼び出すときに使うプロンプトにも、本ガイドで論じてきた原則——明快、簡潔、目標駆動——がそのまま当てはまります。

### 重要な技術ポイント

このプロジェクトには、最初の 2 つのプロジェクトでは出てこなかった技術ポイントがいくつか登場します。

- API キーの管理：キーをコードにハードコードしては絶対だめ。環境変数で管理します
- ストリーミング応答の処理：大規模モデルの返答は通常ストリーミングで返るため、一文字ずつ表示する仕組みが必要です
- トークン使用量の制御：大規模モデルはトークン単位で課金されるため、アプリ側で使用量を抑える必要があります
- プロンプトテンプレートの管理：機能ごとに異なるプロンプトテンプレートを使うので、構造化して管理します

これらの技術ポイントは、すべて AI に実装を手伝わせられます。ただし、それぞれの「なぜ」を理解しておく必要があります。なぜキーをハードコードしてはいけないのか？ なぜストリーミング応答が必要なのか？ なぜトークン使用量を抑える必要があるのか？「なぜ」が分かってはじめて、AI の提案した方式が妥当かどうかを判断できます。

## 四、汎用の実践フローチェックリスト

プロジェクトの大小にかかわらず、次が汎用の AI プログラミング実践フローチェックリストです。新しいプロジェクトを始めるときは、このリストを一通りなぞって、重要なステップの漏れがないか確認しましょう。

| 段階 | やること | 対応する原則 |
|---|---|---|
| 1. 要件を考える | 要件の 5 要素を一通り見て、何を作るかを明確にする | Think Before Coding |
| 2. 要件の分解 | 大きな要件を、独立して検証できる小さなステップに分解する | Simplicity First |
| 3. PRD を書く | 構造化されたドキュメントにまとめ、参考資料にする | Goal-Driven |
| 4. 記憶ファイルを設定する | CLAUDE.md などのファイルを作成する | Harness Engineering |
| 5. 少しずつ実装する | 1 ステップ 1 対話。「指示検証次へ」 | Surgical Changes |
| 6. テストと検証 | 各ステップ完了後すぐに機能の正しさをテストする | Iterate & Verify |
| 7. コードレビュー | アーキテクチャの妥当性、セキュリティ、保守性をチェックする | Iterate & Verify |
| 8. 反復と改善 | テストとレビューの結果をもとに改善を重ねる | Iterate & Verify |

AI プログラミング汎用実践フローチェックリスト

**やってみよう：今週「プロジェクト 1」を完成させよう**

第四十四章第 1 節の「個人ツールサイト」を、あなたの最初の AI プログラミングの宿題にしましょう。5 分かけて 5 要素を確認し、AI に指示を送り（どのツールでも構いません）、3 層の検証で受け入れます。30 分で完成させましょう——あなたの最初の AI プログラミング作品の誕生です。終わったらきちんと保存しておいてください。「読者」から「実践者」へ変わった記念品になります。

**本章のまとめ**

- プロジェクト 1（入門）：フロントエンドだけのツールサイト——全プロセスを体験。30 分で完成できる
- プロジェクト 2（中級）：フルスタック家計簿システム——PRD ＋記憶ファイル＋ 5 ステップの段階的実装。各ステップで「使えるもの」に
- プロジェクト 3（上級）：AI 学習アシスタント——AI を呼び出すコードを AI に書

かせる。プロンプトエンジニアリングに注意

- 汎用フロー 8 ステップ：要件検討分解 PRD 記憶ファイル段階的実装テストレビュー反復
- 各ステップは「指示検証新しい対話」のサイクルに従う

# 第四十五章：品質保証——AI が書いたコードを信頼できるものに

*AI* が書くコードにはひとつ特徴があります。たいてい「正しそうに見える」のです。変数の命名は規範的、コメントは揃っていて、ロジックも明快。しかし「正しそうに見える」は「本当に正しい」と同じではありません。*AI* コードに潜む問題は、手書きのコードより見つけにくいことがよくあります。表面の品質が高すぎて、人は油断しやすいからです。この章では、*AI* コードのために効果的な品質保証の仕組みを作る方法を教えます。テスト・レビュー・バージョン管理という *3* つの防線で、「正しそうに見える」を「本当に正しい」に変えましょう。

## 一、テスト駆動：AI にテストを書かせる

テストは AI プログラミングの品質保証における第一の防線です。朗報があります。AI にテストを書かせるのは、機能のコードを書かせるより簡単で、信頼性も高いということです。テストの期待動作は明確なので、「正しそうに見えて実際は違う」コードが生まれにくいのです。

テスト駆動の AI プログラミングの流れはこうです。まず AI に機能のコードを書かせ、基本動作の検証が通ったら、すぐにテストケースを書かせます。テストケースは正常系、境界値、異常系をカバーすべきです。そのうえで、テストの結果に応じて AI に機能コードを修正させます。この流れの良いところは、テストが一度できあがれば、それが持続的な検証ツールになることです。その後のどんな変更も、テストを実行してリグレッションが起きていないか確認できます。

```
calculate_discount関数のユニットテストを書いてください。
テストケースは以下をカバーすべきです：
1. 正常系：
   - 100元の商品を20%引き -> 80
   - 200元の商品を50%引き -> 100
2. 境界値：
   - 0元の商品を20%引き -> 0
   - 100元の商品を値引きなし -> 100（割引なし）
   - 100元の商品を0%引き -> 100（定価）
3. 異常系：
```

```
   - マイナスの価格 -> エラーを投げること
   - 割引率 > 1 -> エラーを投げること
   - 数値以外の入力 -> エラーを投げること
Vitestフレームワークを使い、完全なテストファイルを示してください。
```

この指示の特徴に注目してください。テストケースの 3 分類（正常・境界・異常）を明示していて、AI が「都合の良いところだけテストする」余地を与えていません。AI はときどき正常系のテストだけ書いて「手早く済ませよう」とします。すべての分類を明示すれば、この問題を避けられます。

## 二、コードレビューのテクニック

テストでカバーしていても、AI が生成したコードを人間がレビューする作業は欠かせません。コードレビューとは 1 行ずつ読むことではありません。時間がかかるうえ、あなたのスキルに寄りかかりすぎます。効果的な AI コードレビューでは、次の重要な観点に注目します。

| レビュー観点 | 何に注目するか | よくある問題 |
| --- | --- | --- |
| セキュリティ | 入力バリデーション、キーの取り扱い、SQL インジェクション対策 | AI は入力チェックを忘れがち。キーをコードにハードコードしてしまう |
| エラー処理 | 例外をキャッチしているか、エラーメッセージは適切か | AI は空の catch ブロックや console.log だけで済ませがち |
| パフォーマンス | N+1 クエリや無限ループなど、明らかな性能問題がないか | AI はときどき $O(n^2)$ のアルゴリズムを書いてくる |
| 保守性 | コード構造は明快か、命名は意味を持つか | AI はときどき抽象化しすぎたコードを生成する |
| 依存関係の管理 | 導入した依存は本当に必要か、バージョンは明確か | AI は不要なライブラリを入れがち |

AI コードレビューの重要な観点

レビューですべて自分で 1 つずつ確認する必要はありません。一次レビューは AI に任せられます。指示の例は「このコードのセキュリティ、エラー処理、パフォーマンスをレビューし、潜在的な問題を指摘してください」。AI のセルフレビュー能力は向上を続けていて、人間のレビューを完全に置き換えることはできませんが、目につく問題の大半をふるい落とすのには役立ちます。

## 三、バージョン管理：AI プログラミングにおける Git の役割

AI プログラミングにおいて Git の役割は、従来のプログラミングよりもさらに重要です。AI の変更はどれも予期しない影響を持ちうるので、頼れる「後悔しないための薬」が必要です。バージョン管理こそが、その薬です。

AI プログラミングにおける Git のベストプラクティスは「こまめにコミット、小刻みに前進」です。検証できる機能がひとつ完成するごとにコミットします。そうすれば、AI のある変更で問題が出ても、直前の使える状態へ正確に戻れ、すべての作業をやり直す必要がありません。

もうひとつの実用的なテクニックが「ブランチでの隔離」です。結果の読めない AI の変更（一括リファクタリングなど）は、まず新しいブランチを作ってそこで行います。結果に満足すればメインブランチへマージし、満足できなければブランチを削除するだけで、メインブランチには影響しません。メインブランチで何度も修正を重ねるより、はるかに安全です。

1. AI に修正させる前に、作業ツリーがきれいな状態（未コミットの変更がない）であることを確認します。そうしておけば、AI の修正が気に入らなければ git checkout . ですぐに戻せます。

2. AI の修正のあと、検証が通ったらすぐにコミットします。コミットメッセージには今回の変更で何をしたかを書きます。たとえば「feat: ユーザー登録機能を追加」のような具合です。

3. 一括リファクタリングでは、まずブランチを作成します：git checkout -b refactor/auth-system。ブランチ上で修正を完成させ、テストが通ってからマージします。

4. diff を確認する習慣をつけましょう。AI が修正するたびに git diff で何が変わったのかを見ます。問題の発見に役立つだけでなく、コードの学びにもなります。

## 四、よくある落とし穴とその回避ガイド

AI プログラミングには繰り返し現れる落とし穴がいくつかあり、ほぼすべての初心者がそこを踏みます。あらかじめ知っておけば、遠回りをかなり減らせます。

### 落とし穴その 1：信じすぎの罠

「AI が書いたコードはプロっぽく見えるから、きっと大丈夫だろう」——これは最も危険な罠です。AI が生成するコードは表面がとてもきれいです。命名は規範的、コメントは揃い、構造は明快。この「表面的なプロっぽさ」が人を油断させます。しかし表面のきれいさは、ロジックの正しさを意味しません。解毒剤：判定は必ずテストによる検証で行い、「見た目が正しそうか」で判断しないこと。

### 落とし穴その 2：肥大化の罠

「AI がコードを書いてくれるなら、機能を盛りだくさんにしてもらおう」——これはコードの肥大化を招きます。AI は「おまけ」を付けるのが得意です。ログインページを頼むと、ユーザー管理システム一式を渡してきます。余計な機能は複雑さを増すだけでなく、バグが出る確率も上げます。解毒剤：Simplicity First の原則を厳守し、AI に「この機能だけ作って、余計なものは追加しないで」とはっきり伝えること。

### 落とし穴その 3：理解ズレの罠

「AI なら私の言いたいことを分かってくれるはず」——そうとは限りません。AI の理解力は確かに高いのですが、曖昧な要件に対してはまだ「推測」します。推測が外れたら、出てくるコードは間違いです。厄介なのは、外れた推測から生まれたコードがたいてい「もっともらしく見える」ことで、理解してくれたと錯覚してしまうことです。解毒剤：指示には具体的な詳細を十分に盛り込み、AI が「推測」する余地を減らすこと。

### 落とし穴その 4：コンテキストの罠

「前の対話で言ったから、AI は覚えているはず」——おそらく覚えていません。対話が長くなったり新しいセッションを開いたりすると、AI は最初のコンテキストを失います。重要な制約がプロジェクト記憶ファイルに書かれていなければ、その後の変更で無視される可能性が高いのです。解毒剤：重要な制約はプロジェクト記憶ファイルに書き込み、対話の記憶に頼らないこと。

**四大落とし穴まとめ**

信じすぎの罠——「正しそう」を鵜呑みにしない。肥大化の罠——AI は「おまけ」を付けたがる。理解ズレの罠——曖昧な要件は「推測」される。コンテキストの罠——会話の記憶は「忘れる」。対応する解毒剤は、テストによる検証、余計なものを削る、細部を十分伝える、記憶ファイルに書き込む、の 4 つです。

**やってみよう：プロジェクトに 3 つの防線を築こう**

すでにプロジェクト 1（ツールサイト）に着手しているなら：(1) AI にコア関数の 3 種類のテストケースを書かせる。(2) 5 つの観点（セキュリティ / エラー処理 / パフォーマンス / 保守性 / 依存関係）でコードを一度レビューする。(3) git でプロジェクトを初期化し、最初のコミットをする。3 つの防線がそろって、はじめてあなたのプロジェクトは本当の意味であなたのものになります。

**本章のまとめ**

- AI コードの「正しそう」≠「本当に正しい」——表面の品質が高すぎるかえって油断しやすい
- AI にテストを書かせる：正常 / 境界 / 異常の 3 種類のケースを明示して「選択的テスト」を防ぐ

- レビューの 5 観点：セキュリティ、エラー処理、パフォーマンス、保守性、依存関係の管理
- Git は「後悔しないための薬」：こまめにコミット、小刻みに前進、ブランチで隔離、diff をこまめに見る
- 四大落とし穴と対策：信じすぎ（テストで検証）、肥大化（余計なものは削る）、理解ズレ（細部まで伝える）、コンテキスト（記憶ファイルに書く）

# 第四十六章：ステップアップへの道——使い手からアーキテクトへ

第 37□44 章の方法論をマスターして、AI を使ったプロジェクト開発をスムーズにこなせるようになったら、次は何を目指すのでしょう？この章では、「AI ツールのユーザー」から「AI システムアーキテクト」へのステップアップの道を探ります。この段階での関心事はもはや「どうすれば AI にいいコードを書かせられるか」ではなく、「どうすれば AI を開発プロセス全体にうまく溶け込ませられるか」です。

## MCP プロトコルとツール拡張

MCP（Model Context Protocol）は、2025 年に登場し、2026 年にかけて徐々に普及したオープンなプロトコルで、AI モデルと外部ツールの間の通信標準を定めています。簡単に言えば、MCP は AI に「ツールを使う」力を与えます。コードを生成するだけでなく、コマンドの実行、データベースへの照会、API の呼び出し、ファイルシステムの操作までできるようになるのです。

MCP が登場する前は、AI ツールごとにツールの統合方式がバラバラで、互換性がありませんでした。MCP の登場が意味するのはこうです。MCP 標準に対応したツールは 1 つ作れば、MCP に対応したどの AI ツールからでも使える。これでツール統合のコストがぐっと下がり、AI の能力の範囲も「テキストの生成」から「アクションの実行」へと広がりました。

ステップアップを目指すユーザーにとって、MCP の意義は「AI のためにカスタムのツール能力を作れる」という点にあります。たとえば、チームに独自のデプロイシステムがあるなら、MCP ツールを 1 つ書けば、AI がそのデプロイシステムを直接呼び出せるようになり、いちいち人がターミナルに切り替えて操作する必要がなくなります。この「AI がツールを呼び出す」能力こそ、Agentic Engineering のインフラなのです。

## カスタムワークフローと Skills

同じ AI プログラミングの流れを何度も繰り返すようになったら、それを「カスタムワークフロー」や「Skills」として固めてしまいましょう。たとえば、チームに「まずセキュリティをチェック、次にパフォーマンス、さらに保守性をチェック」という標準のコードレビ

ュー手順があるとします。この手順を Skill として書いておけば、AI はコードレビューのたびに自動でこの流れに沿って進めてくれます。

カスタムワークフローの核心的価値は「一貫性」です。毎回同じ基準どおりに実行されることを保証し、会話の長さや文脈の違いでブレることがありません。これはチーム開発ではとくに大切です。メンバーみんなが同じワークフローを使えば、生まれてくるコードのスタイルや品質基準もそろっていきます。

カスタムワークフローの作り方はツールごとに違います。Cursor は Rules と Custom Instructions に対応し、Claude Code はカスタムコマンド、Replit は Runme に対応しています。しかし核心の考え方は同じ。あなたのベストプラクティスを「AI が実行できる指示テンプレート」としてコード化するのです。

## マルチエージェント協働のパターン

2026 年に入って、マルチエージェント協働はホットな話題になりつつあります。核心の発想はこうです。タスクを 1 つの AI に任せきりにするのではなく、役割の違う複数の AI に協力させる。たとえば、1 体はコードを書き、1 体はテストを書き、1 体はコードレビューを担当する——それぞれが持ち場に専念し、互いにチェックし合います。

マルチエージェント協働の強みは「分業」と「クロスチェック」にあります。各エージェントが自分の領域に集中できるので品質が上がり、エージェント同士の相互チェックで、自分では見落としがちな問題も見つけられます。一方、弱点もはっきりしています。調整のコストが高く、やり取りのオーバーヘッドが大きく、デバッグも複雑になるのです。

**ワンポイント：ひとりでも「マルチエージェント」ができる**

個人開発者なら、複雑なマルチエージェントシステムを組む必要はありません。あなたが役割を「演じ分ける」ことで、マルチエージェント協働を再現できます。たとえば、まず 1 つ目の会話で AI に機能のコードを書かせます。次に新しい会話を開いて（CLAUDE.md の文脈を渡して）、「コードレビュアーとしてこのコードをレビューして」とお願いします。最後にもう 1 つの会話で「レビューの指摘どおりにコードを直して」と頼みます。この「ひとり多役」方式なら、面倒なインフラなしで、マルチエージェント協働のメリットの大半を得られます。

## 継続学習の道

AI プログラミングは変化の速い分野です。今日のベストプラクティスは、明日には更新が必要かもしれません。学び続けるためのヒントをいくつか挙げます。

1 つ目は、二次的な解説ではなく、情報の「出どころ」を追うこと。Andrej Karpathy 氏の元ツイートと CLAUDE.md、Anthropic や OpenAI の公式ドキュメント、各ツールの更新履歴——これらが一次情報です。コミュニティの記事やチュートリアルはその解説やまとめであり、価値はありますが、ズレが生じていることもあります。一次資料を読む習慣をつけましょう。

2 つ目は、実践から学ぶこと。「学んでからやる」ではなく「やりながら学ぶ」です。新しい種類の問題に出会うたび（初めてのファイルアップロード、初めての外部 API の接続など）をきっかけに、そのつど関連知識を学びましょう。実践の中で学ぶほうが、場面から切り離した理論学習よりはるかに効率が高いのです。

3 つ目は、ツールへの感度は保ちつつ、新しもの追いには走らないこと。1　2 か月に一度は半日ほど時間を取って、最新のツールや動向をチェックしましょう。ただし、新ツールが出るたびに乗り換えるのは禁物です。判断基準は「そのツールは、自分のボトルネックを大きく改善してくれるか」です。そうでなければ、今の組み合わせで十分です。

4 つ目は、自分用のナレッジベースを作ること。複雑な問題を解いたときの対話、つまずいたときの教訓、うまくいった方法を、その都度記録しておきましょう。公開する必要はありません。これはあなた自身の「方法論ノート」です。積み重なるうちに、いちばん貴重な資産になります。

**やってみよう：「つまずきノート」を始めよう**

今日、ドキュメントを 1 つ作りましょう（メモアプリでも.md ファイルでも OK）。そして、これまでの AI プログラミングでぜひ忘れたくない心得を 3 つ書いてください。「聞く前にまず考える」のようなものでも構いません。今後、つまずくたび、新しい技を覚えるたびに、1 つずつ書き足していきます。半年後に見返したとき、今日の自分にきっと感謝するはずです。

**この章のまとめ**

- MCP は AI とツールをつなぐ通信標準：AI がコマンド実行・データベース照会・API 呼び出しをできるようにする
- カスタムワークフロー/Skills の核心は「一貫性」：ベストプラクティスを指示テンプレートにコード化する
- マルチエージェント＝分業＋クロスチェック。個人開発者は「ひとり多役」で代用できる
- 継続学習の 4 つのコツ：一次情報を読む、やりながら学ぶ、新しものを追いすぎない、個人ナレッジベースを作る

# 第四十七章：クイックリファレンスとリソース

この章は、これ *1* つで使えるクイックリファレンスです。原則をうろ覚えのときや、すばやく判断したいときは、直接ここを開いてください。「ポケットカード」として手元に置くのがおすすめです。*5* 大原則の早見表、ツール選択の決定木、よくある質問、学習リソース——どれも飛ばさずどうぞ。

## 思考モデルの早見カード

| 原則 | ひとことで | 操作のポイント |
|---|---|---|
| Think Before Coding | まず考えてから頼む | 3 つの質問に答える：何をする？結果はどんな形？制約は何？ |
| Simplicity First | 最小限のコードで問題を解く | 1 回に 1 機能だけ。不要な複雑さは切り捨てる |
| Surgical Changes | ピンポイントで直し、境界を越えない | 修正範囲を明確に指定。「ついでに直しておきました」は断る |
| Goal-Driven Execution | 「何をするか」を言い、「どうやるか」は言わない | 手順ではなく目標を渡し、道筋は AI に任せる |
| Iterate & Verify | 生成検証修正のループ | 3 層検証：機能、境界、アーキテクチャ |

5 大思考原則の早見カード

## ツール選択の決定木

AI プログラミングツールを選ぶときは、次の判断経路に沿って選びましょう。注意：「どれがいちばん優れているか」のランキングではなく、「どれが自分にいちばん合うか」の判断フレームワークです。

何をしますか？

ゼロベース、まず体験したいクラウド Builder (v0/Bolt/Replit) を選ぶ
日常の開発、コード補完が必要 IDE 内蔵 AI (Cursor/Trae) を選ぶ
大型プロジェクト、一括操作が必要ターミナル Agent (Claude Code) を選ぶ
プロトタイプで素早くアイデアを検証クラウド Builder か Trae SOLO を選ぶ

図 6：AI プログラミングツール選択の決定木

## よくある質問（FAQ）

Q1：プログラミングがまったくできませんが、AI プログラミングは使えますか?

使えます。2026 年の AI プログラミングツールは十分に強力で、自然言語で要件を伝えるだけで、AI がコードを生成してくれます。ただし 2 つアドバイスを。1 つ目は、クラウドの Builder ツール（v0 や Bolt など）から始めること。開発環境の設定がいりません。2 つ目は、使いながら基礎概念を学ぶこと。文法を完璧に知る必要はありませんが、AI が書いたコードの大意を「読める」ことが、きちんと検収するうえで大切です。

Q2：どのプログラミング言語を学ぶべきですか?

何をしたいかによります。Web サイトのフロントエンドなら JavaScript/TypeScript。データ処理や AI なら Python。企業向けのバックエンドなら Java か Go です。でも、もっと大事なことがあります。AI プログラミングの時代には、「言語」の重要度は下がりつつあるということです。文法の細かい部分は AI が引き受けてくれます。あなたに必要なのは「プログラムの構造」の理解、つまり関数とは何か、ループとは何か、データ構造とは何かです。これらの概念は言語をまたいで共通なので、一度身につければ、どの言語に乗り換えても早いのです。

Q3：AI が書いたコードに著作権の問題はありますか?

2026 年 8 月時点では、いまだに意見が分かれる法律上の問題です。一般的には、AI が生成したコードには明確な著作権者がいるとは見なされない司法管轄区が大多数です。とはいえ商用プロジェクトで使うなら、(1) 使う AI ツールの利用規約の知的財産権まわりの条項を確認する、(2) 重要なコードは人間が見直して手を加え、自分の創造的な貢献を含ませる、(3) 著作権で保護されている可能性のあるオープンソースコードを AI にそのままコピーさせない、ことをおすすめします。

Q4：AI はプログラマーに取って代わりますか?

当面、その心配はいりません。AI が置き換えるのは「プログラマー」という職業ではなく、「1 行ずつコードを書く」という作業です。電卓が数学者を消したのではなく、面倒な手計算から解放したのと同じことです。AI プログラミングツールによって、プログラマーは「コードを書く」から「問題を考える」へと移ります。これは実際には、プログラマーの価値の上限を引き上げています。これからの競争力は「何行書けるか」ではなく、「どれだけ複雑な問題を解けるか」にあるのです。

Q5：AI の出力が正しいかどうか、どう見極めればよいですか?

3 層検証法を使いましょう。第 1 層は機能検証——動くか? 入力と出力は合っているか?

これが基本中の基本です。第 2 層は境界検証——空の入力、極端に長い入力、異常な入力でどうなるか？ ここで隠れた問題が見つかります。第 3 層はアーキテクチャ検証——コードの構造は妥当か？ ハードコードはないか？ 拡張しやすいか？ ここにはある程度の技術的判断力がいります。第 3 層まで見極められなくても、最初の 2 層はやりましょう。動いて、境界でも問題がなければ、初学者の大半より先を行けています。

## おすすめ学習リソース

2026 年 8 月時点でのおすすめ学習リソースです。インターネットのリンクは変わることがあります。リンク切れのときは、タイトルで検索すればたいてい最新版が見つかります。

| 種類 | リソース | おすすめの理由 |
|---|---|---|
| 一次資料 | Andrej Karpathy 氏の CLAUDE.md | 5 大原則の源流。70 行ながら、どの 1 行も珠玉 |
| 一次資料 | Anthropic 公式ドキュメント（Claude Code） | ターミナルエージェントのベストプラクティスガイド |
| チュートリアル | Vibe Coding コミュニティがまとめた Skills コレクション | コミュニティの実践者たちがまとめた実戦テクニック集 |
| チュートリアル | Cursor 公式ドキュメントと YouTube チャンネル | IDE 内蔵 AI の権威ある使い方ガイド |
| コミュニティ | GitHub の awesome-claude-code | Claude Code のベストプラクティスとプラグイン集 |
| コミュニティ | Reddit の r/cursor コミュニティ | 世界中の Cursor ユーザーによる経験の共有と議論 |
| 基礎 | MDN Web Docs (developer.mozilla.org) | フロントエンド開発で最も権威あるリファレンス |
| 基礎 | Python 公式チュートリアル（docs.python.org） | Python 学習の公式のスタート地点 |

おすすめ学習リソース一覧

## おわりに：道具は手に、方法論は心に

本書の核心にある信念に立ち返りましょう。道具は 3 か月ごとに更新され、方法論は 3 年かけてようやく進化します。2026 年 8 月の今日、AI プログラミングツールの力は驚くほど強くなりました。ひと言で Web サイトを作り、説明文だけでフルスタックアプリを生み出し、指示 1 つで複雑なリファクタリングを仕上げられるのです。しかし、道具が強くなるほど、方法論の重みはむしろ増します。道具の強さが、あなたの一つひとつの判断を拡大増幅するからです。

要件が明確なら、AI は効率よく実現してくれます。要件があいまいなら、AI は同じ効率で「まちがったもの」を作ります。シンプル第一を貫けば、AI はアイデアの素早い検証を助けてくれます。欲張って全部入れようとすれば、AI は技術的負債を同じ速さで積み上げます。ピンポイントで指示すれば、AI は外科手術のようにコードを最適化します。指示があいまいなら、AI は大鉈を振るうようにバグを持ち込みます。

だからこそ本書は、道具の操作より方法論の解説に多くのページを割いたのです。道具が大切でないからではありません。方法論は道具の「掛ける数」だからです。方法論が良ければ道具の価値は大きくなり、方法論がゼロなら、どんなに強力な道具も「ゼロ × いくら」のままです。

もう一人でプログラミングするのではありません。あなたは指揮官、AI はあなたの軍団です。思考の質が軍団の戦闘力を決めます。今日から方法論で自分の考えを武装し、道具で自分の能力を拡大しよう——それが AI プログラミングのすべての秘密です。

## 参考文献

1. Andrej Karpathy. (2025). Vibe Coding. Social media post introducing the concept of vibe-driven programming with AI.

2. Andrej Karpathy. (2025). CLAUDE.md - Project guidelines for AI-assisted development. GitHub repository, 70+K stars.

3. Anthropic. (2026). Claude Code: Terminal-native AI programming. Official documentation and best practices.

4. Cursor Team. (2026). Cursor 3: The next generation of AI-first IDE. Release notes and documentation.

5. OpenAI. (2026). Codex CLI: Bringing AI to the terminal. Technical documentation.

6. Model Context Protocol (MCP) Specification. (2025-2026). Open protocol for AI-tool integration.

7. Vibe Coding Community. (2026). Community-curated skills and best practices for AI-assisted programming.

8. Spec-Driven Development (SDD) methodology. (2026). Emerging practices in specification-first AI programming.

# 八、Android 開発実践編（第 48-58 章）

# 第四十八章：Android を知る——OS から開発エコシステムまで

この決断、おめでとうございます——「スマホで遊ぶ人」から「スマホアプリを作る人」への一歩です。手を動かす前に、まず地図を広げておきましょう。*Android* はどこから来たのか？なぜ世界中の開発者が *Android* 向けにアプリを作るのか？あなたがキーボードを叩いてから、アプリがユーザーのスマホに届くまで、間で何が起きているのか？

## Android の生い立ち：「小さなロボット」から「世界最大の OS」へ

Android の物語は 2003 年に始まります。Andy Rubin というエンジニアが Android 社を興し、その目標は実に素朴なものでした。「カメラをもっと得意にする」スマート OS を作る、というものです。その後、2005 年に Google が同社を買収し、2007 年には何十社ものメーカーとともに Open Handset Alliance（OHA）を設立。そして 2008 年 9 月、最初の Android スマホ HTC Dream（G1）が正式に発売されました。

その後の話はご存じのとおりです。2026 年には、Android は世界でいちばんシェアの大きいモバイル OS となり、稼働するデバイスは数十億台に上ります。あなたのスマホ、テレビのセットトップボックス、カーナビの大画面、スマートウォッチも、おそらく Android が動いています。これはどういうことでしょう？あなたが書く 1 行 1 行のコードに、世界で最も多くの人に使われるチャンスがある、ということです。

**ワンポイント：Android のオープンソースの血**

Android の中核コード（AOSP、Android Open Source Project）はオープンソースです。どのメーカーも持っていって改造・カスタマイズできます。だから Xiaomi には MIUI（今の HyperOS）、Huawei には HarmonyOS の互換レイヤー、Samsung には One UI があるのです。これは開発者にとって良いことです。あなたが学ぶのは「Android 開発」であって「あるブランドの開発」ではない。1 つの技術を磨けば、どの端末でも通用します。

バージョン番号の歩みもおもしろいものがあります。1.5 の Cupcake（カップケーキ）から始まり、長らくメジャーバージョンにはデザートの名前が付いてきました——ドーナツ、エクレア、フローズンヨーグルト、ジンジャーブレッド……2019 年の Android 10 になって、ようやく数字だけの名前に変わりました。2026 年 8 月時点での最新正式版は Android 17 です。開発で本当に気にすべき「最低サポートバージョン」は、主流アプリなら Android 8（API

26） Android 10（API 29）の間に設定するのが一般的です。数年前の端末までカバーしてこそ、ユーザーの数が大きくなるのです。

| バージョン | コードネーム / 年 | 開発者にとって大事なこと |
|---|---|---|
| Android 8 (API 26) | Oreo ・2017 | 通知チャネル、バックグラウンド制限——多くの旧型端末の下限ライン |
| Android 10 (API 29) | 2019 | スコープストレージの登場。ファイルアクセスのルールが大きく変わる |
| Android 12 (API 31) | 2021 | Material You のダイナミックカラー。UI の見た目が大きく進化 |
| Android 13 (API 33) | 2022 | 実行時の通知権限。「通知を許可しますか?」のポップアップが登場 |
| Android 14-15 (API 34-35) | 2023-2024 | 予測型「戻る」、16KB ページ、プライバシーがさらに厳格化 |
| Android 16-17 (API 36-37) | 2025-2026 | より速いマイナーアップデートのペース、AI 機能の統合（Gemini 拡張など） |

Android バージョン略史（2026 年 8 月時点、開発での注目ポイント）

## どの言語で書く?：Java か、Kotlin か

Android 開発の歴史には、2 つの主流言語があります。Java は「建国の功労者」です。Android 誕生のときから使われてきました。Kotlin は 2017 年に Google が「第一級の言語」に指名した存在で、2019 年からは公式に「Kotlin-first」を明言。2026 年の今では、新規プロジェクトはほぼ全部 Kotlin で書かれ、既存プロジェクトもどんどん移行が進んでいます。

率直に言いましょう。完全な初心者なら、まっすぐ Kotlin を学び、Java には立ち戻らないことです。理由は 3 つ。1 つは、Kotlin のコード量は Java より 30 40% 少ないこと（同じ機能なら、Kotlin のほうが短くて明確）。2 つは、Null Pointer Exception という初心者の悪夢を、言語の仕組みレベルで防いでくれること。3 つは、最近のチュートリアル、公式ドキュメント、AI プログラミングツール（本書の前半を思い出してください）のすべてが Kotlin 基準になっていることです。Java と出会うのは、古いプロジェクトの保守や古いコードを読むときくらいです。

Java 風（くどい版）

```
public class MainActivity extends Activity \{
@Override
```

```
protected void onCreate(Bundle b) \{
super.onCreate(b);
setContentView(R.layout.activity);
\}
\}
```

Kotlin 風（すっきり版）

```
class MainActivity : ComponentActivity() \{
override fun onCreate(b: Bundle?) \{
super.onCreate(b)
setContent \{ App() \}
\}
\}
```

同じ内容なのに、Kotlin では波かっこや定型的な行がひと山減ります。第五十章で正式に学ぶので、今はこれだけ覚えてください。Android 開発の「標準語」の名前は Kotlin です。

## アプリはどうやって生まれるのか？

「アプリ開発」というと、パソコンに向かってひたすらコードを打ち込むイメージを持つ人が多いようです。実際には、アプリはアイデアからユーザーのスマホに届くまで、おおむね次のようなラインをたどります。

アイデア

設計（画面 + 機能）

コードを書く

コンパイルとパッケージ化（APK/AAB）

テスト（エミュレータ + 実機）

リリース（ストア/インストールパッケージ）

図 1：Android アプリがアイデアからストア公開までたどる全工程

この流れの中で、今すぐ知っておきたい概念が 2 つあります。

- APK：Android アプリのインストールパッケージ形式（.apk）。タップするだけでスマホにインストールでき、Windows の.exe のようなものです。普段の自分用テストや、友人に配って遊んでもらうときはこれを使います。
- AAB：Android App Bundle（.aab）。Google Play に公開するための「高機能パッケージ形式」です。ストアがユーザーの機種に応じて動的に切りつぶしてくれるので、誰もがより小さいサイズでダウンロードできます。自分ではインストールできず、ストアに預けるしかありません。

それから、何度も耳にすることばがもう 1 つあります。SDK（Software Development Kit、ソフトウェア開発キット）です。Android 公式が用意する開発装備の一式——各バージョン

の API ライブラリ、デバッグツール、エミュレータのイメージなど——を指します。コードの中で使う Activity や Button といったクラスは、みな SDK から来ています。

## Android 技術スタックの全体図：あなたの「武器リスト」

学びに入る前に、2026 年の主流 Android プロジェクトの標準装備をざっと見ておきましょう。今は分からなくて当然です。本書を読み終えて戻ってくれば、すっきり見えてくるはずです。

| 層 | 主流の選択（2026 年） | 何のためのもの？ |
|---|---|---|
| 言語 | Kotlin 2.x | ロジックを書く言語 |
| UI | Jetpack Compose + Material 3 | コードで UI を描く（本書のメインライン） |
| アーキテクチャ | MVVM + ViewModel | UI とロジックを分けて、コードの混乱を防ぐ |
| データ保存 | Room（データベース）+ DataStore（設定） | データをスマホに保存する |
| ネットワーク | Retrofit + OkHttp + Kotlin コルーチン | サーバーと会話する |
| ビルドツール | Gradle + Kotlin DSL | コードをインストールパッケージにする |
| 開発環境 | Android Studio | Google 公式の「フルセット」IDE |

2026 年の Android 主流技術スタック早見

この英語の並びに気おくれしないでください。これらはキッチンの鍋やボウルのようなものです。今は「料理するにはキッチンへ」と分かれば十分で、各道具の使い方はこの先の章で 1 つずつ解き明かします。しかも朗報です。Android Studio で新規プロジェクトを作れば、この一式を自動でセットアップしてくれます。あとは料理に集中するだけです。

## Android vs iOS：開発者向け最初の「陣取り」の授業

手元に Android 端末と Apple 端末が両方あるかもしれません。開発者としては、両者の体験の違いを前もって知っておく価値があります。「どちらが上か」の派閥争いではなく、「どちらが自分のスタートに向いているか」という現実的な話です。

| 観点 | Android 開発 | iOS 開発 |
|---|---|---|
| 開発マシン | Windows / Mac / Linux いずれも可 | Mac 必須（Xcode は Mac 専用） |
| 開発ツール | Android Studio（無料） | Xcode（無料）＋ストア公開に年間 99 ドル |
| ストア審査 | 緩い。数時間　数日 | 厳しい。1 7 日があたりまえ |
| 端末に入れて試す | APK は自由に配れる、自由に入れられる | 配布は制限あり。テストには署名/TestFlight が必要 |
| 収益環境 | 広告 / 中国国内チャネルが中心 | App Store はアプリ内課金の文化がより定着 |
| 学習コスト | 概念が多い（4 大コンポーネントなど） | エコシステムは閉じているが体系は統一され、ドキュメントも充実 |

Android と iOS の開発パス比較（2026 年 8 月時点）

率直に言えば、お使いのパソコンが Mac でなければ、この問題はそもそも選ぶ必要がありません。Android が唯一の選択肢です。Mac をお持ちなら、本書の第 8 篇と第 9 篇は並行して読めます。プログラミングの考え方は共通なので、片方を学んでからもう片方に入ると、ぐっと速くなります。

**やってみよう：あなたのスマホの中の「Android」をのぞいてみる**

スマホの［設定］［端末情報］（「デバイス情報」など）を開き、「Android バージョン」の欄を探します——そこを何度か連続タップすると、システムの隠しミニゲームが現れます。バージョン番号を書き留めて、上のバージョン表と見比べ、どの世代か確かめてみましょう。あなたのスマホこそ、最初の「開発テスト機」です。

**この章のまとめ**

- Android は 2008 年誕生。2026 年には世界最大シェアのモバイル OS となり、稼働デバイスは数十億台
- 最新正式版は Android 17。主流アプリの最低サポートは Android 8　10。数年前の端末までカバーしてユーザー数を確保
- 新規プロジェクトはとにかく Kotlin：コード量 30% 減、ヌルポ防止、公式のお墨付き
- APK は自分で入れる用、AAB はストア公開用。SDK は公式の開発装備一式
- 2026 年の標準技術スタック：Kotlin ＋ Compose ＋ MVVM ＋ Room ＋ Retrofit ＋ Gradle ＋ Android Studio

# 第四十九章：開発環境を整える——Android Studio と最初のプロジェクト

仕事を仕上げるには、まず道具をととのえること。この章では「キッチン」を組み立てます。*Android Studio* を入れて、エミュレータを動かし、人生初の *Android* プロジェクトを作り、プロジェクト構造をほどいて覗いてみます。環境が整えば、自分の書いたコードが「押せるボタン」に変わる瞬間に、初めて出会えます——この瞬間を一生覚えているという人は少なくありません。

## Android Studio をインストールする

Android Studio は Google 公式の統合開発環境（IDE）です。無料で、機能はフルセット。コードを書く、UI プレビューを見る、デバッグする、パッケージ化する——そのすべてがここに揃っています。インストールは 4 ステップです。

1. 公式サイト developer.android.com/studio を開き、最新の安定版をダウンロードします（2026 年 8 月時点では Narwhal 以降のバージョン。Windows/Mac/Linux 版があります）。
2. インストーラを起動して、ひたすら「次へ」——セットアップウィザードが Android SDK とエミュレータのコンポーネントを自動で入れてくれます。
3. 初回起動時に、案内に従って SDK コンポーネントをダウンロードします（デフォルトのままで OK。数分と数 GB の空き容量がいるかもしれません）。
4. 終わるとウェルカム画面が現れます——おめでとうございます、「キッチン」の完成です。

**注意：ハードウェア要件を頭に入れておく**

Android Studio は「メモリを食べる」タイプです。メモリは 16GB からがおすすめ（8GB でも動きますが、エミュレータを起動すると重くなりがち）。ディスクは 20GB 以上確保し、CPU が仮想化対応（BIOS で VT-x/AMD-V を有効化）か確認しましょう。パソコンがどうしても古い場合は、エミュレータの代わりに「実機デバッグ」を使えば（この章の第 4 節で解説します）、むしろサクサク動きます。

## 最初のプロジェクトを作る

ウェルカム画面で New Project をクリックし、Empty Activity テンプレート（いちばんすっきりした初心者向けテンプレート）を選んで、3 つの項目を入力します。

| 設定項目 | 入力する内容 | 説明 |
|---|---|---|
| Name | My First App | アプリ名。スマホのホーム画面に表示される |
| Package name | com.example.myfirstapp | 世界で一意の識別子。慣習は「ドメインの逆書き」。まずはデフォルトのままで |
| Minimum SDK | API 26（Android 8.0） | 最低サポートバージョン。95% 以上のデバイスをカバー |

プロジェクト作成で入力する 3 つの項目

Finish を押すと、Android Studio が Gradle で依存関係を自動ダウンロードし、プロジェクトをビルドします（初回は数分待ちますが、2 回目からはぐっと速くなります）。ビルドが終われば、動くアプリの完成です——まだ「Hello Android!」と 1 行表示するだけですが。

**ワンポイント：パッケージ名のツボ**

パッケージ名（package name）はアプリの「身分証番号」のようなもので、世界中で重複が許されず、一度公開したら変更できません。個人の練習なら com.example.xxx で十分。正式な製品では、自分のドメインの逆書き（com. あなたの名前. アプリ名など）に変えます。com.example のままで公開すると——ストアに拒まれます。

## 動かしてみる：エミュレータと実機

コードが書けたら、それを動かす「スマホ」が要ります。選択肢は 2 つ。

### エミュレータ（Emulator）

エミュレータは、パソコンの中に Android スマホを 1 台仮想で作る仕組みです。Android Studio で Device Manager Create Device と開き、端末を 1 台選び（Pixel シリーズがおすすめ）システムイメージを 1 つ選ぶ（Google APIs 付きのもの）ダウンロード起動します。そしてツールバーの緑の三角 Run を押せば、数秒後に「スマホ」の画面にあなたのアプリが現れます。

### 実機デバッグ（こちらがおすすめ）

実機は、常にいちばんリアルなテスト環境です。スワイプの感触も、本物のネットワークも、本物のセンサーも、エミュレータでは真似できません。Android の実機デバッグは簡

単です。

1. スマホで［設定］［端末情報］を開き、「ビルド番号」を 7 回連続タップして「開発者向けオプション」を有効にします。
2. ［開発者向けオプション］で USB デバッグをオンにします。
3. USB ケーブルでパソコンにつなぎ、スマホのポップアップで「USB デバッグを許可」を選びます。
4. Android Studio に戻ると、ツールバーのデバイスのドロップダウンにあなたのスマホが現れるので、Run を押します。

**落とし穴注意：USB デバッグはつけっぱなしにしない**

USB デバッグは、パソコンに「裏口」を開けておくようなものです。外で充電するときや、信用できないパソコンにつなぐときは、必ず USB デバッグをオフにして、他人にデータを読まれないようにしましょう。普段の開発でも、使い終わったらオフにする。それを習慣にしてしまいましょう。

## プロジェクト構造の解体：各ファイルの役割

プロジェクトを初めて開くと、左のファイルツリーに目が回るかもしれません。慌てないでください。日常の開発では、時間の 90% を次のいくつかのファイルだけで過ごすことになります。




図 2：初心者が必ず知るプロジェクト構造（合言葉：触るのは app/フォルダだけ）

- MainActivity.kt：アプリの「玄関」です。ユーザーがアイコンをタップすると、最初に目覚めるのがこれ。今のテンプレートでは「Hello Android!」を表示する役割を担っています。

- AndroidManifest.xml：アプリの「住民票」です。このアプリの名前、アイコン、持っている画面、使う権限（カメラ、位置情報など）をここで宣言します。

- res/フォルダ：リソース倉庫。画像、アイコン、文字列、色を置きます。文字をコードに直書きせずリソースファイルに置くのは、Android 開発の良い習慣です（言語やテーマの切り替えが楽になります）。

- build.gradle.kts：ビルドのレシピです。最低サポートバージョンや、依存するライブラリ（Compose など）を宣言します。

## さっそく試す：「Hello」を自分の名前に変える

MainActivity.kt を開くと、Compose テンプレートが生成したコードが見えます。Text(“Hello Android!”) という行を探し、引用符の中を自分の名前に変えてみましょう。たとえば Text(“こんにちは、たろう！これは私の初めてのアプリです”) として、Ctrl+R（Mac なら Cmd+R）で再実行します。数秒後、エミュレータにあなたの名前が表示されます。

```
setContent {
    MyFirstAppTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            // ↓ ここを自分の好きな言葉に変えてみて
            Text("こんにちは、たろう！これは私の初めてのアプリです")
        }
    }
}
```

おめでとうございます。あなたは「コードを変えるコンパイルインストール実行」という 1 周のサイクルを完了しました。このサイクルこそ、これから毎日の開発の日常です。勢いに乗って、もう少し改造してみましょう。文字を大きくする、色を変える——次の章では、こうした「魔法」の正体を学びます。

## よくあるエラーの応急処置：初心者の最初 2 週間の「救急室」

環境構築と初回ビルドの段階では、次の 3 種類のエラーにぶつかる可能性が高いでしょう。慌てないでください。どれにも効く薬があります。

| エラーの症状 | よくある原因 | 効く薬 |
|---|---|---|
| Gradle sync failed / 依存関係のダウンロードが遅い | リポジトリへのネットワーク接続が不安定 | Gradle に国内のミラーリポジトリ（Alibaba Cloud/Tencent Cloud の maven ミラー）を設定する。「Gradle ミラーリポジトリ設定」などで検索し、手順どおりに直す |
| SDK コンポーネントが足りない | インストール時に SDK のダウンロードをスキップした | Settings SDK Manager で、足りないプラットフォームとツールを追加インストール |
| エミュレータが起動しない / 真っ黒な画面 | 仮想化が無効、イメージの破損 | BIOS で VT-x/AMD-V を有効化。エミュレータを削除して作り直す。または実機デバッグに切り替える |

初心者向け環境エラー応急処置表

**ワンポイント：エラー検索の方法論**

エラーメッセージの 1 行目が、いちばん価値のある手がかりです。それをそのまま検索エンジンにコピーし、「android studio」のキーワードを添えましょう。「環境が入らないのですが」といった漠然とした言葉ではなく、「Gradle sync failed: Could not resolve com.android.tools.build」のような具体的なエラー文を検索します。正確な問いにだけ、正確な答えが返ってくるのです。

**やってみよう：「開発の儀式」リストを完成させる**

(1)Android Studio を入れてエミュレータを動かす。(2) 自分のアプリを実機で 1 回動かし、スクリーンショットを残す。(3)Hello の文字を自分の名前に変える。(4) プロジェクトを圧縮して保存する。この 4 つが終わったら、友達にこう言えます。「私も Android 開発者になったよ（証拠写真つきで）」。

**この章のまとめ**

- Android Studio ＝公式の無料 IDE。メモリ 16GB ＋ディスク 20GB ＋仮想化の有効化がおすすめ
- 新規プロジェクトの 3 要素：名前、パッケージ名（世界で一意。com.example のまま公開しない）、最低 SDK（API 26 から）
- 実行方法は 2 つ：エミュレータ（パソコン内の仮想スマホ）と実機（USB デバッグをオン。使い終わったらオフに）
- プロジェクトの合言葉：触るのは app/フォルダだけ——MainActivity.kt は玄関、

Manifest は住民票、res/はリソース倉庫

- 開発の日常＝コードを変える Run 結果を見る、のくり返し

# 第五十章：Kotlin はじめの一歩——Android 開発の第一言語

この章で、いよいよ言語そのものを学びます。緊張はいりません。*Kotlin* は昔から「初心者にやさしい」ことで有名ですし、文法書を丸暗記する必要もありません。文法の *2* 割を押さえるだけで、日常のコードの *8* 割は読めて書けます。残りはプロジェクトを進めながら調べればよいのです。学習の原則を思い出してください。理解する＞暗記する、使える＞完璧、です。

## 変数と型：データに名前を付ける

プログラムとは「データ＋ロジック」です。変数は、データに名前を付けて保存しておく仕組みのこと。Kotlin には 2 つの宣言方法があります。

```
val name = "たろう"        // val: 読み取り専用の変数（デフォルトはこれがおすすめ）
var age = 18              // var: 変更できる変数（あとから変えられる）

age = 19                  // <i class="fas fa-check-circle mr-1"></i> varは変えられる
// name = "はなこ"         // <i class="fas fa-times-circle mr-1"></i> valは変えられない。コンパイルエラー

val score: Int = 100      // 型を明示的に書いてもいい
val pi = 3.14             // 書かなくても自動で推論される
```

よく使う型は数えるほど。1 枚の表で覚えてしまいましょう。

| 型 | 何を入れる？ | 例 |
|---|---|---|
| Int | 整数 | 42 |
| Double | 小数 | 3.14 |
| String | 文字列 | ”こんにちは” |
| Boolean | 真/偽 | true / false |
| List< T> | データの集まり | listOf(1, 2, 3) |

Kotlin のよく使う基本型

**ワンポイント：val 優先の原則**

Kotlin コミュニティの鉄則です。ふだんは val を使い、本当に必要なときだけ var にする。val で足りる場所に var を使うと、IDE が黄色い文字で警告してくれます。この習慣だけで「変数が意図せず書き換わる」系のバグのかなりの割合を防げます。コードの動きが少ないほど、頭の中で追いやすいのです。

## 関数：ロジックを「ボタン」にまとめる

関数とは、名前のついたロジックのひとかたまりです。リモコンのボタンのようなもの——押す（呼び出す）と、一連の動作が実行されます。Kotlin では fun で宣言します。

```
fun greet(name: String): String {
    return "こんにちは、$name！"      // $記号で変数を文字列に差し込む
}

println(greet("たろう"))          // 出力：こんにちは、たろう！

// 1行の関数は式として簡潔に書ける
fun add(a: Int, b: Int) = a + b

println(add(3, 5))               // 出力：8
```

ポイントはいくつか。引数はかっこの中に書く（名前が先、型が後、コロンで区切る）。戻り値の型はかっこの後ろに書く。文字列テンプレートの $ 変数名は、Kotlin 名物の便利機能——Java のように「文字列＋変数」とつなぎ合わせる必要はありません。

## 条件とループ：プログラムに「考える」力をつける

プログラムの賢さは 2 つの力から生まれます。判断できること（if/when）と、繰り返せること（for/while）です。

```
// 判断：if / else
val age = 17
if (age >= 18) {
    println("大人")
} else {
    println("未成年者")
}

// 多分岐：when（switchの百倍使いやすい）
val score = 85
val level = when {
    score >= 90 -> "優秀"
    score >= 60 -> "合格"
    else -> "その調子！"
```

```
}

// 繰り返し：forループ
val fruits = listOf("りんご", "バナナ", "オレンジ")
for (fruit in fruits) {
    println("$fruitが大好き")
}
```

when は Kotlin でいちばんよく使う多分岐の書き方です。Android 開発では毎日お世話になります（たとえば状態に応じて別々の画面を出す、など）。

## Null 安全性：Kotlin の「命綱」

Java の時代でいちばん有名なバグが、NullPointerException（ヌルポインタ例外）です。何も入っていない「空」のものにプログラムがアクセスして、その場でクラッシュします。統計上も、Android のクラッシュランキングの常連 1 位です。Kotlin はこれを言語の仕組みレベルで解決しました。デフォルトでは null を許さないのです。

```
var name: String = "たろう"   // 普通の型：nullは絶対に許されない
// name = null                // <i class="fas fa-times-circle mr-1"></i> コンパイラが直接止めてくれる

var nickname: String? = null // 疑問符を付けると：nullを許す
nickname = "たーくん"

// null許容型を使うときは、安全に扱う必要がある：
println(nickname?.length)    // safe call：nullならnullを返すだけで、落ちない
val len = nickname?.length ?: 0   // Elvis演算子：nullのときはデフォルト値
println(len)
```

**注意：？と?: を取り違えない**

String? は「null を許す」、a ?: b は「a が null なら b を使う」という意味です（形がエルヴィスの髪型に似ているので、Elvis 演算子と呼ばれます）。書き始めの頃にコンパイラから「null 関連のエラー」が出たら、たいていは null 許容型の扱いを忘れているせいです。それは、言語があなたを守ってくれている合図なのです。

## クラスとオブジェクト：「部品」を作る鋳型

クラスはオブジェクトの「鋳型」で、オブジェクトはクラスから作った「製品」です。Android の世界にあるもの——画面、ボタン、データ——は、すべてオブジェクトです。Kotlin には data class（データクラス）という便利な道具があり、データを「入れる」ことに特化していて、Java なら数十行になるのを 1 行で済ませます。

```
// 普通のクラス：鋳型
```

```
class Dog(val name: String) {
    fun bark() = println("$name ワンワン！ ")
}

val dog = Dog("ポチ")       // オブジェクトを作る
dog.bark()                  // ポチ ワンワン！

// data class：データを入れる箱（equals/copyなど付き）
data class User(val id: Int, val name: String, val email: String)

val me = User(1, "たろう", "zhangsan@example.com")
val twin = me.copy(name = "たろう丸")   // コピーして1つのフィールドだけ変える
```

この先、おこづかい帳を作るときは、1 件の記録がそのまま data class になります。data class Record(val amount: Double, val category: String, val date: String)。ほら、もう本物のプロジェクトのコード構造が書けるのです。

## ラムダと高階関数：Kotlin の「シンタックスシュガー」

Android のコードでは、{ ... } という「名前のない関数」を毎日見かけます。これがラムダ（匿名関数）です。Button の onClick = { count++ } も 1 つのラムダで、「クリックされたら何をするか」をひと塊にして、ボタンに手渡しています。

```
// 普通の書き方：名前付き関数を定義してから渡す
fun double(x: Int) = x * 2
val result = listOf(1, 2, 3).map(::double)

// lambdaの書き方：その場で定義。手軽でストレート
val result2 = listOf(1, 2, 3).map { it * 2 }   // it = 現在の要素

// 絞り込み + 変換の連携（実際のプロジェクトで非常によくある形）
val bills = listOf(12.5, 88.0, 5.0, 200.0)
val bigBills = bills.filter { it > 50 }.sorted()
// [88.0, 200.0]――「50を超える請求を並べ替え」
```

使用頻度の高い「コレクションの連係技」を覚えましょう。filter（しぼり込み）、map（変換）、sorted（並べ替え）、sum（合計）。この先のリスト処理の 90% は、この 4 つの組み合わせです。Kotlin の哲学はこうです。1 言で言い切れるロジックを、5 言で書く必要はない。

## 超初心者向けの文法早見カード

1 日で全部の文法を覚えようとしないこと。このカードを保存しておき、コードを書くときに照らし合わせれば、2 週間もすれば自然と身につきます。

| やりたいこと | 書き方 |
|---|---|
| 読み取り専用の変数を定義 | `val x = 1` |
| 変更できる変数を定義 | `var x = 1` |
| 関数を定義 | fun 名前 (引数): 戻り値の型 { ... } |
| 条件分岐 | if (条件) {...} else {...} |
| 多分岐 | when { 条件 -> 結果} |
| 繰り返し | for (x in リスト) { ... } |
| 文字列の組み立て | ” こんにちは、$name” |
| null の扱い | `x?.length ?: 0` |
| データクラスを定義 | `data class X(val a: Int, val b: String)` |

Kotlin 文法早見カード（初心者版）

**やってみよう：ソフトを 1 つも入れずに、まず Kotlin を 10 分遊ぶ**

ブラウザで play.kotlinlang.org（Kotlin 公式のオンライン練習場）を開き、この章のコードを順番に貼り付けて実行してみましょう。数字や文字を書き換えて、出力がどう変わるか観察します。この「遊んでから学ぶ」10 分のおかげで、第五十一章に入ったときの負担が半分になります。

**この章のまとめ**

- val を優先、var は慎重に。型は自動で推論されるが、コロンの後で明示してもよい
- 関数＝まとめたロジック。文字列テンプレートの $ は Kotlin 名物の便利機能
- 判断は if/when、繰り返しは for。when は Android 開発の「何でも屋」
- Null 安全性は Kotlin の命綱。ふつうの型は null を拒み、null 許容型は? / ?: での扱いが必須
- data class はデータを入れる箱。1 フィールド 1 行で、本物のプロジェクトで毎日使う
- 文法の 2 割で仕事の 8 割をこなす。まず使えるようになり、それから極める

# 第五十一章：UI 開発——Jetpack Compose 入門

*UI* とは、ユーザーの目に見えるすべてのものです。*Compose* が登場する前、*Android* の *UI* は *XML* で書かれていました——穴埋め問題のように面倒な作業です。*2021* 年、*Google* は *Jetpack Compose* を発表しました。*Kotlin* コードで直接 *UI* を「描く」方式で、書いたものがそのまま形になり、コードを *1* 行変えればプレビューがすぐ更新されます。*2026* 年の今、*Compose* は新規プロジェクトの揺るぎない主流です。この章では、「お絵描き」を学びます。

## Compose の発想：UI ＝関数

Compose の核心はひと言で言えます。UI ＝関数。画面に表示できるもの（文字、ボタン、画像）は、みな「コンポーザブル関数」（Composable）で、@Composable の印を付けます。それらを積み木のように組み重ねたものが、UI なのです。

```
@Composable
fun Greeting(name: String) {
    Text(text = "こんにちは、$name! ")     // ひとつの「文字ブロック」
}

// 入れ子の組み合わせ：大きいブロックに小さいブロック
@Composable
fun HomeScreen() {
    Column {                           // 縦に並べる
        Greeting(name = "たろう")
        Greeting(name = "はなこ")
        Button(onClick = { /* 押したら何をするか */ }) {
            Text("押して")
        }
    }
}
```

@Composable という「帽子」に注目してください。コンパイラに「これは UI の積み木ですよ」と伝えるためのものです。積み木は無限に組み合わせられます。それが Compose のすべての秘密です。

## 基本コンポーネント：文字、ボタン、入力欄、画像

Compose のコンポーネントは「積み木のブロック」です。いちばんよく使う 4 つを見てみましょう。

| コンポーネント | 役割 | 必須パラメータ |
| --- | --- | --- |
| Text | 文字を表示 | text（表示する文字） |
| Button | ボタン | onClick（クリック時のコールバック） |
| TextField | 入力欄 | value、onValueChange（入力変更のコールバック） |
| Image | 画像を表示 | painter（画像リソース） |

Compose の 4 大基本コンポーネント

```
@Composable
fun BasicWidgets() {
    Column {
        Text(text = "これはテキストです")
        Button(onClick = { println("押された") }) {
            Text("ボタン")
        }
        var input by remember { mutableStateOf("") }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("何か入力") }
        )
    }
}
```

入力欄の 2 行に出てきた remember { mutableStateOf(“”) }——これが Compose の「状態」の魔法です。この章の第 5 節でじっくり解説します。今は、顔見知りになってもらえれば十分です。

## レイアウトの 3 点セット：Column、Row、Box

部品は揃いました。どう並べましょう?「箱」を 3 つ覚えてください。

- Column：縦に並べる（食堂の行列のように）。
- Row：横に並べる（となり同士で並んで座るように）。
- Box：重ねて置く（トランプを積むように。バッジや重ね表示に便利）。

```
Column {
    Text("1行目")
    Row {
        Text("左")
        Text("右")
    }
    Box {
        Text("下の層")
        Text("上に重なる小さなバッジ")
    }
}
```

レイアウトをもっと見栄えよくしたい？コンポーネントに Modifier（修飾子）を付けましょう——積み木に「メイク」をするようなものです。

```
Text(
    text = "見栄えのするテキスト",
    modifier = Modifier
        .padding(16.dp)          // 四方に余白
        .fillMaxWidth()          // 幅いっぱいに
        .background(Color.Blue)  // 背景色
        .clickable { /* クリック */ }// クリックできる
)
// チェーン呼び出し：順番に意味がある（先に padding、後に背景。背景は余白を含まない）
```

**注意：Modifier の順番には意味がある**

padding(16.dp).background(Blue) と background(Blue).padding(16.dp) は、結果が違います。前者は「先に余白をつくってから塗る」（背景に余白を含まない）、後者は「先に塗ってから余白をつくる」（背景に余白を含む）です。初心者を悩ます「変な白いふち」は、たいていここから生まれます。迷ったら順番を入れ替えて試してみてください。プレビューがすぐ答えを教えてくれます。

## 状態：UI を「生きている」ものにする

静的な UI では物足りません。ボタンを押すたびに数字を +1 したり、入力欄で文字を打てるようにするには、状態（State）が要ります。Compose のルールはこうです。UI は、常に状態をそのまま写す。状態が変われば、UI は自動で描き直される。

```
@Composable
fun Counter() {
    // remember + mutableStateOf: 変わる値を覚えておく
    var count by remember { mutableStateOf(0) }

    Column {
```

```
        Text(text = "$count回押しました")
        Button(onClick = { count++ }) {   // 状態を変える
            Text("押して +1")                 // 画面は自動更新
        }
    }
}
```

3 行のコードで、生きているカウンターのできあがりです。ここが従来の書き方より Compose が気持ちいいところです。UI を手動で「更新」する必要はなく、データを変えるだけでよい。大事な 2 つの関数を覚えましょう。remember（画面の再構成で値が消えないようにする）と、mutableStateOf（「この値が変わったら描き直して」と Compose に伝える）です。

ユーザーがタップ

状態が変わる（count++）

Compose が変化を検知

画面を自動で再描画（再構成）

図 3：Compose の「状態 UI」一方向データフロー

## テーマとダークモード：1 回の設定で全体に効く

テンプレートプロジェクトにある MyFirstAppTheme { ... } がテーマです。Material 3 のデザイン規格に基づいて、色、フォント、形をまとめて管理してくれます。ダークモード対応は 3 ステップだけです。

1. テーマの中に、ライト用 / ダーク用の 2 セットの配色を定義する（テンプレートが ColorScheme を自動生成済み）。
2. システムがダークモードに切り替わると、isSystemInDarkTheme() が自動で判定する。
3. 色を直接書かず、テーマが提供する色（MaterialTheme.colorScheme.primary など）を使う。

**ワンポイント：Material 3 のダイナミックカラー**

Android 12 からの「Material You」は、アプリが壁紙から色を取り込めるしくみです。Compose テンプレートはデフォルトで対応しています（dynamicColor）。あなたのアプリは、ユーザーごとのスマホで「自動で着替え」ます。これぞ現代 Android のロマンチックなところです。

## プレビュー、アニメーション、ライブ編集：Compose の「チート機能」

開発体験を上げる Compose の「チート機能」3 種。一度使うともう戻れません。

- @Preview プレビュー：Composable に @Preview アノテーションを付けると、Android Studio の右側にすぐ UI が表示されます。コードを変えた結果がそのまま見えるので、いちいち実機に入れる必要がありません。
- ライブ編集（Live Edit）：2026 年の Android Studio では、エミュレータや実機の画面に一部の変更を「ホット反映」できます。文字の色を変える程度の調整なら、端末上で一瞬で変わります。
- アニメーション API：animateFloatAsState や AnimatedVisibility なら、数行のコードで要素をなめらかに動かせます。「上級者っぽさ」は、たいていこの数十行の差です。

```
@Composable
fun AnimatedBox() {
    var big by remember { mutableStateOf(false) }
    // 値が変わると目標値へ自動でなめらかに遷移
    val size by animateDpAsState(if (big) 120.dp else 60.dp)

    Box(Modifier
        .size(size)
        .background(MaterialTheme.colorScheme.primary)
        .clickable { big = !big }
    )
}
```

**ワンポイント：まず機能、アニメーションはそのあと**

初心者がよくやるのは、「UI がまだ正しく動いていないのに、アニメーションを気にし始める」という流れです。まず機能を通るようにして、最後にまとめて動きを足しましょう。これも「シンプル第一」の原則です。アニメーションは花を添えるもの、機能は雪の中の炭です。

**やってみよう：「気分ボタン」を作る**

テンプレートプロジェクトを、気分チェンジャーに改造しましょう。ボタンを 1 つ置き、押すたびにうれしいかなしいおこるの間をぐるぐる切り替えて、文字と背景色がそれに合わせて変わるようにします。ヒント：when で状態ごとに文言と色を選びましょう。できたら友達に送って、最初の「え、アプリ作れるの!?」をいただきましょう。

**この章のまとめ**

- Compose の発想：UI ＝ @Composable 関数。積み木のように組み合わせる
- 4 大コンポーネント：Text / Button / TextField / Image
- レイアウト 3 点セット：Column は縦並び、Row は横並び、Box は重ね置き
- Modifier はチェーンで「メイク」：padding、fillMaxWidth、background、clickable ——順番に意味がある

- 状態こそ魂：remember ＋ mutableStateOf。データを変えれば UI は自動で描き直される
- テーマは Material 3 ＋ダイナミックカラー。ダークモードは 1 回の設定で全体に効く

# 第五十二章：4 大コンポーネントと Activity のライフサイクル

*Compose* をアプリの「顔」とするなら、この章で扱うのはアプリの「骨格」です。*Android* システムには *4* つの基本コンポーネントがあり、すべてのアプリはそれらの組み合わせでできています。中でもいちばん大事な *Activity* には、すべての *Android* 開発者が脳に刻むべき「ライフサイクル図」があります。この章を終えれば、「バックグラウンドに行って戻ったら、なぜアプリの状態が消えているのか」といった永遠の謎が解けるようになります。

## 4 大コンポーネント：Android の「大黒柱」

Android は公式に 4 種類の基本コンポーネントを定めていて、それぞれが 1 種類の仕事を担います。

| コンポーネント | 役割 | たとえるなら |
|---|---|---|
| Activity | 目に見える画面 | お店の店構え——ユーザーが見て、操作できる部分 |
| Service | バックグラウンドで働く「見えない人」 | 倉庫の作業員——音楽を流す、ファイルをダウンロードする。画面は不要 |
| BroadcastReceiver | システムのブロードキャストを受け取る | 受付——「電池が残り少ない」「ネットが切れた」を告げるスピーカー |
| ContentProvider | アプリをまたいでデータを共有 | 共用の資料棚——たとえば連絡先を別のアプリに共有する |

Android 4 大コンポーネント早見

初心者のうちは、時間の 95% を Activity との付き合いに使います。Service と BroadcastReceiver は音楽再生やプッシュ通知で出番があり、ContentProvider はいちばん登場が少ないです（よく使うデータはシステムがすでにまとめてくれています）。もう 1 つ大事なこと。4 大コンポーネントは、すべて AndroidManifest.xml に「住民登録」が要ります。登録しないと、システムはそれを見つけられません。

## Activity のライフサイクル：Android 開発で最初の「宝の地図」

スマホのアプリは、いつでも中断され得ます。電話がかかってくる、ユーザーが WeChat の返信に切り替える、メモリが足りなくてシステムに止められる……Activity の一生は、この図の上をぐるぐる巡る旅なのです。

onCreate（誕生：初期化）

onStart（見えるようになった）

onResume（前面で操作中）

onPause（一部が隠れた）

onStop（まったく見えない）

onDestroy（破棄）

onRestart / onStart / onResume：前面に戻るときは同じ道をたどる

図 4：Activity のライフサイクル（進む向き：上段が誕生下段が退場。戻るときは同じ道を逆にたどる）

| コールバック | いつ呼ばれる？ | 典型的な用途 |
|---|---|---|
| onCreate | Activity が生成されるとき（1 回だけ） | UI の読み込み、データの初期化 |
| onStart | 画面が見える状態になる | 軽めの処理を始める |
| onResume | フォーカスを得て、操作できるようになる | アニメーションの開始、再生の再開 |
| onPause | 一部が隠れる / 切り替わる瞬間 | アニメーションの一時停止、下書きの保存 |
| onStop | 完全に見えなくなる | 重いリソースの解放、ダウンロードの一時停止 |
| onDestroy | 破棄される前 | 最後のクリーンアップ |

6 つのライフサイクルコールバックの典型的な用途

**落とし穴注意：画面回転は Activity を「殺して」しまう**

初心者にいちばん多い失敗例：アプリの画面を回したら、入力欄の文字が全部消えた。原因：画面を回転すると、デフォルトでは Activity が破棄されて再び作り直されるためです（onDestroy onCreate が一巡します）。解決策：rememberSaveable で状態を保存する（Compose での remember の代わり）。再生成のときに自動で値を取り戻してくれます。第 6・7 章で何度も使うことになります。

## Intent：アプリの内と外をつなぐ「切符」

Intent（インテント）は、コンポーネント同士がやりとりするための「切符」です。ある画面から別の画面へ移動したいときも、ブラウザを開きたいときも、カメラを起動したいときも、必ず「切符を買って乗車」します。

```
// 方法その1：明示的Intent——行き先のページをはっきり指定
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("name", "たろう")      // 車に「荷物」を積むイメージ（データ）
startActivity(intent)

// 方法その2：暗黙的Intent——何をしたいかだけ言う。実行役はシステムが探す
val web = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
startActivity(web)   // システムがブラウザを開く/または選択させる
```

2 つの方式にはそれぞれ役割があります。明示的 Intent は自分のアプリ内の画面遷移に、暗黙的 Intent はシステムや他アプリの機能を呼び出すとき（Web ページを開く、電話をかける、共有するなど）に使います。Compose プロジェクトでは、Composable の中で LocalContext.current で Context を取れば startActivity が使えます。

## 複数画面とナビゲーション

実際のアプリには複数の画面があります。Compose の公式が推奨するナビゲーション手法は Navigation Compose（依存ライブラリ androidx.navigation）です。

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{name}") { entry ->
        val name = entry.arguments?.getString("name")
        DetailScreen(name ?: "不明")
    }
}

// ページ内で遷移：
navController.navigate("detail/たろう")      // 引数付き
navController.popBackStack()                // 前のページへ戻る
```

**ワンポイント：画面が増えてきたら？**

プロジェクトが大きくなったら、すべての画面のルートを 1 つのファイル（たとえば AppNavGraph.kt）にまとめて一元管理しましょう。「どこへ遷移するか」というロジックが各画面に散らばらないようにするのです。これが「動くもの」から「保守できるもの」へ進む第一歩です。

## ついでに知っておく：Fragment とは何か（そして本書で扱わない理由）

古いチュートリアルを読むと必ず Fragment に出会います。Fragment は Activity の中に置ける「再利用できる画面の部品」で、昔ながらの複数画面アーキテクチャでは主役でした。しかし Compose の時代には、画面は Composable 関数＋ Navigation のルートで足りていて、公式も Fragment の立ち位置を弱めています。そのため本書では本筋に含めていません。

アドバイス：新しいプロジェクトで Fragment を進んで使う必要はありません。ただし古いコードを読むときや、面接で聞かれたときには「Activity に組み込む画面の部品」だと分かっていれば十分です。学習リソースは常に 2023 年以降のものを優先しましょう。Android の技術スタックの「古い常識」は、想像以上に速く役目を終えていきます。

**やってみよう：カウンターに「記憶」をつける**

第五十一章のカウンターを改造してみましょう。(1) remember を rememberSaveable に替え、画面を回転しても数字が残ることを確認する。(2)「このアプリについて」ボタンを追加し、Navigation で 2 つ目の画面へ遷移してバージョン番号を表示する。(3) 両方の画面の onCreate/onPause にログ出力（Log.d）を入れ、Logcat でライフサイクルが呼ばれる順番を観察する。この 3 つができれば、ライフサイクルはもう机上の知識ではありません。

**本章のまとめ**

- 4 大コンポーネント：Activity（顔）・Service（バックグラウンド）・BroadcastReceiver（スピーカー）・ContentProvider（資料棚）
- ライフサイクルの 6 つのコールバック：onCreate onStart onResume onPause onStop onDestroy
- 画面回転で Activity は破棄・再生成される：rememberSaveable で状態を守る
- Intent は「切符」：明示的は自アプリの画面遷移、暗黙的はシステム機能の呼び出し
- 複数画面は Navigation Compose で：NavHost ＋ composable ルート

# 第五十三章：データ保存とネットワーク通信

「見る」ことしかできないアプリは生きていません。データを保存できて、ネットにつながるアプリこそが本物です。この章では *Android* データ層の「三種の神器」を学びます。*DataStore*（小さな設定の保存）、*Room*（構造化データの保存）、*Retrofit*（サーバーとのやりとり）の *3* つと、それらを取り仕切る「総指揮者」であるコルーチンです。この章を終える頃には、ユーザーの使い方を覚えて、中身をリアルタイムに更新できる本物のアプリが書けるようになっています。

## 保存方式の選び方：まずこの表を見る

| 方式 | 何の保存に向くか | 特徴 | 本書での使用箇所 |
|---|---|---|---|
| DataStore | こまごました設定（テーマ、スイッチなど） | キーと値のペア。シンプルで安全 | 第五十四章の家計簿の設定 |
| Room | 構造化データ（明細、ユーザーなど） | 本物の SQLite データベースのラッパー。検索できる | 第五十四章の収支の記録 |
| Retrofit | （保存せず、ネットで取得）サーバーのデータ | HTTP 通信ライブラリの事実上の標準 | 第五十五章の天気データ |

データ三種の神器の分担

覚え言葉をひとつ。「小さな設定は DataStore へ、まとまったデータは Room へ、外のデータは Retrofit で」。古いチュートリアルでよく出てくる SharedPreferences は DataStore に、手書き SQL の SQLite は Room に取って代わられました。最初から新しいやり方を学び、古い道へは戻らないことです。

## DataStore：小さな設定を保存する

「ユーザーが前回選んだ通貨」「ダークモードを有効にしているか」のようなこまごました値を覚えておきたいときは、DataStore（Preferences 版）を使います。

```
val Context.dataStore by preferencesDataStore(name = "settings")

// 書く：値を保存
suspend fun saveTheme(isDark: Boolean) {
    context.dataStore.edit { prefs ->
        prefs[THEME_KEY] = isDark
    }
}

// 読む：取り出す（Flowが変化を自動通知）
val themeFlow: Flow<Boolean> = context.dataStore.data
    .map { it[THEME_KEY] ?: false }
```

返ってくるのが Flow、つまり「流れる」データの流れである点に注目しましょう。値がひとたび変われば、それを監視しているすべての場所に新しい値が自動で届きます。この概念（リアクティブ）が現代 Android 開発の土台で、第五十四章で実戦します。

## Room：データに「本物の家」を用意する

家計簿には何百、何千件もの記録がたまり、しかもカテゴリ別・月別に集計もしたい。そこで必要になるのがデータベースです。Room は SQLite の公式ラッパーで、「テーブルを作る・データを引く」を Kotlin を書くのと同じように自然にしてくれます。登場するのは 3 つ。

```
// 1. Entity：テーブル構造（1テーブル = 1 data class）
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val amount: Double,
    val category: String,
    val note: String,
    val date: String
)

// 2. Dao：操作用インターフェース（追加・削除・変更・検索）
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)
```

```
    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT SUM(amount) FROM records WHERE category = :c")
    suspend fun sumByCategory(c: String): Double
}

// 3. Database: 組み立て
@Database(entities = [Record::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun recordDao(): RecordDao
}
```

この「三種の神器」のテンプレートは、そのままどんなプロジェクトにも持ち込めます。フィールド名を変えるだけで OK です。クエリは Flow を返し、データが変われば画面が自動で更新される。なかなか気持ちのいい仕組みです。

## Retrofit：サーバーと会話する

天気、ニュース、為替レート……こうしたデータはスマホの中にはなく、他人のサーバー上にあります。Retrofit は「サーバーにデータをねだる」ことを、関数を呼ぶのと同じくらい簡単にしてくれます。3 ステップで進みます。

```
// 1. インターフェースを定義：HTTPリクエストを関数として書き表す
interface WeatherApi {
    @GET("v1/weather")
    suspend fun getWeather(@Query("city") city: String): WeatherResponse
}

// 2. インスタンスを作成
val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApi::class.java)

// 3. 関数のように呼び出す（コルーチンの中で）
val result = api.getWeather("北京")
```

**落とし穴注意：ネットワークの 2 大鉄則**

(1) ネットワーク通信はメインスレッドで絶対にしてはいけません。画面が固まり、最悪クラッシュします（NetworkOnMainThreadException）。ネットワーク呼び出しは必ずコルーチン（viewModelScope.launch）の中に入れます。(2) 平文の HTTP はデフォルトで禁止です。Android 9 から HTTPS が求められます。テストで http:// のローカルアドレスを使い

たい場合は、Manifest に usesCleartextTraffic を明示的に宣言する必要がありますが、正式な製品は必ずすべて HTTPS にすること。

## JSON 解析：ネットのデータに「翻訳」をつける

サーバーが返すデータの多くは JSON です。入れ子になった辞書のようなテキストのことです。Retrofit はこれを Kotlin オブジェクトに変えてくれる「翻訳者」を必要とします。2026 年の主流は 2 つあります。

| 方式 | 特徴 | おすすめの場面 |
| --- | --- | --- |
| Gson / Moshi | 昔からの定番、リフレクション方式。設定の手間ゼロ | 手早く始めたい小規模プロジェクト、フィールドの少ないもの |
| kotlinx.serialization | 公式のコンパイル時方式。より速く、より安全 | 新規プロジェクトの第一候補（2026 年の主流） |

JSON 解析の 2 大方式

```
// data classに@Serializableアノテーションを付ける
@Serializable
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
)

// Retrofitに対応するConverterを取り付ける（Retrofit 2.11+の公式サポートを例に）：
// .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

どちらの方式でも鉄則は同じです。まず実際に返ってくる JSON を見て、それからフィールドを定義すること。「解析に失敗した」バグの多くは、フィールド名の書き間違いや、入れ子の階層の読み違いが原因です。元の JSON を印字して見比べれば、10 秒で原因がつかめます。

## コルーチン：Android の並行処理の「共通語」

## コルーチン：Android の並行処理の「共通語」

さきほどから何度も出てきた suspend と launch は、Kotlin コルーチン（Coroutine）のものです。Android 公式の並行処理の仕組みです。乱暴に言えば、「少し待つ必要がある」処理（ネットワークやデータベース）が画面を止めないようにするためのもの、と覚えてください。

```
// ViewModelの中でコルーチンを起動
viewModelScope.launch {
    val records = dao.getAll().first()  // suspend: 自動的に待機する
    // 画面の状態を更新……
}

// キーワードの覚え方:
// suspend   ―― 「待ちたい、他を止めないで」
// launch    ―― 「仕事開始！」
// viewModelScope ―― 画面について回る「作業場」。画面が壊れると自動で片付く
```

まずは使い方だけ覚えれば大丈夫です。その裏にある「一時停止と再開」の仕組みは、後からじっくり掘ればいい。ふだんの開発で書くコードの 9 割は、このテンプレートです。

**やってみよう：「今日のひとこと」アプリを作る**

無料の名言 API（hitokoto など）を探して、Retrofit で「今日のひとこと」を取得し、画面の中央に表示しましょう。「別のひとこと」ボタン（コルーチン内で再リクエスト）も付けます。さらに気に入った文は Room で「お気に入り」に保存します。この練習で DataStore/Room/Retrofit/コルーチンを全部使うことになります。これができれば、第 7・8 章は楽勝です。

**本章のまとめ**

- 覚え言葉：小さな設定は DataStore へ、まとまったデータは Room へ、外のデータは Retrofit で
- DataStore は Flow を返す：値が変われば、監視している側に自動で届く
- Room の三種の神器：@Entity（テーブル）＋ @Dao（操作）＋ @Database（組み立て）。テンプレートはそのまま再利用できる
- Retrofit の 3 ステップ：インターフェースを定義インスタンスを作るコルーチン内で呼ぶ
- 2 大鉄則：ネットワークはメインスレッドに入れない。正式な製品は必ず HTTPS
- コルーチン＝画面を止めない待ち方。viewModelScope は画面専用の作業場

# 第五十四章：プロジェクト 1——わたしの家計簿（ローカル版）

いよいよ最初の「きちんとしたプロジェクト」です。この家計簿はネットにはつながりませんが、中身はそろっています。記録を付けられて、履歴が見られて、月ごとの集計ができて、削除も編集もできる。作り終える頃には、「データロジック画面」の全工程を一通り歩いたことになります。コードを急いで写そうとしないこと。まず考えてから手を動かす。これが本書のいつもの方針です。

## 要件を考える：まず 5 つの質問

手を動かす前に、「要件の 5 要素」でアイデアを整理しましょう。

| 要素 | 答え |
|---|---|
| Who（誰に使うか） | 自分用。家族や友人でも OK |
| What（何をするか） | 収支を記録、履歴を見る、月ごとにまとめる、カテゴリ別に集計 |
| Why（なぜ作るか） | 練習＋お金を何に使ったかをはっきりさせる |
| How（どう作るか） | Kotlin ＋ Compose ＋ Room ＋ MVVM。完全ローカルでサーバーなし |
| Constraint（制約） | 1 週間で完成。Android 8 以上に対応。ストアに出さないとしても、出品基準で作っておく |

家計簿の要件 5 要素

次に、「どの段階でも使える」小さなステップに分解します。(1)Room のデータ層を作る（1 件保存できる）。(2) 一覧画面（履歴が見られる）。(3) 入力画面（記録を追加できる）。(4) 削除と編集。(5) 月次のまとめカード。1 ステップ終えるたびに、アプリは「使えるもの」になっています。

## データ層：記録に「住まい」を与える

まず 1 件の記録がどんな形かを定義します。フィールドは足りる分だけで、欲張らないこと。

```
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String,        // "income" 収入 / "expense" 支出
    val amount: Double,      // 金額（元）
    val category: String,    // カテゴリー：食費、交通、給料……
    val note: String = "",   // メモ（任意）
    val date: String         // "2026-08-15"
)
```

Dao に「集計用」のクエリを 2 つ追加します。ここが家計簿の魂です。

```
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC, id DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='income' AND
    date LIKE :month || '%'")
    suspend fun monthIncome(month: String): Double   // monthは「2026-08」の形

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='expense' AND
    date LIKE :month || '%'")
    suspend fun monthExpense(month: String): Double
}
```

**ワンポイント：金額はなぜ Double？**

真面目な金融アプリは、BigDecimal や「分（セント）単位の Int」で浮動小数点の誤差（0.1+0.2≠0.3 という古典的な問題）を避けます。練習のプロジェクトなら Double で十分です。いつか本物の金融の製品を作るときは「分」単位に替えるのを忘れずに。先人たちが無数に踏んできた落とし穴です。

## MVVM：画面とロジックを「別々の家」にする

画面から直接データベースを操作させたら？　動きはしますが、画面のコードは長くて読みにくくなります。MVVM アーキテクチャなら、それぞれの役割がはっきりします。View（画面）は描くことだけ、ViewModel（ロジックの中枢）はデータを担当、Model（Room）は保存を担当です。

View（Compose の画面）

ViewModel（状態とロジック）

Model（Room のデータ層）

図 5：MVVM の 3 層構造——データは一方向に流れ、各層は互いに境界を越えない

```
class RecordViewModel(private val dao: RecordDao) : ViewModel() {

    // Flowを画面から観察できる状態に変換
    val records = dao.getAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), emptyList())

    // ユーザー操作：suspend関数を呼ぶ
    fun addRecord(type: String, amount: Double, category: String, note: String,
    date: String) {
        viewModelScope.launch {
            dao.insert(Record(type = type, amount = amount,
                category = category, note = note, date = date))
        }
    }

    fun deleteRecord(record: Record) {
        viewModelScope.launch { dao.delete(record) }
    }
}
```

画面（Compose）ではこう書きます。val vm: RecordViewModel = viewModel() として、続いて val list by vm.records.collectAsState()。データが変わればリストが自動で更新され、こちらは何もする必要がありません。これが MVVM ＋ Flow ＋ Compose の三本柱がそろった力です。

## 画面：一覧画面＋入力画面

一覧画面には LazyColumn（遅延読み込みリスト）を使います。データがどれだけ増えても動きが重くなりません。

```
@Composable
fun RecordListScreen(vm: RecordViewModel) {
```

```
    val records by vm.records.collectAsState()

    Column {
        // 上部の集計カード（収入-支出=残高）
        SummaryCard(records)

        // 明細リスト
        LazyColumn {
            items(records, key = { it.id }) { record ->
                RecordRow(record) { vm.deleteRecord(record) }
            }
        }

        // 浮遊する「記帳」ボタン　入力ダイアログを出す
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Text("+")
        }
    }
}

@Composable
fun RecordRow(record: Record, onDelete: () -> Unit) {
    Row(Modifier.fillMaxWidth().padding(12.dp)) {
        Text(record.category, fontWeight = FontWeight.Bold)
        Spacer(Modifier.weight(1f))
        Text(if (record.type == "income") "+${record.amount}" else "-${record.
    amount}",
             color = if (record.type == "income") Color(0xFF27AE60) else Color(0
    xFFC0392B))
    }
}
```

入力には AlertDialog（ダイアログ）が一番手軽です。入力欄をいくつか並べて、収入 / 支出の切り替え、カテゴリのプルダウンを付け、「確定」を押したら vm.addRecord(...) を呼ぶだけです。

**注意：入力チェックは手を抜かない**

金額の入力欄は必ずチェックします。空の値、マイナス、「abc」のような不正な入力は全部はじきます（toDoubleOrNull() で判定）。アプリがクラッシュするいちばんよくある原因のひとつが、「ユーザーの入力をいい子だと思い込む」ことです。ユーザーの入力はすべて、敵として吟味する、と覚えてください。

## 編集と削除：データを「入れて出せる」ように

追加だけできて削除できない家計簿は、3 日で使わなくなります。操作を 2 つ足しましょう。

```
// Daoに2つ追加：
@Query("UPDATE records SET amount=:amount, category=:category, note=:note WHERE id
    =:id")
suspend fun update(id: Long, amount: Double, category: String, note: String)

@Query("DELETE FROM records WHERE id=:id")
suspend fun deleteById(id: Long)

// 画面では：リスト項目を長押しして「編集/削除」メニューを出す（ComposeはDropdownMenu
    かAlertDialogを使う）
// 削除前には確認ダイアログを出すのが一番——「うっかり削除」は家計簿アプリのユーザーが
    最も嫌うこと
```

操作まわりの細かいコツが 2 つ。削除は必ず二重確認にする（うっかり操作の防止）。編集はできるだけ元の値を先に入れておく（1 文字だけ直すほうが、打ち直すより楽）。この 2 つの細かい点を舐めてはいけません。製品らしい使い心地は、こうした積み重ねでできているのです。

## 検収：3 層のチェック＋ 2 つの上級チャレンジ

## 検収：3 層のチェック＋ 2 つの上級チャレンジ

プロジェクトができたら、「機能境界アーキテクチャ」の 3 層で検収します。記録できるか、削除できるか、アプリを開き直してもデータが残っているか（機能）。金額に 0 やマイナスを入れてもクラッシュしないか、画面回転でデータが消えないか（境界）。画面とデータが混ざっていないか、値がハードコーディングされていないか（アーキテクチャ）。

基本版の検収を通ったら、自分に「上級の一品」を 2 つ追加しましょう。

1. 月別フィルター：上部に月の切り替え（たとえば 2026-08 v）を付け、リストには当月の履歴だけを表示する。
2. カテゴリ別の円グラフ：Compose の Canvas を使うか、軽量なグラフライブラリを入れて、「お金を何に使ったか」を円グラフにします。ここまでできれば、家計簿はだいぶ「製品っぽく」なります。

**やってみよう：今週中に完成させる**

「データ層 ViewModel 一覧画面入力画面まとめカード」の順に手を動かし、1 日 1 ステップで進めます。つまづいたら、エラーをそのまま検索エンジンや AI に投げましょう（第 7 部『AI プログラミング実戦編』を覚えていますか？　問題を説明するときは「期待す

る動作・実際の動作・再現手順」をはっきり伝えます）。完成したら、1 週間これで家計をつけてみてください。自分で作ったアプリを使うと、達成感が先へ進む力になります。

**本章のまとめ**

- 要件の 5 要素「どの段階でも使える」小ステップに分解してから手を動かす
- データ層：Record エンティティ＋ Dao（月次集計の SQL を含む）＋ Database
- MVVM の分担：View は描くだけ、ViewModel はロジック、Model は保存
- Flow ＋ stateIn ＋ collectAsState：データが変われば画面が自動で更新される
- LazyColumn は遅延読み込みリスト。入力には AlertDialog
- 3 層の検収（機能 / 境界 / アーキテクチャ）＋上級チャレンジ（月別フィルター、カテゴリ別円グラフ）

# 第五十五章：プロジェクト 2——天気チェックアプリ（ネットワーク版）

家計簿が「内功の鍛錬」なら、天気アプリは「外の世界へ出る」プロジェクトです——インターネットと付き合うことになります。ここで学ぶのは、無料 *API* の申し込み、*JSON* の解析、「読み込み中 / 成功 / 失敗」の *3* つの画面状態の処理、そして *API* キーをきちんと隠す方法です。これをやり遂げれば、どんな「ネット接続アプリ」でも作れる力が手に入ります。ニュースも、為替も、翻訳も、やることはまったく同じです。

## 無料の天気 API を申し込む

個人で練習するなら、OpenWeatherMap か、中国国内の「和風天気（QWeather）」がおすすめです。どちらも無料枠があって、登録すればすぐ使えます。OpenWeatherMap を例に見てみましょう。

1. アカウントを登録 API keys ページでキー（apikey）を 1 つ発行する。

2. 無料版で呼べるのは Current Weather（現在の天気）の API です。api.openweathermap.org/data/2.5/weather?q=beijing&appid=あなたのキー&units=metric。

3. まずブラウザで一度試して、返ってくる JSON がどんな形か見ておきましょう。どんな API につなぐ前にも必ずやる「予習」です。

#### 落とし穴注意：キーをコードに直書きしない

API キーはあなたの「財布のパスワード」みたいなものです。コードに書いて GitHub にプッシュすれば、数時間のうちにクローラーに収集されて勝手に使われます。正しいやり方は、local.properties（このファイルは gitignore で無視されています）に置いておき、Gradle の buildConfig 経由でコードに読み込むことです。「Android API Key 隠すベストプラクティス」で検索して、その通りに直せば大丈夫です。

## ネットワーク層：Retrofit ＋ JSON 解析

まず API が返す JSON に合わせて「データモデル」を定義します。対応関係にないフィールドは、すべて無視してかまいません。

```
// JSON: {"weather":[{"description":"晴れ"}],"main":{"temp":26.5,"humidity":40},"
    name":"北京"}
data class WeatherResponse(
    val weather: List<Weather>,
    val main: Main,
    val name: String
)
data class Weather(val description: String)
data class Main(val temp: Double, val humidity: Int)

interface WeatherApi {
    @GET("data/2.5/weather")
    suspend fun getWeather(
        @Query("q") city: String,
        @Query("appid") key: String,
        @Query("units") metric: String = "metric",
        @Query("lang") lang: String = "zh_cn"
    ): WeatherResponse
}
```

Gson は JSON をこれらの data class へ自動で詰め込みます。フィールド名が一致していれば、魔法は自動的に起こります。

## 3 つの画面状態：読み込み中 / 成功 / 失敗

ネット接続アプリの魂は「状態管理」にあります。ネットが遅い、つながらない、都市名を打ち間違えた……どの場合にも、画面はユーザーにきちんと伝える必要があります。sealed class（シールドクラス）で、美しく表現しましょう。

```
sealed class UiState {
    object Loading : UiState()                    // くるくる回る
    data class Success(val data: WeatherResponse) : UiState()  // 成功
    data class Error(val message: String) : UiState()          // 失敗
}

// ViewModel側:
fun search(city: String) {
    viewModelScope.launch {
        _state.value = UiState.Loading
        _state.value = try {
            UiState.Success(api.getWeather(city))
```

```
        } catch (e: Exception) {
            UiState.Error("その都市が見つかりません。別の名前で試してみて？")
        }
    }
}

// 画面側：
when (val s = state) {
    is UiState.Loading -> CircularProgressIndicator()   // 回る
    is UiState.Success -> WeatherContent(s.data)        // 表示
    is UiState.Error -> Text(s.message)                 // エラー表示
}
```

この Loading/Success/Error の三段構えは、ネット接続画面の「万能テンプレート」です。まず暗記してしまえば、この先のどんなネット接続アプリでも同じ形で書けます。

## 実戦の画面コード：天気を描き出す

状態が決まったら、画面はこうなります（要点の抜粋）。

```
@Composable
fun WeatherContent(data: WeatherResponse) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(data.name, fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(
            "${data.main.temp}°C",
            fontSize = 72.sp,
            fontWeight = FontWeight.Light
        )
        Text(
            data.weather.firstOrNull()?.description ?: "不明",
            fontSize = 20.sp,
            color = Color.Gray
        )
        Spacer(Modifier.height(16.dp))
        Row {
            Text("湿度 ${data.main.humidity}%")
            Spacer(Modifier.width(24.dp))
            Text("体感 ${data.main.temp}°C")
        }
    }
}
```

学ぶ価値のある小さな工夫がいくつかあります。firstOrNull() は最初の要素を安全に取り出します（リストが空かもしれないので）。?: “不明”はいざというときの表示を用意しておく——安全に空を扱うという原則は、画面でも同じです。温度は 72.sp の大きい文字で存在感を出し、「細め」のウェイト（Light）で数字を上品に見せる。天気系アプリの定番レシピです。

## 画面と操作の細かい工夫

## 画面と操作の細かい工夫

- 検索ボックス：上部に TextField を置き、都市名を入れて検索（キーボードのアクションを「検索」に設定して、Enter で検索できるように）。
- 天気カード：大きい文字の温度（26.5°）、天気の説明（晴れ）、湿度、都市名。
- お気に入りの都市：よく見る都市を DataStore に保存して、次に開いたときに自動で検索——第六章の復習もかねて。
- 引っ張って更新：Compose では PullToRefreshBox（Material 3 が提供）を使います。操作の感触が一気に「プロっぽく」なります。

**ワンポイント：画像で画面を生き生きと**

天気の種類ごとに絵文字かアイコン（）を割り当て、when(description) で対応づけます。3 行のコードで、画面のぬくもりが倍になります。サードパーティの天気アイコンライブラリを読み込んでもいいですが、練習の段階なら絵文字で十分です。

**やってみよう：天気アプリの検収チェックリスト**

(1) 普通に検索して天気が出る。(2) ネットを切っても、アプリはエラーを上品に表示してクラッシュしない（機内モードで試す！）。(3) 存在しない都市を入れても、やさしい表示が出る。(4) 都市を 2 つお気に入りに入れて、アプリを再起動しても残っている。(5) 画面を回しても状態が消えない。この 5 つを全部クリアして、このプロジェクトは卒業です。

**本章のまとめ**

- まずブラウザで API を試し、JSON を読んでからコードを書く——「予習」で損をすることはない
- API キーは local.properties に隠す。リポジトリには絶対入れない
- Retrofit インターフェース＋ data class：フィールドが合えば、JSON は自動で解析される
- Loading/Success/Error の三段構えは、ネット接続画面の万能テンプレート
- 切断、空の入力、存在しない都市——エラー処理はネット接続アプリの合格ライン

# 第五十六章：仕上げとリリース——テスト、デバッグ、Google Play への出品

アプリができました。それで次は？　この章では「動くもの」から「届けられるもの」へ進むための 3 つのことを扱います。効率よくデバッグしてバグを見つける方法。テストを書いてデグレを防ぐ方法。そしてアプリを *Google Play* に並べて、世界中にダウンロードしてもらう方法です。出品は儀式ではありません。作品を製品に変える分水嶺です。

## デバッグの三種の神器：Logcat、ブレークポイント、UI インスペクター

| ツール | 何をするか | 使い方 |
|---|---|---|
| Logcat | システムや自分が出したログを見る | コードで Log.d(”TAG”,” 値 =$x”) と書き、パネルで TAG を検索して絞り込む |
| ブレークポイントデバッグ | プログラムをある行で止めて、変数の値を見る | 行番号の横をクリックして赤点を出し、Debug モードで実行して 1 行ずつ進める |
| Layout Inspector | 動いている画面の構造を見る | メニューの Tools Layout Inspector で、コンポーネントツリーをリアルタイムに見る |

デバッグの三種の神器

**ワンポイント：エラーの読み方**

クラッシュすると Logcat に赤いスタックトレース（Stacktrace）が出ます。下から上へ向かって、自分のパッケージ名で始まる最初の行（たとえば com.example.yourapp）を探します。その行がほぼ現場です。まず「Exception の種類」（ヌルポインタ？　型変換?）を見て、次に「at ... 自分のファイル: 行番号」を見ます。この技で、クラッシュの 8 割は解決できます。

## テスト：コードに保険をかける

手動テストは頼りになりません。ある機能を直したら、別の機能がこっそり壊れていた——そんなことはよくあります。テストは「後悔しないための薬」であり、「防弾チョッキ」です。Android のテストは 3 層に分かれています。初心者はまず第 1 層をしっかり押さえましょう。

| 層 | 何をテストするか | どこに置くか | 速さ |
|---|---|---|---|
| ユニットテスト | 純粋なロジック（計算、検証） | app/src/test/ | 秒単位 |
| インストゥルメンテーションテスト | 実機上での画面の動き | app/src/androidTest/ | 分単位 |
| 手動テスト | 本物の人間の感覚・体験 | —— | 常に必要 |

テストの 3 層

```
// 「金額文字列を数値へ変換」するロジックを純粋関数として切り出す
fun parseAmount(input: String): Double? {
    val v = input.trim().toDoubleOrNull()
    return if (v != null && v > 0) v else null
}

// テストする（JUnit4）
@Test
fun emptyInputReturnsNull() {
    assertEquals(null, parseAmount(""))
}
@Test
fun negativeReturnsNull() {
    assertEquals(null, parseAmount("-5"))
}
@Test
fun normalValueParses() {
    assertEquals(12.5, parseAmount("12.5"))
}
```

家計簿の金額チェックや集計の計算には、ユニットテストをかぶせておくべきです。テストの実行は、test ディレクトリを右クリック Run。緑で全部通れば、安心してコードを直せます。

## 署名とパッケージング：アプリに「ハンコ」を押す

Android では、すべてのインストールパッケージに署名が必須です。公印を押すのと同じで、「このパッケージは自分が出したもので、改ざんされていない」ことを証明します。デ

バッグ中は Android Studio が自動で debug 署名を使い、リリースでは自分の release 署名を使います。手順はこうです。

1. メニューの Build Generate Signed Bundle / APK を選ぶ。
2. keystore ファイル（キーストア）を作る——パスワードを設定して、大切に保管する。
3. release ＋ AAB 形式を選んで、ビルド完了。

**落とし穴注意：keystore を失う＝アプリを失う**

出品後のアップデートには、必ず同じ署名を使います。そうしないとストアは更新を拒否します。keystore ファイルとパスワードを一度失うと、新しいアプリとして出し直すしかなく、既存のユーザーはみんなつながらなくなります。keystore とパスワードは必ず安全な場所（クラウドの暗号化保存＋メモ）にバックアップしてください。Android 開発者にとって、いちばん高くつく教訓のひとつです。

## Google Play 出品の全ステップ

開発者アカウントを登録
ストア用素材を準備
AAB をアップロード
データセキュリティ表を記入
審査（数時間　数日）
ストアにリリース

図 6：Google Play 出品の 6 ステップ

- 登録：25 ドルの一回払いの登録料。使える Google アカウントが必要。
- 素材：アプリのアイコン（512×512）、スクリーンショット（2 枚以上）、機能の説明、プライバシーポリシーのリンク（データを集める場合は必須）。
- データセーフティフォーム：どんなデータを集めるかを正直に申告する——ネットにつながらない家計簿なら「ゼロ収集」にできます。
- 審査：新しいアカウントの初出品は、人手による審査になることがよくあります。数時間から数日。差し戻されたら指摘に従って直します。

中国国内の読者へ：Google Play には特別なネット環境が必要で、中国大陸向けの配信は普通、国内ストア（Huawei、Xiaomi、Tencent（応用宝）など）を使います。国内ストアへの出品も流れはよく似ていますが、ストアごとに審査の好みがあります。同じ AAB/APK を複数のストアで配信できます。練習としては、まず 1 つでいいので通せば勝ちです。

## ストア素材の作り方：第一印象がそのままダウンロード数になる

ストアページは、ユーザーが「お店をのぞく」ときに目にするショーウィンドーです。3つの素材が第一印象を決めます。

- アイコン：512×512。スクリーンショットで代用しないこと。デザインが苦手なら、AIツールでシンプルな幾何学模様を生成しましょう。スマホのホーム画面で小さくしても、一目で見分けられるかが合格ラインです。
- スクリーンショット：2 枚以上、できれば 4 〜 6 枚で、いちばん核となる機能の場面を見せる。1 枚ごとに、自然なひと言を添える（「1 秒で 1 件メモ」は「効率的な家計簿ソリューション」より 10 倍いい）。
- 説明文：最初の一文で「これが何で、誰のためのものか」をはっきりさせる。「最もプロフェッショナル」「最強」を並べないこと。ストアの検索順位は主にキーワードとダウンロードへの転換で決まります。誠実さのほうが、誇張より効きます。

**ワンポイント：A/B で同業をのぞいてみる**

出品する前に、ストアで同ジャンルの上位 10 アプリを見て回り、スクリーンショットを取って比べましょう。アイコンはどんなスタイル？　スクリーンショットは何を売りにしている？　タイトルはどう書く？　真似は盗作ではありません。型を学んで、中身を自分のものに替える。これはどんな製品でも、最初の一歩を最速で踏み出すための教訓です。

## パフォーマンス最適化とクラッシュ報告の小チェックリスト

アプリを出品する前に、この 2 つにはそれぞれ 30 分かける価値があります。

### パフォーマンスのセルフチェック 4 か条

1. リストは LazyColumn：長いリストを Column で無理に並べないこと——遅延読み込みは、Android の性能の最初の一歩です。
2. 画像は元のままで使わない：Coil 画像ライブラリで自動的に縮小とキャッシュを行う。5MB の元画像 1 枚で、ローエンド機は固まってしまいます。
3. メインスレッドで重い仕事をしない：データベース、ネットワーク、大きなファイルは、すべてコルーチンの中で。
4. Profiler で測る：Android Studio 内蔵のプロファイラーで、メモリ、CPU、消費電力が見られます。まず測ってから最適化する。感覚に頼らないこと。

### クラッシュ報告

自分のテストでクラッシュしなくても、ユーザーのスマホでクラッシュしないとは限りません。出品前に Firebase Crashlytics（Google 公式、無料）を組み込みましょう。数行の設定で、ユーザーがクラッシュするとスタックトレースが自動で送られ、「どの機種・どのバージョン・どこで落ちたか」が管理画面で分かります。クラッシュ監視なしの出品は、目をつぶって車を運転するのと同じです。

**やってみよう：家計簿を一度「引き渡す」**

(1) parseAmount などのロジックにユニットテストを 3 つ追加する。(2) Logcat で、自分で仕込んだクラッシュを実際につかまえて直す。(3) release 署名の APK を生成して、友人に送ってインストールしてもらう（これこそ「作品」の儀式です）。(4) 余裕があれば開発者アカウントを登録して、家計簿を本当に出品する。最初の 3 つは今夜中にできます。

**本章のまとめ**

- デバッグの三種の神器：Logcat でログ、ブレークポイントで変数、Layout Inspector で画面
- クラッシュの読み方：スタックトレースを下から上へ、自分のパッケージ名の最初の行を探す
- テストの 3 層：ユニット（秒単位、まずこれを学ぶ）/ インストゥルメンテーション / 手動
- リリースには署名が必須：keystore とパスワードはバックアップ必須。失うとアプリは「別のアプリ」になる
- 出品の 6 ステップ：登録素材 AAB をアップロードデータセーフティフォーム審査出品

# 第五十七章：ステップアップへの道——権限、通知、モダンなアーキテクチャ

アプリが作れるようになるのは、スタートにすぎません。この章では「作れる」から「うまく作れる」へ進むための関門をいくつか扱います。権限を上品に申請する方法。嫌われない通知の出し方。アーキテクチャをもう一段引き上げる方法。そして、*2026* 年のいま、*AI* と組んで *Android* コードを書く方法です。どれも、実際のプロジェクトですぐに出会うものばかりです。

## 実行時権限：まずノックして、それから入る

カメラ、位置情報、連絡先、通知……こうした「センシティブな領域」については、Android はアプリの実行時にユーザーへ申請することを求めます（Android 6 から）。ルールはこうです。まず宣言（Manifest）、次に申請（コード）、そして拒否されたら上品に縮小して動くこと。Compose では公式ライブラリ Accompanist の rememberPermissionState を使うのが一番楽です。

```
// まずManifestに宣言:
// <uses-permission android:name="android.permission.CAMERA" />

@Composable
fun CameraButton() {
    val permission = rememberPermissionState(
        permission = android.Manifest.permission.CAMERA
    )
    Button(onClick = {
        when {
            permission.status.isGranted -> openCamera()
            permission.status.shouldShowRationale -> showExplainDialog()
            else -> permission.launchPermissionRequest()
        }
    }) { Text("撮影") }
}
```

**注意：完全に拒否されたら？**

ユーザーが「今後表示しない」を選ぶと、launchPermissionRequest はもうダイアログを出しません。その場合は、Intent でアプリ設定のページへ飛ばして、システム設定から手動で有効にするよう誘導し、画面で「この権限を何に使うのか」を説明します。態度を丁寧にすれば、ユーザーが権限を許可してくれる気持ちもぐっと高まります。権限申請の文面は、真剣に書く価値があります。

## 通知：正しいタイミングで、役に立つことを伝える

通知はアプリからユーザーへ届く経路であると同時に、ユーザーに「ブロック」されやすい機能でもあります。通知を出す 3 つの規律。役に立つことだけ送る。ユーザーが切れるようにする。真夜中に攻めない。基本の使い方はこうです。

```
// Android 13+では先にPOST_NOTIFICATIONS権限を申請する必要がある（前の節を参照）

// 1. 通知チャンネルを作成（Android 8+で必須）
val channel = NotificationChannel(
    "reminder", "リマインダー", NotificationManager.IMPORTANCE_DEFAULT
)
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(channel)

// 2. 通知を送る
val notification = NotificationCompat.Builder(this, "reminder")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("記帳リマインダー")
    .setContentText("今日はまだ記帳していないよ")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS) == PackageManager.
    PERMISSION_GRANTED
) {
    manager.notify(1, notification)
}
```

定期タスク（たとえば「毎日 20:00 に記録を促す」）には WorkManager を使います。公式おすすめの「バックグラウンド調整の執事」で、タスクが適切なタイミングで実行されることを保証してくれ、バッテリー最適化を自分で気にする必要がありません。目覚まし系の要望には、これで間違いありません。

## アーキテクチャのステップアップ：依存性注入とモジュール化

プロジェクトが大きくなると、2 つの「うれしい悩み」がやってきます。

- オブジェクトの生成が乱れる：各画面が自分でデータベースを new し、API を new する——Hilt（依存性注入フレームワーク）で一元管理して、「誰が何を欲しければ、口を開けば渡してくれる」ようにする。

- コードが山になる：単一モジュールのプロジェクトは、直すたびに遅くなる——マルチモジュール（app / core / feature-xxx）に分割して、機能ごとに独立して開発・テストする。

**ワンポイント：設計しすぎない**

Hilt もモジュール化も良い道具ですが、個人プロジェクトなら急いで導入しないこと。家計簿は単一モジュール＋依存の手動生成で十分です。コードが数万行を超えて、ビルドが遅くなり、複数人で作業するようになったら、必要に応じて入れましょう。道具は目的のためにあるのであって、見せびらかすためではありません。これが本書で言う「シンプル至上主義」の原則です。

## 2026 年の新しいやり方：AI 支援の Android 開発

2026 年、AI はすでに Android 開発の流れの中に深く入り込んでいます。作業を速くしたいなら、この連携プレーがとても固まっています。

| 段階 | AI の助け方 | あなたの役割 |
|---|---|---|
| コードを書く | Android Studio 内蔵の Gemini やサードパーティ AI プラグインで補完・生成 | 要件をはっきり説明し、成果物をチェックする |
| 調べ物 | 公式ドキュメントやエラー情報を AI に渡して解説してもらう | 大事な結論は公式ドキュメントで確認する |
| テストを書く | AI に「正常 / 境界 / 異常」の 3 種類でテストケースを生成させる | テストケースが業務の意味に合っているか確認する |
| バグを直す | スタックトレース＋関係するコードを貼って、AI が原因の特定と修正を行う | 変更内容を理解し、テストを実行して確認する |

AI 支援 Android 開発の 4 つの場面

具体的な方法論——AI への指示の出し方、検収のやり方、落とし穴の避け方——は、本書第 7 部『AI プログラミング実戦編』（第 37　47 章）に全 11 章でまとまっています。あわせて読むことを強くおすすめします。Android の文法は本編で、AI との協働のコツは第 7 部で、と分担してください。

## Compose の性能最適化とデバッグツール

Compose は書いていて気持ちいいものの、「再構成のしすぎ」で重くなることもあります。実用的な技を 3 つ。

- パラメータを安定させる：データクラスに @Immutable を付けるか、derivedStateOf を使って、関係のない状態変化で広範囲の再構成が起きるのを防ぐ。
- key を忘れない：items(list, key = { it.id }) とすれば、リストの追加・削除のときに動かすべき項目だけが動きます。key がないと、1 項目の削除で表全体が描き直されることもあります。
- Layout Inspector の Compose モード：各 Composable の再構成回数をリアルタイムに見られます。どれが頻繁に再構成されているか一目瞭然なので、その重点箇所を狙って最適化します。

さらに Compose コンパイラのレポート（Metrics）を有効にすると、「どのコンポーネントが不安定か」のレポートが生成できます。ステップアップの道では、データのほうが感覚よりいつでも信頼できます。

**やってみよう：家計簿に「三種の神器」を追加する**

(1) 家計簿に「毎日 20:00 の記録リマインダー」を追加する（WorkManager ＋通知チャネル＋ Android 13 の通知権限）。(2) データベースと API の生成を、Hilt による注入に替える。(3) AI ツールを使って「月別フィルター」機能の実装を速めてみて、3 層検収の方法で成果物をチェックする。3 つそろえば、あなたのプロジェクトも「正規軍」らしくなります。

**本章のまとめ**

- センシティブな権限：Manifest で宣言＋実行時に申請＋拒否されたら上品に誘導
- 通知の 3 規律：役に立つものだけ、切れる、攻めない。チャネル＋権限のどちらも必須
- 定期タスクは WorkManager。車輪の再発明をしない
- オブジェクトは Hilt、コードはモジュール化——個人プロジェクトでは急いで導入しない
- AI 支援の 4 場面：コード作成、調べ物、テスト作成、バグ修正。方法論は第 7 部（第 37　47 章）を参照

# 第五十八章：クイックリファレンスとリソース

この章は、単独で使えるクイックリファレンスです。書き方を思い出せないときや、素早く決めたいときは、直接ここを開いてください。「ポケットカード」として使うのがおすすめです。よく使うコードの速査、よくある質問、学習リソースを、*1* ページも漏らさずまとめました。

## よく使うコードのクイックカード

| やりたいこと | キーコード |
|---|---|
| 可変状態を 1 つ覚えておく | var x by remember { mutableStateOf(初期値) } |
| 画面回転でも状態を失わない | rememberSaveable { mutableStateOf(初期値) } |
| 縦並び / 横並びのレイアウト | `Column { }/Row { }` |
| 遅延読み込みリスト | LazyColumn { items(リスト) { ... } } |
| 画面遷移 | navController.navigate(” ルート”) |
| コルーチンで待ち時間の仕事をする | `viewModelScope.launch { ...}` |
| データフローを監視する | `val list by flow.collectAsState()` |
| 小さな設定を保存する | context.dataStore.edit { it[KEY] = 値} |
| 構造化データを保存する | Room の三種の神器：@Entity / @Dao / @Database |
| ネットからデータを取る | Retrofit インターフェース＋ suspend 関数 |
| 通知を出す | NotificationChannel + NotificationCompat.Builder |
| ログを出す | `Log.d("TAG", "message")` |

# よくある質問 FAQ

Q1：プログラミングがまったくできませんが、Android 開発を学ぶにはどれくらいかかりますか?

本書の道順に従いましょう。第 1 2 週で環境を整えて Kotlin の基礎を学び、第 3 4 週で Compose を学んで画面を作り、第 5 6 週で家計簿プロジェクトを完成させ、第 7 8 週で天気プロジェクトをやり遂げます。2 か月で、ゼロから小さなアプリを独力で作れるまで進めます。条件は、毎日 1 2 時間を投じて、本を読むだけでなく実際にコードを打つことです。

Q2：Java も学ぶ必要がありますか?

不要です。新しいプロジェクト、新しいチュートリアル、新しい求人は、ほぼすべて Kotlin 基準です。大量の Java に出会うのは「10 年前のプロジェクトの保守」を頼まれたときだけです。Kotlin を終えてから、仕事で必要になれば、Java の補習もすぐ済みます。両者の土台は同じ仮想マシンの世界だからです。

Q3：エミュレーター? それとも実機?

開発やデバッグにはエミュレーターが便利です（スナップショット、複数機種）。でも検収は必ず実機で行いましょう。操作感、性能、本物のネットワーク、センサーは、エミュレーターでは代えられません。条件が許すなら、ミドルクラスの実機を 1 台テスト機として買うのが、いちばん費用対効果の高い投資です。

Q4：Android 開発は仕事にありつきやすいですか?

2026 年の相場はこうです。単に「画面が書ける」だけの初級ポストは競争が激しいものの、「Kotlin が分かる＋アーキテクチャができる＋リリース作品がある」開発者は、いまだに引っ張りだこです。学習中のあなたにとって、いちばんいい履歴書は、ストアにある自分で作ったアプリです。

Q5：エラーで先へ進めなくなったら、どうすればいいですか?

黄金の 3 ステップです。(1) エラーの最初の行を、そのまま StackOverflow や検索エンジンで調べる。(2) 公式ドキュメントの該当ページを確認する。(3) それでもだめなら、「期待する動作＋実際の現象＋エラー全文＋関係するコード」をまとめて、AI やコミュニティに質問する。ほとんどの問題は、最初のステップで解決します。

## おすすめ学習リソース

| 種類 | リソース | おすすめの理由 |
|---|---|---|
| 公式 | developer.android.com | Android の公式ドキュメント。Compose や Kotlin のチュートリアルもここにあります。第一の権威 |
| 公式 | Kotlin 公式サイト (kotlinlang.org) | 言語の公式ドキュメント＋オンラインの演習場 |
| 公式 | Google Codelabs | 手取り足取りのステップ式実習コース。初心者へのやさしさは満点 |
| コミュニティ | Stack Overflow（android タグ） | エラーで助けを求める最初の場所 |
| コミュニティ | 掘金 / CSDN の Android コーナー | 中国語の実践記事が多い。新しさの見極めに注意（まず日付を見る） |
| 動画 | Google Developers 公式チャンネル | 毎年の開発者会議での新技術の解説 |
| 本書連動 | 第 7 部『AI プログラミング実戦編』（第 37　47 章） | AI で Android 開発を支援するための完全な心得 |

Android 開発の学習リソース一覧（2026 年 8 月）

## Gradle のよく使うコマンド速査

ふだんはボタンで十分ですが、コマンドラインのほうが効率的なこともあります（特に AI ツールや CI パイプラインと組み合わせるとき）。

| コマンド | 役割 |
|---|---|
| `./gradlew assembleDebug` | デバッグ版 APK をビルドする（Windows では./ を外す） |
| `./gradlew assembleRelease` | リリース版をビルドする（署名の設定が必要） |
| `./gradlew test` | ユニットテストを全部実行する |
| `./gradlew lint` | 静的チェック：潜在的なバグや行儀の悪い書き方を見つける |
| `./gradlew clean` | ビルド成果物を空にする（「再起動の大技」のコンパイル版） |
| `./gradlew :app: dependencies` | 依存ツリーを見る（バージョン競合の調査に便利な道具） |

Gradle のよく使うコマンド速査

**ワンポイント：ビルドでおかしくなったら、まず clean**

「昨日は動いたのに今日は動かない」という不思議な出来事は、十中六七がビルドキャッシュの乱れです。./gradlew clean のあとにビルドし直せば、オカルト的な問題の大半は治ります。それでもだめなら Android Studio を再起動しましょう——この技は File Invalidate Caches と併用するのがお約束です。

## おわりに：最初のアプリから N 個目のアプリへ

## おわりに：最初のアプリから N 個目のアプリへ

振り返れば、あなたは「Activity が何かも知らない」状態から始まって、環境を整え、Kotlin を学び、Compose で画面を描き、アプリにデータベースとネットワークをつなぎ、さらにテスト、パッケージング、出品まで進みました。ゼロから一を生み出す、いちばん大変な一区間を、歩き終えたところです。

技術の世界は変化が速いものです。フレームワークは更新され、道具は世代交代し、AI はますます多くの「キーボードを叩く仕事」を引き受けていきます。でも、ひとつだけ色あせないものがあります。ぼんやりしたアイデアを明確なステップに分解し、それを一歩ずつ実現していく力です。道具は増幅器、考え方は地盤です。増幅器は取り替えられますが、地盤は動かせません。

あなたはもう「スマホが使える人」だけではありません。頭の中のアイデアを、数億台の Android 端末で実際に動くアプリに変えられるのです。今日からキーボードで創造しよう——これが Android 開発者のロマンです。

## 参考文献

1. Android Developers. (2026). Android Developer Guides and Jetpack Compose Documentation. developer.android.com.
2. Kotlin Foundation. (2026). Kotlin Language Documentation. kotlinlang.org.
3. Android Open Source Project. (2026). Android Platform Version History and API Levels.
4. Google Play. (2026). Play Console Help: Publishing and App Signing.
5. AndroidX Release Notes. (2026). Room / DataStore / Navigation / WorkManager releases.
6. Square. (2026). Retrofit Documentation. github.com/square/retrofit.

# 九、iOS 開発実践編（第 59-69 章）

# 第五十九章：iOS を知る――Apple の生態系と開発の生態系

この決断、おめでとうございます。「*iPhone* を使う人」から「*iPhone* のためにアプリを書く人」になる第一歩です。手を動かす前に、まず地図を広げましょう。*iOS* はどこから来たのか？　*Apple* の「壁に囲まれた庭」は開発者にとって何を意味するのか？　アプリが、あなたがキーボードを叩いてからユーザーのスマホに届くまで、何が起きているのか？

## iOS の生い立ち：1 台のスマホがひとつの業界を変えた

2007 年 1 月、ジョブズは発表会で初代 iPhone を取り出しました。当時はサードパーティ製アプリにすら対応していませんでした。2008 年になって Apple が App Store と iPhone SDK を出し、世界中の開発者が iPhone 向けのソフトを書けるようになります。その瞬間に「アプリ経済」が生まれました。今日では App Store には数百万のアプリが並び、開発者への累計の分配額は数千億ドルを超えています。

システム自体も進化しています。最初は iPhone OS という名で、2010 年に iOS へ改名。バージョン番号は iOS 17（2023 年）まで進み、2025 年には Apple が命名を年に合わせる方式へ変えました。iOS 26 は 2025 年にリリースされ、2026 年 8 月時点の最新は iOS 27 です。名前は変わりましたが、変わらないことがひとつあります。iOS ユーザーの支払い習慣とソフトの品質への評判は、モバイルの生態系でいちばん良いと認められていることです。同じアプリでも、iOS 側の収入は Android 側を大きく上回ることがよくあります。

| 節目 | 年 | 開発者にとって重要なこと |
|---|---|---|
| App Store の開始 | 2008 | サードパーティ開発の生態系が始動 |
| Swift の登場 | 2014 | 現代の iOS 開発言語が登場 |
| SwiftUI の登場 | 2019 | 宣言型 UI フレームワーク。本書の本編 |
| iOS 26（改名の初年度） | 2025 | 命名が年号方式へ。Apple Intelligence を全面的に統合 |
| iOS 27 | 2026 | 現行の最新版（本書の基準） |

iOS 開発の年表（2026 年 8 月時点）

## 壁に囲まれた庭：Apple の生態系の「ルール」と「配当」

Apple の生態系は、よく「壁に囲まれた庭」（Walled Garden）と表現されます。ルールが多く、管理が厳しい。アプリは審査を通らなければ出品できず、開発には Mac と Xcode が必須で、配信は主に App Store 経由です。窮屈に聞こえますか？　でも開発者にとって、このルールの代わりに、本物の金になる配当が返ってきます。

- デバイスの断片化がとても小さい：iPhone の型番はごくわずかで、画面サイズもシステムバージョンも高度にそろっています。5 機種に対応すれば、ユーザーの 95% をカバーできます（Android では数十機種が必要になることも）。
- システムバージョンの更新率がとても高い：新しいシステムが出て数週間のうちに、ユーザーの 7　8 割がアップデートします。新しい API も思い切って使えて、「5 年前の古いシステム」への配慮は不要です。
- ユーザーの支払い習慣が良い：App Store のユーザーは、アプリやアプリ内課金にお金を払いやすい。
- 審査は厳しい反面、ストアがきれい：コピー品や迷惑なソフトが少なく、良い製品は見つけてもらいやすい。

**注意：まずコストをはっきり**

iOS 開発には高いハードルがあります。Mac が 1 台必須（Xcode は macOS 専用）で、出品には年 99 ドルの開発者アカウントが必要です（個人開発者の無料アカウントでも、ローカルでの実機デバッグはできますが、出品はできません）。この投資をよく考えてから旅立ちましょう。とはいえ朗報です。学習の段階なら無料アカウントで十分で、第六十章で説明します。

## どの言語で書くか：Objective-C？　それとも Swift？

iOS 開発の歴史にも、2 人の「主役」がいます。Objective-C は開国の功労者で、2008 年から 2014 年まで使われました。Swift は Apple が 2014 年に発表した新しい言語で、その後 Swift 6（2024 年リリース、厳格な並行処理チェックを導入）まで進化を続けてきました。2026 年のいま、新しいプロジェクトはほぼすべて Swift です。

ゼロからの人への、正直なひと言です。まっすぐ Swift を学びましょう。Objective-C は見る必要すらありません——入社後に 10 年前の遺産コードを保守することになる場合を除きます。Swift の利点はこうです。文法がすっきりしている（C 言語時代のセミコロンやヘッダーファイルのしがらみがない）。オプショナル（Optional）の仕組みが、言語のレベルで「ヌルポインタクラッシュ」をなくす。SwiftUI とシームレスに組み合わさる。さらに iPad の Swift Playgrounds で、遊びながら学べる。

Objective-C 風（古代の遺跡）

```
NSString *name = @"山田太郎"; [name stringByAppendingString:@"こんにちは"]; // 角括
    号＋ポインタ＋セミコロンの3点セット
```

Swift 風（現代のすっきりスタイル）

```
var name = "山田太郎" name += "こんにちは" // 自然な言葉のように率直
```

## アプリはどうやって生まれるのか？

どのプラットフォームでも同じように、iOS アプリの誕生にも一連の流れがあります。ただし Apple 版には、「保安検査」が 2 つ余分にあります。

アイデア

設計（画面 + 機能）

コードを書く（Swift + SwiftUI）

コンパイルと署名

テスト（エミュレータ + 実機）

Apple の審査

App Store にリリース

図 1：iOS アプリがアイデアから出品までたどる全工程（余分な「審査」に注目）

流れの中の重要な用語です。

- 署名（Signing）：iOS アプリは必ず「署名」しなければインストールできません。「このパッケージはあなたが作ったもので、改ざんされていない」ことの証明です。学習の段階では Xcode があなたの Apple ID で自動署名し、出品のときには開発者証明書を使います。詳しくは第六十七章で。

- シミュレーター（Simulator）：Mac 上に仮想で立てた iPhone で、開発とデバッグの主戦場です。Mac の CPU で動くので特に軽快で、Android のエミュレーターより体験が格段に良いです。

- TestFlight：Apple 公式の「テスト配布」ツールです。出品の前にテスト版を最大 10,000 人のテスターへ送り、彼らは TestFlight アプリでインストールして試します。iOS の生態系ならではの、上品な仕組みです。

## iOS 技術スタックの全体像：あなたの「武器リスト」

| レイヤー | 主流の選択肢（2026 年） | 何に使う？ |
|---|---|---|
| 言語 | Swift 6 | ロジックを書く言語 |
| UI（画面） | SwiftUI ＋ Apple デザインガイドライン（HIG） | コードで画面を描く（本書のメイン路線） |
| アーキテクチャ | MVVM ＋ @Observable | 画面とロジックを分ける |
| データ保存 | SwiftData（データベース）＋ UserDefaults（設定） | データをスマホに保存する |
| ネットワーク | URLSession ＋ async/await ＋ Codable | サーバーとやり取りする |
| ビルドツール | Xcode Build System / xcode-build | コードをインストール用パッケージに変える |
| 開発環境 | Xcode（macOS 専用） | Apple 公式の IDE |

2026 年の iOS 主流テックスタック早見表

気づきましたか？　このリストは Android よりずっと短い——Apple が多くのことをぜんぶ「お任せ」にしてくれているからです。閉じたエコシステムの代償は自由度、見返りはラクさ：どのネットワークライブラリにするか、どのデータベースにするか悩む必要はなく、公式が用意したものがたいてい最適解です。

**やってみよう：あなたの iPhone の「Apple 戸籍」をのぞいてみる**

iPhone の［設定］［一般］［情報］を開いて、3 つをメモしましょう：iOS のバージョン、機種名、使用可能な容量。さらに App Store の「今日」タブをぶらぶら眺めて、優れたアプリのスクリーンショットや説明文に注目してみてください——今日から、お店を見る目は「ユーザー」から「開発者」へ切り替わります。

**本章のまとめ**

- 2008 年に App Store が登場し「アプリ経済」が誕生。2026 年の現行バージョンは iOS 27
- 「囲われた庭」の収支：ルールは多いが、機種は少なくバージョンは新しく、ユーザーはお金を払い、ストアはきれい
- ハードル：Mac が必須。公開には年 99 ドルの開発者アカウントが必要。学習段階なら無料アカウントで十分
- 言語はいきなり Swift を学ぶ。Objective-C はレガシーコードだけのもの
- 流れは Android より 1 段多い：Apple の審査。TestFlight はベータテスト配布の強い味方
- 2026 年の標準テックスタック：Swift 6 ＋ SwiftUI ＋ MVVM ＋ SwiftData ＋ URLSession ＋ Xcode

# 第六十章：開発環境を整える——Xcodeと最初のプロジェクト

仕事をうまくやるには、まず道具をととのえること。この章では「キッチン」を整えます：*Xcode* を入れて、シミュレータを動かして、人生初の *iOS* プロジェクトを作り、さらにプロジェクトの構造をばらして丸ごと理解します。環境が整えば、自分が書いたコードが「押せるボタン」になる瞬間に初めて出会えます——この瞬間を一生覚えている人は多いものです。

## Mac を 1 台用意する

まずハードルの話：iOS 開発は macOS 上で行う必要があります。Mac を持っていない場合、選択肢は 3 つ。1 台買う（中古の Mac mini や MacBook Air でも十分）、短期だけクラウド Mac を借りる、あるいは先に第 8 編『Android 開発実践編』（Windows で OK）を読む——この 3 つです。

| 項目 | 最低要件 | 快適ライン（おすすめ） |
|---|---|---|
| チップ | Apple Silicon（M1 以降） | M2 以降 |
| メモリ | 8GB | 16GB |
| ストレージ | 40GB 以上を確保（Xcode 本体はかなり大きい） | 256GB 以上 |
| OS | 最新の macOS | 自動更新をオンのまま |

Mac 開発マシンの構成目安（2026 年 8 月）

**ワンポイント：Mac がなくても、言語から先に学べる**

iPad の無料アプリ Swift Playgrounds なら、Mac なしで Swift の基礎が学べます（iPhone にも入れます）。まず第六十一章の言語の知識を Playgrounds で遊びながら慣らしておき、Mac を手に入れたらそのまま飛び立てます。「薪を割る前に斧を研ぐ時間は無駄にならない」というわけです。

## Xcode をインストールする

Xcode は Apple 公式の統合開発環境で、無料です。3 ステップで入れましょう：

1. Mac で App Store を開き、「Xcode」と検索してインストールをクリック——数 GB あるので、ネットが快適な時間帯を選びましょう。

2. インストール後の初回起動で規約に同意し、必要なコンポーネントを入れます（自動で完了します）。

3. メニューの Xcode Settings Locations で、Command Line Tools が選択されているか確認します——コマンドラインツールは、多くのスクリプトやサードパーティツールの前提になるものです。

**注意：バージョンと OS の「組み合わせゲーム」**

新しい Xcode はたいてい最新の macOS を求め、古い Mac は古いバージョンで止まってしまいます。App Store に公開するとき、Apple は比較的新しい Xcode でのビルドを求めます（たいてい猶予期間はあります）。だから OS の更新はサボらないでください——iOS 開発者は「最新版を追いかける」のがいちばん勤勉な集団なのです。

## 最初のプロジェクトを作る

Xcode を開いて Create New Project iOS を選択 App テンプレートを選び、次のように入力します：

| 項目 | 入力するもの | 説明 |
|---|---|---|
| Product Name | MyFirstApp | アプリ名 |
| Team | あなたの Apple ID（無料） | 署名用。個人アカウントで OK |
| Organization Identifier | com.example | Product Name と組み合わせてパッケージ ID になる |
| Interface | SwiftUI | 本書のメインで使うフレームワーク |
| Language | Swift | 説明は不要でしょう |

プロジェクト作成で入力する 5 つの項目

Create を押すと、Xcode はそのまま実行できるプロジェクトを生成します。左上でシミュレータを 1 台選び（たとえば iPhone 17）、> を押して実行——数秒後、シミュレータの画面に「Hello, world!」と書かれたウィンドウが現れます。あなたの初めての iOS アプリが誕生した瞬間です。

## プロジェクト構造の分解：ファイルごとの役割

左側はプロジェクトナビゲータです。初心者がまず知っておけばいいのはこのくらい：

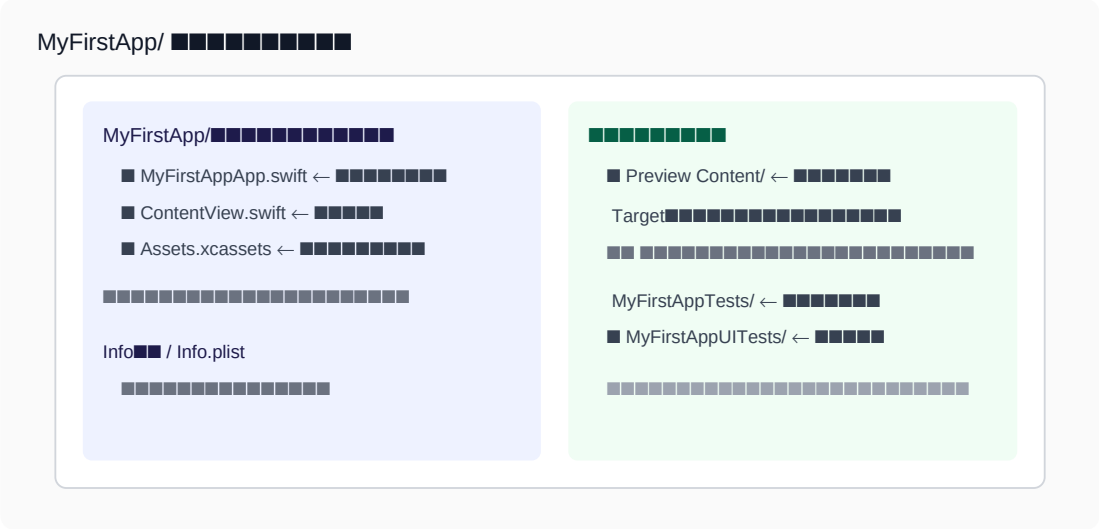



図 2：初心者が必ず知りたい Xcode のプロジェクト構造

- MyFirstAppApp.swift：プログラムの入口。SwiftUI ではわずか数行——「起動して、メイン画面を表示する」だけです。

- ContentView.swift：メイン画面のビュー。いまあなたが見ているものは、ぜんぶここにあります。

- Assets.xcassets：リソースの倉庫——アイコン・画像・色はここに置きます。

- Target の設定：左側の青いプロジェクトアイコンをクリックして入ります。バージョン番号、対応最低 iOS バージョン、署名はぜんぶここにあります。アプリの表示名やアイコンもここで設定します。

## 実機デバッグ：アプリをあなたの iPhone に入れる

## Xcode の時短テクニック：初心者に効く「ショートカット裏ワザ」5 選

これを使いこなせば、あなたはグッと「プロっぽく」見えます：

| 操作 | ショートカット | 効果 |
|---|---|---|
| 実行 | CmdR | ビルドして動かす |
| キャンバスプレビューの表示 / 切り替え | OptionCmd | 見たままがそのまま出るスイッチ |
| インデントを整形 | CtrlI | 選択したコードのインデントを自動で整える |
| ファイルをすばやく開く | CmdO | ファイル名を入れて即オープン（いちばん使う！） |
| コメント化 / コメント解除 | Cmd/ | デバッグ中にコードを手軽にオン・オフする |

Xcode のよく使うショートカット（初心者の基本 5 選）

さらに 2 つの隠れた特典：シミュレータの複数起動（iPhone と iPad を同時に動かしてレイアウトを比較）と、コードスニペット庫（よく使う数行を右側の Snippets パネルにドラッグしておけば、次からワンクリックで挿入）——効率化ツールはまず使ってみると、手感は自然についてきます。

## 実機デバッグ：アプリをあなたの iPhone に入れる

シミュレータも便利ですが、ゴールは実機です。無料の Apple ID でも実機デバッグはできます：

1. ケーブルで iPhone をつなぐ（または同じ Wi-Fi でワイヤレスデバッグ）。
2. Xcode 上部のデバイスリストで自分の iPhone を選びます。初回は Apple ID でのサインインと、署名の自動設定を案内されます。
3. iPhone の［設定］［一般］［VPN とデバイス管理］で、開発者証明書を信頼します。
4. ＞ を押すと、アプリが実機に入ります。

**注意：無料アカウントの「7 日間制限」**

無料 Apple ID で署名したアプリは 7 日後に使えなくなります（もう一度実行すれば更新されます）。これは Apple の不正利用防止の仕組みで、学習段階なら十分です。有料開発者アカウント（年 99 ドル）ならこの制限はありません。詳しくは第六十七章で。

## さっそく試す＋よくあるエラーの応急処置

## さっそく試す＋よくあるエラーの応急処置

ContentView.swift を開いて、Text(“Hello, world!”) の中の文字を自分の名前に変え、CmdR でもう一度実行——数秒後、シミュレータに自分の名前が表示されます。おめでとう

ございます。「コードを直すビルドインストール実行」という、最初の完全なサイクルを終えました。

| エラーの症状 | よくある原因 | 対処法 |
|---|---|---|
| Signing エラー | Team が未選択、証明書の期限切れ | Target Signing で自分のアカウントにチェック。無料アカウントならもう一度実行 |
| シミュレータが真っ暗 / 開かない | イメージが壊れている | メニューの Device Erase All Content。またはシミュレータを削除して作り直す |
| Xcode のダウンロードが遅い / インストールできない | App Store のネットワーク不安定 | 時間帯を変える。または Apple 開発者サイトから.xip オフラインパッケージをダウンロード |

初心者の環境エラー応急処置表

**やってみよう：「開発の儀式」リストを完成させる**

(1)Xcode を入れてシミュレータを動かす。(2) 実機で自分のアプリを 1 回実行し、スクリーンショットを記念に残す。(3)Hello の文字を自分の名前に変える。(4)Target 設定でアプリの表示名を「わたしの初めてのアプリ」に変える。この 4 ステップを終えれば、あなたはもう「作品を持つ人」です。

**本章のまとめ**

- ハードル：Mac ＋ Xcode。Mac がなければ、まず iPad の Swift Playgrounds で言語を学ぶ
- Xcode は App Store から。新バージョンは新しい OS を求めるので、更新はサボらない
- 新規プロジェクトの 5 要素：名前、Team、Organization Identifier、SwiftUI、Swift
- プロジェクトの合言葉：コードはみんな MyFirstApp/フォルダの中。Target 設定がバージョンと署名を担当
- 無料 Apple ID の実機デバッグには 7 日間の署名制限。有料アカウントなら制限なし
- 開発の毎日＝コードを直す CmdR 結果を見る、の繰り返し

# 第六十一章：Swift にすばやく入門——Apple 開発の第一言語

この章から、いよいよ言語を学びます。緊張しないでください——*Swift* は「初心者にやさしい」で有名ですし、文法書を丸暗記する必要もありません。文法の *20%* をまず押さえれば、日常のコードの *80%* は読めて書けます。残りはプロジェクトを進めながら調べれば *OK*。私たちの学習の原則を忘れずに：理解する＞暗記する、使える＞完璧。

## 変数と型：データに名前をつける

プログラムとは「データ＋ロジック」です。変数は、データに名前をつけてしまっておくもの。Swift では let と var で宣言します：

```
let name = "たろう"        // let：定数（デフォルトはこれがおすすめ）
var age = 18              // var：変数（あとから変えられる）

age = 19                  // <i class="fas fa-check-circle mr-1"></i> varは変えられる
// name = "はなこ"         // <i class="fas fa-times-circle mr-1"></i> letは変えられない。コンパイルエラー

let score: Int = 100      // 型を明示的に書いてもいい
let pi = 3.14             // 書かなくても自動で推論される

print("こんにちは、\(name)")   // 文字列補間：バックスラッシュ + 括弧
```

よく使う型は、この 1 枚の表で覚えましょう：

| 型 | 何を入れるか | 例 |
|---|---|---|
| Int | 整数 | 42 |
| Double | 小数 | 3.14 |
| String | 文字列 | “こんにちは” |
| Bool | 真 / 偽 | true / false |
| [Int]（配列） | データのまとまり | [1, 2, 3] |

Swift のよく使う基本型

**ワンポイント：let 優先の原則**

Swift コミュニティの鉄則：デフォルトは let、本当に必要なときだけ var。let で足りる場所に var を使うと、Xcode が黄色い文字で警告してくれます。この習慣があるだけで、「変数が意図せず書き換わる」系のバグの一大カテゴリを減らせます——変化の少ないコードほど、頭の中で追いやすいのです。

## オプショナル（Optional）：Swift の「クラッシュ防止のお守り」

これは Swift でいちばん重要な概念で、ほかにはありません。ひとことで言えば、値が「存在しない」かもしれないとき、Swift はあなたに処理させる——「ヌルポインタクラッシュ」という業界の悪夢を、言語レベルで消し去ったのです。

```
var name: String = "たろう"   // 普通の型：nilは絶対に許されない
// name = nil                 // <i class="fas fa-times-circle mr-1"></i> コンパイラ
    が直接止めてくれる

var nickname: String? = nil  // 疑問符を付けると：nilを許す（オプショナル）
nickname = "たーくん"

// オプショナルを使うときは「ほどく」必要がある：
if let n = nickname {        // if let：安全にほどく
    print("こんにちは、\(n)")
} else {
    print("まだニックネームはない")
}

let len = nickname?.count ?? 0   // ?? でデフォルト値（nil結合演算子）
print(len)
```

**注意：！は「危険な操作」**

コードに nickname!（強制アンラップ）と書くのは、コンパイラに「nil じゃないって保証する！」と叫ぶのと同じ——外れたらアプリはその場でクラッシュします。初心者のうちは if let / ?? を使い、! は使わないのが原則。Xcode がコードを補完するとき、たまに! を勝手に付けてくるので、直すのを忘れずに。

## 関数とクロージャ：ロジックを「ボタン」にまとめる

関数は名前のついたロジックのかたまりで、リモコンのボタンのようなもの。クロージャ（Closure）は「名前のない関数」で、その場でほかに渡せます：

```
// 関数：名前が先、引数と戻り値の型が後
func greet(name: String) -> String {
    return "こんにちは、\(name)！"
```

```
}
print(greet(name: "たろう"))      // 呼び出すときは引数名を付ける

// クロージャ：その場で定義。手軽でストレート
let numbers = [1, 2, 3, 4]
let doubled = numbers.map { $0 * 2 }   // $0 = 現在の要素
// [2, 4, 6, 8]

let big = numbers.filter { $0 > 2 }    // 絞り込み
// [3, 4]
```

SwiftUI ではクロージャが至るところに登場します。ボタンの Button(“押して”) { ... } の後ろの中かっこもクロージャです。map、filter、sorted という「コレクションのコンボ」はしっかり覚えておきましょう——この先のリスト処理の 90%は、この組み合わせです。

## 構造体とプロトコル：Swift の「レゴ」システム

Swift の公式のお好みははっきりしています。基本は構造体（struct）で、継承が必要なときだけクラス（class）。構造体は「値型」——コピーすればそれぞれ独立して動くので、ある種のバグが生まれにくいのです：

```
// struct：データを入れる箱
struct Record {
    var id = UUID()
    var amount: Double
    var category: String
    var date: Date

    // structは「メンバー逐一イニシャライザ」付き：すぐ作れる
}
let r = Record(amount: 25.5, category: "食費", date: Date())

// protocol：「どんな能力が必須か」の約束
protocol Identifiable {
    var id: UUID { get }
}
struct Record: Identifiable { ... }   // protocolへの準拠 = 要件の実装を約束すること
```

SwiftUI ではほぼすべてが struct です（View はプロトコル）。いまはピンとこなくて大丈夫。第六十二章で毎日のように顔を合わせます——Swift の世界では、まず struct を受け入れてしまいましょう。

## ゼロからの初心者のための文法チートシート

| やりたいこと | 書き方 |
|---|---|
| 定数 / 変数の定義 | `let x = 1` / `var x = 1` |
| 文字列への値の埋め込み | “こんにちは、\(name)” |
| 関数の定義 | func 名前 (引数: 型) -> 戻り値の型 { ... } |
| 条件分岐 | if 条件 { ... } else { ... } |
| 繰り返し | for x in 配列 { ... } |
| 安全なアンラップ | if let x = オプショナル値 { ... } |
| デフォルト値 | x ?? デフォルト値 |
| 構造体の定義 | `struct X { var a: Int; var b: String }` |
| クロージャ | `{ $0 * 2 }` |

Swift 文法チートシート（初心者版）

**やってみよう：Playgrounds で Swift を 15 分遊ぶ**

Mac があれば Xcode Playground を開きましょう。なければ iPad の Swift Playgrounds や、ブラウザ版のプレイグラウンドで OK。この章の各コードをすべて実行し、数字や文字をいじって、わざとエラーをいくつか起こして Xcode がどう教えてくれるか見てみてください。この「遊んでから学ぶ」15 分のおかげで、第六十二章に入るときのハードルは半分になります。

**本章のまとめ**

- let を優先、var は慎重に。型は自動推論、コロンのあとに明示宣言しても OK
- オプショナルは Swift の魂：空かもしれないなら必ず処理する。if let / ?? はお守り、! は危険な操作
- 関数呼び出しには引数名をつける。クロージャ＝名前のない関数で、SwiftUI では至るところに登場
- struct を優先し、プロトコル（Protocol）に従う。SwiftUI ではほぼすべてが struct
- 文法の 20%で仕事の 80%：まず使えるようになって、それから極める

# 第六十二章：画面づくり——SwiftUI 入門

画面とは、ユーザーの目に触れるすべてです。*SwiftUI* は *Apple* が *2019* 年に発表した宣言型 *UI* フレームワーク——画面を「宣言」するだけで、描画、ダークモード対応、さまざまな画面への対応はシステムが自動で引き受けます。*2026* 年のいま、*SwiftUI* は新規プロジェクトの圧倒的な主流です。この章では「お絵かき」を学びましょう。

## SwiftUI の考え方：宣言型であって命令型ではない

昔ながらの画面づくりは「命令型」です：システムに「ボタンを作る (20,30) に置く色を設定する」と、一歩ずつ指示します。SwiftUI は「宣言型」:「画面には『押して』と書かれた赤いボタンが 1 つある」と直接記述すれば、どう描くか、いつ描き直すかはシステムが面倒を見てくれます。

```
import SwiftUI

struct ContentView: View {
    var body: some View {      // body: 「画面の見た目」を記述する
        VStack {               // 縦に並べる
            Text("こんにちは、世界！")
                .font(.largeTitle)      // 修飾子：大きな見出し
            Button("押して") {
                print("押された")
            }
            .buttonStyle(.borderedProminent)  // 青い塗りつぶしボタン
        }
        .padding()
    }
}
```

要点：ビュー＝ View プロトコルに従う構造体。body プロパティが画面を記述し、. モディファイア () でつぎつぎ「メイク」していく。これが SwiftUI の基本のすべてです。

## 基本のビュー：テキスト、ボタン、入力欄、画像

| ビュー | 役割 | 例 |
|---|---|---|
| Text | 文字を表示 | Text(” こんにちは”) |
| Button | ボタン | Button(” 押して”) { 動作} |
| TextField | 入力欄 | TextField(” ヒント”, text: $テキスト) |
| Image | 画像を表示 | `Image(systemName: "star.fill")` |

SwiftUI の 4 大基本ビュー

Image(systemName:) に注目——Apple は SF Symbols というアイコンライブラリを内蔵していて（数千個の無料アイコン）、“star.fill” のような名前を書くだけで、塗りつぶしの星がそのまま表示できます。これぞ iOS 開発の隠れた特典：画面づくりでは、ほぼアイコン素材探しが不要です。

## レイアウトの 3 点セット：VStack、HStack、ZStack

- VStack：縦に並べる（食堂の行列のように）。
- HStack：横に並べる（並んで座るように）。
- ZStack：重ねて置く（トランプを積むように。バッジやオーバーレイにぴったり）。

```
VStack(alignment: .leading, spacing: 12) {
    HStack {
        Text("請求書")
        Spacer()                    // 伸びる空間：右端まで押しやる
        Text("-25.5 元").foregroundColor(.red)
    }
    Divider()                       // 区切り線
    Text("メモ：昼ごはん")
}
.padding(16)                        // 四方に余白
.background(Color(.systemGray6)) // 薄いグレーの地
.cornerRadius(12)                   // 角丸

// 修飾子の順番に意味がある：先に padding、後に background。背景は余白を含まない
```

**注意：モディファイアの順番には意味がある**

padding(16).background(グレー) と background(グレー).padding(16) は結果が違います——前者は「余白を取ってから色を塗る」（背景に余白は含まれない）、後者は「色を塗ってから余白を取る」（背景に余白が含まれる）ためです。初心者にいちばんよくある「変

な白いすき間」は、みんなここから来ています。迷ったら順番を入れ替えてみましょう。プレビューがすぐ答えを教えてくれます。

## 状態とバインディング：画面を「生き」させよう

静的な画面では面白くありません。SwiftUI のルール：画面はつねに状態を忠実に映し出し、状態が変われば画面は自動で描き直されます。

```
struct CounterView: View {
    @State private var count = 0   // @State：この値が変わったら再描画

    var body: some View {
        VStack {
            Text("\(count)回押しました")
            Button("押して +1") {
                count += 1          // 状態を変えると画面が自動更新
            }
        }
    }
}
```

入力欄には「双方向バインディング」を使います——$ マークが「値と書き換え権」をセットで渡してくれます：

```
struct InputView: View {
    @State private var text = ""

    var body: some View {
        TextField("何か入力", text: $text)  // $による双方向バインディング
            .textFieldStyle(.roundedBorder)
        Text("入力した内容：\(text)")   // 入力中にリアルタイム更新
    }
}
```

ユーザー操作

状態が変わる（count += 1）

SwiftUI が変化を検知

画面を自動で再描画（body の再計算）

図 3：SwiftUI の「状態画面」一方向データフロー

### プレビュー、テーマ、アニメーション：SwiftUI の「裏ワザ」

- ファイルのいちばん下に #Preview { ContentView() } を足すと、右側のキャンバスがリアルタイムで描画——コードを直すと、見たままがそのまま反映されます。

- ダークモードはタダ：色を直書きせず、システムカラー（Color(.systemGray6)、.primary）を使えばダークモードに自動で対応し、1 行も追加で書く必要がありません。

- アニメーションは 1 行で OK：withAnimation { count += 1 } とするか、ビューに.animation(.default, value: count) を付ける——切り替えがぬるぬる滑らかに。

```
struct AnimatedHeartView: View {
    @State private var liked = false

    var body: some View {
        Button {
            withAnimation(.spring(duration: 0.4)) {
                liked.toggle()
            }
        } label: {
            Image(systemName: liked ? "heart.fill" : "heart")
                .font(.system(size: 48))
                .foregroundColor(liked ? .red : .gray)
                .scaleEffect(liked ? 1.3 : 1.0)
        }
    }
}
```

**ワンポイント：まず機能、アニメーションはそのあと**

初心者にありがちな失敗は「画面がまだ正しくできていないのに、アニメーションで悩み始める」こと。まず機能を通るようにして、動きは最後にまとめて足しましょう——アニメーションは華やかさの上乗せ、機能は困ったときの命綱です。これも私たちの一貫した「シンプル第一」の原則です。

やってみよう：「気分ボタン」を作る

## よく使うモディファイア早見表：この 1 ページがあれば、画面づくりはもう安心

| やりたいこと | モディファイア |
|---|---|
| 余白 | `.padding(16)` |
| 角丸 | `.cornerRadius(12)` |
| 背景色 | `.background(Color(.systemGray6))` |
| フォントと文字サイズ | `.font(.title.bold())/.font(.system(size: 20))` |
| 前景色（文字 / アイコン） | `.foregroundColor(.red)` |
| 幅いっぱいに広げる | `.frame(maxWidth: .infinity)` |
| サイズを固定 | `.frame(width: 100, height: 44)` |
| そろえ方 | `VStack(alignment: .leading)` |
| 不透明度 | `.opacity(0.6)` |
| 影 | `.shadow(radius: 5)` |
| タップ操作 | `.onTapGesture { ...}` |
| キーボードの種類 | `.keyboardType(.decimalPad)` |

SwiftUI のよく使うモディファイア早見表

覚えられなくても大丈夫——このページは調べるためのもの。実際にいちばん使うのは、上の 4 つ：padding、角丸、背景、フォントです。
テンプレートプロジェクトを「気分チェンジャー」に改造しましょう：ボタンを 1 つ用意して、タップするたびにうれしいかなしいおこりの間をぐるぐる切り替え、文字と背景色も連動して変わるようにします。ヒント：3 組（文字＋色）を配列に入れ、@State にインデックスを保存して、タップのたびに (index + 1) % 3。できたら友だちに送って遊んでもらい、「え、アプリ作れるの?」という最初のリアクションをもらいましょう。

本章のまとめ

- SwiftUI は宣言型：どんな見た目かを記述し、描くのも描き直すのもシステムまかせ
- ビュー＝ View プロトコルに従う構造体。body が画面を記述。モディファイアでつぎつぎ「メイク」
- 4 大基本ビュー：Text / Button / TextField / Image。SF Symbols のアイコンライブラリは無料でたっぷり
- レイアウト 3 点セット：VStack は縦並び、HStack は横並び、ZStack は重ね置き。Spacer は伸び縮みするすき間
- @State で画面が生き返る。$ の双方向バインディングで入力欄に値を渡す

- #Preview で見たままを確認。システムカラーならダークモードに自動対応。withAnimation で 1 行アニメーション

# 第六十三章：ビュー、状態、ナビゲーション

*1* ページだけのおもちゃなら、もう作れます。この章で解決するのは「成長」にまつわる *2* つの問題：複数のページの間をどう行き来するか（ナビゲーション）と、データがページの間をどう流れるか（状態管理）です。この章を終えれば、「リスト詳細編集」という定番のマルチページアプリの骨組みが組めるようになります。

## 状態管理の「データフロー全体図」

SwiftUI の状態管理ツールは @State だけではありません。まずこの全体図を見て、「いつどれを使うか」をつかみましょう：

| ツール | 担当するもの | 使う側 | 典型的な場面 |
|---|---|---|---|
| @State | そのページだけの一時的な状態 | いまのビュー | カウンター、入力欄の文字 |
| @Binding | ほかの状態を借りる | 子ビューから親ビューを書き換える | 自作コンポーネント |
| @StateObject / @Observable | ページをまたいで共有するロジック | ビューモデル（ViewModel） | データ読み込み、ビジネスロジック |
| @EnvironmentObject / .environment | アプリ全体で共有する状態 | アプリ全体 | ログイン状態、テーマ設定 |

SwiftUI の状態管理ツール全体図（2026 年、@Observable が主流）

```
// iOS 17+のモダンな書き方：@Observableマクロ（新規プロジェクトにおすすめ）
import Observation

@Observable
class CounterModel {                    // ビューモデル
    var count = 0
    func increment() { count += 1 }
}
```

```
struct CounterView: View {
    @State private var model = CounterModel()   // モデルを保持

    var body: some View {
        VStack {
            Text("\(model.count)回押しました")
            Button("押して") { model.increment() }
        }
    }
}
```

初心者へのぶっきらぼうなアドバイス：ページの中だけで使うものは @State、ページをまたぐもの / ロジックを含むものは @Observable のクラスへ。古いチュートリアルの @StateObject/@ObservableObject は前世代の書き方なので、読めれば OK。新しいコードは上のルールに従いましょう。

## NavigationStack：ページ移動の「スタック」

iOS 16 以降、複数ページのナビゲーションには NavigationStack を使います——お皿を積むようなもので、進むときに 1 枚ずつ積んでいき、戻るときは 1 枚ずつ取り出します：

```
struct HomeView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("たろうの家計簿") {
                    DetailView(name: "たろうの家計簿")   // 行き先のページ
                }
                NavigationLink("はなこの家計簿") {
                    DetailView(name: "はなこの家計簿")
                }
            }
            .navigationTitle("家計簿リスト")   // 上の大きな見出し
        }
    }
}

struct DetailView: View {
    let name: String
    var body: some View {
        Text("\(name) の詳細ページ")
            .navigationTitle(name)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

NavigationLink は「タップできる移動リンク」です。navigationTitle はナビゲーションバーのタイトルと戻るボタンを自動で生成します。プログラムで移動させたいとき（たとえば「保存」を押したら自動で戻る）は、@Environment(\.dismiss) で現在のページを閉じます——この 2 つの合わせ技で、ナビゲーションのニーズの 90%はカバーできてしまいます。

## ポップアップと下部タブ：sheet と TabView

ページを「押し込む」以外にも、定番の操作が 2 つあります：

```
// 1. 下から半画面/全画面で表示（入力フォームの定番）
struct HomeView: View {
    @State private var showAdd = false
    var body: some View {
        Button("記帳する") { showAdd = true }
            .sheet(isPresented: $showAdd) {
                AddRecordView()      // 表示される入力ページ
            }
    }
}

// 2. 下部タブバー（アプリの「骨格」）
struct MainTabView: View {
    var body: some View {
        TabView {
            RecordListView()
                .tabItem { Label("家計簿", systemImage: "book") }
            StatsView()
                .tabItem { Label("統計", systemImage: "chart.pie") }
            SettingsView()
                .tabItem { Label("設定", systemImage: "gear") }
        }
    }
}
```

**ワンポイント：どの場面でどれを使う？**

経験則：メインの流れは NavigationStack でページを push（リスト詳細）、一時的な操作は sheet のポップアップ（追加・編集・確認）、並び立つ大ブロックは TabView（下部タブ）。この 3 つの組み合わせこそ、ほとんどのアプリの骨組みです。

## List：iOS の「国民的リスト」

iOS でいちばんよく使い、いちばん「Apple らしい」コンポーネントが List です——区切り線、スワイプ削除、タップ時のハイライトが、はじめから付いてきます：

```
struct RecordListView: View {
    @State private var records = ["昼ごはん 25.5", "地下鉄 6.0", "給料 +8000"]

    var body: some View {
        List {
            ForEach(records, id: \.self) { r in
                Text(r)
            }
            .onDelete { indexSet in          // 左スワイプで削除
                records.remove(atOffsets: indexSet)
            }
        }
    }
}
```

リスト＋ NavigationLink ＋ sheet ＋ TabView——この 4 つを組み合わせれば、「帳簿リストタップで詳細＋で入力ポップアップ」という完全なアプリの骨組みがもう組めます。次章の家計簿プロジェクトは、まさにこの組み方で作ります。

やってみよう：「3 ページの骨組み」を組む

## アプリのライフサイクル：フォアグラウンドとバックグラウンド

Android の Activity のライフサイクルと同じように、iOS アプリにも「誕生、フォアグラウンド、バックグラウンド、終了」という状態の変化があります。SwiftUI では scenePhase という環境値で観察します：

```
@main
struct MyApp: App {
    @Environment(\.scenePhase) private var phase

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .onChange(of: phase) { _, newPhase in
            switch newPhase {
            case .active:    print("前面に戻る。更新を再開")
            case .inactive:  print("移行中（着信/バックグラウンド切替の瞬間）")
            case .background: print("バックグラウンドへ。下書きを保存")
            @unknown default: break
            }
        }
    }
}
```

典型的な使い方：フォアグラウンドに戻ったらデータを更新、バックグラウンドに入っ

たら下書きを保存。iOS はバックグラウンド動作にとても厳しく——普通のアプリはバックグラウンドに入るとすぐ停止（凍結）されるので、裏でこっそり重い処理をさせるのは期待しないでください。どうしても定期実行したいなら、「バックグラウンド常駐」ではなく第六十八章のローカル通知を使います。

ダミーデータ版の家計簿の骨組みを作りましょう：TabView の 3 つのタブ（帳簿 / 統計 / 設定）帳簿ページは List でダミーデータを表示し、行をタップしたら詳細へ（NavigationStack）右上の「＋」ボタンで入力用の sheet を出す。データ保存も通信もなし、純粋に骨組みの練習——この骨組みができていれば、第六十五章で本物のデータを流し込むのは水到渠成です。

### 本章のまとめ

- 状態管理の 4 ツール：@State（ページ内）、@Binding（借用）、@Observable（ロジック）、.environment（全体）
- 新規プロジェクトのビューモデルは @Observable マクロで書く。古いチュートリアルの @ObservableObject は読めれば OK
- ページ移動は NavigationStack。@Environment(\.dismiss) でプログラムから戻る
- メインの流れはページを push、一時的な操作は sheet、並び立つ大ブロックは TabView
- List は「国民的リスト」：区切り線もスワイプ削除も、はじめから使える

# 第六十四章：データ保存とネットワーク通信

「見る」だけのアプリは生きていません。データを保存できて、ネットにつながるアプリこそが本物です。この章では、*iOS* のデータ層の「*3* 点セット」を紹介します：*UserDefaults*（小さな設定を保存）、*SwiftData*（構造化データを保存）、*URLSession*（サーバーとやり取り）。さらに *JSON* を解析する *Codable* と、現代的な *async/await* も加わります。この章を終えれば、ユーザーの使い方を覚えて、内容をリアルタイムで更新できる本物のアプリが書けるようになります。

## 保存方式の選び方：まずこの表を見る

| 方案 | どんなデータに向くか | 特徴 | 本書での出番 |
| --- | --- | --- | --- |
| UserDefaults | こまごました設定（スイッチ、好み） | キーと値のペア。シンプルでストレート | 第六十五章の設定、第六十六章のお気に入り都市 |
| SwiftData | 構造化データ（収支、記録） | 公式データベースのラッパー。@Model の 1 行で宣言 | 第六十五章の収支記録 |
| URLSession | （保存せず、通信で取得）サーバーのデータ | システム標準のネットワークフレームワーク。サードパーティ依存なし | 第六十六章の天気データ |

データ 3 点セットの役割分担

合言葉をひとつ覚えましょう：「小さな設定は UserDefaults へ、まとまったデータは SwiftData へ、外のデータは URLSession で」。古いチュートリアルの Core Data は今も使えます（SwiftData はその現代版です）。手書きの SQLite は基本さわらなくて OK——新しい方をそのまま学び、古い道は通らないでください。

## UserDefaults：小さな設定を保存する

```
// 書く：値を保存
UserDefaults.standard.set(true, forKey: "isDarkMode")
UserDefaults.standard.set("北京", forKey: "favoriteCity")

// 読む：取り出す（なければデフォルト値を返す）
let isDark = UserDefaults.standard.bool(forKey: "isDarkMode")
let city = UserDefaults.standard.string(forKey: "favoriteCity") ?? "北京"

// 削除：キーを取り除く
UserDefaults.standard.removeObject(forKey: "isDarkMode")
```

注意したいのは、これはこまごました小さな設定にしか向かないということ——数十、数百件の記録は SwiftData に任せましょう。UserDefaults をデータベース代わりに使わないでください（起動が遅くなるうえ、検索も不便です）。

## SwiftData：データに「ちゃんとした家」を用意する

SwiftData は iOS 17 からの公式データベース方式で、「テーブル作成・追加・削除・更新・検索」を極限まで簡単にしてくれます——@Model という注釈を 1 つ付けるだけで、データのクラスがデータベースのテーブルになります：

```
import SwiftData

// 1. @Model：このクラスは自動的に「テーブル」になる
@Model
class Record {
    var amount: Double
    var category: String
    var note: String
    var date: Date

    init(amount: Double, category: String, note: String, date: Date) {
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}

// 2. アプリ入口にコンテナを接続
@main
struct MyApp: App {
    var body: some Scene {
```

```
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Record.self)   // データベース準備完了
    }
}

// 3. viewでは@Queryで読み、@Environmentで書く
struct RecordListView: View {
    @Query(sort: \Record.date, order: .reverse) var records: [Record]
    @Environment(\.modelContext) private var context

    var body: some View {
        List(records) { r in
            Text("\(r.category) \(r.amount) 元")
        }
    }

    func add(amount: Double, category: String) {
        let r = Record(amount: amount, category: category, note: "", date: Date())
        context.insert(r)     // 保存
        try? context.save()   // ディスクに書き込む
    }
}
```

この「@Model ＋ modelContainer ＋ @Query」のテンプレートは、そのままどんなプロジェクトにも持ち込めます——フィールド名を差し替えるだけで OK。データが変わると @Query が画面を自動更新してくれる、なんとも気持ちいい仕掛けです。

## Codable：JSON を「翻訳」する

サーバーからのデータは、たいてい JSON のテキストです。Swift の Codable プロトコルは「JSON 構造体」の変換を自動で行ってくれます——フィールド名が合っていれば、魔法が発動します：

```
// JSON: {"name":"北京","main":{"temp":26.5},"weather":[{"description":"晴れ"}]}
struct WeatherResponse: Codable {
    let name: String
    let main: Main
    let weather: [Weather]
}
struct Main: Codable { let temp: Double }
struct Weather: Codable { let description: String }

// JSON文字列  struct
let decoder = JSONDecoder()
```

```
let data = jsonString.data(using: .utf8)!
let result = try decoder.decode(WeatherResponse.self, from: data)
print(result.weather[0].description)   // "晴れ"
```

鉄則：まず本物の返り JSON を見てから、フィールドを定義すること。「解析に失敗した」系のバグの多くは、フィールド名の書き間違いや型の読み違い（サーバーが“26.5”という文字列を送っているのに、こちらは Double と宣言）です——元の JSON を print して見比べれば、10 秒で原因が突き止められます。

## URLSession ＋ async/await：サーバーと会話する

iOS の通信にはシステム標準の URLSession を使い、Swift の async/await（非同期関数）と組み合わせます。「ネットワークを待つ」のは「宅配便を待つ」のと同じくらい自然になります：

```
struct WeatherService {
    // 非同期関数：asyncは「少し待つ」の印、throwsは「失敗するかも」の印
    func fetchWeather(city: String) async throws -> WeatherResponse {
        let url = URL(string:
            "https://api.example.com/weather?city=\(city)")!
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}

// 呼び出し側（.taskの中）：
.task {
    do {
        let weather = try await service.fetchWeather(city: "北京")
        // 画面を更新……
    } catch {
        print("エラー発生：\(error)")
    }
}
```

**落とし穴注意：ネットワークの 2 大鉄則**

(1) 通信は必ず async/await で。メインスレッドは絶対に止めないこと——画面が固まるのは、クラッシュ並みの最悪体験です。Swift の構造化並行性なら自然に書けますが、try await と do/catch を忘れずに。(2)ATS はデフォルトで平文 HTTP を禁止します：iOS は HTTPS を求めます。ローカルの http://サービスをテストするときだけ、Info.plist で一時的に例外（NSAppTransportSecurity）を開けますが、正式な製品はすべて HTTPS に——審査でもここは見られます。

やってみよう：「今日のひとこと」アプリを作る

### ネットワークエラーも「レベル分け」を：すべてのエラーをユーザーに丸投げしない

通信断、タイムアウト、404、サーバー 500……エラーの種類によって、ユーザーに見せる言葉は変わるべきです：

```
func friendlyMessage(_ error: Error) -> String {
    if let urlError = error as? URLError {
        switch urlError.code {
        case .notConnectedToInternet: return "ネットが切れました。接続を確認してみますか？"
        case .timedOut:               return "リクエストがタイムアウトしました。後でもう一度？"
        case .cannotFindHost:         return "サーバーが見つかりません"
        default:                      return "ネットワークに問題があります：\(urlError.code)"
        }
    }
    if let d = error as? DecodingError {
        return "データの形式が違います（APIが変わったのかも）"
    }
    return "エラーが発生しました：\(error.localizedDescription)"
}
```

ユーザーには「人の言葉」を、ログには「原文」を（error の中身を print）。error.localizedDescription の英語の呪文を、そのままユーザーに投げつけないこと——製品の品質感は、こうした細部に宿ります。

無料の名言 API（hitokoto など）を探して、URLSession ＋ async/await で「今日のひとこと」を取得して画面中央に表示し、「次のひとこと」ボタンを付けます。気に入った文は SwiftData で「お気に入り」に保存しましょう。この練習は、UserDefaults/SwiftData/Codable/URLSession をぜんぶ一度に使います——これをやり終えれば、第 7・8 章は朝メシ前です。

本章のまとめ

- 合言葉：小さな設定は UserDefaults、まとまったデータは SwiftData、外のデータは URLSession
- SwiftData の 3 段構え：@Model でテーブル宣言、modelContainer でコンテナ接続、@Query で読み＋ context.insert で書き
- Codable が JSON を自動「翻訳」：フィールド名が合えば、魔法が発動
- URLSession ＋ async/await：通信は宅配便を待つ感覚。失敗は do/catch で処理
- 2 大鉄則：メインスレッドを止めない。正式な製品は必ず HTTPS（ATS）

# 第六十五章：プロジェクト 1——わたしの家計簿（ローカル版）

あなたの初めての「まともなプロジェクト」へようこそ。この家計簿はネットにはつながりませんが、中身は一通り揃っています：*1* 件の記録ができ、履歴が見られて、月ごとに集計できて、削除・編集もできます。完成すれば、「データロジック画面」の全工程をひととおり歩いたことになります。コードをいきなり写すのは急がないで——まずよく考えてから手を動かす。それが私たちのいつもの原則です。

## 要件を考える：まず 5 つの質問

| 要素 | 答え |
| --- | --- |
| Who（誰が使うか） | 自分用。家族や友だちでも OK |
| What（何を作るか） | 収支を記録、履歴を見る、月ごとにまとめる、カテゴリ別に集計 |
| Why（なぜ作るか） | 練習＋お金を何に使ったかをはっきりさせる |
| How（どう作るか） | Swift ＋ SwiftUI ＋ SwiftData ＋ MVVM。完全ローカル、サーバーなし |
| Constraint（制約） | 1 週間で完成。iOS 17 以上をサポート（SwiftData の要件） |

家計簿の要件 5 要素

そして「どの段階でも使える」小さなステップに分解します：(1)SwiftData のデータ層を作る（1 件保存できる）。(2) リストページ（履歴が見られる）。(3) 入力ポップアップ（記録を追加できる）。(4) 削除と編集。(5) 月次集計カード。1 ステップ終えるごとに、アプリは「使えるもの」になります。

## データ層：記録に「家」を用意する

まず 1 件の記録がどんな形かを定義します——フィールドは十分に、でも欲張らない：

```
import SwiftData

@Model
class Record {
    var type: String      // "income" 収入 / "expense" 支出
    var amount: Double    // 金額（元）
    var category: String  // カテゴリー：食費、交通、給料……
    var note: String      // メモ（任意）
    var date: Date        // 記帳日

    init(type: String, amount: Double, category: String, note: String = "", date:
    Date = Date()) {
        self.type = type
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}
```

**ワンポイント：金額はなぜ Double？**

本格的な金融アプリは、浮動小数点の誤差（0.1 + 0.2≠0.3 という昔からの問題）を避けるために Decimal（正確な十進数）を使います。練習用プロジェクトなら Double で十分です。いつか本物の金融商品を作るときは、Decimal か「分単位の Int」に切り替えるのを忘れずに——これは先人たちが何度も踏んできた落とし穴です。

## MVVM：画面とロジックの「世帯分け」

画面にデータベースを直接操作させる？　動きはしますが、画面のコードが長ったらしくなります。MVVM アーキテクチャなら、それぞれが持ち場を守ります：View（画面）は描くことだけ、ViewModel（ロジックの中枢）はデータを担当、Model（SwiftData）は保存を担当。

View（SwiftUI の画面）

ViewModel（@Observable のロジック中枢）

Model（SwiftData のデータ層）

図 4：MVVM の 3 層構造——データは一方向に流れ、各層は互いに踏み込まない

```
import Observation

@Observable
class RecordViewModel {
    var records: [Record] = []
```

```
    private let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func load() {
        // SwiftDataの#Predicateで絞り込み + 並べ替え
        let descriptor = FetchDescriptor<Record>(
            sortBy: [SortDescriptor(\.date, order: .reverse)]
        )
        records = (try? context.fetch(descriptor)) ?? []
    }

    func add(type: String, amount: Double, category: String, note: String) {
        let r = Record(type: type, amount: amount, category: category, note: note)
        context.insert(r)
        try? context.save()
        load()                          // 読み込み直す。画面が更新される
    }

    func delete(_ record: Record) {
        context.delete(record)
        try? context.save()
        load()
    }
}
```

画面の中で @State private var vm = RecordViewModel(...) としておき、vm.records をそのままリストに描画——データが変わると、画面も自動で更新されます。

## 画面：リストページ＋入力ポップアップ

```
struct RecordListView: View {
    @State private var vm: RecordViewModel
    @State private var showAdd = false

    var body: some View {
        NavigationStack {
            List {
                Section("今月の残高") {          // 集計カード
                    SummaryRow(records: vm.records)
                }
                Section("明細") {
                    ForEach(vm.records) { r in
                        RecordRow(record: r)
                    }
                    .onDelete { idx in
```

```
                        idx.forEach { vm.delete(vm.records[$0]) }
                    }
                }
            }
            .navigationTitle("私の家計簿")
            .toolbar {
                Button { showAdd = true } label: {
                    Image(systemName: "plus")
                }
            }
            .sheet(isPresented: $showAdd) {
                AddRecordView(vm: vm)          // 入力ダイアログ
            }
        }
    }
}
```

入力ポップアップのポイント：金額入力欄のキーボードは.keyboardType(.decimalPad)（数字キーボード）。収入 / 支出はセグメントコントロール（Picker）で、カテゴリはメニューで選択。「保存」を押したら vm.add(...) を呼び、dismiss() で閉じます。

**注意：入力チェックの手抜きは禁物**

金額入力欄は必ずチェックしましょう：空の値、負の数、“abc”のような不正な入力はすべてはじく（Double(text) で判定し、失敗すると nil が返ります）。アプリがクラッシュするいちばんよくある原因のひとつが、「ユーザーの入力をお行儀のいい子だと思い込む」ことです。忘れずに：ユーザーが入力したものは、すべて敵として審査する。

## 編集、削除、集計

- 削除：List の.onDelete で左スワイプ削除＋確認ポップアップ（うっかり削除は、家計簿系アプリのユーザーがいちばん恨むものです）。
- 編集：行をタップして編集ポップアップへ。元の値を先に入れておく（1 文字直すほうが、打ち直すよりずっとラク）。
- 集計：filter ＋ reduce で「今月の収入 / 支出 / 残高」を計算して、上部のカードに表示します：

```
func monthSummary(_ records: [Record]) -> (income: Double, expense: Double) {
    let cal = Calendar.current
    let monthRecords = records.filter {
        cal.isDate($0.date, equalTo: Date(), toGranularity: .month)
    }
    let income = monthRecords
        .filter { $0.type == "income" }
```

```
        .reduce(0) { $0 + $1.amount }
    let expense = monthRecords
        .filter { $0.type == "expense" }
        .reduce(0) { $0 + $1.amount }
    return (income, expense)
}
```

## 検収：3 層チェック＋ 2 つの上級チャレンジ

プロジェクトが完成したら、「機能境界アーキテクチャ」の 3 層で検収します：記録できるか、削除できるか、アプリを再起動してもデータが残っているか（機能）。金額に 0 や負の数を入れてもクラッシュしないか（境界）。画面とデータが混ざっていないか、フィールドがハードコードされていないか（アーキテクチャ）。

基本版の検収をパスしたら、自分に 2 品の「上級メニュー」を出しましょう：

1. カテゴリ別集計：「統計」タブを追加して、Swift Charts でカテゴリ別支出を円グラフ / 棒グラフに——Apple 公式のグラフライブラリで、数行のコードでグラフが描けます。
2. iCloud 同期：SwiftData の iCloud 同期は、設定をいくつか入れるだけ——帳簿が iPhone / iPad の間で自動同期されて、一気に「高級感」が出ます。

**やってみよう：今週中に完成させる**

「データ層 ViewModel リストページ入力ポップアップ集計カード」の順に手を動かし、1 日 1 ステップ。つまづいたら、エラーをそのまま検索エンジンや AI に投げましょう（第 7 編『AI プログラミング実践編』を覚えていますか？　問題を説明するときは「期待した動き、実際の動き、再現手順」をはっきりと）。完成したら、1 週間これで家計簿をつけてみてください——自分で作ったアプリを使うと、達成感が次へ進む力になります。

**本章のまとめ**

- 要件 5 要素「どの段階でも使える」小ステップに分解してから、着手する
- データ層：@Model モデル＋ modelContainer コンテナ＋ FetchDescriptor クエリ
- MVVM の分担：View は描くだけ、@Observable な ViewModel がロジック、SwiftData が保存を担当
- 画面：List ＋ Section ＋ toolbar ＋ sheet の入力ポップアップ
- 3 層検収（機能 / 境界 / アーキテクチャ）＋上級チャレンジ（Swift Charts、iCloud 同期）

# 第六十六章：プロジェクト 2——天気チェックアプリ（ネットワーク版）

家計簿が「内功の鍛錬」なら、天気アプリは「世間を知る」——インターネットと付き合うプロジェクトです。このプロジェクトで学ぶのは：無料 *API* の申し込み、*JSON* の解析、「読み込み中 / 成功 / 失敗」の *3* つの画面状態の扱い、そして *API* キーの隠し方。完成すれば、どんな「ネット接続アプリ」でも作れる力が身につきます——ニュースも為替も翻訳も、やり方はまったく同じです。

## 無料の天気 API を申し込む

個人の練習なら、OpenWeatherMap か中国国内の QWeather がおすすめ——どちらも無料枠があって、登録すればすぐ使えます。OpenWeatherMap を例にすると：

1. アカウント登録 API keys ページでキー（apikey）を 1 つ生成します。
2. 無料版で呼び出せるのは Current Weather API：api.openweathermap.org/data/2.5/weather?q=beijing&appid= あなたのキー &units=metric。
3. まずブラウザで 1 回試して、返ってくる JSON がどんな形か見てみましょう——どんな API をつなぐ前にもやるべき「予習」です。

**落とし穴注意：キーは絶対にコードに直書きしない**

API キーは、あなたの「おサイフの暗証番号」です。コードに書いて GitHub に上げた瞬間、数時間でクローラーに収集されて悪用されます。iOS での正しいやり方：Config.xcconfig（設定ファイル。gitignore で除外済み）に置き、Info.plist 経由でコードに読み込みます。「iOS xcconfig キー管理」で検索して、手順どおりに直せば OK です。

## ネットワーク層：URLSession ＋ Codable

まず API が返す JSON に合わせてモデルを定義してから、サービスを書きます：

```
// JSON: {"weather":[{"description":"晴れ"}],"main":{"temp":26.5,"humidity":40},"
    name":"北京"}
```

```
struct WeatherResponse: Codable {
    let weather: [Weather]
    let main: Main
    let name: String
}
struct Weather: Codable { let description: String }
struct Main: Codable { let temp: Double; let humidity: Int }

struct WeatherService {
    let key: String   // xcconfigから読み込む

    func fetchWeather(city: String) async throws -> WeatherResponse {
        var comps = URLComponents(string: "https://api.openweathermap.org/data/2.5/
    weather")!
        comps.queryItems = [
            URLQueryItem(name: "q", value: city),
            URLQueryItem(name: "appid", value: key),
            URLQueryItem(name: "units", value: "metric"),
            URLQueryItem(name: "lang", value: "zh_cn")
        ]
        let (data, _) = try await URLSession.shared.data(from: comps.url!)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}
```

## 3 つの画面状態：読み込み中 / 成功 / 失敗

ネット接続アプリの魂は「状態管理」です。ネットが遅い、つながらない、都市名を打ち間違えた……どの場合も、画面はユーザーにきちんと説明する必要があります。enum + switch でスマートに表現しましょう：

```
enum LoadState {
    case loading                  // くるくる回る
    case success(WeatherResponse) // 成功
    case error(String)            // 失敗
}

@Observable
class WeatherViewModel {
    var state: LoadState = .loading
    private let service = WeatherService(key: Config.apiKey)

    func search(city: String) async {
        state = .loading
        do {
```

```
            let data = try await service.fetchWeather(city: city)
            state = .success(data)
        } catch {
            state = .error("その都市が見つかりません。別の名前で試してみて？")
        }
    }
}

// 画面側：
switch vm.state {
case .loading: ProgressView("読み込み中…")
case .success(let d): WeatherContent(data: d)
case .error(let msg): Text(msg).foregroundColor(.secondary)
}
```

この loading/success/error の 3 段構えは、ネット接続画面の「万能テンプレート」です。まず丸暗記してしまえば、この先どんなネット接続アプリも同じ型で作れます。

## 実戦の画面コード：天気を描き出す

```
struct WeatherContent: View {
    let data: WeatherResponse

    var body: some View {
        VStack(spacing: 12) {
            Text(data.name)
                .font(.title.bold())
            Text("\(Int(data.main.temp))°C")
                .font(.system(size: 76, weight: .thin))
            Text(data.weather.first?.description ?? "不明")
                .font(.title3)
                .foregroundColor(.secondary)
            HStack(spacing: 24) {
                Label("湿度 \(data.main.humidity)%", systemImage: "humidity")
                Label("体感 \(Int(data.main.temp))°C", systemImage: "thermometer")
            }
        }
        .padding()
    }
}
```

学ぶ価値のある小さなディテールがいくつか：first? は最初の要素を安全に取り出し（リストが空かもしれないので）、?? “不明”がフォールバックの文言。細いウェイトの 76 ポイントの温度数字は、天気系アプリの定番レシピ。Label は「アイコン＋文字」の公式コンポーネントで、HStack を手で組み合わせるより規範的です。

## 操作のディテールとお気に入り都市

- 検索欄：上部に.searchable モディファイアを付けると、システム標準の検索操作になって、確定時に vm.search を呼び出せます。
- お気に入り都市：よく見る都市を UserDefaults に保存すれば、次に開いたとき自動で検索します。
- 下スワイプ更新：.refreshable { await vm.search(...) } の 1 行で完成。
- 天気アイコン：Image(systemName:) で説明に合わせて対応付け（sun.max / cloud.rain / snowflake）。画面が一気に生き生きします。

**ワンポイント：中国語の都市名の「クセ」**

OpenWeatherMap は中国語の都市名への対応がいまひとつで、「北京」と検索すると結果が出ないのに、“beijing”なら正常です。製品化するなら「都市名英語 / 座標」の対応表を内蔵するか、中国国内の天気サービスにつなぐのが定石。練習段階では、ピンイン / 英語の都市名に対応しておけば十分。ガチンコ勝負をしないことです。

**やってみよう：天気アプリの検収チェックリスト**

### ネットワーク通信のデバッグ：出力、パケットキャプチャ、低速ネットの再現

ネット接続アプリで問題が起きたら、十中八九「リクエストが正しく送れていない」か「レスポンスが正しく解析できていない」です。デバッグの手段は 3 つ：

1. 生データを出力する：String(data: data, encoding: .utf8) を印字して生の JSON を確認——フィールド名も型もひと目でわかり、当てずっぽうより 10 倍速いです。
2. パケットキャプチャツール：Mac の無料の Proxyman やオープンソースの Charles なら、App のすべてのリクエストとレスポンスを傍受して、X 線のようにネットワーク層をはっきり見せてくれます。
3. 低速回線の再現：シミュレータのメニュー Features Network Link Conditioner（開発者サイトからツールをダウンロードする必要あり）を使うと、3G や高遅延をエミュレートでき、「ぐるぐる回りっぱなし」といった体験の問題を洗い出せます。

**ワンポイント：リクエストが ATS に阻まれたら？**

リクエストを出したとたん「App Transport Security」エラーが出るなら、十中八九は http:// の平文アドレスが原因です。正式な API は必ず HTTPS に。ローカルデバッグ中だけ例外的に許可し（Info.plist に NSAllowsLocalNetworking を追加）、リリース前には忘れず解除しましょう。

(1) 普通に天気を検索できる。(2) ネットを切っても App はクラッシュせず、やさしくエラーを出す（シミュレータで Mac のネットワークを切って試す！）。(3) 存在しない都市

を入れると親切な表示が出る。(4) 都市を 2 つお気に入りに入れても、App を再起動しても残っている。(5) ダークモードでも画面がまぶしくない。この 5 つがすべて合格してはじめて、このプロジェクトは卒業です。

**この章のまとめ**

- コードを書く前にブラウザで API を試し、JSON を理解する——「予習」で損をすることはありません
- API キーは xcconfig にしまい、決してリポジトリに入れない
- Codable モデル + URLSession の async/await：フィールドが合っていれば JSON は自動でパース
- loading / success / error の 3 段階ステートは、ネットワーク系画面の万能薬
- 通信断・空入力・存在しない都市——エラー処理こそネットワーク App の合格ライン

# 第六十七章：仕上げとリリース——テスト、デバッグ、App Store 公開

*App* が作れた、そのあとは？この章は「動くもの」から「届けられるもの」へ進むための *3* つのテーマに答えます。バグを効率よく見つけるデバッグのやり方、デグレを防ぐテストの書き方、そして *App* を *App Store* に並べて世界中にダウンロードしてもらう方法。公開は単なるセレモニーではありません——作品を製品に変える分水嶺です。

## デバッグの基本三ツール：print、ブレークポイント、プレビュー

| ツール | できること | 使い方 |
| --- | --- | --- |
| print + コンソール | 実行ログを見る | コードに print(” 値 =\(x)”) と書き、Xcode 下部のコンソールで出力を確認 |
| ブレークポイントデバッグ | プログラムを特定の行で止めて変数の値を調べる | 行番号の横をクリックして青いマークを付けると、そこで実行が止まり、1 行ずつ進められます |
| SwiftUI プレビュー + デバッガ | 画面とビュー階層をリアルタイムで確認 | #Preview のキャンバス + 実行時の View Hierarchy インスペクタ |

デバッグの基本三ツール

**ワンポイント：クラッシュしたら何を読む？**

Swift がクラッシュすると、コンソールに「Fatal error」と原因が印字され、Xcode はエラーの行で止まってハイライトします。実機での過去のクラッシュは Xcode Window Organizer Crashes で確認できます（有料アカウントが必要）。検索キーワードは「エラー原因の最初の一文 + swiftui」を優先しましょう——正確な問いだけが、正確な答えを引き出します。

## テスト：コードに保険をかける

手動テストだけでは安心できません——この機能を直したら、別の機能がこっそり壊れていた、というのはよくある話。テストは「後悔しないための薬」であり「防弾チョッキ」です。iOS では XCTest フレームワークを使い、次の 2 種類に分かれます。

| 種類 | テスト対象 | Target | 速度 |
|---|---|---|---|
| ユニットテスト | 純粋なロジック（計算・検証） | MyFirstAppTests | 秒単位 |
| UI テスト | 実際の画面のふるまい | MyFirstAppUITests | 分単位 |
| 手動テスト | 人間の感覚・体験の確認 | —— | 常に必要 |

iOS のテストの種類（テンプレートプロジェクトには Test Target が 2 つ用意済み）

```
import XCTest

// 「金額文字列を数値へ変換」するロジックを純粋関数として切り出す
func parseAmount(_ input: String) -> Double? {
    guard let v = Double(input.trimmingCharacters(in: .whitespaces)),
          v > 0 else { return nil }
    return v
}

final class AmountTests: XCTestCase {
    func testEmptyInputReturnsNil() {
        XCTAssertNil(parseAmount(""))
    }
    func testNegativeReturnsNil() {
        XCTAssertNil(parseAmount("-5"))
    }
    func testNormalValueParses() {
        XCTAssertEqual(parseAmount("12.5"), 12.5)
    }
}
```

テストの実行は CmdU。全部緑になってからコードをいじれば、自信をもって改修できます。家計簿アプリの金額チェックや合計計算にも、ユニットテストの保護を付けましょう。

## 署名とパッケージング：App に「認め印」を押す

iOS の App はすべて署名しなければインストールできません——「このパッケージはあなたのもので、改ざんされていない」ことの証明です。階層関係は次のとおり。

- 学習段階：無料の Apple ID + Xcode の自動署名（Personal Team）なら実機でも動かせます。ただし 7 日ごとの更新が必要。
- 配布段階：有料の開発者アカウント（年間 99 ドル）が発行する証明書とプロビジョニングプロファイルがあって初めて、公開と長期インストールが可能になります。

パッケージング：Xcode のメニュー Product Archive でアーカイブを作り、完了したら Organizer で Distribute App App Store Connect を選んでアップロードします。アーカイブのターゲットは必ず Generic iOS Device に（シミュレータは不可）——ここは初心者がよく忘れる一手です。

**落とし穴注意：証明書を失くす ＝ 更新が途絶える**

署名証明書と秘密鍵を一度失うと、旧 App への更新を配布できなくなります——ユーザーは「真っ新な」アプリを入れ直すしかありません。証明書はキーチェーンから.p12 を書き出すなどしてしっかりバックアップし、パスワードも記録しておきましょう。幸い Apple は「自動署名」とクラウド証明書管理（Xcode の Automatically manage signing にチェック）を用意しており、手作業の落とし穴の大半を回避できます。

## App Store 公開の全ステップ

開発者アカウントを登録

ストア用素材を準備

ビルドパッケージをアップロード

プライバシーと審査情報を記入

Apple の審査（1　7 日）

ストアにリリース

図 5：App Store 公開の 6 ステップ

- 登録：developer.apple.com から。個人アカウントは年間 99 ドル。
- 素材：アイコン（1024×1024。角丸は付けない——Apple が自分で切り取ってくれます）、スクリーンショット（iPhone の各サイズ）、説明文、プライバシーポリシーのリンク。
- プライバシーの設問：収集するデータを正直に申告します——ネットに接続しない家計簿アプリなら「データを一切収集しない」で OK。
- 審査：初回の提出は 1　7 日の待ちもよくあります。却下されたら「審査コメント」に沿って 1 件ずつ修正を——Apple の却下理由は通常とても具体的に書かれているので、指示どおりに対応すれば大丈夫です。
- TestFlight：公開の前にテスト版を友だちに配って内部テストし（最大 10,000 人）、フィードバックを一巡集めてから正式提出——とてもプロらしい進め方です。

## ストア素材の作り方：第一印象がダウンロード数を左右する

ストアページは、ユーザーが「店をのぞく」ときに見るショーウィンドーです。第一印象を決めるのは 3 つの素材。

- アイコン：1024×1024。スクリーンショットで間に合わせないこと。デザインが苦手なら AI ツールでシンプルな図形を生成し、ホーム画面で縮小されてもひと目で判別できるものが合格ライン。
- スクリーンショット：最低 2 セット（iPhone の主流サイズ）。最も核となる機能の場面を見せ、1 枚ごとにひと言のキャプションを添えます（「1 秒で記録」は「効率的な家計管理ソリューション」より 10 倍伝わります）。
- 説明文：最初の一文で「これは何で、誰に使ってほしいのか」をはっきり。「最もプロフェッショナル」「最強」と並べ立てない——誠実さのほうが誇張より効きます。

**やってみよう：家計簿アプリを 1 回「リリース」してみる**

### クラッシュ報告とパフォーマンス改善のミニチェックリスト

App の公開前に、この 2 つにはそれぞれ 30 分を確保する価値があります。

**クラッシュ監視**

自分のテストでクラッシュしなくても、ユーザーの端末では起こるかもしれません。Apple は Xcode の Organizer で無料のクラッシュレポートを提供しています（実機のクラッシュログを見るには有料アカウントが必要）。もっと手軽にやるならサードパーティ SDK（Firebase Crashlytics など）を組み込む方法も。クラッシュ監視なしの公開は、目をつぶったまま車を運転するのと同じです。

**パフォーマンス自己点検の 4 項目**

1. リストには LazyVStack / List を使う：長いリストを普通の VStack で無理に組まない——遅延読み込みはパフォーマンスの第一課。
2. 画像に原寸を詰め込まない：AsyncImage（リモート画像）とシステムのキャッシュを使う。5MB の元画像ひとつで旧機種は軽く悲鳴を上げます。
3. メインスレッドで重い処理をしない：通信も大きなファイルも async/await に任せる。
4. Instruments で測る：Xcode 内蔵のプロファイラ（Time Profiler、メモリリーク検査）——測ってから最適化。勘で語らないこと。

(1) parseAmount などのロジックにユニットテストを 3 つ追加する（CmdU で緑に）。(2) ブレークポイントを使い、自分で仕込んだクラッシュを捕まえて直す。(3) Product Archive でパッケージを作り、TestFlight で友だちを内部テストに招待する。(4) 余力があれば開発者アカウントを登録して、家計簿アプリを本当に提出・公開する。最初の 3 つは今夜中に終えられます。

この章のまとめ

- デバッグの基本三ツール：print ログ、ブレークポイント、SwiftUI プレビューとビューインスペクタ
- XCTest：ユニットテスト（CmdU）＋ UI テスト ＋ 手動テスト。テンプレートに最初から用意済み
- 署名：無料アカウントは自動署名（7 日ごとに更新）。配布には有料アカウントが必要
- パッケージング：Generic iOS Device ＋ Archive ＋ Distribute。証明書は必ずバックアップ
- 公開の 6 ステップ：登録素材アップロードプライバシー設問審査公開。まずは TestFlight で内部テスト

# 第六十八章：ステップアップへの道——権限、通知、モダンなアーキテクチャ

*App* が作れるようになるのは、まだ始まりにすぎません。この章では「作れる」から「うまく作る」までのいくつかのハードル——権限をスマートに要求する方法、嫌われない通知の出し方、アーキテクチャをもう一段引き上げる方法、そして *2026* 年において *AI* と組んで *iOS* コードを書く方法——を取り上げます。どれも、実際のプロジェクトですぐに出会う話ばかりです。

## 権限：まずノック、それから入る

カメラ、位置情報、連絡先、通知……こうした「センシティブな領域」では、iOS は App が使う場面でユーザーにダイアログを出して許可を求めることを求めています。お作法は「宣言 + 要求 + 代替動作の用意」の 3 ステップです。

```
// 1. Info.plistに用途の文を宣言（宣言しないと即クラッシュ！）
// Privacy - Location When In Use Usage Description: "現在地の都市の天気を調べるために使用します"

// 2. 使うときに申請
import CoreLocation

@Observable
class LocationModel {
    private let manager = CLLocationManager()

    func request() {
        switch manager.authorizationStatus {
        case .notDetermined: manager.requestWhenInUseAuthorization()
        case .denied, .restricted: /* 設定画面へ誘導 */
        default: /* 許可済み、始めよう */
        }
    }
}

// 3. 拒否されたときの優雅な代替：案内を表示 + 設定へ誘導
```

```
// UIApplication.shared.open(URL(string: UIApplication.openSettingsURLString)!)
```

**注意：iOS の権限は「一度きり」**

iOS の権限ダイアログは 1 度しか表示されません——ユーザーが拒否したら二度と出ません。だからこそ、要求のタイミングは「ユーザーがまさに使おうとしている瞬間」を選び（起動直後にいきなり何個も出さない）、文言では用途を説明しましょう。拒否されたあとの画面での案内（「設定で位置情報をオンにしよう」）は必須の配慮です。ユーザーを行き止まりに追い込まないこと。

## 通知：正しいタイミングで役立つ言葉を届ける

通知は App がユーザーに届く大切な通路であると同時に、もっとも「ブロック」されやすい機能でもあります。iOS では UserNotifications フレームワークを使います。通知の 3 カ条は、役に立つものだけ送る、ユーザーがオフにできるようにする、深夜に騒がない。基本の使い方は次のとおりです。

```
import UserNotifications

// 1. 通知の権限を申請（ポップアップは1回）
UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge]) { granted, _ in
        print(granted ? "許可された" : "拒否された")
    }

// 2. 「毎日20:00に記帳リマインダー」を予約
func scheduleDailyReminder() {
    let content = UNMutableNotificationContent()
    content.title = "記帳リマインダー"
    content.body = "今日はまだ記帳していないよ"
    content.sound = .default

    var comps = DateComponents()
    comps.hour = 20
    let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)

    let request = UNNotificationRequest(
        identifier: "daily-reminder",
        content: content,
        trigger: trigger
    )
    UNUserNotificationCenter.current().add(request)
}
```

注意：iOS の通知権限も同じく「一度きり」で、ユーザーはシステム設定でいつでもオ

フにできます——だからこそ、通知の中身の「価値」がとても重要です。一度でも乱発すれば、ユーザーを永遠に失いかねません。

## アーキテクチャのステップアップ：MVVM の深掘りとテスト容易性

第六十五章で MVVM を使いました。ここで、レベルアップのためのポイントを 3 つ補足します。

- プロトコル抽象化：ViewModel を具体的なクラスではなくプロトコルに依存させます——テストのときに「偽のデータソース」を注入でき、テスト容易性の土台になります。

- 単一責任：1 つの ViewModel は 1 つのことだけを担当する（家計簿担当は天気を扱わない）。機能が増えたら分割する。

- 設計しすぎない：一人の小さなプロジェクトなら「@Observable クラス + プロトコル抽象化」で十分です。VIPER や Clean Architecture のような重いアーキテクチャは、チームや複雑さが訪れてから考えましょう——ツールは目的に仕えるものであって、見せびらかすためのものではありません。

**ワンポイント：SwiftUI の「エコシステムの恩恵」**

iOS エコシステムには、ここでしか使えないレベルアップの技がもう 2 つあります。ウィジェット（WidgetKit）は App を開かなくても情報を確認できるようにします（家計簿アプリの「今月の支出」をホーム画面に置くイメージ）。ショートカット（App Intents）は、ユーザーが Siri やオートメーションからあなたの機能を呼び出せるようにします。どちらも公式フレームワークが用意され、導入コストは高くないのに、ユーザー体験への追い風は絶大です——ステップアップの道では優先的に解禁したい能力です。

## 2026 年の新常識：AI 支援の iOS 開発

2026 年、AI はすでに Apple の開発フローに深く組み込まれています。作業を速めたいなら、次の組み合わせがすっかり定着しています。

| 場面 | AI の助け方 | あなたの役割 |
|---|---|---|
| コードを書く | Xcode 内蔵のコード補完 + サードパーティ AI プラグインによる生成 | 要件を明確に伝え、生成物をレビューする |
| 調べもの | 公式ドキュメントやエラーメッセージを AI に渡して解説してもらう | 重要な結論は公式ドキュメントで裏取りする |
| テストを書く | 「正常 / 境界 / 異常」の 3 分類で XCTest ケースを AI に生成させる | ケースが業務の意味に合っているか確認する |
| バグを直す | クラッシュログと関係するコードを貼り、AI に原因特定と修正をしてもらう | 変更を理解し、テストを回して検証する |

AI 支援 iOS 開発の 4 つの場面

具体的な方法論——AI への指示の出し方、成果物の受け取り方、落とし穴の避け方——は、本書の第 7 部『AI プログラミング実践編』（第 37　47 章）に全 11 章ぶん収録しています。あわせて読むことを強くおすすめします。Swift の文法はこの部で、AI との協働のコツは第 7 部で、という分担です。

やってみよう：家計簿アプリに「3 点セット」を追加する

## Swift の並行処理の要点：@MainActor と Task

Swift 6 の並行処理の仕組みには、初心者が必ず知っておくべきルールが 2 つあります。

- UI はメインスレッドでしか更新できない：SwiftUI のビューはデフォルトで @MainActor なので、ネットワークのコールバックはメインスレッドに戻ってから状態を更新します（await MainActor.run { ... } を使うか、ViewModel を @MainActor としてマーク）。
- 非同期タスクは Task で管理する：.task { } モディファイアはビューが現れると自動で始まり、消えると自動でキャンセルされます——SwiftUI で非同期処理を始める標準の作法です。body の中に Task をむき出しで書き散らかすのは避けましょう。

```
struct WeatherScreen: View {
    @State private var vm = WeatherViewModel()

    var body: some View {
        Group {
            switch vm.state {
            case .loading: ProgressView()
            case .success(let d): WeatherContent(data: d)
            case .error(let m): Text(m)
```

```
            }
        }
        .task {                          // 現れたら開始、消えたら取消
            await vm.search(city: "北京")
        }
    }
}
```

この 2 つのルールの背景には、Swift 6 の「データ競合安全」設計があります——間違えたとき、App がクラッシュするのを待たずにコンパイラがその場でエラーを出してくれます。コンパイラのエラーは嫌がらせではなく、あなたに代わって弾丸を受けてくれているのです。

(1) 家計簿アプリに「毎日 20:00 の記帳リマインダー」を追加する（通知権限 + ローカル通知）。(2) ViewModel のデータソースをプロトコル抽象化に替え、偽データを注入するユニットテストを 1 つ書く。(3)「今月の支出」のホーム画面ウィジェットを追加する。3 点セットが完成すれば、あなたのプロジェクトは「正規軍」の風格を帯びてきます。

### この章のまとめ

- 権限の 3 ステップ：Info.plist での宣言（抜けると必ずクラッシュ）、使うときに要求、拒否されたら設定への案内
- iOS の権限ダイアログは一度きり：タイミングは「まさに使おうとする瞬間」を選び、文言で用途をはっきり書く
- 通知の 3 カ条：役に立つものだけ、オフにできる、迷惑をかけない。フレームワークは UserNotifications
- MVVM のステップアップ：プロトコル抽象化（テスト可能に）、単一責任、設計しすぎない
- エコシステムの恩恵：WidgetKit ウィジェット + App Intents ショートカット
- AI 支援の 4 つの場面：コーディング、調べもの、テスト作成、バグ修正。方法論は第 7 部（第 37　47 章）参照

# 第六十九章：クイックリファレンスと学習リソース

この章は、それ単体で使えるクイックリファレンスです。書き方を思い出せないときや、すばやく判断したいときは、直接ここを開きましょう。「ポケットカード」として活用するのがおすすめです。よく使うコードの一覧、よくある質問、学習リソース――この *3* つをまるごと収めています。

## よく使うコードのクイックカード

| やりたいこと | キーコード |
|---|---|
| 画面内のプライベートな状態 | @State private var x = 初期値 |
| 画面をまたぐロジック | `@Observable class VM { var x = 0 }` |
| 縦並び / 横並びのレイアウト | `VStack { }/HStack { }` |
| 伸縮する余白 | `Spacer()` |
| リスト | List { ForEach(配列) { ... } } |
| 画面遷移 | `NavigationStack + NavigationLink` |
| ボトムシート | .sheet(isPresented: $show) { 中身} |
| 現在の画面を閉じる | `@Environment(\.dismiss) var dismiss` |
| ネットワークからデータを取得 | `try await URLSession.shared.data(from: url)` |
| JSON のパース | `struct X: Codable+JSONDecoder().decode` |
| 小さな設定を保存 | UserDefaults.standard.set(値, forKey: ”キー”) |
| 構造化データを保存 | `@Model+modelContainer+@Query` |
| ダークモード対応 | システムカラー（.primary / Color(.systemGray6)）を使い、色を決め打ちしない |
| テストを実行 | CmdU |

# よくある質問 FAQ

Q1：Mac を持っていません。iOS 開発は学べますか？

言語の学習はできますが、本格的な開発はできません。iPad / iPhone の Swift Playgrounds なら第六十一章の内容までは学べます。ただし Xcode は macOS 専用なので、最終的には Mac が 1 台必要です。予算が限られているなら、クラウド Mac をレンタルしたり中古の Mac mini を買ってしのいだりして、言語を学び終えてから本格投資を決めるのもよいでしょう。

Q2：iOS 開発の学習には、いくらお金がかかりますか？

学習段階：0 円（Xcode は無料 + 無料の Apple ID で実機デバッグ可能）。リリース段階：開発者アカウントが年間 99 ドル。これに Mac 1 台のハードウェアへの投資を加えたものが総コストです。見返りと比べれば——App Store では小さなツールでも有料ユーザーがつくもの——十分に割に合う勘定です。

Q3：SwiftUI と UIKit、どちらを先に学ぶべき？

SwiftUI です。2026 年の新規プロジェクト・新しいチュートリアル・新しい求人はいずれも SwiftUI が中心で、本書も通して SwiftUI を使います。UIKit は前世代のフレームワークで、多くの古いプロジェクトでは今も使われています——SwiftUI に慣れてから必要に応じて UIKit を補えば（公式に相互運用のチュートリアルがあります）、習得は速いはずです。順序だけは逆にしないこと。

Q4：iOS 開発は仕事探しに強いですか？

2026 年の市況：単に「画面が描けるだけ」の初級ポジションは競争が激しいものの、「Swift + SwiftUI を理解し、リリース実績がある」開発者は今も引く手あまたです。iOS 関連職の給与中央値は、各技術スタックの中で長年上位にあります。学習中のあなたにとって最高の履歴書は、ストアに並んだ自分の App です。

Q5：App が審査で却下されたら、どうすればいい？

慌てなくて大丈夫、ごく普通のことです——初回提出が却下される確率は決して低くありません。Apple の却下理由は通常とても具体的に書かれている（どの画面か、どのガイドラインか）ので、1 件ずつ修正し、App Store Connect で回答を添えて再提出すれば OK。よくある却下理由のビッグ 3 は、権限の用途説明が不明瞭、画面にバグがある、メタデータ（スクリーンショット / 説明文）と実際の中身が合っていない、の 3 つ。事前にこの 3 点をセルフチェックすれば、1 回ぶんの待ち時間を節約できます。

# Xcode のよく使うショートカット一覧

| 操作 | ショートカット |
|---|---|
| 実行 / テスト | CmdR / CmdU |
| キャンバスプレビューを開く | OptionCmd |
| ファイルをすばやく開く | CmdO |
| プロジェクト全体を検索 | CmdF |
| インデントを整形 | CtrlI |
| コメント / コメント解除 | Cmd/ |
| ビルドキャッシュをクリーン | CmdK |
| 定義へジャンプ | Cmd クリック |
| リネーム（全体を安全に改名） | 右クリック Refactor Rename |

Xcode のよく使うショートカット一覧

**ワンポイント：コンパイルの怪奇現象は、まずキャッシュクリア**

「昨日は動いたのに今日は動かない」という怪奇現象は、六七割がビルドキャッシュの不調です。CmdK でクリーンしてから再ビルドすれば、怪奇現象の大半は解決します。それでもだめなら Xcode を再起動（Mac ごとでも）、さらにだめなら「DerivedData フォルダを削除」——iOS 開発者の間で口伝えに受け継がれてきた究極の三連コンボです。

## おすすめ学習リソース

## おすすめ学習リソース

| 種類 | リソース | おすすめの理由 |
|---|---|---|
| 公式 | developer.apple.com | Apple の公式ドキュメント。SwiftUI / Swift のチュートリアルもここに。第一の権威です |
| 公式 | Apple の SwiftUI Tutorials | 公式のきめ細かいハンズオン。初心者へのやさしさは満点 |
| 公式 | Swift 公式サイト（swift.org） | 言語の公式ドキュメントとオンラインの実行環境 |
| コミュニティ | Hacking with Swift | Paul Hudson の無料チュートリアル集。品質は誰もが認めるトップクラス |
| コミュニティ | Stack Overflow（swiftui タグ） | エラーで困ったときの最初の駆け込み先 |
| 動画 | Apple Developer 公式チャンネル（WWDC 動画） | 毎年の新技術を権威者が解説 |
| デザイン | Apple ヒューマンインターフェースガイドライン（HIG） | 「Apple らしい」画面を作るための公式のバイブル |
| 本書の関連 | 第 7 部『AI プログラミング実践編』（第 37 47 章） | AI で iOS 開発を支援するための完全な心得 |

iOS 開発の学習リソース一覧（2026 年 8 月）

## おわりに：最初の App から N 個目の App へ

振り返ってみれば、「SwiftUI が何かも知らなかった」あなたが、Xcode をインストールし、Swift を学び、SwiftUI で画面を描き、App にデータベースとネットワークを接続し、さらにテスト・パッケージング・リリースまで進みました——この道のりで、あなたはもうゼロからイチへ至るいちばん難しい区間を歩き終えています。

技術の世界は変化が速いものです。フレームワークは更新され、ツールは世代交代し、AI は「キーボードを叩く」作業をどんどん引き受けていくでしょう。でも、色あせないものがひとつあります。ぼんやりしたアイデアを明確なステップに分解し、一歩ずつ形にしていくあなたの力です。ツールは増幅器、考え方は土台。増幅器は取り替えられますが、土台は揺るぎません。

あなたはもう「スマホが使える人」だけではありません。頭の中のアイデアを、数億台の iPhone で実際に動くアプリに変えられるのです。今日からキーボードで創造しよう——これが iOS 開発者のロマンです。

## 参考文献

1. Apple Inc. (2026). SwiftUI Framework Documentation. developer.apple.com.
2. Apple Inc. (2026). Swift Programming Language Guide. swift.org.
3. Apple Inc. (2026). SwiftData Framework Documentation. developer.apple.com.
4. Apple Inc. (2026). App Store Review Guidelines. developer.apple.com/app-store/review.
5. Apple Inc. (2026). Human Interface Guidelines (HIG). developer.apple.com/design.
6. Apple Inc. (2026). Xcode 27 Release Notes and App Distribution Guide.

# 十、付録

# 付録 A：新端末の開封「超親切」ガイド

## 開封・検品「3 ステップ」:

- **付録 A：新端末の開封「超親切」ガイド** 1. 開封・検品「3 ステップ」:
- **外観確認：**本体や画面に傷やへこみがないか確認。
- **情報照合：**箱の IMEI/シリアルナンバーと領収書のものが一致するか確認。

## アクティベーションと初期設定の「重要リマインダー」:

- **：電話をかけてイヤホンとマイクを試し、写真を撮ってカメラを確認。**2. アクティベーションと初期設定の「重要リマインダー」:
- **ネットワーク：**まず安定した Wi-Fi に接続。
- **アカウント：**Apple ID、Huawei アカウント、Xiaomi アカウントなどにログイン。
- **基本設定:**「デバイスを探す」をオンにし、しっかりしたロック画面のパスワードを設定。
- **eSIM 設定：**スマートフォンが eSIM に対応している場合（iPhone や一部の Android フラッグシップなど）、設定で eSIM プランを追加でき、物理 SIM カードの挿入は不要です。海外渡航時は eSIM が特に便利で、現地キャリアの eSIM プランを直接購入できます。

## アプリのインストールは「セキュリティ第一」:

- **：古いスマートフォンからデータを移行する際、両方のスマートフォンのバッテリーが十分にあることを確認。iPhone は「クイックスタート」、Android は公式の機種変更アプリを使用。**3. アプリのインストールは「セキュリティ第一」:
- **公式ストアを優先：**Apple は App Store、Android はスマートフォン付属のアプリストアを使用。

# 付録 B：実践練習の参考

- **：インストール時の権限リクエストに対し、「このアプリは本当に必要か？」を考える。**付録 B：実践練習の参考
- **基礎編の練習：**スマートフォンのストレージをクリーンアップ。目標は「何だったか忘れた」古い動画、重複スクショ、長らく使っていないアプリを見つけて削除すること。
- **ソフトウェア編の練習：**WeChat のストレージクリーンアップを設定。パスは通常【自分】-【設定】-【一般】-【ストレージ】。
- **応用編の練習：**バッテリーの健康状態を確認。iPhone は【設定】-【バッテリー】-【バッテリーの状態】にあります。

# 付録 C：ミニマル用語「隠語」対照表

| ：問題に遭遇したら、まず再起動！これは半分以上のスマートフォンの小さな不具合を解決する万能キーです。 | 付録 C：ミニマル用語「隠語」対照表 | 聞いたことがあるかもしれない言葉 |
|---|---|---|
| 実はこれのこと…… | 一言で説明 | SoC / チップ |
| スマートフォンの脳と心臓 | スマートフォンの速さやゲームのカクつきを決めるコアパーツ。 | RAM（メモリ） |
| スマートフォンの「短期作業デスク」 | デスクが大きいほど、同時に開けるアプリが多く、切り替えもスムーズ。 | ROM（ストレージ） |
| スマートフォンの「大倉庫」 | 写真、動画、アプリを保存する場所。128G、256G と言っているのはこれのこと。 | OLED / LCD |
| 画面の 2 種類の素材 | OLED は色鮮やかで省電力、LCD は目に優しくコストが低い。 | NFC |
| 近距離無線通信 | スマートフォンを交通系 IC カード、入退室カード、銀行カードに変える「タッチ」機能。 | OTA アップデート |
| ワイヤレスシステムアップデート | 【設定】でタップするあの「システムアップデート」がこれです。 | Root |
| Android スーパー管理者権限 | 取得するとシステムファイルを変更できますが、リスクもあります。 | BL ロック |
| Bootloader ロック | メーカーがかけるセキュリティロック、解除して初めて ROM 焼きが可能。 | NPU |
| ニューラルネットワークプロセッサ | AI タスクに特化したチップ、CPU より数十倍速く、消費電力も低い。 | オンデバイス AI |
| スマートフォン上でローカル実行される AI | AI がスマートフォン上で直接実行され、ネット接続不要、高速でプライバシーに優れる。 | Qi2 |

# 付録 D：信頼できるリソースはどこで探す？

## 公式サポートが第一選択：

- **付録 D：信頼できるリソースはどこで探す？** 公式サポートが第一選択：
- **Apple ユーザー：** apple.com/cn と iCloud.com。
- **Android ユーザー：** スマートフォンブランドの公式サイトに「サービスとサポート」のセクションがあります。

## 実用サイトとブロガー：

- **：Apple は iTunes、Huawei は Huawei スマートフォンアシスタント、Xiaomi は Xiaomi 機種変更アプリ。**実用サイトとブロガー：
- **テクニックを探す：** Bilibili で「【あなたのスマートフォン型番】テクニック」を検索。
- **レビューを見る：** 複数の異なるブロガーのレビュー動画を見る。

## セキュリティ自己チェック一言リスト：

1. **：** スマートフォン撮影チュートリアルのブロガーをフォロー。
2. セキュリティ自己チェック一言リスト：
3. ロック画面のパスワードは設定しましたか？
4. 「デバイスを探す」はオンにしましたか？
5. 重要アカウントの二要素認証はオンにしましたか？

# 付録 E：感謝とご説明

この「スマホの講座」を最後まで読み通してくれて、ありがとうございます！執筆にあたっては、情報の正確さを保つため、各メーカーの公式マニュアル、ユーザーズガイド、公開されている技術解説記事を数多く参考にしました。

技術は日進月歩で進み、本書の内容は執筆時点の主流の技術と製品に基づいています。あなたがこの本を読む頃には、新しい機能や製品がまた登場しているかもしれません。それでも、この本で伝えた考え方と方法が、これからもっと増えていくデジタルの新しい道具たちと落ち着いて向き合う助けになれば幸いです。

## バージョン 1.6 の更新ポイント

- 「デジタル資産の価値と管理の考え方」を拡充し、「写真とファイルシステムの管理」の章を新設しました（日時＋出来事での命名、分類整理、72 時間自動クリーンアップ、EXIF のプライバシー情報の除去）。
- 「クリーン / 保護モード」と「電子寄生虫」への防護の仕組みを充実させました。
- AI 時代のよくある詐欺の手口（顔すり替え・音声まね / 権威になりすまし / 虚偽の投資案件）と、その突破・防護の考え方を追加しました。
- 「ドクシング（開盒）」とソーシャルエンジニアリング攻撃からの防護を書き直しました。
- アクセシビリティの章を拡充しました（VoiceOver / TalkBack の完全な操作方法、拡大鏡とカラーフィルタ、AI による画像の説明、ライブ字幕、RTT / TTY 通話のデュアルチャンネル）。
- 「役割-背景-タスク-制約」の考え方でプロンプトのフレームワークを書き直し、完全なシナリオ例を 3 つ添えました。
- 技術ステップアップの付録を 3 編追加：第一章付録：Android スマホのメモリ拡張の解説（本質的な定義、zRAM / swap の仕組み、速度のボトルネック、適用場面の境界、各メーカーの名称と有効化方法）。
- 第二十章付録：Termux——Android 端末上のポータブル Linux ターミナル環境（中核機能、プラグインのエコシステム、インストールと設定、モバイル開発 / サーバー運用 / セキュリティテストなどの典型的な活用場面）。

- 第二十三章付録：GitHub 入門ガイド——コード管理からコラボレーション基盤まで（Git と GitHub の違い、核心的な概念、登録と始め方、Pull Request の共同作業フロー、セキュリティ上の注意、国内からのアクセス方法）。

## バージョン 1.7 の更新ポイント

1. 新しい章を 4 章追加

- 第二十九章：デジタルウェルビーイング——スクリーンタイムの分析、App の使用制限、おやすみモード、20-20-20 の目の休憩ルール、首のいたわり習慣、睡眠管理、情報過多への「選択的に目をつぶる」対策、1 週間デジタルヘルスチャレンジ。
- 第三十章：画面ミラーリングと車載連携——AirPlay / Miracast / DLNA の 3 陣営の比較、有線接続の方法、Samsung DeX のデスクトップモード、CarPlay / Android Auto / HiCar / CarLife の 4 大カーコネクティング、会議室での投影の実践。
- 第三十一章：ショート動画づくり——テーマ決めの 3 つの問い、撮影の 5 ステップ設定、カメラワークの 5 技（プッシュ・プル・パン・スライド・フォロー）、剪映（CapCut）の 6 ステップワークフロー、サムネイル作りの要点、「最初の 3 秒で決まる」コンテンツ心得。
- 第三十二章：安全応急ハンドブック——紛失後のゴールデン 30 分の SOP、詐欺被害から 72 時間の対応フロー、マルウェア感染時のセーフモード点検、アカウント乗っ取りからの奪還 6 ステップ、脅迫メッセージへの対応、日常の予防 5 カ条。

2. 新しい章を 2 章追加

- 第二十七章：シニア対応と家族のつながり——簡単モードのメーカー比較表、詐欺の 6 大パターンと 3 つの防御ライン、リモートサポート（MeeTime / 向日葵リモート）、WeChat のケアモードと音声入力、SOS / 転倒検知 / 服薬リマインダー。
- 第二十八章：購入ガイド——3 ステップの選び方、「この 5 項目だけ見る」スペック表、5 段階の予算別おすすめ、7 大シーン別の選択表、購入時の 5 大落とし穴、買うタイミングと購入先の分析。

3. 読み上げと操作の新機能

- フローティング読み上げコントロールバー：再生 / 一時停止、前の段落 / 次の段落、0.5 2.5 倍速の調整、停止に対応。
- 右側のクイックイヤホンボタン：ワンタップで読み上げを開始し、現在の読書位置から自動的に始めます。
- キーボードショートカット：R で読み上げの切り替え、ESC で停止。

- 読み上げ体験の改善：章をまたいで自動的に読み続け、印刷時には読み上げコントロールを自動的に非表示。

4. 全体構成の更新

目次、学習パス（4 ステップ 5 ステップ）、章マップ、バージョン番号（1.61.7）、付録番号（五六）、メタデータをすべて同期して更新しました。

5. 内容の最新化

- プロセッサの基準：Snapdragon 8 Elite Gen5 / Dimensity 9500 を最上位クラスに分類（2026 年基準）。
- フラッグシップ機の参考：iPhone 17 Pro、Samsung S26 Ultra、Xiaomi 17 Ultra、Huawei Pura 80 Ultra、vivo X300 Pro。
- AI の章：DeepSeek R1、ChatGPT-5、エッジ側 NPU 80 TOPS の性能基準に更新。
- 折りたたみスマホ：価格帯が 4,000 5,000 元クラスまで下がってきました。
- 購入ガイド：予算別の全分類とシーン別おすすめを 2026 年の市場に合わせて刷新。
- ネットワークとエコシステム：Wi-Fi 7 / 5G-A / HarmonyOS エコシステム / LE Audio をすべて最新規格に更新。

6. 新機能モジュール：学習ダッシュボード（Hero エリアの下に配置）

- 読書進捗のリングチャート：読み終えた章の割合をリアルタイム表示、SVG の円環アニメーション付き。
- スマホ健康セルフチェック：8 項目のセキュリティチェックリスト（パスワード、二要素認証、端末の検索、クラウドバックアップ、WeChat バックアップ、決済の安全、反詐欺 App、パスワード不要のクリーンアップ）。
- 状態の永続化：チェック状態は localStorage に保存され、健康スコアは動的に計算してグラデーションの進捗バーで表示します。
- 段階別のメッセージ：スコアに応じて「満点 / まずまず / ぎりぎり合格 / 安全上の心配あり」などのフィードバックを表示。

## バージョン 1.8 の更新ポイント

第 7 部『AI プログラミング実践編』（第 37 47 章、全 11 章）を新設：パラダイム革命と Agentic Engineering、AI プログラミングの 5 大思考原則、コンテキストエンジニアリングと仕様駆動開発（SDD）、要件エンジニアリング、2026 年の AI プログラミングツールの全景、大規模モデルの全景と API 実践、会話エンジニアリング、ゼロからイチまでの完全プロジェクトの全工程、品質保証、ステップアップへの道（MCP / マルチ Agent 協働）とク

イックリファレンス。目次、学習パス（5 ステップ 6 ステップ）、章マップ、クイック索引もあわせて更新。内容は『AI プログラミング思考：ゼロからフルスタック実践までの方法論ガイド』2026 年 8 月版に基づきます。

第 8 部『Android 開発実践編』（第 48 – 58 章、全 11 章）と第 9 部『iOS 開発実践編』（第 59 – 69 章、全 11 章）を新設：両プラットフォームとも、環境構築（Android Studio / Xcode）、言語速習（Kotlin / Swift）、UI フレームワーク（Jetpack Compose / SwiftUI）、データとネットワーク、2 つの完全な実践プロジェクト（家計簿アプリ＋天気アプリ）、テストとリリース（Google Play / App Store）、そしてクイックリファレンスを網羅します。目次、学習パス（6 ステップ 8 ステップ）、章マップ、クイック索引もあわせて更新。内容は『ゼロから始める Android 開発』『ゼロから始める iOS 開発』2026 年 8 月版に基づきます。

## コンテンツの追加：実用アプリと迷惑電話・SMS のブロック

- 第十三章に第 6 節「実用システムツール」を追加：クリーンアップアプリの「Qingzhuo」など、システム内蔵のストレージ整理、国家反詐欺センター、各メーカーのスマホ管理アプリ、キャリアの迷惑ブロックと AdGuard を整理し、「システム優先 / 公式で補完 / サードパーティは控えめに」という選び方の原則を示します。
- スマホ内蔵の迷惑ブロック機能のメーカー別対応表：iOS、Xiaomi / Redmi、Huawei / HONOR、OPPO / realme / OnePlus、vivo / iQOO、Samsung の操作手順とおすすめの設定項目を網羅。
- 第八章の迷惑ブロック機能を拡充：発信者の識別、ブラックリスト / ホワイトリスト、SMS キーワードフィルタ、iOS の「知らない差出人を消音」、おやすみモードとの連携などの実践的な説明を補足し、第十三章と相互リンクさせました。
- 3 層防御のすすめ：システムのブロック ＋ キャリアの迷惑対策 ＋ 国家反詐欺センター。ホワイトリストで誤ブロックを防ぐことも強調しています。

## コンテンツの追加：気象警報 / 自然災害警報 / 地震警報

- 第十三章に第 7 節を追加：気象警報（公式気象チャンネル ＋ システムの天気 App ＋ 色による警報レベル）、自然災害警報（気象 / 防災の複数チャンネル）、地震警報の仕組みと誤解（警報 $\neq$ 予報）。
- 地震警報のメーカー別設定表：Xiaomi、Huawei / HONOR、OPPO 系、vivo、iPhone、Samsung などの一般的な設定場所と、権限の注意点。
- 第三十二章に「シーン 0」を追加：地震警報後の避難の優先順位、大雨 / 台風への対応、衛星 SOS / ネットなし通信との連携。応急ハンドブックは第十三章の警報設定と相互リンク。
- 天気のセクションとの連動：旅行・お出かけの天気 App 項目に、警報通知と「通知をオンにしないと受け取れない」注意を追記。

## コンテンツの追加：複数デバイスの連携と効率化ツール

- 第十三章の第 8 節：スマホから PC を遠隔操作（向日葵（Sunlogin）、ToDesk）と、遠隔操作を悪用した詐欺に注意するタブー。AI オフィス（Trae / WorkBuddy / Dumate）とスマホで PC を操るワークフロー。miHoYo「人工デスクトップ」のライブ壁紙。
- LAN 内の高速ファイル送信とクリップボード：LocalSend、飛鴿伝書（FeiGe）、AirDrop。KDE Connect、ユニバーサルクリップボード、各メーカーのスマホ・PC 間送信。
- ワンタイムコード 2FA：Aegis、Microsoft Authenticator。バックアップコードと機種変更時の注意。
- TickTick（滴答清单）と紙のスキャン：スケジュール / タスクの使い方を展開。スキャンツールと撮影テクニックを補完。

スマホを自由に使いこなし、テクノロジーのもたらす便利さと楽しさを味わうと同時に、冷静で自信に満ちたデジタル市民になってくれたらうれしいです。

——あなたのデジタル生活のガイドより

# 付録 F：折りたみ画面の選び方と使い方特集

——あなたのデジタルライフガイド

## 一、折りたたみ画面は今どこまで発展しているか？

折りたたみ画面は「未来のテクノロジー」から「今買える」成熟カテゴリーになりました。しかし、ストレート型スマートフォンとはフォームファクター、使用習慣、メンテナンス方法に大きな違いがあり、買う価値があるか、どう選ぶか、どう使うか、この記事で整理しましょう。**一、折りたたみ画面は今どこまで発展しているか？** 先に結論を言うと：

折りたたみ画面はもはや「お試し玩具」ではありませんが、「誰もが買うべき」主流のフォームファクターでもありません

- **。ニッチなギーク向け製品と大衆消費財の間に位置し、特定のユーザー層に適しています。**現在主流は 2 つのカテゴリーに分かれます：
- **横折り（ブック型）：**閉じると普通のスマートフォンサイズ、開くとほぼ正方形の小型タブレット。代表：Huawei Mate X シリーズ、Samsung Galaxy Z Fold シリーズ、Xiaomi MIX Fold シリーズ、vivo X Fold シリーズ。「1 台のスマートフォン＝スマートフォン＋タブレット」が売り。

2024-2026 年の主要な進歩：ヒンジ寿命が普遍的に 50 万回の折りたたみを保証（1 日 100 回開閉で 13 年使用可）、UTG 超薄ガラスが初期のプラスチックフィルムに取って代わり、折り目は目視で確認できるが手触りはすでに非常に軽微、価格が 1 万元帯から 4000-5000 元帯に下下探。

**：閉じるとパウダーケースのよう、開くと通常サイズのスマートフォン。代表：Huawei Pocket シリーズ、Samsung Galaxy Z Flip シリーズ、OPPO Find N Flip、Xiaomi MIX Flip。「小型携帯＋外画面クイックチェック」が売り。**

2024-2025 年の重要な進歩：ヒンジ寿命が一般的に 50 万回の折りたたみに対応（1 日 100 回開閉で 13 年使用可能）、UTG 超薄ガラスが初期のプラスチックフィルムに代わり、折り目は肉眼で見えるが触感は非常に軽微、価格は 1 万元帯から 5000-6000 元帯に下落。

## 二、どんな人が折りたたみスマートフォンに向いているか？

現在ストレート型スマートフォンで完全に十分で、「画面が足りない」というペインポイントがないなら、折りたたみスマートフォンは買わないでください。それは「錦上花を添える」デバイスであり、「雪中に炭を送る」デバイスではありません。

1. **二、どんな人が折りたたみスマートフォンに向いているか？**実際のユーザーフィードバックに基づくと、以下のタイプの人が折りたたみスマートフォンを買って「マジで良かった」と感じる確率が高いです：
2. **ヘビーリーダー/ドキュメント処理者：**スマートフォンでよく PDF を見たり、ドキュメントに注釈を付けたり、長文を読んだりする人。展開後の約 7.5 インチの内画面なら、A4 PDF をズームなしで見られ、体験が劇的に変わります。
3. **頻繁に出張/会議のあるビジネスユーザー：**タブレットを持ちたくないがスマートフォンの画面が小さいと感じる人。横折りスマートフォンを展開して PPT を見たり、契約書にサインしたり、2 画面でメールを照合したりでき、ストレート型より一段と効率的。
4. **「ハングモード」でのクリエイティブ作業が好きな人：**90 度に折りたたんで机に置き、上半分でファインダー下半分でシャッター（自撮り/低速度撮影）、上半分で動画下半分でメモ、上半分でライブ配信下半分でコメント確認——これはストレート型にはできないフォームファクターです。

**携帯性と大画面を両立したい女性ユーザー**

- **：**縦折りスマートフォンは閉じると小さなバッグに入り、開くと通常のスマートフォン、外画面は化粧鏡や通知確認にも使えます。
- こういう人は買わないで
- スマートフォンをショート動画視聴、WeChat、電話にしか使わない人——折りたたみ画面は何のメリットもなく、むしろより重く厚くなります。

## 三、選び方のポイント：この 6 つの指標を見る

予算が厳しく、1 台のスマートフォンを 4 年以上使いたい人——折りたたみスマートフォンは中古の価値保持率がストレート型より低く、修理コストも高いです。

- **三、選び方のポイント：この 6 つの指標を見る**「折りたたみ」という言葉に目を奪われないで、選ぶときはここを重点的に見て：
- **ヒンジの種類：**水滴ヒンジ（折り目が浅く、寿命が長い、現在主流）が U 字ヒンジ（折り目が深い、初期の設計）より優れています。店員に聞くかレビューで確認。
- **内画面の素材：**実店舗で自分の手で展開して見てください。2026 年の合格ラインは：画面点灯時は正面からほとんど見えず、斜めの光で浅い溝が見え、指で撫でると軽い凹みがあるが手を切らない。

- **折り目の状態：**実店舗で実際に展開して見て。2025 年の合格ラインは：点灯時は正面からはほとんど見えず、斜めの光で浅い溝が見え、指でなぞるとわずかなへこみがあるが手を切りません。
- **外画面サイズ：**横折りスマートフォンは外画面が大きいほど良い（6.4 インチ以上が理想）、閉じた状態で通常のスマートフォンとして使える。縦折りスマートフォンは外画面が大きいほど実用的（4 インチ以上ならメッセージ返信、ナビ確認が可能）。
- **重量と厚さ：**横折り展開後 230-260g が一般的、230g 未満は優秀。閉じた厚さ 12-14mm、12mm 未満は薄い。縦折りは閉じた状態でシャツのポケットに楽に入れられるのが合格ライン。

アフターと保証

1. ：内画面割れの修理価格を重点的に確認。一部ブランドは初年度 1 回無料で内画面交換、自己負担の場合は 2000-4000 元かかることも。これは折りたたみスマートフォン最大の隠れコストです。
2. 実店舗体験チェックリスト
3. 実際に 10 回開閉し、ヒンジのダンピングが滑らかか、異音がないか確認。
4. 展開後、ライトに斜めに向けて折り目を見て、それから正面から点灯画面を見る。
5. 展開状態を片手で 30 秒持ち、重量が許容できるか確認。

## 四、折りたたみスマートフォンならではの使い方のコツ

内画面単独の修理価格と保証ポリシーを確認。

### 四、折りたたみスマートフォンならではの使い方のコツ

買った後、これらの使い方はストレート型では体験できません：

#### 分割画面とマルチタスク

横折りスマートフォン展開後、ほとんどのシステムが上下/左右分割画面に対応。典型的シナリオ：上半分でオンライン講座の動画、下半分でノート、左に WeChat 右にドキュメント、上にマップナビ下に音楽。ストレート型の分割画面よりはるかに実用的で、画面が十分大きく、どちらの半分も窮屈ではありません。

- **2. ハングモード（90 度折りたたみ）**これは折りたたみ画面の「切り札」的な使い方です：
- **自撮り/集合写真：**机に置いて 90 度に折り、上半分でファインダー、下半分でシャッター、人に撮ってもらう必要なし。

- **低速度撮影/夜景：**同じく机に置いて安定撮影、三脚が不要。
- **ビデオ通話：**両手がフリーになり、相手から見る角度もより自然。
- **動画視聴：**机に置いてミニテレビに、飛行機/新幹線のテーブルで特に便利。

### 外画面プレビュー＋内画面撮影

：背面カメラで自分を撮るとき、外画面にリアルタイムで映像が表示され、構図がより正確に。

### 外画面クイックチェック

縦折りスマートフォンを閉じた後、外画面で：時間/通知/天気の確認、WeChat への返信（音声テキストまたはクイック返信）、交通系 IC カードのタッチ、ナビの確認、化粧鏡（インカメラの起動）が可能。毎回展開する必要がなく、省電力で便利。

## 五、メンテナンスのポイント：折りたたみ画面を長持ちさせる

一部の横折りスマートフォン（Huawei、Samsung）は展開後「PC モード」に切り替え可能、外部モニターとキーボード・マウスを接続すれば簡易 PC のように、一時的なドキュメント処理やメール返信には十分。メインの生産性ツールではありませんが、緊急時には非常に便利。

- **五、メンテナンスのポイント：折りたたみ画面を長持ちさせる**折りたたみ画面で最もデリケートなのが内画面とヒンジ、これらの習慣で寿命を大幅に延ばせます：
- **低温では無理に開閉しない：**0℃ 以下では UTG ガラスが脆くなる。開閉する前に懐で数分温めて。これは公式マニュアルにも書いてあるが誰も読まない重要ポイント。
- **爪/硬いもので内画面に触れない：**内画面には保護層があるが外画面より柔らかい。指の腹で操作し、爪で突かない。
- **閉じるときに画面内に異物がないか確認：**一粒の砂が挟まったまま閉じると、内画面を直接圧壊する可能性。閉じる前に一目確認。
- **普通の強化ガラスフィルムは貼らない：**内画面には公式専用フィルムまたは UTG 保護フィルムのみ使用。普通の強化ガラスフィルムは折りたたみとタッチ操作に影響。公式サービスセンターでのフィルム貼りをおすすめ。
- **ヒンジにほこりや液体を入れない：**IPX8 防水を謳うものが多いが、防塵等級は一般的に低い。ビーチや工事現場などの環境では注意。

### 開閉の力を均一に

- ：片手で勢いよく開かない、両手で中央から均等に展開するのが最も安定。

- 内画面破損の多発シーン
- 閉じるときにイヤホンケーブル/充電ケーブルを挟む——内画面が直接圧壊。
- バッグの中で鍵と一緒に入れる——外画面は無事でも、内画面が鍵でひび割れ。

## 六、折りたたみスマートフォンの「落とし穴」と心理的期待値

1 年使って「折り目が深くなった」と感じて純正フィルムを剥がす——フィルムを剥がすと内画面も一緒に破損。

- **六、折りたたみスマートフォンの「落とし穴」と心理的期待値：**2026 年の技術では「完全に痕なしが」は実現できず、「画面点灯時正面からほとんど見えない」までしかできません。完璧主義者で、毎日折り目を見て不快に感じるなら、買わないでください。
- **折り目はずっとある：**2025 年の技術では「完全に跡なし」は実現できず、「点灯時正面からはほとんど見えない」が限界。完璧主義者で、毎日折り目を見てつらいなら、買わないで。
- **より重くより厚い：**横折りスマートフォンは展開後一般的に 230g 以上で、片手持ちはストレート型より疲れる。閉じた厚さ 12-14mm で、ズボンのポケットがより膨らむ。
- **バッテリー持ちは一般的に普通：**ヒンジ機構を収めるため、バッテリー空間が圧縮され、同価格帯のバッテリー持ちは通常ストレート型に劣る。
- **価値保持率が低い：**1 年後の中古価格は通常新品の 50-60% で、ストレート型より下落が早い。
- **修理が高い：**内画面ユニットの修理価格 2000-4000 元、ヒンジ修理 500-1500 元。購入前に保証ポリシーを確認。

アプリの対応がまちまち

：主流アプリは大画面に対応済みだが、ニッチなアプリは単に「引き伸ばし拡大」の可能性があり、体験はいまひとつ。

## 七、主流ブランドの選び方（2026 年）

——折りたたみスマートフォンの選び方の哲学について

- **七、主要ブランドの選び方（2025 年）**具体的な型番は書かず（古くなるため）、ブランドの特徴だけ書き、素早く絞り込めるようにします：
- **Huawei Mate X シリーズ：**横折りのベンチマーク、HarmonyOS エコシステムとの連携が強力、外画面の比率が通常のスマートフォンに近く、閉じた状態でメイン機として使える。価格は高いが価値保持率は相対的に最も良い。

- **Samsung Galaxy Z Fold/Flip：**折りたたみ画面の元祖、ヒンジの調整が成熟、Z Flip は縦折りのルックス担当。中国国内のサービス拠点が少ないのが弱点。
- **Xiaomi MIX Fold/Flip：**コストパフォーマンス路線、スペックが厚く、価格は比較的お手頃。試してみたいがあまりお金をかけたくないユーザーに適している。
- **vivo X Fold シリーズ：**横折りの中で内外画面の比率が最も通常のスマートフォンに近く、閉じてストレート型として使う体験が良い。
- **OPPO Find N シリーズ：**横折りの中でサイズがややコンパクト、片手で持てる、大型折りたたみが重いと感じるユーザーに適している。

Huawei Pocket シリーズ

**最後の注意**

折りたたみ画面技術は毎年イテレーションしており、本付録は 2026 年に執筆。購入前に必ず最新のレビューとユーザーフィードバックを確認し、ヒンジ寿命の実測、折り目の表現、内画面の修理ポリシーの 3 項目に注目。実店舗で直接触れることがレビューを見るより常に重要です。

# おわりに：好奇心を持ち続け、清醒でいる

ここまで読んでくれておめでとうございます。

この本はスマホのハードウェアからソフトウェアエコシステムまで、日常設定から開発者モードまで、セキュリティ保護からデジタル遺産までを扱いました。もし実際の問題を 1 つでも解決できたなら——不必要な権限をオフにすることを覚えただけでも、一度のバックアップ習慣を身につけただけでも——これらの文字は無駄ではありません。

しかし、どんな具体的テクニックよりも重要なのは 2 つのことです：

- **好奇心を保つ：**スマホは毎年変わり、AI が変わり、折りたたみ画面が変わり、衛星通信が変わります。今日学んだ操作手順は明日には古くなるかもしれませんが、「なぜ知りたい」という好奇心は決して古くなりません。「この権限は何のためにあるのか」「このデータはどこに保存されているのか」「この機能はプライバシーを漏洩しないか」ともう一つ問いかけることが、百の操作手順を覚えるより役立ちます。
- **清醒さを保つ：**スマホは道具であり、主人ではありません。それはあなたの注意を引きつけ、時間を占有し、消費を誘導するために設計されています。使うのはあなたの選択、使われるのはあなたのコントロール不全です。定期的にスクリーンタイムを見直し、「私はスマホを使って何かをしているのか、スマホに消費されているのか」と考えることは、感傷ではなく、デジタル時代の生存本能です。

技術は変わりますが、人のニーズは変わりません——私たちには安全感が必要で、つながりが必要で、尊重される必要があります。スマホは単なる媒体です。本当の「スマホの授業」は本の中ではなく、毎日画面をロック解除するその瞬間にあります。

**最後の練習**

この本を閉じ、スマホを手に取って、一つのことをしてください：**30 秒間考えてみてください。あなたのスマホの中で「失くしたら本当に心痛む」ものは 3 つ何ですか？**そして、それらがバックアップされていることを確認してください。この本の価値はそれで十分です。

# 付録 G：本書まるごとクイック索引

五十音順・アルファベット順に並べており、目的の章にすばやくたどり着けます。

### A-B

- BL ロック第二十四章
- ADB 第二十章
- App のインストールと管理第四章
- AI アシスタント第二十二章
- AI スマホ第二十六章
- AI 顔すり替え詐欺第三十二章
- AI プログラミング第三十七　四十七章
- AI プログラミングの 5 大原則第三十八章
- API Key 第四十二章
- Agent（エージェント）第三十七章
- CLAUDE.md 第三十九章
- Android 開発第四十八　五十八章
- Android Studio 第四十九章
- Activity のライフサイクル第五十二章
- App Store への公開第六十七章

### C-D

- カー・スマホ連携第三十四章
- バッテリーと電池持ち第十五章

- パスワード管理第十四章
- パスキー（Passkey）第十四章
- 開発者モード第二十章
- 子どもとスマホ第三十五章
- Claude Code / Cursor 第四十一章
- DeepSeek 第四十二章
- Compose 第五十一章
- DataStore 第五十三章


**F-H**


- 折りたたみスマホ（Foldable）付録 F
- トラブルシューティング第十九章
- Google Play への公開第五十六章
- GaN 充電器第十五章
- iCloud 第十六章
- スマートホーム第三十六章
- コンピュテーショナルフォトグラフィ第十一章付録


**K-R**


- 撮影テクニック第十一章
- 権限管理第四章
- SafetyNet 認証第二十章
- SIM スワップ攻撃第三十二章
- Root 権限第二十章 / 第二十四章
- Termux 第二十章付録
- MCP プロトコル第四十六章
- PRD 要件ドキュメント第四十章
- プロンプトエンジニアリング第四十三章
- Kotlin 第五十章

- MVVM アーキテクチャ第五十七章・第六十八章
- Room データベース第五十三章
- Retrofit 第五十三章


**S-Z**


- デジタル遺産第二十五章
- データバックアップ第十八章
- 迷惑ブロック第八章・第十三章
- 気象警報第十三章
- 自然災害警報第十三章・第三十二章
- 地震警報第十三章・第三十二章
- 実用アプリとクリーンアップツール第十三章
- Qingzhuo（クリーンアップアプリ）第十三章
- UWB（超広帯域無線）第十七章
- ネットワークセキュリティ第二十一章
- 購入ガイド第二十八章
- クラウドサービス第十六章
- SDD（仕様駆動開発）第三十九章
- Token 第四十二章
- Vibe Coding 第三十七章
- Swift 第六十一章
- SwiftUI 第六十二章
- SwiftData 第六十四章
- URLSession 第六十四章
- Xcode 第六十章
- iOS 開発第五十九　六十九章

## その他

- ファイル管理第二章
- シニア向け設定第二十七章
- ツール系 App のおすすめ第十三章
- 迷惑対策 / 国家反詐欺センター第十三章・第二十七章
- PC 遠隔操作（向日葵 / ToDesk）第十三章
- AI オフィス（Trae など）第十三章・第四十一章
- LocalSend / LAN 高速送信第十三章
- KDE Connect / クリップボード同期第十三章・第十七章
- 2FA ワンタイムコード（Aegis など）第十三章・第十四章
- TickTick（滴答清单）第十三章
- 紙の書類スキャン第十三章
- 人工デスクトップ（ライブ壁紙）第十三章
- 地震 / 災害時のスマホ応急対応第三十二章
- 画面ミラーリングと車載連携第三十章
- ショート動画づくり第三十一章
- 安全・応急対応第三十二章
- Git / GitHub 第二十三章付録
- Git バージョン管理第四十五章
- コンテキストエンジニアリング第三十九章
- 要件エンジニアリング第四十章
- 家計簿 App 第五十四章・第六十五章
- 天気 App 第五十五章・第六十六章
- WidgetKit ウィジェット第六十八章

# 付録 H：バージョンロードマップ

この本は更新を続けています。以下は今後のバージョンで計画している方向性です。フィードバックで「いちばん読みたい内容」を教えてもらえるとうれしいです。

**v1.8 で実現：第 7・8・9 部『AI プログラミング実践編』『Android 開発実践編』『iOS 開発実践編』を新設（第 37□69 章、全 33 章）**

AI プログラミングのパラダイム革命、5 大思考原則、コンテキストと要件のエンジニアリング、2026 年のツールとモデルの全景、API 実践、会話エンジニアリング、3 つの完全なプロジェクト、品質保証とステップアップへの道——「AI を使える」から「AI に指揮をとってコードを書かせる」レベルへ引き上げます。さらに Android と iOS の 2 大プラットフォームの完全な開発チェーン：環境構築（Android Studio / Xcode）、言語速習（Kotlin / Swift）、UI フレームワーク（Jetpack Compose / SwiftUI）、データとネットワーク、実践プロジェクト（家計簿アプリ＋天気アプリ）、テストとリリース（Google Play / App Store）、そしてクイックリファレンス——デジタルユーザーから App 開発者へと一気に成長できます。

**次回版（v1.9）の計画テーマ**

- スマホとウェアラブル：Apple Watch、Huawei のスマートウォッチ、スマートリング（Oura / Galaxy Ring）を深く連携させるガイド
- 旅とスマホ：eSIM の実践、海外データ通信プラン、オフライン地図、海外での決済、翻訳 App の比較
- スマホと健康管理：運動と健康、睡眠モニタリング、心拍・血中酸素、女性の健康、メンタルヘルス App
- AI ショート動画づくり：剪映（CapCut）AI、Kling AI、デジタルヒューマン、AI ナレーションの実践チュートリアル
- 縦画面ショートドラマ入門：脚本の構成、撮影テクニック、各プラットフォームのポリシー

**情報の鮮度について**

スマホの技術は非常に速く進化するため、本書の内容は 2026 年半ばの技術と市場の状況に基づいて執筆しました。AI の能力、チップの型番、OS のバージョン、価格帯は、あ

なたが読む時点では変わっているかもしれません。具体的な操作手順については、お使いの端末の実際のシステムバージョンを優先してください。購入の判断には、最新のレビューとユーザーの声を確認しましょう。